达梦数据

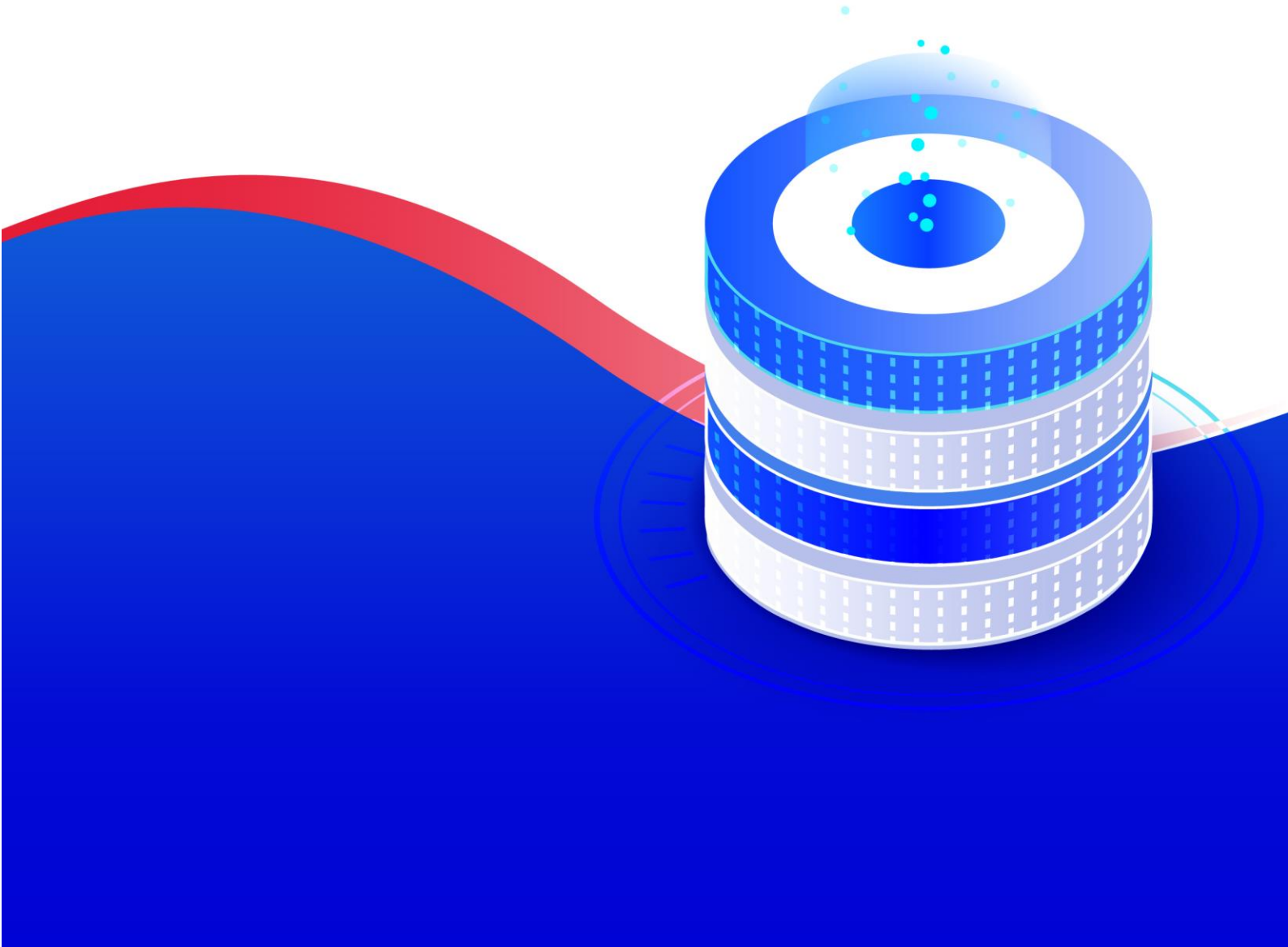
全栈数据产品与解决方案提供商

股票代码 688692

达梦技术手册

DM8 系统包使用手册

Service manual of DM8_System_Packages



前言

概述

本文档主要介绍达梦支持的系统包功能、相关方法和举例说明等。包括 DBMS_ALERT、DBMS_OUTPUT、UTL_FILE 和 UTL_MAIL 等系统包。

读者对象

本文档主要适用于 DM 数据库的：

- 开发工程师
- 测试工程师
- 技术支持工程师
- 数据库管理员

通用约定

在本文档中可能出现下列标志，它们所代表的含义如下：

表 0.1 标志含义

标志	说明
 警告：	表示可能导致系统损坏、数据丢失或不可预知的结果。
 注意：	表示可能导致性能降低、服务不可用。

 小窍门：	可以帮助您解决某个问题或节省您的时间。
 说明：	表示正文的附加信息，是对正文的强调和补充。

在本文档中可能出现下列格式，它们所代表的含义如下：

表 0.2 格式含义

格式	说明
HarmonyOS Sans SC Light	表示正文。
Poppins Light	表示代码或者屏幕显示内容。
粗体	表示命令行中的关键字（命令中保持不变、必须照输的部分）或者正文中强调的内容。标题、警告、注意、小窍门、说明等内容均采用粗体。
< >	语法符号中，表示一个语法对象。
::=	语法符号中，表示定义符，用来定义一个语法对象。定义符左边为语法对象右边为相应的语法描述。
|	语法符号中，表示或者符，限定的语法选项在实际语句中只能出现一个。
{ }	语法符号中，大括号内的语法选项在实际的语句中可以出现 0…N 次(N 为大于 0 的自然数)，但是大括号本身不能出现在语句中。
[ ]	语法符号中，中括号内的语法选项在实际的语句中可以出现 0…1 次，但是中括号本身不能出现在语句中。
关键字	关键字在 DM_SQL 语言中具有特殊意义，在 SQL 语法描述中，关键字以大写形式出现。但在实际书写 SQL 语句时，关键字既可以大写也可以小写。

访问相关文档

如果您安装了 DM 数据库，可在安装目录的“\doc”子目录中找到 DM 数据库的各种手册与技术丛书。

您也可以通过访问我们的网站 www.dameng.com 阅读或下载 DM 的各种相关文档。

联系我们

如果您有任何疑问或是想了解达梦数据库的最新动态消息，请联系我们：

网址：www.dameng.com

技术服务电话：400-991-6599

技术服务邮箱：dmtech@dameng.com

目录

1 概述.....1

1.1 系统包创建、删除语句.....1

1.2 创建、删除指定的包.....1

1.3 创建、删除所有系统包.....3

1.4 检测系统包是否启用.....4

1.5 包间依赖关系.....5

2 DBMS_ADVANCED_REWRITE 包.....7

2.1 相关方法.....7

2.2 字典信息.....12

2.3 使用说明.....13

2.4 举例说明.....15

3 DBMS_ALERT 包.....18

3.1 相关方法.....18

3.2 DBMS_ALERT_INFO 视图.....22

3.3 举例说明.....22

4 DBMS_BINARY 包.....25

4.1 相关方法.....25

4.2 参数说明.....32

4.3 错误处理.....32

4.4 举例说明.....33

5 DBMS_JOB 包 36

5.1 相关方法 36

5.2 相关视图 44

5.3 创建、删除、重建语句 46

5.4 举例说明 47

6 DBMS_LOB 包 49

6.1 相关方法 49

6.2 举例说明 81

7 DBMS_LOCK 包 83

7.1 封锁规则 83

7.2 相关方法 84

7.3 举例说明 90

8 DBMS_LOGMNR 包 93

8.1 相关方法 93

8.2 举例说明 100

9 DBMS_METADATA 包 102

9.1 数据类型 102

9.2 相关方法 104

9.3 错误处理 124

9.4 举例说明 125

10 DBMS_OBFUSCATION_TOOLKIT 包 135

10.1 相关方法.....135
10.2 使用说明.....152
10.3 举例说明.....153
11 DBMS_OUTPUT 包.....159
11.1 相关方法.....159
11.2 举例说明.....162
12 DBMS_PIPE 包.....164
12.1 使用前提.....164
12.2 相关方法.....164
12.3 动态视图.....171
12.4 举例说明.....171
13 DBMS_RANDOM 包.....174
13.1 相关方法.....174
13.2 举例说明.....178
14 DBMS_RLS 包.....179
14.1 策略函数.....179
14.2 策略组.....180
14.3 策略.....182
14.4 上下文.....189
14.5 启用状态检测语句.....191
14.6 举例说明.....191
15 DBMS_SESSION 包.....200

15.1 数据类型.....200
15.2 相关方法.....201
15.3 举例说明.....205
16 DBMS_SPACE 包.....210
16.1 数据类型.....210
16.2 相关方法.....219
16.3 举例说明.....232
17 DBMS_SQL 包.....235
17.1 相关方法.....235
17.2 举例说明.....261
18 DBMS_TRANSACTION 包.....266
18.1 相关方法.....266
18.2 举例说明.....266
19 DBMS_STATS 包.....267
19.1 数据类型.....267
19.2 相关方法.....270
19.3 约束.....319
19.4 举例说明.....320
20 DBMS_UTILITY 包.....326
20.1 数据类型.....326
20.2 相关方法.....326
20.3 举例说明.....329

21 DBMS_WORKLOAD_REPOSITORY 包 332

21.1 相关方法 333

21.2 系统表 354

21.3 创建、检测、删除语句 365

21.4 报表解析说明 366

21.5 举例说明 404

22 DBMS_XMLGEN 包 406

22.1 相关方法 406

22.2 举例说明 414

23 UTL_ENCODE 包 419

23.1 相关方法 419

23.2 举例说明 420

24 UTL_FILE 包 422

24.1 数据类型 422

24.2 相关方法 422

24.3 错误处理 442

24.4 举例说明 442

25 UTL_INADDR 包 447

25.1 相关方法 447

25.2 举例说明 448

26 UTL_MAIL 包 449

26.1 相关方法 449
26.2 举例说明 452
27 UTL_MATCH 包 455
27.1 相关方法 455
27.2 举例说明 457
28 UTL_RAW 包 458
28.1 相关方法 458
28.2 举例说明 472
29 UTL_TCP 包 477
29.1 相关方法 477
29.2 举例说明 487
30 UTL_URL 包 489
30.1 相关方法 489
30.2 举例说明 491
31 DBMS_SCHEDULER 包 492
31.1 相关方法 492
31.2 创建、删除、重建语句 524
31.3 启用状态检测语句 524
31.4 相关视图 525
31.5 日历语法 534
31.6 举例说明 540

32 DBMS_MVIEW 包 547
32.1 相关方法 547
32.2 举例说明 548
33 UTL_SMTP 包 550
33.1 相关方法 550
33.2 举例说明 562
34 UTL_HTTP 包 565
34.1 相关方法 566
34.2 举例说明 585
35 UTL_I18N 包 587
35.1 相关方法 587
35.2 举例说明 589
36 DBMS_AQADM 包 593
36.1 数据类型 593
36.2 相关方法 594
36.3 相关视图 602
36.4 举例说明 602
37 DBMS_AQ 包 604
37.1 数据类型 604
37.2 相关方法 609
37.3 回调说明 611

37.4 举例说明 612

38 DBMS_SQLTUNE 包 616

38.1 相关方法 616

38.2 举例说明 625

39 DBMS_CRYPTO 包 629

39.1 相关方法 629

39.2 使用说明 633

39.3 举例说明 633

40 DBMS_ERRLOG 包 642

40.1 相关方法 642

40.2 使用说明 644

40.3 使用错误日志表 645

40.4 举例说明 646

41 UTL_COMPRESS 包 649

41.1 相关方法 649

41.2 举例说明 659

42 DBMS_APPLICATION_INFO 包 666

42.1 相关方法 666

42.2 举例说明 668

43 DBMS_REDEFINITION 包 670

43.1 相关方法 670

43.2 系统表.....678
43.3 系统视图.....681
43.4 一个完整的重定义过程.....683
43.5 举例说明.....684
44 DBMS_XA 包.....688
44.1 常量.....688
44.2 数据类型.....690
44.3 相关方法.....691
44.4 举例说明.....695
45 DBMS_XMLDOM 包.....698
45.1 数据类型.....698
45.2 相关方法.....701
45.3 举例说明.....757
46 DBMS_DEBUG 包.....759
46.1 相关方法.....759
46.2 举例说明.....765
47 ODCICONST 包.....775
47.1 相关常量.....775
47.2 举例说明.....776
48 DBMS_JSON 包.....778
48.1 数据类型.....778
48.2 JDOM_T 类方法.....778

48.3 JSON_ELEMENT_T 类方法 799
48.4 JSON_SCALAR_T 类方法 807
48.5 JSON_ARRAY_T 类方法 808
48.6 JSON_OBJECT_T 类方法 817
48.7 举例说明 827
49 DBMS_FLASHBACK 包 835
49.1 相关方法 835
49.2 使用说明 836
49.3 举例说明 837
50 DBMS_SYSTEM 包 838
50.1 相关方法 838
50.2 举例说明 839
51 DBMS_AUDIT 包 841
51.1 相关方法 841
51.2 举例说明 843
52 DBMS_XPLAN 包 846
52.1 相关方法 846
52.2 举例说明 847
53 DBMS_DDL 包 849
53.1 相关方法 849
53.2 举例说明 851

54 DBMS_LDAP 包 853

54.1 数据类型 853

54.2 相关方法 853

54.3 举例说明 866

55 DBMS_XMLPARSER 包 868

55.1 数据类型 868

55.2 相关方法 868

55.3 举例说明 875

DM 技术支持 877

手册版本 879

1 概述

本文档主要介绍达梦支持的系统包功能、相关方法和举例说明等。包括 DBMS_ALERT、DBMS_OUTPUT、UTL_FILE 和 UTL_MAIL 等系统包。

1.1 系统包创建、删除语句

DM 在新建库第一次启动服务器时会自动创建所有系统包。

用户也可以通过调用系统过程来创建、删除系统包：一是创建、删除指定的系统包；二是创建、删除所有系统包。用户须具有 DBA 角色权限。



SP_INIT_XXX_SYS()，SP_CREATE_SYSTEM_PACKAGES()使用限制：

1. **不能在 MPP 全局模式下的存储过程中直接调用，在 MPP LOCAL 模式下可在存储过程中直接调用；**
2. **不能在存储过程中带参数进行动态调用。**

1.2 创建、删除指定的包

创建、删除指定系统包的语句有两种：一是 SP_INIT_XXX_SYS(create_flag)；二是 SP_CREATE_SYSTEM_PACKAGES(create_flag, pkgname)。

1.2.1 使用 SP_INIT_XXX_SYS()语句

使用 SP_INIT_XXX_SYS 过程创建或删除指定的系统包。该方法仅支持 DBMS_JOB、DBMS_WORKLOAD_REPOSITORY、DBMS_SCHEDULER 和 DBMS_RLS 包。

这些系统包和创建语句对应关系如下：

包名	创建语句 SP_INIT_XXX_SYS ()

DBMS_JOB	SP_INIT_JOB_SYS(create_flag);
DBMS_WORKLOAD_REPOSITORY	SP_INIT_AWR_SYS(create_flag);
DBMS_SCHEDULER	SP_INIT_DBMS_SCHEDULER_SYS(create_flag);
DBMS_RLS	SP_INIT_RLS_SYS(create_flag);

SP_INIT_XXX_SYS()详细介绍如下:

语法如下:

```
SP_INIT_JOB_SYS (CREATE_FLAG  INT);

SP_INIT_AWR_SYS(CREATE_FLAG  INT);

SP_INIT_DBMS_SCHEDULER_SYS(CREATE_FLAG  INT);

SP_INIT_RLS_SYS(CREATE_FLAG INT);
```

参数详解

- CREATE_FLAG

  为 1 时表示创建对应的系统包；为 0 表示删除对应的系统包。

返回值

无。

举例说明

创建 DBMS_JOB 系统包。

```
SP_INIT_JOB_SYS(1);
```

1.2.2 使用 SP_CREATE_SYSTEM_PACKAGES()语句

SP_CREATE_SYSTEM_PACKAGES 用来创建或删除指定的系统包。除了使用 SP_INIT_XXX_SYS 创建、删除的系统包之外,均可使用 SP_CREATE_SYSTEM_PACKAGES 创建或删除。

SP_CREATE_SYSTEM_PACKAGES()的详细介绍如下：

语法如下：

```
SP_CREATE_SYSTEM_PACKAGES (
    CREATE_FLAG     INT,
    PKGNAME         VARCHAR(128)
)
```

参数详解

- CREATE_FLAG

为 0 时表示删除指定的系统包；为 1 时表示创建指定的系统包；为 2 时表示仅重建指定的系统包定义，即包头和包体，不会重建关联的包系统表、包关联对象（例如：包中的数据类型）等。

- PKGNAME

指定要创建的包名。

返回值

无。

举例说明

例 创建 DBMS_LOB 系统包。

```
SP_CREATE_SYSTEM_PACKAGES(1, 'DBMS_LOB');
```

1.3 创建、删除所有系统包

SP_CREATE_SYSTEM_PACKAGES 用来创建或删除所有系统包，其中不包含 DBMS_JOB、DBMS_WORKLOAD_REPOSITORY、DBMS_SCHEDULER 和 DBMS_RLS 包。若在创建过程中某个系统包由于特定原因未能创建成功，会跳过继续创建后续的系统包。

语法如下：

```
SP_CREATE_SYSTEM_PACKAGES (
    CREATE_FLAG      INT
)
```

参数详解

- CREATE_FLAG

为 0 时表示删除所有系统包；为 1 时表示创建所有系统包，不包括 UTL_MAIL；为 2 时表示仅重建所有系统包的定义，即包头和包体，不会重建关联的包系统表、包关联对象（例如：包中的数据类型）等，不包括 UTL_MAIL。

返回值

无。

举例说明

例 创建所有系统包（不包括 UTL_MAIL）。

```
SP_CREATE_SYSTEM_PACKAGES (1);
```

1.4 检测系统包是否启用

1. SF_CHECK_SYSTEM_PACKAGES

SF_CHECK_SYSTEM_PACKAGES 用来检测系统包的启用状态。系统包的启用状态由是否执行过 SP_CREATE_SYSTEM_PACKAGES(1)过程决定，若执行过，则状态为已启用。

语法如下：

```
INT
SF_CHECK_SYSTEM_PACKAGES()
```

返回值

0：未启用；1：已启用。

举例说明

获得系统包的启用状态。

```
SELECT SF_CHECK_SYSTEM_PACKAGES();
```

2. SF_CHECK_SYSTEM_PACKAGE

SF_CHECK_SYSTEM_PACKAGE 可用来检查某个特定的系统包是否启用。

语法如下:

```
INT
SF_CHECK_SYSTEM_PACKAGE(
    PACKAGE_NAME        VARCHAR
)
```

返回值

0：未启用；1：已启用。

举例说明

例 获得系统包 DBMS_SCHEDULER 的启用状态。

```
SELECT SF_CHECK_SYSTEM_PACKAGE('DBMS_SCHEDULER');
```

1.5 包间依赖关系

部分系统包之间有依赖关系，被依赖包一旦被删除，依赖包就会失效。在 DPC 环境下创建系统包时,若未创建系统包所依赖的包就直接创建系统包,建包语句在 MP 节点执行后，将在该节点打印警告信息，不会向 SP 节点返回警告信息，此时，SP 节点虽然收到了最终执

行语句成功的消息，但实际上该系统包没有创建成功。

针对以下存在依赖关系的系统包，当用户创建这些系统包时，系统会自动创建它们所依赖的包：

- DBMS_ALERT 依赖 DBMS_LOCK 和 DBMS_UTILITY
- DBMS_METADATA 依赖 DBMS_LOB 和 UTL_RAW
- DBMS_LOB 依赖 UTL_RAW
- DBMS_XMLGEN 依赖 DBMS_LOB
- UTL_I18N 依赖 UTL_RAW
- DBMS_AQ 依赖 DBMS_AQADM
- DBMS_SQL 依赖 DBMS_LOB
- DBMS_DDL 依赖 DBMS_SQL

针对以下存在依赖关系的系统包，用户需要先手动创建它们所依赖的包，然后才能成功创建这些系统包：

- DBMS_PIPE 依赖 DBMS_XMLGEN

2 DBMS_ADVANCED_REWRITE 包

在不改变应用程序的前提下，在服务器端将查询语句替换成其他的查询语句执行，此时就需要查询重写（QUERY REWRITE）。DM 使用 DBMS_ADVANCED_REWRITE 包实现该功能，不支持安全策略。DM 支持对原始语句中的某些特定词的替换，以及整个语句的替换，不支持递归和变换替换。

2.1 相关方法

1. DECLARE_REWRITE_EQUIVALENCE

声明一个等价重写规则。

语法如下:

```
PROCEDURE DECLARE_REWRITE_EQUIVALENCE (
    NAME                    VARCHAR(128),
    SOURCE_STMT             VARCHAR(8188),
    DESTINATION_STMT VARCHAR(8188),
    VALIDATE            BOOLEAN,
    REWRITE_MODE        VARCHAR(16)
);
```

参数详解

- name

  重写规则的唯一标识，在会话级唯一。

- source_stmt

原始查询语句。

- destination_stmt

目标语句。

- validate

是否校验原始语句和目标语句等价。取值为 TRUE/FALSE，该参数暂不支持，实际无效。

- rewrite_mode

包含 4 种：disabled，不允许重写；text_match，文本匹配重写；general，变换重写；recursive，递归重写。后面两种暂不支持，可以声明成功，但是不起作用。

使用说明:

(1) 如果一个 SOURCE_STMT 对应多个 DESTINATION_STMT，即用户创建很多相同的重写规则，只是名字不同，在重写时只有第一个才有效。也就是说只执行第一个重写规则，后面的所有重写规则无效。

(2) 不允许 SOURCE_STMT 和 DESTINATION_STMT 完全相同，否则报错：“源语句与目标语句相同”。

(3) SOURCE_STMT 和 DESTINATION_STMT 语句必须为查询语句，否则报错：“源语句与目标语句不兼容”。

(4) SOURCE_STMT 和 DESTINATION_STMT 语句必须经过语法和语义正确解析，否则报错。如果两个语句中都出错，则报 SOURCE_STMT 的错，即首先解析 SOURCE_STMT，如果出错直接报错，不再解析后面的 DESTINATION_STMT。

(5) REWRITE_MODE 为“TEXT_MATCH”时，只允许替换 SOURCE_STMT 中完全相同的语句，不区分大小写。若 SOURCE_STMT 为空，进行正则表达式替换时，是否大小写敏感与数据库建库参数保持一致。

(6) SOURCE_STMT 和 DESTINATION_STMT 语句不允许包含“{}”字符。

(7) 扩展功能说明：如果 SOURCE_STMT 为空，DESTINATION_STMT 格式为“TABLE/[TABLE]”,则可以将当前会话语句中的“TABLE”换成“[TABLE]”,“TABLE”为正则表达式。DM 支持符合 POSIX 标准的正则表达式。如下表 4.1 所示：

表 4.1 正则表达式语法

语法	说明	示例
.	匹配任何除换行符之外的单个字符	DA. 匹配“DAMENG”
*	匹配前面的字符 0 次或多次	A*B 匹配“BAT”中的“B”和“ABOUT”中的“AB”
+	匹配前面的字符一次或多次	AC+ 匹配包含字母“A”和至少一个字母“C”的单词，如“RACE”和“ACE”
^	匹配行首	^CAR 仅当单词“CAR”显示为行中的第一组字符时匹配该单词
$	匹配行尾	END$ 仅当单词“END”显示为可能位于行尾的最后一组字符时匹配该单词
[]	字符集，匹配任何括号间的字符	BE[N-T] 匹配“BETWEEN”中的“BET”、“BENEATH”中的“BEN”和“BESIDE”中的“BES”,但不匹配“BELOW”中的“BEL”
[^]	排除字符集。匹配任何不在括号间的字符	BE[^N-T] 匹配“BEFORE”中的“BEF”、“BEHIND”中的“BEH”和“BELOW”中的“BEL”,但是不匹配“BENEATH”中的“BEN”
(表达式)	在表达式加上括号或标签在替换命令中使用	(ABC)+匹配“ABCABCABC”
|	匹配 OR 符号（|）之前或之后的表达式。最常	(SPONGE|MUD) BATH 匹配“SPONGE

	用在分组中	BATH"和"MUD BATH"
\	按原义匹配反斜杠（\）之后的字符。这使您可以查找正则表达式表示法中使用的字符，如 { 和 ^	\^ 搜索 ^ 字符
{n[,m]}	区间表达式，匹配在它之前的单个字符重现的次数区间。{n}指重复 n 次；{n,}为至少出现 n 次重复；{n,m}为重现 n 至 m 次	zo{2} 匹配"zoone"中的"zoo"，但不匹配"zozo"
[[:alpha:]]	表示任意字母\w ([a-z]+) | ([A-Z]+)	
[[:digit:]]	表示任意数字\d ([0-9]+)	
[[:lower:]]	表示任意小写字母（[a-z]+)	
[[:alnum:]]	表示任意字母和数字([a-z0-9]+)	
[[:space:]]	表示任意空格	
[[:upper:]]	表示任意大写字母([A-Z]+)	
[[:punct:]]	表示任意标点符号	
[[:xdigit:]]	表示任意 16 进制数([0-9a-fA-F]+)	

2. ALTER_REWRITE_EQUIVALENCE

修改重写模式。

语法如下:

```
PROCEDURE ALTER_REWRITE_EQUIVALENCE (
    NAME            VARCHAR(128),
    REWRITE_MODE    VARCHAR(16)
);
```

参数详解

- name

  重写规则的唯一标识。

- rewrite_mode

  包含 4 种，同上。

3. VALIDATE_REWRITE_EQUIVALENCE

校验重写规则是否等效，暂不支持。提供该函数，但是不进行校验。

语法如下:

```
VALIDATE_REWRITE_EQUIVALENCE (
    NAME    VARCHAR(128)
);
```

参数详解

- name

  重写规则的唯一标识。

4. DROP_REWRITE_EQUIVALENCE

删除重写规则。

语法如下:

```
PROCEDURE DROP_REWRITE_EQUIVALENCE (
    NAME      VARCHAR(128)
```

```
);
```

或

```
PROCEDURE DROP_REWRITE_EQUIVALENCE (
    NAME      VARCHAR(128),
    OWNER   VARCHAR(128)
);
```

参数详解

- NAME

  重写规则的唯一标识。

- OWNER

  重写规则所属的用户名称。

2.2 字典信息

所有的重写规则信息都保存到系统表 SYS_REWRITE_EQUIVALENCES 中，该系统表的结构如下表所示。所有的用户创建的语句重写信息都保存到该表中。

表 4.2 系统表 SYS_REWRITE_EQUIVALENCES 结构

序号	列名	类型	是否允许为空	说明
1	OWNER	VARCHAR(128)	N	重写规则的拥有者
2	NAME	VARCHAR(128)	N	重写规则名
3	SOURCE_STMT	VARCHAR(3200)	Y	原始语句
4	DESTINATION_STMT	VARCHAR(3200)	Y	目标语句
5	REWRITE_MODE	VARCHAR(16)	Y	重写模式

6	RESV1	INTEGER	Y	保留字段
7	RESV2	VARCHAR(128)	Y	保留字段

主键（OWNER，NAME）。

通过查询语句可以获得重写规则的信息：

```
SELECT * FROM SYS_REWRITE_EQUIVALENCES;
```

2.3 使用说明

1. 设置会话标记

每个会话对应一个 QUERY_REWRITE_INTEGRITY 标记，该标记只在会话期间起作用，不保存到字典信息中，如果会话结束，则该标记也无效。会话重建后，该标记也需要重新赋值才有效。

QUERY_REWRITE_INTEGRITY 标记用于表示该会话是否可以执行查询语句重写，缺省值为“ENFORCED”，另外两个值为：“TRUSTED”和“STALE_TOLERATED”。默认不允许语句重写；DM 未对后两个参数进行严格区分，一般设置后两个参数都允许查询重写。

语法格式：

```
ALTER SESSION SET QUERY_REWRITE_INTEGRITY = TRUSTED;
```

参数详解

- TRUSTED

表示在保证物化视图数据是正确时才使用该视图替换。

- STALE_TOLERATED

表示能容忍错误的数据，即可以使用不正确的物化视图执行重写。

2. 重写规则的作用域

重写规则只允许在当前会话中执行，禁止跨模式使用其他用户指定的重写规则。SYSDBA 也不可以使用其他用户的重写规则替换。

3. 回滚

执行方法 DECLARE_REWRITE_EQUIVALENCE 后，将自动提交，不能执行回滚。在该方法之前的所有操作也自动提交。

4. 自动重写

自动重写，指的是用户在执行任何查询语句时，可以自动调用重写规则执行重写，而不需要设置会话标记来控制是否执行查询重写。SYSDBA 可以设置某些用户自动重写标记，设置函数为：SP_USER_SET_AUTO_REWRITE_FLAG。该标记和会话标记中有一个有效，则允许查询重写。

语法格式：

```
SP_USER_SET_AUTO_REWRITE_FLAG (
    'USER_NAME',
    VALUE
);
```

参数详解

- USER_NAME

  用户名。

- VALUE

  0 表示不允许自动重写，1 表示允许自动重写。

5. 查询重写时机

查询重写的时机主要发生在如下几个地方：

（1）一般的用户查询，例如：SELECT COUNT(*) FROM T1;

（2）非动态的查询执行时，例如：存储过程中重写：SELECT COUNT(*) FROM T1 WHERE C1 > 1;但是以下语句不可以重写：I INT := 1; SELECT COUNT(*) FROM T1 WHERE C1 > I;

（3）视图中的查询重写，不支持递归替换。例如：CREATE VIEW V1 AS SELECT COUNT(*) FROM T1;如果"SELECT COUNT(*) FROM T1;"存在重写规则，也可以重写。

6. 关闭重用执行计划功能

使用时，可以在 DM.INI 中将重用执行计划标记（USE_PLN_POOL）置为 0。否则，先执行过查询语句后，再设置查询规则，然后对比两次的查询结果会相同，即未被重写。如果不关闭，只能等到设置过查询规则之后，才能执行查询。

2.4 举例说明

使用包内的过程和函数之前，如果还未创建过系统包，请先调用系统过程创建系统包。

```
SP_CREATE_SYSTEM_PACKAGES (1,'DBMS_ADVANCED_REWRITE');
```

例 REWRITE 包的应用，注意 INI 参数 USE_PLN_POOL 应置为 0。

```
DROP TABLE X1;

DROP TABLE X2;

CREATE TABLE X1(C1 INT, C2 CHAR(20));

INSERT INTO X1 VALUES(12, 'TEST12');

INSERT INTO X1 VALUES(13, 'TEST13');

INSERT INTO X1 VALUES(14, 'TEST14');

INSERT INTO X1 VALUES(15, 'TEST15');

CREATE TABLE X2(D1 INT, D2 CHAR(30));
```

```
COMMIT;

BEGIN

DBMS_ADVANCED_REWRITE.DECLARE_REWRITE_EQUIVALENCE(

'TEST_REWRITE',

'SELECT COUNT(*) FROM X1',                                    //原始查询语句

'SELECT COUNT(*) FROM X2',                                    //目标语句

FALSE,

'TEXT_MATCH'

);

END;

/



SELECT COUNT(*) FROM X1;                                      //执行结果 4



ALTER SESSION SET QUERY_REWRITE_INTEGRITY = TRUSTED;     //默认为'ENFORCED'

SELECT COUNT(*) FROM X1;                                      //执行结果 0



DBMS_ADVANCED_REWRITE.DROP_REWRITE_EQUIVALENCE ('TEST_REWRITE');

//DROP REWRITE



BEGIN

    DBMS_ADVANCED_REWRITE.DECLARE_REWRITE_EQUIVALENCE(

     'TEST_REWRITE',

     NULL,                                                    //为空
```

```
    'FRO./FROM',                                    //使用正则表达式
    FALSE,
    'TEXT_MATCH'
    );
END;
/
SELECT COUNT(*) FROK X1;                            //执行结果 4
```

3 DBMS_ALERT 包

DBMS_ALERT 系统包用于生成并传递数据库预警信息，当发生特定数据库事件时能够将预警信息传递给应用程序。

达梦还提供了 DBMS_ALERT_INFO 视图用于查看注册过的预警事件。

DM MPP 环境下不支持 DBMS_ALERT 包。

3.1 相关方法

1．DBMS_ALERT.REGISTER

为当前会话注册一个预警事件，本操作会提交当前事务。

语法如下:

```
DBMS_ALERT.REGISTER (
    NAME     IN   VARCHAR(30)
)
```

参数详解

- NAME

  输入参数，预警事件名。

2．DBMS_ALERT.REMOVE

删除当前会话的一个预警事件，如果该预警事件正在 DBMS_ALERT.WAITONE 或者 DBMS_ALERT.WAITANY 等待，则被阻塞，直到 DBMS_ALERT.WAITONE 或者 DBMS_ALERT.WAITANY 结束。本操作会提交当前事务。

语法如下:

```
DBMS_ALERT. REMOVE (
    NAME      IN  VARCHAR(30)
)
```

参数详解

- NAME

  输入参数，预警事件名。

3．DBMS_ALERT.REMOVEALL

删除当前会话下所有的预警事件，如果该预警事件正在 DBMS_ALERT.WAITONE 或者 DBMS_ALERT.WAITANY 等待，则被阻塞，直到 DBMS_ALERT.WAITONE 或者 DBMS_ALERT.WAITANY 结束。本操作会提交当前事务。

语法如下:

```
DBMS_ALERT. REMOVEALL ()
```

4．DBMS_ALERT.SIGNAL

给所有名为 name 的预警事件发送消息，当前事务提交时生效。

语法如下:

```
DBMS_ALERT. SIGNAL (
    NAME      IN   VARCHAR(30),
    MESSAGE IN   VARCHAR(1800)
)
```

参数详解

- NAME

  输入参数，预警事件名。

- MESSAGE

  输入参数，待发送的消息。

5．DBMS_ALERT.SET_DEFAULTS

设置轮询时间，仅语法支持。

语法如下:

```
DBMS_ALERT. SET_DEFAULTS (
    SENSITIVITY   IN      INT
)
```

参数详解

- SENSITIVITY

  输入参数，轮询时间间隔，单位秒。参数为 NULL，则使用缺省值 5s。

6．DBMS_ALERT.WAITONE

预警事件上等待预警信号，获得 DBMS_ALERT. SIGNAL 发送的消息内容。本操作会提交当前事务。

语法如下:

```
DBMS_ALERT. WAITONE (
    NAME      IN    VARCHAR(30),
    MESSAGE       OUT VARCHAR(1800),
```

```
    STATUS    OUT INT,
    TIMEOUT  IN    INT DEFAULT    86400000
)
```

参数详解

- NAME

  输入参数，预警事件名。

- MESSAGE

  输出参数，获得 DBMS_ALERT. SIGNAL 发送的消息。

- STATUS

  输出参数，表示 WAITONE 是否成功。如果 BMS_ALERT. WAITONE 传入的 NAME 是已经注册过的预警，则 0 表示预警发生；1 表示预警等待超时。

- TIMEOUT

  输入参数，WAITONE 的等待超时时间，单位秒，缺省值为 86400000 秒。

7．DBMS_ALERT. WAITANY

语法如下:

```
DBMS_ALERT. WAITANY (
    NAME      OUT VARCHAR(30),
    MESSAGE       OUT VARCHAR(1800),
    STATUS    OUT INT,
    TIMEOUT  IN    INT DEFAULT    86400000
```

)

参数详解

同 DBMS_ALERT. WAITONE。

功能说明:

等待当前会话任意一个预警信号，获得 DBMS_ALERT. SIGNAL 发送的消息内容。本操作会提交当前事务。会话上没有预警事件时，报错没有注册预警事件。

3.2 DBMS_ALERT_INFO 视图

DBMS_ALERT_INFO 视图可用于查看注册过的预警事件的信息，其列信息如下:

序号	列	数据类型	说明
1	NAME	VARCHAR(30)	预警事件的名称
2	SID	BIGINT	创建预警事件的会话 ID
3	CHANGED	VARCHAR(1)	预警事件状态：Y/N，预警事件被 SIGNAL 则为 Y
4	MESSAGE	VARCHAR(1800)	预警信号所带的消息内容

3.3 举例说明

使用包内的过程和函数之前，如果还未创建过例子中的系统包，请先调用系统过程创建系统包。

```
SP_CREATE_SYSTEM_PACKAGES (1,'DBMS_ALERT');
```

例 1 为当前会话注册一个名为 A1 的预警事件，给预警事件上等待预警信号，获得 DBMS_ALERT. SIGNAL 发送的消息内容，最后删除当前会话的一个预警事件。同时，会话 1 存活期间，可以通过 DBMS_ALERT_INFO 视图查看到 A1 预警事件。

```
会话 1:

    CREATE OR REPLACE PROCEDURE WAIT1 IS
```

```
msg   VARCHAR(1800);
statu INTEGER;
BEGIN
dbms_alert.register('A1');
dbms_alert.waitone('A1', msg, statu, 39);
print 'Msg: ' || msg || ' Statu: ' || statu;
dbms_alert.remove('A1');
END WAIT1;
/
EXEC WAIT1;
```

同时会话 2 执行:

```
CREATE OR REPLACE PROCEDURE SET_ALERT1
is
BEGIN
dbms_alert.signal('A1', 'this is a sig');
COMMIT;
END SET_ALERT1;
/
EXEC SET_ALERT1;
```

会话 1 存活期间，在会话 2 上，通过 DBMS_ALERT_INFO 视图查看 A1。

```
select * from DBMS_ALERT_INFO;
```

查询结果如下:

```
NAME      SID              CHANGED      MESSAGE
A1        83034240         Y            this is a sig
```

4 DBMS_BINARY 包

DBMS_BINARY 系统包用于读写二进制流，实现从一个二进制流指定位置开始对基本数据类型的读写，包括 CHAR、VARCHAR、TINYINT、SMALLINT、INT、BIGINT、FLOAT、DOUBLE 数据类型。

4.1 相关方法

DBMS_BINARY 包所支持的 8 个过程和 8 个函数，分别用来在二进制流中存取数据。这些过程和函数通过调用相应系统内部函数来实现存取数据的过程。如下详细介绍各过程和函数：

1. binary_get_char

返回从二进制流 VB 中偏移 OFFSET 开始的一个 CHAR 类型数据。

语法如下：

```
FUNCTION BINARY_GET_CHAR(
    VB          VARBINARY,
    OFFSET      INT
);
```

返回值

CHAR 类型数据。

2. binary_get_varchar

返回从二进制流 VB 中偏移 OFFSET 开始的一个长度为 LENGTH 的 VARCHAR 类型数据。

语法如下：

```
FUNCTION BINARY_GET_VARCHAR(
    VB          VARBINARY,
    OFFSET      INT,
    LENGTH      INT
);
```

返回值

VARCHAR 类型数据

3. binary_get_tinyint

返回从二进制流 VB 中偏移 OFFSET 开始的一个 TINYINT 型数据。

语法如下:

```
FUNCTION BINARY_GET_TINYINT(
    VB          VARBINARY,
    OFFSET      INT
);
```

返回值

TINYINT 类型数据。

4. binary_get_smallint

返回从二进制流 VB 中偏移 OFFSET 开始的一个 SMALLINT 型数据。

语法如下:

```
FUNCTION BINARY_GET_SMALLINT(
    VB          VARBINARY,
```

```
    OFFSET    INT
);
```

返回值

SMALLINT 类型数据。

5. binary_get_int

返回从二进制流 VB 中偏移 OFFSET 开始的一个 INT 型数据。

语法如下:

```
FUNCTION BINARY_GET_INT(
    VB        VARBINARY,
    OFFSET    INT
);
```

返回值

INT 类型数据。

6. binary_get_bigint

返回从二进制流 VB 中偏移 OFFSET 开始的一个 BIGINT 型数据。

语法如下:

```
FUNCTION BINARY_GET_BIGINT(
    VB        VARBINARY,
    OFFSET    INT
);
```

返回值

BIGINT 类型数据。

7. binary_get_float

返回从二进制流 VB 中偏移 OFFSET 开始的一个 FLOAT 型数据。

语法如下:

```
FUNCTION BINARY_GET_FLOAT(
    VB          VARBINARY,
    OFFSET      INT
);
```

返回值

FLOAT 类型数据。

8. binary_get_double

返回从二进制流 VB 中偏移 OFFSET 开始的一个 DOUBLE 型数据。

语法如下:

```
FUNCTION BINARY_GET_DOUBLE(
    VB          VARBINARY,
    OFFSET      INT
);
```

返回值

DOUBLE 类型数据。

9. binary_set_char

从二进制流 VB 中偏移 OFFSET 开始的位置写入一个 CHAR 型数据。

语法如下:

```
PROCEDURE BINARY_SET_CHAR(
    VB          IN OUT VARBINARY,
    OFFSET      INT,
    VALUE       CHAR
);
```

说明：BINARY_SET_CHAR 仅支持 VALUE 为单字符，若需要写入多字符请使用 binary_set_varchar

10. binary_set_varchar

从二进制流 VB 中偏移 OFFSET 开始的位置写入 VARCHAR 型数据。

语法如下:

```
PROCEDURE BINARY_SET_VARCHAR(
    VB          IN OUT VARBINARY,
    OFFSET      INT,
    VALUE       VARCHAR
);
```

11. binary_set_tinyint

从二进制流 VB 中偏移 OFFSET 开始的位置写入一个 TINYINT 型数据。

语法如下:

```
PROCEDURE BINARY_SET_TINYINT(
    VB          IN OUT VARBINARY,
    OFFSET      INT,
    VALUE       TINYINT
);
```

12. binary_set_smallint

从二进制流 VB 中偏移 OFFSET 开始的位置写入一个 SMALLINT 型数据。

语法如下:

```
PROCEDURE BINARY_SET_SMALLINT(
    VB          IN OUT VARBINARY,
    OFFSET      INT,
    VALUE       SMALLINT
);
```

13. binary_set_int

从二进制流 VB 中偏移 OFFSET 开始的位置写入一个 INT 型数据。

语法如下:

```
PROCEDURE BINARY_SET_INT(
    VB          IN OUT VARBINARY,
    OFFSET      INT,
```

```
    VALUE     INT
);
```

14. binary_set_bigint

从二进制流 VB 中偏移 OFFSET 开始的位置写入一个 BIGINT 型数据。

语法如下:

```
PROCEDURE BINARY_SET_BIGINT(
    VB          IN OUT VARBINARY,
    OFFSET      INT,
    VALUE       BIGINT
);
```

15. binary_set_float

从二进制流 VB 中偏移 OFFSET 开始的位置写入一个 FLOAT 型数据。

语法如下:

```
PROCEDURE BINARY_SET_FLOAT(
    VB          IN OUT VARBINARY,
    OFFSET      INT,
    VALUE       FLOAT
);
```

16. binary_set_double

从二进制流 VB 中偏移 OFFSET 开始的位置写入一个 DOUBLE 型数据。

语法如下:

```
PROCEDURE BINARY_SET_DOUBLE(
    VB          IN OUT VARBINARY,
    OFFSET      INT,
    VALUE       DOUBLE
);
```

4.2 参数说明

因为 DBMS_BINARY 包所支持的 8 个过程和 8 个函数参数意义相同，统一说明如下:

- vb

  用户输入的二进制流。

- offset

  向二进制流中写入数据或者从二进制流取出数据时的偏移量。

- length

  表示从二进制流指定偏移位置开始取多少个字符。

- value

  待写入的数据。

4.3 错误处理

在 DBMS_BINARY 包方法的执行过程中，按下列顺序进行参数检查:

1. 空值处理

如果输入参数中存在空值，则结果返回空值。特别的，参数 LENGTH 为 0 时视为 LENGTH 为 NULL。

例 输入的参数中存在空值。

```
BINARY_GET_CHAR (VB VARBINARY, OFFSET INT)

SELECT DBMS_BINARY.BINARY_GET_CHAR ('ABCDEF', NULL);
```

结果：NULL

2. 非法参数

如果参数 OFFSET 或 LENGTH 为负数，或者偏移量加上所取数据类型的长度大于二进制流的长度，报非法参数错误 EC_RN_INVALID_ARG_DATA，错误码：-6803。

例 输入偏移量为负数。

```
BINARY_GET_CHAR (VB VARBINARY, OFFSET INT)

SELECT DBMS_BINARY.BINARY_GET_CHAR ('ABCDEF', -11);
```

执行结果报错：输入参数非法。

4.4 举例说明

使用包内的过程和函数之前，如果还未创建过系统包，请先调用系统过程创建系统包。

```
SP_CREATE_SYSTEM_PACKAGES (1,'DBMS_BINARY');

SET SERVEROUTPUT ON;  //PRINT 需要设置这条语句，才能打印出消息
```

例 1 使用 BINARY_SET_TINYINT 过程写入一个 TINYINT 类型数据，并使用 BINARY_GET_TINYINT 函数输出该 TINYINT 类型数据。

```
DECLARE

    BIN VARBINARY(50);

    VAL TINYINT;
```

```
BEGIN
    BIN = 'ABCDEF87';
    VAL = 127;
    DBMS_BINARY.BINARY_SET_TINYINT(BIN,0,VAL);
    VAL = DBMS_BINARY.BINARY_GET_TINYINT(BIN,0);
    PRINT VAL;
END;
/
```

结果如下：

```
127
```

例 2 使用 BINARY_SET_VARCHAR 过程写入一个 VARCHAR 类型数据，并使用 BINARY_GET_VARCHAR 函数输出该 VARCHAR 类型数据。

```
DECLARE
    BIN VARBINARY(50);
    VAL VARCHAR;
BEGIN
    BIN = 'ABCDEF87';
    DBMS_BINARY.BINARY_SET_VARCHAR(BIN,0,'AAAA');
    VAL = DBMS_BINARY.BINARY_GET_VARCHAR(BIN,0,4);
    PRINT VAL;
END;
/
```

结果如下：

```
AAAA
```

5 DBMS_JOB 包

DBMS_JOB 系统包用于创建定时任务，支持按指定的时间或间隔执行用户定义的作业。

DM MPP 环境下不支持 DBMS_JOB 包。

5.1 相关方法

1. BROKEN 过程

更新一个已提交的工作的状态，典型地是用来把一个已过期工作标记为未过期工作；或者把一个未过期的工作设置为何时过期。

语法如下:

```
PROCEDURE    BROKEN(
    JOB             IN    INTEGER,
    BROKEN          IN    BOOLEAN,
    NEXT_DATE       IN    DATETIME :=SYSDATE
);
```

参数详解

- job

  工作号，唯一标识一个特定工作。

- broken

  指示此工作是否将标记为过期，TRUE 或 FALSE。TRUE 表明过期，而 FALSE 表明未过期。

- next_date

  指示在什么时候此工作将再次运行。此参数缺省值为当前日期和时间。

2. CHANGE 过程

用来改变指定工作的设置。

语法如下:

```
PROCEDURE    CHANGE (
    JOB             IN    INTEGER,
    WHAT            IN    VARCHAR2(1800),
    NEXT_DATE       IN    DATETIME,
    INTERVAL IN     VARCHAR(500),
    INSTANCEIN      INT DEFAULT 0,
    FORCE           IN    BOOLEAN DEFAULT FALSE
);
```

参数详解

- job

  工作号，唯一标识一个特定工作。

- what

  是由此工作运行的一块 PL/SQL 代码块，长度不能超过 1800 字节。

- next_date

  指示何时此工作将被执行。

- interval

  指示一个工作重执行的频度，长度不能超过 500 字节。

- instance

指定 DSC 集群环境中执行该作业的节点号，取值范围［0,15］。节点号必须是实际存在的节点。

- force

参数仅作兼容用，无实际作用。


注意：**由于 CHANGE 是保留字，调用该过程时需要在过程名上加双引号。**

3. INTERVAL 过程

用来显式地设置重执行一个工作之间的时间间隔数。

语法如下:

```
PROCEDURE    INTERVAL (
    JOB              IN    INTEGER,
    INTERVAL IN    VARCHAR(500)
);
```

参数详解

- job

工作号，唯一标识一个特定工作。

- interval

指示一个工作重执行的频度，长度不能超过 500 字节，该频度是一个日期表达式字符串，以当天 0 点 0 分为起点和该日期表达式字符串计算出来的日期时间之差，以秒为最小间隔单位。例如'sysdate + 1/86400'则表示间隔 1 秒，'sysdate + 1/1440'则表示间隔 1 分钟，

'sysdate + 1/24'则表示间隔 1 小时，'sysdate + 1'则表示间隔 1 天。达梦的 INTERVAL 参数不支持以周，月，季等单位方式指定间隔。


注意：**由于 INTERVAL 是保留字，调用该过程时需要在过程名上加双引号。**

4. ISUBMIT 过程

用来用特定的工作号提交一个工作。这个过程与 Submit()过程的唯一区别在于此 job 参数作为 IN 型参数传递且包括一个由开发者提供的工作号。如果提供的工作号已被使用，将产生一个错误。

在 DSC 环境下，ISUBMIT 创建作业时用户不需要明确指定执行节点号，系统默认指定作业的执行节点号是创建作业的节点编号。若当前节点故障，则在控制节点运行。单机场景中作业执行节点无实际意义。

语法如下:

```
PROCEDURE    ISUBMIT (
    JOB             IN    INTEGER,
    WHAT            IN    VARCHAR2(1800),
    NEXT_DATE       IN    DATETIME,
    INTERVAL IN     VARCHAR(500) DEFAULT NULL,
    NO_PARSE        IN    BOOLEAN:=FALSE
);
```

参数详解

- job

是由用户指定的工作编号，不能为负数，且必须是不存在的工作编号。

- what

是将被执行的 PL/SQL 代码块，长度不能超过 1800 字节。

- next_date

指示何时将运行这个工作。

- interval

何时这个工作将被重执行，长度不能超过 500 字节，不指定时缺省为空，表示仅执行一次。

- no_parse

指示此工作在提交时或执行时是否应进行语法分析。TRUE 指示此 PL/SQL 代码在它第一次执行时应进行语法分析，而 FALSE 指示本 PL/SQL 代码应立即进行语法分析。缺省为 FALSE。

5. NEXT_DATE 过程

用来显式地设定一个工作的执行时间。

语法如下:

```
PROCEDURE    NEXT_DATE(
    JOB              IN    INTEGER,
    NEXT_DATE    IN    DATETIME
);
```

参数详解

- job

标识一个已存在的工作。

- next_date

指示了此工作应被执行的日期与时间。

6. REMOVE 过程

用来删除一个已计划运行的工作。已正在运行的工作不能由调用过程序删除。

语法如下:

```
PROCEDURE    REMOVE(
    JOB IN    INTEGER
);
```

参数详解

- job

唯一地标识一个工作。这个参数的值是由为此工作调用 Submit()过程返回的 job 参数的值。

7. RUN 过程

用来立即执行一个指定的工作。需要说明的是，RUN 过程的调用不影响工作的计划运行时间。

语法如下:

```
PROCEDURE    RUN(
    JOB IN    INTEGER,
    FORCE    BOOLEAN DEFAULT FALSE
);
```

参数详解

- job

标识将被立即执行的工作。

- force

为 FALSE 时在当前会话同步执行；为 TRUE 时以任务的方式异步执行。默认为 FALSE。

8. SUBMIT 过程

使用 Submit()过程，工作被正常地计划好。

语法如下:

```
PROCEDURE    SUBMIT (
    JOB             OUT INTEGER,
    WHAT            IN    VARCHAR2(1800),
    NEXT_DATE       IN    DATE DEFAULT SYSDATE,
    INTERVAL        IN    VARCHAR(500) DEFAULT NULL,
    NO_PARSE        IN    BOOLEAN DEFAULT FALSE,
    INSTANCEIN      INT DEFAULT 0,
    FORCE           IN    BOOLEAN DEFAULT FALSE
);
```

参数详解

- job

是由 Submit()过程返回的工作编号。这个值用来唯一标识一个工作。

- what

是将被执行的 PL/SQL 代码块，长度不能超过 1800 字节。

- next_date

指示将何时运行这个工作，缺省值为当前的日期和时间。

- interval

何时这个工作将被重执行，长度不能超过 500 字节，不指定时缺省为空，表示仅执行一次。

- no_parse

指示此工作在提交时或执行时是否应进行语法分析，TRUE 指示此 PL/SQL 代码在它第一次执行时应进行语法分析，而 FALSE 指示本 PL/SQL 代码应立即进行语法分析。缺省为 FALSE。

- instance

这里的 instance 指定的是 DSC 集群环境中执行该作业的节点号 EP_SEQNO，取值范围［0,15］，执行节点号 EP_SEQNO 比实例号小于 1。比如：要指定集群中第 1 个实例执行作业，这里的 INSTANCE 应该设为 0。指定的节点号必须是实际存在的节点，单节点环境中单机场景中 INSTANCE 参数无实际意义，若不设置，默认在第一个实例执行作业。

- force

参数仅作兼容用，无实际作用。

9. WHAT 过程

允许在工作执行时重新设置此正在运行的命令。

语法如下:

```
PROCEDURE    WHAT (
    JOB        IN    INTEGER,
    WHAT       IN OUT    VARCHAR2(1800)
```

);

参数详解

- job

标识一个存在的工作。

- what

指示将被执行的新的 PL/SQL 代码，长度不能超过 1800 字节，如果传入的 what 参数值为空串，那将返回该任务原来的 PL/SQL 代码。

5.2 相关视图

1.DBA_JOBS 视图

描述数据库中所有的 JOB。

序号	列	数据类型	说明
1	JOB	INTEGER	工作的唯一标识号
2	LOG_USER	VARCHAR(8188)	提交工作的用户
3	PRIV_USER	VARCHAR(8188)	赋予工作权限的用户
4	SCHEMA_USER	VARCHAR(8188)	对工作做语法分析的用户模式
5	LAST_DATE	DATE	最后一次成功运行工作的时间
6	LAST_SEC	TIME(0)	HH:MM:SS 格式的 LAST_DATE 的时间
7	THIS_DATE	DATE	正在运行工作的开始时间
8	THIS_SEC	TIME(0)	HH:MM:SS 格式的 THIS_DATE 的时间
9	NEXT_DATE	VARCHAR(32767)	下一次定时任务运行的时间

10	NEXT_SEC	VARCHAR(3276 7)	HH:MM:SS 格式的 NEXT_DATE 的时间
11	TOTAL_TIME	DOUBLE	该 JOB 每次运行时间的总和，该值会进行累加
12	BROKEN	VARCHAR(1)	是否工作中断标识参数，Y 是，N 否
13	INTERVAL	VARCHAR(8188)	JOB 每次执行的时间间隔表达式
14	FAILURES	BIGINT	工作运行连续没有成功的次数
15	WHAT	TEXT	执行工作的 PL/SQL 块
16	NLS_ENV	VARCHAR(1)	工作运行的 NLS 会话设置
17	MISC_ENV	VARCHAR(1)	工作运行的其他一些会话参数
18	INSTANCE	INTEGER	能够运行或正在运行工作的实例的 id 号。 单节点上默认值为 1，如果是 DM MPP 或 DMDSC 环境。 默认值为实例序号加 1。DPC 环境下无意义

2.USER_JOBS 视图

描述当前用户所拥有的 JOB。视图结构与 DBA_JOBS 相同。

3.DBA_JOBS_RUNNING

显示实例中所有正在执行的作业。

序号	列	数据类型	说明
1	SID	BIGINT	作业执行的 SESSION 的 ID
2	JOB	INTEGER	作业号
3	FAILURES	INTEGER	作业自上一次成功以来的失败次数，暂不支持
4	LAST_DATE	DATE	最后一次成功运行工作的时间

5	LAST_SEC	TIME(0)	HH:MM:SS 格式的 LAST_DATE 的时间
6	THIS_DATE	DATE	本次运行的开始时间
7	THIS_SEC	TIME(0)	HH:MM:SS 格式的 THIS_DATE 的时间
8	INSTANCE	INTEGER	能够运行或正在运行作业的实例的 ID 号。 单节点上默认值为 1，如果是 DM MPP 或 DMDSC 环境，默认值为实例序号加 1

5.3 创建、删除、重建语句

创建或删除 DBMS_JOB 系统包。

语法如下:

```
void
SP_INIT_JOB_SYS(
    CREATE_FLAG      int
)
```

参数详解

- CREATE_FLAG

取值 0、1、2。0 表示删除 DBMS_JOB 包；1 表示创建 DBMS_JOB 包；2 表示重建 DBMS_JOB 包中的相关方法，一般用于版本升级后，包中方法发生了改变的情况下使用。

返回值

无

举例说明

例 创建 DBMS_JOB 系统包。

```
SP_INIT_JOB_SYS(1);
```

5.4 举例说明

用户在使用 DBMS_JOB 包之前，需要提前调用系统过程 SP_INIT_JOB_SYS(1)，创建好调试所需要的包，就可以使用 DBMS_JOB 来调用 PL/SQL 中的过程或函数了。

```
SP_INIT_JOB_SYS(1);
```

创建测试表

```
CREATE TABLE A(A DATETIME);
```

创建一个自定义过程

```
CREATE OR REPLACE PROCEDURE TEST AS
BEGIN
INSERT INTO A VALUES(SYSDATE);
END;
/
```

创建 JOB

```
BEGIN
DBMS_JOB.ISUBMIT(1,'TEST',SYSDATE,'SYSDATE+1/1440'); //每天 1440 分钟，即一分钟运行 TEST 过程一次
    COMMIT;
END;
/
```

查看 JOB 运行结果

```
SELECT TO_CHAR(A,'YYYY/MM/DD HH24:MI:SS') 时间 FROM A;
```

结果如下:

```
时间

----------------------------

2025/01/21    14:34:43

2025/01/21    14:35:43

2025/01/21    14:36:43

2025/01/21    14:37:43
```

删除 JOB

```
BEGIN

    DBMS_JOB.REMOVE(1);

END;

/
```

DMSQL 程序已成功完成。

6 DBMS_LOB 包

DBMS_LOB 包用于对大对象（BLOB、CLOB）进行多种操作。

6.1 相关方法

1. APPEND

追加一个源 LOB 数据到一个目标的 LOB 数据后面。

语法如下:

```
PROCEDURE APPEND(
    DEST_LOB            IN OUT    BLOB,
    SRC_LOB      IN           BLOB
);
PROCEDURE APPEND(
    DEST_LOB            IN OUT    CLOB,
    SRC_LOB      IN           CLOB
);
```

参数详解

- DSET_LOB

  目标临时大对象

- SRC_LOB

  源临时大对象

2. CLOSE

关闭一个之前打开的 LOB 或 BFILE 对象。

语法如下:

```
PROCEDURE CLOSE(
    LOB_LOC      IN OUT    BLOB
);
PROCEDURE CLOSE(
    LOB_LOC      IN OUT    CLOB,
);
PROCEDURE CLOSE(
    FILE_LOC     IN OUT    BFILE
);
```

参数详解

- LOB_LOC

  BLOB 或 CLOB 类型的大对象

- FILE_LOC

  BFILE 类型对象

3. COMPARE

比较 2 个 LOB 的数据，返回一个结果值。

语法如下:

```
FUNCTION COMPARE(
    LOB_1        IN    BLOB,
    LOB_2        IN    BLOB,
    AMOUNT       IN    BIGINT:= DBMS_LOB.LOBMAXSIZE,
    OFFSET_1     IN    BIGINT := 1,
```

```
    OFFSET_2      IN    BIGINT := 1
) RETURN INTEGER;
FUNCTION COMPARE(
    LOB_1         IN    CLOB,
    LOB_2         IN    CLOB,
    AMOUNT        IN    BIGINT:= DBMS_LOB.LOBMAXSIZE,
    OFFSET_1      IN    BIGINT := 1,
    OFFSET_2      IN    BIGINT := 1
) RETURN INTEGER;
FUNCTION COMPARE(
    LOB_1         IN    BFILE,
    LOB_2         IN    BFILE,
    AMOUNT        IN    BIGINT,
    OFFSET_1 IN   BIGINT:= 1,
    OFFSET_2IN    BIGINT:= 1
) RETURN INTEGER;
```

参数详解

- LOB_1

  第一个临时大对象

- LOB_2

  第二个临时大对象

- AMOUNT

  需要比较的总长度，BLOB 为字节长度，CLOB 为字符长度，最大取值为 LOBMAXSIZE。LOBMAXSIZE 是包内变量，为 BIGINT 类型，取值为 BIGINT 的最大值 9223372036854775807。

若调用时设置该参数大于 LOBMAXSIZE，则进行参数转换时就会报错：数据溢出。

- OFFSET_1

第一个临时大对象的起始偏移，最大取值为 LOBMAXSIZE。LOBMAXSIZE 是包内变量，为 BIGINT 类型，取值为 BIGINT 的最大值 9223372036854775807。若调用时设置该参数大于 LOBMAXSIZE，则进行参数转换时就会报错：数据溢出。

- OFFSET_2

第二个临时大对象的起始偏移，最大取值为 LOBMAXSIZE。LOBMAXSIZE 是包内变量，为 BIGINT 类型，取值为 BIGINT 的最大值 9223372036854775807。若调用时设置该参数大于 LOBMAXSIZE，则进行参数转换时就会报错：数据溢出。

返回值

0：表示相等；

-1：表示 LOB_1<LOB_2；

1：表示 LOB_1>LOB_2

NULL：表示 AMOUNT<1，OFFSET_1<1，OFFSET_2<1。

4. COPY

拷贝指定长度的源 LOB 数据插入到目标 LOB。

语法如下:

```
PROCEDURE COPY(
    DEST_LOB            IN OUT  BLOB,
    SRC_LOB             IN      BLOB,
    AMOUNT              IN      BIGINT,
    DEST_OFFSET         IN      BIGINT := 1,
    SRC_OFFSET          IN      BIGINT := 1
```

```
);
PROCEDURE COPY(
    DEST_LOB                IN OUT    CLOB,
    SRC_LOB                 IN        CLOB,
    AMOUNT                  IN        BIGINT,
    DEST_OFFSET             IN        BIGINT := 1,
    SRC_OFFSET              IN        BIGINT := 1
);
```

参数详解

- DEST_LOB

  目标临时大对象。

- SRC_LOB

  源临时大对象。

- AMOUNT

  需要拷贝的字节或字符总数，BLOB 为字节，CLOB 为字符。

- DEST_OFFSET

  写入的起始偏移，BLOB 为字节，CLOB 为字符，如果写入偏移大于原字段的长度，将自动填充原长度到指定偏移间的数据，BLOB 填充 0，CLOB 填充空格。

- SRC_OFFSET

  读取的起始偏移，BLOB 为字节，CLOB 为字符。

5. CREATETEMPORARY

创建一个临时的 LOB 对象，并存储在临时的表空间里。

语法如下:

```
PROCEDURE CREATETEMPORARY(
    LOB_LOC      IN OUT   BLOB,
    CACHE        IN       BOOLEAN,
    DUR          IN       PLS_INTEGER := DBMS_LOB.SESSION
);
PROCEDURE CREATETEMPORARY(
    LOB_LOC      IN OUT   CLOB,
    CACHE        IN       BOOLEAN,
    DUR          IN       PLS_INTEGER := 10
);
```

参数详解

- LOB_LOC

  目标临时大对象。

- CACHE

  仅支持 TRUE 操作，输入值为其它值时不报错，也不产生其它效果。

- DUR

  存活周期，支持 DBMS_LOB.SESSION 和 DBMS_LOB."CALL"。

6. ERASE

擦除指定偏移开始的指定长度 LOB 数据。BLOB 用 0，CLOB 用空格代替原有数据。

语法如下:

```
PROCEDURE ERASE(
    LOB_LOC      IN OUT   BLOB,
    AMOUNT       IN OUT   BIGINT,
```

```
    OFFSET          IN          BIGINT := 1
);
PROCEDURE ERASE(
    LOB_LOC         IN OUT      CLOB,
    AMOUNT          IN OUT      BIGINT,
    OFFSET          IN          BIGINT := 1
);
```

参数详解

- LOB_LOC

  临时大对象。

- AMOUNT

  输入需要擦除数据的总数，BLOB 为字节，CLOB 为字符，输出实际擦除的总数，AMOUNT<=0 将报错，AMOUNT 大于原字段长度，将擦写所有数据。

- OFFSET

  起始偏移。

7. FILECLOSE

关闭一个之前打开的 BFILE 对象。

语法如下:

```
PROCEDURE FILECLOSE(
    FILE_LOC IN OUT BFILE
);
```

参数详解

- FILE_LOC

BFILE 类型对象。

8. FILECLOSEALL

关闭所有之前打开的 BFILE 对象。

语法如下:

```
PROCEDURE FILECLOSEALL();
```

9. FILEEXISTS

判断指定的 BFILE 是否存在。

语法如下:

```
FUNCTION FILEEXISTS(
    FILE_LOC IN     BFILE
)RETURN INTEGER;
```

参数详解

- FILE_LOC

BFILE 类型对象。

返回值

0：不存在

1：存在

10. FILEGETNAME

获取 BFILE 对象的目录名和文件名。

语法如下:

```
PROCEDURE FILEGETNAME(
```

```
    FILE_LOC IN    BFILE,
    DIR_ALIAS      OUT VARHCAR,
    FILEANME OUT VARCHAR
);
```

参数详解

- FILE_LOC

  BFILE 类型对象。

- DIR_ALIAS

  目录对象名。

- FILENAME

  文件名。

11. FILEISOPEN

判断指定的 BFILE 是否已经打开。

语法如下:

```
FUNCITON FILEISOPEN(
    FILE_LOC IN    BFILE
) RETURN INTEGER;
```

参数详解

- FILE_LOC

  BFILE 类型对象。

返回值

0：未打开

1：打开

12. FILEOPEN

打开一个 BFILE 文件。

语法如下:

```
PROCEDURE FILEOPEN(
    FILE_LOC      IN OUT   BFILE,
    OPEN_MODE     IN       INTEGER:=FILE_READONLY
);
```

参数详解

- FILE_LOC

  BFILE 类型对象。

- OPEN_MODE

  打开方式。仅支持对 BFILE 对象使用 FILE_READONLY 参数。对 BLOB 或 CLOB 对象使用参数 LOB_READONLY、LOB_READWRITE 时，仅提供语法支持，不报错，也不产生其它效果。

13. FRAGMENT_DELETE

删除指定偏移开始的指定长度的数据，不重写。

语法如下:

```
PROCEDURE FRAGMENT_DELETE(
    LOB_LOC       IN OUT   BLOB,
    AMOUNT        IN       BIGINT,
    OFFSET        IN       BIGINT
);
```

```
PROCEDURE FRAGMENT_DELETE(
    LOB_LOC      IN OUT   CLOB,
    AMOUNT       IN       BIGINT,
    OFFSET       IN       BIGINT
);
```

参数详解

- LOB_LOC

  临时大对象。

- AMOUNT

  需要删除数据的总数，BLOB 为字节，CLOB 为字符。

- OFFSET

  起始偏移。BLOB 为字节，CLOB 为字符。

错误说明:

OFFSET<1，OFFSET 超过原字段长，AMOUNT 大于原字段长，AMOUNT + OFFSET 大于原字段长，返回错误。

14. FRAGMENT_INSERT

插入指定长度的数据到 LOB 的指定偏移处，最大为 32K。

语法如下:

```
PROCEDURE FRAGMENT_INSERT(
    LOB_LOC      IN OUT   BLOB,
    AMOUNT       IN       INTEGER,
    OFFSET       IN       BIGINT,
    BUFFER       IN       VARBINARY
```

```
);

PROCEDURE FRAGMENT_INSERT(

    LOB_LOC      IN OUT   CLOB,

    AMOUNT       IN       INTEGER,

    OFFSET       IN       BIGINT,

    BUFFER       IN       VARCHAR

);
```

参数详解

- LOB_LOC

  临时大对象。

- AMOUNT

  需要插入数据的总数，BLOB 为字节，CLOB 为字符，最大 32K。

- OFFSET

  写入起始的偏移，BLOB 为字节，CLOB 为字符。

- BUFFER

  待写入数据。

错误说明:

AMOUNT 大于 BUFFER 内数据长度，AMOUNT < = 0，OFFSET < = 0，返回错误。

15. FRAGMENT_MOVE

移动指定长度的数据从原始偏移到新偏移处。

语法如下:

```
PROCEDURE FRAGMENT_MOVE(

    LOB_LOC          IN OUT   BLOB,
```

```
    AMOUNT          IN      BIGINT,
    SRC_OFFSET      IN      BIGINT,
    DEST_OFFSET     IN      BIGINT
);
PROCEDURE FRAGMENT_MOVE(
    LOB_LOC         IN OUT  CLOB,
    AMOUNT          IN      BIGINT,
    SRC_OFFSET      IN      BIGINT,
    DEST_OFFSET     IN      BIGINT
);
```

参数详解

- LOB_LOC

  临时大对象。

- AMOUNT

  需要移动的数据总数，BLOB 为字节，CLOB 为字符。

- SRC_OFFSET

  待移动数据的起始偏移。BLOB 为字节，CLOB 为字符。

- DEST_OFFSET

  目标偏移。BLOB 为字节，CLOB 为字符。DEST_OFFSET 不能在(SRC_OFFSET, SRC_OFFSET + AMOUNT)上取值。

错误说明:

当 SRC_OFFSET 或 DEST_OFFSET 小于 1，SRC_OFFSET 或 DEST_OFFSET 大于原字段长，AMOUNT 小于 0，AMOUNT + SRCOFFSET 大于原字段长度时，返回错误。

16. FRAGMENT_REPLACE

替换从指定偏移位置的指定长度的数据为新数据，长度不超过 32K。

语法如下:

```
PROCEDURE FRAGMENT_REPLACE(
    LOB_LOC       IN OUT   BLOB,
    OLD_AMOUNT    IN       BIGINT,
    NEW_AMOUNT    IN       BIGINT,
    OFFSET        IN       BIGINT,
    BUFFER        IN       VARBINARY
);
PROCEDURE FRAGMENT_REPLACE(
    LOB_LOC       IN OUT   CLOB,
    OLD_AMOUNT    IN       BIGINT,
    NEW_AMOUNT    IN       BIGINT,
    OFFSET        IN       BIGINT,
    BUFFER        IN       VARCHAR2
);
```

参数详解

- LOB_LOC

  临时大对象。

- OLD_AMOUNT

  需要被替换数据的总数，BLOB 为字节，CLOB 为字符。

- NEW_AMOUNT

  替换数据的总数，BLOB 为字节，CLOB 为字符，最大 32K。

- OFFSET

  被替换数据的起始偏移。

- BUFFER

  需要替换的数据。

使用说明

OFFSET 小于 1， OLD_AMOUNT 或 NEW_AMOUNT 小于 0，OLD_AMOUNT 或者 NEW_AMOUNT 大于 32K，NEW_AMOUNT 大于 BUFFER 内数据长度，将返回错误。

17. FREETEMPORARY

释放临时的 LOB 操作符。

语法如下:

```
PROCEDURE FREETEMPORARY(LOB_LOC IN OUT BLOB);

PROCEDURE FREETEMPORARY(LOB_LOC IN OUT CLOB);
```

参数详解

- LOB_LOC

  需要被释放的临时大对象

18. GETLENGTH

返回 LOB 数据的字符长度。

语法如下:

```
FUNCTION GETLENGTH(LOB_LOC IN BLOB) RETURN BIGINT;

FUNCTION GETLENGTH(LOB_LOC IN CLOB) RETURN BIGINT;

FUNCTION GETLENGTH(FILE_LOC IN BFILE) RETURN BIGINT;
```

参数详解

- LOB_LOC

  临时大对象。

- FILE_LOC

  BFILE 类型对象。

  返回值

  临时大对象的数据长度。BLOB 为字节，CLOB 为字符。

19. GET_STORAGE_LIMIT

返回指定 LOB 的存储长度上限。

语法如下:

```
FUNCTION GET_STORAGE_LIMIT(LOB_LOC IN BLOB) RETURN BIGINT;

FUNCTION GET_STORAGE_LIMIT(LOB_LOC IN CLOB) RETURN BIGINT;
```

参数详解

- LOB_LOC

  大字段对象。

  返回值

  大字段存储限制大小。DM 等同于最大长度。

20. INSTR

返回从指定偏移开始的第 NTH 个匹配对象的位置。

语法如下:

```
FUNCTION INSTR(
    LOB_LOC      IN   BLOB,
    PATTERN      IN   VARBINARY,
```

```
    OFFSET          IN    BIGINT := 1,
    NTH             IN    BIGINT:= 1
) RETURN BIGINT;
FUNCTION INSTR(
    LOB_LOC         IN    CLOB,
    PATTERN         IN    VARCHAR2,
    OFFSET          IN    BIGINT:= 1,
    NTH             IN    BIGINT:= 1
) RETURN BIGINT;
```

参数详解

- LOB_LOC

  临时大对象。

- PATTERN

  匹配符。

- OFFSET

  查找起始偏移，BLOB 为字节，CLOB 为字符。

- NTH

  查找的第几个匹配符。

返回值

找到的匹配符出现的偏移。BLOB 为字节，CLOB 为字符，找不到匹配串返回 0。

错误说明:

lob_loc 为 NULL、offset<1 或 nth<1，返回 NULL。

offset>LOBMAXSIZE 或 nth>LOBMAXSIZE，返回错误。

21. ISOPEN

判断指定的 LOB 对象是否已经打开。

语法如下:

```
FUNCTION ISOPEN(LOB_LOC IN BLOB) RETURN INTEGER;

FUNCTION ISOPEN(LOB_LOC IN CLOB) RETURN INTEGER;

FUNCTION ISOPEN(FILE_LOC IN BFILE) RETURN INTEGER;
```

参数详解

- LOB_LOC

  LOB 类型对象。

- FILE_LOC

  BFILE 类型对象。

返回值

0: 未打开

1: 打开

22. ISTEMPORARY

返回是否指定的 LOB 是临时操作符。

语法如下:

```
FUNCTION ISTEMPORARY(LOB_LOC IN BLOB) RETURN INTEGER;

FUNCTION ISTEMPORARY(LOB_LOC IN CLOB) RETURN INTEGER;
```

参数详解

- LOB_LOC

  大对象。

返回值

1：是临时大对象；

0：不是临时大对象；

NULL：输入参数为 NULL 或没有与表的大字段列进行关联的大字段对象。

23. LOADBLOBFROMFILE

从 BFILE 对象加载数据到 BLOB 对象。

语法如下：

```
PROCEDURE LOADBLOBFROMFILE(
    DEST_LOB        IN OUT  BLOB,
    SRC_BFILE       IN      BFILE,
    AMOUNT          IN      BIGINT,
    DEST_OFFSET     IN OUT  BIGINT,
    SRC_OFFSET      IN OUT  BIGINT
);
```

参数详解

- DEST_LOB

  目标 BLOB 对象。

- SRC_BFILE

  源 BFILE 对象。

- AMOUNT

  要从 BFILE 对象加载到 BLOB 对象的字节数。

- DEST_OFFSET

  开始写入 BLOB 对象的偏移。

- SRC_OFFSET

从 BFILE 对象开始读取的偏移。

24. LOADCLOBFROMFILE

从 BFILE 对象加载数据到 CLOB 对象。

语法如下:

```
PROCEDURE LOADCLOBFROMFILE(
    DEST_LOB            IN OUT      CLOB,
    SRC_BFILE           IN          BFILE,
    AMOUNT              IN          BIGINT,
    DEST_OFFSET         IN OUT      BIGINT,
    SRC_OFFSET          IN OUT      BIGINT,
    BFILE_CSID          IN          NUMBER,
    LANG_CONTEXT        IN OUT      INTEGER,
    WARNING             OUT         INTEGER
);
```

参数详解

- DEST_LOB

  输入：目标 CLOB 对象。输出：结果 CLOB 对象。

- SRC_BFILE

  源 BFILE 对象。

- AMOUNT

  要从 BFILE 对象加载到 CLOB 对象的字节数。

- DEST_OFFSET

  输入：开始写入 CLOB 对象的字符偏移。输出：结束写入的字符偏移。

- SRC_OFFSET

输入：从 BFILE 对象开始读取的字节偏移，用户需确保此偏移位置为某字符的第一个字节。输出：结束读取的字节偏移。

- BFILE_CSID

源 BFILE 的编码集。

支持的编码集：

编码集名称	编号	说明
DEFAULT_CSID	0	缺省值，编码与数据库一致
US7ASCII	1	ASCII，保持策略与缺省值一致
CL8KOI8R	196	KOI8-R，俄文编码
JA16EUC	830	EUC-JP，日文编码
ZHS16CGB231280	850	GB2312，中文编码
ZHS16GBK	852	GBK，中文编码
ZHS32GB18030	854	GB18030，中文编码
ZHT16BIG5	865	BIG5，繁体中文编码
UTF8	871	UTF8，未做区分 支持带 BOM（{0XEF，0XBB，0XBF}）或不带 BOM 输入。带 BOM 时去掉 BOM，再进行编码转换；不带 BOM 时直接进行编码转换。编码转换的规则与其他编码相同。若只输入了 BOM，得到的 CLOB 为输入的 CLOB。
AL32UTF8	873	

UTF16	1000	UTF16，自适应大小端 支持带 BOM({0XFE, 0XFF})或不带 BOM 输入。基本逻辑同 UTF8，但会判断输入的大小端标识，并根据编译器大小端进行大小端转换，没有读到大小端标识时默认为小端。
AL16UTF16	2000	UTF16，大端 支持带 BOM({0XFE, 0XFF})或不带 BOM 输入。基本逻辑同 UTF8，但会根据编译器的大小端进行大小端转换。
AL16UTF16LE	2002	UTF16，小端 支持带 BOM({0XFF, 0XFE})或不带 BOM 输入。基本逻辑同 UTF8，但会根据编译器的大小端进行大小端转换。

- LANG_CONTEXT，WARNING

  这两个参数暂时不支持，调用时指定任意值即可。

  错误说明:

  输入参数为 NULL，返回错误。

  AMOUNT、SRC_OFFSET 或 DEST_OFFSET 小于 1 或大于 LOBMAXSIZE，返回错误。

  SRC_OFFSET 大于文件长度、DEST_OFFSET+AMOUNT-1 大于 LOB 最大长度、AMOUNT+SRC_OFFSET-1 超过文件长度，返回错误。

  对于 EUC_KR 编码的数据库，BFILE_CSID 仅支持缺省值，若为其他值，返回错误。

25. LOADFROMFILE

从 BFILE 对象加载数据到 BLOB 对象。

语法如下:

```
PROCEDURE LOADFROMFILE(
    DEST_LOB            IN OUT    BLOB,
```

```
    SRC_FILE        IN          BFILE,
    AMOUNT          IN          BIGINT,
    DEST_OFFSET     IN OUT      BIGINT:=1,
    SRC_OFFSET      IN OUT      BIGINT:=1
);
```

参数详解

- DEST_LOB

  目标 BLOB 对象。

- SRC_FILE

  源 BFILE 对象。

- AMOUNT

  要从 BFILE 对象加载到 BLOB 对象的字节数。

- DEST_OFFSET

  开始写入 BLOB 对象的偏移。

- SRC_OFFSET

  从 BFILE 对象开始读取的偏移。

26. OPEN

打开一个 LOB 对象。

语法如下:

```
PROCEDURE OPEN(
    LOB_LOC         IN OUT      BLOB,
    OPEN_MODE       IN          INTEGER
);
```

```
PROCEDURE OPEN(
    LOB_LOC      IN OUT   CLOB,
    OPEN_MODE    IN       INTEGER
);
PROCEDURE OPEN(
    FILE_LOC     IN OUT   BFILE,
    OPEN_MODE    IN       INTEGER:=FILE_READONLY
);
```

参数详解

- LOB_LOC

  LOB 对象。

- FILE_LOC

  BFILE 类型对象。

- OPEN_MODE

  打开方式。仅支持对 BFILE 对象使用 FILE_READONLY 参数。参数 LOB_READONLY、LOB_READWRITE，仅提供语法支持，使用时不报错，也不产生其它效果。

27. READ

读取指定偏移开始的指定长度数据到 buffer 中。

语法如下:

```
PROCEDURE READ(
    LOB_LOC      IN       BLOB,
    AMOUNT       IN OUT   INTEGER,
    OFFSET       IN       BIGINT,
```

```
    BUFFER          OUT     VARBINARY
);
PROCEDURE READ(
    LOB_LOC         IN      CLOB,
    AMOUNT          IN OUT  INTEGER,
    OFFSET          IN      BIGINT,
    BUFFER          OUT     VARCHAR2
);
PROCEDURE READ(
    FILE_LOC        IN      BFILE,
    AMOUNT          IN OUT  INTEGER,
    OFFSET          IN      BIGINT,
    BUFFER          OUT     VARBINARY
);
```

参数详解

- LOB_LOC

  临时大对象。

- FILE_LOC

  BFILE 对象。

- AMOUNT

  输入需要读取的数据总数，BLOB 为字节，CLOB 为字符，输出为实际读取的总数。

- OFFSET

  读取数据的偏移，BLOB 为字节，CLOB 为字符。

- BUFFER

存储数据的缓冲区。

错误说明:

AMOUNT 为 NULL、小于等于 0 或大于 32767，返回错误。

OFFSET 为 NULL、小于等于 0 或超过原字段长度，返回错误。

28. SUBSTR

获取指定偏移开始的指定长度的子串。

语法如下:

```
FUNCTION SUBSTR(
    LOB_LOC     IN  BLOB,
    AMOUNT      IN  INTEGER := 32767,
    OFFSET      IN  BIGINT := 1
) RETURN VARBINARY;
FUNCTION SUBSTR(
    LOB_LOC     IN  CLOB,
    AMOUNT      IN  INTEGER := 32767,
    OFFSET      IN  BIGINT := 1
) RETURN VARCHAR2;
FUNCTION SUBSTR(
    FILE_LOC    IN  BFILE,
    AMOUNT      IN  INTEGER := 32767,
    OFFSET      IN  BIGINT := 1
) RETURN VARBINARY;
```

参数详解

- LOB_LOC

  临时大对象。

- FILE_LOC

  BFILE 对象。

- AMOUNT

  读取的数据长度，BLOB 为字节，CLOB 为字符，上限为 32767，实际获取为大对象长度与 AMOUNT 的小值。

- OFFSET

  读取数据的起始偏移。

返回值

返回指定偏移开始的指定长度的子串。若输入的 AMOUNT 或 OFFSET 参数非法，返回 NULL。

29. TRIM/TRIM_LOB

截断 LOB 数据到指定的长度。

语法如下:

```
PROCEDURE TRIM(
    LOB_LOC       IN OUT    BLOB,
    NEW_LEN       IN        BIGINT
);
PROCEDURE TRIM(
    LOB_LOC             IN OUT    CLOB,
    NEW_LEN       IN        BIGINT
);
```

```
PROCEDURE TRIM_LOB(
    LOB_LOC          IN OUT   BLOB,
    NEW_LEN          IN       BIGINT
);
PROCEDURE TRIM_LOB(
    LOB_LOC          IN OUT   CLOB,
    NEW_LEN          IN       BIGINT
);
```

参数详解

- LOB_LOC

  临时大对象。

- NEW_LEN

  截断到的新长度，BLOB 为字节，CLOB 为字符。

30. WRITEAPPEND

追加指定长度字符的数据到 LOB 对象的末尾。

语法如下:

```
PROCEDURE WRITEAPPEND(
    LOB_LOC          IN OUT   BLOB,
    AMOUNT  IN       INTEGER,
    BUFFER           IN       VARBINARY
);
PROCEDURE WRITEAPPEND(
    LOB_LOC          IN OUT   CLOB,
```

```
    AMOUNT  IN          INTEGER,
    BUFFER          IN          VARCHAR2
);
```

参数详解

- LOB_LOC

  临时大对象。

- AMOUNT

  追加数据的总数，BLOB 为字节，CLOB 为字符。

- BUFFER

  待追加的数据。

错误说明:

AMOUNT 小于等于 0 或 NULL 或者超过 BUFFER 内数据长度，返回错误。

31. WRITE

写入指定长度字符的数据到 LOB 对象，从 LOB 对象的指定绝对偏移开始。如果指定的位置有数据，则新写入的会覆盖原来的数据。

语法如下:

```
PROCEDURE WRITE (
    LOB_LOC             IN OUT      BLOB,
    AMOUNT  IN          INTEGER,
    OFFSET          IN          BIGINT,
    BUFFER          IN          VARBINARY
);
PROCEDURE WRITE (
```

```
    LOB_LOC             IN OUT    CLOB,
    AMOUNT  IN          INTEGER,
    OFFSET          IN          BIGINT,
    BUFFER          IN          VARCHAR2
);
```

参数详解

- LOB_LOC

  临时大对象。

- AMOUNT

  写入数据的总数，BLOB 为字节，CLOB 为字符。

- OFFSET

  偏移量，起始为 1。BLOB 为字节，CLOB 为字符。

- BUFFER

  待写入的数据。

错误说明:

AMOUNT 小于等于 0 或 NULL 或者超过 BUFFER 内数据长度，返回错误。

32. CONVERTTOBLOB

将指定的 LOB 的指定部分按一定的编码格式完成 BLOB 和 CLOB 的互相转换。

语法如下:

```
PROCEDURE CONVERTTOBLOB(
    DEST_LOB                IN OUT          BLOB,
    SRC_CLOB            IN              CLOB,
    AMOUNT              IN              BIGINT,
```

```
    DEST_OFFSET         IN OUT      BIGINT,
    SRC_OFFSET          IN OUT      BIGINT,
    BLOB_CSID           IN          NUMBER,
    LANG_CONTEXT        IN OUT      INTEGER,
    WARNING             OUT         INTEGER
);

PROCEDURE CONVERTTOCLOB(
    DEST_LOB            IN OUT      CLOB,
    SRC_BLOB            IN          BLOB,
    AMOUNT              IN          BIGINT,
    DEST_OFFSET         IN OUT      BIGINT,
    SRC_OFFSET          IN OUT      BIGINT,
    BLOB_CSID           IN          NUMBER,
    LANG_CONTEXT        IN OUT      INTEGER,
    WARNING             OUT         INTEGER
);
```

参数详解

- DEST_LOB

输入输出参数，转换的结果将存入到这个大字段中，如果这个大字段原来已经有数据，新转换出的数据会根据指定的目标偏移覆盖原数据。

- SRC_LOB

待转换的数据。

- AMOUNT

转换的数据总数，BLOB 为字节，CLOB 为字符。

- DEST_OFFSET

  完成转换的数据向目的大字段存放时的偏移。

- SRC_OFFSET

  源数据开始转换的偏移。

- BLOB_CSID,LANG_CONTEXT,WARNING

  这三个参数暂时不支持，调用时指定任意值即可，CLOB 转 BLOB 时，最后 BLOB 编码格式和 CLOB 的存入格式相同。

错误说明:

如果 AMOUNT 大于 SRC_LOB 的长度、AMOUNT 小于 0、 DEST_OFFSET +AMOUNT 大于大字段限制最大长度（DBMS_LOB.LOBMAXSIZE），返回错误。

33. GETCHUNKSIZE

计算出块大小的字节数。

语法如下:

```
FUNCTION GETCHUNKSIZE(
    LOB_LOC     IN    BLOB
) RETURN INT;

FUNCTION GETCHUNKSIZE(
    LOB_LOC     IN    CLOB
) RETURN INT;
```

参数详解

- LOB_LOC

输入参数，LOB 类型对象。

返回值

返回控制项当前最大项的相对页大小的整数倍乘以页大小的字节数。控制项为 0 时返回当前大字段相对页大小的整数倍乘以页大小的字节数。

6.2 举例说明

使用包内的过程和函数之前，如果还未创建过系统包，请先调用系统过程创建系统包。

```
SP_CREATE_SYSTEM_PACKAGES (1,'DBMS_LOB');
```

例 将输入的 BLOB 类型数据转化为 CLOB 类型。

```
CREATE OR REPLACE FUNCTION BLOBTOCLOB  ( BLOB_IN IN BLOB)
        RETURN CLOB
AS
        V_CLOB CLOB;
        V_VARCHAR VARCHAR2(32767);
        V_START PLS_INTEGER  := 1;
        V_BUFFER PLS_INTEGER := 32767;
BEGIN
        DBMS_LOB.CREATETEMPORARY(V_CLOB, TRUE);
        FOR I IN 1 .. FLOOR(DBMS_LOB.GETLENGTH(BLOB_IN) / V_BUFFER) + 1
        LOOP
            V_VARCHAR := UTL_RAW.CAST_TO_VARCHAR2(DBMS_LOB.SUBSTR(BLOB_IN,
V_BUFFER, V_START));
            DBMS_LOB.WRITEAPPEND(V_CLOB, LENGTH(V_VARCHAR), V_VARCHAR);
            PRINT V_CLOB;
```

```
            V_START := V_START + V_BUFFER;

        END LOOP;

        RETURN V_CLOB;

END BLOBTOCLOB;

/
```

调用函数:

```
SELECT BLOBTOCLOB('B4EFC3CECAFDBEDDBFE2D3D0CFDEB9ABCBBE');
```

结果如下:

```
达梦数据库有限公司
```

7 DBMS_LOCK 包

DBMS_LOCK 包提供了锁管理服务器的接口。该包可以完成下列功能:

1. 提供对设备的独占访问;
2. 提供程序级的读锁;
3. 判断程序之后何时释放锁;
4. 同步程序以及强制顺序执行。

DM MPP 环境下不支持 DBMS_LOCK 包。

7.1 封锁规则

1、封锁类型

DBMS_LOCK 包过程函数对于封锁的类型定义如下表所示，其数据类型为 INTEGER。

名称	意义	值	描述
NL_MODE	NULL	1	
SS_MODE	Sub Shared	2	可以使用在聚合对象以指定在该对象的子部分上获取的共享锁
SX_MODE	Sub eXclusive Row Exclusive Mode	3	可以使用在聚合对象以指定在该对象的子部分上获取的独占锁
S_MODE	Shared Row Exclusive Mode Intended Exclusive	4	共享锁
SSX_MODE	Shared Sub eXclusive Share Row Exclusive Mode	5	该模式表名整个聚合对象占有共享锁，但其某些组成部分会上独占锁（S_MODE+SX_MODE）

X_MODE	eXclusive	6	独占锁

2、兼容规则

若当前锁对象已被其他程序封锁，新封锁的类型兼容规则如下：

当前持有	新加					
	NL	SS	SX	S	SSX	X
NL	Success	Success	Success	Success	Success	Success
SS	Success	Success	Success	Success	Success	Fail
SX	Success	Success	Success	Fail	Fail	Fail
S	Success	Success	Fail	Success	Fail	Fail
SSX	Success	Success	Fail	Fail	Fail	Fail
X	Success	Fail	Fail	Fail	Fail	Fail

7.2 相关方法

1. ALLOCATE_UNIQUE

申请一个锁对象。已经申请的锁对象可在 SYS.DBMS_LOCK_ALLOCATED 表查询。

语法如下：

```
PROCEDURE  ALLOCATE_UNIQUE (
    LOCK_NAME            IN      VARCHAR2,
    LOCK_HANDLE          OUT     VARCHAR2,
    EXPIRATION_SECS      IN      INTEGER  DEFAULT 864000
);
```

参数详解

- LOCK_NAME

输入参数，锁对象名称。

- LOCK_HANDLE

输出参数，锁对象句柄，根据锁名生成的唯一标识；后续过程可以使用该标识对锁对象进行操作。会话对一个新的锁名调用该过程时，会在系统中新生成一个锁对象，在 DBMS_LOCK_ALLOCATED 表中插入一行数据记录锁的相关信息。

- EXPIRATION_SECS

输入参数，锁对象的过期时间。最后一次调用 ALLOCATE_UNIQUE 后允许从 DBMS_LOCK_ALLOCATED 表中删除数据的等待时间，单位为秒。

2. REQUEST

根据指定类型进行封锁。

语法如下:

```
FUNCTION  REQUEST(
    LOCK_ID              IN      INTEGER,
    LOCK_MODE            IN      INTEGER  DEFAULT  X_MODE,
    TIMEOUT              IN      INTEGER  DEFAULT  MAXWAIT,
    RELEASE_ON_COMMIT    IN      BOOLEAN DEFAULT  FALSE
)RETURN INTEGER;
FUNCTION  REQUEST(
    LOCK_HANDLE          IN      VARCHAR2,
    LOCK_MODE            IN      INTEGER DEFAULT X_MODE,
    TIMEOUT              IN      INTEGER DEFAULT MAXWAIT,
    RELEASE_ON_COMMIT    IN      BOOLEAN DEFAULT FALSE
)RETURN INTEGER;
```

参数详解

- LOCK_ID 或 LOCK_HANDLE

输入参数，申请进行封锁动作的锁对象 ID 或句柄。

- LOCK_MODE

  输入参数，上锁的模式。

- TIMEOUT

  输入参数，尝试上锁的等待时间，单位为秒。

- RELEASE_ON_COMMIT

  输入参数，指明在事务结束时是否释放锁。

返回值说明:

返回值	描述
0	成功
1	超时
2	死锁
3	参数错误
4	没有拥有锁
5	无效的锁 id 或句柄

使用说明:

若使用 LOCK_ID 进行封锁，锁对象的 ID 取值范围为 0~1073741823。否则返回 3（参数错误）。

使用名称 ALLOCATE_UNIQUE 的锁对象的 ID 范围为 1073741824~1999999999。也就是说，使用 ALLOCATE_UNIQUE 方式创建的表对象不能通过 LOCK_ID 的方式进行封锁转化和释放。同时可以使用 LOCK_ID 不进行 ALLOCATE_UNIQUE 而直接 REQUEST，但通过 LOCK_ID 方式使用的锁对象不会插入到 DBMS_LOCK_ALLOCATED 表中。

3. CONVERT/CONVERT2

对封锁类型进行修改。

语法如下:

```
FUNCTION  CONVERT(
    LOCK_ID          IN      INTEGER,
    LOCK_MODE        IN      INTEGER,
    TIMEOUT          IN      NUMBER  DEFAULT  MAXWAIT
)RETURN INTEGER;
FUNCTION  CONVERT(
    LOCK_HANDLE      IN      VARCHAR2,
    LOCK_MODE        IN      INTEGER,
    TIMEOUT          IN      NUMBER  DEFAULT  MAXWAIT
)RETURN INTEGER;
FUNCTION  CONVERT2(
    LOCK_ID          IN      INTEGER,
    LOCK_MODE        IN      INTEGER,
    TIMEOUT          IN      NUMBER  DEFAULT  MAXWAIT
)RETURN INTEGER;
FUNCTION  CONVERT2(
    LOCK_HANDLE      IN      VARCHAR2,
    LOCK_MODE        IN      INTEGER,
    TIMEOUT          IN      NUMBER  DEFAULT  MAXWAIT
)RETURN INTEGER;
```

参数详解

- LOCK_ID 或 LOCK_HANDLE

  输入参数，要修改的锁对象 ID 或根据过程 ALLCOATE_UNIQUE 得到的锁对象句柄;

LOCK_ID 的说明同 REQUEST。

- LOCK_MODE

  对锁对象新的封锁模式。

- TIMEOUT

  尝试修改的等待时间，如果在该阶段内没有成功，返回 1（超时）。

返回值说明:

返回值	描述
0	成功
1	超时
2	死锁
3	参数错误
4	没有拥有锁
5	无效的锁的 id 或句柄

使用说明:

如果只有当前程序对该锁对象进行了封锁，可以成功转化到其他任意封锁模式；

如果与其他程序并发封锁，新的上锁模式是否能够成功要取决于其他程序所持锁的模式与新的封锁模式是否兼容，如果不兼容，则上锁失败；

由于 CONVERT 是 DM 数据库的保留字，调用 CONVERT 函数时需要使用""对其进行引用，或者使用同样功能的 CONVERT2。

4. RELEASE 函数

DBMS_LOCK 包默认在会话结束时释放对锁对象的封锁，该函数用于显式地释放当前会话持有的一个锁对象的封锁。

语法如下:

```
FUNCTION  RELEASE (
```

```
    LOCK_ID             IN          INTEGER
)RETURN INTEGER;
FUNCTION   RELEASE (
    LOCK_HANDLE     IN          VARCHAR2
)RETURN INTEGER;
```

参数详解

- LOCK_ID 或 LOCK_HANDLE

  输入参数，要释放封锁的锁对象 ID 或根据过程 ALLCOATE_UNIQUE 返回的句柄；LOCK_ID 的说明同 REQUEST。

返回值说明：

返回值	描述
0	成功
3	参数错误
4	没有拥有锁
5	无效的锁的 id 或句柄

5. SLEEP 过程

使会话休眠一段时间。

语法如下：

```
PROCEDURE SLEEP (
    SECONDS     IN          NUMBER
);
```

参数详解

- SECONDS

输入参数，休眠时间，单位为秒。

7.3 举例说明

使用包内的过程和函数之前，如果还未创建过系统包，请先调用系统过程创建系统包。

```
SP_CREATE_SYSTEM_PACKAGES (1,'DBMS_LOCK');

SET SERVEROUTPUT ON;  //PRINT 需要设置这条语句，才能打印出消息
```

如果要通过名称来进行封锁，首先要通过 ALLOCATE_UNIQUE 过程申请封锁句柄，之后通过该句柄进行封锁和释放操作。如下例以'my_lock'申请句柄并上 X 锁。

```
DECLARE
    HDL VARCHAR(128);
    RET INT;
BEGIN
    DBMS_LOCK.ALLOCATE_UNIQUE('MY_LOCK', HDL);
    SELECT DBMS_LOCK.REQUEST(HDL, 6) INTO RET;
    PRINT RET;
END;
/
```

执行成功后，其他会话同上例执行封锁，会因封锁不兼容而进行等待。

若已经进行了封锁，需要对封锁的类型进行修改，比如，上述例子中进行 X 封锁的级别太高，导致其他会话无法对'my_lock'进行封锁，可将其转换为 S 锁。

```
DECLARE
    HDL VARCHAR(128);
    RET INT;
BEGIN
```

```
    DBMS_LOCK.ALLOCATE_UNIQUE('MY_LOCK', HDL);
    SELECT DBMS_LOCK."CONVERT"(HDL, 4) INTO RET;
    PRINT RET;
END;
/
```

转换成功后，其他会话以与 S 锁兼容的模式（比如 NL，SS，S 模式）进行封锁均可成功。

若已经完成了封锁内需要的操作，可以以如下方式释放该封锁。

```
DECLARE
    HDL VARCHAR(128);
BEGIN
    DBMS_LOCK.ALLOCATE_UNIQUE('MY_LOCK', HDL);
    DBMS_LOCK.RELEASE(HDL);
END;
/
```

上述各例中，均通过句柄进行封锁操作，用户也可直接通过 id 进行封锁。封锁和转换时省略超时时间表示一直等待直至成功，用户可根据需要设置超时时间，也可将超时时间设置为 0 使其立刻返回。

通过名称申请句柄的锁对象，会添加在系统表 SYS.DBMS_LOCK_ALLOCATED 中查询得到。该系统表中的数据在锁对象过期后根据情况进行淘汰。

如上操作后，查询可以得到'MY_LOCK'的相关信息，包括名称、id 及过期时间。

```
SELECT * FROM SYS.DBMS_LOCK_ALLOCATED;
//查询结果如下:
行号        NAME      LOCKID        EXPIRATION
---------- ------- ----------- ----------------------------
```

```
1              MY_LOCK 1073741824    2014-07-20 16:19:59.708000
```

DM 也提供动态性能视图 V$DBMS_LOCKS 供用户查看当前系统中 DBMS_LOCK 封锁的情况。如两个会话并发申请'MY_LOCK'，其中 S 锁封锁成功，X 锁封锁等待的情况，查询可得如下结果：

```
SELECT * FROM V$DBMS_LOCKS;

//查询结果如下:

行号 HANDLE      ID               MODE OWN_SESSWAIT_SESS

- ----------- ---- --------------- ------ ------ ------ -----------

1    10737418241073741824176 1073741824   S      515604448            NULL

2      10737418241073741824176 1073741824   X      516931992            515604448
```

通过记录可以看出 X 封锁在等待 S 封锁。

8 DBMS_LOGMNR 包

用户可以使用 DBMS_LOGMNR 包对归档日志进行挖掘，重构出 DDL 和 DML 等操作，并通过获取的信息进行更深入的分析。

目前 DBMS_LOGMNR 只支持对归档日志进行分析，配置归档后，还需要将 dm.ini 中的 RLOG_APPEND_LOGIC 选项置为 1、2、3 或 4。

DMDPC 使用 DBMS_LOGMNR 时， DBMS_LOGMNR.ADD_LOGFILE 只能添加同一个节点的多个日志同时进行分析，不支持同时分析不同节点的日志。

8.1 相关方法

1. ADD_LOGFILE

增加日志文件。

语法如下:

```
PROCEDURE    ADD_LOGFILE (
    LOGFILENAME         IN          VARCHAR,
    OPTIONS             IN          INT         DEFAULT   DBMS_LOGMNR.ADDFILE,
    RAFTID              IN          INT
);
```

参数详解

- LogFileName

增加的文件的全路径。

- Options

可选配置参数，具体的可选配置包括:

- DBMS_LOGMNR."NEW"结束当前 LOGMNR(调用 LOGMNR_END),并增加指定文件(如果已经 START，则不可增加)
- DBMS_LOGMNR.ADDFILE 在当前 LOGMNR 中增加日志文件（如果已经 START，则不可增加)
- DBMS_LOGMNR.REMOVEFILE 从当前 LOGMNR 中去除一个日志文件

- RAFTID

用于 DMDPC 环境下指定 RAFT 组的 RAFT_ID，后续 DBMS_LOGMNR 包相关操作都在该 RAFT 组的主节点执行。非 DMDPC 环境忽略此参数。当调用 END_LOGMNR()或 REMOVE_LOGFILE()清除所有日志时，指定的 RAFTID 不再生效。若想 RAFTID 继续生效，需要调用 ADD_LOGFILE()重新指定。

2. COLUMN_PRESENT

判断某列是否被包含在指定的一行逻辑记录中。

该方法仅在参数 LOGMNR_DELAY_ANALYZE 取值为 0 时才支持，取值为 1 时使用该方法将报错。对于虚拟列 COLUMN_PRESENT 时，INSERT 语句的 REDO_VALUE 值可挖掘；UNDO_VALUE 值可挖掘；DELETE 语句的 UNDO_VALUE 值可挖掘，REDO_VALUE 值不可挖掘；UPDATE 语句的 REDO_VALUE 值和 UNDO_VALUE 值均不可挖掘。

语法如下:

```
FUNCTION COLUMN_PRESENT (
    SQL_REDO_UNDO   IN          BIGINT,
    COLUMN_NAME         IN          VARCHAR     DEFAULT  ''
) RETURN INT;
```

参数详解

- sql_redo_undo

REDO 或 UNDO 记录的唯一标识，对应 V$LOGMNR_CONTENTS 中的 REDO_VALUE 或 UNDO_VALUE 字段。

- column_name

由模式名.表名.列名组成的字符串。

返回值

0：指定列不存在于该行的逻辑记录中。

1：指定列存在于该行的逻辑记录中。

3. END_LOGMNR

语法如下：

```
PROCEDURE    END_LOGMNR();
```

功能说明：

结束 LOGMNR。

4. MINE_VALUE

以字符串的格式来获取某一条日志中包含的指定列的值。

该方法仅在参数 LOGMNR_DELAY_ANALYZE 取值为 0 时才支持，取值为 1 时使用该方法将报错。在获取某个指定值时，需要确认其可以被挖掘，否则会返回 null。

MINE_VALUE 的挖掘结果与记录日志的内容有关，影响参数包括：RLOG_APPEND_LOGIC、RLOG_LLOG_COMPRESS。

RLOG_APPEND_LOGIC 取值影响如下：

取值 / 操作	0	1	2	3	4
INSERT	不可挖掘	可挖掘	可挖掘	可挖掘	不可挖掘

		REDO_VALUE 值	REDO_VALUE 值	REDO_VALUE 值	
UPDATE	不可挖掘	含有主键列：可挖掘主键列的 REDO_VALUE 值和 UNDO_VALUE 值 不含有主键列：可挖掘 REDO_VALUE 值和 UNDO_VALUE 值	可挖掘 REDO_VALUE 值和 UNDO_VALUE 值	不可挖掘	不可挖掘
DELETE	不可挖掘	含有主键列：可挖掘主键列的 UNDO_VALUE 值 不含有主键列：可挖掘 UNDO_VALUE 值	可挖掘 UNDO_VALUE 值	不可挖掘	不可挖掘

1） 取值为 0 时：不在日志中记录逻辑操作，不可挖掘。

2） 取值为 1 时：如果有主键列，则 DELETE 操作只记录主键列的信息，UPDATE 操作只记录更新列和主键列的信息，此时可挖掘主键列的 UNDO_VALUE 值。如果不含有主键列，则与 RLOG_APPEND_LOGIC 取值为 2 相同。

3） 取值为 2 时:记录 UPDATE 和 DELETE 操作时包含所有非 CLOB 和 BLOB 列的信息。

4） 取值为 3 时：记录 UPDATE 时包含更新列的信息以及 ROWID，记录 DELETE 时只有 ROWID；不可挖掘 UPDATE 和 DELETE 的 UNDO_VALUE 值。

5） 取值为 4 时：只生成事务以及 DDL 相关的逻辑日志，不可挖掘 INSERT\DELETE\UPDATE 的 REDO_VALUE 值和 UNDO_VALUE 值。

RLOG_LLOG_COMPRESS 取值影响:

取值为 1 时开启对 INSERT 逻辑日志的压缩功能，此时不可挖掘 INSERT 操作的 REDO_VALUE 和 UNDO_VALUE 值。

对 UPDATE 逻辑日志挖掘某列的 REDO_VALUE 值时，若该列不是更新列，则返回该列的旧值信息，等效于挖掘 UNDO_VALUE 值。

语法如下:

```
FUNCTION MINE_VALUE(
    SQL_REDO_UNDO IN BIGINT,
    COLUMN_NAME IN VARCHAR DEFAULT ''
) RETURN VARCHAR;
```

参数详解

- sql_redo_undo

REDO 或 UNDO 记录的唯一标识，对应 V$LOGMNR_CONTENTS 中的 REDO_VALUE 或 UNDO_VALUE 字段。

- column_name

由模式名.表名.列名组成的字符串。

返回值

以字符串的格式返回某一条日志中包含的指定列的值。

5. REMOVE_LOGFILE

从日志列表中移除某个日志文件。

语法如下:

```
PROCEDURE REMOVE_LOGFILE (
    LOGFILENAME IN VARCHAR
);
```

参数详解

- LogFileName

待移除的日志的文件全路径。

6. START_LOGMNR

根据指定的模式和条件来开始某个会话上的 LOGMNR，一个会话上仅能 START 一个 LOGMNR。

语法如下:

```
PROCEDURE    START_LOGMNR (
    STARTSCN          IN      BIGINT          DEFAULT 0,
    ENDSCN            IN      BIGINT          DEFAULT 0,
    STARTTIME         IN      DATETIME DEFAULT '1988/1/1',
    ENDTIME           IN      DATETIME DEFAULT '2110/12/31',
    DICTFILENAME      IN      VARCHAR         DEFAULT '',
    OPTIONS           IN      INT             DEFAULT  0
);
```

参数详解

- startScn

  分析时或者加载时的过滤条件，日志起始序列号，缺省为 0，表示无限制。

- endScn

  分析时或者加载时的过滤条件，日志结束序列号，缺省为 0，表示无限制。

- startTime

  分析时或者加载时的过滤条件，日志起始时间，缺省为 1988/1/1。

- endTime

  分析时或者加载时的过滤条件，日志结束时间，缺省为 2110/12/31。

- DictFileName

离线字典的全路径名，缺省为空。目前 DM 暂不支持离线字典功能。

- Options

提供如下表所列的可选模式，各模式可以通过 + 或者按位或来进行组合。其它位的值如 1、4、8 等目前不支持，配置后不会报错，但是没有效果。例如，组合全部模式，则取值计算方法为 2+16+64+2048=2130，那么 OPTIONS 值取就是 2130。

Options	对应值	说明
COMMITTED_DATA_ONLY	2	仅从已提交的事务的日志中挖掘信息
DICT_FROM_ONLINE_CATALOG	16	使用数据字典拼写 SQL 语句。具体如下: 一 表定义不同的两个同名表（例如同名不同列）被认定为是两个不同的表。如果 SQL 中包含的表不存在则该 SQL 语句不能被挖掘。 二 使用数据字典拼写 SQL 语句前，会先判断日志数据的虚拟列信息与表的虚拟列信息是否匹配，若不匹配，则不使用表信息。 三 拼写 SQL 语句时部分数据会受 INI 参数限制而使用特殊值进行替换。具体如下：一对于大字段类型的数据，若参数 LOGMNR_PARSE_LOB=0 则会使用"OUT_BLOB"和"OUT_CLOB"替换 BLOB 和 CLOB 类型的具体数据；二对于 DOUBLE 和 FLOAT 类型的数据，若参数 FLOAT_MODE=0 则会使用 "SPECIAL_DOUBLE" 和 "SPECIAL_FLOAT"替换 DOUBLE 类型和 FLOAT 类型的特殊值：INF/-INF/NAN
NO_SQL_DELIMITER	64	拼写的 SQL 语句最后不添加分隔符

NO_ROWID_IN_STMT	2048	拼写的 SQL 语句中不包含 ROWID

8.2 举例说明

使用包内的过程和函数之前，如果还未创建过系统包，请先调用系统过程创建系统包。

```
SP_CREATE_SYSTEM_PACKAGES (1,'DBMS_LOGMNR');
```

以下示例展示日志分析工具使用流程。

第1步 配置环境

(1)修改 dm.ini 中的参数，如下:

```
ARCH_INI       = 1

RLOG_APPEND_LOGIC = 1
```

(2)dmarch.ini 需要配置本地归档，举例如下:

```
[ARCHIVE_LOCAL1]

      ARCH_TYPE              = LOCAL

      ARCH_DEST              = d:\dmdata\arch

      ARCH_FILE_SIZE         = 128          #单位 Mb

      ARCH_SPACE_LIMIT       = 0        #单位 Mb，0 表示无限制，范围 1024~4294967294M
```

第2步 添加归档日志文件

(1) 查询有哪些归档日志

```
SELECT   NAME , FIRST_TIME , NEXT_TIME , FIRST_CHANGE# , NEXT_CHANGE# FROM

V$ARCHIVED_LOG;
```

设查询结果如下:

```
D:\DMDATA\ARCH\ARCHIVE_LOCAL1_20140505155106577.LOG

 2014-05-05 15:51:06.000000   2014-05-05 15:51:07.000000

 1                          23303
```

（2）添加一个或多个需要分析的归档日志文件

```
DBMS_LOGMNR.ADD_LOGFILE('D:\DMDATA\ARCH\ARCHIVE_LOCAL1_20140505155106577.LOG');
```

如要查看通过 ADD_LOGFILE 添加的归档日志文件，可以通过动态视图 V$LOGMNR_LOGS 进行查询，如下：

```
SELECT LOW_SCN, NEXT_SCN, LOW_TIME, HIGH_TIME, LOG_ID, FILENAME FROM V$LOGMNR_LOGS;
```

查询结果如下：

```
          1                         23303

2014-05-05 15:51:06.000000   2014-05-05 15:51:07.000000   0

D:\DMDATA\ARCH\ARCHIVE_LOCAL1_20140505155106577.LOG
```

第3步 启动归档日志文件分析

```
DBMS_LOGMNR.START_LOGMNR(OPTIONS=>2128 , STARTTIME=>TO_DATE('2014-03-04

17:36:00','YYYY-MM-DD HH24:MI:SS') , ENDTIME=>TO_DATE('2014-06-04 17:36:02','YYYY-MM-DD

HH24:MI:SS'));
```

如要查看归档日志文件的分析结果，可以通过动态视图 V$LOGMNR_CONTENTS 进行查询，如下：

```
SELECT   OPERATION_CODE , SCN, SQL_REDO , TIMESTAMP ,SEG_OWNER, TABLE_NAME   FROM

V$LOGMNR_CONTENTS WHERE SEG_OWNER = 'SYSDBA' AND OPERATION_CODE IN (3,1,2);
```

第4步 终止归档日志文件分析

```
DBMS_LOGMNR.END_LOGMNR();
```

9 DBMS_METADATA 包

DBMS_METADATA 包提供一种方法，可以将元数据以 XML 格式或 CREATION DDL 语句格式进行检索，也可以提交 XML 以重新创建对象。通过此方法，用户可以更方便地管理和查询数据库中的元数据。

需要注意的是，DBMS_METADATA 包中的方法获取系统内建聚集索引时，当 INI 参数 INNER_INDEX_DDL_SHOW=1，结果显示为“INNER CLUSTER INDEX”，当 INI 参数 INNER_INDEX_DDL_SHOW=0，则报错。

9.1 数据类型

DBMS_METADATA包中涉及到类型。如下统一说明。

1. KU$_PARSED_ITEM 记录类型

```
TYPE KU$_PARSED_ITEM IS RECORD (
    ITEM            VARCHAR(30),
    VALUE           VARCHAR(4000),
    OBJECT_ROW      SMALLINT
);
```

参数详解

- ITEM

  被解析对象属性的名称。

- VALUE

  对象属性值，如果没有就是 NULL。

- OBJECT_ROW

对象所在的行数。

KU$_PARSED_ITEMS 为 KU$_PARSED_ITEM 类型的索引表:

```
TYPE KU$_PARSED_ITEMS IS TABLE OF KU$_PARSED_ITEM INDEX BY INT;
```

2. KU$_DDL 记录类型

```
TYPE KU$_DDL IS RECORD(
    DDLTEXT             CLOB,
    PARSEDITEM          KU$_PARSED_ITEMS
);
```

参数详解

- DDLTEXT

  DDL 语句。

- PARSEDITEM

  对象属性。

  KU$_DDLS 为 KU$_DDL 类型的索引表:

```
TYPE KU$_DDLS IS TABLE OF KU$_DDL INDEX BY INT;
```

3. KU$_MULTI_DDL 记录类型

```
TYPE KU$_MULTI_DDL IS RECORD(
    OBJECT_ROW          NUMBER,
    DDLS                KU$_DDLS
);
```

参数详解

- OBJECT_ROW

  对象所在的行数。

- DDLS

  KU$_DDLS 类型。

  KU$_MULTI_DDLS 为 KU$_MULTI_DDL 类型的索引表:

```
TYPE KU$_MULTI_DDLS IS TABLE OF KU$_MULTI_DDL INDEX BY INT;
```

9.2 相关方法

1. GET_DDL

   获取指定对象元数据中的 DDL 语句。

   权限使用说明:

   - 用户只能看到自己被授权的对象定义。拥有 DBA/DB_OBJECT_ADMIN 权限可看到其他用户的对象定义。
   - 用户可查询自己模式下的对象定义。拥有 DBA/DB_OBJECT_ADMIN 权限才能查看 SYS 模式下的对象定义。
   - 三权分立下，拥有 DBA 权限的用户可以查询其他用户的对象定义，但 SYSDBA 用户和普通用户无权限查询 SYSSSO、SYSAUDITOR 的对象定义。
   - 四权分立下，拥有 DB_OBJECT_ADMIN 权限的用户可以查询其他用户的对象定义。但 SYSDBA 和普通用户无权限查询 SYSSSO、SYSAUDITOR 的对象定义，SYSDBO 可以查询 SYSDBA 的对象定义。
   - 拥有 DBA/DB_OBJECT_ADMIN 权限可查看表空间、USER、ROLE、PROFILE、DIR 定义。

   语法如下:

```
FUNCTION GET_DDL(
    OBJECT_TYPE     IN      VARCHAR(30),
    NAME            IN      VARCHAR(128),
    SCHNAME         IN      VARCHAR(128)    DEFAULT NULL
) RETURN CLOB
```

参数详解

- OBJECT_TYPE

对象类型。包括表、视图、物化视图、索引、全文索引、存储过程、函数、包、目录等，详情请见 OPEN 参数详解。其中，OBJECT_TYPE 只能为大写。

- NAME

对象名称，区分大小写。

- SCHNAME

模式，默认是当前用户模式。

返回值

以 DDL 返回对象元数据中的 DDL 语句。

错误处理

INVALID_ARGVAL：如果输入参数中存在空值或非法值。

OBJ_NOT_FOUND：如果指定的对象在数据库中不存在。

2. OPEN

打开对象类型的句柄

语法如下:

```
FUNCTION OPEN (
```

```
    OBJECT_TYPE        IN          VARCHAR
) RETURN INT;
```

参数详解

- OBJECT_TYPE

  可返回的对象类型名称，合法的类型名称见下表 10.1。

表 10.1 OPEN 函数可返回的对象类型名称

类型名称	含义	属性	说明
CLASS	类类型	SN	默认返回类头类体
JCLASS	java class 类型	SN	无
CLASS_HEAD	类型名	SN	无
CLASS_BODY	类型体	SN	无
COL_STATISTICS	列统计	D	无
COMMENT	注释	D	无
CONSTRAINT	约束	SND	不包括聚集主键和非空约束
CONTEXT	上下文	N	无
CONTEXT_INDEX	全文索引	N	无
DATABASE_EXPORT	数据库下的所有对象	H	库级导出
DB_LINK	数据库链接	SN	因此类对象具有所有者,因此将其视为模式级对象。 对于公有连接，它们的所有者是 PUBLIC；对于私有链接，它们的创建者就是它们的所有者

DIRECTORY	目录	N	无
DOMAIN	域	N	无
FUNCTION	存储函数	SN	无
INDEX	索引	SND	不包括系统内部定义的索引
INDEX _STATISTICS	索引统计	D	无
JOB	任务	N	无
OBJECT_GRANT	对象权限	DG	无
PACKAGE	包	SN	默认返回包头包体
PKG_SPEC	包头	SN	无
PKG_BODY	包体	SN	无
POLICY	策略	D	无
PROCEDURE	存储过程	SN	无
ROLE	角色	N	无
ROLE_GRANT	角色权限	G	无
SCHEMA_EXPORT	模式下的所有对象	H	模式级导出
SEQUENCE	序列	SN	无
SYNONYM	同义词	见说明	私有同义词为模式对象，公有同义词为命名对象
SYSTEM_GRANT	系统权限	G	无
TABLE	表	SN	无
TABLE_STATISTICS	表统计信息	D	无

TABLE_EXPORT	表及与其相关的元数据	H	表级导出
TABLESPACE	表空间	N	无
TRIGGER	触发器	SND	无
USER	用户	N	无
VIEW	视图	SN	无
TYPE	用户自定义类型	SN	无
MATERIALIZED_VIEW	物化视图	SN	无
MATERIALIZED_VIEW_LOG	物化视图日志	SN	无
PROFILE	PROFILE 对象	N	无


注意：**属性列中，S 表示模式对象，N 表示命名的对象，D 表示依赖对象，G 表示被授权的对象，H 表示包含不同类型的对象。**

返回值

不透明的句柄。该句柄将用于 SET_FILTER, SET_PARSE_ITEM, FETCH_XXX, CLOSE。

错误处理

INVALID_ARGVAL：参数为 NULL，或者参数值非法。

3. FETCH_DDL

该函数返回符合 OPEN，SET_FILTER，SET_PARSE_ITEM 设定条件的对象元数据。

语法如下:

```
FUNCTION FETCH_DDL (
```

```
    HANDLE          IN          INT

)RETURN KU$ _DDLS;
```

参数详解

- HANDLE

  OPEN 函数返回的句柄。

返回值

对象的元数据或者当所有对象都已返回后返回 NULL。

使用说明:

从表 KU$ _DDLS 中返回 DDL 语句,表 KU$ _DDLS 的每一行在 DDLTEXT 列包含一条 DDL 语句；如果使用解析的话，解析出的对象属性将从 PARSEDITEM 列返回。

错误处理

INVALID_ARGVAL：参数为 NULL，或者参数值非法。

4. FETCH_CLOB

该函数返回符合 OPEN，SET_FILTER 设定条件的对象元数据。

语法如下:

```
FUNCTION  FETCH_CLOB(

    HANDLE          IN          INT

)RETURN CLOB;
```

参数详解

- HANDLE

  OPEN 函数返回的句柄。

返回值

对象的元数据或者当所有对象都已返回后返回 NULL。

使用说明:

将对象以 CLOB 返回。

异常:

INVALID_ARGVAL: 参数为 NULL，或者参数值非法。

5. GET_GRANTED_DDL

该函数用于返回对象的授权语句。权限使用说明和 GET_DDL 一样，请参考 GET_DDL。

语法如下:

```
FUNCTION GET_GRANTED_DDL (
    OBJECT_TYPE     IN      VARCHAR(30),
    GRANTEE         IN      VARCHAR(128) DEFAULT NULL,
    OBJECT_COUNT    IN      INT  DEFAULT  10000
)RETURN CLOB;
```

参数详解

- OBJECT_TYPE

  对象类型。参数要与 OPEN 的对象类型参数相同，不能为 HETEROGENEOUS 对象类型。

- GRANTEE

  被赋予权限的对象。

- OBJECT_COUNT

  返回对象个数的最大值。

返回值

以 DDL 返回对象的元数据

异常:

INVALID_ARGVAL：参数值为 NULL，或者参数值非法。

OBJ_NOT_FOUND：指定的对象数据库中不存在。

6. GET_DEPENDENT_DDL

该函数用于返回依赖对象的 DDL 语句。权限使用说明和 GET_DDL 一样，请参考 GET_DDL。

语法如下:

```
FUNCTION GET_DEPENDENT_DDL (
    OBJECT_TYPE             IN      VARCHAR(30),
    BASE_OBJECT_NAME        IN      VARCHAR(128),
    BASE_OBJECT_SCHEMA      IN      VARCHAR(128)    DEFAULT NULL,
    OBJECT_COUNT            IN      INT         DEFAULT 10000
)RETURN CLOB;
```

参数详解

- OBJECT_TYPE

  对象类型。参数要与 OPEN 的对象类型参数相同，不能为 HETEROGENEOUS 对象类型。

- BASE_OBJECT_NAME

  基对象名称。内部使用 BASE_OBJECT_NAME 过滤器过滤。

- BASE_OBJECT_SCHEMA

  基对象模式。内部使用 BASE_OBJECT_SCHEMA 过滤器过滤。

- OBJECT_COUNT

  返回对象个数的最大值。

返回值

以 DDL 返回对象的元数据。

异常:

INVALID_ARGVAL：参数值为 NULL，或者参数值非法。

OBJ_NOT_FOUND：指定的对象数据库中不存在。

7. ADD_TRANSFORM

该过程用在 FETCH_CLOB 前，表示 FETCH_CLOB 的内容以 DDL 形式展示，即返回对象的 DDL 格式的语句。不调用则默认情况下返回 XML 格式化定义（当 XML 脚本元数据中包含非格式化拼入的特殊字符时，会进行相应的实体引用转换，如表“T< >”转义为“T<>”。会转换的特殊字符有五种：大于号“>”转换为“>”；小于号“<”转换为“<”；“&”转换为“&”；双引号转换为“"”；单引号转换为“&APOS;”）。

因为要支持这个函数，METADATA 包中元数据改造成了 XML 格式。KU$_DDL.DDLTEXT 为元数据。

语法如下:

```
FUNCTION ADD_TRANSFORM (
                HANDLE          IN NUMBER,
                NAME            IN VARCHAR2,
                ENCODING        IN VARCHAR2 DEFAULT NULL,
                OBJECT_TYPE     IN VARCHAR2 DEFAULT NULL)
RETURN NUMBER;
```

参数详解

- HANDLE

OPEN 函数返回的句柄。

- NAME

  参数值有 DDL 和 MODIFY。表示转换类型，目前只支持 DDL，其他字符则不起作用。

- ENCODING

  编码。忽略，暂不支持设置。

- OBJECT_TYPE

  应用本格式化定义的对象类型。由于 TRANSFORM_HANDLE 只支持会话级，因此该参数暂无实际作用。

返回值

TRANSFORM_HANDLE 的会话句柄，表示定义的格式化应用于当前会话。目前暂定从 10000 开始计数，无实际作用。因为此函数可直接设置全局的转换类型。但该句柄可传递给 SET_TRANSFORM_PARAM。当 NAME 为 MODIFY 时，句柄返回值可以用于函数 SET_REMAP_PARAM，但暂不支持。

错误处理

无

8. GET_QUERY

该函数用于获取查询文本。

语法如下:

```
FUNCTION GET_QUERY (
    HANDLE          IN          INT
)RETURN VARCHAR;
```

参数详解

- HANDLE

  OPEN 函数返回的句柄。

  返回值

  将被用于 FETCH_XXX 函数的查询文本。

  错误处理

  INVALID_ARGVAL：参数值为 NULL，或非法。

9. CHECK_HANDLE

打印句柄使用情况。打印结果中 USE 为 1 的句柄表示当前该句柄正在使用。

语法如下:

```
PROCEDURE CHECK_HANDLE ();
```

参数详解

无。

10. CLOSE

关闭 OPEN 打开的句柄，并清理相关环境。

语法如下:

```
PROCEDURE CLOSE (
    HANDLE          IN          INT
);
```

参数详解

- HANDLE

  OPEN 函数返回的句柄。

错误处理

INVALID_ARGVAL：参数为 NULL，或者参数值非法。

11. CLOSE_ALL

关闭全部句柄。

语法如下:

```
PROCEDURE CLOSE_ALL ();
```

参数详解

无。

12. SET_FILTER

该过程用于过滤待返回的对象。

语法如下:

```
PROCEDURE SET_FILTER (
    HANDLE          IN      INT,
    NAME            IN      VARCHAR,
    VAL             IN      VARCHAR,
    OBJECT_TYPE_PATH    IN  VARCHAR DEFAULT NULL
);
```

参数详解

- HANDLE

  句柄，由 DBMS_METADATA.OPEN 输出。

- NAME

过滤器的名称，详见下表。

- VALUE

  过滤器的值，与 NAME 相对应，详见下表 10.2。

- OBJECT_TYPE_PATH

  忽略，暂不支持设置。

表 10.2 SET_FILTER 过程中的过滤器名称

对象类型	名称	数据类型	含义
可命名对象	NAME	文本	通过对象名过滤
可命名对象	NAME_EXPR	文本表达式	通过对象名的 where 表达式过滤，满足条件的返回
可命名对象	EXCLUDE_NAME_EXPR	文本表达式	通过对象名的 where 表达式过滤，不满足条件的返回
模式对象	SCHEMA	文本	通过模式名过滤，如果是公有同义词则指定其模式为 PUBLIC
模式对象	SCHEMA_EXPR	文本表达式	通过对象名的 where 表达式过滤，默认为当前模式
表、索引、表级导出	TS	文本	通过表空间名过滤
表、索引、表级导出	TS_EXPR	文本表达式	通过表空间名的 where 表达式过滤
依赖对象	BASE_OBJECT_NAME	文本	通过基对象名过滤。

依赖对象	BASE_OBJECT_SCHEMA	文本	通过基对象模式过滤。如果使用 BASE_OBJECT_NAME 过滤，默认是当前模式。
依赖对象	BASE_OBJECT_NAME_EXPR	文本表达式	通过基对象名的 where 表达式过滤，满足条件的返回。不适用于模式和库级触发器。
依赖对象	EXCLUDE_BASE_OBJECT_NAME_EXPR	文本表达式	通过基对象名的 where 表达式过滤，不满足条件的返回。不适用于模式和库级触发器。
依赖对象	BASE_OBJECT_SCHEMA_EXPR	文本表达式	通过基对象模式的 where 表达式过滤。
依赖对象	BASE_OBJECT_TYPE	文本	通过基对象的对象类型过滤。
依赖对象	BASE_OBJECT_TYPE_EXPR	文本表达式	通过基对象的对象类型表达式过滤。
依赖对象	BASE_OBJECT_TS	文本	通过基对象的表空间过滤。
依赖对象	BASE_OBJECT_TS_EXPR	文本表达式	通过基对象的表空间 where 表达式过滤。
被授权对象	GRANTEE	文本	被授权的用户或角色。

			AUDIT_obj 只能过滤跟 grantee 有关的其他 grant 过滤器不适用。
被授权对象	ROLE_NAME	文本	通过权限或角色的名称过滤
被授权对象	ROLE_NAME_EXPR	文本表达式	通过权限或角色的 where 表达式名称过滤
被授权对象	GRANTEE_EXPR	文本表达式	通过被授权者名称的 where 表达式过滤，符合条件的返回
被授权对象	EXCLUDE_GRANTEE_EXPR	文本表达式	通过被授权者名称的 where 表达式过滤，不符合条件的返回
对象权限（OBJECT_GRANT）	GRANTOR	文本	通过授权者过滤，对于系统用户（sysdba sysauditor syssso）赋予的权限在权限表里面 ID 记录的都是-1，所以设定了一个 grantor 叫"$SUPER",当 filter 中设定 grantor 为$SUPER 时，系统用户赋予的权限将全部返回而不区分具体是哪个

			系统用户
所有对象	CUSTOM_FILTER	文本	用户自定义过滤，要求填写完整的 where 子句，如“NAME = ‘T1’”
模式级导出	SCHEMA	文本	通过模式名过滤
模式级导出	SCHEMA_EXPR	文本表达式	通过模式名 where 表达式过滤，有以下两种情况：1.获取模式对象；2.获取依赖于此模式的对象。默认情况下，将选中当前用户的对象。
表级导出	SCHEMA	文本	通过模式名过滤
表级导出	SCHEMA_EXPR	文本表达式	通过模式名 where 表达式过滤，有以下两种情况：1.获取模式下的表；2.获取依赖于表的对象。默认情况下，将选中当前用户的对象。
表级导出	NAME	文本	通过表名过滤出表及依赖表的对象
表级导出	NAME_EXPR	文本表达式	通过表名的 where 表达式过滤出表及依赖表的对象


注意：

约束的 BASE_OBJECT_TYPE 是 UTAB，注释的 BASE_OBJECT_TYPE 为

TABLE 或者 VIEW。索引、OBJECT_GRANT 的 BASE_OBJECT_TYPE 为 UTAB，TABLE_STATISTICS，INDEX_STATISTICS，COL_STATISTICS 的 BASE_OBJECT_TYPE 分别对应为 T、I、C。

错误处理

INVALID_ARGVAL：参数为 NULL，或者参数值非法。

INVALID_CALL_ORDER：无效的过程/函数调用顺序。在调用 FETCH_XXX 之后，不允许再调用 SET_FILTER 。

OBJ_N_FLT_NOT_MATCH：参数不一致，包括如下状况：

- 过滤器名字与 DBMS_METADATA.OPEN 中指定的类型不匹配。
- 输入的过滤器的 VALUE 与期待的数据类型不匹配。

13. SET_PARSE_ITEM

该过程用于解析对象的属性。

语法如下：

```
PROCEDURE SET_PARSE_ITEM (
    HANDLE          IN      INT,
    NAME            IN      VARCHAR,
    OBJECT_TYPE     IN      VARCHAR     DEFAULT NULL
);
```

参数详解

- HANDLE

  OPEN 函数返回的句柄。

- NAME

  被解析对象属性的名称，具体的可被解析的对象属性名称见下表 11.3。

- OBJECT_TYPE

  应用于不同对象的集合，不设置时解析所有的对象。

表 10.3 SET_PARSE_ITEM 可解析的对象属性

对象类型	属性名称	含义
所有对象	VERB	如果调用 FETCH_DDL，表 ku$_ddls 每一行 ddltext 列中的动词将被返回. 如果 ddlText 是 SQL DDL 语句, SQL 中的动词（如 CREATE, GRANT, AUDIT）被返回。如果 ddlText 是过程(如 DBMS_METADATA.FETCH_DDL()）那么 package.procedure-name 被返回
所有对象	OBJECT_TYPE	如果调用 FETCH_DDL， ddlText 是 SQL DDL 语句且包含动词 CREATE 或者 ALTER，DDL 语句中的对象类型将被返回（如 TABLE，PKG_BODY 等等）。否则"表 OPEN 函数可返回的对象类型名称"中的对象类型被返回
模式对象	SCHEMA	返回对象的模式，如果不是模式对象则没有返回值
命名的对象	NAME	返回对象的名称，如果不是命名的对象则没有返回值
表、表数据、索引	TABLESPACE	返回对象表空间名称，如果是分区表则返回默认的表空间。对于 TABLE_DATA 对象, 总是返回行所在的表空间
触发器	ENABLE	返回触发器是禁用还是启用状态
依赖对象	BASE_OBJECT_NAME	返回基对象名称，如果不是依赖对象则没有返回值
依赖对象	BASE_OBJECT_S	返回基对象所属的模式，如果不是依赖对象则没有返回值

	CHEMA	
依赖对象	BASE_OBJECT_TYPE	返回基对象所属类型，如果不是依赖对象则没有返回值



对象类型是可解析的对象属性所适用的范围；BASE_OBJECT_NAME，BASE_OBJECT_SCHEMA，BASE_OBJECT_TYPE 这三个属性只解析约束、索引、全文索引、表级/视图级触发器。

使用说明：

FETCH_XXX 函数可以返回对象的 DDL 语句，使用 SET_PARSE_ITEM 则可以返回对象的各个属性，可以多次调用 SET_PARSE_ITEM 来解析和返回多个解析项，返回的解析项存于表 KU$_PARSED_ITEMS 中。

错误处理

INVALID_ARGVAL：参数为 NULL，或者参数值非法。

INVALID_CALL_ORDER：无效的过程/函数调用顺序。SET_PARSE_ITEM 只能用于 FETCH_XXX 函数之前。第一次调用 FETCH_XXX 后，不允许再调用 SET_PARSE_ITEM。

14. SET_COUNT

该过程用于指定一次 FETCH_XXX 函数调用所返回对象个数的最大值。

语法如下：

```
PROCEDURE SET_COUNT (
    HANDLE          IN          INT,
    VALUE           IN          INT
);
```

参数详解

- HANDLE

  OPEN 函数返回的句柄。

- VALUE

  设定的最大值。

错误处理

INVALID_ARGVAL：参数值为 NULL 或非法；

INVALID_CALL_ORDER：无效的过程/函数调用顺序。SET_COUNT 只能用于第一次调用 FETCH_XXX 函数之前，第一次调用 FETCH_XXX 后，不允许再调用 SET_COUNT。

15. SET_TRANSFORM_PARAM

该过程用于格式化过滤会话级约束或存储选项的定义。

语法如下:

```
PROCEDURE SET_TRANSFORM_PARAM (
    TRANSFORM_HANDLE    IN      INT,
    NAME                IN      VARCHAR,
    VALUE               IN      BOOLEAN     DEFAULT TRUE,
    OBJECT_TYPE         IN      VARCHAR     DEFAULT NULL
);
```

参数详解

- TRANSFORM_HANDLE

  会话句柄，目前仅支持 DBMS_METADATA.SESSION_TRANSFORM，表示定义的格式化应用于当前会话。

- NAME

表示待过滤的约束或存储选项的名称，可取值 CONSTRAINTS 或 STORAGE：CONSTRAINTS 表示约束；STORAGE 表示存储选项。与参数 VALUE 配合使用。

- VALUE

  表示是否过滤约束或存储选项的定义，可取值 TRUE 或 FALSE：TRUE 表示过滤；FALSE 表示不过滤。缺省为 TRUE。

- OBJECT_TYPE

  应用本格式化定义的对象类型，由于 TRANSFORM_HANDLE 只支持会话级，因此该参数暂无实际作用。

错误处理

INVALID_ARGVAL：参数值为 NULL 或非法。

9.3 错误处理

错误、异常情况释义：

ER_EXCEED_OPEN_NUM：同时 OPEN 失败数目超过限制 10 次。

INVALID_OBJ_TYPE：无效的对象类型。

INVALID_FLT_TYPE：无效的过滤器名称。

OBJ_N_FLT_NOT_MATCH：对象类型与过滤器不匹配。

INVALID_EXPR_VAL：无效的表达式值。

INVALID_ATTRI_NAME：不可解析的对象属性。

INVALID_ARGVAL：非法的参数数据。

INVALID_CALL_ORDER：无效的过程/函数调用顺序。

OBJ_NOT_FOUND：未找到对象或不允许查询系统定义的内部索引。

9.4 举例说明

使用包内的过程和函数之前，如果还未创建过系统包，请先调用系统过程创建系统包。

```
SP_CREATE_SYSTEM_PACKAGES (1,'DBMS_METADATA');
```

例 1 获取表和存储过程的 DDL 语句。

```
//建表
CREATE TABLE "SYSDBA"."T1"
(
"C1" CHAR(10) NOT NULL,
"C2" CHAR(10),
CONSTRAINT "PK" PRIMARY KEY("C1")) STORAGE(ON "MAIN", CLUSTERBTR);
//获取对象的 DDL 语句
SELECT DBMS_METADATA.GET_DDL('TABLE','T1','SYSDBA');
```

执行结果如下:

```
CREATE TABLE "SYSDBA"."T1"
(
"C1" CHAR(10) NOT NULL,
"C2" CHAR(10),
CONSTRAINT "PK" NOT CLUSTER PRIMARY KEY("C1")) STORAGE(ON "MAIN", CLUSTERBTR) ;
```

例 2 使用 DBMS_METADATA 程序化接口获取元数据。

```
//建表
CREATE TABLE "RESOURCES"."TIMECARDS"
 (      "EMPLOYEE_ID" INT,
        "WEEK" NUMBER(2,0),
```

```
        "JOB_ID" VARCHAR2(10),
        "HOURS_WORKED" NUMBER(4,2),
        FOREIGN KEY ("EMPLOYEE_ID") REFERENCES "RESOURCES"."EMPLOYEE" ("EMPLOYEEID")
    ) STORAGE(ON "MAIN", CLUSTERBTR) ;
//建函数
CREATE OR REPLACE FUNCTION GET_TABLE_MD RETURN CLOB IS
//定义局部变量
H NUMBER; //HANDLE RETURNED BY OPEN
TH NUMBER; //HANDLE RETURNED BY ADD_TRANSFORM
DOC CLOB;
BEGIN
//指定对象类型
H := DBMS_METADATA."OPEN" ('TABLE');

//使用过滤器返回特定的对象
DBMS_METADATA.SET_FILTER(H,'SCHEMA','RESOURCES');
DBMS_METADATA.SET_FILTER(H,'NAME','TIMECARDS');

//获取 DDL 格式的语句
DBMS_METADATA.ADD_TRANSFORM(H,'DDL');

//获取对象
DOC := DBMS_METADATA.FETCH_CLOB(H);
```

```
//释放资源
DBMS_METADATA."CLOSE" (H);
RETURN DOC;
END;
/


SELECT GET_TABLE_MD;
```

查询结果如下:

```
CREATE TABLE "RESOURCES"."TIMECARDS"
(
"EMPLOYEE_ID" INT,
"WEEK" NUMBER(2,0),
"JOB_ID" VARCHAR2(10),
"HOURS_WORKED" NUMBER(4,2),
FOREIGN KEY("EMPLOYEE_ID") REFERENCES "RESOURCES"."EMPLOYEE"("EMPLOYEEID")
) STORAGE(ON "MAIN", CLUSTERBTR) ;
```

例 3 使用 DBMS_METADATA 浏览接口获取元数据。

```
SELECT DBMS_METADATA.GET_DDL('TABLE','TIMECARDS','RESOURCES');
```

查询结果同例 2。

例 4 获取多个对象。

```
DROP TABLE MY_METADATA;
CREATE TABLE MY_METADATA (MD CLOB);
CREATE OR REPLACE PROCEDURE GET_TABLES_MD IS
//定义局部变量
```

```
H          NUMBER;              //HANDLE RETURNED BY 'OPEN'
TH         NUMBER;              //HANDLE RETURNED BY 'ADD_TRANSFORM'
DOC        CLOB;                //METADATA IS RETURNED IN A CLOB
BEGIN
//指定对象类型
 H := DBMS_METADATA."OPEN" ('TABLE');

//使用过滤器指定模式
 DBMS_METADATA.SET_FILTER(H,'SCHEMA','SYSDBA') ;

//获取 DDL 格式的语句
DBMS_METADATA.ADD_TRANSFORM(H,'DDL');

//获取对象
 LOOP
    DOC := DBMS_METADATA.FETCH_CLOB(H);
 //当没有对象可获取时, FETCH_CLOB 返回 NULL
    EXIT WHEN DOC IS NULL;
    //将元数据存入表中
    INSERT INTO MY_METADATA(MD) VALUES (DOC);
    COMMIT;
 END LOOP;
 //释放资源
 DBMS_METADATA."CLOSE" (H);
```

```
END;
/


CALL GET_TABLES_MD;
SELECT * FROM MY_METADATA;
```

例 5 使用 PARSE ITEMS 解析特定的元数据属性。

```
DROP TABLE MY_METADATA;
CREATE TABLE MY_METADATA (
      OBJECT_TYPE     VARCHAR2(30),
      NAME            VARCHAR2(30),
      MD              CLOB);
CREATE OR REPLACE PROCEDURE GET_TABLES_AND_INDEXES IS
//定义局部变量
H1        NUMBER;           //打开表返回的句柄
DOC       DBMS_METADATA.KU$_DDLS;        //KU$_DDLS，返回的元数据
DDL       CLOB;             //对象的 DDL 语句
PI        DBMS_METADATA.KU$_PARSED_ITEMS;    //包含在 KU$_DDL 中的对象返回的解析项
OBJNAME VARCHAR2(30);    //解析对象的名称
BEGIN
//指定对象类型: TABLE.
 H1 := DBMS_METADATA."OPEN" ('TABLE');


//获取 DDL 格式语句
DBMS_METADATA.ADD_TRANSFORM(H1,'DDL');
```

```
//将表名作为解析项返回
 DBMS_METADATA.SET_PARSE_ITEM(H1,'NAME');

//设置循环: 获取 TABLE 对象
 LOOP
    DOC := DBMS_METADATA.FETCH_DDL(H1);

//当没有对象可获取时，FETCH_CLOB 返回 NULL
    EXIT WHEN DOC IS NULL;

//循环 KU$_DDLS 表中的行
    FOR I IN DOC.FIRST..DOC.LAST LOOP
       DDL := DOC(I).DDLTEXT;
       PI := DOC(I).PARSEDITEM;
       //循环返回的解析项
       IF PI IS NOT NULL AND PI.COUNT > 0 THEN
          FOR J IN PI.FIRST..PI.LAST LOOP
             IF PI(J).ITEM='NAME' THEN
                OBJNAME := PI(J).VALUE;
             END IF;
          END LOOP;
       END IF;
       //将该对象的信息插入表 MY_METADATA 中
```

```
        INSERT INTO MY_METADATA(OBJECT_TYPE, NAME, MD)
          VALUES ('TABLE',OBJNAME,DDL);
        COMMIT;
    END LOOP;
 END LOOP;
 DBMS_METADATA."CLOSE" (H1);
END;
/


CALL GET_TABLES_AND_INDEXES;
SELECT * FROM MY_METADATA;
```

例 6 使用 SET_TRANSFORM_PARAM 格式化过滤约束。

```
//建表
DROP TABLE "SYSDBA"."T1";
CREATE TABLE "SYSDBA"."T1"
(
"C1" CHAR(10) NOT NULL,
"C2" CHAR(10),
CONSTRAINT "PK" PRIMARY KEY("C1")) STORAGE(ON "MAIN", CLUSTERBTR);
//格式化过滤约束
DBMS_METADATA.SET_TRANSFORM_PARAM(DBMS_METADATA.SESSION_TRANSFORM,
'CONSTRAINTS', FALSE, '');
//查看表定义
SELECT DBMS_METADATA.GET_DDL('TABLE','T1','SYSDBA');
```

执行结果如下:

```
CREATE TABLE "SYSDBA"."T1"

(

"C1" CHAR(10),

"C2" CHAR(10)) STORAGE(ON "MAIN", CLUSTERBTR);
```

例 7 使用 ADD_TRANSFORM 返回对象的 DDL 格式的语句。

```
CREATE TABLE TRANS1(C1 INT);

CREATE OR REPLACE FUNCTION GET_TABLE_MD RETURN CLOB IS

H NUMBER; //HANDLE RETURNED BY OPEN

TH NUMBER; //HANDLE RETURNED BY ADD_TRANSFORM

DOC CLOB;

BEGIN

H := DBMS_METADATA."OPEN" ('TABLE');

PRINT 'SET_FILTER|'||H;

//使用过滤器返回特定的对象

DBMS_METADATA.SET_FILTER(H,'SCHEMA','SYSDBA');

DBMS_METADATA.SET_FILTER(H,'NAME','TRANS1');

//获取对象

TH:=DBMS_METADATA.ADD_TRANSFORM(H,'DDL1');

DBMS_METADATA.SET_TRANSFORM_PARAM(TH,'STORAGE',FALSE);

DOC := DBMS_METADATA.FETCH_CLOB(H);

DBMS_METADATA."CLOSE" (H);

RETURN DOC;

END;
```

```
/

SELECT GET_TABLE_MD;
```

执行结果:

```
//无 STORAGE 标签

<?XML

VERSION="1.0"?><ROWSET><ROW><TABLE_T><SCHEMANAME>SYSDBA</SCHEMANAME><NAME>T

RANS1</NAME><COL_LIST><COL_LIST_ITEM><COL_NAME>C1</COL_NAME><COLTYPE>INT</CO

LTYPE></COL_LIST_ITEM></COL_LIST></TABLE_T></ROW></ROWSET>
```

例 8 使用 CHECK_HANDLE 打印句柄使用情况。

```
DECLARE

    H NUMBER;

    H1 NUMBER;

BEGIN

  H := DBMS_METADATA."OPEN" ('CONSTRAINT');

  H1 := DBMS_METADATA."OPEN" ('CONSTRAINT');

  DBMS_METADATA.CHECK_HANDLE();

  DBMS_METADATA."CLOSE_ALL";

  DBMS_METADATA.CHECK_HANDLE();

END;

/
```

打印结果如下:

```
HANDLE    USE

1

2
```

```
3

4

5

6

7

8

9   1

10   1

HANDLE     USE

1

2

3

4

5

6

7

8

9   0

10   0
```

10 DBMS_OBFUSCATION_TOOLKIT 包

为了保护敏感数据，DM 提供一个数据加密包 DBMS_OBFUSCATION_TOOLKIT。利用这个包，用户可以对数据进行 DES、DES3 加密，或者对数据进行 MD5 散列。

10.1 相关方法

DBMS_OBFUSCATION_TOOLKIT 包中包含的过程和函数如下详细介绍:

1. DES3DECRYPT

对 VARBINARY 数据进行 DES3 解密。

语法如下:

```
PROCEDURE DES3DECRYPT(
    INPUT              IN      VARBINARY,
    KEY                IN      VARBINARY,
    DECRYPTED_DATA     OUT     VARBINARY,
    WHICH              IN      INT          DEFAULT  0,
    IV                 IN      VARBINARY    DEFAULT  NULL
);
```

返回值

解密后的 VARBINARY 数据。

对 VARCHAR2 数据进行 DES3 解密。

语法如下:

```
PROCEDURE DES3DECRYPT(
```

```
    INPUT_STRING        IN          VARCHAR2,
    KEY_STRING          IN          VARCHAR2,
    DECRYPTED_STRING    OUT         VARCHAR2,
    WHICH               IN          INT             DEFAULT  0,
    IV_STRING           IN          VARCHAR2        DEFAULT  NULL
);
```

返回值

解密后的 VARCHAR2 类型数据。

对 VARBINARY 数据进行 DES3 解密。

语法如下:

```
FUNCTION DES3DECRYPT(
    INPUT           IN          VARBINARY,
    KEY             IN          VARBINARY,
    WHICH           IN          INT             DEFAULT 0,
    IV              IN          VARBINARY       DEFAULT NULL
)RETURN VARBINARY;
```

返回值

解密后的 VARBINARY 数据。

对 VARCHAR2 数据进行 DES3 解密。

语法如下:

```
FUNCTION DES3DECRYPT(
    INPUT_STRING IN         VARCHAR2,
```

```
    KEY_STRING          IN      VARCHAR2,
    WHICH               IN      INT             DEFAULT 0,
    IV_STRING           IN      VARCHAR2        DEFAULT NULL
)RETURN VARCHAR2;
```

返回值

解密后的 VARCHAR2 数据。

参数详解

- INPUT_STRING

  输入参数，需要解密的数据。

- KEY_STRING

  输入参数，解密数据使用的密钥。

- WHICH

  输入参数，解密模式，可选值 0、1。0 表示双密钥 DES3 解密，1 表示三密钥 DES3 解密。

- IV_STRING

  输入参数，解密数据使用的初始化向量。

2. DES3ENCRYPT

对 VARBINARY 数据进行 DES3 加密。

语法如下:

```
PROCEDURE DES3ENCRYPT(
    INPUT               IN      VARBINARY,
    KEY                 IN      VARBINARY,
    ENCRYPTED_DATA      OUT     VARBINARY,
```

```
    WHICH           IN      INT         DEFAULT 0,
    IV              IN      VARBINARY   DEFAULT NULL
);
```

参数详解

- INPUT

  输入参数，需要加密的数据。

- KEY

  输入参数，加密数据使用的密钥。

- ENCRYPTED_DATA

  输出参数，被加密后的数据。

- WHICH

  输入参数，加密模式，可选值 0、1。0 表示双密钥 DES3 加密，1 表示三密钥 DES3 加密。

- IV

  输入参数，加密数据使用的初始化向量。

对 VARCHAR2 数据进行 DES3 加密。

语法如下:

```
PROCEDURE DES3ENCRYPT(
    INPUT_STRING        IN      VARCHAR2,
    KEY_STRING          IN      VARCHAR2,
    ENCRYPTED_STRING    OUT     VARCHAR2,
    WHICH               IN      INT         DEFAULT 0,
    IV_STRING           IN      VARCHAR2    DEFAULT NULL
);
```

参数详解

- INPUT_STRING

  输入参数，需要加密的数据。

- KEY_STRING

  输入参数，加密数据使用的密钥。

- ENCRYPTED_STRING

  输出参数，被加密后的数据。

- WHICH

  输入参数，加密模式，可选值 0、1。0 表示双密钥 DES3 加密，1 表示三密钥 DES3 加密。

- IV_STRING

  输入参数，加密数据使用的初始化向量。

对 VARBINARY 数据进行 DES3 加密。

语法如下:

```
FUNCTION DES3ENCRYPT(
    INPUT       IN      VARBINARY,
    KEY         IN      VARBINARY,
    WHICH       IN      INT             DEFAULT  0,
    IV          IN      VARBINARY       DEFAULT  NULL
)RETURN VARBINARY;
```

参数详解

- INPUT

  输入参数，需要加密的数据。

- KEY

输入参数，加密数据使用的密钥。

- WHICH

  输入参数，加密模式，可选值 0、1。0 表示双密钥 DES3 加密，1 表示三密钥 DES3 加密。

- IV

  输入参数，加密数据使用的初始化向量。

返回值

返回值加密后的 VARBINARY 数据。

对 VARCHAR2 数据进行 DES3 加密。

语法如下:

```
FUNCTION DES3ENCRYPT(
    INPUT_STRING IN      VARCHAR2,
    KEY_STRING   IN      VARCHAR2,
    WHICH        IN      INT          DEFAULT  0,
    IV_STRING    IN      VARCHAR2     DEFAULT  NULL
)RETURN VARCHAR2;
```

参数详解

- INPUT_STRING

  输入参数，需要加密的数据。

- KEY_STRING

  输入参数，加密数据使用的密钥。

- WHICH

  输入参数，加密模式，可选值 0、1。0 表示双密钥 DES3 加密，1 表示三密钥 DES3 加密。

- IV_STRING

输入参数，加密数据使用的初始化向量。

返回值

返回值加密后的 VARCHAR2 数据。

3. DES3GETKEY

生成 VARBINARY 类型数据的 DES3 加密密钥。

语法如下:

```
PROCEDURE DES3GETKEY(
    WHICH       IN      INT             DEFAULT  0,
    SEED        IN      VARBINARY,
    KEY         OUT     VARBINARY
);
```

参数详解

- WHICH

  输入参数，DES3 工作模式，可选值 0、1。0 表示双密钥 DES3，1 表示三密钥 DES3。

- SEED

  输入参数，获取 DES3 密钥使用的种子。至少 80 字节长。

- KEY

  输出参数，VARBINARY 类型的 DES3 加密密钥。

生成 VARCHAR2 类型的 DES3 加密密钥。

语法如下:

```
PROCEDURE DES3GETKEY(
```

```
    WHICH           IN      INT             DEFAULT  0,
    SEED_STRING     IN      VARCHAR2,
    KEY             OUT     VARCHAR2
);
```

参数详解

- WHICH

  输入参数，DES3 工作模式，可选值 0、1。0 表示双密钥 DES3，1 表示三密钥 DES3。

- SEED_STRING

  输入参数，获取 DES3 密钥使用的种子。至少 80 字节长。

- KEY

  输出参数，VARCHAR2 类型的 DES3 加密密钥。

生成 VARBINARY 类型的 DES3 加密密钥。

语法如下:

```
FUNCTION DES3GETKEY(
    WHICH       IN      INT     DEFAULT  0,
    SEED        IN      VARBINARY
)RETURN VARBINARY;
```

参数详解

- WHICH

  输入参数，DES3 工作模式，可选值 0、1。0 表示双密钥 DES3，1 表示三密钥 DES3。

- SEED

  输入参数，获取 DES3 密钥使用的种子。至少 80 字节长。

返回值

返回值 VARBINARY 类型的 DES3 加密密钥。

生成 VARCHAR2 类型的 DES3 加密密钥。

语法如下:

```
FUNCTION DES3GETKEY(
    WHICH           IN      INT     DEFAULT 0,
    SEED_STRING     IN      VARCHAR2
)RETURN VARCHAR2;
```

参数详解

- WHICH

  输入参数，DES3 工作模式，可选值 0、1。0 表示双密钥 DES3，1 表示三密钥 DES3。

- SEED_STRING

  输入参数，获取 DES3 密钥使用的种子。至少 80 字节长。

返回值

返回值 VARCHAR2 类型的 DES3 加密密钥。

4. DESDECRYPT

对 VARBINARY 数据进行 DES 解密。

语法如下:

```
PROCEDURE DESDECRYPT(
    INPUT           IN      VARBINARY,
    KEY             IN      VARBINARY,
    DECRYPTED_DATA  OUT     VARBINARY
```

```
);
```

参数详解

- INPUT

  输入参数，需要解密的数据。

- KEY

  输入参数，解密数据使用的密钥。

- DECRYPTED_DATA

  输出参数，被解密后的数据。

对 VARCHAR2 数据进行 DES 解密。

语法如下:

```
PROCEDURE DESDECRYPT(
    INPUT_STRING        IN          VARCHAR2,
    KEY_STRING          IN          VARCHAR2,
    DECRYPTED_STRING    OUT         VARCHAR2
);
```

参数详解

- INPUT_STRING

  输入参数，需要解密的数据。

- KEY_STRING

  输入参数，解密数据使用的密钥。

- DECRYPTED_STRING

  输出参数，被解密后的数据。

对 VARBINARY 数据进行 DES 解密。

语法如下:

```
FUNCTION DESDECRYPT(
    INPUT       IN      VARBINARY,
    KEY         IN      VARBINARY
)RETURN VARBINARY;
```

参数详解

- INPUT

  输入参数，需要解密的数据。

- KEY

  输入参数，解密数据使用的密钥。

返回值

返回值解密后的 VARBINARY 数据。

对 VARCHAR2 数据进行 DES 解密。

语法如下:

```
FUNCTION DESDECRYPT(
    INPUT_STRING IN     VARCHAR2,
    KEY_STRING      IN      VARCHAR2
)RETURN VARCHAR2;
```

参数详解

- INPUT_STRING

  输入参数，需要解密的数据。

- KEY_STRING

输入参数，解密数据使用的密钥。

返回值

返回值解密后的 VARCHAR2 数据。

5. DESENCRYPT

对 VARBINARY 数据进行 DES 加密。

语法如下:

```
PROCEDURE DESENCRYPT(
    INPUT              IN      VARBINARY,
    KEY                IN      VARBINARY,
    ENCRYPTED_DATA     OUT     VARBINARY
);
```

参数详解

- INPUT

  输入参数，需要加密的数据。

- KEY

  输入参数，加密数据使用的密钥。

- ENCRYPTED_DATA

  输出参数，被加密后的数据。

对 VARCHAR2 数据进行 DES 加密。

语法如下:

```
PROCEDURE DESENCRYPT(
```

```
    INPUT_STRING        IN          VARCHAR2,
    KEY_STRING          IN          VARCHAR2,
    ENCRYPTED_STRING    OUT         VARCHAR2
);
```

参数详解

- INPUT_STRING

  输入参数，需要加密的数据。

- KEY_STRING

  输入参数，加密数据使用的密钥。

- ENCRYPTED_STRING

  输出参数，被加密后的数据。

对 VARBINARY 数据进行 DES 加密。

语法如下:

```
FUNCTION DESENCRYPT(
    INPUT       IN      VARBINARY,
    KEY         IN      VARBINARY
)RETURN VARBINARY;
```

参数详解

- INPUT

  输入参数，需要加密的数据。

- KEY

  输入参数，加密数据使用的密钥。

返回值

加密后的 VARBINARY 数据。

对 VARCHAR2 数据进行 DES 加密。

语法如下:

```
FUNCTION DESENCRYPT(
    INPUT_STRING IN      VARCHAR2,
    KEY_STRING   IN      VARCHAR2
)RETURN VARCHAR2;
```

参数详解

- INPUT_STRING

  输入参数，需要加密的数据。

- KEY_STRING

  输入参数，加密数据使用的密钥。

返回值

返回值加密后的 VARCHAR2 数据。

6. DESGETKEY

生成 VARBINARY 类型的 DES 加密密钥。

语法如下:

```
PROCEDURE DESGETKEY(
    SEED    IN      VARBINARY,
    KEY             OUT     VARBINARY
);
```

参数详解

- SEED

  输入参数，获取 DES 密钥使用的种子。

- KEY

  输出参数，VARBINARY 类型的 DES 加密密钥。

生成 VARCHAR2 类型的 DES 加密密钥。

语法如下:

```
PROCEDURE DESGETKEY(
    SEED_STRING     IN      VARCHAR2,
    KEY             OUT     VARCHAR2
);
```

参数详解

- SEED_STRING

  输入参数，获取 DES 密钥使用的种子。

- KEY

  输出参数，VARCHAR2 类型的 DES 加密密钥。

生成 VARBINARY 类型的 DES 加密密钥。

语法如下:

```
FUNCTION DESGETKEY(
    SEED    IN      VARBINARY
)RETURN VARBINARY;
```

参数详解

- SEED

  输入参数，获取 DES 密钥使用的种子。

  返回值

  返回 VARBINARY 类型的 DES 加密密钥。

  生成 VARCHAR2 类型的 DES 加密密钥。

  语法如下:

```
FUNCTION DESGETKEY(
    SEED_STRING     IN      VARCHAR2
)RETURN VARCHAR2;
```

  参数详解

- SEED_STRING

  输入参数，获取 DES 密钥使用的种子。

  返回值

  VARCHAR2 类型的 DES 加密密钥。

7. MD5

   生成 VARBINARY 类型数据的 MD5 散列值。同一台机器配置，每次运行的结果一致。

   语法如下:

```
PROCEDURE MD5(
    INPUT           IN      VARBINARY,
    CHECKSUM        OUT     VARBINARY
);
```

   参数详解

- INPUT

  输入参数，需要进行散列的数据。

- CHECKSUM

  输出参数，经 MD5 散列后的数据。

生成 VARCHAR2 类型数据的 MD5 散列值。同一台机器配置，每次运行的结果一致。

语法如下:

```
PROCEDURE MD5(
    INPUT_STRING      IN          VARCHAR2,
    CHECKSUM_STRING               OUT         VARCHAR2
);
```

参数详解

- INPUT_STRING

  输入参数，需要进行散列的数据。

- CHECKSUM_STRING

  输出参数，经 MD5 散列后的数据。

生成 VARBINARY 类型数据的 MD5 散列值。同一台机器配置，每次运行的结果一致。

语法如下:

```
FUNCTION MD5(
    INPUT           IN          VARBINARY
)RETURN VARBINARY;
```

参数详解

- INPUT

输入参数，需要进行散列的数据。

返回值

返回 MD5 散列后的 VARBINARY 数据。

生成 VARCHAR2 类型数据的 MD5 散列值。同一台机器配置，每次运行的结果一致。

语法如下:

```
FUNCTION MD5(
    INPUT_STRING IN          VARCHAR2
)RETURN VARCHAR2;
```

参数详解

- INPUT_STRING

输入参数，需要进行散列的数据。

返回值

返回 MD5 散列后的 VARCHAR2 数据。

10.2 使用说明

1. 被加解密的数据不能为空。

2. IV 加解密数据使用的初始化向量目前不使用，默认为 NULL，此时采用系统默认的初始化向量。

3. DES3GETKEY 和 DESGETKEY 生成加密密钥的种子 SEED 目前不使用，因为 DM 用于生成密钥的环境已经配置好，只需要根据密钥的大小生成指定长度的密钥即可。

4. DES 和 DES3 加解密的块大小为 8BYTES，因此用于加解密的数据必须是 8 的整数倍。

5. DES 加解密使用的密钥长度为 8，多于 8 后的字符被忽略。

6. 双密钥 DES3 加解密使用的密钥长度为 16，多于 16 后的字符被忽略。

7. 三密钥 DES3 加解密使用的密钥长度为 24，多于 24 后的字符被忽略。

8. 十六进制在 DM 对应的是 VARBINARY 数据类型，在 ORACLE 中对应的是 RAW 类型。

9. MD5 主要用途是对数据进行散列，用于校验。不同的机器生成的散列值不同。

10.3 举例说明

使用包内的过程和函数之前，如果还未创建过系统包，请先调用系统过程创建系统包。

```
SP_CREATE_SYSTEM_PACKAGES (1,'DBMS_OBFUSCATION_TOOLKIT');

SP_CREATE_SYSTEM_PACKAGES (1,'DBMS_OUTPUT');

SET SERVEROUTPUT ON;  //DBMS_OUTPUT.PUT_LINE 需要设置这条语句，才能打印出消息
```

例 1　分别使用 DES3ENCRYPT 加密算法、DES3DECRYPT 解密算法对 VARBINARY 类型数据进行加密解密。

```
DECLARE

    RAW_INPUT         VARBINARY(128) := '74696765727469676572746967657274';

//'TIGERTIGERTIGERT'

    RAW_KEY           VARBINARY(128) := '73636F747473636F747473636F747473';

//'SCOTTSCOTTSCOTTS'

    ENCRYPTED_RAW                    VARBINARY(2048);

    DECRYPTED_RAW                    VARBINARY(2048);



BEGIN

    DBMS_OUTPUT.PUT_LINE('> ========== BEGIN TEST RAW DATA ==========');

    DBMS_OUTPUT.PUT_LINE('> RAW INPUT                                  : ' ||
```

```
                    RAW_INPUT);


        DBMS_OBFUSCATION_TOOLKIT.DES3ENCRYPT(RAW_INPUT,
                    RAW_KEY, ENCRYPTED_RAW );
        DBMS_OUTPUT.PUT_LINE('> ENCRYPTED HEX VALUE                    : ' ||
                    ENCRYPTED_RAW);
        DBMS_OBFUSCATION_TOOLKIT.DES3DECRYPT(ENCRYPTED_RAW,
                    RAW_KEY, DECRYPTED_RAW);
        DBMS_OUTPUT.PUT_LINE('> DECRYPTED RAW OUTPUT                   : ' ||
                        DECRYPTED_RAW);
        DBMS_OUTPUT.PUT_LINE('>   ');
        IF RAW_INPUT =
                        DECRYPTED_RAW THEN
            DBMS_OUTPUT.PUT_LINE('> RAW DES ENCYPTION AND DECRYPTION SUCCESSFUL');
        END IF;
END;
/
```

结果如下:

```
> ========== BEGIN TEST RAW DATA ==========
> RAW INPUT                          : 74696765727469676572746967657274
> ENCRYPTED HEX VALUE                : F77BB98AF8E7F59E329C31027CAF6C98
> DECRYPTED RAW OUTPUT               : 74696765727469676572746967657274
>
> RAW DES ENCYPTION AND DECRYPTION SUCCESSFUL
```

例 2　分别使用 DESENCRYPT 加密算法、DESDECRYPT 解密算法对 VARCHAR2 类型的数据进行加密、解密。本示例建库时，未指定 unicode 参数（unicode=0）。

```
DECLARE
    INPUT_STRING        VARCHAR2(8)   := 'DM123456';
    KEY_STRING          VARCHAR2(8)   := 'PASSWORD';

    ENCRYPTED_STRING    VARCHAR2(2048);
    DECRYPTED_STRING    VARCHAR2(2048);

BEGIN
    DBMS_OBFUSCATION_TOOLKIT.DESENCRYPT(
                INPUT_STRING, KEY_STRING, ENCRYPTED_STRING );
    DBMS_OUTPUT.PUT_LINE('> ENCRYPTED STRING                : ' ||
                ENCRYPTED_STRING);
    DBMS_OBFUSCATION_TOOLKIT.DESDECRYPT(
                ENCRYPTED_STRING,
                KEY_STRING,
                DECRYPTED_STRING);
    DBMS_OUTPUT.PUT_LINE('> DECRYPTED OUTPUT                : ' ||
                DECRYPTED_STRING);
    DBMS_OUTPUT.PUT_LINE('>  ');
    IF INPUT_STRING =
                DECRYPTED_STRING THEN
    DBMS_OUTPUT.PUT_LINE('> DES ENCRYPTION AND DECRYPTION SUCCESSFUL');
```

```
    END IF;
END;
/
```

结果如下：

```
> ENCRYPTED STRING                : 櫽C\5g
> DECRYPTED OUTPUT                : DM123456
>
> DES ENCRYPTION AND DECRYPTION SUCCESSFUL
```

例 3　使用 MD5 算法对 VARBINARY 类型数据散列，查看其散列值。

```
DECLARE
    RETVAL VARBINARY(100);
    INPUT_STRING VARBINARY(100) := '746967657274696765727469676572746967657274';
BEGIN
    DBMS_OUTPUT.PUT_LINE('> ========== BEGIN TEST VARBINARY DATA ==========');
    DBMS_OUTPUT.PUT_LINE('> INPUT STRING                    : '
                         || INPUT_STRING);
    RETVAL := DBMS_OBFUSCATION_TOOLKIT.MD5(INPUT_STRING);
    DBMS_OUTPUT.PUT_LINE( '> MD5 STRING OUTPUT              : ' ||
                  RETVAL);
END;
/
```

结果如下：

```
> ========== BEGIN TEST VARBINARY DATA ==========
```

```
> INPUT STRING                    : 74696765727469676572746967657274

> MD5 STRING OUTPUT               : 831F2AA79221A0F8F330E43A76B53323
```

例 4　使用 DES3GETKEY，生成 VARCHAR2 类型的 DES3 加密密钥，输出加密密钥。

```
DECLARE
    RETVAL VARCHAR2(100);
    INPUT_STRING VARCHAR2(100) :=
'0123456789012345678901234567890123456789012345678901234567890123456789012345678901234567890123456789
0123456789';
BEGIN
    DBMS_OUTPUT.PUT_LINE('> ========== BEGIN TEST STRING DATA ==========');
    DBMS_OUTPUT.PUT_LINE('> INPUT STRING                    : '
                         || INPUT_STRING);
    DBMS_OBFUSCATION_TOOLKIT.DES3GETKEY(SEED_STRING => INPUT_STRING, KEY =>
RETVAL);
    DBMS_OUTPUT.PUT_LINE('> DECRYPTED STRING OUTPUT         : ' ||
                 RETVAL);
END;
/
```

结果如下：

```
> ========== BEGIN TEST STRING DATA ==========
> INPUT STRING                    : 0123456789012345678901234567890123456789012
3456789012345678901234567890123456789012345678901234567890123456789
> DECRYPTED STRING OUTPUT         : 4OXDW4FKLE8386JQ   //每次生成密码均不一样
```

11 DBMS_OUTPUT 包

DBMS_OUTPUT 系统包提供了将文本行写入内存、供以后提取和显示的功能。

11.1 相关方法

1．DBMS_OUTPUT.DISABLE

禁用 DBMS_OUTPUT 包。

语法如下:

```
DBMS_OUTPUT.DISABLE
```

举例说明

```
DBMS_OUTPUT.DISABLE(); //禁用 DBMS_OUTPUT 包
```

2．DBMS_OUTPUT.ENABLE

启用 DBMS_OUTPUT 包，并设置缓冲区大小。

语法如下:

```
DBMS_OUTPUT.ENABLE (
    BUFFER_SIZE        IN        INT        DEFAULT  20000
)
```

参数详解

- buffer_size

指定缓冲区大小，以字节为单位，默认值为 20000。当指定的 buffer_size 小于 2000 时，实际 buffer_size 为 2000；当指定的 buffer_size 大于 1000000 时，实际 buffer_size 为 1000000。

举例说明

```
DBMS_OUTPUT.ENABLE(10000); //启用 DBMS_OUTPUT 包，并指定缓冲区大小为 10000
```

3．DBMS_OUTPUT.PUT_LINE

输出字符串到缓冲区。DIsql 中需要设置 SET SERVEROUTPUT ON;语句才可以打印消息到客户端。客户端打印字符串单行最大长度 32767 字节，字符串总长度最大 1MB，超长报错。

语法如下:

```
DBMS_OUTPUT.PUT_LINE(
    STR     IN      VARCHAR
)
```

参数详解

- STR

  输出的字符串，VARCHAR 类型。

4．DBMS_OUTPUT.GET_LINE

从缓冲区中读取一行信息。

语法如下:

```
DBMS_OUTPUT.GET_LINE(
    LINE    OUT     VARCHAR,
    STATUS      OUT     INT
)
```

参数详解

- LINE

缓冲区中的内容。

- STATUS

0 或 1。读取成功返回 0，反之为 1。

5．DBMS_OUTPUT.GET_LINES

从打印缓冲区中检索行数组。

语法如下:

```
DBMS_OUTPUT.GET_LINES(
    LINES          OUT      CHARARR,
    NUMLINES       IN OUT       INT
)
```

语法如下:

```
DBMS_OUTPUT.GET_LINES(
    LINES          OUT      DBMSOUTPUT_LINESARRAY,
    NUMLINES       IN OUT       INT
)
```

参数详解

- LINES

返回已经缓存的行数组。每行的最大大小为 32767 字节。

- NUMLINES

输入需要获取的行数，输出实际获取的行数。如果输出的行数小于需要获取的行数则表明缓冲区中已经没有剩余的数据了。

6．DBMS_OUTPUT.PUT

向行缓冲区中写入部分数据。

语法如下:

```
DBMS_OUTPUT.PUT(
    ITEM        IN          VARCHAR2
)
```

参数详解

- ITEM

向行缓冲区中写入的内容。

7．DBMS_OUTPUT.NEW_LINE

向行缓冲区中放置一个行结束标记。

语法如下:

```
DBMS_OUTPUT.NEW_LINE()
```

11.2 举例说明

使用包内的过程和函数之前，如果还未创建过系统包，请先调用系统过程创建系统包。

```
SP_CREATE_SYSTEM_PACKAGES (1,'DBMS_OUTPUT');
SET SERVEROUTPUT ON;  //DBMS_OUTPUT.PUT_LINE 需要设置这条语句，才能打印出消息
```

例 放入内容到缓冲区，并从缓冲区中读出

```
//启用 DBMS_OUTPUT 包
```

```
DBMS_OUTPUT.ENABLE();
//使用 put_line 和 get_line
declare
buffer varchar;
status int;
begin
    dbms_output.put_line('达梦数据库有限公司');
    dbms_output.get_line(buffer,status);
    dbms_output.put_line('BUFFER:'||buffer||' status:'||status);
    commit;
end;
/
//打印结果：BUFFER:达梦数据库有限公司 status:0
```

12 DBMS_PIPE 包

DBMS_PIPE 包用于同一实例的不同会话之间通过管道进行通信。这里的管道(PIPE)类似于 UNIX 系统的管道，但它不是采用操作系统机制实现的。管道信息被存储在本地消息缓冲区中，当关闭实例时会丢失管道信息。

管道的发送端和接收端都有相应的本地消息缓冲区进行接收和发送消息。DBMS_PIPE 包中函数的使用流程大致为：首先，发送端创建（CREATE_PIPE）管道、打包（PACK_MESSAGE）管道消息、发送（SEND_MESSAGE）消息；其次，接收端接收（RECEIVE_MESSAGE）消息、取出（UNPACK_MESSAGE）消息。

12.1 使用前提

DBMS_PIPE 包的使用依赖 DBMS_XMLGEN 包。所以，在创建 DBMS_PIPE 包之前，请先成功创建 DBMS_XMLGEN 包。

12.2 相关方法

1. CREATE_PIPE

显式创建一个公用或者私有管道。

语法如下:

```
DBMS_PIPE.CREATE_PIPE(
    PIPENAME        IN          VARCHAR2,
    MAXPIPESIZE         IN          INTEGER         DEFAULT 8192,
    IS_PRIVATE          IN          BOOLEAN         DEFAULT TRUE
)RETURN INTEGER;
```

参数详解

- PIPENAME

  指定管道的名称。

- MAXPIPESIZE

  指定管道消息的最大尺寸。取值范围：8KB~2GB。

- IS_PRIVATE

  指定管道类型。TRUE 表示建立私有管道，FALSE 建立公有管道。公用管道是指所有数据库用户都可以访问的管道，而私有管道只有建立管道的数据库用户才能访问。

返回值

如果该函数返回 0，则表示建立管道成功，否则表示建立管道失败。

使用说明:

通过CREATE_PIPE创建的管道的方法为显式创建;在向指定的管道发送消息的过程中，如果指定的管道不存在，则系统会隐式创建管道，这种方法称为隐式创建。

2. **NEXT_ITEM_TYPE**

获取本地消息缓冲区中下一条消息的数据类型,在调用了 RECEIVE_MESSAGE 之后调用该函数。

语法如下:

```
DBMS_PIPE. NEXT_ITEM_TYPE    RETURN INTEGER;
```

返回值

返回 0，则表示管道没有任何消息。

返回 6，则表示下一项的数据类型为 NUMBER。

返回 9，则表示下一项的数据类型为 VARCHAR2。

返回 12，则表示下一项的数据类型为 DATE。

返回 23，则表示下一项的数据类型为 BINARY。

3. REMOVE_PIPE

删除指定的管道。

语法如下:

```
DBMS_PIPE.REMOVE_PIPE (
    PIPENAME IN          VARCHAR2
)RETURN INTEGER;
```

参数详解

- PIPENAME

  管道名称。

使用说明:

1) 对于隐式创建的管道来说，管道为空时自动被删除，不需要调用该函数。

2) 对于显式创建的管道来说，只能通过调用 REMOVE_PIPE 或者关闭数据库实例来删除管道。

返回值

0：删除成功

其他错误码：删除失败

4. RESET_BUFFER

清空本地发送消息缓冲区。因为一个会话中所有的管道都共享一个本地管道缓冲区，所以为防止将本地缓存中原有的数据发送到管道中，在发送消息之前应该复位管道缓冲区。

语法如下:

```
DBMS_PIPE.RESET_BUFFER;
```

5. PURGE

清空指定管道中的消息。

语法如下:

```
DBMS_PIPE.PURGE (
    PIPENAME IN          VARCHAR2
);
```

参数详解

- PIPENAME

  管道名称。

6. PACK_MESSAGE

在本地消息缓冲区中指定消息打包。

为了给管道发送信息，首先需要使用过程 PACK_MESSAGE 将本地消息缓冲区中的指定消息打包好，然后使用过程 SEND_MESSAGE 将打包好的消息发送到管道。

语法如下:

```
DBMS_PIPE.PACK_MESSAGE (ITEM  IN  VARCHAR);
DBMS_PIPE.PACK_MESSAGE (ITEM  IN  INT);
DBMS_PIPE.PACK_MESSAGE (ITEM  IN  BIGINT);
DBMS_PIPE.PACK_MESSAGE (ITEM  IN  DOUBLE);
DBMS_PIPE.PACK_MESSAGE (ITEM  IN  NUMBER);
DBMS_PIPE.PACK_MESSAGE (ITEM  IN  DATE);
```

```
DBMS_PIPE.PACK_MESSAGE_ROWID (ITEM  IN  ROWID);
```

参数详解

- ITEM

  指定管道消息。其输入值可以是字符，数字，日期等多种数据类型。

7. RECEIVE_MESSAGE

接收指定管道中的消息到本地消息缓冲区。

语法如下:

```
DBMS_PIPE.RECEIVE_MESSAGE (
    PIPENAME IN         VARCHAR2,
    TIMEOUT      IN         INTEGER        DEFAULT MAXWAIT
)RETURN INTEGER;
```

参数详解

- PIPENAME

  管道名称。

- TIMEOUT

  最大等待时间。取值范围 0~8640000，单位：秒。

返回值

返回 0，表示接收消息成功。

返回 1，则表示出现超时。

返回 2，则表示本地缓冲区不能容纳管道消息。

返回 3，则表示发生中断。

使用说明:

1） 接收之后，消息从管道删除。

2） 一条消息只能被接收一次。

3） 对于隐式创建的管道来说，管道中最后一条消息被删除后，该管道被自动删除。

8. SEND_MESSAGE

向指定管道上发送一条消息。

语法如下:

```
DBMS_PIPE.SEND_MESSAGE (
    PIPENAME        IN          VARCHAR2,
    TIMEOUT         IN          INTEGER         DEFAULT MAXWAIT,
    MAXPIPESIZE     IN          INTEGER         DEFAULT 8192
) RETURN INTEGER;
```

参数详解

- PIPENAME

  管道名称。

- TIMEOUT

  最大等待时间。取值范围 0~8640000，单位：秒。

- MAXPIPESIZE

  指定管道消息的最大尺寸。取值范围：8KB~2GB。

使用说明:

1) 消息是通过调用 PACK_MESSAGE 生成并存储在本地缓冲区内。

2) 如果指定的管道不存在，则自动隐式创建。

返回值

返回 0，表示接收消息成功。

返回 1，表示出现超时。

返回 2，则表示本地缓冲区不能容纳管道消息。

返回 3，则表示发生中断。

9. UNIQUE_SESSION_NAME

返回一个会话的唯一标识名称。对于同一会话来说，其值不会改变。

语法如下:

```
DBMS_PIPE.UNIQUE_SESSION_NAME

RETURN VARCHAR2;
```

10. UNPACK_MESSAGE

用于取出消息缓冲区中的内容。在使用函数 RECEIVE_MESSAGE 将管道消息接收到本地消息缓冲区中之后，应该使用过程 UNPACK_MESSAGE 取得本地消息缓冲区的消息。

当使用过程 UNPACK_MESSAGE 取出消息缓冲区的消息时，每次只能取出一条消息。如果要取出多条消息，则需要多次调用过程 UNPACK_MESSAGE。

语法如下:

```
DBMS_PIPE.UNPACK_MESSAGE (ITEM   OUT VARCHAR2);

DBMS_PIPE.UNPACK_MESSAGE (ITEM   OUT INT);

DBMS_PIPE.UNPACK_MESSAGE (ITEM   OUT BIGINT);

DBMS_PIPE.UNPACK_MESSAGE (ITEM   OUT DOUBLE);

DBMS_PIPE.UNPACK_MESSAGE (ITEM   OUT NUMBER);

DBMS_PIPE.UNPACK_MESSAGE (ITEM   OUT DATE);
```

```
DBMS_PIPE.UNPACK_MESSAGE_ROWID (ITEM    OUT ROWID);
```

参数详解

- ITEM

  指定管道消息，其输入值可以是字符，数字，日期等多种数据类型。

12.3 动态视图

V$DB_PIPES

记录管道的相关信息。

列名	类型	说明
NAME	VARCHAR(128)　NOT NULL	管道的名字
OWNERID	BIGINT　NOT NULL	管道拥有者 ID
TYPE	VARCHAR(7)	管道类型:PUBLIC/PRIVATE
PIPE_SIZE	BIGINT	管道的长度

12.4 举例说明

使用包内的过程和函数之前，如果还未创建过系统包，请先调用系统过程创建系统包。

```
SP_CREATE_SYSTEM_PACKAGES (1,'DBMS_XMLGEN');

SP_CREATE_SYSTEM_PACKAGES (1,'DBMS_PIPE');

SP_CREATE_SYSTEM_PACKAGES (1,'DBMS_OUTPUT');

SET SERVEROUTPUT ON;  //DBMS_OUTPUT.PUT_LINE 和 PRINT 需要设置这条语句，才能打印出消息
```

举一个两个会话之间通信的实例。

第一步，以 SYSDBA 身份连接会话 1，发送消息。

```
declare

v_pipename varchar2(30):='pipe2';
```

```
v_status integer;
begin
DBMS_PIPE.PURGE(v_pipename);
DBMS_PIPE.RESET_BUFFER;
v_status:=DBMS_PIPE.create_pipe(v_pipename,8192,FALSE);
if v_status !=0 then
raise_application_error(-20099, 'create_pipe error status = '|| v_status);
end if;
dbms_pipe.pack_message(' hello,this is sending process!');
v_status:=dbms_pipe.send_message(v_pipename);
if v_status !=0 then
dbms_output.put_line('error!');
end if;
end;
/
```

第二步，以 USER01 身份连接会话 2，接收消息。

```
declare
v_pipename varchar2(30):='pipe2';
v_status integer;
v_msg varchar2(40);
begin
v_status:=dbms_pipe.receive_message(v_pipename);
if v_status !=0 then
dbms_output.put_line('error');
```

```
end if;

dbms_pipe.unpack_message(v_msg);

print v_msg;

end;

/
```

结果打印出：hello,this is sending process!

13 DBMS_RANDOM 包

DBMS_RANDOM 系统包用于随机产生 INT 类型或 NUMBER 类型数、随机字符串以及符合正态分布的随机数。

使用 DBMS_RANDOM 包生成随机数或随机字符串步骤如下:

1. 调用 INITIALIZE 方法初始化随机种子。若不初始化，则系统将自动生成随机种子;
2. 调用相应方法生成随机数或随机字符串。

13.1 相关方法

1. INITIALIZE

初始化随机种子。

语法如下:

```
PROCEDURE INITIALIZE(
    VAL         IN          INT
);
```

参数详解

- VAL

任意整数，取值范围为：-2147483648 ~ +2147483647。该值将作为随机种子。

2. SEED

重置随机种子。

语法如下:

```
PROCEDURE SEED(
    VAL     IN      INT
);
或
PROCEDURE SEED(
    VAL     IN      VARCHAR2
);
```

参数详解

- VAL

任意整数或字符串。整数取值范围为：-2147483648 ~ +2147483647。若输入字符串，则 SEED 方法会将字符串转换为整数。输入的整数或转换后的整数将作为随机种子。

3. TERMINATE

仅语法支持，无实际作用。

语法如下：

```
PROCEDURE TERMINATE;
```

4. RANDOM_NORMAL

生成符合正态分布的随机数。

语法如下：

```
FUNCTION RANDOM_NORMAL RETURN NUMBER;
```

5. RANDOM

生成随机整数。

语法如下:

```
FUNCTION RANDOM RETURN INTEGER;
```

6. RANDOM_STRING/STRING

生成随机字符串。

语法如下:

```
FUNCTION RANDOM_STRING (
    OPT     IN      CHAR,
    LEN     IN      NUMBER
)RETURN VARCHAR2;
或
FUNCTION STRING (
    OPT     IN      CHAR,
    LEN     IN      NUMBER
)RETURN VARCHAR2;
```

参数详解

- OPT

  CHAR 类型，表示字符串模式，模式解释如下:

  'u', 'U': 只产生随机的大写字母字符串;

  'l', 'L': 只产生随机的小写字母字符串;

  'a', 'A': 产生大小写混合的字母字符串;

'x', 'X'：返回大写字母和数字随机字符串；

'p', 'P'：返回随机可打印字符字符串。

当模式字符不是上述给定的字符时，系统会为按'U'的方式处理。

- LEN

表示生成字符串的长度，最大为 32767，单位为字节。若超过 32767，则置为 32767。

7. value

生成一个指定范围内的随机数。

语法如下:

```
FUNCTION VALUE (
    LOW      IN          NUMBER        DEFAULT 0,
    HIGHIN          NUMBER        DEFAULT 1
)RETURN NUMBER;
```

参数详解

- LOW、HIGH

指定随机数的范围区间。当 LOW 小于等于 HIGH 时，生成大于等于 LOW 小于等于 HIGH 的随机数；当 LOW 大于 HIGH 时，生成大于等于 HIGH 小于等于 LOW 的随机数。

8. NORMAL

生成符合正态分布的随机数。

语法如下:

```
FUNCTION NORMAL RETURN NUMBER;
```

13.2 举例说明

使用包内的过程和函数之前，如果还未创建过系统包，请先调用系统过程创建系统包。

```
SP_CREATE_SYSTEM_PACKAGES (1,'DBMS_RANDOM');
```

例 1 初始化随机种子。

```
CALL DBMS_RANDOM.INITIALIZE(15);
```

例 2 返回大于等于 10 小于 100 的 NUMBER 类型随机数。

```
SELECT   DBMS_RANDOM.VALUE(10,100);
```

结果如下：

```
10.112613
```

14 DBMS_RLS 包

在某些系统中，权限控制必须定义到数据行访问权限的控制，此需求一般出现在同一系统中，不同的相对独立机构使用的情况。例如：集团下属多个子公司，所有子公司使用同一套数据表，但不同子公司的数据相对隔离。绝大多数人会选择在表或视图加上 WHERE 子句来进行数据访问控制，此方法编码工作量大、系统适应用户管理体系的弹性空间较小，因为一旦权限逻辑发生变动，就可能需要修改权限体系，导致所有的表或视图都必须修改。

为了解决上述问题，达梦提供了 DBMS_RLS 包，DBMS_RLS 包通过策略 POLICY 管理来实现数据行级安全（ROW LEVEL SECURITY，简称 RLS）访问控制。

在 INI 参数 ENABLE_RLS 为 1 的前提下，用户需要提前调用系统过程 SP_INIT_RLS_SYS(1);创建 DBMS_RLS 包之后，方可使用 DBMS_RLS 包中的函数或过程来实现数据行级安全访问控制。开启了 ENABLE_RLS 参数后，需初始化 DBMS_RLS 系统包，否则即使没有为用户表、视图对象设置策略，也无法对这些对象进行 DML 操作。

为兼容 V8.4.1.113 之前的版本，支持通过 SP_CREATE_SYSTEM_PACKAGES(create_flag, pkgname) 且指定 pkgname 为 'DBMS_RLS'的方式创建或删除 DBMS_RLS 系统包，但建议使用 SP_INIT_RLS_SYS 进行创建或删除 DBMS_RLS 系统包。

需要说明的是：1. 策略对于 SYSDBA 用户不会生效；2. DM MPP 环境下不支持 DBMS_RLS 包。

14.1 策略函数

策略函数的模板如下所示:

```
FUNCTION POLICY_FUNCTION (
    OBJECT_SCHEMA   IN          VARCHAR2,    // 模式名。如果为空则表示当前模式。
```

```
    OBJECT_NAME          IN          VARCHAR2     // 对象名。仅支持表或视图
)RETURN VARCHAR2
```



注意： **如果不是策略函数模板形式的函数，执行时会报错"策略函数 xx 错误"。**

策略函数中两个输入参数，一个是用户输入参数，一个是对象输入参数。两个输入参数不能不写，尽管可以在函数中没有用到，否则报错返回。

策略函数返回一个字符串，作为 WHERE 子句中的条件表达式。DMDPC 环境下仅支持返回简单类型的条件表达式，例如简单的布尔表达式。

14.2 策略组

14.2.1 创建策略组

创建一个新的策略组。

语法如下:

```
PROCEDURE CREATE_POLICY_GROUP(
    OBJECT_SCHEMA   IN          VARCHAR := NULL,
    OBJECT_NAME          IN          VARCHAR,
    POLICY_GROUP     IN          VARCHAR
);
```

参数详解

- OBJECT_SCHEMA

  包含由策略保护的表或视图的模式。

- OBJECT_NAME

  由策略保护的表或视图的名称。

- POLICY_GROUP

添加到该对象的策略组名称。

使用说明:

1．如果 OBJECT_SCHEMA 为 NULL，则使用调用过程的用户的当前模式。

2．对于受保护的每个对象（表或视图），策略组必须唯一。

14.2.2 删除策略组

删除一个策略组。

语法如下:

```
PROCEDURE DELETE_POLICY_GROUP(
    OBJECT_SCHEMA   IN      VARCHAR := NULL,
    OBJECT_NAME     IN      VARCHAR,
    POLICY_GROUP    IN      VARCHAR
);
```

参数详解

- OBJECT_SCHEMA

  包含由策略保护的表或视图的模式。

- OBJECT_NAME

  由策略保护的表或视图的名称。

- POLICY_GROUP

  要删除该对象的策略组名称。

使用说明:

如果 OBJECT_SCHEMA 为 NULL，则使用调用过程的用户的当前模式。

14.3 策略

14.3.1 创建策略

创建一个新的策略。

语法如下:

```
PROCEDURE ADD_POLICY(
    OBJECT_SCHEMA           IN      VARCHAR := NULL,
    OBJECT_NAME             IN      VARCHAR,
    POLICY_NAME             IN      VARCHAR,
    FUNCTION_SCHEMA         IN      VARCHAR := NULL,
    POLICY_FUNCTION         IN      VARCHAR,
    STATEMENT_TYPES         IN      VARCHAR := NULL,
    UPDATE_CHECK            IN      INT   := 0,
    ENABLE                  IN      INT   := 1,
    STATIC_POLICY           IN      INT   := 0,
    POLICY_TYPE             IN      INT := NULL,
    LONG_PREDICATE                  INT   := 0,
    SEC_RELEVANT_COLS       IN      VARCHAR   := NULL,
    SEC_RELEVANT_COLS_OPT   IN      INT := NULL
);
```

参数详解

- OBJECT_SCHEMA

  包含由策略保护的表或视图的模式。

- OBJECT_NAME

由策略保护的表或视图的名称。

- POLICY_NAME

  添加到该对象的策略名称。

- FUNCTION_SCHEMA

  拥有策略函数的模式。

- POLICY_FUNCTION

  策略函数名称。

- STATEMENT_TYPES

  应用策略的 DML 语句类型。

- UPDATE_CHECK

  该参数对于 INSERT 或 UPDATE 类型是可选项，缺省值为 0。

- ENABLE

  表示添加该策略时是否启用它，缺省值为 1。

- STATIC_POLICY

  表策略是静态的，即对任何访问该对象的人产生相同的谓词字符串，该参数默认值为 0。

- POLICY_TYPE

  表示策略静态或动态类型，缺省为 NULL。如果 POLICY_TYPE 不是 NULL，则覆盖 STATIC_POLICY。可允许的值是 STATIC、SHARED_STATIC、CONTEXT_SENSITIVE、SHARED_CONTEXT_SENSITIVE 和 DYNAMIC。

- LONG_PREDICATE

  表示谓词字符串是否是长谓词，该参数缺省为 0。

- SEC_RELEVANT_COLS

  只应用于表和视图，在列表中指定受保护的列，使用逗号或空格作为分隔符。

- SEC_RELEVANT_COLS_OPT

该参数的缺省值为 NULL；如果不是缺省值，则必须指定 DBMS_RLS.ALL_ROWS，用于显示敏感列为 NULL 的所有列。

使用说明:

1．如果 OBJECT_SCHEMA 和 FUNCTION_SCHEMA 为 NULL，则使用调用过程的用户的当前模式。

2．对于受保护的每个对象（表或视图），策略名必须唯一。

3．POLICY_FUNCTION 为针对 OBJECT_NAME 的策略生成谓词。如果函数是程序包的一部分，则 POLICY_FUNCTION 必须指定程序包名，用于限定策略函数名。

4．STATEMENT_TYPES 允许的值以逗号或空格分隔，可以是 SELECT、INSERT、UPDATE、DELETE 和 INDEX 的任意组合。默认情况下，除了 INDEX 之外的所有类型都适用。对于列存储表（即 HUGE 表），仅支持 SELECT 类型语句。

5. 如果 UPDATE_CHECK 参数为 1，则会检查 INSERT 或 UPDATE 对应的数据是否满足策略函数。不满足，则报错“违反策略检查约束”。

6. 不管 LONG_PREDICATE 是 0，还是 1，谓词字符串最多可为 4000 字节。

7. STATIC_POLICY 和 POLICY_TYPE 暂不支持，所有谓词都是动态的。

8. ADD_POLICY 没有指定策略组，则使用系统默认的策略组 SYS_DEFAULT。

9. 不支持对临时表添加策略。

创建一个新的策略。用法与 ADD_POLICY 基本相同，只是指定了策略所属的策略组 POLICY_GROUP， POLICY_GROUP 默认值为系统策略组 SYS_DEFAULT。

语法如下:

```
PROCEDURE ADD_GROUPED_POLICY(
    OBJECT_SCHEMA                IN      VARCHAR := NULL,
    OBJECT_NAME                  IN      VARCHAR,
```

```
    POLICY_GROUP                IN      VARCHAR := 'SYS_DEFAULT',
    POLICY_NAME                 IN      VARCHAR,
    FUNCTION_SCHEMA             IN      VARCHAR := NULL,
    POLICY_FUNCTION     IN      VARCHAR,
    STATEMENT_TYPES     IN      VARCHAR := NULL,
    UPDATE_CHECK                IN      INT   := 0,
    ENABLE                      IN      INT   := 1,
    STATIC_POLICY               IN      INT   := 0,
    POLICY_TYPE                 IN      INT := NULL,
    LONG_PREDICATE                      INT   := 0,
    SEC_RELEVANT_COLS           IN      VARCHAR   := NULL,
    SEC_RELEVANT_COLS_OPT   IN      INT := NULL
);
```

14.3.2 启用策略

启用一个策略。

语法如下:

```
PROCEDURE ENABLE_POLICY(
    OBJECT_SCHEMA   IN      VARCHAR := NULL,
    OBJECT_NAME     IN      VARCHAR,
    POLICY_NAME     IN      VARCHAR,
    ENABLE          IN      INT := 1
);
```

参数详解

- OBJECT_SCHEMA

  包含由策略保护的表或视图的模式。

- OBJECT_NAME

  由策略保护的表或视图的名称。

- POLICY_NAME

  启用该对象的策略名称。

- ENABLE

  表示是否启用该策略，缺省值为 1。0 表示禁用，1 表示启用。

使用说明:

1．如果 OBJECT_SCHEMA 为 NULL，则使用调用过程的用户的当前模式。

2．ENABLE_POLICY 没有指定策略组，则使用系统默认的策略组 SYS_DEFAULT。

启用一个策略。用法与 ENABLE_POLICY 基本相同，只是指定了策略所属的策略组 GROUP_NAME。

语法如下:

```
PROCEDURE ENABLE_GROUPED_POLICY(
    OBJECT_SCHEMA   IN      VARCHAR := NULL,
    OBJECT_NAME     IN      VARCHAR,
    GROUP_NAME      IN      VARCHAR,
    POLICY_NAME     IN      VARCHAR,
    ENABLE          IN      INT := 1
);
```

14.3.3 禁用策略

禁用一个策略。

语法如下:

```
PROCEDURE DISABLE_GROUPED_POLICY(
    OBJECT_SCHEMA   IN      VARCHAR := NULL,
    OBJECT_NAME     IN      VARCHAR,
    GROUP_NAME      IN      VARCHAR,
    POLICY_NAME     IN      VARCHAR
);
```

参数

- OBJECT_SCHEMA

  包含由策略保护的表或视图的模式。

- OBJECT_NAME

  由策略保护的表或视图的名称。

- GROUP_NAME

  指定策略所属的策略组名称。

- POLICY_NAME

  禁用该对象的策略名称。

使用说明:

如果 OBJECT_SCHEMA 为 NULL，则使用调用过程的用户的当前模式。

14.3.4 删除策略

删除一个策略。

语法如下:

```
PROCEDURE DROP_POLICY(
    OBJECT_SCHEMA   IN      VARCHAR := NULL,
    OBJECT_NAME     IN      VARCHAR,
    POLICY_NAME     IN      VARCHAR
);
```

参数详解

- OBJECT_SCHEMA

  包含由策略保护的表或视图的模式。

- OBJECT_NAME

  由策略保护的表或视图的名称。

- POLICY_NAME

  要删除该对象的策略名称。

使用说明:

1．如果 OBJECT_SCHEMA 为 NULL，则使用调用过程的用户的当前模式。

2．DROP_POLICY 没有指定策略组，则使用系统默认的策略组 SYS_DEFAULT。

删除一个策略。用法与 DROP_POLICY 基本相同，只是指定了策略所属的策略组 POLICY_GROUP， POLICY_GROUP 默认值为系统策略组 SYS_DEFAULT。

语法如下:

```
PROCEDURE DROP_GROUPED_POLICY(
    OBJECT_SCHEMA   IN      VARCHAR := NULL,
    OBJECT_NAME     IN      VARCHAR,
    POLICY_GROUP    IN      VARCHAR := 'SYS_DEFAULT',
```

```
    POLICY_NAME        IN          VARCHAR
);
```

14.4 上下文

14.4.1 创建上下文策略

创建一个上下文策略。

语法如下:

```
PROCEDURE ADD_POLICY_CONTEXT(
    OBJECT_SCHEMA        IN          VARCHAR := NULL,
    OBJECT_NAME          IN          VARCHAR,
    NAMESPACE            IN          VARCHAR,
    ATTRIBUTE            IN          VARCHAR
);
```

参数详解

- OBJECT_SCHEMA

  包含由策略保护的表或视图的模式。

- OBJECT_NAME

  由策略保护的表或视图的名称。

- NAMESPACE

  添加到该对象的上下文名称。

- ATTRIBUTE

  策略对应上下文的属性名称。

使用说明:

1．如果 OBJECT_SCHEMA 为 NULL，则使用调用过程的用户的当前模式。

2．对于受保护的每个对象（表或视图），同一上下文的属性名必须唯一。

14.4.2 删除上下文策略

删除一个上下文策略。

语法如下:

```
PROCEDURE DROP_POLICY_CONTEXT(
    OBJECT_SCHEMA       IN      VARCHAR := NULL,
    OBJECT_NAME         IN      VARCHAR,
    NAMESPACE       IN      VARCHAR,
    ATTRIBUTE           IN      VARCHAR
);
```

参数详解

- OBJECT_SCHEMA

  包含由策略保护的表或视图的模式。

- OBJECT_NAME

  由策略保护的表或视图的名称。

- NAMESPACE

  添加到该对象的上下文名称。

- ATTRIBUTE

  策略对应上下文的属性名称。

使用说明:

如果 OBJECT_SCHEMA 为 NULL，则使用调用过程的用户的当前模式。

14.5 启用状态检测语句

系统的 DBMS_RLS 系统包启用状态检测。

语法如下:

```
INTEGER

SF_CHECK_RLS_SYS();
```

返回值

0：未启用；1：已启用

举例说明

获得 DBMS_RLS 系统包的启用状态。

```
select SF_CHECK_RLS_SYS();
```

14.6 举例说明

创建 DBMS_RLS 包之前，需先将 INI 参数 ENABLE_RLS 设置为 1，之后才可以成功使用包中的过程或函数。

```
ALTER SYSTEM SET 'ENABLE_RLS' =1 SPFILE; --因为是静态参数，设置完需重启服务器
```

调用系统过程创建系统包。

```
SP_INIT_RLS_SYS(1);
```

例 1 通过策略过滤用户对表数据的访问控制。

```
数据准备:

    CONN SYSDBA/DMdba_123   --以 SYSDBA 身份连接服务器

    SP_INIT_RLS_SYS(1);

    CREATE USER USER01 IDENTIFIED BY "USER01_psd";

    GRANT DBA TO USER01;
```

```
CONN USER01/USER01_psd  --以 USER01 身份连接服务器
```

第一步：建立测试数据表(T_POLICY)：

```
CREATE TABLE T_POLICY( T1  VARCHAR2(10), T2  NUMBER(10));
INSERT INTO T_POLICY VALUES('A',10);
INSERT INTO T_POLICY VALUES('B',20);
INSERT INTO T_POLICY VALUES('C',30);
COMMIT;
```

第二步：建立测试 POLICY 的函数:

```
CREATE OR REPLACE FUNCTION FN_GETPOLICY(
    P_SCHEMA IN VARCHAR2,
    P_OBJECT IN VARCHAR2
) RETURN VARCHAR2 IS RESULT VARCHAR2(1000);
BEGIN
    RESULT:='T2 NOT IN (10)';
    RETURN(RESULT);
END;
/
```

第三步：加入 POLICY:

```
DBMS_RLS.ADD_POLICY(
OBJECT_SCHEMA =>'USER01',  //数据表(或视图)所在的 SCHEMA 名称
OBJECT_NAME =>'T_POLICY', //数据表(或视图)的名称
POLICY_NAME =>'T_TESTPOLICY', //POLICY 的名称，主要用于将来对 POLICY 的管理
FUNCTION_SCHEMA =>'USER01',  //返回 WHERE 子句的函数所在 SCHEMA 名称
POLICY_FUNCTION =>'FN_GETPOLICY', //返回 WHERE 子句的函数名称
```

```
    STATEMENT_TYPES =>'SELECT,INSERT,UPDATE,DELETE', //要使用该 POLICY 的 DML 类型
    UPDATE_CHECK =>1, //仅适用于 STATEMENT_TYPE 为'INSERT,UPDATE'，值为 1 或 0
    ENABLE =>1    //是否启用，值为 1 或 0
    );
```

第四步：查询数据表(T_POLICY)：

```
    SELECT * FROM T_POLICY;
    //查询结果如下:
    行号          T1 T2
    1             B   20
    2             C   30
    //从查询结果中可以看出少了 T2=10 这行数据。
```

第五步：检查更新数据

```
    UPDATE T_POLICY SET T2 = 10;
    //运行结果如下:
    [-6614]:违反策略检查约束.
    //注: 如果 UPDATE_CHECK 设为 1, 则用户插入的值不符合 POLICY_FUNCTION 返回条件时, 该 DML
```

执行返回错误信息。

第六步：禁用 POLICY

```
    DBMS_RLS.ENABLE_POLICY('USER01','T_POLICY','T_TESTPOLICY',0);
    SELECT * FROM T_POLICY;
    //查询结果如下:
    行号          T1 T2
    1             A   10
    2             B   20
```

```
3              C   30

//从查询结果中可以看到 T2=10 这行数据。

第七步：删除 POLICY

DBMS_RLS.DROP_POLICY('USER01','T_POLICY','T_TESTPOLICY');
```

例 2 上下文在数据访问控制中的使用。

```
数据准备:

CONN SYSDBA/DMdba_123    --以 SYSDBA 身份连接服务器

CREATE USER MYKGIS IDENTIFIED BY "MYKGIS_psd_123";

GRANT DBA TO MYKGIS;

CONN MYKGIS/MYKGIS_psd_123    --以 MYKGIS 身份连接服务器

SP_INIT_RLS_SYS(1);

SP_CREATE_SYSTEM_PACKAGES(1,'DBMS_SESSION');


DROP TABLE EMPLOYEE;

CREATE TABLE EMPLOYEE (

    EMPID INT, FIRSTNAME VARCHAR(20),

    LASTNAME VARCHAR(20),

    LOCATION VARCHAR(20) ,

    DEPARTMENT VARCHAR(20)

);

INSERT INTO EMPLOYEE VALUES(1, 'STEVE', 'MILLER', 'GA', 'IT');

INSERT INTO EMPLOYEE VALUES(2, 'SCOTT', 'TIGER', 'GA', 'HR');

INSERT INTO EMPLOYEE VALUES(3, 'TOM', 'LUTZ', 'FL', 'HR');

INSERT INTO EMPLOYEE VALUES(4, 'HARRY', 'MYKGIS', 'FL', 'IT');
```

```
COMMIT;


DROP TABLE ORDERS;
CREATE TABLE ORDERS(
    ORDER_ID INT, AMOUNT   FLOAT,
    LOCATION VARCHAR(20),
    DEPARTMENT VARCHAR(20)
);
INSERT INTO ORDERS VALUES(1, 3454.45, 'GA', 'IT');
INSERT INTO ORDERS VALUES(2, 324893.34, 'FL', 'IT');
INSERT INTO ORDERS VALUES(3, 34545.11,   'FL', 'HR');
INSERT INTO ORDERS VALUES(4, 234.99, 'GA', 'HR');
COMMIT;
```

第一步：创建上下文

```
CREATE CONTEXT DRIVECTX USING MYKGIS.APPSEC;


//创建上下文所使用的包
CREATE OR REPLACE PACKAGE MYKGIS.APPSEC AS
V_DEPARTMENT VARCHAR2(2);
V_LOCATION VARCHAR2(2);
PROCEDURE SETCONT(POLICY_GROUP VARCHAR2);
END;
/
```

```
//创建上下文所使用的包体
CREATE OR REPLACE PACKAGE BODY MYKGIS.APPSEC AS
PROCEDURE SETCONT(POLICY_GROUP VARCHAR2) AS
V_DEPARTMENT VARCHAR2(2);
V_LOCATION VARCHAR2(2);
BEGIN
DBMS_SESSION.SET_CONTEXT('DRIVECTX','ACTIVE_APP',POLICY_GROUP);
SELECT DEPARTMENT INTO V_DEPARTMENT
FROM MYKGIS.EMPLOYEE WHERE LASTNAME = SYS_CONTEXT('USERENV','SESSION_USER');
DBMS_SESSION.SET_CONTEXT('DRIVECTX','DEPARTMENT',V_DEPARTMENT);
SELECT LOCATION INTO V_LOCATION
FROM MYKGIS.EMPLOYEE WHERE LASTNAME = SYS_CONTEXT('USERENV','SESSION_USER');
DBMS_SESSION.SET_CONTEXT('DRIVECTX','LOCATION',V_LOCATION);
END;
END;
/
```

第二步：为表 ORDERS 定义上下文

```
DBMS_RLS.ADD_POLICY_CONTEXT('MYKGIS','ORDERS','DRIVECTX','ACTIVE_APP');
```

第三步：创建策略函数

```
CREATE OR REPLACE FUNCTION MYKGIS.DEP_POLICY(D1 VARCHAR2, D2 VARCHAR2)
RETURN VARCHAR2 AS
```

```
BEGIN
RETURN 'DEPARTMENT = SYS_CONTEXT("DRIVECTX", "DEPARTMENT")';
END;
/
CREATE OR REPLACE FUNCTION MYKGIS.LOC_POLICY(D1 VARCHAR2, D2 VARCHAR2)
RETURN VARCHAR2 AS
BEGIN
RETURN 'LOCATION = SYS_CONTEXT("DRIVECTX", "LOCATION")';
END;
/
```

第四步：创建策略组

```
DBMS_RLS.CREATE_POLICY_GROUP('MYKGIS','ORDERS','DEPARTMENT');
DBMS_RLS.CREATE_POLICY_GROUP('MYKGIS','ORDERS','LOCATION');
```

第五步：创建策略

```
DBMS_RLS.ADD_GROUPED_POLICY('MYKGIS','ORDERS','DEPARTMENT','DEP_SECURITY','MYKGIS','DEP_POLICY');
DBMS_RLS.ADD_GROUPED_POLICY('MYKGIS','ORDERS','LOCATION','LOC_SECURITY','MYKGIS','LOC_POLICY');
```

第六步：设置上下文中的策略组

```
MYKGIS.APPSEC.SETCONT('DEPARTMENT');
//查询策略函数返回结果：
```

```
SELECT MYKGIS.DEP_POLICY('MYKGIS', 'EMPLOYEE') FROM DUAL;

//查询结果如下:

DEPARTMENT = SYS_CONTEXT('DRIVECTX', 'DEPARTMENT')

//查询上下文的属性值

SELECT * FROM V$CONTEXT WHERE NAMESPACE = 'DRIVECTX';

//查询结果如下:

NAMESPACE ATTRIBUTE    VALUE

--------- ---------- ----------

DRIVECTX    ACTIVE_APP DEPARTMENT

DRIVECTX    DEPARTMENT IT

DRIVECTX    LOCATION     FL
```

第七步：查询表

```
SELECT * FROM ORDERS;

//查询结果如下:

ORDER_ID       AMOUNT                                LOCATION DEPARTMENT

---------- ------------------------ -------- ----------

1                   3.4544500000000000E+003       GA           IT

2                    3.2489334000000000E+005       FL           IT

//从查询结果中可以看出过滤掉了 DEPARTMENT 不是 IT 的数据。
```

第八步：重新设置上下文中的策略组

```
MYKGIS.APPSEC.SETCONT('LOCATION');

//查询策略函数返回结果:
```

```
SELECT MYKGIS.LOC_POLICY('MYKGIS', 'EMPLOYEE') FROM DUAL;

//查询结果如下:

LOCATION = SYS_CONTEXT('DRIVECTX', 'LOCATION')



SELECT * FROM ORDERS;

//查询结果如下:

ORDER_ID        AMOUNT                                LOCATION DEPARTMENT

----------- ------------------------- -------- ----------

2                    3.248933400000000E+005       FL           IT

3                    3.454511000000000E+004       FL           HR

//从查询结果中可以看出过滤掉了 LOCATION 不是 FL 的数据。
```

15 DBMS_SESSION 包

DBMS_SESSION 包用来访问或者设置会话信息。

15.1 数据类型

DBMS_SESSION 包中涉及到类型。如下统一说明。

APPCTXRECTYP 类型是 DBMS_SESSION 专有类型。用户不能引用和改变该记录的内容。

APPCTXRECTYP 记录类型定义如下：

```
TYPE APPCTXRECTYP IS RECORD(
    NAMESPACE       VARCHAR2(30),
    ATTRIBUTE       VARCHAR2(30),
    VALUE           VARCHAR2(256)
);
```

参数详解

- NAMESPACE

  上下文的名称。

- ATTRIBUTE

  上下文的属性名称。

- VALUE

  上下文的属性值。

例 APPCTXTABTYP 为 APPCTXRECTYP 类型的索引表：

```
TYPE APPCTXTABTYP IS TABLE OF APPCTXRECTYP INDEX BY BINARY_INTEGER;
```

15.2 相关方法

1. CLEAR_ALL_CONTEXT 过程

清理当前会话的所有上下文。该过程需通过 namespace 关联的 package 调用。

语法如下:

```
DBMS_SESSION.CLEAR_ALL_CONTEXT(
    NAMESPACE    VARCHAR(30)
);
```

参数详解

- NAMESPACE

  不区分大小写。

2. CLEAR_CONTEXT 过程

清理 namespace 中的 attribute 值。该过程需通过上下文 namespace 关联的 package 调用。

语法如下:

```
DBMS_SESSION.CLEAR_CONTEXT(
    NAMESPACE              VARCHAR(30),
    CLIENT_IDENTIFIER   VARCHAR(30)
    ATTRIBUTE              VARCHAR(30)
);
```

参数详解

- NAMESPACE 和 ATTRIBUTE

  不区分大小写。

- CLIENT_IDENTIFIER

  用于全局访问的 CLIENT 标识，不起作用，如果指定，则忽略。

3. IS_SESSION_ALIVE 函数

会话是否为活动会话，如果是，则返回 TRUE，否则返回 FALSE。

语法如下:

```
DBMS_SESSION.IS_SESSION_ALIVE (
    UNIQUEID          BIGINT
)RETURN VARCHAR;
```

参数详解

- UNIQUEID

  为会话的唯一标识。

4. LIST_CONTEXT 过程

返回当前会话所有上下文属性和值。

语法如下:

```
DBMS_SESSION.LIST_CONTEXT(
    CONTEXT_LIST      OUT          APPCTXTABTYP,
    CONTEXT_SIZE      OUT          NUMBER
);
```

参数详解

- CONTEXT_LIST

  输出参数，类型为索引表。

- CONTEXT_SIZE

输出参数，该索引表中元素的总数。

5. RESET_PACKAGE 过程

清除当前会话上的所有 package 对象，即可以释放所有 package 对象占用的内存空间。

语法如下:

```
DBMS_SESSION.RESET_PACKAGE;
```

参数详解

无。

6. SET_CONTEXT 过程

设置上下文 namespace 的属性和值。

语法如下:

```
DBMS_SESSION.SET_CONTEXT (
    NAMESPACE       VARCHAR(30),
    ATTRIBUTE       VARCHAR(30),
    VALUE           VARCHAR(4000),
    USERNAME        VARCHAR(128),
    CLIENT_ID       VARCHAR(64)
);
```

参数详解

- NAMESPACE 和 ATTRIBUTE 不区分大小写。
- USERNAME 和 CLIENT_ID 参数不起作用，如果指定，则忽略。

7. UNIQUE_SESSION_ID 函数

返回唯一会话 id。

语法如下:

```
DBMS_SESSION.UNIQUE_SESSION_ID

RETURN VARCHAR;
```

8. CLOSE_DATABASE_LINK 过程

关闭一个指定的打开的外部链接，若该外部链接正在使用中（可通过 V$DBLINK 进行查询），则报错。

语法如下:

```
DBMS_SESSION.CLOSE_DATABASE_LINK (
    DBLINK          VARCHAR(128)
);
```

参数详解

- DBLINK

  为指定的要关闭的外部链接名。

9. SET_IDENTIFIER 过程

用于设置当前会话 ID。设置之后的会话 ID 可通过 V$SESSIONS 中 CLIENT_IDENTIFIER 字段查看。

语法如下:

```
DBMS_SESSION.SET_IDENTIFIER(
    SESSIONID           VARCHAR(64)
);
```

参数详解

- SESSIONID

  会话 ID。

15.3 举例说明

使用包内的过程和函数之前，如果还未创建过系统包，请先调用系统过程创建系统包。

```
SP_CREATE_SYSTEM_PACKAGES (1,'DBMS_SESSION');

SET SERVEROUTPUT ON;  //PRINT 需要设置这条语句，才能打印出消息
```

初始化环境：

```
//创建上下文关联的包

CREATE OR REPLACE PACKAGE TEST_PK AS

    PROCEDURE SET_CONTEXT(TS_NAME VARCHAR, KEY VARCHAR, VALUE VARCHAR);

    PROCEDURE CLEAR_CONTEXT(TS_NAME VARCHAR, CLIENTID VARCHAR, KEY VARCHAR);

    PROCEDURE CLEAR_ALL_CONTEXT(TS_NAME VARCHAR);

END TEST_PK;

/

CREATE OR REPLACE PACKAGE BODY TEST_PK AS

    PROCEDURE SET_CONTEXT(TS_NAME VARCHAR, KEY VARCHAR, VALUE VARCHAR) AS

    BEGIN

        DBMS_SESSION.SET_CONTEXT(TS_NAME, KEY, VALUE);

    END;

    PROCEDURE CLEAR_CONTEXT(TS_NAME VARCHAR, CLIENTID VARCHAR, KEY VARCHAR) AS

    BEGIN

        DBMS_SESSION.CLEAR_CONTEXT(TS_NAME, CLIENTID, KEY);
```

```
    END;
    PROCEDURE CLEAR_ALL_CONTEXT(TS_NAME VARCHAR) AS
    BEGIN
        DBMS_SESSION.CLEAR_ALL_CONTEXT(TS_NAME);
    END;
END TEST_PK;
/


//创建上下文 C_TEST
CREATE OR REPLACE CONTEXT C_TEST USING TEST_PK;
```

例 1 设置上下文 C_TEST 的属性和值。

```
CALL TEST_PK.SET_CONTEXT('C_TEST', 'K1', 'V1');
CALL TEST_PK.SET_CONTEXT('C_TEST', 'K2', 'V2');
```

例 2 查询上下文 C_TEST 的属性 K1 的值。

```
SELECT SYS_CONTEXT('C_TEST', 'K1') FROM DUAL;
```

结果如下:

```
V1
```

例 3 清除上下文 C_TEST 属性 K1 的值。

```
CALL TEST_PK.CLEAR_CONTEXT('C_TEST', NULL, 'K1');
```

再次查询名字空间 C_TEST 的属性 K1 的值。

```
SELECT SYS_CONTEXT('C_TEST', 'K1') FROM DUAL;
```

查询结果为空。

例 4 当前会话是否为活动会话

```
SELECT DBMS_SESSION.IS_SESSION_ALIVE(DBMS_SESSION.UNIQUE_SESSION_ID) FROM DUAL;
```

结果如下：

```
TRUE
```

例 5 打印当前会话所有的上下文属性和值

```
DECLARE
RES  DBMS_SESSION.APPCTXTABTYP;
NUM INT;
BEGIN
    DBMS_SESSION.LIST_CONTEXT (RES, NUM);
    FOR I IN 0..NUM-1 LOOP
      PRINT RES(I).NAMESPACE;
      PRINT RES(I).ATTRIBUTE;
      PRINT RES(I).VALUE;
    END LOOP;
END;
/
```

结果如下：

```
C_TEST
K1
NULL
C_TEST
K2
V2
```

例 6 删除上下文 C_TEST

```
DROP CONTEXT C_TEST;
```

例 7 清理当前会话的 package 对象

```
CREATE OR REPLACE PACKAGE TEST_P IS

CNT NUMBER := 0;

PROCEDURE PRINT_STATUS;

END TEST_P;

/



CREATE OR REPLACE PACKAGE BODY TEST_P IS

PROCEDURE PRINT_STATUS IS

BEGIN

 PRINT 'TEST_P.CNT = ' || CNT;

END;

END;

/
```

初始化并打印值:

```
BEGIN

TEST_P.CNT := 5236;

TEST_P.PRINT_STATUS;

END;

/
```

输出结果如下:

```
TEST_P.CNT = 5236
```

打印 CNT 值:

```
BEGIN
```

```
    TEST_P.PRINT_STATUS;

END;

/
```

输出结果如下:

```
TEST_P.CNT = 5236
```

清理包对象:

```
CALL DBMS_SESSION.RESET_PACKAGE;
```

再打印 CNT 值:

```
BEGIN

    TEST_P.PRINT_STATUS;

END;

/
```

输出结果如下:

```
TEST_P.CNT = 0
```

16 DBMS_SPACE 包

为了展示所有物理对象（文件、页）和逻辑对象（表空间、簇、段）的存储空间信息。通过 DBMS_SPACE 包来获取表空间、文件、页、簇、段的内容。DM DPC 环境下不支持 DBMS_SPACE 包。

16.1 数据类型

DBMS_SPACE 包中涉及到的变量和记录类型。如下统一说明：

包内变量和记录类型	解释
TS_T	表空间记录类型，用于记录表空间的信息
TS_ARR_T	表空间记录类型数组
TS_ALL_ARR_T	表空间 ID 数组
FILE_T	文件记录类型，用于记录文件的信息
FILE_ARR_T	文件记录类型数组
FILE_ALL_ARR_T	文件的 ID 数组
SEG_T	段记录类型，用于记录段的信息
SEG_ARR_T	段记录类型数组
SEG_ID_ARR_T	段 ID 数组
EXTENT_T	簇记录类型，用于记录簇的信息
EXTENT_ARR_T	簇记录类型数组
EXTENT_SIZE	簇的大小（页为单位）
PAGE_ADDR_T	页地址记录类型，用于记录页地址的信息
PAGE_ADDR_ARR_T	页地址记录类型数组

PAGE_N	描述页能描述的页数

下面为 DBMS_SPACE 包中记录类型的具体定义:

1. TS_T

TS_T 为表空间记录类型,用于记录表空间的信息,具体数据信息来源于 V$TABLESPACE,类型定义如下:

```
TYPE TS_T IS RECORD(
    ID              INT,
    NAME            VARCHAR(128),
    CACHE           VARCHAR(20),
    TYPE            TINYINT,
    STATUS          TINYINT,
    MAX_SIZE        BIGINT,
    TOTAL_SIZE      BIGINT,
    FILE_NUM        SMALLINT
);
```

参数详解

- ID

  表空间 ID。

- NAME

  表空间名称。

- CACHE

  缓冲区名。

- TYPE

  表空间类型，1：DB 类型，2：临时表空间。

- STATUS

  表空间状态。0：ONLINE；1：OFFLINE；2：RES_OFFLINE；3：CORRUPT。

- MAX_SIZE

  表空间的最大大小，以页为单位。0 代表只受操作系统限制（暂无实际意义）。

- TOTAL_SIZE

  表空间的总大小，以页为单位。

- FILE_NUM

  包含的文件数。

2. TS_ARR_T

TS_ARR_T 为表空间记录类型数组，类型定义如下：

```
TYPE TS_ARR_T IS ARRAY TS_T[];
```

3. TS_ALL_ARR_T

TS_ALL_ARR_T 为表空间 ID 数组，类型定义如下：

```
TYPE TS_ALL_ARR_T IS ARRAY INT[];
```

4. FILE_T

FILE_T 为文件记录类型，用于记录文件的信息，具体数据信息来源于 V$DATAFILE，类型定义如下：

```
TYPE FILE_T IS RECORD(
    PATH                VARCHAR(256),
    CREATE_TIME         TIMESTAMP,
    MODIFY_TIME         TIMESTAMP,
```

```
    TOTAL_SIZE              BIGINT,
    FREE_SIZE               BIGINT,
    FREE_PAGE_NO            BIGINT,
    PAGE_SIZE               INT,
    AUTOEXTEND              INT,
    MAX_SIZE                INT,
    NEXT_SIZE               INT
);
```

参数详解

- PATH

  数据库文件路径。

- CREATE_TIME

  数据库文件创建时间。

- MODIFY_TIME

  文件修改的时间。

- TOTAL_SIZE

  文件的总大小，以页为单位。

- FREE_SIZE

  未分配的空闲大小，以页为单位。

- FREE_PAGE_NO

  数据文件中连续空白页的起始页号。

- PAGE_SIZE

  页大小，单位 BYTE。

- AUTOEXTEND

是否支持自动扩展，1：支持；0：不支持。

- MAX_SIZE

  文件最大大小，以 MB 为单位。

- NEXT_SIZE

  文件每次扩展大小，以 MB 为单位。创建文件时没有指定则返回 0。

5. FILE_ARR_T

FILE_ARR_T 为文件记录类型数组，类型定义如下：

```
TYPE FILE_ARR_T IS ARRAY FILE_T[];
```

6. FILE_ALL_ARR_T

FILE_ALL_ARR_T 为文件的 ID 数组，类型定义如下：

```
TYPE FILE_ALL_ARR_T IS ARRAY INT[];
```

7. SEG_T

SEG_T 为段记录类型，用于记录段的信息，类型定义如下：

```
TYPE SEG_T IS RECORD(
    SEGID                   INT,
    INODE_FILE_ID           SMALLINT,
    INODE_PAGE_NO           INT,
    INODE_OFFSET            SMALLINT,
    N_FULL_EXTENT           INT,
    N_FRAG_EXTENT           INT,
    N_FREE_EXTENT           INT,
```

```
    N_USED_IN_FRAG      INT
);
```

参数详解

- SEG_ID

  段 ID。

- INODE_FILE_ID

  段 INODE 项所在的文件 ID。

- INODE_PAGE_NO

  段 INODE 项所在的页号。

- INODE_OFFSET

  段 INODE 项的页偏移。

- N_FULL_EXTENT

  全满簇的个数。

- N_FRAG_EXTENT

  半满簇的个数。

- N_FREE_EXTENT

  空闲簇的个数。

- N_USED_IN_FRAG

  半满簇中使用页的总数量。

8. SEG_ARR_T

SEG_ARR_T 为段记录类型数组，类型定义如下：

```
TYPE SEG_ARR_T IS ARRAY SEG_T[];
```

9. SEG_ID_ARR_T

SEG_ID_ARR_T 为段 ID 数组，类型定义如下:

```
TYPE SEG_ID_ARR_T IS ARRAY INT[];
```

10. EXTENT_T

EXTENT_T 为簇记录类型，用于记录簇的信息，类型定义如下:

```
TYPE EXTENT_T IS RECORD(
    TSID                    SMALLINT,
    SEGID                   INT,
    STATE                   VARCHAR(5),
    BITMAP                  VARCHAR(64),
    EXTENT_FILE_ID          SMALLINT,
    EXTENT_PAGE_NO          INT,
    EXTENT_OFFSET           SMALLINT,
    FIRST_PAGE_NO           INT,
    LAST_PAGE_NO            INT,
    NEXT_LINK_FILE_ID       SMALLINT,
    NEXT_LINK_PAGE_NO       INT,
    NEXT_LINK_OFFSET        SMALLINT
);
```

参数详解

- TS_ID

  表空间 ID。

- SEG_ID

段 ID。

- STATE

簇状态，NOUSE：未使用；FULL：全满；FRAG：半满；FREE：空闲；UNKNOWN：未知。

- BITMAP

簇的页标记位图。

- EXTENT_FILE_ID

簇描述项的文件 ID。

- EXTENT_PAGE_NO

簇描述项所在页号。

- EXTENT_OFFSET

簇描述项的页偏移。

- FIRST_PAGE_NO

簇的起始页号。

- LAST_PAGE_NO

簇的终止页号。

- NEXT_LINK_FILE_ID

下一个簇描述项的文件号 ID。

- NEXT_LINK_PAGE_NO

下一个簇描述项的页号。

- NEXT_LINK_OFFSET

下一个簇描述项的页偏移。

11. EXTENT_ARR_T

EXTENT_ARR_T 为簇记录类型数组，类型定义如下：

```
TYPE EXTENT_ARR_T IS ARRAY EXTENT_T[];
```

12. PAGE_ADDR_T

PAGE_ADDR_T 为页记录类型，用于记录页的信息，类型定义如下：

```
TYPE PAGE_ADDR_T IS RECORD(
    TS_ID                   SMALLINT,
    FILE_ID                 SMALLINT,
    PAGE_NO                 INT
);
```

参数详解

- TS_ID

  页所在的表空间 ID。

- FILE_ID

  页所在的文件 ID。

- PAGE_NO

  页号。

13. PAGE_ADDR_ARR_T

PAGE_ADDR_ARR_T 为页记录类型数组，类型定义如下：

```
TYPE PAGE_ADDR_ARR_T IS ARRAY PAGE_ADDR_T[];
```

16.2 相关方法

DBMS_SPACE 包中包含的过程和函数如下详细介绍：

1. TS_LOAD/ TS_GET

根据输入的表空间 ID，获得表空间信息。过程和函数功能相同。

语法如下：

```
PROCEDURE TS_LOAD(
    TS_ID          IN        SMALLINT,
    TS_ARR         OUT       TS_ARR_T
);
```

语法如下：

```
FUNCTION   TS_GET (
    TS_ID          IN        SMALLINT
)RETURN TS_ARR_T;
```

参数详解

- TS_ID

  表空间 ID。

- TS_ARR_T

  表空间记录类型数组。

2. TS_N_LOAD / TS_N_GET

获得数据库中表空间的个数。过程和函数功能相同。

语法如下：

```
PROCEDURE TS_N_LOAD(
    NUMBER      OUT     INT
);
```

语法如下:

```
FUNCTION TS_N_GET
RETURN INT;
```

参数详解

- NUMBER

  表空间的个数。

3. TS_ALL_LOAD/TS_ALL_GET

   获得所有表空间的 ID。过程和函数功能相同。

   语法如下:

```
PROCEDURE TS_ALL_LOAD(
    TS_ALL_ARR  OUT     TS_ALL_ARR_T,
    OFFSET      IN      INT     DEFAULT 1,
    NUM         IN      INT     DEFAULT 100
);
```

语法如下:

```
FUNCTION TS_ALL_GET(
    OFFSET      IN      INT     DEFAULT 1,
    NUM         IN      INT     DEFAULT 100
```

```
) RETURN TS_ALL_ARR_T;
```

参数详解

- OFFSET

  数组起始位置。

- NUM

  数组长度

- TS_ALL_ARR

  表空间 ID 数组

4. FILE_LOAD/ FILE_GET

   根据输入的表空间 ID、文件 ID，获得文件信息。过程和函数功能相同。

   语法如下：

```
PROCEDURE FILE_LOAD(
    TS_ID          IN        SMALLINT,
    FILE_ID        IN        SMALLINT,
    FILE_ARR  OUT        FILE_ARR_T
);
```

   语法如下：

```
FUNCTION FILE_GET(
    TS_ID          IN        SMALLINT,
    FILE_ID        IN        SMALLINT
) RETURN FILE_ARR_T;
```

参数详解

- TS_ID

  表空间 ID。

- FILE_ID

  文件号。

- FILL_ARR

  文件记录类型数组。

5. FILE_ALL_LOAD/ FILE_ALL_GET

获得所有文件的 ID。过程和函数功能相同。

语法如下:

```
PROCEDURE FILE_ALL_LOAD(
    TS_ID              IN      SMALLINT,
    FILE_ALL_ARR  OUT     FILE_ALL_ARR_T,
    OFFSET             IN      INT       DEFAULT 1,
    NUM                IN      INT       DEFAULT 100
);
```

语法如下:

```
FUNCTION FILE_ALL_GET(
    TS_ID          IN      SMALLINT,
    OFFSET         IN      INT       DEFAULT 1,
    NUM            IN      INT       DEFAULT 100
```

```
)RETURN FILE_ALL_ARR_T;
```

参数详解

- TS_ID

  表空间 ID。

- FILE_ALL_ARR

  文件的 ID 数组。

- OFFSET

  数组起始位置。

- NUM

  数组的长度。

6. SEG_LOAD_BY_ID/SEG_GET_BY_ID

根据输入的表空间 ID、段号获得段信息。过程和函数功能相同。

语法如下:

```
PROCEDURE SEG_LOAD_BY_ID(
    TS_ID          IN        SMALLINT,
    SEG_ID         IN        INT,
    SEG_INFO       OUT       SEG_T
);
```

语法如下:

```
FUNCTION SEG_GET_BY_ID(
    TS_ID          IN        SMALLINT,
    SEG_ID         IN        INT
```

```
)RETURN SEG_T;
```

参数详解

- TS_ID

  表空间 ID。

- SEG_ID

  段号。

- SEG_INFO

  段的信息。

7. SEG_LOAD_BY_HEADER/ SEG_GET_BY_HEADER

根据输入的表空间 ID、段头的文件号、页号、页内偏移，获得段信息，其中页内偏移取值只能为 62 或 72。过程和函数功能相同。

语法如下:

```
PROCEDURE SEG_LOAD_BY_HEADER(
    TS_ID                IN      SMALLINT,
    HEADER_FILE_ID       IN      SMALLINT,
    HEADER_PAGE_NO       IN      INT,
    HEADER_OFFSET        IN      SMALLINT,
    SEG_INFO             OUT         SEG_T
);
```

语法如下:

```
FUNCTION SEG_GET_BY_HEADER(
    TS_ID                IN      SMALLINT,
```

```
    HEADER_FILE_ID          IN          SMALLINT,
    HEADER_PAGE_NO          IN          INT,
    HEADER_OFFSET           IN          SMALLINT
)RETURN SEG_T;
```

参数详解

- TS_ID

  表空间 ID。

- HEADER_FILE_ID

  段头文件号。

- HEADER_PAGE_NO

  段头页号。

- HEADER_OFFSET

  段头偏移量。

- SEG_INFO

  段的信息。

8. SEG_ALL_LOAD/SEG_ALL_GET

根据表空间 ID、数组相对起始位置、数组长度，获得数据库中表空间中所有段的 ID 信息。过程和函数功能相同。

语法如下:

```
PROCEDURE SEG_ALL_LOAD(
    TS_ID               IN          SMALLINT,
    SEG_ID_ARR          OUT         SEG_ID_ARR_T,
```

```
    OFFSET          IN      INT     DEFAULT 1,
    NUM             IN      INT     DEFAULT 100
);
```

语法如下:

```
FUNCTION SEG_ALL_GET(
    TS_ID           IN      SMALLINT,
    OFFSET          IN      INT     DEFAULT 1,
    NUM             IN      INT     DEFAULT 100
)RETURN SEG_ID_ARR_T;
```

参数详解

- TS_ID

  表空间 ID。

- OFFSET

  数组起始位置。

- NUM

  数组长度。

- SEG_ID_ARR

  段 ID 信息。

9. SEG_EXTENT_LST_LOAD/SEG_EXTENT_LST_GET

根据输入的表空间 ID、段号、链表类型、链表方向、链表相对起始位置、指定链表长度，获得 FULL、FREE 或 FRAG 链表的信息。过程和函数功能相同。

语法如下:

```
PROCEDURE SEG_EXTENT_LST_LOAD(
    TS_ID           IN      SMALLINT,
    SEG_ID          IN      INT,
    EXTENT_ARR      OUT         EXTENT_ARR_T,
    LINKTYPE    IN      VARCHAR(5)      DEFAULT 'FULL',
    DIRECT          IN      VARCHAR(10)     DEFAULT 'FORWARD',
    OFFSET          IN      INT             DEFAULT 1,
    NUM                 IN      INT             DEFAULT 100
);
```

语法如下:

```
FUNCTION SEG_EXTENT_LST_GET(
    TS_ID           IN      SMALLINT,
    SEG_ID          IN      INT,
    LINKTYPE        IN      VARCHAR(5)      DEFAULT 'FULL',
    DIRECT          IN      VARCHAR(10)     DEFAULT 'FORWARD',
    OFFSET          IN      INT             DEFAULT 1,
    NUM             IN      INT             DEFAULT 100,
) RETURN EXTENT_ARR_T;
```

参数详解

- TS_ID

  表空间 ID。

- SEG_ID

  段 ID。

- LINKTYPE

链表类型。

- DIRECT

  链表方向。

- OFFSET

  链表相对起始位置。

- NUM

  指定链表长度。

- EXTENT_ARR

  簇信息结构体链表

10. EXTENT_SIZE_LOAD/EXTENT_SIZE_GET

    获得簇的大小（页为单位）。过程和函数功能相同。

    语法如下:

```
PROCEDURE EXTENT_SIZE_LOAD(
    EXTENT_SIZE    OUT              INT
);
```

    语法如下:

```
FUNCTION    EXTENT_SIZE_GET    RETURN INT;
```

    参数详解

- EXTENT_SIZE

  簇的大小。

11. EXTENT_XDESC_PAGE_NO_LOAD/EXTENT_XDESC_PAGE_NO_GET

根据输入的表空间 ID、文件 ID，获得所有簇描述页的信息。过程和函数功能相同。

语法如下：

```
PROCEDURE EXTENT_XDESC_PAGE_NO_LOAD(
    TS_ID                   IN      SMALLINT,
    FILE_ID                 IN      SMALLINT,
    XPAGE_ADDR_ARR      OUT         PAGE_ADDR_ARR_T,
    OFFSET                  IN      INT     DEFAULT 1,
    NUM                     IN      INT     DEFAULT 100
);
```

语法如下：

```
FUNCTION EXTENT_XDESC_PAGE_NO_GET(
    TS_ID               IN      SMALLINT,
    FILE_ID             IN      SMALLINT,
    OFFSET              IN      INT     DEFAULT 1,
    NUM                 IN      INT     DEFAULT 100
)RETURN PAGE_ADDR_ARR_T;
```

参数详解

- TS_ID

  表空间 ID。

- FILE_ID

  文件 ID。

- OFFSET

  数组的起始位置。

- NUM

  指定输出的长度。

- XPAGE_ADDR_ARR

  页地址记录类型数组。

12. PAGE_N_LOAD/ PAGE_N_GET

获得一个描述页能描述的页数。过程和函数功能相同。

语法如下:

```
PROCEDURE PAGE_N_LOAD(
    PAGE_N      OUT         INT
);
```

语法如下:

```
FUNCTION PAGE_N_GET
RETURN INT;
```

参数详解

- PAGE_N

  描述页能描述的页数。

13. IPAGE_NO_ALL_LOAD/ IPAGE_NO_ALL_GET

输入表空间 ID、INODE 页链表类型（FULL 或 FREE)、链表方向、链表相对起始位置、指定链表长度，输出所有 INODE 页的页号。过程和函数功能相同。

语法如下:

```
PROCEDURE IPAGE_NO_ALL_LOAD(
    TS_ID           IN      SMALLINT,
```

```
    PAGE_ADDR_ARR  OUT          PAGE_ADDR_ARR_T ,
    LINKTYPE       IN       VARCHAR(5)      DEFAULT 'FULL',
    DIRECT         IN       VARCHAR(10)     DEFAULT 'FORWARD',
    OFFSET         IN       INT             DEFAULT 1,
    NUM                IN       INT             DEFAULT 100
);
```

语法如下:

```
FUNCTION IPAGE_NO_ALL_GET(
    TS_ID          IN       SMALLINT,
    LINKTYPE       IN       VARCHAR(5)      DEFAULT 'FULL',
    DIRECT         IN       VARCHAR(10)     DEFAULT 'FORWARD',
    OFFSET         IN       INT             DEFAULT 1,
    NUM                IN       INT             DEFAULT 100
)RETURN PAGE_ADDR_ARR_T;
```

参数详解

- TS_ID

  表空间 ID。

- PAGE_ADDR_ARR

  页地址记录类型数组。

- LINKTYPE

  页链表类型：FULL 或 FREE。

- DIRECT

  链表的方向。

- OFFSET

  链表的相对起始位置。

- NUM

  指定链表长度。

16.3 举例说明

使用包内的过程和函数之前，如果还未创建过系统包，请先调用系统过程创建系统包。

```
SP_CREATE_SYSTEM_PACKAGES(1,'DBMS_SPACE');

SET SERVEROUTPUT ON;  //PRINT 需要设置这条语句，才能打印出消息
```

例 1 获取表空间信息（表空间 ID、表空间名、表空间类型：1 DB 类型，2 临时文件组、表空间状态、表空间的最大空间、表空间的总大小（页）、包含文件的个数）。

```
DECLARE

  TS_INFO    DBMS_SPACE.TS_ARR_T;

BEGIN

    TS_INFO = DBMS_SPACE.TS_GET(4);

    IF TS_INFO IS NOT NULL

    THEN

        SELECT * FROM ARRAY TS_INFO;

    ELSE PRINT 'TABLESPACE IS NULL';

    END IF;

END;

/
```

结果如下：

```
行号    ID    NAME    CACHE TYPE    STATUS    MAX_SIZE    TOTAL_SIZE          FILE_NUM
```

```
---------- ---------- ---- ----- ---------- ---------- --------------------

1       4       MAIN          1               0               0               16384                   1
```

例 2 获得数据库中表空间的个数

```
DECLARE

TS_NUM INT;

BEGIN

TS_NUM = DBMS_SPACE.TS_N_GET;

PRINT 'TS_NUM:';

PRINT TS_NUM;

END;

/
```

结果如下：

```
TS_NUM:

5                   //个数根据实际表空间个数不同而不同
```

例 3　获得文件信息（文件路径、文件创建时间、文件修改的时间、文件的总大小（MB）、文件的空闲大小（MB）、数据文件中连续空白页的起始页号、页大小（KB）、文件可扩展标记、文件最大大小（MB）、文件每次扩展大小（MB））

```
DECLARE

  FILE_INFO        DBMS_SPACE.FILE_ARR_T;

BEGIN

    FILE_INFO = DBMS_SPACE.FILE_GET(4,0);

    IF FILE_INFO IS NOT NULL

    THEN

        SELECT * FROM ARRAY FILE_INFO;
```

```
    ELSE PRINT 'FILE IS NULL';

    END IF;

END;

/
```

结果如下:

```
行号    PATH    CREATE_TIME    MODIFY_TIME    TOTAL_SIZE    FREE_SIZE    FREE_PAGE_NO

PAGE_SIZE    AUTOEXTEND MAX_SIZE     NEXT_SIZE

--------- ------------------------------ -------------------------

1    D:\DATABASE0718\DAMENG\MAIN.DBF    2012-07-19 09:47:55.000000    2012-08-07

09:43:33.000000    16384    16348    368     8192          1             0             0
```

17 DBMS_SQL 包

支持在 PL/SQL 中使用动态 SQL 语句。

17.1 相关方法

DBMS_SQL 包中所用到的结构定义如下：

```
TYPE BINARY_FLOAT_TABLE IS TABLE OF  FLOAT INDEX BY INTEGER;

TYPE BLOB_TABLE IS TABLE OF BLOB INDEX BY INTEGER;

TYPE CLOB_TABLE IS TABLE OF CLOB INDEX BY INTEGER;

TYPE DATE_TABLE IS TABLE OF DATETIME INDEX BY INTEGER;

TYPE DESC_TAB IS TABLE OF DESC_REC INDEX BY INTEGER;

TYPE DESC_TAB2 IS TABLE OF DESC_REC2 INDEX BY INTEGER;

TYPE DESC_TAB3 IS TABLE OF DESC_REC3 INDEX BY INTEGER;

TYPE INTERVAL_DAY_TO_SECOND_TABLE IS TABLE OF  INTERVAL DAY TO SECOND INDEX BY
INTEGER;

TYPE INTERVAL_YEAR_TO_MONTH_TABLE IS TABLE OF  INTERVAL YEAR TO MONTH INDEX BY
INTEGER;

TYPE INT_TABLE IS TABLE OF INT INDEX BY INTEGER;

TYPE NUMBER_TABLE IS TABLE OF NUMBER INDEX BY INTEGER;

TYPE TIME_TABLE IS TABLE OF TIME INDEX BY INTEGER;

TYPE TIME_WITH_TIME_ZONE_TABLE IS TABLE OF  TIME WITH TIME ZONE  INDEX BY INTEGER;

TYPE TIMESTAMP_TABLE IS TABLE OF DATETIME INDEX BY INTEGER;

TYPE TIMESTAMP_WITH_LTZ_TABLE IS TABLE OF  TIMESTAMP WITH LOCAL TIME ZONE INDEX BY
```

```
INTEGER;
TYPE TIMESTAMP_WITH_TIME_ZONE_TABLE IS TABLE OF  TIMESTAMP WITH TIME ZONE INDEX BY
INTEGER;
TYPE VARCHAR2_TABLE IS TABLE OF VARCHAR2(2000) INDEX BY INTEGER;
TYPE VARCHAR2A IS TABLE OF VARCHAR2(32767) INDEX BY INTEGER;
```

DESC_REC 的结构如下:

```
TYPE DESC_REC IS RECORD(
        COL_TYPE                    INTEGER := 0,
        COL_MAX_LEN                 INTEGER := 0,
        COL_NAME                    VARCHAR2(128)    := '',
        COL_NAME_LEN                INTEGER := 0,
        COL_SCHEMA_NAME             VARCHAR2(128)    := '',
        COL_SCHEMA_NAME_LEN INTEGER := 0,
        COL_PRECISION               INTEGER := 0,
        COL_SCALE                   INTEGER := 0,
        COL_CHARSETID               INTEGER := 0,
        COL_CHARSETFORM             INTEGER := 0,
        COL_NULL_OK                 BOOLEAN          := TRUE);
```

使用 DESCRIBE_COLUMNS 函数可以将查询项的数据类型填充到 DESC_REC 中，其中服务器部分数据类型和 DBMS_SQL 中的数据类型的映射关系如下:

```
TYPECODE_VARCHAR                INTEGER :=    1;
TYPECODE_NUMBER                 INTEGER :=    2;
TYPECODE_INT                    INTEGER :=    3;
```

```
TYPECODE_SMALLINT               INTEGER :=    4;
TYPECODE_TINYINT                INTEGER :=    5;
TYPECODE_BIGINT                 INTEGER :=    6;
TYPECODE_FLOAT                  INTEGER :=    7;
TYPECODE_VARCHAR2               INTEGER :=    9;
TYPECODE_DATE                   INTEGER :=   12;
TYPECODE_VARBINARY              INTEGER :=   23;
TYPECODE_RAW                    INTEGER :=   95;
TYPECODE_CHAR                   INTEGER :=   96;
TYPECODE_MLSLABEL               INTEGER :=   105;
TYPECODE_AOT                    INTEGER :=   109; // TYPECODE_AOT 为用户自定义类型
TYPECODE_BLOB                   INTEGER := 113;
TYPECODE_BFILE                  INTEGER := 114;
TYPECODE_CLOB                   INTEGER := 112;
TYPECODE_CFILE                  INTEGER := 115;
TYPECODE_TIME                   INTEGER := 170;
TYPECODE_TIME_TZ                INTEGER := 171;
TYPECODE_TIMESTAMP              INTEGER := 180;
TYPECODE_TIMESTAMP_TZ           INTEGER := 181;
TYPECODE_TIMESTAMP_LTZ          INTEGER := 232;
TYPECODE_INTERVAL_YM            INTEGER := 182;
TYPECODE_INTERVAL_DS            INTEGER := 183;
TYPECODE_REF                    INTEGER := 110;
TYPECODE_OBJECT                 INTEGER := 108;
```

```
TYPECODE_VARRAY                INTEGER := 247;
```

DBMS_SQL 包中包含的过程和函数如下详细介绍:

1. BIND_ARRAY

按照 SQL 语句中变量名将变量值绑定到游标的索引表上。

语法如下:

```
DBMS_SQL.BIND_ARRAY (
    CURID                IN      INTEGER,
    NAME                 IN      VARCHAR2,
    <TABLE_VARIABLE>     IN      <DATATYPE>
);
或
DBMS_SQL.BIND_ARRAY (
    CURID                IN      INTEGER,
    NAME                 IN      VARCHAR2,
    <TABLE_VARIABLE>     IN      <DATATYPE>,
    INDEX1               IN      INTEGER,
    INDEX2               IN      INTEGER
);
```

参数详解

- CURID

游标。

- NAME

  变量名称。

- <TABLE_VARIABLE>

  索引表，<DATATYPE>是 DBMS_SQL 包中的索引表类型，如下所示：

  - binary_float_table
  - blob_table
  - clob_table
  - date_table
  - interval_day_to_second_Table
  - interval_year_to_month_table
  - Int_Table
  - number_table
  - time_table
  - time_with_time_zone_table
  - timestamp_table
  - timestamp_with_ltz_table
  - timestamp_with_time_zone_Table
  - varchar2_table
  - varchar2a

- index1

  绑定数组表的起始下标。

- index2

  绑定数组表的结束下标。

2. BIND_VARIABLE

按照 SQL 语句中变量名将变量值绑定到游标的基本数据类型变量上。

语法如下:

```
DBMS_SQL.BIND_VARIABLE (
    C              IN      INTEGER,
    NAME           IN      VARCHAR2,
    VALUE          IN      <DATATYPE>
    CHAR_SIZE      IN      INTEGER
);
或
DBMS_SQL.BIND_VARIABLE (
    C              IN      INTEGER,
    NAME           IN      VARCHAR2,
    VALUE          IN      <DATATYPE>,
    CHAR_SIZE      IN      INTEGER
);
```

参数详解

- C

  游标。

- NAME

  变量名称。

- VALUE

  变量值。可以为任何类型，类型<DATATYPE>是 DBMS_SQL 包中基本数据类型，如下所示:

  - FLOAT
  - CLOB
  - DATETIME
  - INTERVAL DAY TO SECOND
  - NUMBER
  - TIME
  - TIME WITH TIME ZONE
  - TIMESTAMP WITH TIME ZONE
  - VARCHAR
  - INTERVAL YEAR TO MONTH
  - INT
  - SMALLINT
  - TINYINT

- BIGINT
- VARBINARY
- BLOB

- CHAR_SIZE

  允许绑定参数输入输出的最大字符数量，仅对 VARBINARY、VARCHAR 数据类型有效。

3. BIND_VARIABLE_RAW

绑定二进制数据类型变量，可通过 OUT_VALUE_SIZE 指定 VARBINARY 长度。

语法如下:

```
DBMS_SQL.BIND_VARIABLE_RAW (
    C               IN      INTEGER,
    NAME            IN      VARCHAR2,
    VALUE           IN      VARBINARY
);
或
DBMS_SQL.BIND_VARIABLE_RAW (
    C               IN      INTEGER,
    NAME            IN      VARCHAR2,
    VALUE           IN      VARBINARY,
    OUT_VALUE_SIZE  IN      INTEGER
);
```

参数详解

- C

  游标。

- NAME

  变量名称。

- VALUE

  变量值，数据类型为 VARBINARY。

- OUT_VALUE_SIZE

  指定 VARBINARY 的长度。

4. BIND_VARIABLE_CHAR

绑定 VARCHAR 数据类型变量，可通过 OUT_VALUE_SIZE 指定 VARCHAR 长度。

语法如下:

```
DBMS_SQL.BIND_VARIABLE_CHAR (
    C               IN      INTEGER,
    NAME            IN      VARCHAR2,
    VALUE           IN      VARCHAR
);
或
DBMS_SQL.BIND_VARIABLE_CHAR (
    C               IN      INTEGER,
    NAME            IN      VARCHAR2,
    VALUE           IN      VARCHAR,
    OUT_VALUE_SIZE  IN  INTEGER
```

```
);
```

参数详解

- C

  游标。

- NAME

  变量名称。

- VALUE

  变量值，数据类型为 VARCHAR。

- OUT_VALUE_SIZE

  指定 VARCHAR 的长度。

5. CLOSE_CURSOR

关闭所给定的游标。

语法如下:

```
DBMS_SQL.CLOSE_CURSOR (
    C           IN OUT          INTEGER
);
```

参数详解

- C

  游标。

6. COLUMN_VALUE

根据列的位置，返回游标中的列值，通常在 FETCH_ROWS 后被调用。

语法如下:

```
DBMS_SQL.COLUMN_VALUE (
    C                   IN      INTEGER,
    POSITION            IN      INTEGER,
    VALUE               OUT     <DATATYPE>
);
或
DBMS_SQL.COLUMN_VALUE (
    C                   IN      INTEGER,
    POSITION            IN      INTEGER,
    VALUE               OUT     <DATATYPE>,
    COLUMN_ERROR        OUT     NUMBER
);
或
DBMS_SQL.COLUMN_VALUE (
    C                   IN      INTEGER,
    POSITION            IN      INTEGER,
    VALUE               OUT     <DATATYPE>,
    COLUMN_ERROR        OUT     NUMBER,
    ACTUAL_LENGTH       OUT     INTEGER
);
```

参数详解

- C

游标。

- POSITION

  为对应动态 SQL 中列的位置。

- VALUE

  列值。<DATATYPE>可以为任何类型。

- COLUMN_ERROR

  仅语法兼容，无实际意义。

- ACTUAL_LENGTH

  仅语法兼容，无实际意义。

7. COLUMN_VALUE_CHAR

根据列的位置，返回数据类型为 CHAR 的列值。

语法如下:

```
DBMS_SQL.COLUMN_VALUE_CHAR (
    C                   IN      INTEGER,
    POSITION            IN      INTEGER,
    VALUE               OUT         CHAR
);
或
DBMS_SQL.COLUMN_VALUE_CHAR (
    C                   IN      INTEGER,
    POSITION            IN      INTEGER,
    VALUE               OUT         CHAR,
```

```
    COLUMN_ERROR        OUT         NUMBER
);
```

或

```
DBMS_SQL.COLUMN_VALUE_CHAR (
    C                   IN      INTEGER,
    POSITION            IN      INTEGER,
    VALUE               OUT         CHAR,
    COLUMN_ERROR        OUT         NUMBER,
    ACTUAL_LENGTH       OUT         INTEGER
);
```

参数详解

- C

  游标。

- POSITION

  为对应动态 SQL 中列的位置。

- VALUE

  列值，数据类型为 CHAR。

- COLUMN_ERROR

  仅语法兼容，无实际意义。

- ACTUAL_LENGTH

  仅语法兼容，无实际意义。

8. COLUMN_VALUE_RAW

根据列的位置，返回数据类型为 BINARY 的列值。

语法如下:

```
DBMS_SQL.COLUMN_VALUE_RAW (
    C                   IN      INTEGER,
    POSITION            IN      INTEGER,
    VALUE               OUT         BINARY
);
或
DBMS_SQL.COLUMN_VALUE_RAW (
    C                   IN      INTEGER,
    POSITION            IN      INTEGER,
    VALUE               OUT         BINARY,
    COLUMN_ERROR        OUT         NUMBER
);
或
DBMS_SQL.COLUMN_VALUE_RAW (
    C                   IN      INTEGER,
    POSITION            IN      INTEGER,
    VALUE               OUT         BINARY,
    COLUMN_ERROR        OUT         NUMBER,
    ACTUAL_LENGTH       OUT         INTEGER
);
```

参数详解

- C

游标。

- POSITION

  为对应动态 SQL 中列的位置。

- VALUE

  列值，数据类型为 BINARY。

- COLUMN_ERROR

  仅语法兼容，无实际意义。

- ACTUAL_LENGTH

  仅语法兼容，无实际意义。

9. DEFINE_ARRAY

定义索引表类型的接收列，用于接收 FETCH 结果集，一次可获取多行数据。

语法如下:

```
DBMS_SQL.DEFINE_ARRAY (
    C                   IN      INTEGER,
    POSITION            IN      INTEGER,
    <TABLE_VARIABLE>    IN      <DATATYPE>,
    LEAST_FETCH         IN      INTEGER,
    START_OP            IN      INTEGER
);
```

参数详解

- C

  游标。

- POSITION

  为对应动态 SQL 中列的位置。

- <TABLE_VARIABLE>

  索引表，<DATATYPE>是 DBMS_SQL 包中的索引表类型，具体可参考 BIND_ARRAY 的参数介绍。

- LEAST_FETCH

  至少 FETCH 的记录行数。

- START_OP

  指定数组表下标，从指定位置开始填充 FETCH 结果集。

10. DEFINE_COLUMN

定义基本数据类型接收列，用于接收 FETCH 结果集，一次获取一行数据。

语法如下:

```
DBMS_SQL.DEFINE_COLUMN (
    C                   IN      INTEGER,
    POSITION            IN      INTEGER,
    COL                 IN      <DATATYPE>
);
```

参数详解

- C

  游标。

- POSITION

  为对应动态 SQL 中的位置(从 1 开始)。

- COL

  该值所对应的变量，可以为任何类型。

11. EXECUTE

执行给定游标内的 SQL 语句。

语法如下:

```
DBMS_SQL.EXECUTE (
    C           IN          INTEGER
) RETURN INTEGER;
```

参数详解

- C

  游标。

返回值

返回处理的数据记录行数，仅对 INSERT、DELETE、UPDATE 操作有效，对于其他操作返回 0。

12. EXECUTE_AND_FETCH

执行给定游标内的 SQL 语句，同时将 FETCH 数据。

语法如下:

```
DBMS_SQL.EXECUTE_AND_FETCH (
    C               IN          INTEGER,
    EXACT           IN          BOOLEAN     DEFAULT FALSE
```

```
)RETURN INTEGER;
```

参数详解

- C

  游标。

- EXACT

  仅语法兼容，无实际意义。

13. FETCH_ROWS

FETCH 指定游标中的数据，同时返回 FETCH 的行数。为 0 时表示已经取到游标末端。

语法如下:

```
DBMS_SQL.FETCH_ROWS (
    C               IN          INTEGER
)RETURN INTEGER;
```

参数详解

- C

  游标。

14. IS_OPEN

判断游标是否打开，若打开则返回 TRUE，否则返回 FALSE。

语法如下:

```
DBMS_SQL.IS_OPEN (
    C               IN          INTEGER
```

```
)RETURN BOOLEAN;
```

参数详解

- C

  游标。

15. OPEN_CURSOR

打开一个游标，返回游标号。

语法如下:

```
DBMS_SQL.OPEN_CURSOR

RETURN INTEGER;
```

16. PARSE

解析 SQL 语句。

语法如下:

```
DBMS_SQL.PARSE (

   C                    IN        INTEGER,

   STATEMENT            IN        VARCHAR2,

   LANGUAGE_FLAG        IN        NUMBER

);
```

参数详解

- C

  游标。

- sTATEMENT

  动态游标所提供的 SQL 语句。

- LANGUAGE_FLAG

  仅语法兼容，无实际意义。

17. TO_CURSOR_NUMBER

将一个外部游标转化为 DBMS_SQL 包内部的游标，返回游标号。

语法如下:

```
DBMS_SQL.TO_CURSOR_NUMBER(
    RC                  IN OUT      SYS_REFCURSOR
)RETURN INTEGER;
```

参数详解

- RC

  外部游标。

18. TO_REFCURSOR

将 DBMS_SQL 包内的游标转换为 PL/SQL 游标。

语法如下:

```
DBMS_SQL.TO_REFCURSOR(
    CURSOR_NUMBER   IN OUT      INTEGER
)RETURN SYS_REFCURSOR;
```

参数详解

- CURSOR_NUMBER

  游标。

19. DESCRIBE_COLUMNS

获取查询项的描述信息，需要一个执行过查询操作的游标作为输入参数。

语法如下:

```
DBMS_SQL.DESCRIBE_COLUMNS (
    C           IN      INTEGER,
    COL_CNT     OUT     INTEGER,
    DESC_T      OUT     DESC_TAB
);
```

参数详解

- C

  游标。

- COL_CNT

  查询项数量。

- DESC_T

  查询项描述信息，数据类型为数组表。

20. VARIABLE_VALUE

获取变量的值，通常需要先执行 BIND_VARIABLE。

语法如下:

```
DBMS_SQL.VARIABLE_VALUE (
    C               IN      INTEGER,
    NAME            IN      VARCHAR2,
    VALUE           OUT     <DATATYPE>
);
```

参数详解

- C

  游标。

- NAME

  变量名。

- VALUE

  变量值，可以为任何类型。

21. VARIABLE_VALUE_CHAR

获取数据类型为 VARCHAR 的变量值。

语法如下:

```
DBMS_SQL.VARIABLE_VALUE_CHAR (
    C               IN      INTEGER,
    NAME            IN      VARCHAR2,
    VALUE           OUT     VARCHAR
);
```

参数详解

- C

游标。

- NAME

  变量名。

- VALUE

  变量值，数据类型为 VARCHAR。

22. VARIABLE_VALUE_RAW

获取数据类型为 VARBINARY 的变量值。

语法如下:

```
DBMS_SQL.VARIABLE_VALUE_RAW (
    C               IN      INTEGER,
    NAME            IN      VARCHAR2,
    VALUE           OUT     VARBINARY
);
```

参数详解

- C

  游标。

- NAME

  变量名。

- VALUE

  变量值，数据类型为 VARBINARY。

23. DEFINE_COLUMN_CHAR

定义 VARCHAR 数据类型接收列，用于接收 FETCH 结果集，一次获取一行数据。

语法如下:

```
DBMS_SQL.DEFINE_COLUMN_CHAR(
    C               IN      INTEGER,
    POSITION        IN      INTEGER,
    COLUMN          IN      VARCHAR,
    COLUMN_SIZE     IN      INTEGER
);
```

参数详解

- C

  游标。

- POSITION

  为对应动态 SQL 中列的位置。

- COLUMN

  列值。

- COLUMN_SIZE

  可接收的最大列值。

24. DEFINE_COLUMN_RAW

定义 RAW 数据类型接收列，用于接收 FETCH 结果集，一次获取一行数据。

语法如下:

```
DBMS_SQL.DEFINE_COLUMN_RAW(
    C               IN      INTEGER,
```

```
    POSITION        IN      INTEGER,
    COLUMN          IN      RAW,
    COLUMN_SIZE     IN      INTEGER
);
```

参数详解

- C

  游标。

- POSITION

  为对应动态 SQL 中列的位置。

- COLUMN

  列值。

- COLUMN_SIZE

  可接收的最大列值。

25. DEFINE_COLUMN_LONG

定义 CLOB 数据类型接收列，用于接收 FETCH 结果集，一次获取一行数据。

语法如下:

```
DBMS_SQL.DEFINE_COLUMN_LONG(
    C               IN      INTEGER,
    POSITION        IN      INTEGER
);
```

参数详解

- C

游标。

- POSITION

  为对应动态 SQL 中列的位置。

26. DESCRIBE_COLUMNS3

获取查询项的描述信息，需要一个执行过查询操作的游标作为输入参数。

DESCRIBE_COLUMNS3 和 DESCRIBE_COLUMNS 的唯一区别是 DESCRIBE_COLUMNS3 在描述时可以增加获取类型名称信息。

语法如下:

```
DBMS_SQL.DESCRIBE_COLUMNS3 (
    C               IN          INTEGER,
    COL_CNT         OUT         INTEGER,
    DESC_T              OUT         DESC_TAB3
);
```

参数详解

- C

  游标。

- COL_CNT

  查询项数量。

- DESC_T

  查询项描述信息，数据类型为数组表。

17.2 举例说明

使用包内的过程和函数之前，如果还未创建过例子中的系统包，请先调用系统过程创建系统包。

```
SP_CREATE_SYSTEM_PACKAGES (1, 'DBMS_SQL');

SP_CREATE_SYSTEM_PACKAGES (1,'DBMS_OUTPUT');

SET SERVEROUTPUT ON;  //DBMS_OUTPUT.PUT_LINE 需要设置这条语句，才能打印出消息
```

下面是利用 DBMS_SQL 动态执行语句的例子。

例 1　一次获取一行数据

```
CREATE TABLE T1(N1 NUMBER);

INSERT INTO T1 VALUES(1.1),(2.1),(3.2),(4.3);

//非 COLLECTION 类型

DECLARE

  C          NUMBER;

  D1         NUMBER;

BEGIN

  C := DBMS_SQL.OPEN_CURSOR;  //打开游标

  DBMS_SQL.PARSE(C, 'SELECT N1 FROM SYSDBA.T1', '1');  //解析 SQL 语句

  DBMS_SQL.DEFINE_COLUMN(C, 1, D1);//定义列，将来可用 COLUMN_VALUE 获取列值

  DBMS_SQL.EXECUTE(C);//执行语句

  DBMS_SQL.FETCH_ROWS(C);//获取结果集

  DBMS_SQL.COLUMN_VALUE(C, 1, D1);//获取列值

  DBMS_SQL.CLOSE_CURSOR(C);//关闭游标

  DBMS_OUTPUT.PUT_LINE(D1);
```

```
END;
/


//输出结果:
1.1
```

例 2 一次获取多行数据

```
DECLARE
   C            NUMBER;
   D1           DBMS_SQL.NUMBER_TABLE;
BEGIN
   C := DBMS_SQL.OPEN_CURSOR;//打开游标
   DBMS_SQL.PARSE(C, 'SELECT N1 FROM SYSDBA.T1', 1);//解析 SQL 语句
   DBMS_SQL.DEFINE_ARRAY(C, 1, D1,3,1);//定义列，将来可用 COLUMN_VALUE 获取列值
   DBMS_SQL.EXECUTE(C);//执行语句
   DBMS_SQL.FETCH_ROWS(C);//获取结果集
   DBMS_SQL.COLUMN_VALUE(C, 1, D1);//获取列值
   DBMS_SQL.CLOSE_CURSOR(C);//关闭游标
   DBMS_OUTPUT.PUT_LINE(D1(2));//输出 D1 中第二条列值
END;
/


//输出结果:
2.1
```

例 3 绑定参数

```
DECLARE
   C            NUMBER;
   D1           NUMBER;
BEGIN
       D1    :=9.12;
   C := DBMS_SQL.OPEN_CURSOR;//打开游标
   DBMS_SQL.PARSE(C, 'INSERT INTO SYSDBA.T1 VALUES(:C1)','1');//解析 SQL 语句
   DBMS_SQL.BIND_VARIABLE(C,'C1',D1);
   DBMS_SQL.EXECUTE(C);//执行语句
   DBMS_SQL.CLOSE_CURSOR(C);//关闭游标
END;
/


SELECT * FROM T1;
//查询结果:
行号          N1
---------- ----------------------
1             1.1
2              2.1
3              3.2
4              4.3
5              9.12
```

例 4　批量绑定

```
DECLARE
```

```
  C           NUMBER;
  D1          DBMS_SQL.NUMBER_TABLE;
BEGIN
      D1(1)       :=12.3;
      D1(2)       :=6.12;
      D1(3)       :=8.12;
      D1(4)       :=342.12;
  C := DBMS_SQL.OPEN_CURSOR;//打开游标
  DBMS_SQL.PARSE(C, 'INSERT INTO SYSDBA.T1 VALUES(:C1)', 1');//解析 SQL 语句
  DBMS_SQL.BIND_ARRAY(C,'C1',D1);
  DBMS_SQL.EXECUTE(C);//执行语句
  DBMS_SQL.CLOSE_CURSOR(C);//关闭游标
END;
/


SELECT * FROM T1;
//查询结果:
行号          N1
---------- -------------------------
1             1.1
2              2.1
3              3.2
4              4.3
5              9.12
```

6	12.3
7	6.12
8	8.12
9	342.12

18 DBMS_TRANSACTION 包

DBMS_TRANSACTION 包提供获得当前活动事务号的功能。

18.1 相关方法

返回当前活动事务号。

语法如下:

```
FUNCTION LOCAL_TRANSACTION_ID (
    CREATE_TRANSACTION BOOLEAN := FALSE
) RETURN VARCHAR2;
```

参数详解

- CREATE_TRANSACTION

表示如果当前没有活动事务，是否创建一个新的事务。TRUE：创建；FALSE：不创建，返回空。缺省为 FALSE。

18.2 举例说明

使用包内的过程和函数之前，如果还未创建过系统包，请先调用系统过程创建系统包。

```
SP_CREATE_SYSTEM_PACKAGES (1,'DBMS_TRANSACTION');
```

例 查询当前活动事务号。

```
SELECT DBMS_TRANSACTION.LOCAL_TRANSACTION_ID(FALSE);
```

结果如下:

```
6755
```

需要注意的是，该查询语句本身也是活动事务，因此该语句必然返回一个活动事务号。

19 DBMS_STATS 包

优化统计信息描述了数据库中的对象细节。查询优化使用这些信息选择最合适的执行计划。使用 DBMS_STATS 包来收集统计、删除信息，将收集的统计信息记录在数据字典中。

19.1 数据类型

DBMS_STATS 包中涉及到类型。如下统一说明。

1. OBJECTELEM 类型

OBJECTELEM 类型是 DBMS_STATS 包内专有类型。用户不能引用和改变该记录的内容。OBJECTELEM 记录类型定义如下:

```
TYPE OBJECTELEM IS RECORD (
    OWNNAME         VARCHAR(128),
    OBJTYPE         VARCHAR(6),
    OBJNAME         VARCHAR(128),
    PARTNAME        VARCHAR(128),
    SUBPARTNAME     VARCHAR(128)
);
```

参数详解

- OWNNAME

  模式名。

- OBJTYPE

  对象类型，TABLE 或 INDEX。

- OBJNAME

  对象（表或索引）名称，区分大小写。

- PARTNAME

  分区名称，区分大小写。

- SUBPARTNAME

  子分区名称，区分大小写。

  例 OBJECTTAB 为 OBJECTELEM 类型的索引表:

```
TYPE OBJECTTAB IS TABLE OF OBJECTELEM INDEX BY INT;
```

2. STATREC 类型

STATREC 类型是 DBMS_STATS 包内专有类型。用户不能引用和改变该记录的内容。

STATREC 记录类型定义如下:

```
TYPE STATREC IS RECORD (
    EPC             NUMBER,
    MINVAL          VARBINARY(8188),
    MAXVAL          VARBINARY(8188),
    EKHCNTS         NUMARRAY,
    NOVALS          NUMARRAY,
    CHVALS          CHARARRAY,
    DTVALS          DATEARRAY,
    EHCNTS          NUMARRAY,
    EDCNTS          NUMARRAY
);
```

参数详解

- EPC

  ENDPOINT 的个数（即桶数）。

- MINVAL

最小值。

- MAXVAL

  最大值。

- EKHCNTS

  对应 ENDPOINT_KEYHEIGHT，每个 ENDPOINT 的重复次数。

- NOVALS

  对应 ENDPOINT_VALUE，具体表示数值类型。

- CHVALS

  对应 ENDPOINT_VALUE，具体表示字符类型。

- DTVALS

  对应 ENDPOINT_VALUE，具体表示日期类型。

- EHCNTS

  对应 ENDPOINT_HEIGHT，小于每个 ENDPOINT 的数据个数。

- EDCNTS

  对应 ENDPOINT_DISTINCT， 小于每个 ENDPOINT 的唯一值个数。

例 定义一个 STATREC 类型的变量：

```
V_SREC                    DBMS_STATS.STATREC;
```

3. NUMARRAY 类型

NUMARRAY 类型是 DBMS_STATS 包提供的数据类型。自定义列统计信息直方图时，列类型为数字类型的都通过 NUMARRAY 存储，包括 INT、DOUBLE、FLOAT、DEC 等所有支持收集统计信息的数字类型。

```
TYPE NUMARRAY             IS VARRAY(2050) OF NUMBER;
```

例 定义一个 NUMARRAY 类型的变量：

```
ARRAY_COL_VAL             DBMS_STATS.NUMARRAY;
```

4. CHARARRAY 类型

CHARARRAY 类型是 DBMS_STATS 包提供的数据类型。自定义列统计信息直方图时，列类型为字符类型都通过 CHARARRAY 存储，包括 CHAR、VARCHAR、BINARY 等所有支持收集统计信息的字符类型。

```
TYPE CHARARRAY          IS VARRAY(2050) OF VARCHAR2(4000);
```

例 定义一个 CHARARRAY 类型的变量：

```
ARRAY_COL_VAL           DBMS_STATS.CHARARRAY ;
```

5. DATEARRAY 类型

DATEARRAY 类型是 DBMS_STATS 包提供的数据类型。自定义列统计信息直方图时，列类型为日期类型都通过 DATEARRAY 存储，包括 DATETIME、DATE、DATETIME WITH TIME ZONE、TIMESTAMP、TIME、TIME WITH TIME ZONE、INTERVAL YEAR TO MONTH、INTERVAL DAY TO SECOND。

```
TYPE DATEARRAY          IS VARRAY(2050) OF VARCHAR2(100);
```

例 定义一个 DATEARRAY 类型的变量：

```
ARRAY_COL_VAL           DBMS_STATS.DATEARRAY;
```

19.2 相关方法

DBMS_STATS 包中包含的过程和函数如下详细介绍。其中，使用表、索引统计信息相关的过程和方法需要具有表查询权限；使用列统计信息相关的过程和方法需要具有列查询权限。

1. COLUMN_STATS_SHOW

根据模式名，表名和列名获得该列的统计信息。用于经过 GATHER_TABLE_STATS 或 GATHER_SCHEMA_STATS 收集之后展示。返回两个结果集：一个是列的统计信息；另一

个是直方图的统计信息。

语法如下:

```
PROCEDURE COLUMN_STATS_SHOW (
    OWNNAME     IN      VARCHAR(128),
    TABNAME     IN      VARCHAR(128),
    COLNAME     IN      VARCHAR(128)
);
```

列的统计信息和直方图的统计信息，格式分别如下表 21.1、表 21.2:

表 19.1 列统计信息

名称	解释
NUM_DISTINCT	不同列值的个数
LOW_VALUE	列最小值
HIGH_VALUE	列最大值
NUM_NULLS	空值的个数
NUM_BUCKETS	直方图桶的个数
SAMPLE_SIZE	样本容量
HISTOGRAM	直方图的类型

表 19.2 直方图的统计信息

名称	解释
OWNER	模式名
TABLE_NAME	表名
COLUMN_NAME	列名
HISTOGRAM	直方图类型

ENDPOINT_VALUE	样本值
ENDPOINT_HEIGHT	对于频率直方图，样本值的个数；对于等高直方图，小于样本值大于前一个样本值的个数。
ENDPOINT_KEYHEIGHT	对于频率直方图无效；对于等高直方图，样本值的个数。
ENDPOINT_DISTINCT	对于频率直方图无效；对于等高直方图，小于样本值大于前一个样本值之间不同样本的个数。

参数详解

- OWNNAME

  模式名，区分大小写。

- TABNAME

  表名，区分大小写。

- COLNAME

  列名，区分大小写。

2. TABLE_STATS_SHOW

根据模式名，表名获得该表的统计信息。用于经过 GATHER_TABLE_STATS 或 GATHER_SCHEMA_STATS 收集之后展示。

语法如下:

```
PROCEDURE TABLE_STATS_SHOW (
    OWNNAME         IN      VARCHAR(128),
    TABNAME         IN      VARCHAR(128)
);
```

表的统计信息，格式如下表 21.3:

表 19.3 表统计信息

名称	解释
NUM_ROWS	表的总行数
LEAF_BLOCKS	总的页数
LEAF_USED_BLOCKS	已经使用的页数

参数详解

- OWNNAME

  模式名，区分大小写。

- TABNAME

  表名，区分大小写。

3. INDEX_STATS_SHOW

根据模式名，索引名获得该索引的统计信息。用于经过 GATHER_TABLE_STATS、GATHER_INDEX_STATS 或 GATHER_SCHEMA_STATS 收集之后展示。返回两个结果集：一个是索引的统计信息；另一个是直方图的统计信息。

语法如下：

```
PROCEDURE INDEX_STATS_SHOW (
    OWNNAME         IN      VARCHAR(128),
    INDEXNAME       IN      VARCHAR(128)
);
```

索引的统计信息和直方图的统计信息，格式分别如下表 21.4、表 21.5：

表 19.4 索引统计信息

名称	解释

BLEVEL	B_Tree 的层次
LEAF_BLOCKS	页数
DISTINCT_KEYS	不同样本的个数
CLUSTERING_FACTOR	聚集因子，表示索引的数据分布与物理分布之间的关系，暂不支持
NUM_ROWS	行数
SAMPLE_SIZE	样本容量

表 19.5 直方图的统计信息

名称	解释
OWNER	模式名
NAME	索引名
COLUMN_NAME	列名
HISTOGRAM	直方图类型
ENDPOINT_VALUE	样本值
ENDPOINT_HEIGHT	对于频率直方图，样本值的个数；对于等高直方图，小于样本值大于前一个样本值的个数。
ENDPOINT_KEYHEIGHT	对于频率直方图无效；对于等高直方图，样本值的个数。
ENDPOINT_DISTINCT	对于频率直方图无效；对于等高直方图，小于样本值大于前一个样本值之间不同样本的个数。

参数详解

- OWNNAME

  模式名，区分大小写。

- INDEXNAME

  索引名，区分大小写。

4. GATHER_TABLE_STATS

根据设定的参数，收集表、表中的列和表上的索引的统计信息。其中，对于表，只搜集表的总行数、总的页数、已经使用的页数等基本信息。本操作会提交当前事务。

语法如下：

```
PROCEDURE GATHER_TABLE_STATS (
    OWNNAME             VARCHAR(128),
    TABNAME             VARCHAR(128),
    PARTNAME            VARCHAR(128)    DEFAULT NULL,
    ESTIMATE_PERCENT DOUBLE         DEFAULT
TO_ESTIMATE_PERCENT_TYPE(GET_PREFS('ESTIMATE_PERCENT')),
    BLOCK_SAMPLE    BOOLEAN     DEFAULT FALSE,
    METHOD_OPT      VARCHAR     DEFAULT GET_PREFS('METHOD_OPT'),
    DEGREE          INT         DEFAULT TO_DEGREE_TYPE(GET_PREFS('DEGREE')),
    GRANULARITY     VARCHAR     DEFAULT GET_PREFS('GRANULARITY'),
    CASCADE         BOOLEAN     DEFAULT TO_CASCADE_TYPE(GET_PREFS('CASCADE')),
    STATTAB         VARCHAR     DEFAULT NULL,
    STATID          VARCHAR     DEFAULT NULL,
    STATOWN             VARCHAR     DEFAULT NULL,
    NO_INVALIDATE   BOOLEAN     DEFAULT
TO_NO_INVALIDATE_TYPE(GET_PREFS('NO_INVALIDATE')),
    FORCE           BOOLEAN     DEFAULT FALSE
    MERGE_STAT      BOOLEAN   DEFAULT FALSE
);
```

参数详解

- OWNNAME

  模式名，区分大小写。

- TABNAME

  表名，区分大小写。

- PARTNAME

  分区表名，缺省为 NULL，区分大小写。

- ESTIMATE_PERCENT

  收集的百分比，范围为 0~100，区间(0,0.000001)内的值，按 0.000001 取值计算。默认系统自定。

- BLOCK_SAMPLE

  TRUE 表示以数据页为单位采样；FALSE 表示以数据行为单位采样。缺省为 FALSE。

- METHOD_OPT

  控制列的统计信息集合和直方图的创建的格式，缺省为 FOR ALL COLUMNS SIZE DEFAULT。其中 BLOB、IMAGE、LONGVARBINARY、CLOB、TEXT、LONGVARCHAR、自定义类型列和空间类型列等列类型不能被收集。使用 FOR COLUMNS 语法时，后跟可选填的<size_clause>部分和必填的<column_name>部分，多个<column_name>之间必须用“,”分隔。格式选项如下：

```
FOR ALL [INDEXED | HIDDEN] COLUMNS [<size_clause>]
<size_clause>::= SIZE { integer | REPEAT | AUTO | SKEWONLY}
或者
FOR COLUMNS { <size clause> | [<size clause>] <column_name> [<size_clause>]} {,
<column_name> [<size_clause>]}
```

  各参数解释如下：

- INDEXED | HIDDEN　表示只统计索引或者隐藏的列，缺省为都统计。
- SIZE integer　表示直方图的桶数。其中的 integer 必须指定某个具体的数值，取值范围 1~10000。
- SIZE REPEAT　只统计已经有直方图的列。
- SIZE AUTO　根据数据分布和工作量自动决定统计直方图的列。设置为默认取值 FOR ALL COLUMNS SIZE AUTO 时，DM 会根据数据分布和工作量自动决定统计直方图的列。
- SIZE SKEWONLY　根据数据分布决定统计直方图的列。
- SIZE DEFAULT　表示统计列的直方图且由系统自定义桶数。

- DEGREE

  收集的并行度，缺省为 1。

- GRANULARITY

  收集的粒度，缺省为 AUTO；GRANULARITY 可选参数如下：AUTO | DEFAULT | ALL | PARTITION | SUBPARTITION | GLOBAL | GLOBAL AND PARTITION。

  各参数解释如下：

  - ALL　收集全部的统计信息（SUBPARTITION，PARTITION 和 GLOBAL）。
  - AUTO　缺省值，根据分区的类型来决定如何收集。
  - DEFAULT　和 AUTO 功能相同。
  - GLOBAL　收集 GLOBAL 表的统计信息。
  - GLOBAL AND PARTITION　收集 GLOBAL 和 PARTITION 表的统计信息。
  - PARTITION　收集 PARTITION 表的统计信息。
  - SUBPARTITION 收集 SUBPARTITION 表的统计信息。当子分区表个数超过 50 时，因为采用跳跃采样，所以统计信息会有误差。即使采样率为 100，也还是会有误差。可以通过减少统计层次的方式降低这种误差，即将 GRANULARITY=>'SUBPARTITION'修改

成 GRANULARITY=>'GLOBAL AND PARTITION'或 GRANULARITY=>'GLOBAL'。

- CASCADE

  是否收集索引信息。TRUE 是；FALSE 否，缺省为 TRUE。

- STATTAB

  统计信息存放的表，缺省为 NULL。

- STATID

  统计信息的 ID，缺省为 NULL。

- STATOWN

  统计信息的模式，缺省为 NULL。

- NO_INVALIDATE

  保留参数，是否让依赖游标失效，缺省为 TRUE。NO_INVALIDATE 可选参数如下:TRUE | FALSE | DBMS_STATS.AUTO_INVALIDATE。

  各参数解释如下:

  ◆ TRUE 游标不失效，原有的执行计划保持原状态。

  ◆ FALSE 游标失效，移除原有的相关执行计划。

  ◆ DBMS_STATS.AUTO_INVALIDATE 和 TRUE 用法相同。

- FORCE

  是否忽略统计信息锁，强制收集统计信息，缺省为 FALSE；为 TRUE 表示被锁住也强制收集统计信息，为 FALSE 表示没有被锁住才能收集统计信息。

- MERGE_STAT

  为分区子表收集统计信息后，是否级联更新祖先对应的统计信息，TRUE 是，FALSE 否；缺省为 FALSE。

5. GATHER_INDEX_STATS

根据设定的参数，收集索引的统计信息。本操作会提交当前事务。

语法如下：

```
PROCEDURE GATHER_INDEX_STATS (
    OWNNAME                 VARCHAR(128),
    INDNAME                 VARCHAR(128),
    PARTNAME                VARCHAR(128)    DEFAULT NULL,
    ESTIMATE_PERCENT DOUBLE  DEFAULT
TO_ESTIMATE_PERCENT_TYPE(GET_PREFS('ESTIMATE_PERCENT')),
    STATTAB                 VARCHARDEFAULT NULL,
    STATID                  VARCHARDEFAULT NULL,
    STATOWN                 VARCHARDEFAULT NULL,
    DEGREE                  INT         DEFAULT TO_DEGREE_TYPE(GET_PREFS('DEGREE')),
    GRANULARITY             VARCHARDEFAULT GET_PREFS('GRANULARITY'),
    NO_INVALIDATE           BOOLEAN   DEFAULT
TO_NO_INVALIDATE_TYPE(GET_PREFS('NO_INVALIDATE')),
    FORCE                   BOOLEAN DEFAULT FALSE
);
```

参数详解

- OWNNAME

  模式名，区分大小写。

- INDNAME

  索引名，区分大小写。

- PARTNAME

分区索引名，缺省为 NULL，区分大小写。

- ESTIMATE_PERCENT

  收集的百分比，范围为 0~100，区间(0,0.000001)内的值，按 0.000001 取值计算。默认系统自定。

- STATTAB

  保留参数，统计信息存放的表，缺省为 NULL。

- STATID

  保留参数，统计信息的 ID，缺省为 NULL。

- STATOWN

  保留参数，统计信息的模式，缺省为 NULL。

- DEGREE

  收集的并行度，缺省为 1。

- GRANULARITY

  收集的粒度，缺省为 ALL。

- NO_INVALIDATE

  是否让依赖游标失效，缺省为 TRUE。具体用法请参考 GATHER_TABLE_STATS 方法的 NO_INVALIDATE 参数。

- FORCE

  是否忽略统计信息锁，强制收集统计信息，缺省为 FALSE；为 TRUE 表示被锁住也强制收集统计信息，为 FALSE 表示没有被锁住才能收集统计信息

6. GATHER_SCHEMA_STATS

收集模式下对象的统计信息。只收集有查询权限的对象，对于没有查询权限的对象会直接跳过。本操作会提交当前事务。

语法如下：

```
PROCEDURE GATHER_SCHEMA_STATS (
    OWNNAME                 VARCHAR(128),
    ESTIMATE_PERCENT DOUBLE  DEFAULT
TO_ESTIMATE_PERCENT_TYPE(GET_PREFS('ESTIMATE_PERCENT')),
    BLOCK_SAMPLE            BOOLEAN DEFAULT FALSE,
    METHOD_OPT              VARCHARDEFAULT GET_PREFS('METHOD_OPT'),
    DEGREE                  INT       DEFAULT TO_DEGREE_TYPE(GET_PREFS('DEGREE')),
    GRANULARITY             VARCHARDEFAULT GET_PREFS('GRANULARITY'),
    CASCADE                 BOOLEAN DEFAULT TO_CASCADE_TYPE(GET_PREFS('CASCADE')),
    STATTAB                 VARCHARDEFAULT NULL,
    STATID                  VARCHARDEFAULT NULL,
    OPTIONS                 VARCHARDEFAULT 'GATHER',
    OBJLIST        OUT      OBJECTTAB DEFAULT NULL,
    STATOWN                 VARCHARDEFAULT NULL,
    NO_INVALIDATE           BOOLEAN DEFAULT
TO_NO_INVALIDATE_TYPE(GET_PREFS('NO_INVALIDATE')),
    FORCE                   BOOLEAN DEFAULT FALSE,
    OBJ_FILTER_LIST         OBJECTTAB DEFAULT NULL
);
```

参数详解

- OWNNAME

  模式名，区分大小写。

- ESTIMATE_PERCENT

  收集的百分比，范围为 0~100，区间(0,0.000001)内的值，按 0.000001 取值计算。默认系统自定。

- BLOCK_SAMPLE

  TRUE 表示以数据页为单位采样；FALSE 表示以数据行为单位采样。缺省为 FALSE。

- METHOD_OPT

  控制列的统计信息集合和直方图的创建；缺省为 FOR ALL COLUMNS SIZE DEFAULT；只支持其中一种格式：

```
FOR ALL [INDEXED | HIDDEN] COLUMNS [<size_clause>]
<size_clause>::= SIZE {integer | REPEAT | AUTO | SKEWONLY}
```

  各参数解释如下：

  - INDEXED | HIDDEN　表示只统计索引或者隐藏的列，缺省为都统计。
  - SIZE integer　表示直方图的桶数。其中的 integer 必须指定某个具体的数值，取值范围 1~10000。
  - SIZE REPEAT　只统计已经有直方图的列。
  - SIZE AUTO　根据数据分布和工作量自动决定统计直方图的列。设置为默认取值 FOR ALL COLUMNS SIZE AUTO 时，DM 会根据数据分布和工作量自动决定统计直方图的列。
  - SIZE SKEWONLY　根据数据分布决定统计直方图的列。
  - SIZE DEFAULT　表示统计列的直方图且由系统自定义桶数。

- DEGREE

  收集的并行度，缺省为 1。

- GRANULARITY

  收集的粒度，缺省为 ALL。

- CASCADE

  是否收集索引信息，TRUE 或 FALSE。缺省为 TRUE。

- STATTAB

  统计信息存放的表，缺省为 NULL。

- STATID

  统计信息的 ID，缺省为 NULL。

- OPTIONS

  控制收集的列，缺省为 GATHER；选项如下：GATHER|GATHER AUTO|GATHER STALE|GATHER EMPTY|LIST AUTO|LIST STALE|LIST EMPTY。各选项解释如下:

  - GATHER：收集模式下所有对象的统计信息。
  - GATHER AUTO：自动收集需要的统计信息。系统隐含的决定哪些对象需要新的统计信息,以及怎样收集这些统计信息。此时,只有 OWNNAME,STATTAB，STATID,OBJLIST AND STATOWN有效，返回收集统计信息的对象。
  - GATHER STALE：对旧的对象收集统计信息。返回找到的旧的对象。
  - GATHER EMPTY：收集没有统计信息对象的统计信息。返回这些对象。
  - LIST AUTO：返回GATHER AUTO方式处理的对象。
  - LIST STALE：返回旧的对象信息。
  - LIST EMPTY：返回没有统计信息的对象。

- OBJLIST

  返回 OPTIONS 选项对应的链表，缺省为 NULL。

- STATOWN

  统计信息的模式，缺省为 NULL。

- NO_INVALIDATE

  是否让依赖游标失效，缺省为 TRUE。具体用法请参考 GATHER_TABLE_STATS 方法的

NO_INVALIDATE 参数。

- FORCE

  是否忽略统计信息锁，强制收集统计信息，缺省为 FALSE。为 TRUE 表示被锁住也强制收集统计信息，为 FALSE 表示没有被锁住才能收集统计信息。

- OBJ_FILTER_LIST

  存放过滤条件的模式名、表名和子表名，缺省为 NULL。

7. DELETE_TABLE_STATS

根据设定参数，删除与表相关对象的统计信息。本操作会提交当前事务。

语法如下:

```
PROCEDURE DELETE_TABLE_STATS (
    OWNNAME             VARCHAR(128),
    TABNAME             VARCHAR(128),
    PARTNAME            VARCHAR(128)    DEFAULT NULL,
    STATTAB             VARCHAR         DEFAULT NULL,
    STATID              VARCHAR         DEFAULT NULL,
    CASCADE_PARTS       BOOLEAN         DEFAULT TRUE,
    CASCADE_COLUMNS     BOOLEAN         DEFAULT TRUE,
    CASCADE_INDEXES     BOOLEAN         DEFAULT TRUE,
    STATOWN             VARCHAR         DEFAULT NULL,
    NO_INVALIDATE       BOOLEAN         DEFAULT
TO_NO_INVALIDATE_TYPE(GET_PREFS('NO_INVALIDATE')),
    FORCE               BOOLEAN         DEFAULT FALSE
```

);

参数详解

- OWNNAME

  模式名，区分大小写。

- TABNAME

  表名，区分大小写。

- PARTNAME

  分区表名，缺省为 NULL，区分大小写。

- STATTAB

  保留参数，统计信息存放的表，缺省为 NULL。

- STATID

  保留参数，统计信息的 ID，缺省为 NULL。

- CASCADE_PARTS

  是否级联删除分区表信息，缺省为 TRUE。

- CASCADE_COLUMNS

  是否级联删除表中列的信息，取值 TRUE 或 FALSE。缺省为 TRUE。

- CASCADE_INDEXES

  是否级联删除表的索引信息，取值 TRUE 或 FALSE。缺省为 TRUE。

- STATOWN

  保留参数，统计信息的模式，缺省为 NULL。

- NO_INVALIDATE

  是否让依赖游标失效，缺省为 TRUE。具体用法请参考 GATHER_TABLE_STATS 方法的 NO_INVALIDATE 参数。

- FORCE

是否忽略统计信息锁，强制删除统计信息，缺省为 FALSE；为 TRUE 表示被锁住也强制删除统计信息，为 FALSE 表示没有被锁住才能删除统计信息。

8. DELETE_SCHEMA_STATS

根据设定参数，删除模式下对象的统计信息。本操作会提交当前事务。

语法如下:

```
PROCEDURE DELETE_SCHEMA_STATS (
    OWNNAME             VARCHAR(128),
    STATTAB             VARCHAR         DEFAULT NULL,
    STATID              VARCHAR         DEFAULT NULL,
    STATOWN             VARCHAR         DEFAULT NULL,
    NO_INVALIDATE   BOOLEAN     DEFAULT
TO_NO_INVALIDATE_TYPE(GET_PREFS('NO_INVALIDATE')),
    FORCE               BOOLEAN         DEFAULT FALSE
);
```

参数详解

- OWNNAME

  模式名，区分大小写。

- STATTAB

  保留参数，统计信息存放的表，缺省为 NULL。

- STATID

  保留参数，统计信息的 ID，缺省为 NULL。

- STATOWN

保留参数，统计信息的模式，缺省为 NULL。

- NO_INVALIDATE

是否让依赖游标失效，缺省为 TRUE。具体用法请参考 GATHER_TABLE_STATS 方法的 NO_INVALIDATE 参数。

- FORCE

是否忽略统计信息锁，强制删除统计信息，缺省为 FALSE。为 TRUE 表示被锁住也强制删除统计信息，为 FALSE 表示没有被锁住才能删除统计信息。

9. DELETE_INDEX_STATS

根据设定参数，删除索引的统计信息。本操作会提交当前事务。

语法如下：

```
PROCEDURE DELETE_INDEX_STATS (
    OWNNAME             VARCHAR(128),
    INDNAME             VARCHAR(128),
    PARTNAME            VARCHAR(128)    DEFAULT NULL,
    STATTAB             VARCHAR             DEFAULT NULL,
    STATID              VARCHAR             DEFAULT NULL,
    CASCADE_PARTS       BOOLEAN             DEFAULT TRUE,
    STATOWN             VARCHAR             DEFAULT NULL,
    NO_INVALIDATE       BOOLEAN             DEFAULT
    TO_NO_INVALIDATE_TYPE(GET_PREFS('NO_INVALIDATE')),
    FORCE               BOOLEAN             DEFAULT FALSE
);
```

参数详解

- OWNNAME

  模式名，区分大小写。

- INDNAME

  索引名，区分大小写。

- PARTNAME

  分区索引名，缺省为 NULL，区分大小写。

- STATTAB

  保留参数，统计信息存放的表，缺省为 NULL。

- STATID

  保留参数，统计信息的 ID，缺省为 NULL。

- CASCADE_PARTS

  是否级联删除分区表信息，TRUE 或 FALSE。缺省为 TRUE。

- STATOWN

  保留参数，统计信息的模式，缺省为 NULL。

- NO_INVALIDATE

  是否让依赖游标失效，缺省为 TRUE。具体用法请参考 GATHER_TABLE_STATS 方法的 NO_INVALIDATE 参数。

- FORCE

  是否忽略统计信息锁，强制删除统计信息，缺省为 FALSE；为 TRUE 表示被锁住也强制删除统计信息，为 FALSE 表示没有被锁住才能删除统计信息。

10. DELETE_COLUMN_STATS

根据设定参数，删除列的统计信息。本操作会提交当前事务。

语法如下：

```
PROCEDURE DELETE_COLUMN_STATS (
    OWNNAME             VARCHAR(128),
    TABNAME             VARCHAR(128),
    COLNAME             VARCHAR(128),
    PARTNAME            VARCHAR(128)    DEFAULT NULL,
    STATTAB             VARCHAR             DEFAULT NULL,
    STATID              VARCHAR             DEFAULT NULL,
    CASCADE_PARTS       BOOLEAN             DEFAULT TRUE,
    STATOWN             VARCHAR             DEFAULT NULL,
    NO_INVALIDATE       BOOLEAN             DEFAULT
TO_NO_INVALIDATE_TYPE(GET_PREFS('NO_INVALIDATE')),
    FORCE               BOOLEAN             DEFAULT FALSE,
    COL_STAT_TYPE       VARCHAR             DEFAULT 'ALL'
);
```

参数详解

- OWNNAME

  模式名，区分大小写。

- TABNAME

  表名，区分大小写。

- COLNAME

  列名，区分大小写。

- PARTNAME

  分区表名，缺省为 NULL，区分大小写。

- STATTAB

  保留参数，统计信息存放的表，缺省为 NULL。

- STATID

  保留参数，统计信息的 ID，缺省为 NULL。

- CASCADE_PARTS

  是否级联删除分区表信息，TRUE 或 FALSE。缺省为 TRUE。

- STATOWN

  保留参数，统计信息的模式，缺省为 NULL。

- NO_INVALIDATE

  是否让依赖游标失效，缺省为 TRUE。具体用法请参考 GATHER_TABLE_STATS 方法的 NO_INVALIDATE 参数。

- FORCE

  是否忽略统计信息锁，强制删除统计信息，缺省为 FALSE。为 TRUE 表示被锁住也强制删除统计信息，为 FALSE 模式没有被锁住才能删除统计信息。

- COL_STAT_TYPE

  保留参数，是否只删除直方图信息，缺省为 ALL。

11. UPDATE_ALL_STATS

    更新已有的统计信息。本操作会提交当前事务。

    语法如下：

```
PROCEDURE UPDATE_ALL_STATS ();
```

12. CONVERT_RAW_VALUE

把二进制流转换为指定类型。

语法如下:

```
PROCEDURE CONVERT_RAW_VALUE (
    I_RAW                   VARBINARY(8188),
    M_N         OUT         NUMBER
);
PROCEDURE CONVERT_RAW_VALUE (
    I_RAW                   VARBINARY(8188),
    M_N         OUT         DATETIME
);
PROCEDURE CONVERT_RAW_VALUE (
    I_RAW                   VARBINARY(8188),
    M_N         OUT         VARCHAR(8188)
);
PROCEDURE CONVERT_RAW_VALUE (
    I_RAW                   VARBINARY(8188),
    M_N         OUT         DOUBLE
);
```

参数详解

- I_RAW

输入参数，VARBINARY 类型的最大或最小值，期望来源于 SYSSTATS 表的 V_MAX 或

V_MIN 列。

- M_N

  输出参数，由 I_RAW 转换而来，支持 NUMBER、DATETIME、VARCHAR(8188)、DOUBLE 类型。

13. CREATE_STAT_TABLE

根据用户指定的名称创建一个增加了前缀的临时统计信息表，用于保存待导出的统计信息。统计信息保存到该表中后，可以使用 DM 的数据导入导出工具进行跨实例导入导出。MPP 环境下暂不支持统计信息导入导出功能。本操作会提交当前事务。DMDPC 下 GLOBAL_TEMPORARY=TRUE 时要求表空间是 TEMP_TS。

统计信息表属于内建表，为跟普通用户表区分，系统内部会为表名增加前缀"STAT$_"。例如用户指定统计信息表名为 STATTAB，查询该统计信息表应该使用 STAT$_STATTAB。对于用户，统计信息表不支持使用 CREATE、DROP、ALTER、TRUNCATE、INSERT、DELETE、UPDATE、GRANT 操作，会报错。统计信息表支持通过包方法 DROP_STAT_TABLE 进行删除。

语法如下：

```
PROCEDURE CREATE_STAT_TABLE (
    STATOWN             VARCHAR(128),
    STATTAB             VARCHAR(128),
    TABLESPACE          VARCHAR(128)    DEFAULT NULL,
    GLOBAL_TEMPORARY    BOOLEAN         DEFAULT FALSE
);
```

参数详解

- STATOWN

  统计信息表的模式名，区分大小写。

- STATTAB

  统计信息表名，区分大小写，系统实际创建的表会增加前缀。

- TABLESPACE

  表空间名，缺省为 NULL，区分大小写。

- GLOBAL_TEMPORARY

  是否创建为会话级的全局临时表。

14. DROP_STAT_TABLE

    删除统计信息表。本操作会提交当前事务。

    语法如下:

```
PROCEDURE DROP_STAT_TABLE (
    STATOWN                 VARCHAR(128),
    STATTAB             VARCHAR(128)
);
```

    参数详解

- STATOWN

  统计信息表的模式名，区分大小写。

- STATTAB

  创建时指定的统计信息表名，区分大小写。

15. EXPORT_TABLE_STATS

把目标表的统计信息导出到指定统计信息表中。本操作会提交当前事务。

语法如下:

```
PROCEDURE EXPORT_TABLE_STATS(
    OWNNAME          VARCHAR(128),
    TABNAME          VARCHAR(128),
    PARTNAME         VARCHAR(128)    DEFAULT NULL,
    STATTAB          VARCHAR(128),
    STATID           VARCHAR(128)    DEFAULT '',
    CASCADE          BOOLEAN         DEFAULT TRUE,
    STATOWN          VARCHAR(128)    DEFAULT NULL,
    STAT_CATEGORY    VARCHAR(128)    DEFAULT NULL
);
```

参数详解

- OWNNAME

  目标表的模式名，区分大小写。

- TABNAME

  目标表名，区分大小写。

- PARTNAME

  目标表分区名，如果不指定分区，则一起导出所有子表的统计信息，缺省为 NULL。

- STATTAB

  统计信息表名，区分大小写。

- STATID

由用户指定的统计信息标识名，缺省为空字符串。

- CASCADE

  是否连列和索引的统计信息一起导出，缺省为 TRUE。

- STATOWN

  统计信息表的模式名，区分大小写，缺省为 NULL。

- STAT_CATEGORY

  保留参数，无实际意义。

16. IMPORT_TABLE_STATS

把统计信息表中的统计信息导入到目标表中。本操作会提交当前事务。

语法如下：

```
PROCEDURE IMPORT_TABLE_STATS(
    OWNNAME         VARCHAR(128),
    TABNAME         VARCHAR(128),
    PARTNAME        VARCHAR(128)    DEFAULT NULL,
    STATTAB         VARCHAR(128),
    STATID          VARCHAR(128)    DEFAULT '',
    CASCADE         BOOLEAN             DEFAULT TRUE,
    STATOWN             VARCHAR(128)    DEFAULT NULL,
    NO_INVALIDATE       BOOLEAN         DEFAULT NULL,
    FORCE           BOOLEAN         DEFAULT FALSE,
    STAT_CATEGORY       VARCHAR(128)    DEFAULT NULL
);
```

参数详解

- OWNNAME

  目标表的模式名，区分大小写。

- TABNAME

  目标表名，区分大小写。

- PARTNAME

  目标表分区名，如果不指定分区，则一起导入所有子表的统计信息，缺省为 NULL。

- STATTAB

  统计信息表名，区分大小写。

- STATID

  由用户指定的统计信息标识名，缺省为空字符串。

- CASCADE

  是否连列和索引的统计信息一起导入，缺省为 TRUE

- STATOWN

  统计信息表的模式名，区分大小写，缺省为 NULL。

- NO_INVALIDATE

  是否让依赖游标失效，缺省为 TRUE。具体用法请参考 GATHER_TABLE_STATS 方法的 NO_INVALIDATE 参数。

- FORCE

  是否忽略统计信息锁, 强制把统计信息表中的统计信息导入到目标表中, 缺省为 FALSE。

  为 TRUE 表示目标表被锁住也强制导入，为 FALSE 表示目标表没有被锁住才能导入。

- STAT_CATEGORY

  保留参数，无实际意义。

使用说明:

统计信息导入时，导入对象完全由对象名称来确定，如果存在由系统根据 ID 自动创建的对象，例如在分区表主表上建立索引时，子表上自动建立的索引将根据索引 ID 来命名，统计信息表里存储的索引名称可能与新建立的分区表子表上的索引名称不一致，此时名称不一致的索引统计信息将无法导入。

17. SET_TABLE_PREFS

该过程设置普通用户表的静态属性值。本操作会提交当前事务。

语法如下:

```
PROCEDURE SET_TABLE_PREFS (
    OWNNAME         VARCHAR(128),
    TABNAME     VARCHAR(128),
    PPNAME      VARCHAR(128),
    PVALUE      VARCHAR(128)
);
```

参数详解

- OWNNAME

  模式名，不区分大小写。

- TABNAME

  表名，不区分大小写。

- PPNAME

  属性名，不区分大小写。可以取 STALE_PERCENT 、ESTIMATE_PERCENT、METHOD_OPT 、 DEGREE 、 GRANULARITY 、 CASCADE 、 NO_INVALIDATE 、PREFERENCE_OVERRIDES_PARAMETER 、 MONITOR_MODIFICATIONS_FLAG 、MONITOR_MODIFICATIONS_RM_PLN_FLAG 和

MONITOR_MODIFICATIONS_REGATHER_COL。

STALE_PERCENT 决定表的统计信息过时且需要重新收集统计信息时表的行数百分比。有效取值范围 0~100，缺省值是 10，即一张表有超过 10%的数据改变则可认为统计信息过时。将该值置为 0 时，系统会自动将该表的 STALE_PERCENT 属性值置为缺省值 10。特别的，当 INI 参数 AUTO_STAT_OBJ=0 或 2 时，对表 T 设置 STALE_PERCENT=0，表示不再监控表 T，将其从系统表 SYSSTATTABLEIDU 中移除（设置的动作即表示将表 T 从系统表 SYSSTATTABLEIDU 中移除，尽管此时查询 STALE_PERCENT 值仍为缺省值 10）；对表 T 设置 STALE_PERCENT>0，表示要监控表 T，将其加入系统表 SYSSTATTABLEIDU，并将系统表对应记录的 MONITOR_FLAG 列置为 2。当 INI 参数 AUTO_STAT_OBJ=1（表示对所有表进行监控）时，对表 T 设置 STALE_PERCENT=0 仍会监控表 T，且当系统表 SYSSTATTABLEIDU 存在该表信息时会将 MONITOR_FLAG 列置为 1，若不存在则当该表发生数据变动时将该表加入系统表 SYSSTATTABLEIDU 中（进行收集统计信息操作后可以查询到），并将 MONITOR_FLAG 列置为 1；对表 T 设置 STALE_PERCENT>0 表示要监控表 T，将其加入系统表 SYSSTATTABLEIDU，并将 MONITOR_FLAG 列置为 2。

PREFERENCE_OVERRIDES_PARAMETER 决定是否用静态属性值覆盖参数的输入值。为 TRUE 时，忽略参数的输入值，使用对应的属性值；为 FALSE 时，使用参数的输入值。

MONITOR_MODIFICATIONS_FLAG 决定是否对表进行数据变化监控。为 TRUE 时，监控表的数据变化；为 FALSE 时，不监控表的数据变化。缺省值为 TRUE。

MONITOR_MODIFICATIONS_RM_PLN_FLAG 决定对表进行数据变化监控时，当数据变化达到该阈值时，会移除该表相关的缓存计划，同时按照指定方式重新收集该表的统计信息，取值范围[-1,2147483647]。若后续未指定 MONITOR_MODIFICATIONS_REGATHER_COL，则重新对该表所有存在频率直方图统计信息的列和索引收集统计信息；若后续指定了 MONITOR_MODIFICATIONS_REGATHER_COL，则只检测 MONITOR_MODIFICATIONS_REGATHER_COL 监控的列及包含该列的索引，并只对存在统计信息

的重新收集统计信息，同一张表上可指定多个不同的列。必须先指定 MONITOR_MODIFICATIONS_RM_PLN_FLAG，才可以指定 MONITOR_MODIFICATIONS_REGATHER_COL。

其他属性值范围可以参考 GATHER_TABLE_STATS 中的参数描述。

- PVALUE

属性名对应的属性值，不区分大小写。

18. GET_PREFS

该函数返回普通用户表的静态属性值。

语法如下:

```
FUNCTION GET_PREFS(
    PPNAME          VARCHAR(128),
    OWNNAME         VARCHAR(128),
    TABNAME         VARCHAR(128)
)RETURN VARCHAR;
```

参数详解

- PPNAME

属性名，不区分大小写。可以取 STALE_PERCENT、ESTIMATE_PERCENT、METHOD_OPT、DEGREE、GRANULARITY、CASCADE、NO_INVALIDATE、PREFERENCE_OVERRIDES_PARAMETER、MONITOR_MODIFICATIONS_FLAG、MONITOR_MODIFICATIONS_RM_PLN_FLAG 和 MONITOR_MODIFICATIONS_REGATHER_COL。

特别的，当 PPNAME 指定为 MONITOR_MODIFICATIONS_REGATHER_COL 时，返回的数值代表对应列在表中的序号（第一列的序号为 0，第二列的序号是 1，如此类推）；

若存在多列，则返回诸如"0,1,3"这样的形式。

- OWNNAME

  模式名，不区分大小写。

- TABNAME

  表名，不区分大小写。

19. COPY_TABLE_STATS

该过程将基表中指定源分区的统计信息复制到同一基表的指定的目的分区，目的分区原有统计信息将被覆盖。本操作会提交当前事务。

语法如下:

```
PROCEDURE COPY_TABLE_STATS(
    OWNNAME                 VARCHAR(128),
    TABNAME                 VARCHAR(128),
    SRCPARTNAME             VARCHAR(128),
    DSTPARTNAME             VARCHAR(128),
    SCALE_FACTOR            VARCHAR(128) DEFAULT '1',
    FORCE                   BOOLEAN      DEFAULT FALSE
);
```

参数详解

- OWNNAME

  模式名，区分大小写。

- TABNAME

  基表名，区分大小写。

- SRCPARTNAME

  源分区名，区分大小写。

- DSTPARTNAME

  目标分区名，区分大小写。

- SCALE_FACTOR

  保留参数，无实际意义。

- FORCE

  是否忽略统计信息锁，强制将基表中指定源分区的统计信息复制到同一基表的指定的目的分区，缺省为 FALSE。为 TRUE 表示基表被锁住也强制复制，为 FALSE 表示基表没有被锁住才能复制。

20. SET_TABLE_STATS

该过程设置指定表的指定统计信息。本操作会提交当前事务。

语法如下:

```
PROCEDURE SET_TABLE_STATS (
    OWNNAME          VARCHAR2,
    TABNAME          VARCHAR2,
    PARTNAME         VARCHAR2 DEFAULT NULL,
    STATTAB          VARCHAR2 DEFAULT NULL,
    STATID           VARCHAR2 DEFAULT NULL,
    NUMROWS          NUMBER DEFAULT NULL,
    NUMBLKS          NUMBER DEFAULT NULL,
    AVGRLEN          NUMBER DEFAULT NULL,
```

```
    FLAGS               NUMBER DEFAULT NULL,
    STATOWN             VARCHAR2 DEFAULT NULL,
    NO_INVALIDATE   BOOLEAN   DEFAULT TRUE,
    CACHEDBLK           NUMBER   DEFAULT NULL,
    CACHEHIT            NUMBER   DEFAULT NULL,
    FORCE               BOOLEAN DEFAULT FALSE,
    IM_IMCU_COUNT   NUMBER   DEFAULT NULL,
    IM_BLOCK_COUNT NUMBER   DEFAULT NULL,
    SCANRATE            NUMBER   DEFAULT NULL
);
```

参数详解

- OWNNAME

  模式名，区分大小写。

- TABNAME

  表名，区分大小写。

- PARTNAME

  分区表名，区分大小写；缺省为 NULL。

- STATTAB

  自定义统计信息表，区分大小写；缺省为 NULL，统计信息将直接更新到系统视图 SYS.SYSSTATS 中。

- STATID

  统计信息 ID，只有当 STATTAB 不为 NULL 的时候有效；缺省为 NULL。

- NUMROWS

  表/分区行数，缺省为 NULL，为 NULL 时如果待设置统计信息的表

(STATTAB/SYS.SYSSTATS)中存在行数时不做修改，否则插入默认值 2000；若不为 NULL，则更新为指定行数，大小不超过 9223372036854775807(2^63-1)。

- NUMBLKS

  表/分区块数，缺省为 NULL，为 NULL 时如果待设置统计信息的表(STATTAB/SYS.SYSSTATS)中存在块数时不做修改，否则插入默认值 100；若不为 NULL，则更新为指定块数，大小不超过 9223372036854775807(2^63-1)。

- AVGRLEN

  表/分区平均行长，缺省为 NULL，为 NULL 时如果待设置统计信息的表(STATTAB/SYS.SYSSTATS)中存在平均列宽时不做修改，否则插入默认值 100；若不为 NULL，则更新为指定行长，大小不超过 2147483647(2^31-1)。

- FLAGS

  保留参数，仅供内部使用。

- STATOWN

  自定义统计信息表所属的模式名。

- NO_INVALIDATE

  是否让依赖游标失效，缺省为 TRUE。目前仅为保留参数，无实际作用。

- CACHEDBLK

  保留参数，段(索引/表/索引分区/表分区)在 BUFFER CACHE 中的平均块数；缺省为 NULL，仅供内部使用。

- CACHEHIT

  保留参数，段的平均缓存命中率；缺省为 NULL；仅供内部使用。

- FORCE

  是否忽略统计信息锁，强制设置指定表的指定统计信息，缺省为 FALSE；为 TRUE 表示被锁住也设置统计删除信息，为 FALSE 表示没有被锁住才能设置统计信息。

- IM_IMCU_COUNT

  表/分区中内存压缩单元的个数。目前仅为保留参数，无实际作用。

- IM_BLOCK_COUNT

  表/分区中内存块的个数。目前仅为保留参数，无实际作用。

- SCANRATE

  数据库扫描外部表的速率，以 MB/S 为单位。目前仅为保留参数，无实际作用。

21. IMPORT_SCHEMA_STATS

将所有对象的统计信息从指定的 STATTAB 导入到由 OWNNAME 标识的用户中，并将统计信息存入系统表。本操作会提交当前事务。

统计信息导入时，导入对象完全由对象名称来确定，如果存在由系统根据 ID 自动创建的对象，例如在分区表主表上建立索引时，子表上自动建立的索引将根据索引 ID 来命名，统计信息表里存储的索引名称可能与新建立的分区表子表上的索引名称不一致，此时名称不一致的索引统计信息将无法导入。

不支持多列统计信息的导入。

语法如下:

```
PROCEDURE IMPORT_SCHEMA_STATS(
        OWNNAME         VARCHAR(128),
        STATTAB          VARCHAR(128),
        STATID           VARCHAR(128) DEFAULT '',
        STATOWN          VARCHAR(128) DEFAULT NULL,
        NO_INVALIDATE    BOOLEAN to_no_invalidate_type(GET_PREFS('NO_INVALIDATE')),
        FORCE            BOOLEAN         DEFAULT FALSE,
```

```
    STAT_CATEGORY   VARCHAR(128) DEFAULT NULL
);
```

参数详解:

- OWNNAME

  待导入统计信息的 SCHEMA 名，区分大小写；缺省为当前模式。

- STATTAB

  存储统计信息的表，区分大小写。

- STATID

  统计信息 ID，用于区分表存储在同一个 STATTAB 中的多份统计信息； 缺省为空字符串。

- STATOWN

  STATTAB 所在的模式，区分大小写；缺省为当前模式。

- NO_INVALIDATE

  是否让依赖游标失效，缺省为 TRUE。具体用法参考 GATHER_TABLE_STATS 方法的 NO_INVALIDATE 参数。

- FORCE

  忽略统计信息锁；为 TRUE 表示即使表锁住了也导入统计信息，FALSE 表示只导入未上锁的统计信息；缺省为 FALSE。

- STAT_CATEGORY

  仅作兼容 ORACLE 用，无实际作用。

22. IMPORT_DATABASE_STATS

将所有对象的统计信息从指定的 STATTAB 导入到数据库中，并将统计信息存入系统表。

统计信息导入时，导入对象完全由对象名称来确定，如果存在由系统根据 ID 自动创建的对象，例如在分区表主表上建立索引时，子表上自动建立的索引将根据索引 ID 来命名，统计信息表里存储的索引名称可能与新建立的分区表子表上的索引名称不一致，此时名称不一致的索引统计信息将无法导入。

不支持多列统计信息的导入。本操作会提交当前事务。

语法如下:

```
PROCEDURE IMPORT_DATABASE_STATS(
        STATTAB             VARCHAR(128),
        STATID              VARCHAR(128) DEFAULT '',
        STATOWN             VARCHAR(128) DEFAULT NULL,
        NO_INVALIDATE    BOOLEAN   to_no_invalidate_type(GET_PREFS('NO_INVALIDATE')),
        FORCE               BOOLEAN      DEFAULT FALSE,
        STAT_CATEGORY    VARCHAR(128) DEFAULT NULL
);
```

参数详解:

- STATTAB

  指定要存储统计信息的表，区分大小写。

- STATID

  统计信息 ID，用于区分表存储在同一个 STATTAB 中的多份统计信息； 缺省为空字符串。

- STATOWN

  存储统计信息表的拥有者，区分大小写；缺省为当前模式。

- NO_INVALIDATE

是否让依赖游标失效，缺省为 TRUE。具体用法参考 GATHER_TABLE_STATS 方法的 NO_INVALIDATE 参数。

- FORCE

  忽略统计信息锁；TRUE 表示即使表锁住了也导入统计信息，FALSE 表示只导入未上锁的统计信息；缺省为 FALSE。

- STAT_CATEGORY

  仅作兼容 ORACLE 用，无实际作用

23. export_schema_stats

导出由 OWNNAME 标识的 SCHEMA 中全部对象的统计信息，并将它们存储到由 STATTAB 标识的用户统计信息表。

不支持多列统计信息的导出。本操作会提交当前事务。

语法如下:

```
PROCEDURE EXPORT_SCHEMA_STATS(
    OWNNAME          VARCHAR(128),
    STATTAB          VARCHAR(128),
    STATID           VARCHAR(128) DEFAULT '',
    STATOWN          VARCHAR(128) DEFAULT NULL,
    STAT_CATEGORY    VARCHAR(128) DEFAULT NULL
);
```

参数详解:

- OWNNAME

待导出统计信息的 SCHEMA 名，区分大小写；缺省为当前模式。

- STATTAB

  指定要存储统计信息的表，区分大小写。

- STATID

  统计信息 ID，用于区分表存储在同一个 STATTAB 中的多份统计信息； 缺省为空字符串。

- STATOWN

  存储统计信息表的拥有者，区分大小写；缺省为当前模式。

- STAT_CATEGORY

  仅作兼容 ORACLE 用，无实际作用。

24. EXPORT_DATABASE_STATS

导出数据库中所有对象的统计信息，并存储将他们存储到由 STATOWN.STATTAB 标识的用户统计信息表。

不支持多列统计信息的导出。本操作会提交当前事务。

语法如下:

```
PROCEDURE EXPORT_DATABASE_STATS(
    STATTAB             VARCHAR(128),
    STATID              VARCHAR(128) DEFAULT '',
    STATOWN             VARCHAR(128) DEFAULT NULL,
    STAT_CATEGORY       VARCHAR(128) DEFAULT NULL
);
```

参数详解

- STATTAB

  指定要存储统计信息的表，区分大小写。

- STATID

  统计信息 ID，用于区分表存储在同一个 STATTAB 中的多份统计信息； 缺省为空字符串。

- STATOWN

  存储统计信息表的拥有者，区分大小写；缺省为当前模式。

- STAT_CATEGORY

  仅作兼容 ORACLE 用，无实际作用

25. CONV_DATA

用于将给定数据的字符串形式转换成系统表 SYSSTATS 中的 MIN/MAX 的格式。

例如：(1)输入 INT 类型数据对应的字符串'1038'，最终输出'0X0E040000'。

(2)输入 INTERVAL YEAR TO MONTH 类型数据的字符串'INTERVAL "1-02" YEAR(2) TO MONTH'（数据中有'需要转移成"），最终输出'0100000002000000026010000'。

实际上，在当前包方法的处理中，即使通过该函数设置了 MIN/MAX，执行 SET_COLUMN_STATS 时也会检查直方图中的数据，根据用户自定义桶边界 ENDPOINT_VALUE 重新设置 MIN/MAX 以保证直方图合法。

语法如下：

```
PROCEDURE CONV_DATA(
    OWNNAME             VARCHAR(128),
    TABNAME             VARCHAR(128),
    COLNAME             VARCHAR(128),
```

```
    DATA                VARCHAR2,
    RESVAL              OUT     VARCHAR2
);
```

参数详解

- OWNNAME

  用户名，区分大小写。输入参数。

- TABNAME

  表名，区分大小写。只能是普通表表名或者分区表根表表名。输入参数。

- COLNAME

  列名，区分大小写。输入参数。

- DATA

  待转换数据的字符串形式。输入参数。

- RESVAL

  输出的数据。输出参数。

26. GET_COLUMN_STATS

用于从系统表 SYSSTATS 或者用户自定义统计信息表 STATTAB 中获取统计信息，输出到用户自定义的变量中。

默认从系统表 SYSSTATS 中读取数据；当指定用户自定义统计信息表 STATTAB 时，从 STATTAB 中读取。如果系统表 SYSSTATS 或者用户自定义统计信息表 STATTAB 中没有对应表的统计信息时会报错。

使用该方法时，可以输出的信息包括：DISTCNT、NULLCNT、SREC.EPC、SREC.MINVAL、SREC.MAXVAL、AVGCLEN，其他直方图信息仍需要用户手动设置。

语法如下:

```
PROCEDURE GET_COLUMN_STATS(
    OWNNAME         VARCHAR2,
    TABNAME         VARCHAR2,
    COLNAME         VARCHAR2,
    PARTNAME        VARCHAR2 DEFAULT NULL,
    STATTAB         VARCHAR2 DEFAULT NULL,
    STATID          VARCHAR2 DEFAULT NULL,
    DISTCNT         OUT NUMBER,
    DENSITY         OUT NUMBER,
    NULLCNT         OUT NUMBER,
    SREC            OUT STATREC,
    AVGCLEN         OUT NUMBER,
    STATOWN         VARCHAR2 DEFAULT NULL,
    REALTIME_STATS  BOOLEAN DEFAULT TRUE
);
```

参数详解

- OWNNAME

  用户名，区分大小写。输入参数。

- TABNAME

  表名，区分大小写。输入参数。

- COLNAME

  列名，区分大小写。输入参数。

- PARTNAME

分区名，区分大小写。输入参数。

- STATTAB

  自定义统计信息表名，区分大小写。输入参数。

- STATID

  自定义统计信息 ID。输入参数。

- STATOWN

  自定义统计信息表所属用户。输入参数。

- REALTIME_STATS

  无实际作用，仅做兼容 ORACLE 使用。输入参数。

- DISTCNT

  唯一值个数。输出参数。

- DENSITY

  无实际作用，仅做兼容 ORACLE 使用。输出参数。

- NULLCNT

  空值个数。输出参数。

- SREC

  存储自定义统计信息相关信息的内部结构。输出参数。

- AVGCLEN

  平均列宽。输出参数。

27. PREPARE_COLUMN_VALUES

用于将用户自定义的桶边界数组写入到自定义的 STATREC 结构中。

语法如下:

```
PROCEDURE  PREPARE_COLUMN_VALUES(
    SREC          IN OUT STATREC,
    CHARVALS      CHARARRAY
);
PROCEDURE PREPARE_COLUMN_VALUES(
    SREC          IN OUT STATREC,
    NUMVALS       NUMARRAY
);
PROCEDURE PREPARE_COLUMN_VALUES(
    SREC          IN OUT STATREC,
    TSVALS        DATEARRAY
);
```

参数详解

- CHARVALS

字符类型都通过 CHARARRAY 类型存储，包括 CHAR、VARCHAR、BINARY 等所有支持收集统计信息的字符类型。输入参数。

- NUMVALS

数字类型都通过 NUMARRAY 类型存储，包括 INT、DOUBLE、FLOAT、DEC 等所有支持收集统计信息的数字类型。输入参数。

- TSVALS

日期类型都通过 DATEARRAY 类型存储，包括 DATETIME、DATE、DATETIME WITH TIME ZONE、TIMESTAMP、TIME、TIME WITH TIME ZONE、INTERVAL YEAR TO MONTH、INTERVAL DAY TO SECOND。输入参数。

- SREC.CHVALS

输出参数，存储自定义统计信息相关信息的内部结构，当输入为 CHARVALS 时，将输出此项。

- SREC.NOVALS

输出参数，存储自定义统计信息相关信息的内部结构，当输入为 NUMVALS 时，将输出此项。

- SREC.DTVALS

输出参数，存储自定义统计信息相关信息的内部结构，当输入为 TSVALS 时，将输出此项。

28. SET_COLUMN_STATS

用于将用户自定义的统计信息写入到指定的统计信息表。

默认写入系统表 SYSSTATS; 当指定用户自定义统计信息表 STATTAB 时, 写入 STATTAB。

语法如下:

```
PROCEDURE SET_COLUMN_STATS(
    OWNNAME         VARCHAR2,
    TABNAME         VARCHAR2,
    COLNAME         VARCHAR2,
    PARTNAME        VARCHAR2    DEFAULT NULL,
    STATTAB         VARCHAR2    DEFAULT NULL,
    STATID          VARCHAR2    DEFAULT '',
    DISTCNT         NUMBER      DEFAULT NULL,
    DENSITY         NUMBER      DEFAULT NULL,
    NULLCNT         NUMBER      DEFAULT NULL,
    SREC            STATREC     DEFAULT NULL,
```

```
    AVGCLEN             NUMBER      DEFAULT NULL,
    FLAGS               NUMBER      DEFAULT NULL,
    STATOWN             VARCHAR2    DEFAULT NULL,
    NO_INVALIDATE       BOOLEAN     DEFAULT NULL,
    FORCE               BOOLEAN     DEFAULT FALSE
);
```

参数详解

- OWNNAME

  用户名，区分大小写。

- TABNAME

  表名，区分大小写。

- COLNAME

  列名，区分大小写。

- PARTNAME

  分区名，区分大小写。

- STATTAB

  自定义统计信息表，区分大小写。

- STATID

  自定义统计信息表中的 ID。

- DISTCNT

  唯一值个数，缺省为 NULL。

- DENSITY

  无实际作用，仅做兼容 ORACLE 使用。

- NULLCNT

空值个数，缺省为 NULL。

- SREC

  存储自定义统计信息相关信息的内部结构，缺省为 NULL。

- AVGCLEN

  平均列宽，缺省为 NULL。

- FLAGS

  无实际作用，仅做兼容 ORACLE 使用。

- STATOWN

  存储自定义统计信息相关信息的用户。

- NO_INVALIDATE

  是否让依赖游标失效，缺省为 TRUE。

- FORCE

  忽略统计信息锁；TRUE 表示即使表锁住了也允许设置自定义统计信息，FALSE 表示只允许设置未上锁的统计信息。

使用说明:

用户自定义统计信息不能生成合法的直方图时，系统将进行报错提示，如下表所示。

表 19.6 SET_COLUMN_STATS 错误码

代码	解释
-4138	桶内数据个数数组 SREC.EHCNTS 或桶内数据唯一值个数数组 SREC.EDCNTS 错误
-4139	桶边界数组错误，可能是 SREC.NOVALS、SREC.CHVALS、SREC.DTVALS
-4140	自定义数据超过表行数，可能是 DISTCNT、SAMPLE、SREC.EPC、SAMPLE+NULLCNT
-4141	桶边界数组未严格按升序排序，可能是 SREC.NOVALS、SREC.CHVALS、SREC.DTVALS
-4142	桶边界个数数组 SREC.EKHCNTS 或 桶内数据唯一值个数数组 SREC.EDCNTS 错误
-4143	不支持的数据类型

29. IMPORT_TABLE_STATS_CUSTOM

把统计信息表中的统计信息导入到目标表中。

语法如下:

```
PROCEDURE IMPORT_TABLE_STATS_CUSTOM(
    OWNNAME             VARCHAR(128),
    TABNAME             VARCHAR(128),
    PARTNAME            VARCHAR(128)    DEFAULT NULL,
    STATTAB             VARCHAR(128),
    STATID              VARCHAR(128)    DEFAULT '',
    CASCADE             BOOLEAN         DEFAULT TRUE,
    STATOWN             VARCHAR(128)    DEFAULT NULL,
    NO_INVALIDATE       BOOLEAN         DEFAULT NULL,
    FORCE               BOOLEAN         DEFAULT FALSE,
    STAT_CATEGORY       VARCHAR(128)    DEFAULT NULL,
    SRC_OWNNAME         VARCHAR(128)    DEFAULT NULL,
    SRC_TABNAME         VARCHAR(128)    DEFAULT NULL,
    SRC_PARTNAME        VARCHAR(128)    DEFAULT NULL
);
```

参数详解

- OWNNAME

  目标表的模式名，区分大小写。

- TABNAME

  目标表名，区分大小写。

- PARTNAME

  目标表分区名，如果不指定分区，则一起导入所有子表的统计信息，缺省为 NULL。

- STATTAB

  统计信息表名，区分大小写。

- STATID

  由用户指定的统计信息标识名，缺省为空字符串。

- CASCADE

  是否连列和索引的统计信息一起导出，缺省为 TRUE。

- STATOWN

  统计信息表的模式名，区分大小写，缺省为 NULL。

- NO_INVALIDATE

  是否让依赖游标失效，缺省为 TRUE。具体用法请参考 GATHER_TABLE_STATS 方法的 NO_INVALIDATE 参数。

- FORCE

  是否忽略统计信息锁, 强制把统计信息表中的统计信息导入到目标表中, 缺省为 FALSE。为 TRUE 时表示忽略统计信息锁，强制向被锁住的目标表导入统计信息；为 FALSE 时表示仅当目标表没有被锁住才能导入统计信息。

- STAT_CATEGORY

  保留参数，无实际意义。

- SRC_OWNNAME

  源表模式名，区分大小写；如果不指定源表模式名，则默认为目标表的模式名。

- SRC_TABNAME

  源表名，区分大小写；如果不指定源表名，则默认为目标表名。

- SRC_PARTNAME

源表分区名；如果不指定分区名，则一起导入所有子表的统计信息，缺省为 NULL。

使用说明:

使用 IMPORT_TABLE_STATS_CUSTOM 进行统计信息导入时需要注意:

（1）如果 SRC_OWNNAME、SRC_TABNAME、SRC_PARTNAME 均设置为 NULL，则方法功能与 IMPORT_TABLE_STATS 无区别;

（2）如果设置了 SRC_OWNNAME、SRC_TABNAME、SRC_PARTNAME，导入对象统计信息时将根据源对象名称来确定；用户名和表名不一致时，分区名\列名\索引名一致允许导入对应的统计信息，否则跳过不导入；目标表和源表的层次一致时，允许导入统计信息，否则报错“-4136，源分区和目的分区类型不同”。

如下例表示把自定义统计信息表 USER1.STAT_TAB 中，STATID 为 A，OWNNAME 为 'USER1'，TABNAME=>'T1'的统计信息导入到 USER2.T2 中。

```
DBMS_STATS.IMPORT_TABLE_STATS_CUSTOM(OWNNAME=>'USER2',TABNAME=>'T2',STATTAB=>'S
TAT_TAB',STATID=>'A',STATOWN=>'USER1',SRC_OWNNAME=>'USER1',SRC_TABNAME=>'T1');
```

19.3 约束

以下对象不支持统计信息:

1. 视图、物化视图、外部表、DBLINK 远程表、动态视图表、记录类型数组所用的临时表、局部临时表。
2. 位图索引、位图连接索引、虚索引、空间索引、数组索引、无效的索引。
3. 所在表空间为 OFFLINE 的对象。
4. BLOB、IMAGE、LONGVARBINARY、CLOB、TEXT、LONGVARCHAR、自定义类型列和空间类型列等列类型不支持统计信息，在使用 GATHER_TABLE_STATS()方法收集列统计信息时，这些类型的列会被忽略。
5. ROWID，ROWNUM 等特殊列。

19.4 举例说明

使用包内的过程和函数之前，如果还未创建过系统包，请先调用系统过程创建系统包。

```
SP_CREATE_SYSTEM_PACKAGES (1,'DBMS_STATS');

SET SERVEROUTPUT ON;  //PRINT 需要设置这条语句，才能打印出消息
```

例 1 收集模式 PERSON 下表 ADDRESS 的统计信息，并打印收集的表信息。

```
BEGIN
    DECLARE
    OBJTAB DBMS_STATS.OBJECTTAB;
    OBJ_FILTER_LIST DBMS_STATS.OBJECTTAB;
    BEGIN
        OBJ_FILTER_LIST(0).OWNNAME = 'PERSON';
        OBJ_FILTER_LIST(0).OBJTYPE = 'TABLE';
        OBJ_FILTER_LIST(0).OBJNAME = 'ADDRESS';
    DBMS_STATS.GATHER_SCHEMA_STATS(
        'PERSON',
        100,
        FALSE,
        'FOR ALL COLUMNS SIZE AUTO',
        1,
        'AUTO',
        TRUE,
        NULL,
        NULL,
```

```
        'GATHER',
        OBJTAB,
        NULL,
        TRUE,
        TRUE,
        OBJ_FILTER_LIST
    );
    PRINT OBJTAB.COUNT;
        FOR I IN 0..OBJTAB.COUNT-1 LOOP
            PRINT OBJTAB(I).OWNNAME;
            PRINT OBJTAB(I).OBJTYPE;
            PRINT OBJTAB(I).OBJNAME;
            PRINT '-------- ';
        END LOOP;
    END;
END;
/
```

打印结果:

```
1
PERSON
TABLE
ADDRESS
--------
```

例 2 获得 PERSON 模式下表 ADDRESS 中列 ADDRESSID 的统计信息。

```
DBMS_STATS.COLUMN_STATS_SHOW('PERSON','ADDRESS','ADDRESSID');
```

返回结果集 1:

```
LINEID NUM_DISTINCT   LOW_VALUE   HIGH_VALUE   NUM_NULLS   NUM_BUCKETS   SAMPLE_SIZE
HISTOGRAM
---------- -------------------- --------- ---------- ----------------------------1
6          1          6          0          6          6          FREQUENCY
```

结果集 2:

```
OWNER TABLE_NAME COLUMN_NAME HISTOGRAM ENDPOINT_VALUE ENDPOINT_HEIGHT
ENDPOINT_KEYHEIGHT ENDPOINT_DISTINCT
PERSON ADDRESS ADDRESSID FREQUENCY 1 1 NULL NULL
PERSON ADDRESS ADDRESSID FREQUENCY 2 1 NULL NULL
PERSON ADDRESS ADDRESSID FREQUENCY 3 1 NULL NULL
PERSON ADDRESS ADDRESSID FREQUENCY 4 1 NULL NULL
PERSON ADDRESS ADDRESSID FREQUENCY 5 1 NULL NULL
PERSON ADDRESS ADDRESSID FREQUENCY 6 1 NULL NULL
```

例 3 删除 PERSON 模式下表 ADDRESS 的统计信息。

```
BEGIN
    DBMS_STATS.DELETE_TABLE_STATS ('PERSON', 'ADDRESS');
END;
/
```

例 4 自动收集用户表的统计信息。

第一步，打开监控。

设置 INI 参数 AUTO_STAT_OBJ 为 1 或 2。1：对所有表进行监控；2：只对用户通过 DBMS_STATS. SET_TABLE_PREFS 设置过 STALE_PERCENT 属性的表对象进行监控。如果

AUTO_STAT_OBJ=2，需进一步使用 DBMS_STATS.SET_TABLE_PREFS 设置 STALE_PERCENT 属性。

用 AUTO_STAT_OBJ=1 打开对 T 表的监控。

```
CREATE TABLE T(A INT);
SP_SET_PARA_VALUE(1,'AUTO_STAT_OBJ',1);
```

用 AUTO_STAT_OBJ=2 打开对 T 表的监控。

```
CREATE TABLE T(A INT);
SP_SET_PARA_VALUE(1,'AUTO_STAT_OBJ',2);
DBMS_STATS.SET_TABLE_PREFS('SYSDBA','T','STALE_PERCENT',15);
--对修改行数占比达到 STALE_PERCENT 要求即占比达到 15%的对象收集统计信息
```

下面以 AUTO_STAT_OBJ=2 为例。

第二步，执行统计信息收集操作。自动收集统计信息使用 SP_CREATE_AUTO_STAT_TRIGGER 设置触发器。

```
SP_CREATE_AUTO_STAT_TRIGGER(1, 1, 1, 1,'11:20', '2024/12/10',0,1);
```

此处，也可将自动收集方法换成手动收集方法，其它三步用法不变。

第三步，对用户表 T 进行增删改操作。

```
INSERT INTO T SELECT LEVEL CONNECT BY LEVEL<=20;
COMMIT;
```

第四步，通过系统表 SYSSTATTABLEIDU 和动态视图 V$AUTO_STAT_TABLE_IDU 查看监控信息，通过系统表 SYSSTATS 查看统计信息。

打开监控后并执行统计信息收集操作之后才能查看到 SYSSTATTABLEIDU 和 SYSSTATS 的变化。SYSSTATTABLEIDU 为 AUTO_STAT_OBJ 等于 1 或 2 时对上一次的统计信息的监控。V$AUTO_STAT_TABLE_IDU 为 AUTO_STAT_OBJ 等于 1 时的统计信息实时监控，不需要执行收集统计信息操作也能查看。

下面以 AUTO_STAT_OBJ=2 为例。

查看监控信息:

```
//在触发器触发之后查看

SELECT * FROM SYSSTATTABLEIDU;



行号          ID              TOTAL_ROWS              INSERT_ROWS             DELETE_ROWS

UPDATE_ROWS         LAST_STAT_DT            MONITOR_FLAG RSVD1         TRUNCATED

---------- ----------- -------------------- --------------------

1             1061          20                       0                       0                       0

2024-12-19 14:09:16.978000 2                0XFFFFFFFF 0
```

查看统计信息:

```
SELECT * FROM SYSSTATS WHERE ID=1061;

LINEID    ID      COLID     T_FLAG T_TOTAL   N_SAMPLE     N_DISTINCT     N_NULL     V_MIN

V_MAX   BLEVEL    N_LEAF_PAGES   N_LEAF_USED_PAGES   CLUSTER_FACTOR N_BUCKETS

DATA     COL_AVG_LEN LAST_GATHERED   INFO1         INFO2

-------------------------------- ------------------------- -------------------

1             1061          -1             T        20                      0                       0

0                       NULL          NULL          0             1                       1

              0                 0

              0X0000FFFF00000700

              -1              2024-12-19 14:09:17.001081 NULL          NULL



2             1061          0              C        20                      20                      20

0                       0X01000000 0X14000000 0             1                       1
```

```
        0               20
0X140001000800070001000000010000000100000002000000010000000300000001000000040
000000100000005000000010000000600000001000000070000000100000008000000010000000
009000000010000000A000000010000000B000000010000000C000000010000000D000000010
000000E000000010000000F0000000100000010000000010000001100000001000000120000000010
000000130000000100000014000000
        4           2024-12-19 14:09:16.998000 NULL          NULL
```

如果用户想进一步对监控统计信息进行收集，可以参考《DM8 系统管理员手册》的 22.5 统计信息章节 SYSDBA.GET_AUTO_STAT_INFO_FUNC 用法。

20 DBMS_UTILITY 包

DBMS_UTILITY 包用于查看异常信息。

20.1 数据类型

DBMS_UTILITY 定义了一种索引表类型 LNAME_ARRAY，此类型存储长名称，包括完全限定的属性名称。LNAME_ARRAY 类型定义如下：

语法如下：

```
TYPE LNAME_ARRAY IS TABLE OF VARCHAR2(4000) INDEX BY INTEGER;
```

20.2 相关方法

1. FORMAT_ERROR_STACK

获取 DMSQL 程序异常的堆栈信息。

语法如下：

```
FUNCTION  FORMAT_ERROR_STACK (
)RETURN VARCHAR2;
```

2. GET_HASH_VALUE

对于指定的字符串，返回范围在[base, base+hase_size-1]的散列值。

语法如下：

```
FUNCTION  GET_HASH_VALUE(
    NAME          VARCHAR,
    BASE      INT,
```

```
    HASH_SIZE      INT
)RETURN INT;
```

参数详解

- NAME 指定的要被 HASH 的字符串
- BASE 返回的 HASH 值的基准值，即返回值的最小值
- HASH_SIZE 指定的 HASH 表的大小

3. GET_TIME

返回一个代表当前时间的以 1/100 秒为精度的值，该值并不真正表示当前的时间值，此方法常用来计算两个时间点的间隔。

语法如下:

```
FUNCTION  GET_TIME() RETURN NUMBER;
```

参数详解

无

4. FORMAT_CALL_STACK

返回 DMSQL 程序当前调用的堆栈，包括每层的栈帧方法名。

语法如下:

```
FUNCTION  FORMAT_CALL_STACK() RETURN VARCHAR2;
```

5. FORMAT_ERROR_BACKTRACE

返回 DMSQL 程序发生异常时的堆栈信息，包括每层的栈帧地址和方法名。

语法如下:

```
FUNCTION  FORMAT_ERROR_BACKTRACE() RETURN VARCHAR2;
```

6. COMMA_TO_TABLE

将用户输入的字符串依据逗号进行分割，返回分割后字符串的索引表。

语法如下:

```
PROCEDURE COMMA_TO_TABLE(
   LIST        IN          VARCHAR2,
   TABLEN      OUT             BINARY_INTEGER,
   TAB         OUT             LNAME_ARRAY
);
```

参数详解

- LIST 输入参数，待分割的字符串列表，若其中的单个字符串未用双引号括起来，则仅支持该字符串格式为 A[.B]*，其中的 A 和 B 均为不包含特殊字符的字符串；若列表中的单个字符串需要包含部分特殊字符，则需要将该字符串用双引号括起来，COMMA_TO_TABLE 不会对双引号中的字符串做字符转换处理。
- TABLEN 输出参数，索引表中的成员个数。
- TAB 输出参数，对用户输入的字符串列表依据逗号进行分割后得到的索引表。

使用说明:

输入的字符串参数 LIST 不能为空，且多个字符串之间需用逗号进行分隔。双引号内的逗号视作正常字符处理。

7. TABLE_TO_COMMA

TABLE_TO_COMMA 是 COMMA_TO_TABLE 的逆过程，TABLE_TO_COMMA 将字符串的索引表合并成以逗号为分隔符的字符串列表。

语法如下:

```
PROCEDURE TABLE_TO_COMMA(
    TAB         IN      LNAME_ARRAY,
    TABLEN      OUT     BINARY_INTEGER,
    LIST        OUT     VARCHAR2
);
```

参数详解

- TAB 输入参数，待合并字符串的索引表。
- TABLEN 输出参数，索引表中的成员个数。
- LIST 输出参数，将索引表中的字符串以逗号为分隔符进行合并后的字符串列表。

使用说明:

输入的索引表的最后一个成员必须为 NULL，否则报错。

8. EXEC_DDL_STATEMENT

执行 DDL 语句。

语法如下:

```
PROCEDURE EXEC_DDL_STATEMENT(
    PARSE_STRING   IN   VARCHAR2
);
```

参数详解

- PARSE_STRING 输入参数，待执行的 DDL 语句。

20.3 举例说明

使用包内的过程和函数之前，如果还未创建过系统包，请先调用系统过程

SP_CREATE_SYSTEM_PACKAGES 创建系统包。

```
SP_CREATE_SYSTEM_PACKAGES(1,'DBMS_OUTPUT');

SP_CREATE_SYSTEM_PACKAGES(1,'DBMS_UTILITY');

SET SERVEROUTPUT ON;   //DBMS_OUTPUT.PUT_LINE 需要设置这条语句，才能打印出消息
```

例 1 COMMA_TO_TABLE 过程的使用。

```
DECLARE
   LEN INTEGER := 0;
   ARRAY1 DBMS_UTILITY.LNAME_ARRAY;
   P_LIST VARCHAR2 := 'HOPE,HOPE.WORLD.PEACE,"#*!:,\A123(";
BEGIN
   DBMS_UTILITY.COMMA_TO_TABLE(P_LIST, LEN, ARRAY1);
   FOR I IN 1..LEN LOOP
      DBMS_OUTPUT.PUT_LINE(ARRAY1[I]);
   END LOOP;
END;
/
```

执行结果如下：

```
HOPE
HOPE.WORLD.PEACE
"#*!:,\A123("
```

例 2 TABLE_TO_COMMA 过程的使用。

```
DECLARE
   LEN INTEGER := 0;
```

```
  ARRAY1 DBMS_UTILITY.LNAME_ARRAY;
  STR VARCHAR(200) := '';
BEGIN
  ARRAY1[1] := 'AA*($#^';
  ARRAY1[2] := 'BB.,123';
  ARRAY1[3] := NULL;
  DBMS_UTILITY.TABLE_TO_COMMA(ARRAY1, LEN, STR);
  DBMS_OUTPUT.PUT_LINE(STR);
END;
/
```

执行结果如下：

```
AA*($#^,BB.,123
```

21 DBMS_WORKLOAD_REPOSITORY 包

数据库快照是一个只读的静态的数据库。DM 快照功能是基于数据库实现的，每个快照是基于数据库的只读镜像。通过检索快照，可以获取源数据库在快照创建时间点的相关数据信息。

为了方便管理自动工作集负载信息库 AWR（Automatic Workload Repository）的信息，系统为其所有重要统计信息和负载信息执行一次快照，并将这些快照存储在 AWR 中。

用户在使用 DBMS_WORKLOAD_REPOSITORY 包之前，需要提前调用系统过程 SP_INIT_AWR_SYS(1)创建包。DM 数据库在创建该包时，默认创建一个名为 SYSAUX 的表空间，对应的数据文件为 SYSAWR.DBF，该表空间用于存储该包生成快照的数据，表空间大小上限为 16GB。如果该包被删除，那么 SYSAUX 表空间也对应地被删除。若包已创建成功，再次执行 SP_INIT_AWR_SYS(1)，不会影响当前已创建的表和数据。

AWR 功能默认是关闭的，如果需要开启，则调用 DBMS_WORKLOAD_REPOSITORY.AWR_SET_INTERVAL 过程设置快照的间隔时间。DBMS_WORKLOAD_REPOSITORY 包还负责 snapshot（快照）的管理。

DM MPP 环境下不支持 DBMS_WORKLOAD_REPOSITORY 包。

ASH（Active Session History）数据报表记录当前活动会话的等待信息，对应 V$ACTIVE_SESSION_HISTORY。ASH 报表生成时依赖 DBA_USERS 对象，该对象在数据库创建过程中，会默认随 SP_CREATE_SYSTEM_VIEWS(1)自动创建，若该对象不存在时则会报错。

AWR 支持使用基线（BASELINE）功能对已经创建或未来即将创建的快照进行保留。

21.1 相关方法

 注意：

DBMS_WORKLOAD_REPOSITORY 包不支持打开 ASM 文件系统上的路径。

DBMS_WORKLOAD_REPOSITORY 包创建、删除及其相关方法不支持在主备集群的备库模式下执行。

1. AWR_CLEAR_HISTORY

清理之前的所有 SNAPSHOT 记录。

语法如下：

```
PROCEDURE  AWR_CLEAR_HISTORY();
```

2. AWR_SET_INTERVAL

设置生成 SNAPSHOT 的时间间隔。

语法如下：

```
PROCEDURE  AWR_SET_INTERVAL(
    AWR_INTERVAL    IN      INT     DEFAULT 60
);
```

参数详解

- AWR_INTERVAL

时间间隔。单位分钟，取值范围为 10~525600，缺省值为 60。参数为 0 时，关闭快照（关闭时参数值为 57816000 分钟(110 年)，是一个无效的值)。

3. AWR_REPORT_HTML

定义 1：

将 AWR 数据报表生成 HTML 格式的报告。

```
FUNCTION AWR_REPORT_HTML(
    START_SNAP_ID    IN        INT,
    END_SNAP_ID      IN        INT
)RETURN AWRRPT_ROW_TYPE PIPELINED;
```

参数详解

- START_SNAP_ID

  起始 SNAPSHOT_ID。

- END_SNAP_ID

  终止 SNAPSHOT_ID。

返回值

返回包含报告的全部 HTML 脚本信息的嵌套表类型 AWRRPT_ROW_TYPE。

定义 2:

把 AWR 数据报表生成到指定路径的 HTML 文件。

```
PROCEDURE  SYS.AWR_REPORT_HTML(
    START_ID    IN        INT,
    END_ID      IN        INT,
    DEST_DIR    IN        VARCHAR(128),
    DEST_FILE   IN        VARCHAR(128)
);
```

参数详解

- START_ID

  起始 SNAPSHOT_ID。

- END_ID

终止 SNAPSHOT_ID。

- DEST_DIR

  指定生成报告的目标路径。

- DEST_FILE

  指定生成报告的目标文件名，文件名需要以.htm 和.html 结尾。

4. AWR_REPORT_TEXT

定义 1:

生成 TEXT 格式的报告。

```
FUNCTION AWR_REPORT_TEXT(
    START_SNAP_ID    IN        INT,
    END_SNAP_ID      IN        INT
) RETURN AWRRPT_ROW_TYPE PIPELINED;
```

参数详解

- START_SNAP_ID

  起始 SNAPSHOT_ID

- END_SNAP_ID

  终止 SNAPSHOT_ID。

返回值

返回包含报告的全部 text 脚本信息的嵌套表类型 AWRRPT_ROW_TYPE。

定义 2:

把 AWR 数据报表生成到指定路径的 TEXT 文件。

```
PROCEDURE   SYS.AWR_REPORT_TEXT(
```

```
    START_IDIN          INT,
    END_ID          IN          INT,
    DEST_DIRIN          VARCHAR(128),
    DEST_FILEIN         VARCHAR(128)
);
```

参数详解

- START_ID

  起始 SNAPSHOT_ID。

- END_ID

  终止 SNAPSHOT_ID。

- DEST_DIR

  指定生成报告的目标路径。

- DEST_FILE

  指定生成报告的目标文件名，文件名需要以.TXT 结尾。

5. CREATE_SNAPSHOT

创建一次快照 SNAPSHOT

语法如下:

```
FUNCTION CREATE_SNAPSHOT(
    FLUSH_LEVEL         IN          VARCHAR2    DEFAULT 'TYPICAL'
) RETURN INT;
```

参数详解

- FLUSH_LEVEL

  'TYPICAL' OR 'ALL'；如果为空，则缺省为'TYPICAL'，该值会影响快照生成数据的大小，

如果是'ALL'，则将全部历史数据保存，如果是'TYPICAL'则会刷部分数据。目前该参数只形式支持，暂未支持实际功能。

返回值

返回创建的快照 ID 值。

6. DROP_SNAPSHOT_RANGE

删除指定快照 ID 范围内的快照。

语法如下:

```
PROCEDURE DROP_SNAPSHOT_RANGE(
    LOW_SNAP_ID          IN      INT,
    HIGH_SNAP_ID         IN      INT,
    DBID                 IN      INT          DEFAULT NULL,
    FREE_TS_SIZE         IN      BOOLEAN      DEFAULT FALSE
);
```

参数详解

- LOW_SNAP_ID

  范围的起始值。

- HIGH_SNAP_ID

  范围的结束值。

- DBID

  表示 SNAPSHOT 所在的 DB 唯一标识，缺省为 NULL，表示当前 DB，目前该参数暂时忽略不支持。

- FREE_TS_SIZE

  是否不等待触发器执行，直接释放被删除快照对应分区表空间。TRUE：是；FALSE：否。

缺省为 FALSE，表示等待系统触发器执行时统一释放空间。目前该参数暂时忽略不支持。

7. DROP_SNAPSHOT_BY_TIME

删除快照创建时间小于等于指定时间戳的所有快照。

语法如下:

```
PROCEDURE DROP_SNAPSHOT_BY_TIME(
    END_DT              IN  TIMESTAMP,
    FREE_TS_SIZE        IN  BOOLEAN DEFAULT FALSE
);
```

参数详解:

- END_DT

  指定时间戳，若指定为 NULL，则指定时间戳为当前系统时间。

- FREE_TS_SIZE

  是否不等待触发器执行，直接释放被删除快照对应分区表空间。TRUE：是；FALSE：否。

缺省为 FALSE，表示等待系统触发器执行时统一释放空间。目前该参数暂时忽略不支持。

8. MODIFY_SNAPSHOT_SETTINGS

设置 snapshot 的属性值。

语法如下:

```
PROCEDURE MODIFY_SNAPSHOT_SETTINGS(
    RETENTION           IN      INT     DEFAULT NULL,
    AWR_INTERVAL        IN      INT     DEFAULT NULL,
    TOPNSQL             IN      INT     DEFAULT NULL,
    DBID            IN      INT     DEFAULT NULL
);
```

```
PROCEDURE MODIFY_SNAPSHOT_SETTINGS(
    RETENTION       IN      INT         DEFAULT NULL,
    AWR_INTERVAL    IN      INT         DEFAULT NULL,
    TOPNSQL         IN      VARCHAR2,
    DBID            IN      INT         DEFAULT NULL
);
```

参数详解

- RETENTION

表示 SNAPSHOT 在数据库中保留的时间，以分钟为单位，最小值为 1 天，最大值为 100 年；如果值为 0，则表示永久保留；如果值为 NULL，则表示本次设置的该值无效，保留以前的旧值。

- AWR_INTERVAL

表示每次生成 SNAPSHOT 的间隔时间，以分钟为单位，最小值为 10 分钟，最大值为 1 年；如果值为 0，则 SNAPSHOT 会失效。如果值为 NULL，则表示本次设置的该值无效，保留以前的旧值。

- TOPNSQL

如果为 NULL，则保留当前设置的值。

如果为 INT 类型，则表示按照 SQL 的衡量标准（执行时间，CPU 时间，消耗内存等）获取保存的 SQL 个数，最小值为 30，最大值为 50000。

如果为 VARCHAR2 类型, 则可以设置如下 3 个值: DEFAULT、MAXIMUM 和 N。DEFAULT 对应值为 100；MAXIMUM 对应值为 50000；值 N 是数字串，但要求转化为 INT 类型之后，值必须在 30 至 50000 之间。

目前该参数暂时忽略不支持。

- DBID

表示 SNAPSHOT 所在的 DB 唯一标识，缺省为 NULL，表示当前 DB。目前该参数暂时忽略不支持。

9. AWR_CLEAR_TMP_TABLE

清理创建快照时产生的中间表。

创建快照时系统会创建中间表临时存储数据，预期在创建快照阶段末尾进行删除，若发生异常导致中间表未能如期删除，可以调用此接口对中间表进行手动清理。

语法如下:

```
PROCEDURE  AWR_CLEAR_TMP_TABLE();
```

10. ASH_REPORT_HTML

定义 1:

将 ASH 数据报表生成 html 格式的报告。

```
FUNCTION ASH_REPORT_HTML(
    START_SNAP_ID   IN      INT,
    END_SNAP_ID     IN      INT
)RETURN AWRRPT_ROW_TYPE PIPELINED;
```

参数详解

- START_ID

  起始 SNAPSHOT_ID。

- END_ID

  终止 SNAPSHOT_ID。

返回值

返回包含报告的全部 html 脚本信息的嵌套表类型 AWRRPT_ROW_TYPE。

定义 2:

把 ASH 数据报表（目前支持单机、主备和 DPC）生成到指定路径的 HTML 文件。

```
PROCEDURE  SYS.ASH_REPORT_HTML(
    START_ID     IN        INT,
    END_ID       IN        INT,
    DEST_DIR     IN        VARCHAR(128),
    DEST_FILE    IN        VARCHAR(128)
);
```

参数详解

- START_ID

  起始 SNAPSHOT_ID。

- END_ID

  终止 SNAPSHOT_ID。

- DEST_DIR

  指定生成报告的目标路径。

- DEST_FILE

  指定生成报告的目标文件名，文件名需要以.HTM 或.HTML 结尾。

存在限制

目标路径 DEST_DIR 的长度与目标文件名 DEST_FILE 的长度的加和小于 256。

11. CREATE_BASELINE

定义 1:

功能描述

根据快照范围创建基线。

语法格式

```
PROCEDURE CREATE_BASELINE(
    START_SNAP_ID       IN   NUMBER,
    END_SNAP_ID         IN   NUMBER,
    BASELINE_NAME       IN   VARCHAR2,
    DBID                IN   NUMBER   DEFAULT NULL,
    EXPIRATION          IN   INT    DEFAULT   NULL,
    MAX_SNAP_CNT        IN   INT    DEFAULT   100
);
```

参数详解

- START_SNAP_ID

  基线覆盖起始 SNAPSHOT_ID。

- END_SNAP_ID

  基线覆盖终止 SNAPSHOT_ID。

- BASELINE_NAME

  基线名称。

- DBID

  表示 SNAPSHOT 所在的 DB 唯一标识，缺省为 NULL，表示当前 DB，目前该参数暂时忽略不支持。

- EXPIRATION

  基线有效期。

- MAX_SNAP_CNT

  基线涵盖的快照数量上限，缺省为 100。

定义 2:

功能描述

根据时间范围创建基线。

语法格式

```
PROCEDURE CREATE_BASELINE(
    START_TIME      IN  DATETIME,
    END_TIME        IN  DATETIME,
    BASELINE_NAME   IN  VARCHAR2,
    DBID            IN  NUMBER  DEFAULT NULL,
    EXPIRATION      IN  INT DEFAULT  NULL,
    MAX_SNAP_CNT    IN  INT  DEFAULT  100
);
```

参数详解

- START_TIME

  基线覆盖起始时间。

- END_TIME

  基线覆盖终止时间。

- BASELINE_NAME

  基线名称。

- DBID

  表示 SNAPSHOT 所在的 DB 唯一标识，缺省为 NULL，表示当前 DB，目前该参数暂时忽略不支持。

- EXPIRATION

  基线有效期。

- MAX_SNAP_CNT

基线涵盖的快照数量上限，缺省为 100。

使用限制

1）新创建的基线不能与已经存在的基线同名。

2）创建基线时指定的 START_SNAP_ID 与 END_SNAP_ID 必须有效。START_SNAP_ID 必须是正整数且小于 END_SNAP_ID。

3）END_TIME 不允许超过 SYSDATE 且必须大于 START_TIME。若 START_TIME 超过现存快照的最早开始创建时间或 END_TIME 与超过现存快照的最晚结束创建时间，则将 START_TIME 规约至最早开始创建时间，END_TIME 规约至最晚结束创建时间。

12. CREATE_BASELINE_TEMPLATE

定义 1:

功能描述

创建 SINGLE 类型的基线模板。

语法格式

```
PROCEDURE CREATE_BASELINE_TEMPLATE(
    START_TIME                  IN DATETIME,
    END_TIME                    IN DATETIME,
    SINGLE_BASELINE_NAME        IN VARCHAR2,
    RELATED_TEMPLATE_NAME       IN VARCHAR2,
    EXPIRATION                  IN INT,
    DBID                        IN NUMBER DEFAULT NULL,
    MAX_SNAP_CNT                IN INT DEFAULT 100
);
```

参数详解

- START_TIME

  基线覆盖起始时间。

- END_TIME

  基线覆盖终止时间。

- SINGLE_BASELINE_NAME

  基线名称。

- RELATED_TEMPLATE_NAME

  关联 REPEATING 类型的基线模板名称。若为用户创建 的 SINGLE 类型的基线模板，此列应为 NULL。

- EXPIRATION

  基线模板有效期。

- DBID

  表示 SNAPSHOT 所在的 DB 唯一标识，缺省为 NULL，表示当前 DB，目前该参数暂时忽略不支持。

- MAX_SNAP_CNT

  基线模板对应基线涵盖的快照数量上限，缺省为 100。

定义 2:

功能描述

创建 REPEATING 类型的基线模板。

语法格式

```
PROCEDURE CREATE_BASELINE_TEMPLATE(
    DAY_OF_WEEK                 IN VARCHAR2,
    HOUR_IN_DAY                 IN INT,
    DURATION                    IN INT,
```

```
    START_TIME                  IN DATE,
    END_TIME                    IN DATE,
    BASELINE_NAME_PREFIX        IN VARCHAR2,
    BSL_TEMPLATE_NAME           IN VARCHAR2,
    EXPIRATION                  IN INT,
    DBID                        IN NUMBER DEFAULT NULL,
    MAX_SNAP_CNT                IN INT DEFAULT 100
);
```

参数详解

- DAY_OF_WEEK

  基线在一周中重复的时间节点，取值'SUNDAY'，'MONDAY'，'TUESDAY'，'WEDNESDAY'，'THURSDAY'，'FRIDAY'，'SATURDAY'，大小写不敏感。

- HOUR_IN_DAY

  指定基线在当天开始的时间，取值 0~23。

- DURATION

  指定基线一天中应持续的小时数（几小时）。

- START_TIME

  REPEATING 类型基线覆盖起始时间。

- END_TIME

  REPEATING 类型基线覆盖终止时间。

- BASELINE_NAME_PREFIX

  REPEATING 类型基线指定前缀，由 REPEATING 类型模板生成的 SINGLE 类型的基线模板共享同一名称前缀，即生成的 SINGLE 类型的模板名字组成为’BASELINE_NAME_PREFIX_YYYY_MM_DD_HH_MM_SS’。

- DBID

表示 SNAPSHOT 所在的 DB 唯一标识，缺省为 NULL，表示当前 DB，目前该参数暂时忽略不支持。

- EXPIRATION

生成的 SINGLE 类型基线模板有效期。

- MAX_SNAP_CNT

生成的 SINGLE 类型基线模板所涵盖的快照数量上限，缺省为 100。

使用限制

1）新创建的基线模板不能与已经存在的基线模板同名。

2）基线模板名字只能包含字母， 数字， _ ，$，#。

3）START_TIME 不允许小于 SYSDATE 且不允许小于等于 END_TIME。

4）SINGLE 类型的基线模板同时存储在基线表 SYS.WRM$_BASELINE 与基线模板表 SYS.WRM$_BASELINE_TEMPLATE 中，且基线名称与基线模板名称均由 SINGLE_BASELINE_NAME 指定。

13. DROP_BASELINE

功能描述

删除基线。

语法格式

```
PROCEDURE DROP_BASELINE(
    DROP_BASELINE_NAME          IN VARCHAR2,
    CASCADE_FLAG                IN BOOLEAN DEFAULT FALSE
);
```

参数详解

- DROP_BASELINE_NAME

  基线名称。

- CASCADE_FLAG

  是否级联删除基线对应快照，TRUE 表示级联删除，FALSE 表示不级联删除，缺省为 FALSE。

14. DROP_BASELINE_TEMPLATE

功能描述

删除基线模板。

语法格式

```
PROCEDURE DROP_BASELINE_TEMPLATE(
    DROP_TEMPLATE_NAME      IN VARCHAR2
);
```

参数详解

- DROP_TEMPLATE_NAME

  基线名称。

使用限制

1）删除 REPEATING 类型的基线模板时，不会删除任何已经由该模板在之前已经生成的 SINGLE 类型的基线数据，那些基线会继续保留在 AWR 中直到他们的保留期过期或调用 DROP_BASELINE 进行手动删除。

2）删除模板后不再根据该模板的规则自动生成新的基线。

15. MODIFY_BASELINE_EXPIRATION

功能描述

修改基线过期时间。

语法格式

```
PROCEDURE MODIFY_BASELINE_EXPIRATION(
    MDF_BASELINE_NAME       IN VARCHAR2,
    MDF_EXPIRATION          IN NUMBER
);
```

参数详解

- MDF_BASELINE_NAME

  基线名称。

- MDF_EXPIRATION

  新的过期时间。

16. MODIFY_BASELINE_TEMPLATE_EXPIRATION

功能描述

修改基线模板的过期时间。

语法格式

```
PROCEDURE MODIFY_BASELINE_TEMPLATE_EXPIRATION(
    MDF_TEMPLATE_NAME       IN VARCHAR2,
    MDF_EXPIRATION          IN NUMBER
);
```

参数详解

- MDF_TEMPLATE_NAME

  基线模板名称。

- MDF_EXPIRATION

  新的过期时间。

使用限制

仅影响还未创建的基线，已经生成的基线可以通过 MODIFY_BASELINE_EXPIRATION 方法进行修改。

17. MODIFY_BASELINE_MAXCNT

功能描述

修改基线的快照最大数量上限。

语法格式

```
PROCEDURE MODIFY_BASELINE_TEMPLATE_EXPIRATION(
    MDF_BASELINE_NAME       IN VARCHAR2,
    MDF_MAXCNT              IN INT
);
```

参数详解

- BASELINE_NAME

  基线名称。

- MAXCNT

  新的快照数量上限。

18. MODIFY_BASELINE_TEMPLATE_MAXCNT

功能描述

修改基线模板的最大数量上限。

语法格式

```
PROCEDURE MODIFY_BASELINE_TEMPLATE_MAXCNT(
    MDF_TEMPLATE_NAME       IN VARCHAR2,
    MAXCNT                  IN INT
);
```

参数详解

- MDF_TEMPLATE_NAME

  基线模板名称。

- MAXCNT

  新的快照数量上限。

使用限制

仅影响还未创建的基线，已经生成的基线可以通过 MODIFY_BASELINE_MAXCNT 方法进行修改。

19. AWR_DIFF_REPORT_HTML

功能描述

在 DSC、DPC 环境下生成含有多个快照实例的使用两个时间段进行比较的 HTML 格式的报告。

语法格式

```
FUNCTION AWR_DIFF_REPORT_HTML(
    START_SNAP_ID_1ST          IN INT,
    END_SNAP_ID_1ST            IN INT,
    START_SNAP_ID_2ND          IN INT,
    END_SNAP_ID_2ND            IN INT
) RETURN AWRRPT_ROW_TYPE PIPELINED;
```

参数详解

- START_SNAP_ID_1ST

  时间段 1 的起始 SNAPSHOT_ID。

- END_SNAP_ID_1ST

时间段 1 的终止 SNAPSHOT_ID。

- START_SNAP_ID_2ND

  时间段 2 的起始 SNAPSHOT_ID。

- END_SNAP_ID_2ND

  时间段 2 的终止 SNAPSHOT_ID。

  支持 START_SNAP_ID 与 END_SNAP_ID 来自于不同实例。

返回值

返回包含报告的全部 HTML 脚本信息的嵌套表类型 AWRRPT_ROW_TYPE。

20. AWR_DIFF_REPORT_HTML

定义 1:

功能描述

生成使用两个时间段进行比较的 HTML 格式的报告。

语法格式

```
FUNCTION AWR_DIFF_REPORT_HTML(
    START_SNAP_ID_1ST          IN INT,
    END_SNAP_ID_1ST            IN INT,
    START_SNAP_ID_2ND          IN INT,
    END_SNAP_ID_2ND            IN INT
)RETURN AWRRPT_ROW_TYPE PIPELINED;
```

参数详解

- START_SNAP_ID_1ST

  时间段 1 的起始 SNAPSHOT_ID。

- END_SNAP_ID_1ST

时间段 1 的终止 SNAPSHOT_ID。

- START_SNAP_ID_2ND

  时间段 2 的起始 SNAPSHOT_ID。

- END_SNAP_ID_2ND

  时间段 2 的终止 SNAPSHOT_ID。

返回值

返回包含报告的全部 HTML 脚本信息的嵌套表类型 AWRRPT_ROW_TYPE。

定义 2:

功能描述

把 AWR_DIFF 数据报表（包含单实例与 DSC 环境下多实例）生成到指定路径的 HTML 文件。

语法格式

```
PROCEDURE  SYS.AWR_DIFF_REPORT_HTML(
    START_ID_1ST     IN     INT,
    END_ID_1ST       IN     INT,
    START_ID_2ND     IN     INT,
    END_ID_2ND       IN     INT,
    DEST_DIR         IN     VARCHAR(128),
    DEST_FILE        IN     VARCHAR(128)
);
```

参数详解

- START_ID_1ST

  时间段 1 的起始 SNAPSHOT_ID。

- END_ID_1ST

  时间段 1 的终止 SNAPSHOT_ID。

- START_ID_2ND

  时间段 2 的起始 SNAPSHOT_ID。

- END_ID_2ND

  时间段 2 的终止 SNAPSHOT_ID。

- DEST_DIR

  指定生成报告的目标路径。

- DEST_FILE

  指定生成报告的目标文件名，文件名需要以.HTM 和.HTML 结尾。

  使用限制

  目标路径 DEST_DIR 的长度与目标文件名 DEST_FILE 的长度的加和小于 256。

21.2 系统表

DBMS_WORKLOAD_REPOSITORY 包相关的系统表包括快照信息管理表及快照信息表，这些系统表都只有创建了 DBMS_WORKLOAD_REPOSITORY 包后才能查看。

21.2.1 快照信息管理表

1. SYS.WRM$_WR_CONTROL

记录快照的相关控制信息。

列名	类型	说明
DBID	INTEGER	数据库 ID（NULL）
SNAP_INTERVAL	INTERVAL DAY(5) TO SECOND(1)	快照间隔

RETENTION	INTERVAL DAY(5) TO SECOND(1)	快照保留时间（单位:INTERVAL DAY TO SECOND 类型）
TOPNSQL	INTEGER	每次快照搜集 sql 条数
STATUS_FLAG	INTEGER	AWR 状态（1：开启；0：关闭）

2. SYS.WRM$_SNAPSHOT

记录快照的相关信息。

列名	类型	说明
SNAP_ID	INTEGER	快照 ID
DBID	INTEGER	数据库 ID（NULL）
INSTANCE_NUMBER	INTEGER	实例编号
STARTUP_TIME	DATETIME(6)	数据库启动时间
BEGIN_INTERVAL_TIME	DATETIME(6)	开始快照时间
END_INTERVAL_TIME	DATETIME(6)	结束快照时间
FLUSH_ELAPSED	INTERVAL DAY(5) TO SECOND(1)	生成快照的时间（NULL）
SNAP_LEVEL	INTEGER	快照等级，恒为 1
STATUS	INTEGER	快照是否成功生成，0：成功；1：失败（NULL）
ERROR_COUNT	INTEGER	快照生成失败时的错误数（NULL）
BL_MOVED	INTEGER	保留字段（NULL）
SNAP_FLAG	INTEGER	快照生成方式（NULL）
SNAP_TIMEZONE	INTERVAL DAY(1) TO SECOND(0)	快照生成的时区和 UTC 时区的时间差

BEGIN_INTERVAL_TIME_TZ	DATETIME(6) WITH TIME ZONE	带时区的开始快照时间
END_INTERVAL_TIME_TZ	DATETIME(6) WITH TIME ZONE	带时区的结束快照时间

21.2.2 快照信息表

快照信息表记录快照中数据库的一些具体信息。

1. SYS.WRH$_RESOURCE_LIMIT

显示所有快照中表、用户的空间限制信息。

列名	类型	说明
SNAP_ID	INTEGER	快照 ID
NAME	VARCHAR(128)	表/用户名
ID	INTEGER	表/用户 ID
TYPE	VARCHAR(64)	表/用户类型
SPACE_LIMIT	INTEGER	空间限制大小，以页为单位
SPACE_USED	INTEGER	空间实际使用大小，以页为单位

2. SYS.WRH$_SQL_HISTORY

当 INI 参数 ENABLE_MONITOR=1 时，显示所有快照中执行 SQL 的历史记录信息，其中不同快照的 SQL_ID 和 START_TIME 肯定不相同。

列名	类型	说明
SNAP_ID	INTEGER	快照 ID
SQL_ID	INTEGER	当前语句的 SQL ID
START_TIME	DATETIME(6)	SQL 执行的起始时间
TIME_USED	BIGINT	SQL 执行所使用时间（单位：usec）

BYTES_DYNAMIC_ALLOCED	BIGINT	动态分配字节数
N_LOGIC_READ	INTEGER	语句逻辑读的次数
N_PHY_READ	INTEGER	语句物理读的次数
AFFECTED_ROWS	INTEGER	语句影响的行数
HARD_PARSE_FLAG	INTEGER	语句硬解析标记，0：软解析；1：语义解析；2：硬解析
EXEC_ID	BIGINT	SQL 执行 ID

3. SYS.WRH$_SQLTEXT

显示所有快照的缓冲区中的 SQL 语句信息。

列名	类型	说明
SNAP_ID	INTEGER	快照 ID
SQL_ID	INTEGER	语句的 SQL ID
N_EXEC	INTEGER	语句执行次数
SQL_TEXT	TEXT	SQL 语句内容，超过 7168 的串不分行显示，而是显示成一行
SQL_NTH	INTEGER	SQL 语句段号（从 0 开始）

4. SYS.WRH$_SQLTEXT_TMP

SYS.WRH$_SQLTEXT 的辅助表，显示所有快照中缓冲区中的 SQL 语句信息。

列名	类型	说明
SNAP_ID	INTEGER	快照 ID
SQL_ID	INTEGER	语句的 SQL ID
N_EXEC	INTEGER	语句执行次数

SQL_TEXT	TEXT	SQL 语句内容，与 v$SQLTEXT 里的一致，超过 7168 的串分行显示
SQL_NTH	INTEGER	SQL 语句段号（从 0 开始）

5. SYS.WRH$_SESSIONS

显示所有快照中会话的具体信息，如执行的 sql 语句、主库名、当前会话状态、用户名等。

列名	类型	说明
SNAP_ID	INTEGER	快照 ID
N_SESSIONS	INTEGER	会话数
N_STMTS	INTEGER	已使用的 STMT 数量之和

6. SYS.WRH$_SESSION_WAIT_HISTORY

显示所有快照中会话等待事件的历史信息。

列名	类型	说明
SNAP_ID	INTEGER	快照 ID
SQL_ID	INTEGER	语句的 SQL ID
EVENT#	SMALLINT	事件编号
EVENT	VARCHAR(32)	事件名称
TIME_WAITED_MICRO	INTEGER	等待时间（单位：微秒）
WAIT_CLASS#	SMALLINT	等待事件类别编号
SESSION#	INTEGER	会话 ID

7. SYS.WRH$_ACTIVE_SESSION_HISTORY

显示当前活动会话的等待信息。

此表与其他快照信息表的不同之处是，向此表内插入数据的时间周期与 ASH 刷盘的时间周期保持一致，而不与创建快照的时间周期一致。这就导致执行创建快照的操作后此表内可能没有数据写入，或者未执行创建快照的操作但此表内已写入数据两种情况均属正常现象。例如：由于一个快照创建时间需要至少几秒钟，而 ASH 自身采样频率为 1s，故快照 BEGIN_INTERVAL_TIME 与 END_INTERVAL_TIME 之间可能出现查询 WRH$_ACTIVE_SESSION_HISTORY 表数据不同批次的情况。

基于上述差异，SNAP_ID 这一列定义也存在不同。此视图中 SNAP_ID 为插入数据时的时间戳，不是快照表 WRM$_SNAPSHOT 中的快照 ID，所以在生成 ASH 报表时部分模块需要首先获取开始和结束快照对应的 BEGIN_INTERVAL_TIME，即数据插入时的系统时间。

可以通过以下语句，获取实际进行 ASH 报表模块生成的、可以映射在 WRH$_ACTIVE_SESSION_HISTORY 快照信息表中的 SNAP_ID 范围。

```
--获取指定快照 id 在 WRH$_ACTIVE_SESSION_HISTORY 中的时间戳范围
--语句中涉及的 START_SNAP_ID 和 END_SNAP_ID 分别表示生成 ASH 报表时指定的起始 SNAP_ID 和结束 SNAP_ID。
DECLARE
    TMP_START          BIGINT;
    TMP_END            BIGINT;
    CURSOR CUR1 FAST IS
        SELECT         SNAP_ID,         TO_NUMBER(YEAR(BEGIN_INTERVAL_TIME))         TYEAR,
TO_NUMBER(MONTH(BEGIN_INTERVAL_TIME)) TMONTH,
           TO_NUMBER(DAY(BEGIN_INTERVAL_TIME))                                        TDAY,
TO_NUMBER(HOUR(BEGIN_INTERVAL_TIME)) THOUR,
           TO_NUMBER(MINUTE(BEGIN_INTERVAL_TIME))                                     TMINUTE,
TO_NUMBER(SECOND(BEGIN_INTERVAL_TIME)) TSECOND
```

```
        FROM SYS.WRM$_SNAPSHOT WHERE SNAP_ID IN (START_SNAP_ID, END_SNAP_ID);
BEGIN
    FOR X IN CUR1 LOOP
        IF X.SNAP_ID = 1 THEN
          TMP_START = X.TYEAR * 10000000000 + X.TMONTH * 100000000 + X.TDAY * 1000000 +
X.THOUR * 10000 + X.TMINUTE * 100 + X.TSECOND;
        ELSE
          TMP_END = X.TYEAR * 10000000000 + X.TMONTH * 100000000 + X.TDAY * 1000000 +
X.THOUR * 10000 + X.TMINUTE * 100 + X.TSECOND;
        END IF;
    END LOOP;
  PRINT TMP_START;
  PRINT TMP_END;
END;
/
```

该表的列说明中提到的 SESSION_STATE 对应于 V$ACTIVE_SESSION_HISTORY 系统视图的 SESSION_STATE 列。

列名	类型	说明
SNAP_ID	BIGINT	执行数据插入时的系统时间，格式为 YYYYMMDDHH24MISS，与其他信息表不同
INSTANCE_NUMBER	INTEGER	实例个数，单机下为 1
SAMPLE_ID	BIGINT	采样 ID

SAMPLE_TIME	DATETIME(3)	采样时戳
SESSION_ID	BIGINT	会话 ID
SESSION_SERIAL#	INTEGER	会话序列号，用来唯一标识会话
USER_ID	INTEGER	当前用户 ID
SQL_ID	INTEGER	采样时正在执行的语句 ID
IS_SQLID_CURRENT	VARCHAR(1)	SQL_ID 列所标识的语句是否正在执行(Y/N)
SESSION_TYPE	VARCHAR(10)	FOREGROUND/BACKGROUND FOREGROUND 表示用户创建的会话; BACKGROUND 表示系统内部创建的会话
SQL_OPCODE	INTEGER	SQL 语句类型
SQL_OPNAME	VARCHAR(64)	SQL 语句类型名
EVENT	VARCHAR(64)	如果 SESSION_STATE=WAITING,则表示会话在采样时的等待事件。否则为 NULL
EVENT_ID	SMALLINT	采样时等待事件的 ID
P1TEXT	VARCHAR(64)	等待事件对应的参数 1 说明
P1	VARCHAR(256)	等待事件对应的参数 1 值
P2TEXT	VARCHAR(64)	等待事件对应的参数 2 说明
P2	BIGINT	等待事件对应的参数 2 值
P3TEXT	VARCHAR(64)	等待事件对应的参数 3 说明
P3	BIGINT	等待事件对应的参数 3 值
P4TEXT	VARCHAR(64)	等待事件对应的参数 4 说明
P4	BIGINT	等待事件对应的参数 4 值

WAIT_CLASS	VARCHAR(64)	等待会话在采样时等待的事件的类名
WAIT_CLASS_ID	INTEGER	等待会话在采样时等待的事件的类 ID
WAIT_TIME	BIGINT	如果是在 SESSION_STATE=ON CPU 采样，则表示会话上次等待的事件的总等待时间，单位为微秒，否则为 0
TIME_WAITED	BIGINT	如果 SESSION_STATE=WAITING，则表示会话实际花费在等待该事件的时间，单位微秒
PROGRAM	VARCHAR(128)	程序名
MODULE	VARCHAR(64)	采样时执行模块的名称(DBMS_APPLICATION_INFO.SET_MODULE)
ACTION	VARCHAR(64)	采样时执行模块的动作(DBMS_APPLICATION_INFO.SET_ACTION)
CLIENT_ID	VARCHAR(64)	客户端标识
SQL_EXEC_ID	BIGINT	SQL 执行 ID
TOP_LEVEL_SQL_ID	INTEGER	顶层 SQL 语句 ID
TOP_LEVEL_SQL_OPCODE	INTEGER	顶层 SQL 语句类型
MACHINE	VARCHAR(128)	客户端的机器名
PORT	INTEGER	客户端的端口号

快照信息表共有 39 张表，每张快照信息表都有其对应的动态性能视图，快照信息表名以“WRH$_”开头，相应的动态性能视图名以“V$”开头。例如快照信息表

SYS.WRH$_ARCH_QUEUE 对应的动态性能视图为 V$ARCH_QUEUE。除了上述 7 张快照信息表与其对应的动态性能视图在结构上有些许不同，其余的快照信息表仅比对应的动态性能视图多一列名为 SNAP_ID 的整型列，其余结构完全相同，如 SYS.WRH$_ARCH_QUEUE 仅比 V$ARCH_QUEUE 多一列 SNAP_ID，这里不再赘述，仅给出快照信息表的名称，具体字段介绍可以参考对应的动态性能视图，动态性能视图介绍可参考《DM8 系统管理员手册》附录 2。

剩余的快照信息表名称如下：

1） SYS.WRH$_ARCH_QUEUE

2） SYS.WRH$_BUFFERPOOL

3） SYS.WRH$_CKPT_HISTORY

4） SYS.WRH$_DATAFILE

5） SYS.WRH$_DB_CACHE

6） SYS.WRH$_DEADLOCK_HISTORY

7） SYS.WRH$_DMSQL_EXEC_TIME

8） SYS.WRH$_DM_INI

9） SYS.WRH$_HASH_MERGE_USED_HISTORY

10） SYS.WRH$_LARGE_MEM_SQLS

11） SYS.WRH$_LOCK

12） SYS.WRH$_MAL_INFO

13） SYS.WRH$_MEM_POOL

14) SYS.WRH$_MTAB_USED_HISTORY

15) SYS.WRH$_PSEG_ITEMS

16) SYS.WRH$_RLOG

17) SYS.WRH$_SCP_CACHE

18) SYS.WRH$_SESSION_HISTORY

19) SYS.WRH$_SORT_HISTORY

20) SYS.WRH$_SQL_NODE_HISTORY

21) SYS.WRH$_SQL_PLAN_NODE

22) SYS.WRH$_SQL_STAT_HISTORY

23) SYS.WRH$_SYSSTAT

24) SYS.WRH$_SYSTEMINFO

25) SYS.WRH$_SYSTEM_EVENT

26) SYS.WRH$_TABLESPACE

27) SYS.WRH$_TASK_QUEUE

28) SYS.WRH$_TRACE_QUEUE

29) SYS.WRH$_VIRTUAL_MACHINE

30) SYS.WRH$_WAIT_HISTORY

31) SYS.WRH$_TRX

32) SYS.WRH$_LONG_EXEC_SQLS

21.3 创建、检测、删除语句

21.3.1 SP_INIT_AWR_SYS

创建或删除 DBMS_WORKLOAD_REPOSITORY 系统包。

语法如下:

```
VOID
SP_INIT_AWR_SYS(
    CREATE_FLAG          INT
);
```

参数详解

- CREATE_FLAG

  为 1 时表示创建 DBMS_WORKLOAD_REPOSITORY 包；为 0 表示删除该系统包。

返回值

无

举例说明

创建 DBMS_WORKLOAD_REPOSITORY 系统包。

```
SP_INIT_AWR_SYS(1);
```

21.3.2 SF_CHECK_AWR_SYS

系统的 DBMS_WORKLOAD_REPOSITORY 系统包启用状态检测。

语法如下:

```
INT
SF_CHECK_AWR_SYS ()
```

返回值

0：未启用；1：已启用

举例说明

获得 DBMS_WORKLOAD_REPOSITORY 系统包的启用状态。

```
SELECT SF_CHECK_AWR_SYS;
```

21.4 报表解析说明

21.4.1 信息表介绍


注意：**下面对各报表含义的详细介绍中，由于部分项的含义是较为明显的，存在对部分内容做忽略处理的情况，实际包含内容以生成的报表为准。**

自动工作集负载信息库 AWR（Automatic Workload Repository）提供了某一时间段内整个系统的报表数据，通过生成并解读该报告，用户能够对指定时间段内数据库系统的性能进行分析。后文中按照报表标题名细分出了若干小节，并依次给出了 AWR 报告中每个表格中的字段名、相应的说明和字段的计算方式。信息表的各个部分含义如下：

- HEAD INFO

AWR 报告的头部信息，包含数据库基本信息（DB INFO）、实例运行环境基本信息（HOST INFO）、快照统计基本信息（SNAP INFO）三个部分。

- Report Summary

AWR 报告的摘要，包含内存池信息报告（Memory Pool）、缓存池信息报告（Buffer Pool）、两次快照之间的数据库负载概况（Load Profile）、内存命中率及数据库实例操作的效率报告（Instance Efficiency Percentages）、系统中最严重的 5 个等待（Top 5 Timed Foreground Events）。

- Main Report

AWR 报告的主体部分，包含等待事件统计信息等待事件统计名称（Wait Events Statistics）、SQL 语句相关统计（SQL Statistics）、实例活动统计信息（Instance Activity Statistics）、IO 信息（IO Stats）、缓存区统计信息（Buffer Pool Statistics）、等待事件统计信息（Wait Statistics）、撤销的统计信息（Undo Statistics）、锁统计信息（Lock Statistics）、死锁统计信息（Dead Lock Statistics）、隔离区活动统计信息（Seg Activity Statistics）、数据库缓存信息（Dictionary Cache Statistics）、库缓存统计信息（Library Cache Statistics）、内存池的统计信息（Memory Statistics）、分类统计信息（Sort Statistics）、空间限制信息（Resource Limit Statistics）、邮箱视图信息（Mail Statistics）、dm.ini 参数信息（dm.ini Parameters）、顶级等待事件中的高负载 SQL 语句（Top SQL with Top Events）。

除了 AWR 数据报表以外，用户也可自行生成并解读 ASH 数据报表。由于二者存在部分重叠内容，因此放在一起介绍。ASH 数据报表各方面含义如下：

- HEAD INFO

ASH 报告的头部信息，包含 DB INFO 和 SNAP INFO。其中大部分项的解释与 AWR 一致，在 AWR 的 HEAD INFO 中统一介绍。

- ASH_REPORT

ASH 报告的主体部分，包括顶级等待事件（Top Event），负载情况（Load Profile），顶级 SQL 语句（Top SQL），顶级等待会话（Top Sessions），分析期间的活动与工作负载（Activity Over Time）。各内容将在 AWR 的 Main Report 中介绍。

21.4.1.1 HEAD INFO

21.4.1.1.1 DB INFO

显示数据库的基本信息，每个字段说明及值的计算参见下表。

表 21.1 DB INFO 说明

字段名	说明	计算值

DB Name	数据库名	初始化时指定的数据库名
DB Id	数据库唯一标识	\
Instance	实例名	生成报表时指定的起始快照 ID 对应的实例名
Inst num	实例数	单机环境为 1
Startup Time	起始快照服务器启动时间	V$INSTANCE 中的 START_TIME
Release	起始快照服务器的版本	V$INSTANCE 中的 SVR_VERSION
DSC	是否为 DSC 环境	单机环境为 NO, DSC 环境为 YES

21.4.1.1.2 HOST INFO

显示实例运行环境，每个字段说明及值的计算参见下表。

表 21.2 HOST INFO 说明

字段名	说明	计算值
Host Name	主机名	V$INSTANCE 中 HOST_NAME
Platform	实例运行平台	\
Cpus	cpu 数	V$SYSTEMINFO 中 N_CPU
Cores	核数	CORE_NUM*SOCKET_NUMBER
Sockets	socket 数	SOCKET_NUMBER
Memory(GB)	内存	物理内存大小，单位 GB

21.4.1.1.3 SNAP INFO

显示快照统计的基本信息，每个字段说明及值的计算参见下表。

表 21.3 SNAP INFO 说明

字段名	说明	计算值
Snap Id	快照 ID	传入参数：快照的范围值[start,end]
Snap Time	快照生成的时间	起始和结束快照生成的时间。AWR 对应的是起始

		和结束快照的结束生成时间，即 WRM$_SNAPSHOT 中的 END_INTERVAL_TIME，ASH 对应的是及时和结束快照的开始生成时间，即 WRM$_SNAPSHOT 中的 BEGIN_INTERVAL_TIME
Sessions	生成快照时的会话数	V$SESSIONS count(*)
stmts/Session	每个会话的语句数	V$SESSIONS SUM(N_USED_STMT)/Sessions
Elapsed	起始快照和结束快照持续的时间	起始快照和结束快照的时间差
DB Time	在起始和结束快照间的数据库时间	V$SYSSTAT 中的 db_time (ms)，选择起始和结束快照的差值
Average Active Session	平均活动会话	WRH$_ACTIVE_SESSION_HISTORY 信息表中获取到采样 SQLID 总数/去重后会话数

21.4.1.2 Report Summary

21.4.1.2.1 Memory Pool

记录 dm memory pool 的使用情况，每个字段说明及值的计算参见下表。每项显示 begin 和 end 的值。

表 21.4 Memory Pool 说明

字段名	说明	计算值
Memory Pool Size(MB)	内存池大小	V$SYSSTAT 中' memory pool size in bytes'
Memory Used(MB)	内存池使用大小	V$SYSSTAT 中'memory used bytes'
Bytes Allocated From OS(MB)	从 OS 申请内存数	V$SYSSTAT 中'bytes allocated from os'

Memory used bytes from os(MB)	内存池使用字节数	V$SYSSTAT 中'memory used bytes from os'
Memory Usage %	memory pool 使用率	memory used / memory pool size
% SQL with executions>1	执行次数大于 1 的 sql 比率	V$SYSSTAT 中的 sql cache hit count / sql executed count
% plan with executions>1	执行次数大于 1 的计划比率	V$SYSSTAT 中的 plan cache hit count / sql executed count
%rowset cache hit count	rowset 比率	V$SYSSTAT 中的 rowset cache hit count / rowset total count

21.4.1.2.2 Buffer Pool

显示 buffer pool 的信息，每个字段说明及值的计算参见下表。每项显示 begin 和 end 的值。

表 21.5 Buffer Pool 说明

字段名	说明	计算值
Name	缓冲池名称	V$BUFFERPOOL 中'NAME'
Page Size(Bytes)	基缓冲池页大小	V$BUFFERPOOL 中'PAGE_SIZE'，单位 Bytes
Total Pages	页数	V$BUFFERPOOL 中' N_PAGES'
Free	空闲页数目	V$BUFFERPOOL 中'FREE'
Logic Reads	READ 命中的次数	V$BUFFERPOOL 中 'N_LOGIC_READS'
Discards	淘汰的页数	V$BUFFERPOOL 中'N_DISCARD'
Rat Hit %	命中率	V$BUFFERPOOL 中'RAT_HIT'

21.4.1.2.3 Load Profile

主要记录两次快照之间的数据库负载概况，每个字段的含义和说明，参见下表。

表 21.6 Load Profile 说明

字段名	说明	计算值
DB Time(s)	每秒/每事务的 DB TIME	总的 db 时间、快照持续时间或总的事务数，根据 V$sysstat 计算，取两次快照的差值
DB CPU(s)	每秒/每事务的 cpu 时间	总的 cpu 时间、快照持续时间或总的事务数，根据 V$sysstat 计算，取两次快照的差值
Redo size	每秒/每事务 redo 日志的字节大小	redo 日志的大小(根据 V$sysstat 的 redo log size in pages 计算）与总的 db 时间或者总的事务数的相除
Logical reads	每秒/每事务逻辑读的块数	logic read count / db time 或者 total_trx
Block changes	每秒/每事务修改的块数	V$SYSSTAT 里添加 block changes 事件
Physical reads	每秒/每事务物理读的块数	physical read count / db time
Physical writes	每秒/每事务物理写的块数	physical write count / db time
User calls	每秒/每事务用户 call 次数	V$SYSSTAT 中的 command received count 总数
Parses	每秒/每事 SQL 解析的次数	添加 parse 解析次数，prepare_low2
Hard parses	每秒/每事务 SQL 硬解	V$SYSSTAT 添加硬解析次数，每走一次 phc 算一次

	析的次数	
Logons	每秒/每事务登录的次数	session alloc count
Logoff	每秒/每事务登出的次数	session free count
Executes	每秒/每事务 SQL 执行次数	V$SYSSTAT 中的 sql execution count/快照持续时间
Rollbacks	每秒/每事务的回滚率	transaction rollback count
Commit	每秒/每事务的提交率	transaction commit count
Deadlock	每秒/每事务的死锁率	transaction deadlock count
Transactions	每秒事务数.每秒产生的事务数，反映数据库任务繁重与否	transaction total count
%Changes changed per read	每个逻辑读产生的数据页变更	data page changes / logic read count
Rows per transaction	每事务行数	V$SYSSTAT 中的 optimized sort count+one-way sort count+one-way sort count
Rows per Sort	每事务排序行数	V$SORT_HISTORY 中 SORT_ROWS

21.4.1.2.4 Instance Efficiency Percentages（Target 100%）

本节统计内存命中率及数据库实例操作的效率，每个字段的含义和说明参见下表。

表 21.7 Instance Efficiency Percentages 说明

字段名	说明	计算值
Buffer Nowait	在内存获得数据的未等待比例，在缓冲区	V$SYSTEM_EVENT 中的 buffer busy w

%	中获取 Buffer 的未等待比率	ait 事件
Buffer Hit %	数据块的命中率	V$BUFFER_POOL 中的 N_LOGIC_READS / N_LOGIC_READS + N_PHY_READS
Redo NoWait %	LOG 缓冲区获得 BUFFER 的未等待比例	V$SYSTEM_EVENT 中的(100-redo log system busy 的 TOTAL_WAITS)/ TOTAL_WAITS
Library Hit %	从 Library Cache 中检索到一个解析过的 SQL 或 PL/SQL 语句的比率	V$SYSSTAT 中的(plan cache hit count + rowset cache hit count + sql cache hit count) / (plan cache hit count + rowset cache hit count + sql cache hit count + plan total count)
Execute to Parse %	语句执行与分析的比例，如果要 SQL 重用率高，则这个比例会很高. 该值越高表示一次解析后被重复执行的次数越多。该值<0 通常说明 shared pool 设置或者语句效率存在问题，造成反复解析，reparse 可能较严重,或者是可能同 snapshot 有关，通常说明数据库性能存在问题。	V$SYSSTAT 中的 100 * (1 - parse count/Executions)
In-memory Sort %	在内存中排序的比率	V$SYSSTAT 中的 optimized sort count / (optimized sort count + one-way sort count + multi-way sort co

		unt)
Soft Parse %	软解析的百分比(softs/(softs+hards))，近似当作 sql 在共享区的命中率	V$SYSSTAT 中的 parse time - hard parse time(ms) / parse time
% Non-Parse CPU	SQL 实际运行时间/(SQL 实际运行时间+SQL 解析时间) 值越小表示解析消耗时间过多	V$SYSSTAT 中的(1-PARSE_CPU)/db time(ms)

21.4.1.2.5 Top 5 Timed Foreground Events

显示系统中最严重的 5 个等待，按所占等待时间的比例倒序显示，每个字段的说明参见下表。

表 21.8 Top 5 Timed Foreground Events 说明

字段名	说明	计算值
event	等待时间的名称	V$SYSTEM_EVENT 中的 Event
Waits	等待的次数	V$SYSTEM_EVENT 中的 Total_waits
Times(s)	等待的时间	V$SYSTEM_EVENT 中的 Time_waited_micro(s)
Avg wait (ms)	平均等待时间	V$SYSTEM_EVENT 中的 Average_wait_micro(ms)
% DB time	等待时间和 db time 的比例	V$SYSTEM_EVENT 中的 Time_waited_micro / db time
Wait Class	等待类别	Wait_class

21.4.1.3 Main Report

21.4.1.3.1 Wait Events Statistics

21.4.1.3.1.1 Time Model Statistics

记录各类型任务占用 CPU 时间，每个字段的含义和说明参见下表。

表 21.9 Time Model Statistics 说明

字段名	说明	计算值
Statistic Name	统计名称	根据 V$SYSSTAT 细分的时间类型划分
Time(s)	各种类型任务所占用的 CPU 时间	\
% of DB Time	该任务所占用的 CPU 时间占总的 DB TIME 的时间比	Time/DB Time

21.4.1.3.1.2 Operating System Statistics

记录起始快照与结束快照之间的系统信息，每个字段的含义和说明参见下表。

表 21.10 Operating System Statistics 说明

字段名	说明	计算值
Statistic	统计名称	V$SYSTEMINFO 系统信息
Value	起始快照对应统计值	\
End Value	结束快照对应统计值	仅在与起始快照对应值不同时进行展示

21.4.1.3.1.3 Foreground Wait Class

记录等待事件类型的次数与所用时间，每个字段的含义和说明参见下表。

表 21.11 Foreground Wait Class 说明

字段名	说明	计算值
Wait class	等待事件类别	V$SYSTEM_EVENT 中 WAIT_CLASS
Waits	各种类型事件发生次数	V$SYSTEM_EVENT 中 TOTAL_WAITS
%Time -outs	各种类型事件所占用的 CPU 时间占总的 DB CPU 的时间比	V$SYSTEM_EVENT 中的 MAX(TIME_WAITED_MICRO)/DB CPU
Total Wait Ti	各种类型事件所占用的总的 CPU 时间	V$SYSTEM_EVENT 中的 MAX(TIME_WAIT

me（s）		ED_MICRO）
Avg Wait(ms)	各种类型事件的平均 CPU 时间	Total Wait Time/ Waits
%DB time	各种类型事件所占用的 CPU 时间占总的 DB TIME 的时间比	Total Wait Time/DB TIME

21.4.1.3.1.4 Foreground Wait Events

记录等待事件的次数与所用时间，每个字段的含义和说明参见下表。

表 21.12 Foreground Wait Events 说明

字段名	说明	计算值
Event	事件名称	V$SYSTEM_EVENT 中 EVENT
Waits	该事件等待次数	V$SYSTEM_EVENT 中 TOTAL_WAITS
%Time -outs	各种类型事件所占用的 CPU 时间占总的 DB CPU 的时间比	Total Wait Time/DB CPU
Total Wait Time (s)	事件所占用的总的 CPU 时间	V$SYSTEM_EVENT 中 TIME_WAITED_MICROSS
Avg Wait（ms）	事件的平均 CPU 时间	Total Wait Time/ Waits
Waits /txn	每个事务发生的等待次数总的事务个数	Waits / transaction total count
%DB time	事件所占用的 CPU 时间占总的 DB TIME 的时间比	Total Wait Time/DB TIME

21.4.1.3.1.5 Wait Event Histogram

事件的等待次数与具体等待时间占比，每个字段的含义和说明参见下表。

表 21.13 Wait Event Histogram 说明

字段名	说明	计算值
Event	事件名称	V$SESSION_WAIT_HISTORY 中 EVENT
Total waits	等待次数	V$SESSION_WAIT_HISTORY 中 EVENT, COUNT(*)
% of Waits	以<1ms 为例，表示等待时间<1ms 的百分比	V$SESSION_WAIT_HISTORY 里每种事件的等待时间<1ms 的等待次数/总的等待事件次数

21.4.1.3.2 Sql Statistics

21.4.1.3.2.1 SQL ordered by Elapsed Time

sql、pl/sql 按执行时间降序排序。每个字段的含义和说明参见下表。

表 21.14 SQL ordered by Elapsed Time 说明

字段名	说明	计算值
Elapsed Time（s）	该条语句执行时间	V$SQL_STAT_HISTORY 的 exec_time
Executions	该条语句执行次数	V$SQLTEXT 中 N_EXEC
Elapsed Time per Exec（s）	每次执行的时间	Elapsed Time / Executions
%Total	执行时间占 DB TIME 总时间的百分比	Elapsed Time / DB TIME
%CPU	每条 sqlcpu 时间占执行时间的百分比	V$SESSION_WAIT_HISTORY 中 TIME_WAITED_MICRO/Elapsed Time
%IO	每条 sqlIO 时间占执行时间的百分比	V$SQL_STAT_HISTORY 中 IO_WAIT_TIME/Elapsed Time
SQL Id	语句编号	V$SQL_STAT_HISTORY 中 SQL_ID
SQL Text	SQL 语句内容	V$SQLTEXT 中 SQL_TEXT

21.4.1.3.2.2 SQL ordered by CPU Time

sql、pl/sql 按每条语句的 cpu 时间降序排序。每个字段的含义和说明参见下表。

表 21.15 SQL ordered by CPU Time 说明

字段名	说明	计算值
CPU Time（s）	该条语句 cpu 时间	V$SESSION_WAIT_HISTORY 中 TIME_WAITED_MICRO
Executions	该条语句执行次数	V$SQLTEXT 中 N_EXEC
CPU per Exec（s）	每次执行的 cpu 时间	CPU Time（s）/Executions
%Total	该条语句 cpu 时间占总的 cpu 时间的百分比	CPU Time（s）/V$SYSSTAT 中在指定快照范围内的总的 CPU 时间
Elapsed Time（s）	该条语句执行时间	V$SQL_STAT_HISTORY 的 exec_time
%CPU	每条语句 cpu 时间占执行时间的百分比	CPU Time（s）/Elapsed Time（s）
%IO	每条语句 IO 时间占执行时间的百分比	V$SQL_STAT_HISTORY 中 IO_WAIT_TIME/Elapsed Time（s）
SQL Id	语句编号	V$SQL_STAT_HISTORY 中 SQL_ID
SQL Text	SQL 语句内容	V$SQLTEXT 中 SQL_TEXT

21.4.1.3.2.3 SQL ordered by User I/O Wait Time

sql、pl/sql 按 User I/O Wait 时间降序排序。每个字段的含义和说明参见下表。

表 21.16 SQL ordered by User I/O Wait Time 说明

字段名	说明	计算值
User I/O Time（s）	该条语句 User IO wait time	V$SQL_STAT_HISTORY 中 IO_

		WAIT_TIME
Executions	该条语句执行次数	V$SQLTEXT 中 N_EXEC
UIO per Exec (s)	每次执行的 IO wait time	User I/O Time (s)/Executions
%Total	该条语句 cpu 时间占总的 cpu 时间的百分比	CPU Time (s)/V$SYSSTAT 中在指定快照范围内的总的 CPU 时间
Elapsed Time (s)	该条语句执行时间	V$SQL_STAT_HISTORY 的 exec_time
%CPU	每条语句 cpu 时间占执行时间的百分比	V$SESSION_WAIT_HISTORY 中 TIME_WAITED_MICRO/Elapsed Time
%IO	每条语句 IO 时间占执行时间的百分比	V$SQL_STAT_HISTORY 中 IO_WAIT_TIME/Elapsed Time (s)
SQL Id	语句编号	V$SQL_STAT_HISTORY 中 SQL_ID
SQL Text	SQL 语句内容	V$SQLTEXT 中 SQL_TEXT

21.4.1.3.2.4 SQL ordered by Gets

sql、pl/sql 按 buffer gets 时间降序排序。每个字段的含义和说明参见下表。

表 21.17 SQL ordered by Gets 说明

字段名	说明	计算值
Buffer Gets	该条语句 Buffer Gets	V$SQL_STAT_HISTORY LOGIC_READ_CNT
Executions	该条语句执行次数	V$SQLTEXT 中 N_EXEC

Gets per Exec	每次执行的 Buffer Gets	Buffer Gets /Executions
%Total	该条语句 Buffer Gets 占总的 Buffer Gets 的百分比	Buffer Get/ V$SYSSTAT logic read count
Elapsed Time（s）	该条语句执行时间	V$SQL_STAT_HISTORY 的 exec_time
%CPU	每条语句 cpu 时间占执行时间的百分比	V$SESSION_WAIT_HISTORY 中 TIME_WAITED_MICRO/Elapsed Time
%IO	每条语句 IO 时间占执行时间的百分比	V$SQL_STAT_HISTORY 中 IO_WAIT_TIME/Elapsed Time（s）
SQL Id	语句编号	V$SQL_STAT_HISTORY 中 SQL_ID
SQL Text	SQL 语句内容	V$SQLTEXT 中 SQL_TEXT

21.4.1.3.2.5 SQL ordered by Reads

sql、pl/sql 按 physical reads 降序排序。每个字段的含义和说明参见下表。

表 21.18 SQL ordered by Reads 说明

字段名	说明	计算值
Physical Reads	该条语句 Physical Reads	V$SQL_STAT_HISTORY 中 PHY_READ_CNT
Executions	该条语句执行次数	V$SQLTEXT 中 N_EXEC
Reads per Exec	每次执行的 Buffer Gets	Physical Reads /Executions
%Total	该条语句 Physical Reads 占总的 Disk Reads 的百分比	Physical Reads/ 总的 Disk Reads 的次数
Elapsed Time（s）	该条语句执行时间	V$SQL_STAT_HISTORY 的 exec_time
%CPU	每条语句 cpu 时间占执行时间的百分比	V$SESSION_WAIT_HISTORY 中

		TIME_WAITED_MICRO/Elapsed Time
%IO	每条语句 IO 时间占执行时间的百分比	V$SQL_STAT_HISTORY 中 IO_WAIT_TIME/Elapsed Time（s）
SQL Id	语句编号	V$SQL_STAT_HISTORY 中 SQL_ID
SQL Text	SQL 语句内容	V$SQLTEXT 中 SQL_TEXT

21.4.1.3.2.6 SQL ordered by Executions

sql、pl/sql 按执行次数降序排序。每个字段的含义和说明参见下表。

表 21.19 SQL ordered by Executions 说明

字段名	说明	计算值
Executions	该条 SQL 执行次数	V$SQLTEXT 中 N_EXEC
Rows Processed	该条 SQL 影响行数	V$SQL_HISTORY 中 AFFECTED_ROWS
Rows per Exec	每次执行影响行数	Rows Processed/ Executions
Elapsed Time（s）	该条语句执行时间	V$SQL_STAT_HISTORY 的 exec_time
%CPU	每条语句 cpu 时间占执行时间的百分比	V$SESSION_WAIT_HISTORY 中 TIME_WAITED_MICRO/Elapsed Time
%IO	每条语句 IO 时间占执行时间的百分比	V$SQL_STAT_HISTORY 中 IO_WAIT_TIME/Elapsed Time（s）

SQL Id	语句编号	V$SQL_STAT_HISTORY 中 SQL_ID
SQL Text	SQL 语句内容	V$SQLTEXT 中 SQL_TEXT

21.4.1.3.2.7 SQL ordered by Parse Calls

sql、pl/sql 按解析次数降序排序。每个字段的含义和说明参见下表。

表 21.20 SQL ordered by Parse Calls 说明

字段名	说明	计算值
Parse Calls	该条语句解析次数	V$SQL_STAT_HISTORY 中 HARD_PARSE_CNT
Executions	该条语句执行次数	V$SQLTEXT 中 N_EXEC
%Total Parses	该条语句分析次数占总的解析次数的百分比	Parse Calls/V$SYSSTAT 中 pl/sql parse count
SQL Id	语句编号	V$SQL_STAT_HISTORY 中 SQL_ID
SQL Text	SQL 语句内容	V$SQLTEXT 中 SQL_TEXT

21.4.1.3.2.8 SQL ordered by Sharable Memory

sql、pl/sql 按共享内存大小降序排序。每个字段的含义和说明参见下表。只显示共享内存>1048576 字节的字段。

表 21.21 SQL ordered by Sharable Memory 说明

字段名	说明	计算值
Sharable Mem(b)	该条语句共享内存	V$LARGE_MEM_SQLS 中 MEM_USED_BY_K，单位 KB
Executions	该条语句执行次数	V$SQLTEXT 中 N_EXEC

%Total	该条语句共享内存占总的共享内存的百分比	Sharable Mem(b)/总的共享内存
SQL Id	语句编号	V$SQL_STAT_HISTORY 中 SQL_ID
SQL Text	SQL 语句内容	V$SQLTEXT 中 SQL_TEXT

21.4.1.3.2.9 SQL Plan ordered by Cost

sql、pl/sql 按代价大小降序排序。每个字段的含义和说明参见下表。

表 21.22 SQL Plan ordered by Cost 说明

字段名	说明	计算值
Pln_addr(b)	计划地址	V$SQL_PLAN_NODE 中 PLN_ADDR
Node_addr	节点地址	V$SQL_PLAN_NODE 中 NODE_ADDR
Operation	操作符名称	V$SQL_PLAN_NODE 中 OPERATION
Depth	节点层次	V$SQL_PLAN_NODE 中 DEPTH
SQL Id	语句编号	V$SQL_PLAN_NODE 中 中 SQL_ID
card	行数	V$SQL_PLAN_NODE 中 CARD
cost	代价	V$SQL_PLAN_NODE 中 COST
cpu_cost	CPU 代价	V$SQL_PLAN_NODE 中 CPU_COST
io_cost	IO 代价	V$SQL_PLAN_NODE 中

		IO_COST
SQL Text	SQL 语句内容	V$SQLTEXT 中 SQL_TEXT

21.4.1.3.2.10 SQL Node ordered by Time

sql、pl/sql 按节点执行耗时降序排序。每个字段的含义和说明参见下表。

表 21.23 SQL Node ordered by Time 说明

字段名	说明	计算值
seq_no	操作符在计划中的序号	V$SQL_NODE_HISTORY 中 SEQ_NO
exec_id	执行 ID	V$SQL_NODE_HISTORY 中 EXEC_ID
node	节点 ID	V$SQL_NODE_HISTORY 中 NODE
time_used(us)	节点执行耗时	V$SQL_NODE_HISTORY 中 TIME_USED,单位微秒
SQL Id	语句编号	V$SQLTEXT 中 SQL_ID
n_enter	节点进入次数	V$SQL_NODE_HISTORY 中 N_ENTER
name	代价	V$SQL_NODE_NAME 中 NAME
SQL Text	SQL 语句内容	V$SQLTEXT 中 SQL_TEXT

21.4.1.3.2.11 DMSQL ordered by exec time

sql、pl/sql 按执行时间降序排序。每个字段的含义和说明参见下表。

表 21.24 DMSQL ordered by exec time 说明

字段名	说明	计算值

exec_id	语句执行号	V$DMSQL_EXEC_TIME 中 EXEC_ID
type$	语句类型	V$DMSQL_EXEC_TIME 中 TYPE$
seq	相对序号	V$DMSQL_EXEC_TIME 中 SEQ
level	执行层次	V$DMSQL_EXEC_TIME 中 LEVEL
caller	调用方法名	V$DMSQL_EXEC_TIME 中 CALLER
method	方法名	V$DMSQL_EXEC_TIME 中 METHOD
exec_cnt	执行次数	V$DMSQL_EXEC_TIME 中 EXEC_CNT
last_exec_tick	最后一次执行时戳	V$DMSQL_EXEC_TIME 中 LAST_EXEC_TICK，单位微秒
time_used(ms)	执行时间	V$DMSQL_EXEC_TIME 中 LAST_EXEC_TICK，单位毫秒
SQL Text	SQL 语句内容	V$SQLTEXT 中 SQL_TEXT

21.4.1.3.3 Instance Activity Statistics

21.4.1.3.3.1 System Statistics

记录统计对象在两快照间的统计信息。每个字段的含义和说明参见下表。

表 21.25 System Statistics 说明

字段名	说明	计算值
Statistic Name	统计对象名	V$SYSSTAT 中 NAME
Class Id	统计对象所属类别	V$SYSSTAT 中 CLASSID

ID	统计对象 ID	V$SYSSTAT 中 ID
Max Stat Val	最大统计值	V$SYSSTAT 中 MAX(STAT_VAL)
Min Stat Val	最小统计值	V$SYSSTAT 中 MIN(STAT_VAL)
Avg Stat Val	平均统计值	V$SYSSTAT 中 AVG(STAT_VAL)
Total	指定快照范围内统计值总计	V$SYSSTAT 中 MAX(STAT_VAL)-MIN(STAT_VAL)
Per Second	每秒统计值	Total/Elapsed
Per Trans	每事务统计值	Total/Transaction

21.4.1.3.3.2 VM Statistics

记录活动的虚拟机信息。每个字段的含义和说明参见下表。

表 21.26 VM Statistics 说明

字段名	说明	计算值
id	虚拟机 ID	V$VIRTUAL_MACHINE 中 ID
sessid	会话 ID	V$VIRTUAL_MACHINE 中 SESSID
stmt_id	语句 ID	V$VIRTUAL_MACHINE 中 STMT_ID
exp_flag	表达式标识	V$VIRTUAL_MACHINE 中 EXP_FLAG
vstack_size	栈大小	V$VIRTUAL_MACHINE 中 VSTACK_SIZE
vtop	栈顶离基址距离	V$VIRTUAL_MACHINE 中 VTOP
vused	栈已用空间大小	V$VIRTUAL_MACHINE 中 VUSED
stkfrm_depth	栈深度	V$VIRTUAL_MACHINE 中 STKFRM_DEPTH
vstack	栈基址	V$VIRTUAL_MACHINE 中 VSTACK
free_stkfrms	已释放的栈	V$VIRTUAL_MACHINE 中 FREE_STKFRM
curr_frm	当前栈	V$VIRTUAL_MACHINE 中 CURR_FRM

ip	IP 地址	V$VIRTUAL_MACHINE 中 IP
sql_no	SQL 语句树的信号	V$VIRTUAL_MACHINE 中 SQL_NO
reuse_cnt	虚拟机重用次数	V$VIRTUAL_MACHINE 中 REUSE_CNT

21.4.1.3.3.3 Session history Statistics

记录指定快照范围内会话历史的记录信息，属性列均源于动态视图 V$SESSION_HISTORY。

21.4.1.3.4 IO Stats

21.4.1.3.4.1 Tablespace IO Stats

统计各表空间 IO 信息，根据读写次数降序排序。每个字段的含义和说明参见下表。

表 21.27 Tablespace IO Stats 说明

字段名	说明	计算值
Tablespace	表空间名称	V$TABLESPACE 中 NAME
Reads	读页	V$DATAFILE 中 PAGES_READ，根据表空间 ID 进行 SUM
Av Reads/s	每秒读页	Reads/DB time
Writes	写页	V$DATAFILE 中 PAGES_WRITE，根据表空间 ID 进行 SUM
Av Writes/s	每秒写页	Writes/DB time

21.4.1.3.4.2 File IO Stats

记录数据库文件信息，每个字段的含义和说明参见下表。

表 21.28 File IO Stats 说明

字段名	说明	计算值
Tablespace	表空间名称	V$TABLESPACE 中 NAME

Path	数据库文件路径	V$DATAFILE 中 PATH
Reads	读页	V$DATAFILE 中 PAGES_READ
Av Reads/s	每秒读页	Reads/DB time
Writes	写页	V$DATAFILE 中 PAGES_WRITE
Av Writes/s	每秒写页	Writes/DB time

21.4.1.3.5 Buffer Pool Statistics

21.4.1.3.5.1 Buffer Pool Statistics

记录缓冲区动态性能信息，每个字段的含义和说明参见下表。

表 21.29 Buffer Pool Statistics 说明

字段名	说明	计算值
Name	缓冲区名称	V$BUFFERPOOL 中 NAME
Id	缓冲区 ID	V$BUFFERPOOL 中 ID
Page Size	基缓冲区页大小	V$BUFFERPOOL 中 PAGES_SIZE
Pages	页数	V$BUFFERPOOL 中 N_PAGES
Total Pages	页大小	V$BUFFERPOOL 中 N_TOTAL_PAGES
Max Pages	最多的页数	V$BUFFERPOOL 中 N_MAX_PAGES
Fixed	数据页被引用的次数	V$BUFFERPOOL 中 N_FIXED
Pool Hit %	READ 命中次数占比	V$BUFFERPOOL 中 N_LOGIC_READS/N_LOGIC_READS+N_PHY_READS
Logical Reads	READ 命中次数	V$BUFFERPOOL 中 N_LOGIC_READS
Physical Reads	READ 未命中次数	V$BUFFERPOOL 中 N_PHY_READS
Exp Bufferpools	扩展缓冲区个数	V$BUFFERPOOL 中 N_EXP_BUFFERPOOL

21.4.1.3.5.2 Checkpoint Activity

显示检查点历史信息，每个字段的含义和说明参见下表。

表 21.30 Checkpoint Activity 说明

字段名	说明	计算值
Thread Id	线程 ID	V$CKPT_HISTORY 中 THREAD_ID
Ckpt Cmd	检查点	V$CKPT_HISTORY 中 CKPT_CMD
Start Time	基缓冲区页大小	V$CKPT_HISTORY 中 START_TIME
Total Used Time(s)	页数	V$CKPT_HISTORY 中 TIME_USED
Total Flused Pages	页大小	V$CKPT_HISTORY 中 PAGE_FLUSHED
Min Ckpt Lsn	最多的页数	V$CKPT_HISTORY 中 MIN(CKPT_LSN)
Max Ckpt Lsn	数据页被引用的次数	V$CKPT_HISTORY 中 MAX(CKPT_LSN)

21.4.1.3.5.3 Rlog Statistics

显示日志的总体信息，每个字段的含义和说明参见下表。

表 21.31 Rlog Statistics 说明

字段名	说明	计算值
ckpt_lsn	最近一次检查点 LSN	V$RLOG 中 CKPT_LSN
file_lsn	已经到盘上的 LSN	V$RLOG 中 FILE_LSN
flush_lsn	当前准备刷盘的 LSN	V$RLOG 中 FLUSH_LSN
cur_lsn	当前的 LSN	V$RLOG 中 CUR_LSN
next_seq	下一个日志包序号	V$RLOG 中 NEXT_SEQ
flush_pages	Flush 链表中的总页数	V$RLOG 中 FLUSH_PAGES
flushing_pages	正在刷盘的总页数	V$RLOG 中 FLUSHING_PAGES
free_space	可用的日志空间	V$RLOG 中 FREE_SPACE，单位为字节

total_space	日志总空间	V$RLOG 中 TOTAL_SPACE，单位为字节
suspend_time	挂起时间戳	V$RLOG 中 SUSPEND_TIME
upd_ctl_lsn	系统修改控制文件的 RLOG_RESERVE_THRESHOLD ->lsn	V$RLOG 中 UPD_CTL_LSN
n_reserve_wait	日志空间预申请等待个数	V$RLOG 中 N_RESERVE_WAIT

21.4.1.3.6 Wait Statistics

21.4.1.3.6.1 Wait History

记录指定快照范围内等待事件的具体信息，属性列均源于动态视图 V$WAIT_HISTORY。

21.4.1.3.6.2 Enqueue Activity

记录当前归档任务队列信息，每个字段的含义和说明参见下表。

表 21.32 Enqueue Activity 说明

字段名	说明	计算值
queue_name	归档类型	V$ARCH_QUEUE 中 ARCH_TYPE
readys	已处理任务数	V$ARCH_QUEUE 中 READY
total_wait	已处理任务的总等待时间	V$ARCH_QUEUE 中 TOTAL_WAIT_TIME
avg_wait	已处理任务的平均等待时间	total_wait/readys

21.4.1.3.7 Undo Statistics

21.4.1.3.7.1 Pseg Items

记录回滚系统中回滚项信息，每个字段的含义和说明参见下表。

表 21.33 Pseg Items 说明

字段名	说明	计算值
Snap Id	快照 ID	\
Total Pages	总回滚页数	V$PSEG_ITEMS 中 SUM(N_PAGES)
Total Extends	总扩展回滚段次数	V$PSEG_ITEMS 中 SUM(N_EXTEND)
Total Purge Pages	总 purge 页数	V$PSEG_ITEMS 中 SUM(N_PURGED_PAGES)
Total Used Pages	总正在使用的页数	V$PSEG_ITEMS 中 SUM(N_USED_PAGES)
Total Urec Used(MB)	总生成回滚记录的总字节数	V$PSEG_ITEMS 中 SUM(N_UREC_BYTES)
Total Commit Trx	总已经提交未 PURGE 的事务数	V$PSEG_ITEMS 中 SUM(N_COMMIT_TRX)
Max Reserve Time	最大最长的保留时间，单位秒	V$PSEG_ITEMS 中 MAX(RESERVE_TIME)
Free Stack Total Pages	最小空闲回滚页堆栈空间大小，单位页数	V$PSEG_ITEMS 中 MIN(STACK_SIZE)
Free Stack Free Pages	最小空闲回滚页堆栈中空闲页数	V$PSEG_ITEMS 中 MIN(STACK_SP)

21.4.1.3.7.2 Undo Segment Summary

记录系统中回滚项相关信息，每个字段的含义和说明参见下表。

表 21.34 Undo Segment Summary 说明

字段名	说明	计算值
Num Undo Blocks	当前回滚项的回滚块数	V$PSEG_ITEMS 中 N_PAGES
Number of Transactions	当前回滚表空间上执行过的事务	V$PSEG_ITEMS 中 N_TRXS

	总数	
Max Exec Len	持续最久的查询时间	V$LONG_EXEC_SQLS 中 MAX(EXEC_TIME)
Max Tx Concy	最大事务并发量	V$TRX 中 MAX(COUNT(ID))
Min/Max TR (mins)	最小和最大的 undo retention	V$PSEG_ITEMS 中 MAX(TUNED_UNDO_RETENTION) /MIN(TUNED_UNDO_RETENTION)

21.4.1.3.8 Lock Statistics

记录活动的事务锁信息，每个字段的含义和说明参见下表。

表 21.35 Lock Statistics 说明

字段名	说明	计算值
addr	锁地址	V$LOCK 中 ADDR
trx_id	所属事务 ID	V$LOCK 中 TRX_ID
ltype	锁类型	V$LOCK 中 LTYPE
lmode	锁模式	V$LOCK 中 LMODE
blocked	是否阻塞	V$LOCK 中 BLOCKED
table_id	对于对象锁，表示表对象或字典对象的 ID；对于 TID 锁，表示封锁记录对应的表 ID	V$LOCK 中 TABLE_ID
row_idx	TID 锁封锁记录行信息	V$LOCK 中 ROW_IDX

21.4.1.3.9 Dead Lock Statistics

21.4.1.3.9.1 Dead Lock Stats

记录死锁的历史信息，每个字段的含义和说明参见下表。

表 21.36 Dead Lock Stats 说明

字段名	说明	计算值
SeqNo	编号	V$DEADLOCK_HISTORY 中 SEQNO
trx_id	事务 ID	V$DEADLOCK_HISTORY 中 TRX_ID
Sess Id	会话 ID	V$DEADLOCK_HISTORY 中 SESS_ID
Sess Seq	会话序列号	V$DEADLOCK_HISTORY 中 SESS_SEQ
Happen Time	死锁发生时间	V$DEADLOCK_HISTORY 中 HAPPEN_TIME
Sql Text	产生死锁的 SQL 语句	V$DEADLOCK_HISTORY 中 SQL_TEXT

21.4.1.3.10 Seg Activity Statistics

记录指定快照范围内统计对象 ID 在 98 至 106 区间内的具体信息，属性列均源于动态视图 V$SYSSTAT。

21.4.1.3.11 Dictionary Cache Statistics

记录两快照间数据字典的实时信息，每个字段的含义和说明参见下表。

表 21.37 Dictionary Cache Statistics 说明

字段名	说明	计算值
Snap Id	快照编号	\
Total Size(MB)	总缓存池空间，单位字节	V$DB_CACHE 中 SUM(TOTAL_SIZE)
Used %	总实际使用空间，单位字节	V$DB_CACHE 中 SUM(USED_SIZE)/SUM(TOTAL_SIZE)
Dict Num	总缓存池中字典对象	V$DB_CACHE 中 SUM(DICT_NUM)
Total LRU Discards Used(MB)	总被淘汰字典对象空间的总和，单位字节	V$DB_CACHE 中 SUM(SIZE_LRU_DISCARD)

Total LRU Discards	总字典对象被淘汰次数	V$DB_CACHE 中 SUM(LRU_DISCARD)
DDL Discards	总 DDL 操作导致字典对象被淘汰的次数	V$DB_CACHE 中 SUM(DDL_DISCARD)

21.4.1.3.12 Library Cache Statistics

记录两快照间缓存池信息，每个字段的含义和说明参见下表。

表 21.38 Library Cache Statistics 说明

字段名	说明	计算值
Pln_addr	缓存池地址	V$SCP_CACHE 中 SCP_ADDR
Pln_num	计划缓存总数	V$SCP_CACHE 中 PLN_CNT
Sql_num	SQL 缓存总数	V$SCP_CACHE 中 SQL_CNT
RS_num	结果集缓存总数	V$SCP_CACHE 中 RS_CNT
Discard_Mem	缓存对象淘汰大小	V$SCP_CACHE 中 DISCARD_MEM
Discard_cnt	缓存对象淘汰次数	V$SCP_CACHE 中 DISCARD

21.4.1.3.13 Memory Statistics

记录指定快照范围内所有内存池的具体信息，属性列均源于动态视图 V$MEM_POOL。

21.4.1.3.14 Sort Statistics

21.4.1.3.14.1 Hash Merge Used History

记录指定快照范围内 HASH MERGE 连接操作符使用的缓存信息，属性列均源于动态视图 V$HASH_MERGE_USED_HISTORY。

21.4.1.3.14.2 Memory Table Used History

显示快照范围内使用 MTAB 空间最多的操作符信息，每个字段的含义和说明参见下表。

表 21.39 Memory Table Used History 说明

字段名	说明	计算值
Exec Id	语句执行 ID	V$MTAB_USED_HISTORY 中 EXEC_ID
Op Seq No	操作符在执行计划中序号	V$MTAB_USED_HISTORY 中 OP_SEQ_NO
Op Type	节点名字	V$SQL_NODE_NAME 中 NAME
Mtab Used(MB)	操作符使用的 MTAB 空间，单位 MB	V$MTAB_USED_HISTORY 中 MTAB_USED_BY_M
Sql Text	SQL 文本	V$MTAB_USED_HISTORY 中 SQL_TEXT

21.4.1.3.14.3 Sort History

记录指定快照范围内当 INI 参数 ENABLE_MONITOR=1 都打开时，显示系统自启动以来使用排序页数最多的 50 个操作符信息，属性列均源于动态视图 V$SORT_HISTORY。

21.4.1.3.15 Resource Limit Statistics

记录指定起始快照与结束快照对应的表的空间限制信息，属性列均源于动态视图 V$RESOURCE_LIMIT。

21.4.1.3.16 Mail Statistics

记录指定快照范围内 mail 邮箱视图信息，属性列均源于动态视图 V$MAL_INFO。

21.4.1.3.17 dm.ini Parameters

显示快照范围内 ini 参数和 dminit 建库参数信息，每个字段的含义和说明参见下表。

表 21.40 dm.ini Parameters 说明

字段名	说明	计算值
Parameter Name	参数名称	V$DM_INI 中 PARA_NAME
Begin value	起始快照系统参数值	V$DM_INI 中 PARA_VALUE
End value (if differ	结束快照系统参数值	V$DM_INI 中 PARA_VALUE，仅与 Begin value

ent)		不同时显示

21.4.1.3.18 Top SQL with Top Events

显示采样的 SQL 语句及其对应的等待事件相关信息，来源 V$ACTIVE_SESSION_HISTORY。由于动态视图的采样周期与快照生成周期不一致，故此表仅分析在指定快照范围对应的采样时间的采样数据。每个字段的含义和说明参见下表。

表 21.41 Top SQL with Top Events 说明

字段名	说明	计算值
SQL ID	SQL 语句 ID	V$ACTIVE_SESSION_HISTORY 中 SQL_ID
Samples	SQL 语句的采样次数	V$ACTIVE_SESSION_HISTORY 中 COUNT(SQL_ID)
%Activity	SQL执行时间对DB Time的占比	V$SQL_STAT_HISTORY 中 SUM(EXEC_TIME) / DB TIME
Event	SQL 语句对应等待事件	V$ACTIVE_SESSION_HISTORY 中 EVENT
Waits	事件等待次数	V$SYSTEM_EVENT 中 TOTAL_WAITS，取结束快照与起始快照的差值
%Event	平均等待时间对 DB Time 的占比	V$SYSTEM_EVENT 中 AVERAGE_WAIT_MICRO / DB Time
Sql Text	采样的 SQL 语句	V$SQL_STAT_HISTORY 中 SQL_TXT

21.4.1.3.19 Top Events

Top Events 分为以下几个模块：顶级用户事件（Top User Events）、（顶级后台事件）Top Background Events、（顶级等待事件参数）Top EVENT P1/P2/P3/P4 Values。

21.4.1.3.19.1 Top User Events

Top User Events 显示顶级用户事件，其中有以下几项需要注意：

%Event：类名为 User I/O, Concurrency, Commit, Transaction, Network 中所有等待事件数量 / 起始快照之间所有的等待事件数量。

Avg Wait Time：平均等待时间。从 WRH$_ACTIVE_SESSION_HISTORY 信息表中，类名 WAIT_CLASS 在 User I/O, Concurrency, Commit, Transaction, Network 中根据事件数量排序取出前 30 条，再从 WRH$_SYSTEM_EVENT 表中限制 EVENT IS NOT NULL 即事件类型为等待事件，根据事件名称 EVENT 取出各事件名称在 START_SNAP_ID 以及 END_SNAP_ID 期间的对应的平均等待时间 AVERAGE_TIME_MICRO（单位微秒）。若查询时 SYS.WRH$_SYSTEM_EVENT 中不包含所查询的快照数据，查询数据失败会导致该字段全部写入'/ '。

 注意：

WRH$_SYSTEM_EVENT 信息表记录的是系统启动以来所有等待事件的信息，最后的报表按照等待事件数量进行排序。

21.4.1.3.19.2 Top Background Events

显示顶级后台事件。在 WRH$_ACTIVE_SESSION_HISTORY 信息表中，限制字段 SESSION_TYPE = BACKGROUND 代表收集到事件都为 Background，，再从 WRH$_SYSTEM_EVENT 表中限制 EVENT IS NOT NULL 即事件类型为等待事件，根据事件名称 EVENT 取出各事件名称在 START_SNAP_ID 以及 END_SNAP_ID 期间的对应的平均等待时间 AVERAGE_TIME_MICRO（单位微秒）。

21.4.1.3.19.3 Top Event P1/P2/P3/P4 Values

显示顶级等待事件的相关参数，来源于 WRH$_ACTIVE_SESSION_HISTORY 表。根据起始快照 ID，取值 WRH$_ACTIVE_SESSION_HISTORY 中的 P1/P1TEXT，对应 P1/Parameter1 列,分别表示等待事件的参数和参数取值。同样的还有 P2,P3,P4。

21.4.1.3.20 Load Profile

该模块包括顶级 SQL 语句类型（Top SQL Command Types）。

Top SQL Command Types 显示采样的 SQL 语句类型以及相关信息，来源于 V$ACTIVE_SESSION_HISTORY，详见下表。

表 21.42 Top SQL Command Types 说明

字段名	说明	计算值
SQL Command Type	SQL 语句类型	V$ACTIVE_SESSION_HISTORY 中 SQL_OPNAME，对应 WRH$_ACTIVE_SESSION_HISTORY 信息表中的 SQL 语句类型
Distinct SQLIDS	去重后的 SQL 语句的采样次数	V$ACTIVE_SESSION_HISTORY 中 COUNT(DISTINCT(SQL_ID))，执行的语句 ID 数量,实现逻辑中进行去重
%Activity	分组 SQL 语句采样次数/去重后会话数	V$ACTIVE_SESSION_HISTORY 中分组 COUINT(SQL_ID) /COUNT(DISTINCT(SESSION_ID) 即分组 SQL 语句采样次数/去重后会话数，报表也根据该值进行排序
Avg Active Sessions	平均活跃会话数	V$ACTIVE_SESSION_HISTORY 中 COUNT(SQL_ID)/COUNT(DISTINCT(SESSION_ID))，即分组 SQLID 数/去重后的会话 ID 数量

21.4.1.3.21 Top SQL

该模块包括包括对等待事件顶级占比 SQL 语句（TOP SQL WITH TOP EVENTS）。

TOP SQL WITH TOP EVENTS 显示采样的 SQL 语句及其对应的等待事件的相关信息，来源于 WRH$_ACTIVE_SESSION_HISTORY 以及 WRH$_SQL_STAT_HISTORT 信息表。由于动态视图的采样周期与快照生成周期不一致，故此表仅分析在指定快照范围对应的采样时

间的采样数据。每个字段的含义和说明参见下表。

表 21.43 Top SQL with Top Events 说明

字段名	说明	计算值
SQL ID	SQL 语句 ID	WRH$_ACTIVE_SESSION_HISTORY 中 SQL_ID
Samples	SQL 语句的采样次数	WRH$_ACTIVE_SESSION_HISTORY 中 COUNT(SQL_ID)，限制 EVENT IS NOT NULL,并且是在同一个 EVENT 事件中
%Activity	SQL 语句采样次数对执行总次数的占比	WRH$_ACTIVE_SESSION_HISTORY 中 COUNT(SQL_ID) / 快照期间该 SQL_ID 在 WRH$_SQL_STAT_HISTORY 中的 COUNT(SQL_ID) 若 WRH$_SQL_STAT_HISTORY 中不存在该 SQL_ID 则输出‘/’,且 SQL Text 也为‘/’
Event	SQL 语句对应等待事件	WRH$_ACTIVE_SESSION_HISTORY 中 EVENT
Waits	事件等待时间，单位为微秒	WRH$_ACTIVE_SESSION_HISTORY 中根据 SQL_ID,EVENT,SESSION_ID,SQL_EXEC_ID 分组得到 MAX(TIME_WAITED)，除以 1000 输出，若为 0 则输出‘/’ 报表根据该值进行排序
%Event	事件等待时间对 DB Time 的占比	Waits/DB Time 若 DB_TIME 或 Waits 为 0，输出‘/’
Sql Text	采样的 SQL 语句	WRH$_SQL_STAT_HISTORY 中 SQL_TXT

21.4.1.3.22 Top Sessions

包括顶级会话（Top Sessions）和顶级阻塞会话（Top Blocking Sessions）。

21.4.1.3.22.1 Top Sessions

显示采样时的会话以及相关信息，数据来源于 DBA_HIST_ACTIVE_SESS_HISTORY 以及 DBA_USER 系统视图，通过 USER_ID 进行连接，限定 SQL_ID 列所标识的语句为正在执行状态，并且为会话在采样时的等待事件，根据会话 ID，会话序列号，事件名称，用户名称，程序名进行分组，根据每组会话总数进行排序。每个字段说明具体见下表。

表 21.44 Top Sessions 说明

字段名	说明	计算值
Sid,Serial#	会话 ID 以及序列号	DBA_HIST_ACTIVE_SESS_HISTORY 中的 SESSION_ID 拼接 SESSION_SERIAL#
%Activity	会话对总会话占比	DBA_HIST_ACTIVE_SESS_HISTORY 中分组的会话数/总会话数
Event	事件名称	DBA_HIST_ACTIVE_SESS_HISTORY 中 EVENT
%Event	事件数对总事件数占比	DBA_HIST_ACTIVE_SESS_HISTORY 中按照上述分组后的每组事件数/总事件数
User	用户名称	DBA_USER 中 USERNAME
Program	程序名	DBA_HIST_ACTIVE_SESS_HISTORY 中 PROGRAM
#Sample Active	样本活跃度	ELAPSED 作为起始快照的时间差，若 ELAPSED 为 0 为每组会话数 '/' 总会话数 || [0%] 否则最后拼接的为[AVG(TIME_WAITED)/1000000/ELAPSED * 100 '%']

21.4.1.3.22.2 Top Blocking Sessions

显示阻塞会话的情况，数据来源于 WRH$_SESSION_WAIT_HISTORY 快照信息表,该表

显示所有快照中会话等待时间的历史信息，按照会话 ID 以及事件名称分组,逻辑部分与 Top Sessions 一致，报表根据%Activity 进行排序。下表为该信息表内容。

表 21.45 WRH$_SESSION_WAIT_HISTORY 快照信息表说明

列名	类型	说明
SNAP_ID	INTEGER	快照 ID
SQL_ID	INTEGER	语句的 SQL ID
EVENT#	SMALLINT	事件编号
EVENT	VARCHAR(32)	事件名称
TIME_WAITED_MICRO	INTEGER	等待时间（单位：微秒）
WAIT_CLASS#	SMALLINT	等待事件类别编号
SESSION#	INTEGER	会话 ID

21.4.1.3.23 Activity Over Time

显示数据库随时间推移的活动情况，提供了期间活动以及工作负载的信息。将起始快照之间的采样数据，以 2 分钟为一个窗口切割成连续的时间窗口，并且统计每个窗口的活动情况,每个窗口只统计前三项最活跃的事件，数据来源于 WRH$_ACTIVE_SESSION_HISTORY 信息表，详见下表。

表 21.46 Activity Over Time 说明

字段名	说明	计算值
Slot Time	当前窗口的结束时间	每个窗口结束时间由快照对应时间开始逐步加 2mins，若剩余时间不足两分钟，则使用剩余时间作为 slot
Slot Count	窗口期间采样 ID 总数量	WRH$_ACTIVE_SESSION_HISTORY 中 COUNT(SAMPLE_ID)

Event	事件名称	WRH$_ACTIVE_SESSION_HISTORY 中 EVENT
Event Count	会话在采样时的等待事件的数量	WRH$_ACTIVE_SESSION_HISTORY 中 COUNT(EVENT) WHERE EVENT IS NOT NULL 报表根据该值进行排序
%Event	在采样时等待事件数量对快照起始间分组采样 ID 总数量的占比	Event Count / COUNT(SAMPLE_ID) * 100

21.4.2 快照存储说明

快照信息管理表 WRM$_SNAPSHOT 中 SNAP_ID 为自增列，默认值为 1，每当新建一个快照该参数就自增"1"。快照信息表为水平分区表，且为范围（Range）分区类型，初始分区为 Part0 不存储信息，分区表均存储于 SYSAUX 表空间上。

分区表均以快照 ID 进行分区，新建快照时会在每个分区表末尾新增新的分区 Parti（i 为变量，Part 为常量），其中 i 为当前快照 ID，同时向 Parti 中插入此快照对应的数据。

暂未提供接口查询指定快照的大小。可以使用系统函数 TABLE_USED_SPACE 获取每个水平分区表中指定快照所占用的页大小，如获取 SYS.WRH$_SESSIONS 表中快照 8 所占用的页大小可执行以下 SQL 语句：

```
SELECT TABLE_USED_SPACE('SYS', 'WRH$_SESSIONS_PART8');
```

执行结果如下：

```
行号          TABLE_USED_SPACE('SYS','WRH$_SESSIONS_PART8')

---------- ----------------------------------------------

1             16
```

21.4.3 快照删除说明

目前 AWR 支持四种对快照进行删除的方式：

1） 过期快照定期删除

快照保留时间由快照信息管理表 WRM$_WR_CONTROL 中 RETENTION 列决定，RETENTION 列默认为 8 天，可以使用 MODIFY_SNAPSHOT_SETTINGS 接口进行修改。每次创建新的快照创建新的分区子表前对当前所有快照进行扫描，找到已过期的快照。

2） 指定快照范围删除

调用接口 DROP_SNAPSHOT_RANGE 指定待删除的 LOW_SNAP_ID 与 HIGH_SNAP_ID 对[LOW_SNAP_ID, HIGH_SNAP_ID]范围内的快照进行删除。

3） 指定快照创建时间删除

调用接口 DROP_SNAPSHOT_BY_TIME 指定待删除的快照创建时间 END_DT，对快照开始创建时间小于等于 END_DT 范围内的快照进行删除。

上述删除快照的方式均会将待删除快照存入中间表 SYS.WRM$_SNAPSHOT_D，并立即对 SYS.WRM$_SNAPSHOT 表中待删除快照的数据进行 DELETE。

每间隔 11min 会对中间表 SYS.WRM$_SNAPSHOT_D 中存储的快照 ID 对应的快照信息表中的分区子表进行 DROP，且经过系统 purge 时间后对应空间会真正释放。DROP 操作结束后将已删除的快照从中间表 SYS.WRM$_SNAPSHOT_D 中进行 DELETE。

4） 对全部快照进行删除

调用接口 AWR_CLEAR_HISTORY，对全部快照信息表先进行 TRUNCATE 操作释放空间，再对表进行 DROP 操作。删除后再执行对全部信息表的重建操作，同时对中间表 SYS.WRM$_SNAPSHOT_D 与 SYS.WRM$_SNAPSHOT 表进行数据清理。

21.5 举例说明

用户在使用 DBMS_WORKLOAD_REPOSITORY 包之前，需要提前调用系统过程并设置间隔时间：

```
SP_INIT_AWR_SYS(1);
```

下面语句设置间隔为 10 分钟，也可以是其他值：

```
CALL DBMS_WORKLOAD_REPOSITORY.AWR_SET_INTERVAL(10);
```

设置成功后，可以使用 CREATE_SNAPSHOT 手动创建快照，也可以等待设置的间隔时间后系统自动创建快照，快照 id 从 1 开始递增：

手动创建快照：

```
DBMS_WORKLOAD_REPOSITORY.CREATE_SNAPSHOT();
```

查看创建的快照信息，包括快照 id：

```
SELECT * FROM SYS.WRM$_SNAPSHOT;
```

查看 snapshot 的 id 在 1~2 范围内的 AWR 分析报告的带 html 格式的内容。然后复制到文本文件中，保存成 html 格式即可查看。

```
SELECT * FROM TABLE (DBMS_WORKLOAD_REPOSITORY.AWR_REPORT_HTML(1,2));
```

把 snapshot 的 id 在 1~2 范围内的 AWR 分析报告生成到 c 盘 awr1.html 文件。

```
SYS.AWR_REPORT_HTML(1,2,'C:\','AWR1.HTML');
```

通过 DBMS_WORKLOAD_REPOSITORY 包还可以对快照本身做增删改操作。

例 1 删除 id 在 22~32 之间的 snapshot，并立即释放相应分区表空间。

```
CALL DBMS_WORKLOAD_REPOSITORY.DROP_SNAPSHOT_RANGE(22,32,NULL,TRUE);
```

例 2 删除“2023-11-17 17:01:29.488000”时间点之前创建的所有 snapshot，并立即释放相应分区表空间。

```
CALL DBMS_WORKLOAD_REPOSITORY.DROP_SNAPSHOT_BY_TIME('2023-11-17 17:01:29.488000',
```

```
TRUE);
```

例 3 修改 snapshot 的间隔时间为 30 分钟、保留时间为 1 天。

```
CALL DBMS_WORKLOAD_REPOSITORY.MODIFY_SNAPSHOT_SETTINGS(1440,30);
```

查询设置后快照参数。

```
SELECT * FROM SYS.WRM$_WR_CONTROL;
```

例 4 创建一次 snapshot。

```
CALL DBMS_WORKLOAD_REPOSITORY.CREATE_SNAPSHOT();
```

例 5 清理全部 snapshot。

```
CALL DBMS_WORKLOAD_REPOSITORY.AWR_CLEAR_HISTORY();
```

例 6 设置 snapshot 的间隔为 10 分钟。

```
CALL DBMS_WORKLOAD_REPOSITORY.AWR_SET_INTERVAL(10);
```

例 7 手动清理创建快照时产生的中间表。

```
CALL DBMS_WORKLOAD_REPOSITORY.AWR_CLEAR_TMP_TABLE();
```

例 8 执行下述语句，将 SNAPSHOT 的 ID 在 1~2 范围内的 ASH 分析报告生成到 /home/test 路径下的 ash_test.html 文件中。

```
CALL SYS.ASH_REPORT_HTML(1,2, '/home/test', 'ash_test.html');
```

22 DBMS_XMLGEN 包

DBMS_XMLGEN 系统包用于将 SQL 查询结果转换成 XML 文档。

DBMS_XMLGEN 包的使用依赖 DBMS_LOB 包。在创建 DBMS_XMLGEN 包时，系统会自动创建 DBMS_LOB 包。

22.1 相关方法

1. NEWCONTEXT()

申请上下文句柄。

语法如下:

```
FUNCTION NEWCONTEXT (
    QUERYSTRING      IN      VARCHAR(32767)
) RETURN CTXHANDLE;
或
FUNCTION NEWCONTEXT (
    QUERY            IN      CURSOR
) RETURN CTXHANDLE;
```

参数详解

- QUERYSTRING 查询语句，字符串类型。
- QUERY 查询语句的引用游标。

返回值

ctxHandle：上下文句柄。

2. GETXML()

GETXML()方法用来获取 XML 数据，有以下三种调用方式。

1) 根据指定的最大获取行数，将获取的数据添加在传入的 clob 的内容之后。使用该函数可以重用 clob，避免额外的 clob 复制。

语法如下:

```
FUNCTION GETXML(
    CTX                 IN      INTEGER,
    TMPCLOB             INOUT   CLOB,
    DTDORSCHEMA         IN      INTEGER
) RETURN INTEGER;
```

参数详解

- CXT 之前根据查询语句获取的上下文句柄。
- TMPCLOB 写入 XML 数据的 CLOB。
- DTDORSCHEMA 仅支持 0，缺省为 0，可以不指定。

返回值

总是为 0。

2) 此方法会将获取的 XML 数据放在一个临时生成的 clob 中，并返回这个临时 clob，该临时 clob 必须通过 DBMS_LOB.FREETEMPORARY 进行释放。

语法如下:

```
FUNCTION GETXML(
    CTX                 IN      INTEGER,
    DTDORSCHEMA         IN      INTEGER
) RETURN CLOB;
```

参数详解

- CXT 之前根据查询语句获取的上下文句柄。
- DTDORSCHEMA 仅支持 0，缺省为 0，可以不指定。

返回值

包含获取的 XML 数据的 clob。

3） 此方法根据指定的 SQL 语句，生成内部临时上下文句柄，获取 XML 数据并返回，并释放临时上下文句柄。

语法如下:

```
FUNCTION GETXML(
    SQLQUERY        IN      VARCHAR(32767),
    DTDORSCHEMA     IN      INTEGER
)RETURN CLOB;
```

参数详解

- SQLQUERY 查询语句。
- DTDORSCHEMA 仅支持 0，缺省为 0，可以不指定。

返回值

包含获取的 XML 数据的 clob。

3. GETNUMROWSPROCESSED()

调用 GETXML 函数之后，使用该函数获取实际获取的数据行数，该函数的返回值不包含跳过的数据。

语法如下:

```
FUNCTION GETNUMROWSPROCESSED(
    CTX     IN      INTEGER
)RETURN BIGINT;
```

参数详解

- CTX 操作的上下文句柄。

返回值

GETXML 实际获取的行数。

4. CLOSECONTEXT()

关闭上下文句柄并释放相关资源。

语法如下:

```
PROCEDURE CLOSECONTEXT(
    CTX       IN        INTEGER
)RETURN BIGINT;
```

参数详解

- CTX 待关闭的上下文句柄。

5. CONVERT()

将 XML 数据在是否需要保留特殊字符之间进行转换。

由于 CONVERT 为系统保留字，使用该方法时需要在方法名外添加双引号，即 DBMS_XMLGEN."CONVERT"。

语法如下:

```
FUNCTION CONVERT(
    XMLDATA      IN        VARCHAR(32767),
    FLAG      IN        INTEGER
)RETURN VARCHAR(32767);
```

或

```
FUNCTION CONVERT(
    XMLDATA     IN      CLOB,
    FLAG        IN      INTEGER
)RETURN CLOB;
```

参数详解

- XMLDATA 需要转换的 XML 数据。
- FLAG 转换标记，取值 ENTITY_ENCODE 和 ENTITY_DECODE，缺省为 ENTITY_ENCODE。

返回值

转换之后的数据。

6. RESTARTQUERY()

重新查询。

语法如下:

```
PROCEDURE RESTARTQUERY(
    CTX     IN      INTEGER
);
```

参数详解

- CTX 操作的上下文句柄。

7. SETCONVERTSPECIALCHARS()

设置获取结果集 XML 时特殊字符是否转化。

语法如下:

```
PROCEDURE SETCONVERTSPECIALCHARS(
    CTX     IN      INTEGER,
```

```
    CONV    IN          BOOLEAN
);
```

参数详解

- CTX 操作的上下文句柄。
- CONV 是否要转化特殊字符。

8. SETMAXROWS()

设置每次调用 GETXML 时的最大获取行数。

语法如下:

```
PROCEDURE SETMAXROWS(
    CTX             IN          INTEGER,
    MAXROWS         IN          BIGINT
);
```

参数详解

- CTX 操作的上下文句柄。
- MAXROWS 设置的最大获取行数。

9. SETNULLHANDLING()

设置 XML 中对 NULL 值的表示方法。

语法如下:

```
PROCEDURE SETNULLHANDLING(
    CTX     IN          INTEGER,
    FLAG    IN          INTEGER
);
```

参数详解

- CTX 操作的上下文句柄。
- FLAG NULL 值的表示方法，可有如下三种取值。
  - DROP_NULLS0，也是缺省值，直接跳过值为 NULL 的列。
  - NULL_ATTR1，在值为 NULL 的列的列名标签中显示 xsi:nil="true"。
  - EMPTY_TAG2，值为 NULL 的列仅显示列名标签。

10. SETROWSETTAG()

设置 XML 文档中 ROWSET 标签的名称，默认的标签名为 ROWSET。

语法如下:

```
PROCEDURE SETROWSETTAG(
    CTX                 IN      INTEGER,
    ROWSETTAGNAME   IN      VARCHAR(32767)
);
```

参数详解

- CTX 操作的上下文句柄。
- ROWSETTAGNAME 设置的标签名。

11. SETROWTAG()

设置 XML 文档中 ROW 标签的名称，默认的标签名为 ROW。

语法如下:

```
PROCEDURE SETROWTAG(
    CTX                 IN      INTEGER,
    ROWTAGNAME      IN      VARCHAR(32767)
```

```
);
```

参数详解

- CTX 操作的上下文句柄。
- ROWTAGNAME 设置的标签名。

12. SETSKIPROWS()

设置每次调用 GETXML 函数时跳过的行数，可以用在 XML 或 HTML 数据获取时各页数据的生成，第一页时，设置为 0，后面各页时，设置为前面已经获取的数据行数。

语法如下:

```
PROCEDURE SETSKIPROWS(
    CTX         IN      INTEGER,
    SKIPROWS    IN      BIGINT
);
```

参数详解

- CTX 操作的上下文句柄。
- SKIPROWS 跳过的行数。

13. USEITEMTAGSFORCOLL()

设置生成的 XML 中，集合类型元素的标签名。DM 目前仅仅语法支持，调用后无实际效果。

语法如下:

```
PROCEDURE USEITEMTAGSFORCOLL(
    CTX     IN      INTEGER
);
```

参数详解

- CTX 操作的上下文句柄。

14. USENULLATTRIBUTEINDICATOR()

另一种设置 NULL 值表现方式的过程，设置是否用一个 XML 属性来表示 NULL，或者忽略 NULL。

语法如下:

```
PROCEDURE USENULLATTRIBUTEINDICATOR(
    CTX             IN      INTEGER,
    ATTRIND         IN      BOOLEAN
);
```

参数详解

- CTX 操作的上下文句柄。
- ATTRIND TRUE 对应 SETNULLHANDLING 的 NULL_ATTR，为缺省值；FALSE 对应 SETNULLHANDLING 的 DROP_NULLS。

22.2 举例说明

使用包内的过程和函数之前，如果还未创建过系统包，请先调用系统过程创建系统包。

```
SP_CREATE_SYSTEM_PACKAGES (1,'DBMS_XMLGEN');
SP_CREATE_SYSTEM_PACKAGES (1,'DBMS_OUTPUT');
SET SERVEROUTPUT ON;    //DBMS_OUTPUT.PUT_LINE 需要设置这条语句，才能打印出消息
```

1. 数据准备

```
CREATE TABLE TEST_XML(A INT, B VARCHAR(30));
INSERT INTO TEST_XML VALUES(0, NULL);
```

```
DECLARE
    I INT;
BEGIN
    FOR I IN 1..100 LOOP
        INSERT INTO TEST_XML VALUES(I, '<XML>$');
    END LOOP;
END;
/
COMMIT;
```

2. 最简单的获取 XML 数据的例子

```
DECLARE
    DTA    CLOB;
BEGIN
    DTA := DBMS_XMLGEN.GETXML('SELECT * FROM TEST_XML WHERE ROWNUM < 4');
    DBMS_OUTPUT.PUT_LINE(DTA);
    DBMS_LOB.FREETEMPORARY(DTA);
END;
/
```

在这个例子中，直接指定查询语句调用 GETXML 方法，在 GETXML 中会生成临时的上下文句柄，这个句柄的所有属性都会使用缺省值。GETXML 返回一个临时生成的 CLOB，之后由程序员负责调用 DBMS_LOB.FREETEMPORARY 来释放它。

上述 PL/SQL 执行后将打印出以下的 XML 数据。

```
<?xml version = '1.0'?>
<ROWSET>
```

```
<ROW>
<A>0</A>
</ROW>
<ROW>
<A>1</A>
<B>&lt;XML&gt;$</B>
</ROW>
<ROW>
<A>2</A>
<B>&lt;XML&gt;$</B>
</ROW>
</ROWSET>
```

3. 典型的获取 XML 数据的例子

```
DECLARE
    CTX    DBMS_XMLGEN.CTXHANDLE;
    DTA    CLOB;
BEGIN
    CTX := DBMS_XMLGEN.NEWCONTEXT('SELECT * FROM TEST_XML');
    DBMS_XMLGEN.SETMAXROWS(CTX, 2);
    DTA := DBMS_XMLGEN.GETXML(CTX);
    DBMS_OUTPUT.PUT_LINE(DTA);
    DBMS_LOB.FREETEMPORARY(DTA);
    DBMS_XMLGEN.SETSKIPROWS(CTX, 5);
    DTA := DBMS_XMLGEN.GETXML(CTX);
```

```
    DBMS_OUTPUT.PUT_LINE(DTA);
    DBMS_LOB.FREETEMPORARY(DTA);
    DBMS_XMLGEN.CLOSECONTEXT(CTX);
END;
/
```

这是一个较为典型的获取 XML 数据的例子，首先申请一个上下文句柄，然后使用这个上下文句柄进行一些属性设置，再调用 GETXML 获取 XML 数据，最后关闭上下文句柄。

上述 PL/SQL 执行后将打印出以下的 XML 数据。

```
<?xml version = '1.0'?>
<ROWSET>
<ROW>
<A>0</A>
</ROW>
<ROW>
<A>1</A>
<B>&lt;XML&gt;$</B>
</ROW>
</ROWSET>


<?xml version = '1.0'?>
<ROWSET>
<ROW>
<A>7</A>
<B>&lt;XML&gt;$</B>
```

```
</ROW>

<ROW>

<A>8</A>

<B>&lt;XML&gt;$</B>

</ROW>

</ROWSET>
```

23 UTL_ENCODE 包

UTL_ENCODE 包提供了对 varbinary 和 blob 类型数据进行编码和解码的功能。

23.1 相关方法

1. BASE64_ENCODE

将 varbinary 或 blob 格式的数据 R 编码成 base64 字符集格式，再以 varbinary 类型返回。

语法如下:

```
FUNCTION UTL_ENCODE.BASE64_ENCODE (
    R     IN     VARBINARY
)RETURN VARBINARY;
或
FUNCTION UTL_ENCODE.BASE64_ENCODE (
    R     IN     BLOB
)RETURN VARBINARY;
```

2. BASE64_DECODE

将 base64 字符集的编码 R，解码成原始的 varbinary 类型数据。

在遇到输入值中存在非 base64 编码的内容时，报错。

语法如下:

```
FUNCTION UTL_ENCODE.BASE64_DECODE (
    R     IN     VARBINARY
```

```
)RETURN VARBINARY;
```

3. BASE64_DECODE2

将 base64 字符集的编码 R，解码成原始的 varbinary 类型数据。

在遇到输入值中存在非 base64 编码的内容时，返回 NULL，不报错。

语法如下:

```
FUNCTION UTL_ENCODE.BASE64_DECODE2 (
    R     IN     VARBINARY
)RETURN VARBINARY;
```

23.2 举例说明

使用包内的过程和函数之前，如果还未创建过系统包，请先调用系统过程创建系统包。

```
SP_CREATE_SYSTEM_PACKAGES (1,'UTL_ENCODE');
```

例使用 BASE64_ENCODE 和 BASE64_deCODE 对数据进行编码和解码。

```
//数据准备:
DROP TABLE T;
CREATE TABLE T (I VARBINARY (10));
INSERT INTO T VALUES ('420921197908051523');
//使用 BASE64_ENCODE 和 BASE64_DECODE:
SELECT UTL_ENCODE.BASE64_ENCODE (I) FROM T;
查询结果为：0X51676B6847586B494252556A
SELECT UTL_ENCODE.BASE64_DECODE (UTL_ENCODE.BASE64_ENCODE (I)) FROM T;
查询结果为：0X420921197908051523
```

24 UTL_FILE 包

UTL_FILE 包为 PL/SQL 程序提供读和写操作系统数据文件的功能。它提供一套严格的使用标准操作系统文件 I/O 方式：OPEN、 PUT、 GET 和 CLOSE 操作。

当用户想读取或写一个数据文件的时候，可以使用 FOPEN 来返回文件句柄。这个文件句柄将用于随后在文件上的所有操作。例如，过程 PUT_LINE 写 TEXT 字符串和行终止符到一个已打开的文件句柄，以及使用 GET_LINE 来读取指定文件句柄的一行到提供的缓存。

24.1 数据类型

UTL_FILE 定义了一种 FILE_TYPE 记录类型。FILE_TYPE 类型是 UTL_FILE 专有类型。用户不能引用和改变该记录的内容。FILE_TYPE 记录类型定义如下：

语法如下：

```
TYPE FILE_TYPE IS RECORD (
    ID              INTEGER,
    DATATYPE        INTEGER,
    BYTE_MODE       BOOLEAN
);
```

参数详解

- ID 需要处理的外部文件句柄。
- DATATYPE 文件的类型：1 表示 CHAR 、2 表示 NCHAR、3 表示 BINARY。
- BYTE_MODE 文件打开后的类型：TRUE 表示二进制模式；FALSE 表示文本模式。

24.2 相关方法

UTL_FILE 包中的 23 个过程和函数详细介绍如下：


注意：

UTL_FILE 包只能访问目录对象中的数据文件，并且执行文件操作的用户必须具有针对当前目录对象的相关操作权限。

UTL_FILE 包不支持打开 ASM 文件系统上的路径。

以下详细介绍各过程和函数：

1. FCLOSE

关闭一个已打开文件。

语法如下：

```
PROCEDURE FCLOSE(
    FILE  IN OUT          FILE_TYPE
);
```

参数详解

- FILE 通过 FOPEN 调用，返回的活动文件句柄。

2. FCLOSE_ALL

关闭在这个会话里打开的所有文件。可用作紧急情况下清理程序，比如，当 PL/SQL 存在异常时。在 FCLOSE_ALL 之前打开的所有文件都无法进一步读或写。

语法如下：

```
PROCEDURE FCLOSE_ALL;
```

3. FCOPY

把一个文本文件中指定的连续的行数据拷贝到另外一个文件中。源文件需要以可读模式

打开。

语法如下:

```
PROCEDURE FCOPY(
    SRC_LOCATION    IN      VARCHAR(128),
    SRC_FILENAME    IN      VARCHAR(128),
    DEST_LOCATION   IN      VARCHAR(128),
    DEST_FILENAME   IN      VARCHAR(128),
    START_LINE      IN      INTEGER     DEFAULT 1,
    END_LINE        IN      INTEGER     DEFAULT NULL
);
```

参数详解

- SRC_LOCATION 源文件所在的目录对象名。
- SRC_FILENAME 被拷贝的源文件名称。若参数中包含路径，则直接忽略路径，只获取最后的文件名称。SRC_LOCATION 与 SRC_FILENAME 拼成的路径串总长度不能超过 256。
- DEST_LOCATION 新创建的目标文件所在的目录对象名。
- DEST_FILENAME 源文件的新名称。若参数中包含路径，则直接忽略路径，只获取最后的文件名称。DEST_LOCATION 与 DEST_FILENAME 拼成的路径串总长度不能超过 256。
- START_LINE 拷贝的起始行，默认为文件的第 1 行。
- END_LINE 拷贝的终止行，缺省为 NULL，即为文件的最后一行。

4. FFLUSH

以物理写入的方式将待定的数据写入到文件句柄所指定的文件中。通常地，待写入文件

的数据是存放在缓冲区中的。FFLUSH 程序强制将缓冲区中的数据写入到文件中。数据必须以换行符结束。

语法如下:

```
PROCEDURE FFLUSH(
    FILE  IN        FILE_TYPE
);
```

参数详解

- FILE 由 FOPEN 调用，返回的文件句柄。

5. FGETATTR

读取并返回磁盘文件的属性。

语法如下:

```
PROCEDURE FGETATTR(
    LOCATION        IN      VARCHAR(128),
    FILENAME        IN      VARCHAR(128),
    FEXISTS         OUT     BOOLEAN,
    FILE_LENGTH     OUT     NUMBER,
    BLOCKSIZE       OUT     NUMBER
);
```

参数详解

- LOCATION 源文件所在的目录对象名。
- FILENAME 被检查的文件名称。若参数中包含路径，则直接忽略路径，只获取最后的文件名称。LOCATION 与 FILENAME 拼成的路径串总长度不能超过 256。

- FEXISTS 文件是否存在，存在为 1，不存在为 0。
- FILE_LENGTH 文件的长度（字节），若文件不存在，则长度为 NULL。
- BLOCKSIZE 文件系统中数据块的大小（字节），若文件不存在，则长度为 NULL。

6. FGETPOS

指定文件中当前相对位置偏移量（字节）。

语法如下:

```
FUNCTION  FGETPOS(
    FILE        IN          FILE_TYPE
)RETURN INTEGER;
```

参数详解

- FILE 通过 FOPEN 调用，返回的活动文件句柄。

返回值

返回值 FGETPOS 以字节的方式返回一个打开文件相对位置偏移量。如果文件未打开则发生异常。如果当前位置在文件的最开始，则返回 0。

注意事项:

如果文件以 BYTE 的模式打开，那么会发生 INVALID OPERATION。

7. FOPEN

打开指定文件并返回一个文件句柄。FOPEN 不支持访问数据库内建文件（归档日志和联机日志除外），例如 dmarch.ini。

语法如下:

```
FUNCTION FOPEN(
    LOCATION        IN      VARCHAR(128),
    FILENAME        IN      VARCHAR(128),
    OPEN_MODE       IN      VARCHAR(128),
    MAX_LINESIZE    IN      INTEGER     DEFAULT 1024
)RETURN FILE_TYPE;
```

参数详解

- LOCATION　源文件所在的目录对象名。
- FILENAME　文件名称。若参数中包含路径，则直接忽略路径，只获取最后的文件名称。在 UNIX 系统中，文件名不能包含转义符：“/”。 LOCATION 与 FILENAME 拼成的路径串总长度不能超过 256。
- OPEN_MODE　文件打开模式。包括：R 只读模式；W 写模式；A 附加模式；RB 只读打开一个二进制文件，只允许读；WB 只写打开或建立一个二进制文件，只允许写数据；AB 追加打开一个二进制文件，并在文件末尾写数据。当以“A”或“AB”的方式打开文件时，若该文件不存在，则以“W”的方式创建该文件。
- MAX_LINESIZE　文件每行最大的字符数，包括换行符。最小为 1，最大为 32767。

注意事项:

目录对象名和文件名必须加上引号；对文件的访问和操作需要满足目录对象的权限控制要求。

8. FREMOVE

在有足够权限的情况下，从目录中删除一个文件。

语法如下:

```
PROCEDURE FREMOVE(
    LOCATION    IN      VARCHAR(128),
    FILENAME    IN      VARCHAR(128)
);
```

参数详解

- LOCATION 被删除文件所在的目录对象名。
- FILENAME 被删除文件名称。若参数中包含路径，则直接忽略路径，只获取最后的文件名称。LOCATION 与 FILENAME 拼成的路径串总长度不能超过 256。

注意事项:

FREMOVE 存储过程不会在删除文件之前验证权限，但是操作系统会验证文件和目录对象名称的正确性。没有权限，或者文件和目录对象名称不正确，则返回删除失败信息。

9. FRENAME

修改一个文件的名称。

语法如下:

```
PROCEDURE FRENAME(
    SRC_LOCATION    IN      VARCHAR(128),
    SRC_FILENAME    IN      VARCHAR(128),
    DEST_LOCATION   IN      VARCHAR(128),
    DEST_FILENAME   IN      VARCHAR(128),
    OVERWRITE       IN      BOOLEAN         DEFAULT FALSE
```

```
);
```

参数详解

- SRC_LOCATION 要改名文件所在的目录对象名。
- SRC_FILENAME 要改名的源文件名称。若参数中包含路径，则直接忽略路径，只获取最后的文件名称。SRC_LOCATION 与 SRC_FILENAME 拼成的路径串总长度不能超过 256。
- DEST_LOCATION 改名后文件所在的目录对象名。
- DEST_FILENAME 修改后的新名称。若参数中包含路径，则直接忽略路径，只获取最后的文件名称。DEST_LOCATION 与 DEST_FILENAME 拼成的路径串总长度不能超过 256。
- OVERWRITE 设置是否能够重写。如果设置为“TRUE”，则在 SRC_LOCATION 目录中覆盖任何名为 DEST_FILENAME 的文件。若设置为“FALSE”，就会产生异常。缺省为 FALSE。

10. FSEEK

根据指定的字节数，调整文件指针前进或后退。

语法如下:

```
PROCEDURE FSEEK(
    FILE                IN      FILE_TYPE,
    ABSOLUTE_OFFSET     IN      INTEGER     DEFAULT NULL,
    RELATIVE_OFFSET     IN      INTEGER     DEFAULT NULL
);
```

参数详解

- FILE 通过 FOPEN 调用，返回的活动文件句柄。
- ABSOLUTE_OFFSET 要寻找的字节的绝对路径，缺省为 NULL。

- RELATIVE_OFFSET 指针前进或后退的字节数。用正整数表示向前查找，负整数表示向后查找。

注意事项:

使用 FSEEK，可以读取先前的行而不用关闭文件再重新打开。但是使用者必须知道要跳过的字符数。

该存储过程根据 relative_offset 参数指定的字节数进行查找。

如果在指定字节数之前就已经到达了文件的开头，那么文件指针指定在文件开头。如果在指定字节数之前就已经到达了文件的尾部，那么 INVALID_OFFSET 错误就会发生。

以 RB 方式打开的文件，支持通过 FSEEK 设置偏移；以其余二进制模式打开的文件，使用 FSEEK 时会发生 INVALID OPERATION 异常。

当 ABSOLUTE_OFFSET 不为 0 时，设置 RELATIVE_OFFSET 不为 0 不生效，即如需使用相对偏移，需要将 ABSOLUTE_OFFSET 参数置为 0。

11. GET_LINE

读取指定文件的一行到提供的缓存。读取的文本不包括该行的终结符，或者直到文件的末尾，或者到指定的长度。并且不能超过 FOPEN 时指定的 MAX_LINESIZE。

语法如下:

```
PROCEDURE  GET_LINE(
    FILE            IN          FILE_TYPE,
    BUFFER          OUT             VARCHAR,
    LEN             IN          INTEGER         DEFAULT NULL
);
```

参数详解

- FILE 通过 FOPEN 调用，返回的活动文件句柄。
- BUFFER 接收数据文件中行读的数据缓冲区。
- LEN 从文件中读取的字节数。默认情况为 NULL，NULL 表示读取的字节数为 MAX_LINESIZE。

注意事项:

如果该行不匹配缓冲区，则发生 READ_ERROR 异常。如果到文件结束了仍没有文本读取，发生 NO_DATA_FOUND 异常。如果 FILE 是以 BYTE 模式打开的，则产生 INVALID_OPERATION 异常。

由于行的结束符不被读取，读取空行则返回空字符串。

缓冲区最大大小为 32767 字节，除非 FOPEN 指定了更小的值。如果未指定，缺省值为 1024。

12. GET_RAW

从文件中读取二进制字符串，并将文件指针向前调整读取的字节数。GET_RAW 函数忽略掉行结束符。

语法如下:

```
PROCEDURE GET_RAW(
    FILE        IN          FILE_TYPE,
    BUFFER          OUT         VARBINARY,
    LEN             IN          INTEGER         DEFAULT NULL
);
```

参数详解

- FILE 通过 FOPEN 调用，返回的活动文件句柄。
- BUFFER 接收从文件中读取的数据行的缓冲区。
- LEN 从文件中读取的字节数。默认情况为 NULL，NULL 表示读取的字节数为 MAX_LINESIZE。

注意事项:

如果子程序读取超过文件末尾，那么将产生 NO_DATA_FOUND 的异常。程序在执行循环时应该允许捕获这个异常。

13. IS_OPEN

判断文件句柄指定的文件是否已打开，如果已打开则返回 TRUE，否则返回 FALSE。不保证用户在尝试使用该文件句柄时没有操作系统的权限。

语法如下:

```
FUNCTION  IS_OPEN(
    FILE        IN          FILE_TYPE
)RETURN BOOLEAN;
```

参数详解

- FILE 通过 FOPEN 调用，返回的活动文件句柄。

14. NEW_LINE

在当前位置输出一个或多个行终止符。

语法如下:

```
PROCEDURE NEW_LINE(
```

```
    FILE        IN          FILE_TYPE,
    LINES       IN          INTEGER     DEFAULT 1
);
```

参数详解

- FILE 通过 FOPEN 调用，返回的活动文件句柄。
- LINES 写入文件的行终结符数量。

异常:

INVALID_FILEHANDLE

INVALID_OPERATION

WRITE_ERROR

15. PUT

把缓冲区中的文本字符写入到数据文件。文件必须是以写模式打开的。UTL_FILE.PUT 输出数据时不会附加行终止符。使用 NEW_LINE 来添加行结束符或者使用 PUT_LINE 来写一行带有结束符的数据。

语法如下:

```
PROCEDURE PUT(
    FILE        IN          FILE_TYPE,
    BUFFER      IN          VARCHAR
);
```

参数详解

- FILE 通过 FOPEN 调用，返回的活动文件句柄。

- BUFFER 包含要写入文件的数据缓存。

注意事项:

如果 FOPEN 没有指定一个更小的值，BUFFER 参数的最大值将是 32767。如果未指定，FOPEN 默认的行最大值为 1024。连续调用 PUT 的总和在没有缓冲区刷页的情况下不能超过 32767。

异常:

INVALID_FILEHANDLE

INVALID_OPERATION

WRITE_ERROR

16. PUT_LINE

把缓冲区中的文本字符串写入数据文件，同时输出一个与系统有关的行终止符。

语法如下:

```
PROCEDURE PUT_LINE(
    FILE        IN          FILE_TYPE,
    BUFFER      IN          VARCHAR,
    AUTOFLUSH   IN          BOOLEAN             DEFAULT FALSE
);
```

参数详解

- FILE 由 FOPEN 返回的文件句柄。
- BUFFER 包含要写入文件的数据缓存。
- AUTOFLUSH 数据写完后，自动清理缓冲区。

注意事项:

缓区大小最大为 32767。

如果文件有 byte 模式打开，则产生 INVALID_OPERATION 异常。

异常:

INVALID_FILEHANDLE

INVALID_OPERATION

WRITE_ERROR

17. PUT_RAW

接收输入的原始数据并将其写入缓存。

语法如下:

```
PROCEDURE PUT_RAW(
    FILE        IN          FILE_TYPE,
    BUFFER      IN          VARBINARY,
    AUTOFLUSH   IN          BOOLEAN             DEFAULT FALSE
);
```

参数详解

- FILE 由 FOPEN 返回的文件句柄。
- BUFFER 包含要写入文件的数据缓存;
- AUTOFLUSH 数据写完后，自动清理缓冲区。

注意事项:

通过设置第三个参数可以使缓冲区自动冲刷。

Buffer 最大值为 32767。

18. PUTF

以一个模版样式输出至多 5 个字符串，类似 C 语言中的 PRINTF()。

语法如下:

```
PROCEDURE PUTF(
    FILE        IN      FILE_TYPE,
    FORMAT      IN      VARCHAR,
    ARG1        IN      VARCHAR     DEFAULT NULL,
    ARG2        IN      VARCHAR     DEFAULT NULL,
    ARG3        IN      VARCHAR     DEFAULT NULL,
    ARG4        IN      VARCHAR     DEFAULT NULL,
    ARG5        IN      VARCHAR     DEFAULT NULL
);
```

参数详解

- FILE 由 FOPEN 返回的文件句柄。
- FORMAT 决定格式的格式串。格式串可使用以下样式:

1）%s 格式符。在格式串中最多可以识别 5 个格式符%s，且格式符%s 会被后面的参数依次填充，如果此时没有足够的参数，没有对应参数的%s 会被忽视，不被写入文件；当格式符%s 超过 5 个时，超出 5 个的部分不会被视为格式符，而会被识别为字符串"%s"直接写入文件（对于其余不是%s 或\n 的格式符也会直接作为字符串写入文件）;

2）\n 换行符。在格式串中没有个数限制。

- ARGn 可选的 5 个参数，最多 5 个（n 为参数编号）。

注意事项:

如果文件以 BYTE 模式打开，返回 INVALID_OPERATION 错误。

%s，\n 与 C 语言中的类似。

19. FOPEN_NCHAR

打开指定 UTF8 格式文件并返回一个文件句柄。FOPEN_NCHAR 不支持访问数据库内建文件，例如 DMARCH.INI。

语法如下:

```
FUNCTION FOPEN_NCHAR (
    LOCATION        IN      VARCHAR(128),
    FILENAME        IN      VARCHAR(128),
    OPEN_MODE       IN      VARCHAR(128),
    MAX_LINESIZE    IN      INTEGER     DEFAULT 1024
);
```

参数详解

- LOCATION 源文件所在的目录对象名。
- FILENAME 文件名称。若参数中包含路径，则直接忽略路径，只获取最后的文件名称。在 UNIX 系统中，文件名不能包含转义符：“/”。 LOCATION 与 FILENAME 拼成的路径串总长度不能超过 256。
- OPEN_MODE 文件打开模式。包括：R 只读模式；W 写模式；A 附加模式；RB 只读打开一个二进制文件，只允许读；WB 只写打开或建立一个二进制文件，只允许写数据；AB 追加打开一个二进制文件，并在文件末尾写数据。当以“A”或“AB”的方式打开文件时，

若该文件不存在，则以“W”的方式创建该文件。

- MAX_LINESIZE　文件每行最大的字符数，包括换行符。取值范围为 1~32767，缺省为 1024。

注意事项:

目录对象名和文件名必须加上引号；对文件的访问和操作需要满足目录对象的权限控制要求；FOPEN_NCHAR 以 UTF8 格式打开文件，读写需调用带 NCHAR 的读写过程。

20. GET_LINE_NCHAR

读取指定的 UTF8 格式文件中的一行到指定的缓存。读取的文本不包括该行的终结符，当读取到文件的末尾，或者到指定的长度时结束读取，并且不能超过 FOPEN_NCHAR 时指定的 MAX_LINESIZE。

语法如下:

```
PROCEDURE GET_LINE_NCHAR (
    FILE            IN      FILE_TYPE,
    BUFFER          OUT             VARCHAR,
    LEN             IN      INTEGER     DEFAULT NULL
);
```

参数详解

- FILE 通过 FOPEN_NCHAR 调用，返回的活动文件句柄。
- BUFFER 接收数据文件中行读的数据缓冲区。
- LEN 从文件中读取的字节数。默认情况为 NULL，NULL 表示读取的字节数为 MAX_LINESIZE。

注意事项:

如果该行不匹配缓冲区，则发生 READ_ERROR 异常。如果到文件结束了仍没有读取文本，则发生 NO_DATA_FOUND 异常。如果 FILE 是以 BYTE 模式打开的，则发生 INVALID_OPERATION 异常。

由于行的结束符不被读取，读取空行则返回空字符串。

缓冲区最大大小为 32767 字节，除非 FOPEN_NCHAR 指定了更小的值。如果未指定，缺省值为 1024。

21. PUT_NCHAR

把缓冲区中的文本字符以 UTF8 格式写入到数据文件。文件必须是以写模式打开的。UTL_FILE.PUT_NCHAR 输出数据时不会附加行终止符。可以使用 NEW_LINE 来添加行结束符或者使用 PUT_LINE_NCHAR 来写一行带有结束符的数据。

语法如下:

```
PROCEDURE PUT_NCHAR (
    FILE        IN          FILE_TYPE,
    BUFFER          IN          VARCHAR
);
```

参数详解

- FILE 通过 FOPEN_NCHAR 调用，返回的活动文件句柄。
- BUFFER 包含要写入文件的数据缓存。

注意事项:

如果 FOPEN_NCHAR 中没有指定一个更小的值，BUFFER 参数的最大值将是 32767；如果 FOPEN_NCHAR 中未指定 FOPEN_NCHAR，默认的行最大值为 1024。连续调用 PUT_NCHAR 的总和在没有缓冲区刷页的情况下不能超过 32767。

异常:

INVALID_FILEHANDLE

INVALID_OPERATION

WRITE_ERROR

22. PUT_LINE_NCHAR

把缓冲区中的文本字符串以 UTF8 格式写入数据文件，同时输出一个与系统有关的行终止符。

语法如下:

```
PROCEDURE PUT_LINE_NCHAR (
    FILE        IN          FILE_TYPE,
    BUFFER      IN          VARCHAR,
    AUTOFLUSH   IN          BOOLEAN         DEFAULT FALSE
);
```

参数详解

- FILE 由 FOPEN_NCHAR 返回的文件句柄。
- BUFFER 包含要写入文件的数据缓存。
- AUTOFLUSH 数据写完后，是否自动清理缓冲区。缺省为 FALSE。

注意事项:

缓区大小最大为 32767。

如果文件以 BYTE 模式打开，则产生 INVALID_OPERATION 异常。

异常:

INVALID_FILEHANDLE

INVALID_OPERATION

WRITE_ERROR

23. PUTF_NCHAR

以一个模版样式并用 UTF8 格式输出至多 5 个字符串，类似 C 语言中的 PRINTF()。

语法如下:

```
PROCEDURE PUTF_NCHAR (
    FILE        IN      FILE_TYPE,
    FORMAT      IN      VARCHAR,
    ARG1        IN      VARCHAR         DEFAULT NULL,
    ARG2        IN      VARCHAR         DEFAULT NULL,
    ARG3        IN      VARCHAR         DEFAULT NULL,
    ARG4        IN      VARCHAR         DEFAULT NULL,
    ARG5        IN      VARCHAR         DEFAULT NULL
);
```

参数详解

- FILE 由 FOPEN_NCHAR 返回的文件句柄。
- FORMAT 决定格式的格式串。格式串可使用以下样式:

1）%s 格式符。在格式串中最多可以识别 5 个格式符%s，且格式符%s 会被后面的参数依次填充，如果此时没有足够的参数，没有对应参数的%s 会被忽视，不被写入文件；当格式符%s 超过 5 个时，超出 5 个的部分不会被视为格式符，而会被识别为字符串"%s"直接写入文件（对于其余不是%s 或\n 的格式符也会直接作为字符串写入文件）；

2）\n 换行符。在格式串中没有个数限制。

- ARGn 可选的 5 个参数，最多 5 个（n 为参数编号）。

注意事项:

如果文件以 BYTE 模式打开，返回 INVALID_OPERATION 错误。

%s，\n 与 C 语言中的类似。

24.3 错误处理

错误、异常情况释义:

1. INVALID_PATH：无效路径名
2. INVALID_MODE：无效文件模式名
3. INVALID_FILEHANDLE：无效文件处理
4. WRITE_ERROR：写入错误
5. FILE_OPEN：打开文件失败
6. CHARSETMISMATCH：字符集不匹配
7. INVALID_OFFSET：无效偏移
8. READ_ERROR：读文件失败
9. INTERNAL_ERROR：内部错误
10. INVALID_FILENAME：无效文件名
11. ACCESS_DENIED：拒绝访问
12. RENAME_FAILED：重命名失败
13. DELETE_FAILED：删除失败

24.4 举例说明

首先在路径 e:\utl_file_temp 下创建文件 u12345.tmp。文件的内容为:

```
THIS IS LINE 1.

THIS IS LINE 2.

THIS IS LINE 3.

THIS IS LINE 4.

THIS IS LINE 5.
```

登录 SYSDBA 用户，使用包内的过程和函数之前，如果还未创建过系统包，请先调用系统过程创建系统包。

```
SP_CREATE_SYSTEM_PACKAGES (1,'UTL_FILE');
```

创建用户 USER01, 创建目录对象 DIR01, 该目录对应的文件路径为 e:\utl_file_temp，并将该目录对象的读写权限授予用户 USER01。另外还要将创建存储过程的权限授予用户 USER01，该权限在示例 3 中将会用到。

```
CREATE USER USER01 IDENTIFIED BY "USER01_psd";

CREATE DIRECTORY DIR01 AS 'E:\UTL_FILE_TEMP';

GRANT READ ON DIRECTORY DIR01 TO USER01;

GRANT WRITE ON DIRECTORY DIR01 TO USER01;

GRANT CREATE PROCEDURE TO USER01;
```

登录用户 USER01，首先执行下述语句，然后即可执行下面 3 个示例。

```
SET SERVEROUTPUT ON;  //PRINT 需要设置这条语句，才能打印出消息
```

例 1 使用 GET_LINE 过程读文件

```
DECLARE

    V1 VARCHAR2(8186);

    F1 UTL_FILE.FILE_TYPE;

BEGIN

  //本例子中 MAX_LINESIZE 小于 GET_LINE 的长度要求，所以返回的字节数小于等于 256
```

```
  F1 := UTL_FILE.FOPEN('DIR01','u12345.tmp','R',256);
  UTL_FILE.GET_LINE(F1,V1,32767);
  PRINT 'GET_LINE:'||V1;
  UTL_FILE.FCLOSE(F1);
  //FOPEN 的 MAX_LINESIZE 是 NULL ， NULL 即为默认的 1024，所以返回的字节数小于等于 1024
  F1 := UTL_FILE.FOPEN('DIR01','u12345.tmp','R');
  UTL_FILE.GET_LINE(F1,V1,32767);
  PRINT 'GET_LINE:'||V1;
  UTL_FILE.FCLOSE(F1);

  //GET_LINE 未指定字节数，所以默认为 NULL1024，NULL 即为 1024
  F1 := UTL_FILE.FOPEN('DIR01','u12345.tmp','R');
  UTL_FILE.GET_LINE(F1,V1);
  PRINT 'GET_LINE:'||V1;
  UTL_FILE.FCLOSE(F1);
END;
/
```

每个过程都是输出第一行，这个例子的结果如下所示:

```
GET LINE: THIS IS LINE 1.
GET LINE: THIS IS LINE 1.
GET LINE: THIS IS LINE 1.
```

例 2 使用 PUTF 过程向文件尾部添加内容。

```
DECLARE
    HANDLE UTL_FILE.FILE_TYPE;
```

```
    MY_WORLD   VARCHAR2(4) := 'ZORK';
BEGIN
    HANDLE := UTL_FILE.FOPEN('DIR01','u12345.tmp','A');
    UTL_FILE.PUTF(HANDLE,'\nHELLO, WORLD! \nI COME FROM %s WITH %s.\n',MY_WORLD,'GREETINGS
FOR ALL EARTHLINGS');
    UTL_FILE.FFLUSH(HANDLE);
    UTL_FILE.FCLOSE(HANDLE);
END;
/
```

该例子在 U12345.TMP 文件的末尾添加了如下内容:

```
HELLO, WORLD!
I COME FROM ZORK WITH GREETINGS FOR ALL EARTHLINGS.
```

例 3 使用 GET_RAW 过程从 DIR01 目录下的 u12345.tmp 文件中读取原始信息。

首先，将 u12345.tmp 文件中的内容写为：HELLO WORLD!

```
DECLARE
    HANDLE UTL_FILE.FILE_TYPE;
BEGIN
    HANDLE := UTL_FILE.FOPEN('DIR01','u12345.tmp','W');
    UTL_FILE.PUT(HANDLE,'HELLO WORLD!');
    UTL_FILE.FFLUSH(HANDLE);
    UTL_FILE.FCLOSE(HANDLE);
END;
/
```

然后创建一个存储过程，使用 GET_RAW 读取 u12345.tmp 文件中的原始信息。

```
CREATE OR REPLACE PROCEDURE GETRAW(N IN VARCHAR2) IS
    H       UTL_FILE.FILE_TYPE;
    BUF     BLOB;
    AMNT    CONSTANT BINARY_INTEGER := 32767;
BEGIN
    H := UTL_FILE.FOPEN('DIR01', N, 'R', 32767);
    UTL_FILE.GET_RAW(H, BUF, AMNT);
    PRINT 'THIS IS THE RAW DATA:';
    SELECT BUF;
    UTL_FILE.FCLOSE (H);
END;
/
```

过程建好后，运行如下语句：

```
begin
    getraw('u12345.tmp');
end;
/
```

输出结果为：

```
THIS IS THE RAW DATA:


行号        BUF
---------- --------------------------
1             0X48454C4C4F20574F524C4421
```

25 UTL_INADDR 包

为了在 PL/SQL 中，提供网络地址转换的支持，开发 UTL_INADDR 包，提供一组 API，实现主机的 IP 地址和主机名之间的转换。

25.1 相关方法

1. get_host_address

依据给定的主机名返回其 IP 地址。返回给定 HOST 主机的 IP 地址（以 CHAR 的形式展现）。

语法如下:

```
FUNCTION GET_HOST_ADDRESS (
    HOST      IN          VARCHAR2     DEFAULT NULL
) RETURN VARCHAR2;
```

参数详解

- HOST 主机名，VARCHAR2 类型。默认值为 NULL，用本机的主机名代替，表示获取本机的 IP 地址;

2. get_host_name

依据给定的 IP 地址返回主机名。

语法如下:

```
FUNCTION GET_HOST_NAME(
    IP          IN          VARCHAR2     DEFAULT NULL
) RETURN VARCHAR2;
```

参数详解

- IP　IP 地址，VARCHAR2 类型。默认值为 NULL，用本机的 IP 地址代替，表示获取本机的主机名。

25.2 举例说明

使用包内的过程和函数之前，如果还未创建过系统包，请先调用系统过程创建系统包。

```
SP_CREATE_SYSTEM_PACKAGES (1,'UTL_INADDR');
```

例 获得 192.168.0.30 机器的名称，以及名称为 TEST 机器的 IP 地址。

```
SELECT UTL_INADDR.GET_HOST_NAME('192.168.0.30');

SELECT UTL_INADDR.GET_HOST_ADDRESS('TEST');
```

26 UTL_MAIL 包

为了在 PL/SQL 中，提供 EMAIL 发送的支持，开发 UTL_MAIL 包，提供一组 API，实现发送简单的邮件和包含附件的邮件。服务器默认不安装 UTL_MAIL 包，需要用户主动通过系统函数 SP_CREATE_SYSTEM_PACKAGES 指定包名方式进行创建。UTL_MAIL 包中的配置信息将以明文传输，存在邮件密码泄露的风险，建议谨慎开启 UTL_MAIL 包，可由 UTL_SMTP 包替代相关功能。

26.1 相关方法

1. INIT

包需要的配置信息。

语法如下:

```
PROCEDURE INIT(
    SMTP_SERVER     VARCHAR2(128),
    SMTP_PORT       INT         DEFAULT 25,
    SENDER          VARCHAR2(128),
    PASSWORD        VARCHAR2(128)
);
```

参数详解

- SMTP_SERVER　SMTP 服务器的地址，如 SMTP.163.COM。
- SMTP_PORT　SMTP 的端口号，一般都是 25。
- SENDER 设置发送的邮箱，也就是要认证的邮箱。
- PASSWORD 设置发送邮箱的密码。

2. SEND

发送 MESSAGE 的信息。

语法如下:

```
PROCEDURE SEND (
    SENDER          IN          VARCHAR2,
    RECIPIENTS      IN          VARCHAR2,
    CC              IN          VARCHAR2      DEFAULT NULL,
    BCC             IN          VARCHAR2      DEFAULT NULL,
    SUBJECT         IN          VARCHAR2      DEFAULT NULL,
    MESSAGE         IN          VARCHAR2,
    MIME_TYPE       IN          VARCHAR2      DEFAULT 'TEXT/PLAIN; CHARSET=US-ASCII',
    PRIORITY  IN          INT           DEFAULT NULL
);
```

参数详解

- SENDER 发送的邮箱地址。
- RECIPIENTS 接收邮件的地址。
- CC 抄送的邮件的地址。
- BCC 秘密抄送的邮件的地址。
- SUBJECT 邮件的主题。
- MESSAGE 邮件的内容，正文部分。
- MIME_TYPE 邮件的内容格式，DEFAULT IS 'TEXT/PLAIN; CHARSET=US-ASCII'。
- PRIORITY 优先级（不用设置）。

3. SEND_ATTACH_RAW

发送带附件的邮件。

语法如下:

```
PROCEDURE SEND_ATTACH_RAW (
    SENDER          IN      VARCHAR2,
    RECIPIENTS      IN      VARCHAR2,
    CC              IN      VARCHAR2    DEFAULT NULL,
    BCC             IN      VARCHAR2    DEFAULT NULL,
    SUBJECT         IN      VARCHAR2    DEFAULT NULL,
    MESSAGE         IN      VARCHAR2    DEFAULT NULL,
    MIME_TYPE       IN      VARCHAR2    DEFAULT 'TEXT/PLAIN; CHARSET=US-ASCII',
    PRIORITY        IN      INT         DEFAULT NULL,
    ATTACHMENT      IN      BLOB,
    ATT_INLINE      IN      BOOLEAN     DEFAULT TRUE,
    ATT_MIME_TYPE   IN      VARCHAR2    DEFAULT 'APPLICATION/OCTET-STREAM',
    ATT_FILENAME    IN      VARCHAR2    DEFAULT NULL
);
```

参数详解

- SENDER 发送的邮箱地址。
- RECIPIENTS 接收邮件的地址。
- CC 抄送的邮件的地址。
- BCC 秘密抄送的邮件的地址。

- SUBJECT 邮件的主题。
- MESSAGE 邮件的内容，正文部分。
- MIME_TYPE 邮件的内容格式：文本、HTML 等。缺省为'TEXT/PLAIN;CHARSET=US-ASCII'。
- PRIORITY 优先级（不用设置）。
- ATTACHMENT 附件。
- ATT_INLINE 附件在文件中显示是否使用下划线。
- ATT_MIME_TYPE 邮件附件的类型，如文本。
- ATT_FILENAME 附件的文件名。

26.2 举例说明

使用包内的过程和函数之前，如果还未创建过系统包，请先调用系统过程创建系统包。

```
SP_CREATE_SYSTEM_PACKAGES (1,'UTL_MAIL');
```

例 在使用该包前，需要先初始化该包，调用 INIT 函数，设置使用 SMTP 服务器，端口（一般默认 25，设置用于认证的邮箱，认证的密码。设置完后，就可以使用包的函数：SEND 和 SEND_ATTACH_RAW。

```
//初始化包的函数
UTL_MAIL.INIT(
    'SMTP.163.COM',  //SMTP 服务器
    25,  //端口
    'UTL_MAIL@163.COM',  //认证的邮箱
    'DM_UTL_MAIL'  //认证邮箱的密码
);
//发送不带附件的邮件
```

```
UTL_MAIL.SEND(
    'UTL_MAIL@163.COM',   //发送的邮箱地址
    'RECIPIENT@DAMENG.COM',   //接受的邮箱地址
    '',
    '',
    'TEST',   //主题
    'THIS IS A MAIL FOR TEST .',   //正文
    'TEXT/PLAIN; CHARSET=US-ASCII',   //类型
    3
);
//发送带附件的邮件
UTL_MAIL.SEND_ATTACH_RAW(
    'UTL_MAIL@163.COM',
    'RECIPIENT@DAMENG.COM',
    '',
    '',
    'TEST',
    'THIS IS A MAIL FOR TEST .',
    'TEXT/PLAIN; CHARSET=US-ASCII',
    3,
    HEXTORAW('12345678905645318131694341964316513983208431264654654654654615 64'),
//附件二进制的内容
    FALSE,
    'APPLICATION/OCTET-STREAM',   //附件的类型
```

```
    'ATTACH.ZIP'        //附件的名字
);
```

27 UTL_MATCH 包

UTL_MATCH 包用于计算两个字符串差异字符个数和相似度。

27.1 相关方法

UTL_MATCH 包所支持的 2 个函数，分别计算源字符串和目标字符串的差异字符个数和相似度。

1. EDIT_DISTANCE

输出字符串 S1 转化为 S2 所需要被插入、删除或替换字符的个数。

语法如下：

```
FUNCTION EDIT_DISTANCE(
    S1          IN          VARCHAR,
    S2          IN          VARCHAR
)RETURN INTEGER;
```

参数详解

- S1 输入参数，VARCHAR 数据。
- S2 输入参数，VARCHAR 数据。

2. EDIT_DISTANCE_SIMILARITY

输出两个字符串的相似度。通过字符串 S1 转化为 S2 所需要被插入、删除或替换字符的个数，计算出字符串 S1 和 S2 的相似度。返回 1～100 之间的整数，0 代表完全不匹配，100 代表完全匹配。

语法如下：

```
FUNCTION EDIT_DISTANCE_SIMILARITY (
```

```
    S1          IN          VARCHAR,

    S2          IN          VARCHAR

)RETURN INTEGER;
```

参数详解

- S1 输入参数，VARCHAR 数据。
- S2 输入参数，VARCHAR 数据。

3. JARO_WINKLER

采用 JARO_WINKLER 算法评估两个字符串的相似度。

语法如下:

```
FUNCTION JARO_WINKLER(

    S1                  VARCHAR,

    S2                  VARCHAR

)RETURN DOUBLE;
```

参数详解

- S1 输入参数，VARCHAR 数据。
- S2 输入参数，VARCHAR 数据。

4. JARO_WINKLER_SIMILARITY

采用 JARO_WINKLER 算法计算两个字符串的相似度指标。相似度指标范围为：0 ~ 100 之间的整数，0 代表完全不匹配，100 代表完全匹配。

语法如下:

```
FUNCTION JARO_WINKLER_SIMILARITY(

    S1                  VARCHAR,

    S2                  VARCHAR

)RETURN INT;
```

参数详解

- S1 输入参数，VARCHAR 数据。
- S2 输入参数，VARCHAR 数据。

27.2 举例说明

使用包内的过程和函数之前，如果还未创建过系统包，请先调用系统过程创建系统包。

```
SP_CREATE_SYSTEM_PACKAGES (1,'UTL_MATCH');
```

例 UTL_MATCH 包的应用。

例1 EDIT_DISTANCE

```
SELECT UTL_MATCH.EDIT_DISTANCE('china','chin') FROM DUAL;
```

查询结果如下:

```
1
```

例2 EDIT_DISTANCE_SIMILARITY

```
SELECT UTL_MATCH.EDIT_DISTANCE_SIMILARITY('china','chin') FROM DUAL;
```

查询结果如下:

```
80
```

例3 JARO_WINKLER

```
SELECT UTL_MATCH.JARO_WINKLER('china','chin') FROM DUAL;
```

查询结果如下:

```
9.600000000000000E-001
```

例4 JARO_WINKLER_SIMILARITY

```
SELECT UTL_MATCH.JARO_WINKLER_SIMILARITY('china','chin') FROM DUAL;
```

查询结果如下:

```
96
```

28 UTL_RAW 包

UTL_RAW 包提供了一系列十六进制类型相关函数，可以将十六进制类型数据转化为其它类型数据。

28.1 相关方法

UTL_RAW 包所支持的 20 个函数，分别实现十六进制的相关转化函数。这些函数通过调用相应系统内部函数来实现存取数据的过程。如下详细介绍各函数：

1. XRANGE

输出包含 START_BYTE~END_BYTE 在内的，十六进制单字节区间内的全部字节。

语法如下：

```
FUNCTION XRANGE(
    START_BYTE   IN   VARBINARY   DEFAULT NULL,
    END_BYTE     IN   VARBINARY   DEFAULT NULL
) RETURN VARBINARY;
```

参数详解

- START_BYTE 输入参数，单字节 VARBINARY 数据。
- END_BYTE 输入参数，单字节 VARBINARY 数据。

注意：1）要求 START_BYTE 和 END_BYTE 都是单字节。若超过了一个字节，则只从左取第一个高位字节；2）如果 START_BYTE 值>END_BYTE 值，则返回以 START_BYTE 十六进制字节开始的，通过 FF 至 00，再到 END_BYTE 的全部十六进制字节；3）如果 START_BYTE 为 NULL 默认为 00，END_BYTE 为 NULL 默认为 FF，二者均为 NULL 则 START_BYTE 默认为 00，END_BYTE 默认为 FF。

2. BIT_AND

输出 R1 与 R2 的按位运算的逻辑与。

语法如下:

```
FUNCTION BIT_AND(
    R1    IN    VARBINARY,
    R2    IN    VARBINARY
) RETURN VARBINARY;
```

参数详解

- R1 输入参数，VARBINARY 数据。
- R2 输入参数，VARBINARY 数据。

注意：如果 R1、R2 长度不等，则对长度较小的参数低位补 0XFF 字节至与另一个参数对齐后再做按位运算。

3. BIT_COMPLEMENT

输出 R 的按位运算的反码。

语法如下:

```
FUNCTION BIT_COMPLEMENT(
    R     IN    VARBINARY
) RETURN VARBINARY;
```

参数详解

- R 输入参数，VARBINARY 数据。

4. BIT_OR

输出 R1 与 R2 的按位运算的逻辑或。

语法如下:

```
FUNCTION BIT_OR(
    R1    IN    VARBINARY,
    R2    IN    VARBINARY
) RETURN VARBINARY;
```

参数详解

- R1 输入参数，VARBINARY 数据。
- R2 输入参数，VARBINARY 数据。

注意：如果 R1、R2 长度不等，则对长度较小的参数低位补 0X00 字节至与另一个参数对齐后再做按位运算。

5. BIT_XOR

输出 R1 与 R2 的按位运算的逻辑异或。

语法如下：

```
FUNCTION BIT_XOR(
    R1    IN    VARBINARY,
    R2    IN    VARBINARY
) RETURN VARBINARY;
```

参数详解

- R1 输入参数，VARBINARY 数据。
- R2 输入参数，VARBINARY 数据。

注意：如果 R1、R2 长度不等，则对长度较小的参数低位补 0X00 字节至与另一个参数对齐后再做按位运算。

6. COMPARE

比较十六进制串数据 R1 和 R2。如果 R1 串长度< R2 串长度，R1 串用 PAD 填充至二者长度相等后再作比较。返回结果：数字，表示从几个数字开始不相等。0 表示相等。

语法如下：

```
FUNCTION COMPARE(
    R1              IN      VARBINARY,
    R2              IN      VARBINARY,
    PAD             IN      VARBINARY       DEFAULT 0X00
) RETURN NUMBER;
```

参数详解

- R1 输入参数，VARBINARY 数据。
- R2 输入参数，VARBINARY 数据。
- PAD 输入参数，VARBINARY 数据，缺省为 0X00。

7. CONCAT

按参数顺序分别连接各个十六进制串数。最多可连接 12 个。返回连接后的十六进制串数值。

语法如下：

```
FUNCTION CONCAT(
    R1      IN      VARBINARY   DEFAULT NULL,
    R2      IN      VARBINARY   DEFAULT NULL,
    R3      IN      VARBINARY   DEFAULT NULL,
    R4      IN      VARBINARY   DEFAULT NULL,
    R5      IN      VARBINARY   DEFAULT NULL,
    R6      IN      VARBINARY   DEFAULT NULL,
```

```
    R7          IN          VARBINARY     DEFAULT NULL,
    R8          IN          VARBINARY     DEFAULT NULL,
    R9          IN          VARBINARY     DEFAULT NULL,
    R10         IN          VARBINARY     DEFAULT NULL,
    R11         IN          VARBINARY     DEFAULT NULL,
    R12         IN          VARBINARY     DEFAULT NULL
) RETURN VARBINARY;
```

参数详解

- R1 输入参数，VARBINARY 数据，缺省为 NULL。
- R2 输入参数，VARBINARY 数据，缺省为 NULL。
- R3 输入参数，VARBINARY 数据，缺省为 NULL。
- R4 输入参数，VARBINARY 数据，缺省为 NULL。
- R5 输入参数，VARBINARY 数据，缺省为 NULL。
- R6 输入参数，VARBINARY 数据，缺省为 NULL。
- R7 输入参数，VARBINARY 数据，缺省为 NULL。
- R8 输入参数，VARBINARY 数据，缺省为 NULL。
- R9 输入参数，VARBINARY 数据，缺省为 NULL。
- R10 输入参数，VARBINARY 数据，缺省为 NULL。
- R11 输入参数，VARBINARY 数据，缺省为 NULL。
- R12 输入参数，VARBINARY 数据，缺省为 NULL。

8. CONVERT_RAW

将十六进制串数值从后指定的字符集到前指定的字符集转换。支持 UTF8 与其它字符集(GBK、BIG5、ISO_8859_9、EUC_JP、EUC_KR、KOI8R、GB18030、SQL_ASCII、

ISO_8859_1)的相互转换。

语法如下:

```
FUNCTION CONVERT_RAW (
    R              IN      VARBINARY,
    TO_CHARSET     IN      VARCHAR2,
    FROM_CHARSET   IN      VARCHAR2
) RETURN VARBINARY;
```

参数详解

- R 输入参数，VARBINARY 数据。
- TO_CHARSET 输入参数，字符集。指定转换后的字符集。
- FROM_CHARSET 输入参数，字符集。转换前的字符集。

9. COPIES

将十六进制串数值拷贝 N 次，依次排在前一个数的后面。返回新的十六进制数值。

语法如下:

```
FUNCTION COPIES(
    R    IN    VARBINARY,
    N    IN    NUMBER
) RETURN VARBINARY;
```

- R 输入参数，VARBINARY 数据。
- N 输入参数，NUMBER 类型，指拷贝 N 次。

10. LENGTH

返回十六进制串的字节长度。

语法如下:

```
FUNCTION LENGTH(
    R     IN    VARBINARY
) RETURN NUMBER;
```

- R 输入参数，VARBINARY 数据。

11. OVERLAY_RAW

用十六进制串 OVERLAY_STR 覆盖源十六进制串 TARGET 中指定的十六进制子串。

语法如下:

```
FUNCTION OVERLAY_RAW(
    OVERLAY_STR     IN      VARBINARY,
    TARGET          IN      VARBINARY,
    POS             IN      INTEGER     DEFAULT 1,
    LEN             IN      INTEGER     DEFAULT NULL,
    PAD             IN      VARBINARY   DEFAULT NULL
) RETURN VARBINARY;
```

参数详解

- OVERLAY_STR 十六进制串。
- TARGET 源十六进制串。
- POS 子串在源串 TARGET 中的起始位置。
- LEN 子串的长度。
- PAD OVERLAY_STR 串长度若小于 LEN 或 POS 超过了 TARGET 的长度，用 PAD 值代替。

12. REVERSE_RAW

将输入十六进制串的字符顺序反转后返回。

语法如下:

```
FUNCTION REVERSE_RAW (
    R     IN    VARBINARY
) RETURN VARBINARY;
```

- R 输入参数，VARBINARY 数据。

13. SUBSTR

返回十六进制串中从字节位置 POS 开始的 LEN 个字节。

语法如下:

```
FUNCTION SUBSTR(
    R          IN          VARBINARY,
    POS        IN          INTEGER,
    LEN        IN          INTEGER        DEFAULT NULL
) RETURN VARBINARY;
```

- R 输入参数，VARBINARY 数据。
- POS 子串在源串 TARGET 中的起始位置。
- LEN 子串的长度。

14. TRANSLATE

十六进制替换函数。将 R 字符串中的 FROM_SET 串，全部替换成 TO_SET 串。

语法如下:

```
FUNCTION TRANSLATE(
```

```
    R              IN      VARBINARY,
    FROM_SET       IN      VARBINARY,
    TO_SET         IN      VARBINARY
) RETURN VARBINARY;
```

参数详解

- R 源十六进制串。
- FROM_SET 被替换的十六进制串。
- TO_SET 替换后的十六进制串。

15. TRANSLITERATE

功能和 TRANSLATE 一样。

语法如下:

```
FUNCTION TRANSLITERATE(
    R              IN      VARBINARY,
    TO_SET         IN      VARBINARY    DEFAULT NULL,
    FROM_SET       IN      VARBINARY    DEFAULT NULL,
    PAD            IN      VARBINARY    DEFAULT NULL
) RETURN VARBINARY;
```

参数详解

- R 源十六进制串。
- FROM_SET 被替换的十六进制串。
- TO_SET 替换后的十六进制串。
- PAD 如果 TO_SET 小于 FROM_SET，少的部分用 PAD 值代替。如果 PAD 为 NULL，则以 0 从低位开始填充。

16. CAST_TO_RAW

ASCII 字符转化成十六进制数字。

语法如下:

```
FUNCTION CAST_TO_RAW(
    C    IN    VARCHAR2
) RETURN VARBINARY;
```

- C 输入参数，ASCII 字符。

17. CAST_TO_VARCHAR2

十六进制数字转化成对应的 ASCII 字符。

语法如下:

```
FUNCTION CAST_TO_VARCHAR2(
    R    IN    VARBINARY
) RETURN VARCHAR2;
```

- R 输入参数，VARBINARY 数据。

18. CAST_TO_INT

十六进制数字转化成整型十进制。

语法如下:

```
FUNCTION CAST_TO_INT(
    R              IN          VARBINARY,
    ENDIANESS      IN          INTEGER         DEFAULT 1
)RETURN   INTEGER;
```

参数详解

- R 输入参数，VARBINARY 数据。
- ENDIANESS 读取 R 数据的顺序，取值范围：1、2、3，缺省为 1。1 表示使用大端方式，2 表示使用小端方式，3 表示使用机器的大小端方式。

19. CAST_FROM_INT

整型十进制转化成十六进制数字。

语法如下:

```
FUNCTION CAST_FROM_INT(
    N               IN      INTEGER,
    ENDIANESS       IN      INTEGER     DEFAULT 1
) RETURN VARBINARY;
```

- N 输入参数，整型十进制数据。
- ENDIANESS 读取 N 数据的顺序，取值范围：1、2、3，缺省为 1。1 表示使用大端方式，2 表示使用小端方式，3 表示使用机器的大小端方式。

20. CAST_TO_BINARY_INTEGER

十六进制数字转化成整型十进制。

语法如下:

```
FUNCTION CAST_TO_BINARY_INTEGER (
    R               IN      VARBINARY,
    ENDIANESS       IN      INTEGER     DEFAULT 1
)RETURN  INTEGER;
```

参数详解

- R 输入参数，VARBINARY 数据。
- ENDIANESS 读取 R 数据的顺序，取值范围：1、2、3，缺省为 1。1 表示使用大端方式，2 表示使用小端方式，3 表示使用机器的大小端方式。

21. CAST_FROM_BINARY_INTEGER

整型十进制转化成十六进制数字。

语法如下:

```
FUNCTION CAST_FROM_BINARY_INTEGER (
    N               IN      INTEGER,
    ENDIANESS       IN      INTEGER     DEFAULT 1
) RETURN VARBINARY;
```

参数详解

- N 输入参数，整型十进制数据。
- ENDIANESS 读取 N 数据的顺序，取值范围：1、2、3，缺省为 1。1 表示使用大端方式，2 表示使用小端方式，3 表示使用机器的大小端方式。

22. CAST_TO_BINARY_DOUBLE

十六进制数字转化成 DOUBLE。

语法如下:

```
FUNCTION CAST_TO_BINARY_DOUBLE (
    R               IN      VARBINARY,
    ENDIANESS       IN      INTEGER     DEFAULT 1
)RETURN  DOUBLE;
```

参数详解

- R 输入参数，VARBINARY 数据。
- ENDIANESS 读取 R 数据的顺序，取值范围：1、2、3，缺省为 1。1 表示使用大端方式，2 表示使用小端方式，3 表示使用机器的大小端方式。

23. CAST_FROM_BINARY_DOUBLE

DOUBLE 转化成十六进制数字。

语法如下:

```
FUNCTION CAST_FROM_BINARY_DOUBLE (
    N           IN      DOUBLE,
    ENDIANESS   IN      INTEGER     DEFAULT 1
) RETURN VARBINARY;
```

参数详解

- N 输入参数，DOUBLE 类型数据。
- ENDIANESS 读取 N 数据的顺序，取值范围：1、2、3，缺省为 1。1 表示使用大端方式，2 表示使用小端方式，3 表示使用机器的大小端方式。

24. CAST_TO_BINARY_FLOAT

十六进制数字转化成 FLOAT。

语法如下:

```
FUNCTION CAST_TO_BINARY_FLOAT(
    R           IN      VARBINARY,
    ENDIANESS   IN      INTEGER     DEFAULT 1
)RETURN  FLOAT;
```

参数详解

- R 输入参数，VARBINARY 数据。
- ENDIANESS 读取 R 数据的顺序，取值范围：1、2、3，缺省为 1。1 表示使用大端方式，2 表示使用小端方式，3 表示使用机器的大小端方式。

25. CAST_FROM_BINARY_FLOAT

FLOAT 转化成十六进制数字。

语法如下:

```
FUNCTION CAST_FROM_BINARY_FLOAT(
    N               IN      DOUBLE,
    ENDIANESS       IN      INTEGER     DEFAULT 1
) RETURN VARBINARY;
```

参数详解

- N 输入参数，DOUBLE 类型数据。
- ENDIANESS 读取 N 数据的顺序，取值范围：1、2、3，缺省为 1。1 表示使用大端方式，2 表示使用小端方式，3 表示使用机器的大小端方式。

26. CAST_TO_NUMBER

十六进制数字转化成 NUMBER。

语法如下:

```
FUNCTION CAST_TO_NUMBER(
    R               IN      VARBINARY
)RETURN  NUMBER;
```

参数详解

- R 输入参数，VARBINARY 数据。

27. CAST_FROM_NUMBER

将 NUMBER 类型（DEC、DECIMAL）转化成十六进制数据。

语法如下:

```
FUNCTION CAST_FROM_NUMBER(
    N     IN     NUMBER
) RETURN VARBINARY;
```

参数详解

- N 输入参数，NUMBER 类型数据。

28.2 举例说明

使用包内的过程和函数之前，如果还未创建过系统包，请先调用系统过程创建系统包。

```
SP_CREATE_SYSTEM_PACKAGES (1,'UTL_RAW');
```

例 UTL_RAW 包的应用。

例1 xrange

```
select utl_raw.xrange(utl_raw.cast_to_raw('A'),utl_raw.cast_to_raw('Z')) from dual;
```

查询结果如下:

```
0x414243444454647484 94A4B4C4D4E4F505152535455565758595A
```

例2 bit_and

```
select utl_raw.bit_and('0102F3','F30201') from dual;
```

查询结果如下:

```
0x010201
```

例3 bit_complement

```
select utl_raw.bit_complement('0102F3') from dual;
```

查询结果如下:

```
0xFEFD0C
```

例4 bit_or

```
select utl_raw.bit_or('0102F3','F30201') from dual;
```

查询结果如下:

```
0XF302F3
```

例5 bit_xor

```
SELECT utl_raw.bit_xor('0102F3','F30201') from dual;
```

查询结果如下:

```
0XF200F2
```

例6 compare

```
select utl_raw.compare(utl_raw.cast_to_raw('ABC'),utl_raw.cast_to_raw('ABCD')) from
dual;
```

查询结果如下:

```
4
```

例7 concat

```
select utl_raw.concat('A','41','B','42') from dual;
```

查询结果如下:

```
0x0A410B42
```

例8 convert_raw

```
select UTL_RAW.CONVERT_raw('E4B8ADE69687', 'GBK', 'UTF8') from dual;
```

查询结果如下:

```
0xD6D0CEC4
```

```
select UTL_RAW.CONVERT_raw('D6D0CEC4','UTF8', 'GBK') from dual;
```

查询结果如下：

```
0xE4B8ADE69687
```

例9 copies

```
select utl_raw.copies('A',6) from dual;
```

查询结果如下：

```
0x0A0A0A0A0A0A
```

例10 length

```
select utl_raw.LEngth(UTL_RAW.CAST_TO_RAW('ABCD')) from dual;
```

查询结果如下：

```
 4
```

例11 overlay_raw

```
select utl_raw.overlay_raw (utl_raw.cast_to_raw('D'),utl_raw.cast_to_raw('AAAC'),1,1) from
dual;
```

查询结果如下：

```
0x44414143
```

```
select
utl_raw.overlay_raw(utl_raw.cast_to_raw('D'),utl_raw.cast_to_raw('AAAC'),1,2,utl_raw.cast_
to_raw(' ')) from dual;
```

查询结果如下：

```
0X44204143
```

例12 reverse_raw

```
select utl_raw.reverse_raw(utl_raw.cast_to_raw('123')) from dual;
```

查询结果如下:

```
0x333231
```

例13 substr

```
select utl_raw.substr(0x41424344454142434445414243444 5,6,2) from dual;
```

查询结果如下:

```
0x4142
```

例14 translate

```
select
UTL_RAW.translate(utl_raw.cast_to_raw('aDe'),utl_raw.cast_to_raw('AD'),utl_raw.cast_to_raw('ad')) from dual;
```

查询结果如下:

```
0x616465  //即 ade
```

例15 translITERATE

```
select
UTL_RAW.translITERATE(utl_raw.cast_to_raw('FaDe'),utl_raw.cast_to_raw('adef'),utl_raw.cast_to_raw('ADEF'),utl_raw.cast_to_raw(' ')) from dual;
```

查询结果如下:

```
0x66616465      //即 fade
```

例16 CAST_TO_RAW

```
select UTL_RAW.CAST_TO_RAW(10)  from dual;
```

查询结果如下:

```
0x3130
```

例17 cast_to_varchar2

```
select utl_raw.cast_to_varchar2('40') from dual;
```

查询结果如下:

```
@
```

例18 cast_to_INT

```
select utl_raw.cast_to_int('00000064',1) from dual;
```

查询结果如下:

```
100
```

```
select utl_raw.cast_to_int('00000064',2) from dual;
```

查询结果如下:

```
1677721600
```

例19 cast_from_INT

```
select utl_raw.cast_from_int(10) from dual;
```

查询结果如下:

```
0x0000000A
```

```
select utl_raw.cast_FROM_int(94311,2) from dual;
```

查询结果如下:

```
0x67700100
```

例20 cast_from_number

```
select UTL_RAW.cast_FROM_number(10) from dual;
```

查询结果如下:

```
0xC10B
```

29 UTL_TCP 包

许多的应用程序都是基于 TCP/IP 协议的，UTL_TCP 包提供了与外部的 TCP/IP 服务器通讯和基本的收发数据的功能。

29.1 相关方法

包中使用到的类型：

1. CONNECTION

定义 CONNECTION 为 RECORD 类型，为包内的过程或函数使用。

语法如下：

```
TYPE CONNECTION IS RECORD (
    REMOTE_HOST     VARCHAR2(255),
    REMOTE_PORT     INT,
    LOCAL_HOST      VARCHAR2(255),
    LOCAL_PORT      INT,
    CHARSET         VARCHAR2(30),
    NEWLINE         VARCHAR2(2),
    TX_TIMEOUT      INT,
    PRIVATE_SD      INT
);
```

参数详解

- REMOTE_HOST，REMOTE_PORT

TCP 远程服务器的名字（或 IP 地址）及端口号，由 OPEN_CONNECTION 过程传入。

- LOCAL_HOST，LOCAL_PORT

本地服务器的相关信息，保留字段，暂无意义。

- CHARSET

字符数据传输时采用的字符集编码。

- NEWLINE

换行符序列，用于将文本行转换为在线字符集，默认为 0d0a。

- TX_TIMEOUT

尝试连接到远程主库的等待响应时间，同样由 OPEN_CONNECTION 过程传入。

- PRIVATE_SD

为内部标识 CONNECTION 的 ID。

2. AVAILABLE

当前能够读取的连接字符数(最大不超过 96 字节)。

语法如下:

```
FUNCTION AVAILABLE(
    C           IN OUT      CONNECTION,
    TIMEOUT     IN          INT         DEFAULT 0
)RETURN INT;
```

参数详解

- C　一个 OPEN_CONNECTION 打开的、有效的 CONNECTION。
- TIMEOUT　如果没有数据，则在指定的 TIMEOUT 时间返回。

返回值

当前获取的一个 TCP/IP 连接字节数。

3. CLOSE_ALL_CONNECTION

关闭所有的连接。

语法如下:

```
PROCEDURE CLOSE_ALL_CONNECTION();
```

4. CLOSE_ALL_CONNECTIONS

关闭所有的连接，用法同 CLOSE_ALL_CONNECTION()。

语法如下:

```
PROCEDURE CLOSE_ALL_CONNECTIONS();
```

5. CLOSE_CONNECTION

关闭指定的连接。

语法如下:

```
PROCEDURE CLOSE_CONNECTION (
    C           IN OUT          CONNECTION
);
```

参数详解

- C 一个 OPEN_CONNECTION 打开的、有效的 CONNECTION。

6. GET_RAW

读取指定长度（二进制的数据）的数据。

语法如下:

```
FUNCTION GET_RAW  (
    C          IN OUT        CONNECTION,
    LEN        IN            INT             DEFAULT 1,
    PEEK       IN            BOOLEAN         DEFAULT FALSE
)RETURN VARBINARY;
```

参数详解

- C 一个 OPEN_CONNECTION 打开的、有效的 CONNECTION。
- LEN 要收到的数据的长度，取值范围 0~32767，默认值 1。当参数值超出取值范围时报错。
- PEEK 该选项为 TRUE, 则指只是查看队列的数据, 不会从队列中删除, 下次还能够读到; 为 FALSE，则会删除队列中的数据。默认为 FALSE。

返回值

读取到的二进制数据。

7. OPEN_CONNECTION

打开一个连接到外部服务器的连接。

语法如下:

```
FUNCTION OPEN_CONNECTION(
    REMOTE_HOST      IN        VARCHAR2,
    REMOTE_PORT      IN        INT,
    LOCAL_HOST       IN        VARCHAR2    DEFAULT NULL,
```

```
    LOCAL_PORT      IN      INT         DEFAULT NULL,
    CHARSET         IN      VARCHAR2    DEFAULT NULL,
    NEWLINE         IN      VARCHAR2    DEFAULT CRLF,
    TX_TIMEOUT      IN      INT         DEFAULT NULL
) RETURN CONNECTION;
```

参数详解

- REMOTE_HOST 远程主机的名字，或者 IP 地址。
- REMOTE_PORT 远程主机的端口。
- LOCAL_HOST 本地主机的名字（一般不用设置）。
- LOCAL_PORT 本地主机的端口（一般不用设置）。
- CHARSET 用于编码转换的在线字符集。
- NEWLINE 换行符序列，用于将文本行转换为在线字符集。
- TX_TIMEOUT 尝试连接到远程主机的等待时间。

返回值

一个连接到指定外部服务器的链接。

8. READ_RAW

以二进制形式读取一行数据。

语法如下:

```
FUNCTION READ_RAW (
    C       IN OUT      CONNECTION,
    DATA    OUT         VARBINARY,
    LEN     IN          INT         DEFAULT 1,
```

```
    PEEK        IN              BOOLEAN             DEFAULT FALSE
) RETURN INT;
```

参数详解

- C 一个 OPEN_CONNECTION 打开的、有效的 CONNECTION。
- DATA 用于存放读到的数据。
- LEN 要收到的数据的长度，取值范围 0~32767，默认值 1。当参数值大于 32767 时，按照 32767 读取。
- PEEK 该选项为 TRUE，则指只是查看队列的数据，不会从队列中删除，下次还能够读到；为 FALSE，则会删除队列中的数据。默认为 FALSE。

返回值

收到的数据的字节长度。

9. WRITE_RAW

发送一行二进制数据。

语法如下:

```
FUNCTION WRITE_RAW(
    C           IN OUT      CONNECTION,
    DATA        IN          VARBINARY,
    LEN         IN          INT             DEFAULT NULL
) RETURN INT;
```

参数详解

- C 一个 OPEN_CONNECTION 打开的、有效的 CONNECTION。
- DATA 存放要发送的数据。

- LEN　要发送数据的长度，指定的长度不能超过 DATA 中的实际数据长度，否则报错，默认为 NULL。

返回值

发送的二进制数据的字节长度。只有成功发送才会有返回值。

10. WRITE_LINE

将文本行转换为在线字符集，并发送至连接的服务器。函数 OPEN_CONNECTION 中设置的换行符序列 NEWLINE 会在消息传输之前附加到消息中。

语法如下:

```
FUNCTION WRITE_LINE(
    C          IN OUT        CONNECTION,
    DATA       IN            VARCHAR2        DEFAULT NULL
) RETURN INT;
```

参数详解

- C　一个 OPEN_CONNECTION 打开的、有效的 CONNECTION。
- DATA　存放要发送的数据。

返回值

实际写入的字符数。

11. WRITE_TEXT

将文本行转换为在线字符集，并发送至连接的服务器。

语法如下:

```
FUNCTION WRITE_TEXT(
```

```
    C           IN OUT      CONNECTION,
    DATA        IN          VARCHAR2,
    LEN         INT         DEFAULT NULL
) RETURN INT;
```

参数详解

- C 一个 OPEN_CONNECTION 打开的、有效的 CONNECTION。
- DATA 存放要发送的数据。
- LEN 要发送数据的长度，指定的长度不能超过 DATA 中的实际数据长度，否则报错，默认为 NULL。。

返回值

实际写入的字符数。

12. GET_LINE

读取一行数据。

语法如下:

```
FUNCTION GET_LINE(
    C               IN OUT  CONNECTION,
    REMOVE_CRLF     IN      BOOLEAN     DEFAULT FALSE,
    PEEK            IN      BOOLEAN     DEFAULT FALSE
) RETURN VARCHAR2;
```

参数详解

- C 一个 OPEN_CONNECTION 打开的、有效的 CONNECTION。
- REMOVE_CRLF 用于表示是否需要移除接收数据尾部的换行符，默认为 FALSE。

- PEEK 该选项为 TRUE，则指只是查看队列的数据，不会从队列中删除，下次还能够读到；为 FALSE，则会删除队列中的数据。默认为 FALSE。

返回值

读取到的一行数据。

13. READ_TEXT

读取一行数据。

语法如下:

```
FUNCTION READ_TEXT(
    C          IN OUT    CONNECTION,
    DATA       IN OUT    VARCHAR2,
    LEN        IN        INT              DEFAULT 1,
    PEEK       IN        BOOLEAN          DEFAULT FALSE
) RETURN INT;
```

参数详解

- C 一个 OPEN_CONNECTION 打开的、有效的 CONNECTION。
- DATA 用于存放接收到的字符数据。
- LEN 用于指定希望接收数据的长度，取值范围 0~32767，默认为 1。当参数值大于 32767 时，按照 32767 读取。
- PEEK 该选项为 TRUE，则指只是查看队列的数据，不会从队列中删除，下次还能够读到；为 FALSE，则会删除队列中的数据。默认为 FALSE。

返回值

实际接收到的数据长度。

14. GET_TEXT

读取数据。

语法如下:

```
FUNCTION GET_TEXT(
    C       IN OUT  CONNECTION,
    LEN     IN      INT         DEFAULT 1,
    PEEK    IN      BOOLEAN     DEFAULT FALSE
) RETURN VARCHAR2;
```

参数详解

- C 一个 OPEN_CONNECTION 打开的、有效的 CONNECTION。
- LEN 用于指定希望读取的数据的长度，取值范围 0~32767，单位：字符，默认为 1。当参数值超出取值范围时报错。
- PEEK 该选项为 TRUE，则指只是查看队列的数据，不会从队列中删除，下次还能够读到；为 FALSE，则会删除队列中的数据。默认为 FALSE。

返回值

读取到的数据。

15. FLUSH

立刻将缓冲区中数据发送。

语法如下:

```
PROCEDURE FLUSH(
    C   IN OUT  CONNECTION
```

```
);
```

参数详解

- C 一个 OPEN_CONNECTION 打开的、有效的 CONNECTION。

29.2 举例说明

使用包内的过程和函数之前，如果还未创建过系统包，请先调用系统过程创建系统包。

```
SP_CREATE_SYSTEM_PACKAGES (1,'UTL_TCP');

SET SERVEROUTPUT ON;  //PRINT 需要设置这条语句，才能打印出消息
```

例 UTL_TCP 包示例。

```
DECLARE
    HOST VARCHAR(128);
    PORT INT;
    RET INT;
    LEN INT;
    DATA VARBINARY(512);
    SHOW VARBINARY(128);
    C UTL_TCP.CONNECTION;
BEGIN
    HOST := '223.254.17.100';
    PORT := 25;
    C := UTL_TCP.OPEN_CONNECTION(HOST,PORT);
    RET := UTL_TCP.WRITE_RAW(C, RAWTOHEX('DAMENG DATABASE'),15);
    PRINT RET;
    LEN := 48;
```

```
    RET := UTL_TCP.AVAILABLE(C,0);
    PRINT RET;
    SHOW := UTL_TCP.GET_RAW(C,LEN);  //UTL_TCP 包仅用于发送请求，响应信息由远程 HOST 决
定，此处获取的二进制流信息根据不同的远程 HOST 可能为不同的响应结果，故下述打印结果以实际为准
    PRINT SHOW;
    RET := UTL_TCP.READ_RAW(C,DATA,LEN);
    UTL_TCP.CLOSE_CONNECTION(C);
END;
/
```

结果如下：

```
15
0
323230206D61696C2E64616D656E672E636F6D2045534D545020506F73746669780D0A3432312
0342E342E32206D6169
```

30 UTL_URL 包

UTL_URL 系统包用于对 URL 进行转码与解码操作。URL 是一个指向网页、图片等网络资源的字符串。访问网络资源使用的是 HTTP 协议。例如，DM 网站的 URL 为：http://www.dameng.com。通常，一个正确的 URL 字符串包含字母、数字、标点符号三种规范字符。这些字符都是非保留字。如果 URL 中含有非规范字符，那么就需要转码；如果 URL 中含有保留字符，则根据用户需要决定是否转码。

URL 中可以使用的规范字符和保留字符。如下所示：

- 规范字符包括：

[A-Z]、[a-z]、[0-9]；

减号(-)、下划线(_)、句点(.)、感叹号(!)、波浪号(~)、星号(*)、重音符号(')、左圆括号((()、右圆括号())。

- 保留字符包括：

分号(;)、斜线(/)、问号(?)、冒号(:)、(@)、地址符(&)、等号(=)、加号(+)、美元符号($)、逗号(,)。

30.1 相关方法

1. UTL_URL.ESCAPE

对 URL 中的非法字符（及保留字符）进行转码。

语法如下：

```
UTL_URL.ESCAPE (
    URL                     IN      VARCHAR,
    ESCAPE_RESERVED_CHARS   IN      BOOLEAN     DEFAULT FALSE,
```

```
    URL_CHARSET                 IN          VARCHAR2      DEFAULT   'GBK'
)RETURN VARCHAR2;
```

参数详解

- URL 待转码的 URL。
- ESCAPE_RESERVED_CHARS 指明是否转义 URL 的保留字符。
- URL_CHARSET

  指定 URL 的转码格式使用的字符集。字符集包含：UTF8、GBK、BIG5、ISO_8859_9、EUC_JP、EUC_KR、KOI8R、ISO_8859_1、SQL_ASCII、GB18030、ISO_8859_11。

返回值

返回转码后的 URL。

使用说明:

此存储函数转义基于 RFC2396 标准的 URL 中包含的非规范字符。

2. UTL_URL.UNESCAPE

对 URL 进行解码。

语法如下:

```
UTL_URL.UNESCAPE (
    URL                 IN          VARCHAR,
    URL_CHARSET         IN          VARCHAR2      DEFAULT 'GBK'
)RETURN VARCHAR2;
```

参数详解

- URL 待解码 URL。
- URL_CHARSET

待解码 URL 的编码格式使用的字符集。字符集包含：UTF8、GBK、BIG5、ISO_8859_9、EUC_JP、EUC_KR、KOI8R、ISO_8859_1、SQL_ASCII、GB18030、ISO_8859_11。

返回值

返回解码后的 URL。

30.2 举例说明

使用包内的过程和函数之前，如果还未创建过系统包，请先调用系统过程创建系统包。

```
SP_CREATE_SYSTEM_PACKAGES (1,'UTL_URL');
```

第一步，对 URL（http://www.dameng.com/a url with space.html）使用 UTF8 字符集进行转码：

```
SELECT UTL_URL.ESCAPE ('http://www.dameng.com/a url with space.html', FALSE, 'UTF8') FROM DUAL;
```

查询结果如下：

```
http://www.dameng.com/a%20url%20with%20space.html
```

第二步，对上面得到的 URL 进行解码：

```
SELECT UTL_URL.UNESCAPE ('http://www.dameng.com/a%20url%20with%20space.html', 'UTF8');
```

查询结果如下：

```
http://www.dameng.com/a url with space.html
```

比较可以发现，经过转码再解码后的 URL 与原来相同。

31 DBMS_SCHEDULER 包

DBMS_SCHEDULER 包提供一系列调度相关的存储过程及方法供 PL/SQL 调用。

31.1 相关方法

31.1.1 SCHEDULE 对象

1. CREATE_SCHEDULE 过程

创建一个调度。

语法如下:

```
PROCEDURE DBMS_SCHEDULER.CREATE_SCHEDULE(
    SCHEDULE_NAME       IN      VARCHAR,
    START_DATE          IN      TIMESTAMP WITH TIME ZONE DEFAULT NULL,
    REPEAT_INTERVAL     IN      VARCHAR,
    END_DATE            IN      TIMESTAMP WITH TIME ZONE DEFAULT NULL,
    COMMENTS                IN      VARCHAR     DEFAULT NULL
);
```

参数详解

- SCHEDULE_NAME

调度名称。调度名称不区分大小写，要求在包模式下名称唯一。不能为空，若未设置，则报错。

- START_DATE

调度第一次有效日期或者时间。对于重复的调度，START_DATE 是个参照值，这种情况

START_DATE 不是调度的开始时间，调用的开始决定于 REPEAT_INTERVAL 的设置。

START_DATE 用于决定调度的第一个实例。

- REPEAT_INTERVAL

  调度重复间隔，用于指定调用隔多久重复一次。它是基于日历语法的表达式。对于命名的调度，REPEAT_INTERVAL 不支持 PL/SQL 表达式。

- END_DATE

  作业停止运行时间。如果为未设置，作业则一直有效。

- COMMENTS

  调度相关的评论或注释。默认为空。

2. DROP_SCHEDULE 过程

删除一个调度。

语法如下:

```
PROCEDURE DBMS_SCHEDULER.DROP_SCHEDULE(
    SCHEDULE_NAME       IN      VARCHAR,
    FORCE               IN      BOOLEAN         DEFAULT FALSE
);
```

参数详解

- SCHEDULE_NAME

  调度名称。仅支持单独一个调度名称。

- FORCE

  设为 FALSE，说明调度删除前不能被任何作业引用，否则报错。

设为 TRUE，所有引用这个调度的作业在调度删除之前都将调为无效。

使用该调度，但正在运行的作业不受影响。

如果为 NULL，则取值为 FALSE。

31.1.2 PROGRAM 对象

1. CREATE_PROGRAM 过程

创建一个程序。通过 CREATE_PROGRAM 方法创建的 PROGRAM 对象，在视图 DBA_SCHEDULER_PROGRAMS 或 USER_SCHEDULER_PROGRAMS 中可以查看到。

语法如下:

```
PROCEDURE DBMS_SCHEDULER.CREATE_PROGRAM(
    PROGRAM_NAME              IN      VARCHAR,
    PROGRAM_TYPE              IN      VARCHAR,
    PROGRAM_ACTION            IN      VARCHAR,
    NUMBER_OF_ARGUMENTS       IN      INT         DEFAULT 0,
    ENABLED                   IN      BOOLEAN     DEFAULT FALSE,
    COMMENTS                  IN      VARCHAR     DEFAULT NULL
);
```

参数详解

- PROGRAM_NAME

程序名称。程序名称不区分大小写，要求在包模式下名称唯一。若未设置，则报错。

- PROGRAM_TYPE

程序类型。未设置，则报错。支持类型如下:

PLSQL_BLOCK: 说明程序是一个 PL/SQL 语句块, 即 DM 的 SQL 程序设计里的语句块。此类型的程序或者作业不允许设置参数个数，也就是说设置的参数个数必须为 0;

STORED_PROCEDURE: 说明程序是一个 PL/SQL 存储过程。支持存储过程，不支持带有返回值的函数。包含输入输出或者输出参数的 PL/SQL 存储过程不支持。

- PROGRAM_ACTION

定义程序的动作。可能的动作如下:

对于 PL/SQL 语句块，程序动作就是去执行 PL/SQL 代码。

对于存储过程，程序动作是存储过程的名称。如果存储过程与作业不属于同一个模式，则需要指定存储过程名称的时候，指定模式名。

若未指定程序动作，则报错。

- number_of_arguments

定义程序包含的参数的个数。若未设置，则默认为 0。一个程序最多可以指定 255 个参数。

- enabled

指定程序是否以激活的方式创建。若设置为 TRUE，则执行合法性检测，检测成功，则创建程序并置 ENABLED 状态。默认设置为 FALSE，那么程序不以激活的方式创建，可以在程序使用之前通过调用 ENABLE 过程来激活。

如果为 NULL，则取值为 FALSE。

- COMMENTS

程序的相关评论或者注释。默认为空。

2. DEFINE_PROGRAM_ARGUMENT 过程

该过程为重载过程。通过 DEFINE_PROGRAM_ARGUMENT 方法定义的 PROGRAM 参数信息，可以在 DBA_SCHEDULER_PROGRAM_ARGS 或 USER_SCHEDULER_PROGRAM_ARGS 视图中查看得到。

1) 定义一个程序参数，不带默认值。

语法如下:

```
PROCEDURE DBMS_SCHEDULER.DEFINE_PROGRAM_ARGUMENT(
    PROGRAM_NAME            IN      VARCHAR,
    ARGUMENT_POSITION       IN      INT,
    ARGUMENT_NAME           IN      VARCHAR     DEFAULT NULL,
    ARGUMENT_TYPE           IN      VARCHAR,
    OUT_ARGUMENT            IN      BOOLEAN     DEFAULT FALSE
);
```

2) 定义一个程序参数，带默认值。

语法如下:

```
PROCEDURE DBMS_SCHEDULER.DEFINE_PROGRAM_ARGUMENT(
    PROGRAM_NAME            IN      VARCHAR,
    ARGUMENT_POSITION       IN      INT,
    ARGUMENT_NAME           IN      VARCHAR     DEFAULT NULL,
    ARGUMENT_TYPE           IN      VARCHAR,
```

```
    DEFAULT_VALUE           IN      VARCHAR,
    OUT_ARGUMENT            IN      BOOLEAN         DEFAULT FALSE
);
```

参数详解

- PROGRAM_NAME

  待定义参数的程序名。该程序必须已经存在。

- ARGUMENT_POSITION

  待定义参数的位置，可选值，从 1 开始到 NUMBER_OF_ARGUMENTS。若指定位置已经定义过，则将被覆盖。

- ARGUMENT_NAME

  定义参数名称，可选项。定义参数名称不区分大小写，若设置，则要确保在程序所有参数名中唯一。若设置，则可能被其他包过程使用，包括 SET_JOB_ARGUEMENT_VALUE 过程。

- ARGUMENT_TYPE

  定义参数使用数据的类型。调度不证实也不使用这个类型。只有在程序用户决定赋值给参数时才使用。所有可用的 SQL 数据类型均允许，不支持用户自定义类型。

- DEFAULT_VALUE

  参数默认值。若作业未给参数设值，则将默认值赋值给对应参数。

- OUT_ARGUMENT

  预留参数，以后使用。必须设置为 FALSE。如果为 NULL，则取值为 FALSE。

3. DROP_PROGRAM 过程

删除一个程序。

语法如下:

```
PROCEDURE DBMS_SCHEDULER.DROP_PROGRAM(
    PROGRAM_NAME        IN      VARCHAR,
    FORCE               IN      BOOLEAN         DEFAULT FALSE
);
```

参数详解

- PROGRAM_NAME

  待删除的程序名。

- FORCE

  设为 FALSE，说明程序删除前不能被任何作业引用，否则报错。

  设为 TRUE，所有引用这个程序的作业在调度删除之前都将调为无效。

  使用该程序，且正在运行的作业不受影响，并允许继续运行。

  如果为 NULL，则取值为 FALSE。

31.1.3 JOB 对象

1. CREATE_JOB 过程

创建作业。通过 CREATE_JOB 创建的 JOB 对象，可以在 DBA_SCHEDULER_JOBS 或 USER_SCHEDULER_JOBS 视图中查看得到。

1） 不使用已经存在的程序（PROGRAM）或者调度（SCHEDULE）创建作业。

语法如下:

```
PROCEDURE DBMS_SCHEDULER.CREATE_JOB(
    JOB_NAME                IN      VARCHAR,
    JOB_TYPE                IN      VARCHAR,
    JOB_ACTION              IN      VARCHAR,
    NUMBER_OF_ARGUMENTS     IN      INT             DEFAULT 0,
    START_DATE              IN      TIMESTAMP WITH TIME ZONE DEFAULT NULL,
    REPEAT_INTERVAL         IN      VARCHAR     DEFAULT NULL,
    END_DATE                IN      TIMESTAMP WITH TIME ZONE DEFAULT NULL,
    JOB_CLASS               IN      VARCHAR         DEFAULT 'DEFAULT_JOB_CLASS',
    ENABLED                 IN      BOOLEAN         DEFAULT FALSE,
    AUTO_DROP               IN      BOOLEAN         DEFAULT TRUE,
    COMMENTS                IN      VARCHAR         DEFAULT NULL,
    CREDENTIAL_NAME         IN      VARCHAR         DEFAULT NULL,
    DESTINATION_NAME        IN      VARCHAR         DEFAULT NULL
);
```

2） 使用命名程序（PROGRAM）和命名调度（SCHEDULE）创建作业。

语法如下:

```
PROCEDURE DBMS_SCHEDULER.CREATE_JOB(
    JOB_NAME                IN      VARCHAR,
```

```
    PROGRAM_NAME        IN      VARCHAR,
    SCHEDULE_NAME       IN      VARCHAR,
    JOB_CLASS           IN      VARCHAR         DEFAULT 'DEFAULT_JOB_CLASS',
    ENABLED             IN      BOOLEAN         DEFAULT FALSE,
    AUTO_DROP           IN      BOOLEAN         DEFAULT TRUE,
    COMMENTS            IN      VARCHAR         DEFAULT NULL,
    JOB_STYLE           IN      VARCHAR         DEFAULT 'REGULAR',
    CREDENTIAL_NAME     IN      VARCHAR         DEFAULT NULL,
    DESTINATION_NAME    IN      VARCHAR         DEFAULT NULL
);
```

3） 使用命名程序（PROGRAM）和内置调度（SCHEDULE）创建作业。

语法如下:

```
PROCEDURE DBMS_SCHEDULER.CREATE_JOB(
    JOB_NAME            IN      VARCHAR,
    PROGRAM_NAME        IN      VARCHAR,
    START_DATE          IN      TIMESTAMP WITH TIME ZONE DEFAULT NULL,
    REPEAT_INTERVAL     IN      VARCHAR         DEFAULT NULL,
    END_DATE            IN      TIMESTAMP WITH TIME ZONE DEFAULT NULL,
    JOB_CLASS           IN      VARCHAR         DEFAULT 'DEFAULT_JOB_CLASS',
    ENABLED             IN      BOOLEAN         DEFAULT FALSE,
    AUTO_DROP           IN      BOOLEAN         DEFAULT TRUE,
    COMMENTS            IN      VARCHAR         DEFAULT NULL,
```

```
    JOB_STYLE               IN      VARCHAR         DEFAULT 'REGULAR',
    CREDENTIAL_NAME         IN      VARCHAR         DEFAULT NULL,
    DESTINATION_NAME        IN      VARCHAR         DEFAULT NULL
);
```

4） 使用命名调度（SCHEDULE）和内置程序（PROGRAM）创建作业。

语法如下:

```
PROCEDURE CREATE_JOB (
    JOB_NAME                IN      VARCHAR,
    SCHEDULE_NAME           IN      VARCHAR,
    JOB_TYPE                IN      VARCHAR,
    JOB_ACTION              IN      VARCHAR,
    NUMBER_OF_ARGUMENTS     IN      INT             DEFAULT 0,
    JOB_CLASS               IN      VARCHAR         DEFAULT 'DEFAULT_JOB_CLASS',
    ENABLED                 IN      BOOLEAN         DEFAULT FALSE,
    AUTO_DROP               IN      BOOLEAN         DEFAULT TRUE,
    COMMENTS                IN      VARCHAR         DEFAULT NULL,
    CREDENTIAL_NAME         IN      VARCHAR         DEFAULT NULL,
    DESTINATION_NAME        IN      VARCHAR         DEFAULT NULL
);
```

参数详解

- JOB_NAME

作业名称。作业名称不区分大小写，要求在包模式下名称唯一，在其他模式下使用时，必须加模式名前缀。若未设置，则报错。使用 GENERATE_JOB_NAME 函数可以自动生成一个作业名。详见 **GENERATE_JOB_NAME 函数**。

- JOB_TYPE

  创建作业类型。未设置，则报错。支持类型如下：

  PLSQL_BLOCK：指定作业是一个匿名的 PL/SQL 语句块。此类型的程序或者作业不允许设置参数个数，也就是说设置的参数个数必须为 0；

  STORED_PROCEDURE：指定作业是一个 PL/SQL 存储过程。支持存储过程，不支持带有返回值的函数。

- JOB_ACTION

  定义作业的动作。可能的动作如下：

  对于 PL/SQL 语句块，程序动作就是去执行 PL/SQL 代码。

  对于存储过程，程序动作是存储过程的名称。如果存储过程与作业不属于同一个模式，则需要指定存储过程名称的时候，指定模式名。

  若未指定作业动作，则报错。

- number_of_arguments

  定义作业预期的参数个数。若未设置，则默认为 0。一个程序最多可以指定 255 个参数。

- PROGRAM_NAME

  和作业关联的 program 名称，不区分大小写。

- SCHEDULE_NAME

作业使用的调度名称，不区分大小写。

- START_DATE

作业起始日期。如果 START_DATE 和 REPEAT_INTERVAL 都为 NULL，表示作业在 ENABLE 后立即执行。

- REPEAT_INTERVAL

作业重复执行及间隔时长。支持日历语法格式和 PL/SQL 表达式。

- END_DATE

作业结束日期。

- JOB_CLASS

不支持，仅作兼容参数用。

- enabled

指定作业创建时是否启用。默认为 FALSE。作业如果未启用，则系统不会调度执行。作业创建时设置为 TRUE，会去检查作业是否有效。作业 ENABLE 时，如果作业所使用的 PROGRAM 无效，作业创建失败。

如果为 NULL，则取值为 FALSE。

- AUTO_DROP

如果为 TRUE，作业自动删除。默认为 TRUE。自动删除的条件如下:

  - 作业的结束日期已经过期
  - 作业不是重复执行作业，并且只执行一次

如果为 NULL，则取值为 FALSE，作业不会自动删除。

- COMMENTS

  程序的相关评论或者注释。默认为空。

- JOB_STYLE

  不支持，只作兼容参数用。

- CREDENTIAL_NAME

  不支持，只作兼容参数用。

- DESTINATION_NAME

  不支持，只作兼容参数用。

2. SET_JOB_ARGUMENT_VALUE 过程

重载过程。通过 SET_JOB_ARGUMENT_VALUE 设置的 JOB 参数值，可以在 DBA_SCHEDULER_JOB_ARGS 或 USER_SCHEDULER_JOB_ARGS 视图中查询得到。

1) 通过指定位置设置参数值。

语法如下:

```
PROCEDURE DBMS_SCHEDULER.SET_JOB_ARGUMENT_VALUE(
    JOB_NAME                IN      VARCHAR,
    ARGUMENT_POSITION       IN      INT,
    ARGUMENT_VALUE          IN      VARCHAR
);
```

2） 通过指定参数名称，设置相应参数值。仅适用于作业使用了已经存在的程序对象，并且程序参数通过调用 DEFINE_PROGRAM_ARGUMENT 过程定义了参数名称。

语法如下:

```
PROCEDURE DBMS_SCHEDULER.SET_JOB_ARGUMENT_VALUE(
    JOB_NAME                IN      VARCHAR,
    ARGUMENT_NAME           IN      VARCHAR,
    ARGUMENT_VALUE          IN      VARCHAR
);
```

参数详解

- JOB_NAME

  待设置参数值的作业名称。

- ARGUMENT_NAME

  待设置参数值的程序参数名称，不区分大小写。

- ARGUMENT_POSITION

  待设置参数值的程序参数位置，从 1 开始。

- ARGUMENT_VALUE

  设置到程序参数上的新值，暂仅支持字符串类型的值。

3. DROP_JOB 过程

删除一个作业过程。作业删除时，作业上所有设置参数值也都删除。

语法如下:

```
PROCEDURE DBMS_SCHEDULER.DROP_JOB(
    JOB_NAME                IN      VARCHAR,
    FORCE                   IN      BOOLEAN     DEFAULT FALSE,
    DEFER                   IN      BOOLEAN     DEFAULT FALSE,
    COMMIT_SEMANTICS        IN      VARCHAR2    DEFAULT 'STOP_ON_FIRST_ERROR'
);
```

参数详解

- JOB_NAME

  待删除的作业名。

- FORCE

  设为 TRUE，调度首先尝试中止正在运行的作业实例（通过调用 STOP_JOB 过程，参数 force 设置 FALSE）。如果为 NULL，则取值为 FALSE。参数 DEFER 和参数 FORCE 都设为 TRUE 时无意义，因此不允许同时设为 TRUE。

- DEFER

  设为 TRUE，调度运行正在运行的作业完成，然后删除。如果为 NULL，则取值为 FALSE。参数 DEFER 和参数 FORCE 都设为 TRUE 时无意义，因此不允许同时设为 TRUE。

- COMMIT_SEMANTICS

  提交语义，暂不处理。

4. RUN_JOB 过程

立即执行一次作业。

语法如下:

```
PROCEDURE RUN_JOB (
    JOB_NAME                IN      VARCHAR,
    USE_CURRENT_SESSION     IN      BOOLEAN     DEFAULT TRUE
);
```

参数详解

- JOB_NAME

待运行的作业名。

- USE_CURRENT_SESSION

作业是否运行在和存储过程被调用的同一个会话中。USE_CURRENT_SESSION 为 TRUE 时在当前会话同步执行；为 FALSE 时以任务的方式异步执行。默认为 TRUE。

5. STOP_JOB 过程

停止正在运行的作业。当作业的 AUTO_DROP 配置为 TRUE，停止作业时会检查作业是否需要删除，满足删除条件时将作业删除。

语法如下:

```
PROCEDURE STOP_JOB (
    JOB_NAME            IN      VARCHAR,
    FORCE               IN      BOOLEAN     DEFAULT FALSE,
    COMMIT_SEMANTICS    IN      VARCHAR     DEFAULT 'STOP_ON_FIRST_ERROR'
```

);

参数详解

- JOB_NAME

  待停止的作业名。

- FORCE

  FALSE 表示中断作业，DM 会用中断会话的方式停止作业，VM 里面会定期检测会话是否中断；

  TRUE 表示立即停止作业，目前不支持这种方式。

  如果为 NULL，则取值为 FALSE。

- COMMIT_SEMANTICS

  仅语法支持。

31.1.4 全局调度属性

全局调度属性是作用于整个 SCHEDULER 中所有作业上。

表 33.1 全局属性列表

属性名称	取值	备注
Default_timezone	Start_date 未指定时，用于重复作业和窗口应用日历语法格式串时获取时区信息	不支持，仅作兼容使用
Event_expiry_time	以秒为单位，作业从调度事件队列中产生的过期事件；若为 NULL，则过期时间为 24 小时。	不支持，仅作兼容使用
Log_history	作业日志和窗口日志的保存天数，默认 30 天。	不支持，仅作兼容使用

Max_job_slave_processes	特殊系统配置和加载的从进程的最大个数。默认是 NULL，有效值范围为 1-999.	不支持，仅作兼容使用
Email_server	调度用于发送作业状态事件邮件通知使用的 SMTP 服务器的地址。如果为 NULL，则不能发送。	支持
Email_port	调度用于发送作业状态事件邮件通知使用的 SMTP 服务器的端口号，默认 25。	扩展支持，DM 特有的选项
Email_password	邮件发送连接 SMTP 服务器的密码。	扩展支持，加密保存。
Email_sender	发送作业状态邮件通知的默认邮件地址，默认为 NULL	支持
Email_mimetype	邮件的 MIMETYPE，设置邮件使用的内容和编码方式，默认 TEXT/PLAIN; CHARSET=US-ASCII; 中文可调整为 TEXT/PLAIN; CHARSET=GB18030	扩展支持，DM 特有的选项
Current_open_window	当前打开窗口的名字	ORACLE 特有属性, DM 不支持，也不兼容，仅做介绍
File_watcher_count	当前数据库中文件监视器的数量	ORACLE 特有属性, DM 不支持，也不兼容，仅做介绍
Last_observed_event	指调度器最近观察到的事件	ORACLE 特有属性, DM 不支持，也不兼容，

		仅做介绍

1. SET_SCHEDULER_ATTRIBUTE 过程

设置全局调度的一个属性。设置属性值会立即有效，但结果不一定立即出现。全局调度属性可以在 DBA_SCHEDULER_GLOBAL_ATTRIBUTE 视图中查询得到。

语法如下:

```
PROCEDURE SET_SCHEDULER_ATTRIBUTE(
    ATTRIBUTE       IN      VARCHAR,
    VALUE           IN      VARCHAR
);
```

参数详解

- ATTRIBUTE

  属性名称，详见上表。

- VALUE

  属性值，详见上表说明，根据实际情况设置。

2. GET_SCHEDULER_ATTRIBUTE 过程

获取全局调度的一个属性。

语法如下:

```
PROCEDURE GET_SCHEDULER_ATTRIBUTE(
    ATTRIBUTE       IN      VARCHAR,
```

```
    VALUE                   OUT         VARCHAR
);
```

参数详解

- ATTRIBUTE

  属性名称，详见上表。

- VALUE

  属性值，详见上表说明，根据实际情况设置。

31.1.5 邮件通知

编辑作业事件的邮件通知。

表 33.2 在邮件中 SUBJECT 和 BODY 参数中可使用的变量

变量名称	释义
%job_owner%	创建的作业的模式
%job_name%	-
%job_subname%	用于基于事件的作业，设置了 parallel_instances 属性或者用于 CHAIN 步骤（STEPS）
%event_type%	见表 33.3 调度抛出的事件类型
%event_timestamp%	事件时间戳
%log_id%	指向视图*_SCHEULER_JOB_LOG 和*_SCHEDULER_JOB_RUN_DETAILS 中的 LOG_ID 列。
%error_code%	错误码
%error_message%	错误信息

%run_count%	运行次数
%failure_count%	失败次数
%retry_count%	重试次数

表 33.3 调度抛出的事件类型

事件类型	释义
Job_all_events	不是一个事件，是一个常量，用于提供一种简单的方法激活所有事件
Job_broken	作业必须已经 disabled 并且调整到 BROKEN 状态，因为执行失败次数超过了作业属性 max_failures 设置值
Job_chain_stalled	运行 CHAIN 的作业进入 CHAIN_STALLED 状态。A running chain becomes stalled if there are no steps running or scheduled to run and the chain evaluation_interval is set to NULL. No progress will be made in the chain unless there is manual intervention.
Job_completed	作业成功完成，达到 max_runs 或者 end_date
Job_disabled	调度本身调整作业 disabled，或者调用 SET_ATTRIBUTE
Job_failed	作业失败，可以是抛出错误，也可以是异常中断
Job_over_max_dur	作业执行超过了 max_run_duration 属性设置的最大运行周期。(注意：没必要使用作业 raise_events 属性来激活这个事件；总是激活状态)
Job_run_completed	作业运行或者失败、或者成功或者停止
Job_sch_lim_reached	达到作业调用 limit。作业未开始，因为启动作业延迟超过了作业属性 schedule_limit 定义值
Job_started	作业启动了
Job_stopped	调用 STOP_JOB 停止了作业

Job_succeeded	作业成功完成

1. ADD_JOB_EMAIL_NOTIFICATION 过程

为作业添加一个邮件通知。无论哪个指定的作业状态事件触发，邮件会发送到指定链表中的接收端。

语法如下:

```
PROCEDURE ADD_JOB_EMAIL_NOTIFICATION(
    JOB_NAME      IN      VARCHAR,
    RECIPIENTS    IN      VARCHAR,
    SENDER        IN      VARCHAR DEFAULT NULL,
    SUBJECT       IN      VARCHAR DEFAULT
DBMS_SCHEDULER.DEFAULT_NOTIFICATION_SUBJECT,
    BODY          IN      VARCHAR DEFAULT
DBMS_SCHEDULER.DEFAULT_NOTIFICATION_BODY,
    EVENTS        IN      VARCHAR DEFAULT
'JOB_FAILED,JOB_BROKEN,JOB_SCH_LIM_REACHED,JOB_CHAIN_STALED,JOB_OVER_MAX_DUR',
    FILTER_CONDITION  IN      VARCHAR DEFAULT NULL
);
```

参数详解

- JOB_NAME

添加邮件通知的目标作业名称，不能为 NULL。

- Recipients

逗号隔开的接收端的 EMAIL 地址链表。列出的所有邮件通知都会发送到所有的接收端地址。不能为 NULL。

- Sender

  发送端的 EMAIL 地址。若为 NULL 或者忽略，则使用全局调度属性中的 email_sender。

- Subject

  邮件主题。目前仅支持默认主题：Dameng Scheduler Job Notification - %job_owner%.%job_name% %event_type%。其中，%中间代表是要插入的变量值。

- Body

  邮件内容。目前仅支持默认邮件内容格式：

  'Job: %job_owner%.%job_name%

  Event: %event_type%

  Date: %event_timestamp%

  Log id: %log_id%

  Run count: %run_count%

  Failure count: %failure_count%

  Retry count: %retry_count%

  Error code: %error_code%

  Error message:%error_message%'

  其中，%中间代表是要插入的变量值。

- Events

  以逗号隔开的发送邮件通知的作业状态事件。不能为 NULL。列表中任何一个事件触发都会向接收端发送通知。如果忽略，则以下默认事件发送通知：

JOB_FAILED、JOB_BROKEN、JOB_SCH_LIM_REACHED、JOB_CHAIN_STALLED、JOB_OVER_MAX_DUR。

- Filter_condition

用于过程要发送邮件通知的事件。如果为 NULL，所有指定事件发生时机都会产生一个邮件通知并发送。Filter_condition 必须是一个布尔类型 SQL WHERE 表达式，引用:event 绑定变量。绑定变量自动绑定到代表抛出事件的对象上（实现中并未支持，仅作兼容处理）。

例如，过滤条件为：

:event.error_code=600 或者 :error.error_code=700

2. REMOVE_JOB_EMAIL_NOTIFICATION 过程

移除作业的邮件通知。可以移除所有的邮件通知，或者仅移除指定接收者或指定事件的邮件通知。执行作业删除时，会将作业上所有的邮件通知移除。

语法如下：

```
PROCEDURE REMOVE_JOB_EMAIL_NOTIFICATION(
    JOB_NAMEIN              VARCHAR,
    RECIPIENTS      IN      VARCHAR     DEFAULT NULL,
    EVENTS          IN      VARCHAR     DEFAULT NULL
);
```

参数详解

- JOB_NAME

移除邮件通知的目标作业。不能为 NULL。

- recipients

逗号隔开的要移除满足配置指定接收者的邮件通知。若为 NULL，则移除邮件通知时，忽略接口端的过滤。

- Events

逗号隔开的要移除满足配置直指定事件的邮件通知。若为 NULL，则移除邮件通知时，忽略指定事件的过滤。

31.1.6 其他

1. GENERATE_JOB_NAME 函数

返回一个唯一的作业名称。名称格式为{前缀}N，其中 N 是从一个序列获取的数字。若未指定前缀，则默认产生名字为 JOB$_1、JOB$_2、JOB$_3 等。若设置前缀为'DM'，则生成 JOB 名称为 DM1、DM2、DM3 等。

语法如下:

```
FUNCTION DBMS_SCHEDULER.GENERATE_JOB_NAME(
    PREFIX          IN          VARCHAR       DEFAULT 'JOB$_'
)RETURN VARCHAR;
```

参数详解

- PREFIX

生成作业名的前缀

2. EVALUATE_CALENDAR_STRING 过程

计算日历字符串。

语法如下:

```
PROCEDURE DBMS_SCHEDULER.EVALUATE_CALENDAR_STRING (
    CALENDAR_STRING          IN      VARCHAR,
    START_DATE               IN      TIMESTAMP WITH TIME ZONE,
    RETURN_DATE_AFTER        IN      TIMESTAMP WITH TIME ZONE,
    NEXT_RUN_DATE            OUT     TIMESTAMP WITH TIME ZONE
);
```

参数详解

- CALENDAR_STRING

  待计算的日历格式字符串。必须是按照日历语法。

- START_DATE

  日历字符串有效的起始日期和时间。若指定为 NULL，则起始时间为当前时间。

- RETURN_DATE_AFTER

  使用 START_DATE 和 CALENDAR_STRING，调度就有足够的信息来决定所有有效执行日期。设置这个参数，调度知道匹配哪个可能的时间退出。若未设置，则系统直接使用系统时间戳。

- NEXT_RUN_DATE

  匹配 CALENDAR_STRING 和 START_DATE，并在 RETURN_DATE_AFTER 参数值之后发生的第一个时间戳。

3. SET_ATTRIBUTE 过程

设置对象的一个属性，目前仅支持 JOB 对象的属性设置。

语法如下:

```
PROCEDURE SET_ATTRIBUTE(
    NAME            IN      VARCHAR,
    ATTRIBUTE       IN      VARCHAR,
    VALUE           IN      {BOOLEAN |
                             VARCHAR |
                             TIMESTAMP |
                             TIMESTAMP WITH TIME ZONE |
                             INTERVAL DAY TO SECOND}
);
```

参数详解

- NAME

设置的对象名称。目前仅支持对 JOB 对象属性设置。

- ATTRIBUTE

属性名称。

➢ JOB 对象支持的属性有以下属性:

➢ auto_drop: 作业是否自动删除

如果为 TRUE，作业自动删除。默认为 TRUE。自动删除的条件如下:

1) 作业的结束日期已经过期

2) 作业不是重复执行作业，并且只执行一次

- program_name：作业执行的程序名称，若 JOB 对象设置了 JOB_TYPE、JOB_ACTION、NUMBER_OF_ARGUMENTS，则不允许再设置 PROGRAM_NAME 属性。
- schedule_name：作业所使用的调度名称，若 JOB 对象设置了 START_DATE、END_DATE、REPEAT_INTERVL 属性，则不允许再设置 SCHEDULE_NAME 属性。
- repeat_interval：作业执行的重复/间隔规则，支持日历语法，详见日历语法章节，同时也支持 PL/SQL 表达式。若 JOB 对象设置了 SCHEDULE_NAME 属性，则不允许设置该属性。
- end_date：作业截止日期。若 JOB 对象设置了 SCHEDULE_NAME 属性，则不允许设置该属性。
- start_date：作业起始日期。若 JOB 对象设置了 SCHEDULE_NAME 属性，则不允许设置该属性。


说明：**Repeat_interval、end_date、start_date 属性设置的前提是 JOB 对象未设置 SCHEDULE_NAME 属性。若事先设置了 SCHEDULE_NAME 属性，但后来这个 SCHEDULE 对象被删除了，那么 JOB 对象对应的 SCHEDULE_NAME 属性会被置空，用户可以通过组合设置 repeat_interal、end_date、start_date 属性或者重新设置 SCHEDULE_NAME 属性，完成调度相关内容的补充，确保 JOB 对象可以正常执行调度。**

- max_run_duration：作业允许运行的最大时间长度。参数数据类型为 INTERVAL DAY TO SECOND，必须为有效的间隔类型串。
- comments：作业的相关评论或注释。
- number_of_arguments：作业的参数个数，等同 create_job 中的

number_of_arguments 参数。若 JOB 对象设置了 PROGRAM_NAME 属性，则不允许设置该属性。

- job_action：作业的动作内容，等同 create_job 中的 job_action 参数。若 JOB 对象设置了 PROGRAM_NAME 属性，则不允许设置该属性。
- job_type：作业的动作类型，等同 create_job 中 job_type 参数。若 JOB 对象设置 PROGRAM_NAME 属性，则不允许设置该属性。


说明：**Number_of_arguments、job_action、job_type 属性设置的前提是 JOB 对象未设置 PROGRAM_NAME 属性。若事先设置了 PROGRAM_NAME 属性，但后来这个 PROGRAM 对象被删除了，那么 JOB 对象对应的 PROGRAM_NAME 属性会被置空，用户可以通过组合设置 number_of_arguments、job_action、job_type 属性或者重新设置 PROGRAM_NAME 属性，完成程序相关内容的补充，确保 JOB 对象可以正常执行工作。**

- 以下属性仅作兼容，未实现功能：

restartable

allow_runs_in_restricted_mode

follow_default_timezone

instance_stickiness

parallel_instances

stop_on_windows_close

instance_id

max_failures

max_runs

job_priority

logging_level

credential_name

database_role

destination

destination_name

event_spec

job_class

job_weight

raise_events

schedule_limit

- VALUE

  设置的属性值。

4. PURGE_LOG 过程

   清理日志。

   语法如下:

```
PROCEDURE PURGE_LOG (
```

```
    LOG_HISTORY         IN      INT         DEFAULT 0,
    WHICH_LOG           IN      VARCHAR     DEFAULT 'JOB_AND_WINDOW_LOG',
    JOB_NAME            IN      VARCHAR     DEFAULT NULL
)
```

参数详解

- LOG_HISTORY

  历史记录保留天数。取值范围 0-999，如果为 0，则清空历史记录。

- WHICH_LOG

  指定清理的日志类型，不支持，仅作兼容参数用。注：DBMS_SCHEDULER 日志只支持 JOB_LOG 类型日志，WHICH_LOG 指定与否，PURGE_LOG 均删除作业日志。

- JOB_NAME

  指定清理日志历史记录的作业名称。

5. DISABLE 过程

   使指定的对象失效。目前仅支持 PROGRAM 和 JOB。

   语法如下:

```
PROCEDURE DISABLE (
    NAME                    IN      VARCHAR,
    FORCE                   IN      BOOLEAN     DEFAULT FALSE,
    COMMIT_SEMANTICS        IN      VARCHAR     DEFAULT 'STOP_ON_FIRST_ERROR'
)
```

参数详解

- NAME

  DISABLE 对象名称。目前仅支持 JOB 和 PROGRAM

- FORCE

  是否强制 DISABLE 对象。参数不支持，仅作兼容用，目前处理都为强制 DISABLE

- COMMIT_SEMANTICS

  不支持，仅作兼容用。

6. ENABLE 过程

   使指定的对象生效。目前仅支持 PROGRAM 和 JOB。

   语法如下:

```
PROCEDURE ENABLE(
    NAME                IN      VARCHAR,
    COMMIT_SEMANTICS    IN      VARCHAR         DEFAULT 'STOP_ON_FIRST_ERROR'
)
```

   参数详解

- NAME

  ENABLE 对象名称。目前仅支持 JOB 和 PROGRAM。

  作业 ENABLE 时，如果作业所使用的 PROGRAM 无效，作业创建失败。

- COMMIT_SEMANTICS

不支持，仅作兼容用。

31.2 创建、删除、重建语句

创建、删除或者重建 DBMS_SCHEDULER 系统包。

语法如下:

```
void
SP_INIT_DBMS_SCHEDULER_SYS (
    CREATE_FLAG      INT
)
```

参数详解

- CREATE_FLAG

取值为 0、1、2。0 表示删除 DBMS_SCHEDULER 包；1 表示创建 DBMS_SCHEDULER 包；2 表示重建 DBMS_SCHEDULER 包中的相关方法，一般在版本升级后，包中方法发生了改变的情况下使用。

返回值

无

举例说明

创建 DBMS_SCHEDULER 系统包。

```
SP_INIT_DBMS_SCHEDULER_SYS (1);
```

31.3 启用状态检测语句

系统的 DBMS_SCHEDULER 系统包启用状态检测。

语法如下:

```
INTEGER

SF_CHECK_DBMS_SCHEDULER_SYS();
```

返回值

0：未启用；1：已启用

举例说明

获得 DBMS_SCHEDULER 系统包的启用状态。

```
select SF_CHECK_DBMS_SCHEDULER_SYS();
```

31.4 相关视图

MPP 环境下，作业只在节点号最小的节点上执行。所有视图，都只显示本站点的相关信息。

1. DBA_SCHEDULER_PROGRAMS

记录 PROGRAM 对象信息。

序号	列	说明
1	OWNER	PROGRAM 所有者
2	PROGRAM_NAME	PROGRAM 名称
3	PROGRAM_TYPE	PROGRAM 类型，仅支持 PLSQL_BLOCK 和 STORED_PROCEDURE 类型
4	PROGRAM_ACTION	PROGRAM 执行的动作
5	NUMBER_OF_ARGUMENTS	参数个数
6	ENABLED	是否启用
7	DETACHED	不支持，仅作兼容用
8	SCHEDULE_LIMIT	不支持，仅作兼容用

9	PRIORITY	不支持，仅作兼容用
10	WEIGHT	不支持，仅作兼容用
11	MAX_RUNS	不支持，仅作兼容用
12	MAX_FAILURES	不支持，仅作兼容用
13	MAX_RUN_DURATION	不支持，仅作兼容用
14	NLS_ENV	不支持，仅作兼容用
15	COMMENTS	PROGRAM 评论或注释

2. DBA_SCHEDULER_PROGRAM_ARGS

记录 PROGRAM 对象的参数定义信息。

序号	列	说明
1	OWNER	PROGRAM 所有者
2	PROGRAM_NAME	PROGRAM 名称
3	ARGUMENT_NAME	参数名称
4	ARGUMENT_POSITION	当前参数在参数列表中所在位置
5	ARGUMENT_TYPE	参数的数据类型
6	METADATA_ATTRIBUTE	不支持，仅作兼容用
7	DEFAULT_VALUE	参数默认值，字符串格式
8	DEFAULT_ANYDATA_VALUE	不支持，仅作兼容用
9	OUT_ARGUMENT	是否为输出参数

3. DBA_SCHEDULER_SCHEDULES

记录 SCHEDULE 对象信息。

序号	列	说明
1	OWNER	调度的所有者
2	SCHEDULE_NAME	调度名称
3	SCHEDULE_TYPE	调度类型，取值 ONCE,CALENDAR
4	START_DATE	起始时间
5	REPEAT_INTERVAL	调度的重复/间隔规则
6	EVENT_QUEUE_OWNER	不支持，仅作兼容用
7	EVENT_QUEUE_NAME	不支持，仅作兼容用
8	EVENT_QUEUE_AGENT	不支持，仅作兼容用
9	EVENT_CONDITION	不支持，仅作兼容用
10	FILE_WATCHER_OWNER	不支持，仅作兼容用
11	FILE_WATCHER_NAME	不支持，仅作兼容用
12	END_DATE	结束时间
13	COMMENTS	调度的评论或注释

4. DBA_SCHEDULER_JOBS

记录 JOB 对象信息。

序号	列	说明
1	OWNER	作业的所有者
2	JOB_NAME	作业名称
3	JOB_SUBNAME	作业子名称，不支持，仅作兼容用
4	JOB_STYLE	作业风格，仅支持 REGULAR
5	JOB_CREATOR	作业原始的创建者

6	CLIENT_ID	不支持，仅作兼容用
7	GLOBAL_UID	不支持，仅作兼容用
8	PROGRAM_OWNER	和作业关联的 PROGRAM 的所有者
9	PROGRAM_NAME	和作业关联的 PROGRAM 的名称
10	JOB_TYPE	作业类型，取值 PLSQL_BLOCK， STORED_PROCEDURE， 不支持 EXECUTABLE 和 CHAIN
11	JOB_ACTION	作业执行的动作
12	NUMBER_OF_ARGUMENTS	作业参数个数
13	SCHEDULE_OWNER	调度的所有者
14	SCHEDULE_NAME	和作业相关联的调度名称
15	SCHEDULE_TYPE	调度的类型
16	START_DATE	作业起始时间
17	REPEAT_INTERVAL	作业的重复/间隔规则
18	EVENT_QUEUE_OWNER	不支持，仅作兼容用
19	EVENT_QUEUE_NAME	不支持，仅作兼容用
20	EVENT_QUEUE_AGENT	不支持，仅作兼容用
21	EVENT_CONDITION	不支持，仅作兼容用
22	EVENT_RULE	不支持，仅作兼容用
23	FILE_WATCHER_OWNER	不支持，仅作兼容用
24	FILE_WATCHER_NAME	不支持，仅作兼容用
25	END_DATE	作业对束时间
26	JOB_CLASS	不支持，仅作兼容用

27	ENABLED	作业是否启用
28	AUTO_DROP	作业结束时是否自动删除
29	RESTARTABLE	作业执行失败是否重新启动。不支持，仅作兼容用
30	STATE	当前作业的状态
31	JOB_PRIORITY	不支持，仅作兼容用
32	RUN_COUNT	不支持，仅作兼容用
33	MAX_RUNS	不支持，仅作兼容用
34	FAILURE_COUNT	不支持，仅作兼容用
35	MAX_FAILURES	不支持，仅作兼容用
36	RETRY_COUNT	不支持，仅作兼容用
37	LAST_START_DATE	不支持，仅作兼容用
38	LAST_RUN_DURATION	不支持，仅作兼容用
39	NEXT_RUN_DATE	作业下一次执行时间
40	SCHEDULE_LIMIT	不支持，仅作兼容用
41	MAX_RUN_DURATION	作业执行的最大持续时间
42	LOGGING_LEVEL	不支持，仅作兼容用
43	STOP_ON_WINDOW_CLOSE	不支持，仅作兼容用
44	INSTANCE_STICKINESS	不支持，仅作兼容用
45	RAISE_EVENTS	不支持，仅作兼容用
46	SYSTEM	不支持，仅作兼容用
47	JOB_WEIGHT	不支持，仅作兼容用
48	NLS_ENV	不支持，仅作兼容用
49	SOURCE	不支持，仅作兼容用

50	NUMBER_OF_DESTINATIONS	不支持，仅作兼容用
51	DESTINATION_OWNER	不支持，仅作兼容用
52	DESTINATION	不支持，仅作兼容用
53	CREDENTIAL_OWNER	不支持，仅作兼容用
54	CREDENTIAL_NAME	不支持，仅作兼容用
55	INSTANCE_ID	不支持，仅作兼容用
56	DEFERRED_DROP	不支持，仅作兼容用
57	ALLOW_RUNS_IN_RESTRICTED_MODE	不支持，仅作兼容用
58	COMMENTS	作业的评论或注释
59	FLAGS	内部使用字段

5. DBA_SCHEDULER_JOB_ARGS

记录 JOB 对象的参数值信息。

序号	列	说明
1	OWNER	参数所属作业的所有者
2	JOB_NAME	参数所属作业名称
3	ARGUMENT_NAME	参数名称
4	ARGUMENT_POSITION	当前参数在参数列表中所在位置
5	ARGUMENT_TYPE	参数的数据类型
6	VALUE	参数值，字符串格式
7	ANYDATA_VALUE	其他格式的参数值
8	OUT_ARGUMENT	是否为输出参数

6. DBA_SCHEDULER_JOB_RUN_DETAILS

记录 JOB 运行过程中的详细信息。

序号	列	说明
1	LOG_ID	日志唯一 ID
2	LOG_DATE	作业执行结束时间
3	OWNER	作业的所有者
4	JOB_NAME	作业名称
5	JOB_SUBNAME	作业子名称，不支持，仅作兼容用
6	STATUS	作业运行状态
7	ERROR#	作业出错时的错误码
8	REQ_START_DATE	作业请求运行时间，不支持，仅作兼容用
9	ACTUAL_START_DATE	作业实际执行起始时间
10	RUN_DURATION	作业执行时长
11	INSTANCE_ID	不支持，仅作兼容用
12	SESSION_ID	作业执行所在会话 ID
13	SLAVE_PID	不支持，仅作兼容用
14	CPU_USED	不支持，仅作兼容用
15	CREDENTIAL_OWNER	不支持，仅作兼容用
16	CREDENTIAL_NAME	不支持，仅作兼容用
17	DESTINATION_OWNER	不支持，仅作兼容用
18	DESTINATION	不支持，仅作兼容用
19	ADDITIONAL_INFO	作业运行过程中相关信息

7. DBA_SCHEDULER_JOB_LOG

记录作业日志信息。

序号	列	说明
1	LOG_ID	日志唯一 ID
2	LOG_DATE	日志记录的日期
3	OWNER	作业的所有者
4	JOB_NAME	作业名称
5	JOB_SUBNAME	作业子名称，不支持，仅作兼容用
6	JOB_CLASS	不支持，仅作兼容用
7	OPERATION	不支持，仅作兼容用
8	STATUS	作业运行状态
9	USER_NAME	用户名
10	CLIENT_ID	不支持，仅作兼容用
11	GLOBAL_UID	不支持，仅作兼容用
12	CREDENTIAL_OWNER	不支持，仅作兼容用
13	CREDENTIAL_NAME	不支持，仅作兼容用
14	DESTINATION_OWNER	不支持，仅作兼容用
14	DESTINATION	不支持，仅作兼容用
16	ADDITIONAL_INFO	作业运行过程中相关信息

8. DBA_SCHEDULER_GLOBAL_ATTRIBUTE

记录 SCHEDULER 包的全局属性。

序号	列	说明
1	ATTRIBUTE_NAME	属性名称, 不包括 EMAIL_PASSWORD

2	VALUE	属性值

9. DBA_SCHEDULER_RUNNING_JOBS

描述数据库中由 DBMS_SCHEDULER 包创建的且正在执行的作业。

序号	列	说明
1	OWNER	作业所有者
2	JOB_NAME	作业名称
3	JOB_SUBNAME	作业子名称，不支持，仅作兼容用
4	JOB_STYLE	作业风格，仅支持 REGULAR
5	DETACHED	不支持，仅作兼容用
6	SESSION_ID	作业执行的会话 ID
7	SLAVE_PROCESS_ID	不支持，仅作兼容用
8	SLAVE_OS_PROCESS_ID	不支持，仅作兼容用
9	RUNNING_INSTANCE	执行作业的实例号
10	RESOURCE_CONSUMER_GROUP	不支持，仅作兼容用
11	ELAPSED_TIME	作业已经执行的时长
12	CPU_USED	不支持，仅作兼容用
13	DESTINATION_OWNER	不支持，仅作兼容用
14	DESTINATION	不支持，仅作兼容用
14	CREDENTIAL_OWNER	不支持，仅作兼容用
16	CREDENTIAL_NAME	不支持，仅作兼容用
17	LOG_ID	不支持，仅作兼容用

10. USER_SCHEDULER_RUNNING_JOBS

描述当前用户拥有的由 DBMS_SCHEDULER 包创建的且正在执行的作业，结构与 DBA_SCHEULER_RUNNING_JOBS 相同。

31.5 日历语法

31.5.1 日历语法格式

DM 支持的日历语法格式：

```
repeat_interval: :  =
<frequency 子句>[;<interval 子句>] [;<bymonth 子句>] [;<byweekno 子句>][;<byyearday 子句>] [;<bymonthday 子句>][;<byday 子句>] [;<byhour 子句>] [;<byminute 子句>][;<bysecond 子句>]

<frequency 子句>::= FREQ = <predefined_frequency>

<predefined_frequency>:: = YEARLY | MONTHLY | WEEKLY | DAILY |  HOURLY | MINUTELY | SECONDLY
<interval 子句> ::= INTERVAL = <intervalnum>
<intervalnum> ::= <取值 1 到 99>
<bymonth 子句> ::= BYMONTH = <monthlist>
<monthlist> ::= <month>{,<month>}
<month> ::= <numeric_month> | <char_month>
<numeric_month> ::= 1 | 2 | 3 ...  |12
<char_month> ::= JAN | FEB | MAR | APR | MAY | JUN | JUL | AUG | SEP | OCT | NOV | DEC
```

<byweekno 子句>::= BYWEEKNO = <weeknumber_list>

<weeknumber_list> ::= <weeknumber>{,<weeknumber>}

<weeknumber> ::= [<minus>] <weekno>

<weekno> ::= <取值 1 到 53>

<byyearday 子句> ::= BYYEARDAY = <yearday_list>

<yearday_list> ::= <yearday>{, <yearday>}

<yearday> ::= [<minus>] <yeardaynum>

<yeardaynum> ::= <取值 1 到 366>

<bymonthday 子句> ::= BYMONTHDAY = <monthday_list>

<monthday_list> ::= <monthday>{, <monthday>}

<monthday> ::= [<minus>] <monthdaynum>

<monthdaynum> ::= <取值 1 到 31>

<byday 子句> ::= BYDAY = <byday_list>

<byday_list> ::= <byday>{,<byday>}

<byday> ::= [<weekdaynum>] <day>

<weekdaynum> ::= [<minus>] <daynum>

<daynum> ::= <取值 1 到 53> /* 当 frequency 为 yearly */

<daynum> ::= <取值 1 到 5> /* 当 frequency 为 monthly */

<day> ::= MON | TUE | WED | THU | FRI | SAT | SUN

<byhour 子句> ::= BYHOUR = <hour_list>

```
<hour_list> ::= <hour>{, <hour>}

<hour> ::= <取值 0 到 23>



<byminute 子句> ::= BYMINUTE = <minute_list>

<minute_list> ::= <minute>{, <minute>}

<minute> := <取值 0 到 59>



<bysecond 子句> ::= BYSECOND = <second_list>

<second_list> ::= <second>{, <second>}

<second> ::= <取值 0 到 59>



<minus> ::= -
```

参数详解

- REPEAT_INTERVAL 是整个重复规则的顶层结构，可包含一个或者多个子句，必须要指定 FREQ=频率或 INTERVAL 中的一项作为重复的基础,其余子句用于进一步约束或者筛选执行时间。例如：FREQ=MONTHLY;BYDAY=MON;BYHOUR=9 表示每月的每周一上午 9 点执行一次。
- <FREQUENCY 子句>指定事件的基准重复频率。例如:

  1） FREQ=YEARLY 表示以年为单位重复（每年）;

  2） FREQ=MONTHLY 表示以月为单位重复（每月）;

  3） FREQ=WEEKLY 表示以周为单位重复（每周）;

  4） FREQ=DAILY 表示以天为单位重复（每天）;

  5） FREQ = HOURLY / MINUTELY / SECONDLY:分别以小时/分钟/秒为单位重复，粒度更细。

- <INTERVAL 子句>指定频率的间隔倍数。例如:

  1） FREQ=DAILY;INTERVAL=2 表示每 2 天执行 1 次;

  2） FREQ=MONTHLY;INTERVAL=3 表示每 3 个月执行 1 次。

- <BYMONTH 子句>按“月份”进行筛选执行时间，支持数字（1-12）或英文缩写（JAN-DEC），常用来指定年内哪些月份执行。例如:

  1） FREQ=YEARLY;BYMONTH=3 表示每年 3 月执行;

  2） FREQ=YEARLY;BYMONTH=MAR,MAY,DEC 表示每年在 3 月、5 月、12 月执行。

- <BYWEEKNO 子句>按“年内第几周”进行筛选执行时间，允许正负数，负数表示从年尾向前计数。只有在 FREQ=YEARLY 时有效。例如:

  1） FREQ=YEARLY;BYWEEKNO=1 表示每年第一周执行;

  2） FREQ=YEARLY;BYWEEKNO=-1 表示每年最后一周执行。

- <BYYEARDAY 子句>按“年内第几天”进行筛选执行时间，允许正负数，负数表示从年尾向前计数，考虑闰年。例如:

  1） FREQ=YEARLY;BYYEARDAY=1 表示每年的 1 月 1 日;

  2） FREQ=YEARLY;BYYEARDAY=-1 表示每年的 12 月 31 日;

  3） FREQ=YEARLY;BYYEARDAY=365 表示非闰年的 12 月 31 日;

  4） FREQ=YEARLY;BYYEARDAY=366 表示闰年的 12 月 31 日，在闰年 366 才有意义。

- <BYMONTHDAY 子句>按“每月的第几天”进行筛选执行时间，允许正负数，负数表示从月末向前计数。例如:

  1） FREQ=MONTHLY;BYMONTHDAY=1,15 表示每月 1 日和 15 日;

  2） FREQ=MONTHLY;BYMONTHDAY=-1 表示每月最后一天，对不同月份最后一天自动适应。

- <BYDAY 子句>按“星期几”进行筛选执行时间，若带前缀数字 WEEKDAYNUM，表示

当前频率范围内第 WEEKDAYNUM 周的星期几；若不带前缀数字 WEEKDAYNUM，表示当前频率范围内所有的星期几。当 FREQ=YEARLY 时，<DAYNUM>的取值范围为 1~53；当 FREQ=MONTHLY 时，<DAYNUM>的取值范围为 1~5。例如：

1） FREQ=MONTHLY;BYDAY=MON 表示每月所有的星期一；

2） FREQ=YEARLY;BYDAY=2MON 表示每年第二周的星期一；

3） FREQ=MONTHLY;BYDAY=-1FRI 表示每月最后一个星期五；

4） FREQ=WEEKLY;BYDAY=MON,WED,FRI 表示每周的星期一、三、五。

- <BYHOUR 子句>按“小时”进行筛选执行时间，为 24 小时制，取值范围为 0~23。例如：

  1） FREQ=DAILY;BYHOUR=9 表示每天 09:00:00 执行一次；

  2） FREQ=DAILY;BYHOUR=9,17 表示每天 09:00:00 和 17:00:00 各执行一次。

- <BYMINUTE 子句>按“分钟”进行筛选执行时间。例如：

  FREQ=DAILY;BYHOUR=9;BYMINUTE=0,30 表示每天 09:00:00 和 09:30:00 各执行一次。

- <BYSECOND 子句>按“秒”进行筛选执行时间，粒度最细。在 DBMS_SCHEDULER 包中进行秒级设置需要把 INI 参数 TIMER_TRIG_CHECK_INTERVAL 置为 1。例如：

  FREQ=HOURLY;BYMINUTE=0;BYSECOND=0,30 表示每小时的第 0 分 0 秒和每小时的第 0 分 30 秒各执行一次。

- <MINUS>为负数（-），表示从后向前计数。例如：-1 表示“最后一个”或“倒数第一个”。

31.5.2 版本升级

达梦数据库产品注重向下兼容性，虽然版本不断地更新升级，但在绝大多数情况下，使用高版本的执行码启动低版本的数据库时，会自动执行一系列更新升级动作。对于待升级的

数据库，用户只需要保证它之前是正常退出的即可。

在数据库升级操作前，建议先进行数据备份。

需要注意的是，从 V7.6.1.67 版本和 V8.1.1.107 开始，对日历表达式触发器进行了改进。使得老版本升级后的新版本需要进行额外的升级操作，下面进行详细的说明。

31.5.2.1 日历表达式触发器升级

日历表达式触发器就是使用了“日历语法”的时间触发器。主要应用于 DBMS_SCHEDULER 包中。当使用 PROCEDURE DBMS_SCHEDULER.CREATE_JOB 创建 JOB 时，只要使用了 REPEAT_INTERVAL 参数，按照日历语法格式指定了调度重复间隔，那么系统内部就会自动生成一个日历表达式触发器。

老版本中，触发器日历表达式 encode 到 sysobjects 视图 info6 字段时，指针偏移多移动了 31 字节，decode 时也同样多移动了 31 字节，导致日历表达式涉及秒级间隔触发，部分秒数实际未触发。在 V7.6.1.67 以上的 DM7 版本和 V8.1.1.107 以上的 DM8 版本中，此问题已被修正，均不存在此问题。

31.5.2.1.1 版本说明

DM7 支持从 DM7 老版本升级到 V7.6.1.67 以上版本，DM8 支持从 DM8 老版本升级到 V8.1.1.107 以上版本。但是，从老版本升级的新版本，需要手动重新创建日历触发器，使 sysobjects 视图 info6 字段重新填写，来修正上述问题。

非日历表达式触发器无此问题，无需修正。

31.5.2.1.2 升级步骤

待升级的日志表达式触发器分为两种情况：一种是 DBMS_SCHEDULER 包相关，另一种是非 DBMS_SCHEDULER 包相关。下面详细介绍如下：

1. DBMS_SCHEDULER 包相关

由 DBMS_SCHEDULER 包创建的作业，内部会生成日历表达式触发器，对于仍然有效的

日历表达式触发器，只需要重新 enable 该作业，系统内部会重建该触发器。步骤如下:

1） 查询系统视图，获取预期仍然有效的作业名称。

```
SELECT JOB_NAME,ENABLED FROM DBA_SCHEDULER_JOBS WHERE ENABLED='TRUE';

//查询结果如下:

行号          JOB_NAME     ENABLED

---------- -------- -------

1                  TEST_JOB      TRUE
```

2） 重新生成作业的触发器。

```
begin

        dbms_scheduler.enable(name=>'TEST_JOB');

end;

/
```

3） 至此，升级完成。

2. 非 DBMS_SCHEDULER 包相关

其它非 DBMS_SCHEDULER 包相关的日历表达式触发器，可以通过查询系统表或 MANAGER 工具获取触发器 DDL 创建脚本，然后重新执行一遍 DDL 脚本即可，以此来完成升级工作。

31.6 举例说明

使用包内的过程和函数之前，如果还未创建过系统包，请先调用系统过程创建系统包。

```
SP_INIT_DBMS_SCHEDULER_SYS(1);
```

准备工作:

```
drop table test cascade;

create table test(c1 varchar, c2 varchar);

create or replace procedure proc1(c1 varchar, c2 varchar)

as

begin

insert into test values(c1, c2);

commit;

end;

/
```

创建 PROGRAM：

```
begin

dbms_scheduler.create_program(

program_name=>'PROG1',

program_type=>'STORED_PROCEDURE',

program_action=>'PROC1',

number_of_arguments=>2);

end;

/
```

定义 program 参数：

```
begin

dbms_scheduler.define_program_argument(

program_name=>'PROG1',

argument_position=>1,

argument_name=>'C1',
```

```
argument_type=>'VARCHAR'
);
end;
/


begin
dbms_scheduler.define_program_argument(
program_name=>'PROG1',
argument_position=>2,
argument_name=>'C2',
argument_type=>'VARCHAR'
);
end;
/
```

设置 PROGRAM 为 ENABLE 状态:

```
begin
dbms_scheduler.enable('PROG1');
end;
/
```

创建作业:

```
begin
dbms_scheduler.create_job(
job_name=>'test_job',
program_name=>'PROG1',
```

```
start_date=>'2024-09-01 8:55:00',
repeat_interval=>'FREQ=MINUTELY;INTERVAL=1',
end_date=>'2030-09-01 9:15:00',
enabled=>FALSE,
auto_drop=>FALSE,
comments=>'test_job',
job_style=>'REGULAR'
);
end;
/
```

设置作业参数值：

```
begin
dbms_scheduler.set_job_argument_value(
job_name=>'test_job',
argument_position=>1,
argument_value=>'abc'
);
end;
/
begin
dbms_scheduler.set_job_argument_value(
job_name=>'test_job',
argument_name=>'C2',
argument_value=>'abc'
```

```
);
end;
/
```

配置全局调度属性:

```
begin
DBMS_SCHEDULER.SET_SCHEDULER_ATTRIBUTE(
    'email_server',
    '192.168.0.212');
end;
/


begin
DBMS_SCHEDULER.SET_SCHEDULER_ATTRIBUTE(
    'email_sender',
    'shenning@dameng.shanghai');
end;
/


begin
DBMS_SCHEDULER.SET_SCHEDULER_ATTRIBUTE(
    'email_port',
    '25');
end;
/
```

```
begin
DBMS_SCHEDULER.SET_SCHEDULER_ATTRIBUTE(
        'email_password',
        '888888');
end;
/

begin
DBMS_SCHEDULER.SET_SCHEDULER_ATTRIBUTE(
        'email_mimetype',
        'TEXT/PLAIN; CHARSET=GB18030');
end;
/
```

添加作业邮件通知：

```
BEGIN
DBMS_SCHEDULER.ADD_JOB_EMAIL_NOTIFICATION(
        Job_name => 'test_job',
        Recipients => 'shenning@dameng.shanghai',
      Sender     => 'shenning@dameng.shanghai',
events       => 'JOB_ALL_EVENTS');
END;
/
```

```
//设置作业为 ENABLE
begin
dbms_scheduler.enable(name=>'test_job');
end;
/
```

删除作业邮件通知：

```
BEGIN
DBMS_SCHEDULER.REMOVE_JOB_EMAIL_NOTIFICATION(
    Job_name => 'test_job');
END;
/
```

32 DBMS_MVIEW 包

DBMS_MVIEW 包提供了一个可以一次性刷新多个物化视图的方法。

32.1 相关方法

1. REFRESH

刷新物化视图。

语法如下:

```
PROCEDURE REFRESH(
    LST                     IN      VARCHAR,
    METHOD                  IN      VARCHAR             DEFAULT NULL,
    ROLLBACK_SEG            IN      VARCHAR             DEFAULT NULL,
    PUSH_DEFERRED_RPC       IN      BOOL                DEFAULT TRUE,
    REFRESH_AFTER_ERRORS    IN      BOOL                DEFAULT FALSE,
    PURGE_OPTION            IN      BINARY_INTEGER      DEFAULT 1,
    PARALLELISM             IN      BINARY_INTEGER      DEFAULT 0,
    HEAP_SIZE               IN      BINARY_INTEGER      DEFAULT 0,
    ATOMIC_REFRESH          IN      BOOL                DEFAULT TRUE,
    NESTED                  IN      BOOL                DEFAULT FALSE
);
```

参数详解

- LST

指定的要刷新的物化视图名称，多个物化视图则用逗号隔开。

- METHOD

  对应 LST 中的物化视图的刷新方式，'F'表示快速刷新，'C'表示完全刷新，空串表示默认刷新方式。另外，'A'和'C'等价，'P'和空串等价。标记不区分大小写。

- ATOMIC_REFRESH

  指明 LST 中的所有物化视图如果有失败的话，是提交已成功刷新的记录还是全部不提交。TRUE 表示全部不提交，FALSE 表示提交已成功刷新的记录。缺省为 TRUE。

- ROLLBACK_SEG/PUSH_DEFERRED_RPC/REFRESH_FATER_ERRORS/PURGE/OPTION/PARALLELISM/HEAP_SIZE/NESTED 仅语法支持。

32.2 举例说明

使用包内的过程和函数之前，如果还未创建过 DBMS_MVIEW 包，请先调用系统过程创建 DBMS_MVIEW 包。

```
SP_CREATE_SYSTEM_PACKAGES (1,'DBMS_MVIEW');
```

数据准备

```
//创建模式和表
CREATE SCHEMA PURCHASING1 AUTHORIZATION SYSDBA;
CREATE TABLE PURCHASING1.VENDOR
(VENDORID INT IDENTITY(1,1) PRIMARY KEY,
ACCOUNTNO VARCHAR(15) NOT NULL,
NAME VARCHAR(50) NOT NULL,
ACTIVEFLAG BIT NOT NULL,
WEBURL VARCHAR(1024),
CREDIT INT NOT NULL CHECK(CREDIT IN(1,2,3,4,5)));
```

```
//创建物化视图日志
CREATE MATERIALIZED VIEW LOG ON PURCHASING1.VENDOR WITH
ROWID(VENDORID,ACCOUNTNO,NAME,ACTIVEFLAG,WEBURL,CREDIT) PURGE      START WITH
SYSDATE     REPEAT INTERVAL '1' DAY;


//创建物化视图 PURCHASING1.MV_VENDOR_EXCELLENT1
 CREATE MATERIALIZED VIEW    PURCHASING1.MV_VENDOR_EXCELLENT1
REFRESH WITH ROWID START WITH SYSDATE+1 NEXT SYSDATE + 2 AS
SELECT ROWID A, VENDORID, ACCOUNTNO, NAME, ACTIVEFLAG, CREDIT
FROM    PURCHASING1.VENDOR
WHERE    CREDIT = 1;


//创建物化视图 SYSDBA.MV_VENDOR_EXCELLENT2
CREATE MATERIALIZED VIEW    SYSDBA.MV_VENDOR_EXCELLENT2
REFRESH WITH ROWID START WITH SYSDATE+1 NEXT SYSDATE + 2 AS
SELECT ROWID A, VENDORID, ACCOUNTNO, NAME, ACTIVEFLAG, CREDIT
FROM    PURCHASING1.VENDOR
WHERE    CREDIT = 1;
```

一次性刷新模式 PURCHASING1 模式 MV_VENDOR_EXCELLENT1 物化视图和 SYSDBA 模式 MV_VENDOR_EXCELLENT2 物化视图，前者使用快速刷新，后者使用完全刷新。

```
CALL DBMS_MVIEW.REFRESH('PURCHASING1.MV_VENDOR_EXCELLENT1,
SYSDBA.MV_VENDOR_EXCELLENT2', 'FC');
```

33 UTL_SMTP 包

UTL_SMTP 包的功能是提供对 smtp 服务器的基本访问请求以及通过 SMTP 服务器发送邮件的功能。

使用 UTL_SMTP 包发送邮件的大致流程为：首先，声明连接，并开启连接；其次，遵照 SMTP 协议与服务器进行握手，并根据 RFC821 简单邮件传输协议进行命令和邮件内容等的发送；最后，关闭会话，关闭连接。

33.1 相关方法

下面对各个函数和过程进行详细说明。

1. CONNECTION

定义 CONNECTION 为 RECORD 类型，为下面的过程或函数使用。

语法如下:

```
TYPE CONNECTION IS RECORD (
    REMOTE_HOST             VARCHAR2(255),
    REMOTE_PORT             INT,
    TX_TIMEOUT              INT,
    PRIVATE_HNDL    BIGINT
);
```

参数详解

- REMOTE_HOST，REMOTE_PORT

为 SMTP 服务器的地址及端口号，由 OPEN_CONNECTION 过程传入。

- TX_TIMEOUT

为等待响应时间，同样由 OPEN_CONNECTION 过程传入。

- PRIVATE_HNDL

为内部标识 CONNECTION 的 ID

2. REPLY

支持新类型 REPLY，表示一行 SMTP 服务器的回复

语法如下:

```
TYPE REPLY IS RECORD (
    CODE    INTEGER,
    TEXT    VARCHAR2(508)
);
```

参数详解

- CODE

为服务器返回的返回码。

- TEXT

为服务器返回的消息内容。

3. REPLIES

支持新类型 REPLIES，表示多行 SMTP 服务器的回复

语法如下:

```
TYPE REPLIES IS TABLE OF REPLY INDEX BY BINARY_INTEGER;
```

4. OPEN_CONNECTION

用于开启与 SMTP 服务器的连接或回复。

语法如下:

```
FUNCTION OPEN_CONNECTION (
    HOST                                IN   VARCHAR2,
    PORT                                IN   PLS_INTEGER DEFAULT 25,
    TX_TIMEOUT                          IN   PLS_INTEGER DEFAULT NULL,
    WALLET_PATH                         IN   VARCHAR2      DEFAULT NULL,
    WALLET_PASSWORD                     IN   VARCHAR2      DEFAULT NULL,
    SECURE_CONNECTION_BEFORE_SMTP       IN   BOOLEAN       DEFAULT FALSE,
    SECURE_HOST                         IN   VARCHAR2      DEFAULT NULL
)RETURN CONNECTION;

FUNCTION OPEN_CONNECTION (
    HOST                                IN       VARCHAR2,
    PORT                                IN       PLS_INTEGER DEFAULT 25,
    C                                   OUT      CONNECTION,
    TX_TIMEOUT                          IN       PLS_INTEGER      DEFAULT NULL,
    WALLET_PATH                         IN       VARCHAR2      DEFAULT NULL,
    WALLET_PASSWORD                     IN       VARCHAR2      DEFAULT NULL,
    SECURE_CONNECTION_BEFORE_SMTP       IN       BOOLEAN       DEFAULT FALSE,
    SECURE_HOST                         IN       VARCHAR2      DEFAULT NULL
)RETURN   REPLY;
```

参数详解

- HOST

SMTP 服务器地址。

- PORT

SMTP 服务器端口号。

- C

使用的 SMTP 连接。

- TX_TIMEOUT

等待时间，NULL 代表一直等待，0 代表不等待。

- WALLET_PATH

保留参数。

- WALLET_PASSWORD

保留参数。

- SECURE_CONNECTION_BEFORE_SMTP

在 SMTP 通信之前，是否使用安全通信连接。

- SECURE_HOST

保留参数。

返回值

CONNECTION 表示使用的 SMTP 连接，REPLY 表示一行 SMTP 服务器的回复。

5. HELO/EHLO

用于向 SMTP 服务器打招呼。EHLO 指令表示需要 SMTP 认证，HELO 指令表示不需要 SMTP 认证。

语法如下:

```
PROCEDURE HELO (
    C               IN OUT      CONNECTION,
    R_DOMAIN        IN          VARCHAR2
);

PROCEDURE EHLO (
    C               IN OUT      CONNECTION,
    R_DOMAIN        IN          VARCHAR2
);

FUNCTION  HELO  (
    C           IN  OUT  NOCOPY     CONNECTION,
    R_DOMAIN    IN                  VARCHAR2
)RETURN  REPLY;

FUNCTION  EHLO  (
    C           IN  OUT  NOCOPY     CONNECTION,
    R_DOMAIN    IN                  VARCHAR2
)  RETURN  REPLIES;
```

参数详解

- C

使用的 SMTP 连接。

- R_DOMAIN

SMTP 连接的域名。

返回值

REPLY：一行 SMTP 服务器的回复；REPLIES：多行 SMTP 服务器的回复。

6. COMMAND

用于向 SMTP 服务器发送命令。命令不由此程序判断正确性，发送错误命令，程序会报错。

语法如下:

```
PROCEDURE COMMAND (
    C           IN OUT      CONNECTION,
    CMD         IN          VARCHAR2,
    ARG         IN          VARCHAR2 DEFAULT NULL
);

FUNCTION COMMAND (
    C   IN OUT  NOCOPY      CONNECTION,
    CMD     IN          VARCHAR2,
    ARG     IN          VARCHAR2    DEFAULT  NULL
)RETURN  REPLY;
```

参数详解

- C

使用的连接。

- CMD　　命令名

准备登陆为 AUTH LOGIN(AUTH LOGIN 命令使用后，要分别再使用 COMMAND 发送

用户名和密码)； 握手为 HELO；更多的命令详见 RFC821 简单邮件传输协议。

- ARG 命令参数

可选参数，不同的 CMD 使用不同的 ARG 。例如: AUTH LOGIN 无参数，但必须接下来继续使用 COMMAND 发送用户名和密码； 而 HELO 后面的参数为服务器地址。

返回值.

一行 SMTP 服务器的回复。

7. MAIL

用于告知服务器发件人。

语法如下:

```
PROCEDURE MAIL (
    C                IN OUT      CONNECTION,
    SENDER           IN          VARCHAR2,
    M_PARAMETERS     IN          VARCHAR2    DEFAULT NULL
);

FUNCTION  MAIL  (
    C              IN  OUT  NOCOPY    CONNECTION,
    SENDER         IN                 VARCHAR2,
    M_PARAMETERS   IN                 VARCHAR2  DEFAULT  NULL
)  RETURN  REPLY;
```

参数详解

- C

使用的 SMTP 连接。

- SENDER

发件人邮箱。

- M_PARAMETERS

可选参数。公认的几种 SMTP 的扩展将会用到 MAIL FROM 和 RCPT TO 的额外参数，详见 RFC1869 第 6 部分，格式要求必须符合 "XXX=XXX (XXX=XXX ....)"。

返回值

一行 SMTP 服务器的回复。

8. RCPT

用于告知 SMTP 服务器收件人信息。

语法如下:

```
PROCEDURE RCPT (
    C                   IN OUT      CONNECTION,
    RECIPIENTS      IN              VARCHAR2,
    R_PARAMETERS    IN              VARCHAR2    DEFAULT NULL
);

FUNCTION  RCPT  (
    C               IN  OUT  NOCOPY     CONNECTION,
    RECIPIENTS      IN                  VARCHAR2,
```

```
    R_PARAMETERS   IN                    VARCHAR2   DEFAULT   NULL

)RETURN   REPLY;
```

参数详解

- C

使用的 SMTP 连接。

- RECIPIENTS

收件人邮箱。

- R_PARAMETERS

可选参数。公认的几种 SMTP 的扩展将会用到 MAIL FROM 和 RCPT TO 的额外参数，详见 RFC1869 第 6 部分，格式要求必须符合 "XXX=XXX (XXX=XXX ....)"。

返回值

一行 SMTP 服务器的回复。

9. OPEN_DATA

用于告知 SMTP 服务器开始发送数据。只有使用过该过程才可以发送数据。

语法如下:

```
PROCEDURE OPEN_DATA (
    C                IN OUT          CONNECTION
);

FUNCTION   OPEN_DATA   (
    C        IN   OUT   NOCOPY   CONNECTION
```

```
) RETURN  REPLY;
```

参数详解

- C

使用的 SMTP 连接。

返回值

一行 SMTP 服务器的回复。

10. WRITE_DATA

用于发送邮件内容（VARCHAR 类型）。

语法如下:

```
PROCEDURE WRITE_DATA (
    C               IN OUT      CONNECTION,
    DATA            IN          VARCHAR2
);
```

参数详解

- C

使用的 SMTP 连接。

- DATA

发送的内容，VARCHAR 类型。

11. WRITE_RAW_DATA

用来发送邮件内容（VARBINARY 类型）。

语法如下:

```
PROCEDURE WRITE_RAW_DATA (
    C               IN OUT      CONNECTION,
    DATA            IN          VARBINARY
);
```

参数详解

- C

使用的 SMTP 连接。

- DATA

邮件内容，VARBINARY 类型。

12. CLOSE_DATA

关闭数据流，告知服务器数据发送完毕。

语法如下:

```
PROCEDURE CLOSE_DATA (
    C               IN OUT      CONNECTION
);

FUNCTION CLOSE_DATA(
    C   IN OUT  NOCOPY      CONNECTION
)RETURN  REPLY;
```

参数详解

- C

所使用 SMTP 连接。

返回值.

一行 SMTP 服务器的回复。

13. QUIT

用来结束会话。

语法如下:

```
PROCEDURE QUIT (
    C                IN OUT          CONNECTION
);

FUNCTION  QUIT(
   C  IN  OUT  NOCOPY  CONNECTION
)RETURN  REPLY;
```

参数详解

- C

使用的 SMTP 连接。

返回值

一行 SMTP 服务器的回复。

14. CLOSE_CONNECTION

关闭 SMTP 连接。

语法如下:

```
PROCEDURE CLOSE_CONNECTION (
    C                IN OUT          CONNECTION
);
```

参数详解

- C

所需要关闭的 SMTP 连接。

33.2 举例说明

使用包内的过程和函数之前，如果还未创建过系统包，请先调用系统过程创建系统包。

```
SP_CREATE_SYSTEM_PACKAGES(1, 'UTL_SMTP');
SP_CREATE_SYSTEM_PACKAGES(1, 'UTL_TCP');
SP_CREATE_SYSTEM_PACKAGES(1, 'UTL_RAW');
```

举一个使用 UTL_SMTP 包发邮件的例子。发件人 yhx@dameng.shanghai，收件人 gyf@dameng.shanghai，发送内容为邮件头部信息(收件人、发件人、发送时间、主题、编码等)，重复 10 次的 “达梦数据库第 i 行”。

```
DECLARE
  C UTL_SMTP.CONNECTION; //声明连接
  MSG VARCHAR2(4000);
BEGIN
  C := UTL_SMTP.OPEN_CONNECTION('192.168.0.212'); //开启连接
  UTL_SMTP.HELO(C, '192.168.0.212');    //握手
  UTL_SMTP.COMMAND(C, 'AUTH LOGIN');   //准备登陆
```

```
UTL_SMTP.COMMAND(C,UTL_RAW.CAST_TO_VARCHAR2(UTL_ENCODE.BASE64_ENCODE(UTL_R
AW.CAST_TO_RAW('yhx@DAMENG.SHANGHAI'))));  //用户名

UTL_SMTP.COMMAND(C,UTL_RAW.CAST_TO_VARCHAR2(UTL_ENCODE.BASE64_ENCODE(UTL_R
AW.CAST_TO_RAW('888888'))));     //密码
  UTL_SMTP.MAIL(C, 'yhx@DAMENG.SHANGHAI'); //发件人
  UTL_SMTP.RCPT(C, 'GYF@DAMENG.SHANGHAI');     //收件人
  UTL_SMTP.OPEN_DATA(C);     //开始发送邮件内容
  MSG := 'CONTENT-TYPE: TEXT/PLAIN; CHARSET=GB2312' || UTL_TCP.CRLF || 'DATE:' ||
  TO_CHAR(SYSDATE, 'MM DD YYYY HH24:MI:SS') || UTL_TCP.CRLF || 'FROM:' ||
  'YHX@DAMENG.SHANGHAI' || UTL_TCP.CRLF || 'SUBJECT: HOW TO USE SMTP' || UTL_TCP.CRLF ||
  'TO: ' || 'GYF@DAMENG.SHANGHAI' ||   UTL_TCP.CRLF ||
  'CONTENT-TYPE: TEXT/PLAIN; CHARSET=GB2312' || UTL_TCP.CRLF ;
     //MSG 为邮件头信息, 邮件头信息需要与正文信息, 通过 UTL_TCP.CRLF 来分隔开
  UTL_SMTP.WRITE_RAW_DATA(C, UTL_RAW.CAST_TO_RAW(MSG));   //发送邮件头信息
  UTL_SMTP.WRITE_DATA(C, UTL_TCP.CRLF ||'达梦数据库 SMTP 包邮件示例:' || UTL_TCP.CRLF);   //
发送邮件正文信息
  FOR I IN 1 .. 10 LOOP
        UTL_SMTP.WRITE_DATA(C, '达梦数据库 DM  第' || i || '行' || UTL_TCP.CRLF);
  END LOOP;
  UTL_SMTP.CLOSE_DATA(C);    //邮件内容发送完毕
  UTL_SMTP.QUIT(C);   //结束会话
  END;
```

所收到的邮件内容(包含 msg 中所写的收件人，发件人，时间信息)，见图 35.1:

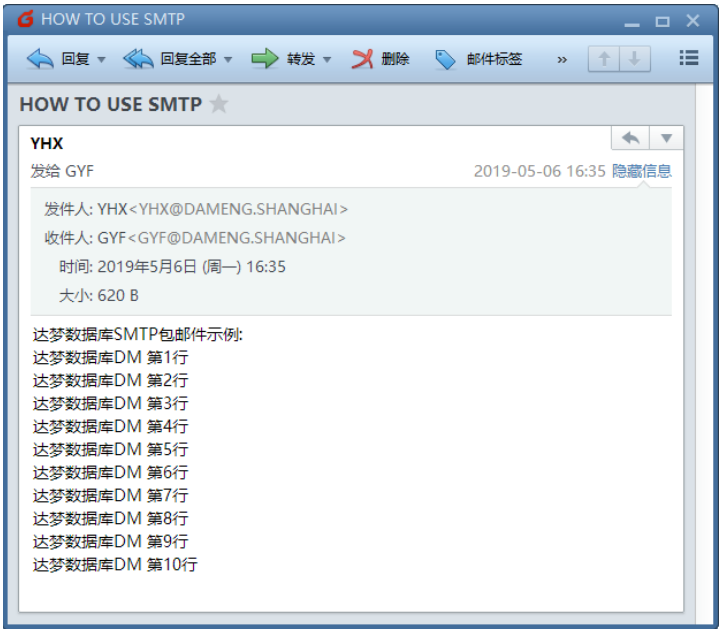


图 35.1 收到的邮件内容

34 UTL_HTTP 包

UTL_HTTP 包提供了通过 HTTP 协议获取网页内容的功能。

在发起 HTTP 请求和读取 HTTP 响应消息的过程中，各函数的调用顺序如下图 36.1 所示。

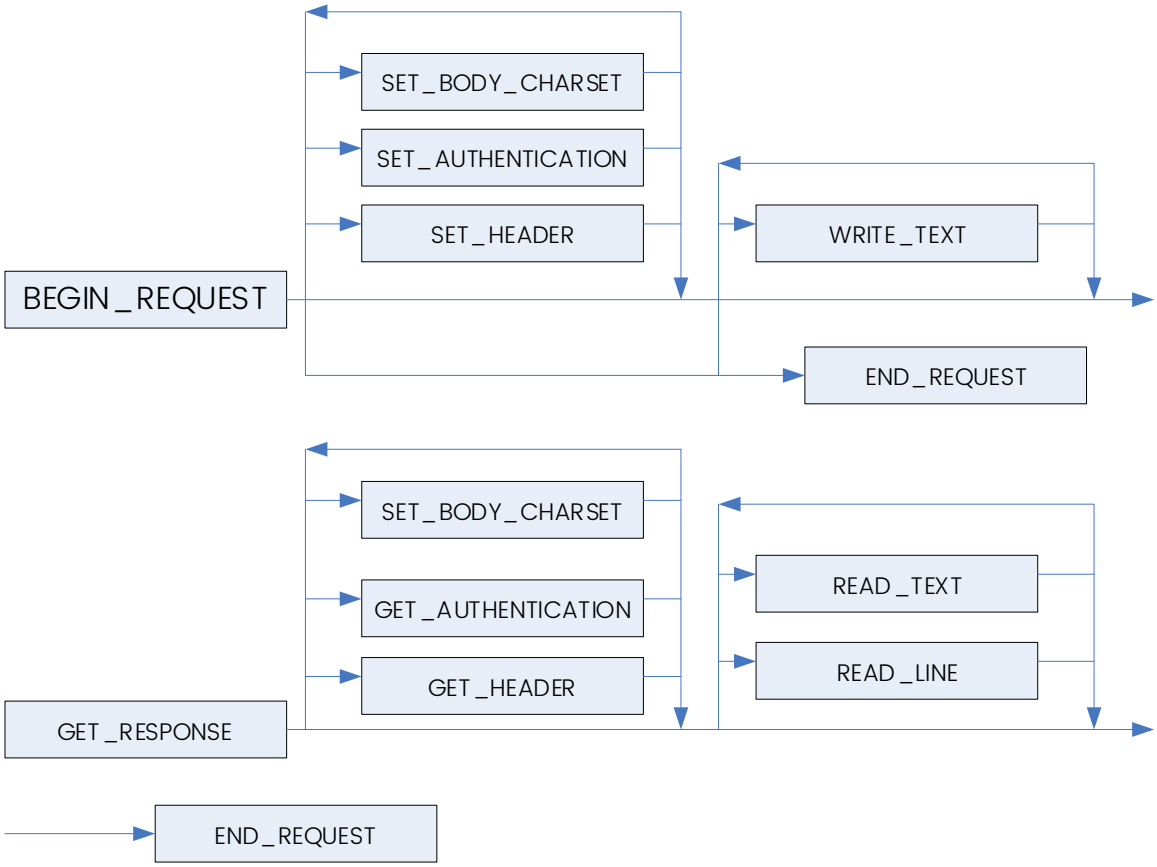


图 36.1 HTTP 请求/响应消息相关函数使用顺序图

REQUEST、SET_TRANSFER_TIMEOUT、SET_WALLET、SET_DETAILED_EXCP_SUPPORT、GET_DETAILED_EXCP_SUPPORT 和 SET_RESPONSE_ERROR_CHECK 不参与上述使用顺序图，用户可在任意时刻调用上述函数和过程。REQUEST 函数相当于上图中各函数的缺省配置，最终返回指定网页的前 2000 个字节。

另外，达梦支持基于 HTTP 1.1 协议的持久连接功能，与该功能相关的所有过程及函数也

不参与上述使用顺序图，用户可在任意时刻调用相关过程或函数，以对当前会话的持久连接进行相应设置。

34.1 相关方法

下面对各个函数和过程进行详细说明。

1. REQ

定义 REQ 为 RECORD 类型，用来存放请求消息。

语法如下:

```
TYPE REQ IS RECORD (
    URL             VARCHAR2(32767),
    METHOD          VARCHAR2(64),
    HTTP_VERSION    VARCHAR2(64)
);
```

参数详解

- URL　WEB 服务器的 URL 地址。URL 中可以包含用户名密码。
- METHOD　对指定 URL 发起的操作命令。常见的命令有 GET、POST，默认为 GET。
- HTTP_VERSION　HTTP 协议版本号，UTL_HTTP 包提供 1.0 和 1.1 两个版本宏定义，若此参数设为 NULL，则默认使用最新的 HTTP 协议版本。此参数默认为 NULL。

2. RESP

定义 RESP 为 RECORD 类型，用来存放响应消息。

语法如下:

```
TYPE RESP IS RECORD (
```

```
    STATUS_CODE             PLS_INTEGER,
    REASON_PHRASE       VARCHAR2(256),
    HTTP_VERSION        VARCHAR2(64)
);
```

参数详解

- STATUS_CODE　WEB 服务器返回的状态码。
- REASON_PHRASE　关于状态码的简单文本描述。
- HTTP_VERSION　HTTP 协议版本号，UTL_HTTP 包提供 1.0 和 1.1 两个版本宏定义，若此参数设为 NULL，则默认使用最新的 HTTP 协议版本。此参数默认为 NULL。

3. CONNECTION

定义 CONNECTION 为 RECORD 类型，用来存放 HTTP 请求完成后的持久连接信息。

语法如下:

```
TYPE CONNECTION IS RECORD (
    HOST            VARCHAR2(256),
    PORT            INTEGER,
    PROXY_HOST          VARCHAR2(256),
    PROXY_PORT          INTEGER,
    SSL         BOOLEAN
);
```

参数详解

- HOST WEB 服务器 IP 地址。
- PORT WEB 服务器端口号。

- PROXY_HOST　WEB 服务器代理 IP 地址。
- PROXY_PORT　WEB 服务器代理端口号。
- SSL　是否使用 SSL，保留参数。

4. CONNECTION_TABLE

定义 CONNECTION_TABLE 为 CONNECTION 的索引表类型，用来存放所有持久连接相关信息。

语法如下：

```
TYPE CONNECTION_TABLE IS TABLE OF CONNECTION INDEX BY BINARY_INTEGER;
```

5. BEGIN_REQUEST

用于发起一个新的 http 请求。单个会话若调用 BEGIN_REQUEST 后未调用 END_REQUEST 结束，则最多同时发起 20 个 http 请求。

语法如下：

```
FUNCTION    BEGIN_REQUEST (
    URL                 IN      VARCHAR2,
    METHOD              IN      VARCHAR2            DEFAULT 'GET',
    HTTP_VERSION        IN      VARCHAR2            DEFAULT NULL,
    REQUEST_CONTEXT     IN      REQUEST_CONTEXT_KEY DEFAULT NULL
)RETURN REQ;
```

参数详解

- URL　WEB 服务器的 URL 地址。URL 中可以包含用户名密码。
- METHOD 对指定 URL 发起的操作命令。

- HTTP_VERSIONHTTP 协议版本号，UTL_HTTP 包提供 1.0 和 1.1 两个版本宏定义，若此参数设为 NULL，则默认使用最新的 HTTP 协议版本。此参数默认为 NULL。
- REQUEST_CONTEXT 目前达梦不支持此参数，仅用作兼容参数。

返回值

HTTP 请求消息。

6. END_REQUEST

用于结束 http 请求。

语法如下:

```
PROCEDURE END_REQUEST (
    R           IN OUT          REQ
);
```

参数详解

- R HTTP 请求句柄。

7. GET_RESPONSE

用于读取 http 响应消息。

语法如下:

```
FUNCTION GET_RESPONSE (
    R           IN OUT          REQ
) RETURN RESP;
```

参数详解

- R HTTP 请求消息。

返回值

HTTP 请求句柄。

8. END_RESPONSE

用于结束 http 响应操作。

语法如下:

```
PROCEDURE END_RESPONSE (
    R           IN OUT          RESP
);
```

参数详解

- R HTTP 响应句柄。

9. WRITE_TEXT

用于向 http 请求 body 中填写数据。

语法如下:

```
PROCEDURE WRITE_TEXT(
    R           IN OUT          REQ,
    DATA        IN              VARCHAR2 CHARACTER SET ANY_CS
);
```

参数详解

- R HTTP 请求消息。
- DATA 准备向 BODY 写入的数据。

10. READ_TEXT

用于读取 http 响应消息中 body 的内容并将内容输出到指定的缓冲区中。body 中的数据将自动转换成数据库所指定的字符集。

语法如下:

```
PROCEDURE READ_TEXT(
    R          IN OUT          RESP,
    DATA       OUT                 VARCHAR2 CHARACTER SET ANY_CS,
    LEN        IN              PLS_INTEGER DEFAULT NULL
);
```

参数详解

- R　HTTP 请求消息。
- DATA　读取 BODY 中的数据。
- LEN　请求读取的数据长度。

11. READ_RAW

用于读取 http 响应消息中 body 的内容并将内容输出到指定的缓冲区中。不对 body 中的数据进行字符集转码操作。

语法如下:

```
PROCEDURE READ_RAW(
    R          IN OUT          RESP,
    DATA       OUT                 VARBINARY,
    LEN        IN              PLS_INTEGER DEFAULT NULL
);
```

参数详解

- R HTTP 请求消息。
- DATA 读取 BODY 中的数据。
- LEN 请求读取的数据长度。

12. READ_LINE

用于按行读取 http 响应消息中 body 的内容并将内容输出到指定的缓冲区中。body 中的数据将自动转换成数据库所指定的字符集。

语法如下:

```
PROCEDURE READ_LINE(
    R                IN OUT    RESP,
    DATA             OUT       VARCHAR2,
    REMOVE_CRLF      IN        BOOLEAN DEFAULT FALSE
);
```

参数详解

- R HTTP 请求消息。
- DATA 读取 BODY 中的数据。
- REMOVE_CRLF 读取的数据是否去除换行标识，默认不去除。

13. SET_HEADER

用于设置 http 请求的头信息。设置头信息的请求将被立即发送到 web 服务器。

语法如下:

```
PROCEDURE SET_HEADER (
```

```
    R           IN OUT      REQ,
    NAME        IN          VARCHAR2,
    VALUE       IN          VARCHAR2    DEFAULT NULL
);
```

参数详解

- R HTTP 请求消息。
- NAME HEADER 中请求设置的属性名。
- VALUE HEADER 中请求设置的属性值。

14. GET_HEADER

用于获取 http 响应头信息。过程将响应头中的第 n 个属性名及属性值返回。

语法如下:

```
PROCEDURE GET_HEADER (
    R           IN OUT      RESP,
    N           IN          PLS_INTEGER,
    NAME        OUT             VARCHAR2,
    VALUE       OUT             VARCHAR2
);
```

参数详解

- R HTTP 响应消息。
- N HEADER 中的第 N 个属性。
- NAME 属性名。
- VALUE 属性值。

15. SET_BODY_CHARSET

用于设置 body 的字符集，至于是请求还是响应消息中的 body，则需要看上下文。

语法如下:

```
PROCEDURE SET_BODY_CHARSET (
    CHARSET      IN          VARCHAR2 DEFAULT NULL
);
```

或者

```
PROCEDURE SET_BODY_CHARSET(
    R            IN OUT      REQ,
    CHARSET      IN          VARCHAR2 DEFAULT NULL
);
```

或者

```
PROCEDURE SET_BODY_CHARSET(
    R            IN OUT      RESP,
    CHARSET      IN          VARCHAR2 DEFAULT NULL
);
```

- R HTTP 请求（或响应）句柄。
- CHARSET 字符集名称。

16. SET_AUTHENTICATION

用于设置 http 请求消息中的权限。WEB 服务器需要这些权限信息用于授权访问。

语法如下:

```
PROCEDURE SET_AUTHENTICATION(
    R           IN OUT      REQ,
    USERNAME    IN          VARCHAR2,
    PASSWORD    IN          VARCHAR2,
    SCHEME      IN          VARCHAR2 DEFAULT 'BASIC',
    FOR_PROXY   IN          BOOLEAN DEFAULT FALSE
);
```

参数详解

- R HTTP 请求消息。
- USENAME WEB 服务器端所需的用户名。
- PASSWORD WEB 服务器端所需的密码。
- SCHEME 访问模式，仅支持"BASIC"。
- FOR_PROXY 是否通过代理来访问 WEB，默认为 FALSE。

17. GET_AUTHENTICATION

从 http 响应消息中获取 WEB 服务器所需要的授权信息。

语法如下:

```
PROCEDURE GET_AUTHENTICATION(
    R           IN OUT      RESP,
    SCHEME      OUT             VARCHAR2,
    REALM       OUT             VARCHAR2,
    FOR_PROXY   IN          BOOLEAN   DEFAULT FALSE
```

```
);
```

参数详解

- R HTTP 响应消息。
- SCHEME WEB 服务器的授权模式。
- REALM WEB 服务器需要授权的区域。
- FOR_PROXY 是否返回访问代理所需的权限而不是 WEB 服务器的权限，默认 FALSE。

18. REQUEST

返回网页的前 2000 个字节。该函数可以直接用在 SQL 查询中。

语法如下:

```
FUNCTION    REQUEST (
    URL             IN          VARCHAR2,
    PROXY           IN          VARCHAR2 DEFAULT NULL
)RETURN VARCHAR;
```

参数详解

- URL WEB 服务器的 URL 地址。URL 中可以包含用户名密码。
- PROXY 代理服务器的 URL。

返回值

返回网页的前 2000 个字节。

19. SET_TRANSFER_TIMEOUT

设置 HTTP 相关连接的超时时长。

语法如下:

```
PROCEDURE SET_TRANSFER_TIMEOUT(
    TIMEOUT      IN      INT      DEFAULT 60
);
```

或者

```
PROCEDURE SET_TRANSFER_TIMEOUT(
    R            IN OUT  REQ,
    TIMEOUT      IN      INT      DEFAULT 60
);
```

参数详解

- R HTTP 请求句柄。若指定该参数，则仅设置指定的 HTTP 相关连接的超时时长；若不指定该参数，则设置当前会话后续所有 HTTP 相关连接的超时时长。
- TIMEOUT 超时时长，若 HTTP 连接超过该时长未响应，则报错：传输超时。单位为秒，取值范围为 1 ~ 2147483647，缺省为 60。

20. SET_PERSISTENT_CONN_SUPPORT

打开或关闭当前会话/当前请求的持久连接功能。

语法如下:

```
PROCEDURE SET_PERSISTENT_CONN_SUPPORT (
    ENABLE       IN          BOOLEAN,
    MAX_CONNS    IN          INTEGER          DEFAULT 0
);
```

或者

```
PROCEDURE SET_PERSISTENT_CONN_SUPPORT(
```

```
    R               IN OUT      REQ,
    ENABLE          IN          BOOLEAN             DEFAULT FALSE
);
```

参数详解

- ENABLE　是否打开持久连接功能。
- MAX_CONNS　持久连接个数的最大值。
- R　HTTP 请求句柄。

21. GET_PERSISTENT_CONN_SUPPORT

获取当前会话的持久连接设置信息。

语法如下:

```
PROCEDURE GET_PERSISTENT_CONN_SUPPORT(
    ENABLED         OUT         BOOLEAN,
    MAX_CONNS       OUT         INTEGER
);
```

参数详解

- ENABLED 当前会话是否已打开持久连接功能。
- MAX_CONNS 持久连接个数的最大值。

22. GET_PERSISTeNT_CONN_count

获取当前会话的持久连接个数。

语法如下:

```
FUNCTION GET_PERSISTENT_CONN_COUNT() RETURN INTEGER;
```

返回值

当前会话的持久连接个数。

23. GET_PERSISTENT_CONNS

获取当前会话所有持久连接的相关信息。

语法如下:

```
PROCEDURE GET_PERSISTENT_CONNS(
    CONNECTIONS      IN OUT          CONNECTION_TABLE
);
```

参数详解

- CONNECTIONS 当前会话持久连接的索引表。

24. CLOSE_PERSISTENT_CONN

关闭指定的持久连接。

语法如下:

```
PROCEDURE CLOSE_PERSISTENT_CONN(
    CONN     IN          CONNECTION
);
```

参数详解

- CONN 持久连接句柄。

25. CLOSE_PERSISTENT_CONNS

关闭指定的持久连接或所有持久连接。

语法如下:

```
PROCEDURE CLOSE_PERSISTENT_CONNS(
    HOST            IN      VARCHAR2        DEFAULT NULL,
    PORT            IN      INTEGER         DEFAULT NULL,
    PROXY_HOST      IN      VARCHAR2        DEFAULT NULL,
    PROXY_PORT      IN      INTEGER         DEFAULT NULL,
    SSL             IN      BOOLEAN         DEFAULT NULL
);
```

参数详解

- HOST 待关闭持久连接的 IP 地址。若为 NULL，则关闭当前会话的所有持久连接。
- PORT 待关闭持久连接的端口号。若为 NULL，则关闭指定 HOST 的所有持久连接。
- PROXY_HOST 待关闭持久连接的代理 IP 地址。
- PROXY_PORT 待关闭持久连接的代理端口号。
- SSL 待关闭持久连接是否使用 SSL，保留参数，暂时忽略。

26. WRITE_RAW

用于向 HTTP 请求消息的 BODY 中填写 BINARY 数据。

语法如下:

```
PROCEDURE WRITE_RAW(
    R           IN OUT      REQ,
    DATA        IN          VARBINARY
);
```

参数详解

- R HTTP 请求消息。
- DATA 准备向 BODY 写入的数据。

27. GET_HEADER_BY_NAME

用于获取 HTTP 响应头信息。过程将获取响应头中的第 N 个属性名为指定属性名的属性值。

语法如下:

```
PROCEDURE GET_HEADER_BY_NAME (
    R           IN OUT      RESP,
    NAME        IN          VARCHAR2,
    VALUE       OUT             VARCHAR2,
    N           IN          PLS_INTEGER DEFAULT 1
);
```

参数详解

- R HTTP 响应消息。
- NAME 指定的属性名。
- VALUE 属性值。
- N HEADER 中的第 N 个指定属性名的属性。

28. SET_PROXY

用于配置 HTTP 代理服务器，便于数据库会话进行 HTTP 请求时经过代理服务器。

语法如下:

```
PROCEDURE SET_PROXY (
    PROXY                   IN VARCHAR2,
    NO_PROXY_DOMAINS    IN VARCHAR2 DEFAULT NULL
);
```

参数详解

- PROXY 指定代理服务器的 URL。
- NO_PROXY_DOMAINS 用于指定不需要经过代理服务器的 URL，用“,”隔开。

29. GET_PROXY

用于获取 HTTP 代理服务器的配置，获取的就是用 SET_PROXY 配置的内容。

语法如下:

```
PROCEDURE GET_PROXY (
    PROXY                   OUT NOCOPY VARCHAR2,
    NO_PROXY_DOMAINS    OUT NOCOPY VARCHAR2
);
```

参数详解

- PROXY 指定代理服务器的 URL。
- NO_PROXY_DOMAINS 用于指定不需要经过代理服务器的 URL，用“,”隔开。

30. SET_WALLET

用于设置钱包文件路径，钱包文件中包含客户端相关证书密钥等信息。该方法目前仅做语法兼容，暂无实际效果。

语法如下:

```
PROCEDURE SET_WALLET(
    PATH            IN VARCHAR2,
    PASSWORD        IN VARCHAR2 DEFAULT NULL
);
```

参数详解

- PATH 钱包文件路径。
- PASSWORD 钱包文件密码。

31. SET_DETAILED_EXCP_SUPPORT

发起 HTTP 请求过程中发生异常时，是否抛出详细的异常信息。

语法如下:

```
PROCEDURE SET_DETAILED_EXCP_SUPPORT(
    ENABLE_FLAG       IN BOOLEAN DEFAULT FALSE
);
```

参数详解

- ENABLE_FLAG 是否抛出详细异常。TRUE：是；FALSE：否。默认值为 FALSE，此时请求异常时会固定抛出 REQUEST_FAILED 异常。

32. GET_DETAILED_EXCP_SUPPORT

获取当前 HTTP 请求异常配置信息。

语法如下:

```
PROCEDURE GET_DETAILED_EXCP_SUPPORT(
    ENABLE_FLAG       OUT BOOLEAN
```

```
);
```

参数详解

- ENABLE_FLAG 是否抛出详细异常。

33. SET_RESPONSE_ERROR_CHECK

是否检查 HTTP 响应码。若是，则响应码为 4XX（4 开头的三位数字）时抛出 HTTP_CLIENT_ERROR 异常，响应码为 5XX（5 开头的三位数字）时抛出 HTTP_SERVER_ERROR 异常。

语法如下:

```
PROCEDURE SET_RESPONSE_ERROR_CHECK(
    ENABLE_FLAG       IN BOOLEAN DEFAULT FALSE
);
```

参数详解

- ENABLE_FLAG 是否检查 HTTP 响应码。TRUE：是；FALSE：否。默认值为 FALSE。

34. GET_DETAILED_SQLCODE

获取 UTL_HTTP 包执行过程中抛出的异常码。

语法如下:

```
FUNCTION GET_DETAILED_SQLCODE RETURN INTEGER;
```

35. GET_DETAILED_SQLERRM

获取 UTL_HTTP 包执行过程中抛出的异常错误描述。

语法如下:

```
FUNCTION GET_DETAILED_SQLERRM RETURN VARCHAR;
```

34.2 举例说明

使用包内的过程和函数之前，如果还未创建过系统包，请先调用系统过程创建系统包。

```
SP_CREATE_SYSTEM_PACKAGES (1,'UTL_HTTP');

SP_CREATE_SYSTEM_PACKAGES (1,'DBMS_OUTPUT');

//DBMS_OUTPUT.PUT_LINE 需要设置这条语句，才能打印出消息

SET SERVEROUTPUT ON;
```

举一个使用 UTL_HTTP 包获取网页信息的例子。

```
declare

req utl_http.REQ;

resp utl_http.resp;

data varchar2(32563);

receiveDate varchar;

begin

req :=

utl_http.begin_request('http://192.168.0.104/blog/2015_06_05/CSDN_public_1433381803730.

html', 'POST');

data := ' ';

utl_http.write_text(req, data);

resp := utl_http.get_response(req);

utl_http.set_body_charset(resp, 'utf-8');

//使用 read_text 时需注意 LEN 参数取值，当其超出数据实际长度时会报错"[-20018]:已达到正文的末尾"

utl_http.read_text(resp, receiveDate, 1024);

dbms_output.put_line(receiveDate);
```

```
utl_http.end_response(resp);

end;
```

输出的网页内容为：

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"

"www.w3.org/TR/xhtml1/DTD/xhtml1-strict.dtd">

<html xmlns="www.w3.org/1999/xhtml">

 <head>

  <script type="text/javascript" src="c.csdnimg.cn/pubfooter/js/tracking.js"

charset="utf-8"></script>

  <script type="text/javascript">

          var protocol = window.location.protocol;

          document.write('<script type="text/javascript"

src="csdnimg.cn/pubfooter/js/repoAddr2.js?v=' + Math.random() + '"></' + 'script>');

     </script>

  <script id="allmobilize" charset="utf-8"

src="a.yunshipei.com/46aae4d1e2371e4aa769798941cef698/allmobilize.min.js"></script>

  <meta http-equiv="Cache-Control" content="no-siteapp">

  <link rel="alternate" media="handheld" href="#">

  <title>使用 UTL_HTTP 包获取网页内容 - IndexMan 的专栏

          - 博客频道 - CSDN.NET</title>

  <meta http-equiv="Content-Type" content="text/html; charset=utf-8">

  <meta name="description" content="UTL_HTTP 包提供了容易的方式通过HTTP协议获取网页内容">
```

35 UTL_I18N 包

UTL_I18N 包提供字符串与十六进制编码的相互转换功能。

UTL_I18N 依赖于 UTL_RAW。创建 UTL_I18N 时，系统会自动创建 UTL_RAW 包。

35.1 相关方法

下面对各个函数和过程进行详细说明。

1. string_to_raw

将字符串转换成指定字符集下的十六进制串数值。

语法如下:

```
FUNCTION STRING_TO_RAW(
    DATA            IN      VARCHAR,
    TO_CHARSET      IN      VARCHAR     DEFAULT NULL
) RETURN VARBINARY
```

参数详解

- DATA　输入参数，字符串。
- TO_CHARSET　输入参数，字符集。指定转换后的字符集。

返回值

转换后的十六进制串数值。若字符集输错了，转换失败，返回 NULL；单字节 0~255 用任何字符集编码都可以转成 ASCII 值；三种字符集库（GBK 库、UTF8 库和韩文库）下都支持转换到 UTF8 编码，只有 UTF8 库支持转到字符集（BIG5、ISO_8859_9、EUC_JP、EUC_KR、KOI8R、SQL_ASCII、ISO_8859_1）。字符串输入 NULL，返回当前字符集下的编码。

2. raw_to_char

将从指定的字符集下的十六进制串数值转到字符串。

语法如下:

```
FUNCTION RAW_TO_CHAR(
    DATA            IN      VARBINARY,
    FROM_CHARSET    IN      VARCHAR         DEFAULT NULL
) RETURN VARCHAR
```

参数详解

- DATA　输入参数，VARBINARY 数据。
- FROM_CHARSET　输入参数，字符集。转换前的字符集。

返回值

转换后的字符串。注意：若字符集输错了，转换失败，返回 NULL；ASCII 值用任何字符集编码都可以转成单字节；只有 UTF8 库支持指定字符集（BIG5、ISO_8859_9、EUC_JP、EUC_KR、KOI8R、SQL_ASCII、ISO_8859_1）。十六进制串输入 NULL，返回在当前字符集下的字符串。

3. ESCAPE_REFERENCE

将字符串中的特殊字符转义成引用表示形式。

语法如下:

```
FUNCTION ESCAPE_REFERENCE(
    STR             IN      VARCHAR,
    PAGE_CS_NAME    IN      VARCHAR         DEFAULT NULL
) RETURN VARCHAR
```

参数详解

- STR　输入参数，待转义的字符串。
- PAGE_CS_NAME　输入参数，页字符集。目前仅做语法兼容，无实际作用。

返回值

转换后的字符串。

4. UNESCAPE_REFERENCE

将字符串中的引用表示形式反转义成特殊字符。

语法如下:

```
FUNCTION UNESCAPE_REFERENCE (
    STR                 IN          VARCHAR
) RETURN VARCHAR
```

参数详解

- STR　输入参数，待反转义的字符串。

返回值

转换后的字符串。

35.2 举例说明

使用包内的过程和函数之前，如果还未创建过系统包，请先调用系统过程 SP_CREATE_SYSTEM_PACKAGES 创建系统包。

```
SP_CREATE_SYSTEM_PACKAGES(1,'UTL_I18N');
```

例 1 STRING_TO_RAW 函数的使用。

一 将 GBK 库中，GBK 编码的字符串转换成指定字符集下的十六进制串数值。

```
select utl_i18n.string_to_raw('中国','utf8') from dual;
```

查询结果如下:

```
0xE4B8ADE59BBD



select utl_i18n.string_to_raw('中国','GBK') from dual;

查询结果如下:

0xD6D0B9FA



select utl_i18n.string_to_raw('abcdef','utf8') from dual;

查询结果如下:

0x616263646566



select utl_i18n.string_to_raw('abcdef','GBK') from dual;

查询结果如下:

0x616263646566
```

二 UTF8 库中，将 UTF8 编码的字符串转换成指定字符集下的十六进制串数值。

```
select utl_i18n.string_to_raw('備註','utf8');

查询结果如下:

0xB3C6B5F9
```

三 韩文库中，将韩文编码的字符串转换成指定字符集下的十六进制串数值。

```
select utl_i18n.string_to_raw('철수','EUC_KR') from dual;

查询结果如下:

0xC3B6BCF6
```

例 2 RAW_TO_CHAR 函数的使用。

一 从指定字符集下的十六进制串数值转为 GBK 编码（例如，本机为 GBK 库）字符串。

```
select UTL_I18N.RAW_TO_CHAR('61', 'utf8') from dual;

查询结果如下:

a



select UTL_I18N.RAW_TO_CHAR('E4B8ADE59BBD', 'UTF8') from dual;

查询结果如下:

中国



select UTL_I18N.RAW_TO_CHAR('D6D0B9FA', 'GBK') from dual;

查询结果如下:

中国



select UTL_I18N.RAW_TO_CHAR('616263', 'GBK') from dual;

查询结果如下:

abc



select UTL_I18N.RAW_TO_CHAR('616263', 'UTF8') from dual;

查询结果如下:

abc
```

二 从指定字符集下的十六进制串数值转为 UTF8 编码 (例如, 本机为 UTF8 库) 字符串。

```
select UTL_I18N.RAW_TO_CHAR(utl_i18n.string_to_raw('備註',,'utf8'),'utf8') from dual;

查询结果如下:

備註
```

三 从指定字符集下的十六进制串数值转为韩文编码（例如，本机为韩文库）字符串。

```
select UTL_I18N.RAW_TO_CHAR(0xC3B6BCF6,'EUC_KR') from dual;

查询结果如下:

철수
```

36 DBMS_AQADM 包

DBMS_AQADM 包提供高级队列的以下常用功能：创建、修改、删除队列表和队列，添加、删除订阅者等。

36.1 数据类型

1. SYS.AQ$_AGENT

系统对象类型，用于描述订阅者信息。

语法如下：

```
TYPE SYS.AQ$_AGENT IS OBJECT (
    NAME            VARCHAR(30),
    ADDRESS         VARCHAR(1024),
    PROTOCOL        NUMBER
);
```

参数详解

- NAME 指定订阅者名，非正规表示符要用""括起来，如"@@#"。
- 剩余参数仅做语法支持。

2. DBMS_AQADM. AQ$_PURGE_OPTIONS_T

DBMS_AQADM 包专有类型，指定清理队列表选项。

语法如下：

```
TYPE AQ$_PURGE_OPTIONS_T IS RECORD(
```

```
    BLOCK                   BOOLEAN DEFAULT FALSE,
    DELIVERY_MODE           INTEGER DEFAULT 1
);
```

参数详解

- 参数仅做语法支持。

36.2 相关方法

1. CREATE_QUEUE_TABLE

创建队列表。

语法如下:

```
PROCEDURE CREATE_QUEUE_TABLE(
    QUEUE_TABLE             IN      VARCHAR,
    QUEUE_PAYLOAD_TYPE      IN      VARCHAR,
    STORAGE_CLAUSE          IN      VARCHAR         DEFAULT NULL,
    SORT_LIST               IN      VARCHAR         DEFAULT NULL,
    MULTIPLE_CONSUMERS      IN      BOOLEAN         DEFAULT FALSE,
    MESSAGE_GROUPING        IN      INTEGER         DEFAULT NONE,
    TAB_COMMENT             IN      VARCHAR         DEFAULT NULL,
    AUTO_COMMIT             IN      BOOLEAN         DEFAULT TRUE,
    PRIMARY_INSTANCE        IN      INTEGER     DEFAULT 0,
    SECONDARY_INSTANCE      IN      INTEGER     DEFAULT 0,
    COMPATIBLE              IN      VARCHAR         DEFAULT NULL,
    SECURE                  IN      BOOLEAN         DEFAULT FALSE
);
```

参数详解

- QUEUE_TABLE 指定要创建的队列表名，格式为：[模式名.]表名。非正规表示符要用""括起来，如"@@#".MYTAB。
- QUEUE_PAYLOAD_TYPE 指定消息载荷类型，格式为：[模式名.]数据类型名。可取值包括 RAW\VARBINARY\VARBINARY(8188)或自定义 CLASS\OBJECT 类型。
- STORAGE_CLAUSE 指定创建队列表的存储选项，即建表语法 STORAGE()中的内容。
- MULTIPLE_CONSUMERS 指定队列是否可以添加多个订阅者，为 TRUE 时才可使用添加订阅、注册通知和触发回调等功能。
- TAB_COMMENT 指定队列表注释。
- 剩余参数仅做语法支持。

2. ALTER_QUEUE_TABLE

修改队列表。

语法如下:

```
PROCEDURE ALTER_QUEUE_TABLE (
    QUEUE_TABLE             IN      VARCHAR,
    TAB_COMMENT             IN      VARCHAR     DEFAULT NULL,
    PRIMARY_INSTANCE        IN      INTEGER     DEFAULT NULL,
    SECONDARY_INSTANCE      IN      INTEGER     DEFAULT NULL
);
```

参数详解

- QUEUE_TABLE 指定要修改的队列表名，格式为：[模式名.]队列名。
- TAB_COMMENT 指定要修改的队列表注释。

- 剩余参数仅做语法支持。

3. DROP_QUEUE_TABLE

删除队列表。

语法如下:

```
PROCEDURE DROP_QUEUE_TABLE(
    QUEUE_TABLE                 IN      VARCHAR,
    FORCE                       IN      BOOLEAN         DEFAULT FALSE,
    AUTO_COMMIT                 IN      BOOLEAN         DEFAULT TRUE
);
```

参数详解

- QUEUE_TABLE 指定要删除的队列表名，格式为: [模式名.]队列名。
- FORCE 指定是否强制删除关联的队列。
- 剩余参数仅做语法支持。

4. PURGE_QUEUE_TABLE

清理队列表中的消息。

语法如下:

```
PROCEDURE PURGE_QUEUE_TABLE(
    QUEUE_TABLE                 IN      VARCHAR,
    PURGE_CONDITION             IN      VARCHAR,
    PURGE_OPTIONS               IN      AQ$_PURGE_OPTIONS_T
);
```

参数详解

- QUEUE_TABLE　指定要清理的队列表名，格式为：[模式名.]队列名。
- 剩余参数仅做语法支持。

5. CREATE_QUEUE

创建队列。

语法如下:

```
PROCEDURE CREATE_QUEUE(
    QUEUE_NAME              IN      VARCHAR,
    QUEUE_TABLE             IN      VARCHAR,
    QUEUE_TYPE              IN      INTEGER     DEFAULT NORMAL_QUEUE,
    MAX_RETRIES             IN      NUMBER      DEFAULT NULL,
    RETRY_DELAY             IN      NUMBER      DEFAULT 0,
    RETENTION_TIME          IN      NUMBER      DEFAULT 0,
    DEPENDENCY_TRACKING     IN      BOOLEAN         DEFAULT FALSE,
    QUEUE_COMMENT               IN      VARCHAR         DEFAULT NULL,
    AUTO_COMMIT                 IN      BOOLEAN         DEFAULT TRUE
);
```

参数详解

- QUEUE_NAME 指定要创建的队列名，格式为：[模式名.]队列名。非正规表示符要用""括起来，如"@@#".MYQUEUE。
- QUEUE_TABLE　指定队列所属的队列表，格式为：[模式名.]队列表名。队列和队列表必须属于同一模式。

- QUEUE_COMMENT 指定队列注释。
- 剩余参数仅做语法支持。

6. ALTER_QUEUE

修改队列。

语法如下:

```
PROCEDURE ALTER_QUEUE (
    QUEUE_NAME          IN      VARCHAR,
    MAX_RETRIES         IN      NUMBER      DEFAULT NULL,
    RETRY_DELAY         IN      NUMBER      DEFAULT NULL,
    RETENTION_TIME      IN      NUMBER      DEFAULT NULL,
    QUEUE_COMMENT               IN      VARCHAR         DEFAULT NULL
);
```

参数详解

- QUEUE_NAME 指定要修改的队列名，格式为：[模式名.]队列名。
- QUEUE_COMMENT 指定队列注释。
- 剩余参数仅做语法支持。

7. DROP_QUEUE

删除队列，删除前必须先 STOP 队列。

语法如下:

```
PROCEDURE DROP_QUEUE(
    QUEUE_NAME          IN      VARCHAR,
```

```
    AUTO_COMMIT                 IN      BOOLEAN     DEFAULT TRUE
);
```

参数详解

- QUEUE_NAME 指定要删除的队列名，格式为：[模式名.]队列名。
- 剩余参数仅做语法支持。

8. START_QUEUE

启用队列功能。

语法如下：

```
PROCEDURE START_QUEUE(
    QUEUE_NAME              IN      VARCHAR,
    ENQUEUE                 IN      BOOLEAN     DEFAULT TRUE,
    DEQUEUE                 IN      BOOLEAN     DEFAULT TRUE
);
```

参数详解

- QUEUE_NAME 指定要启用功能的队列名，格式为：[模式名.]队列名。
- ENQUEUE 指定是否启用入队操作。
- DEQUEUE 指定是否启用出队操作。

9. STOP_QUEUE

禁用队列功能。

语法如下：

```
PROCEDURE STOP_QUEUE(
```

```
    QUEUE_NAME              IN      VARCHAR,
    ENQUEUE                 IN      BOOLEAN         DEFAULT TRUE,
    DEQUEUE                 IN      BOOLEAN         DEFAULT TRUE,
    WAIT                    IN      BOOLEAN         DEFAULT TRUE
);
```

参数详解

- QUEUE_NAME　指定要禁用功能的队列名，格式为：[模式名.]队列名。
- ENQUEUE　指定是否禁用入队操作。
- DEQUEUE　指定是否禁用出队操作。
- 剩余参数仅做语法支持。

10. ADD_SUBSCRIBER

添加订阅者。

语法如下:

```
PROCEDURE ADD_SUBSCRIBER (
    QUEUE_NAME              IN      VARCHAR,
    SUBSCRIBER              IN      SYS.AQ$_AGENT,
    RULE                    IN      VARCHAR         DEFAULT NULL,
    TRANSFORMATION          IN      VARCHAR         DEFAULT NULL,
    QUEUE_TO_QUEUE          IN      BOOLEAN         DEFAULT FALSE,
    DELIVERY_MODE           IN      INTEGER         DEFAULT DBMS_AQADM.PERSISTENT
);
```

参数详解

- QUEUE_NAME　指定要添加订阅者的队列名，格式为：[模式名.]队列名。
- SUBSCRIBER　指定订阅者，队列的订阅者最大数量为 1024。参考 37.1 节中的数据类型说明。
- 剩余参数仅做语法支持。

11. REMOVE_SUBSCRIBER

删除订阅者。

语法如下：

```
PROCEDURE REMOVE_SUBSCRIBER (
    QUEUE_NAME              IN      VARCHAR,
    SUBSCRIBER              IN      SYS.AQ$_AGENT
);
```

参数详解

- QUEUE_NAME　指定要删除订阅者的队列名，格式为：[模式名.]队列名。
- SUBSCRIBER　指定要删除的订阅者，参考 37.1 节中数据类型的说明。

12. ALTER_SUBSCRIBER

修改订阅者属性。该方法目前仅做语法兼容，暂无实际效果。

语法如下：

```
PROCEDURE ALTER_SUBSCRIBER (
    QUEUE_NAME              IN      VARCHAR,
    SUBSCRIBER              IN      SYS.AQ$_AGENT,
    RULE                    IN      VARCHAR,
```

```
    TRANSFORMATION          IN          VARCHAR
);
```

参数详解

- QUEUE_NAME　指定要修改的队列名，格式为：[模式名.]队列名。
- SUBSCRIBER　指定要修改的订阅者，参考 36.1 节中数据类型的说明。
- RULE　定义消息、PL/SQL 等内容的布尔表达式。
- TRANSFORMATION 定义订阅者出队消息的转换形式。

36.3 相关视图

1． SYS.ALL_INT_DEQUEUE_QUEUES 视图

显示当前用户有权访问的所有出队队列。

序号	列	数据类型	说明
1	OWNER	VARCHAR(128)	队列所属模式
2	NAME	VARCHAR(128)	队列名称
3	QID	INTEGER	队列 ID

36.4 举例说明

使用包过程和函数之前，如果还未创建过系统包，请先调用系统过程创建。

```
SP_CREATE_SYSTEM_PACKAGES (1,'DBMS_AQADM');
```

例 1 创建消息载荷类型 MY_MSG_TYPE 和模式 SSS，并在其上创建队列表"@TAB"。

```
CREATE OR REPLACE TYPE MY_MSG_TYPE AS OBJECT(MESSAGE VARCHAR(4000));
/
CREATE SCHEMA SSS;
```

```
/

DBMS_AQADM.CREATE_QUEUE_TABLE(

        QUEUE_TABLE             => 'SSS."@TAB"',

        QUEUE_PAYLOAD_TYPE    => 'MY_MSG_TYPE',

         MULTIPLE_CONSUMERS    => TRUE

    );
```

例 2 创建队列 MY_QUEUE 并启动入队出队功能。

```
DBMS_AQADM.CREATE_QUEUE(

        QUEUE_NAME    => 'SSS.MY_QUEUE',

        QUEUE_TABLE => 'SSS."@TAB"'

     );

DBMS_AQADM.START_QUEUE(QUEUE_NAME    => 'SSS.MY_QUEUE');
```

例 3 为队列添加订阅者 MY_SUB。

```
DBMS_AQADM.ADD_SUBSCRIBER(

        QUEUE_NAME => 'SSS.MY_QUEUE',

        SUBSCRIBER => SYS.AQ$_AGENT('MY_SUB', NULL, NULL)

    );
```

例 4 清理环境，包括禁用、删除队列，删除队列表。

```
DBMS_AQADM.STOP_QUEUE(QUEUE_NAME => 'SSS.MY_QUEUE');

DBMS_AQADM.DROP_QUEUE(QUEUE_NAME => 'SSS.MY_QUEUE');

DBMS_AQADM.DROP_QUEUE_TABLE(QUEUE_TABLE => 'SSS."@TAB"');
```

37 DBMS_AQ 包

DBMS_AQ 包提供高级队列的以下常用功能：消息入队出队，注册、注销通知等。

DBMS_AQ 包的使用依赖 DBMS_AQADM。在创建 DBMS_AQ 包时，系统会自动创建 DBMS_AQADM 包。

37.1 数据类型

1. SYS.AQ$_RECIPIENT_LIST_T

系统对象类型，用于描述订阅者信息组。

语法如下:

```
TYPE SYS.AQ$_RECIPIENT_LIST_T IS TABLE OF SYS.AQ$_AGENT INDEX BY INTEGER;
```

2. SYS.AQ$_REG_INFO

系统对象类型，用于描述注册信息。

语法如下:

```
TYPE SYS.AQ$_REG_INFO AS OBJECT(
    NAME                VARCHAR(128),
    NAMESPACE           NUMBER,
    CALLBACK            VARCHAR(256),
    CTX                 VARBINARY(2000),
    ANYCTX              "<ADT_1>",
    CTXTYPE             NUMBER,
```

```
    QOSFLAGS            NUMBER,
    PAYLOADCBK          VARCHAR(4000),
    TIMEOUT             NUMBER
);
```

参数详解

- NAME 指定注册信息，格式为：[模式名.]队列名:订阅者。非正规表示符要用""括起来，如 SYSDBA."@QUE_NAME":SUB_NAME。
- NAMESPACE 指定订阅者命名空间，取值包括 NAMESPACE_ANONYMOUS 或 NAMESPACE_AQ。仅做语法支持。
- CALLBACK 指定回调过程名，格式为：PLSQL://过程名。
- 剩余参数仅做语法支持。

3. SYS.AQ$_REG_INFO_LIST

系统对象类型，用于描述注册信息组。

语法如下:

```
TYPE SYS.AQ$_REG_INFO_LIST AS VARRAY(1024) OF SYS.AQ$_REG_INFO;
```

4. SYS.AQ$_DESCRIPTOR

系统对象类型，用于描述回调过程参数。

语法如下:

```
TYPE SYS.AQ$_DESCRIPTOR AS OBJEC (
    QUEUE_NAME          VARCHAR(65),
    CONSUMER_NAME       VARCHAR(30),
```

```
    MSG_ID                  VARBINARY(16)
);
```

参数详解

- QUEUE_NAME　指定触发回调的队列名，格式为：模式名.队列名。
- CONSUMER_NAME　指定触发回调的订阅者名。
- MSG_ID　为触发回调的消息 ID。

5. DBMS_AQ. ENQUEUE_OPTIONS_T

DBMS_AQ 包专有类型，指定入队选项。

语法如下:

```
TYPE ENQUEUE_OPTIONS_T IS RECORD (
    VISIBILITY              INTEGER         DEFAULT ON_COMMIT,
    RELATIVE_MSGID          VARBINARY(16)   DEFAULT NULL,
    SEQUENCE_DEVIATION      INTEGER         DEFAULT NULL
);
```

参数详解

- 参数仅做语法支持。

6. DBMS_AQ.DEQUEUE_OPTIONS_T

DBMS_AQ 包专有类型，指定出队选项。

语法如下:

```
TYPE DEQUEUE_OPTIONS_T IS RECORD (
    CONSUMER_NAME           VARCHAR(30)     DEFAULT NULL,
```

```
    DEQUEUE_MODE            INTEGER             DEFAULT REMOVE,
    NAVIGATION              INTEGER             DEFAULT NEXT_MESSAGE,
    VISIBILITY              INTEGER             DEFAULT ON_COMMIT,
    WAIT                    INTEGER             DEFAULT FOREVER,
    MSGID                   VARBINARY(16)       DEFAULT NULL
);
```

参数详解

- DEQUEUE_MODE　指定出队模式，取值包括：BROWSE（直接读取消息）、REMOVE（读取后删除消息）。
- WAIT　指定没有消息时的等待时间（秒），值 FOREVER 和 NO_WAIT 分别表示一直等待和不等待。
- 剩余参数仅做语法支持。

7. DBMS_AQ. MESSAGE_PROPERTIES_T

DBMS_AQ 包专有类型，指定消息属性。

语法如下:

```
TYPE MESSAGE_PROPERTIES_T IS RECORD (
    PRIORITY            INTEGER     NOT NULL    DEFAULT 1,
    DELAY               INTEGER     NOT NULL    DEFAULT 0,
    EXPIRATION          INTEGER     NOT NULL    DEFAULT -1,
    CORRELATION         VARCHAR(128)            DEAFULT NULL,
    ATTEMPTS            INTEGER,
    RECIPIENT_LIST      SYS.AQ$_RECIPIENT_LIST_T,
```

```
    EXCEPTION_QUEUE      VARCHAR(61)              DEFAULT NULL,
    ENQUEUE_TIME         DATE,
    STATE                INTEGER
);
```

参数详解

- CORRELATION：消息的标识关联符。
- ENQUEUE_TIME：消息的入队时间。
- 剩余参数仅做语法支持。

8. SYS.AQ$_SIG_PROP

用于标识或配置与队列消息相关的信号或属性。

语法如下:

```
TYPE SYS.AQ$_SIG_PROP AS OBJECT (
    SIGNATURE    RAW(2000),
    CANALGO      VARCHAR2(2000),
    DIGALGO      VARCHAR2(2000),
    SIGALGO      VARCHAR2(2000),
    CERTIFICATE  VARCHAR2(2000),
    DIGVAL       RAW(2000)
);
```

参数详解

- 参数仅做语法支持。

37.2 相关方法

1. REGISTER

注册订阅者和回调信息。

语法如下:

```
PROCEDURE REGISTER(
    REG_LIST        IN          SYS.AQ$_REG_INFO_LIST
    REG_COUNT       IN          NUMBER
);
```

参数详解

- REG_LIST 指定注册信息数组。参考 38.2 数据类型。
- REG_COUNT 指定 REG_LIST 中的注册信息个数。

2. UNREGISTER

注销订阅者和回调信息。

语法如下:

```
PROCEDURE UNREGISTER(
    REG_LIST        IN          SYS.AQ$_REG_INFO_LIST
    REG_COUNT       IN          NUMBER
);
```

参数详解

- REG_LIST 指定注销信息数组。参考 38.2 数据类型。
- REG_COUNT 指定 REG_LIST 中的注销信息个数。

3. ENQUEUE

消息入队，若队列注册过订阅者和回调信息，则入队消息后会尝试为每个订阅者执行其注册的回调过程。

语法如下:

```
PROCEDURE ENQUEUE(
    QUEUE_NAME              IN      VARCHAR,
    ENQUEUE_OPTIONS         IN      ENQUEUE_OPTIONS_T,
    MESSAGE_PROPERTIES      IN      MESSAGE_PROPERTIES_T,
    PAYLOAD                 IN      "<ADT_1>",
    MSGID                   OUT         VARBINARY
);
```

参数详解

- QUEUE_NAME 指定要入队的队列名，格式为：[模式名.]队列名。
- ENQUEUE_OPTIONS 指定入队选项，仅做语法支持。参考 38.2 数据类型。
- MESSAGE_PROPERTIES 指定消息属性，仅做语法支持。参考 38.2 数据类型。
- PAYLOAD 指定消息载荷，必须与创建队列表指定的消息载荷类型对应。
- MSGID 输出生成的入队消息 ID。

4. DEQUEUE

消息出队，队列中存在多条消息时总是优先出队第一条。

语法如下:

```
PROCEDURE DEQUEUE(
```

```
    QUEUE_NAME              IN      VARCHAR,
    DEQUEUE_OPTIONS         IN      DEQUEUE_OPTIONS_T,
    MESSAGE_PROPERTIES      OUT     MESSAGE_PROPERTIES_T,
    PAYLOAD                 OUT         "<ADT_1>",
    MSGID                   OUT         VARBINARY
);
```

参数详解

- QUEUE_NAME 指定要出队的队列名，格式为：[模式名.]队列名。
- DEQUEUE_OPTIONS 指定出队选项。参考 38.2 数据类型。
- MESSAGE_PROPERTIES 指定消息属性，仅做语法支持。参考 38.2 数据类型。
- PAYLOAD 输出消息载荷，必须与创建队列表指定的消息载荷类型对应。
- MSGID 输出消息 ID。

37.3 回调说明

每个订阅者可注册多个回调过程，但消息入队时最多仅触发一个回调。触发的回调过程必须有效，与队列处于同一模式，且参数数目、类型和顺序必须满足如下定义，不符合预期的回调会跳过执行：

```
PROCEDURE CALLBACK(
    CONTEXT1            RAW,
    REGINFO             SYS.AQ$_REG_INFO,
    DESCR               SYS.AQ$_DESCRIPTOR,
    PAYLOAD             RAW,
    PAYLOADL            NUMBER
);
```

参数详解

- REGINFO 指定触发回调的注册信息，同过程 REGISTER 参数。参考 38.2 数据类型。
- DESCR 指定触发回调的消息和队列信息。参考 38.2 数据类型。
- 剩余参数仅做语法支持。

37.4 举例说明

使用包过程和函数之前，如果还未创建过系统包，请先调用系统过程创建。

```
SP_CREATE_SYSTEM_PACKAGES (1,'DBMS_AQ');

SET SERVEROUTPUT ON;  //PRINT 需要设置这条语句，才能打印出消息
```

例 1 在 37.3 节例 3 之后执行以下操作：创建回调过程 MY_CALLBACK 并注册到订阅者 MY_SUB。

```
CREATE OR REPLACE PROCEDURE SSS.MY_CALLBACK(
    CONTEXT1      RAW,
    REGINFO       SYS.AQ$_REG_INFO,
    DESCR         SYS.AQ$_DESCRIPTOR,
    PAYLOAD       RAW,
    PAYLOADL      NUMBER
) AS
    DOP DBMS_AQ.DEQUEUE_OPTIONS_T;
     MSG SYSDBA.MY_MSG_TYPE;
     MSGID RAW(16);
BEGIN
    DOP.DEQUEUE_MODE := DBMS_AQ.BROWSE;
    DBMS_AQ.DEQUEUE(
```

```
        QUEUE_NAME              => DESCR.QUEUE_NAME,
        DEQUEUE_OPTIONS         => DOP,
        MESSAGE_PROPERTIES      => NULL,
        PAYLOAD                 => MSG,
        MSGID                   => MSGID
    );
    PRINT MSG.MESSAGE;
END;
/
DBMS_AQ.REGISTER(
    SYS.AQ$_REG_INFO_LIST(
        SYS.AQ$_REG_INFO(
            'SSS.MY_QUEUE:MY_SUB',
            DBMS_AQ.NAMESPACE_AQ,
            'PLSQL://MY_CALLBACK',
            HEXTORAW('FF')
        )
    ),
    1);
```

例 2 消息入队，入队后会触发回调 MY_CALLBACK，从而打印入队的消息载荷。

```
DECLARE
    EOP DBMS_AQ.ENQUEUE_OPTIONS_T;
    MSG MY_MSG_TYPE;
    MSGID RAW(16);
```

```
BEGIN
    MSG := MY_MSG_TYPE(TO_CHAR(SYSTIMESTAMP, 'DD-MON-YYYY HH24:MI:SS.FF3'));
    DBMS_AQ.ENQUEUE(
        QUEUE_NAME          => 'SSS.MY_QUEUE',
        ENQUEUE_OPTIONS     => EOP,
        MESSAGE_PROPERTIES  => NULL,
        PAYLOAD             => MSG,
        MSGID               => MSGID
    );
    COMMIT;
END;
/
```

例 3 消息以 REMOVE 模式出队，出队后消息会被删除。

```
DECLARE
    DOP DBMS_AQ.DEQUEUE_OPTIONS_T;
     MSG MY_MSG_TYPE;
     MSGID RAW(16);
BEGIN
    DBMS_AQ.DEQUEUE(
        QUEUE_NAME          => 'SSS.MY_QUEUE',
        DEQUEUE_OPTIONS     => DOP,
        MESSAGE_PROPERTIES  => NULL,
        PAYLOAD             => MSG,
        MSGID               => MSGID
```

```
    );

    PRINT MSG.MESSAGE;

END;

/
```

38 DBMS_SQLTUNE 包

DBMS_SQLTUNE 包提供一系列对实时 SQL 监控的方法。当 SQL 监控功能开启后，DBMS_SQLTUNE 包可以实时监控 SQL 执行过程中的信息，包括：执行时间、执行代价、执行用户、统计信息等情况。

SQL 监控功能开启的方法是将 DM.INI 参数 ENABLE_MONITOR 和 MONITOR_SQL_EXEC 均设置为 1。MPP 环境下，当前节点执行的 SQL，只能在当前节点查询监控报告，其他节点无法查询。

值得注意的是，由于包内相关方法中存在 SQL 文本、调优任务名称等的字符串比较和匹配等操作，所以建库参数 BLANK_PAD_MODE 和 dm.ini 参数 SPACE_COMPARE_MODE 会影响字符串尾部空格的处理策略，会对相关包方法的执行结果产生影响。例如，如果设置 BLANK_PAD_MODE=0 且 SPACE_COMPARE_MODE=0 忽略字符串尾部空格差异，则调优任务名'TASK1'和'TASK1 '(尾部空格数不同)会被认为是相同的任务名。

38.1 相关方法

1. REPORT_SQL_MONITOR

返回单个 SQL 的实时监控报告。

语法如下:

```
FUNCTION REPORT_SQL_MONITOR(
    SQL_ID                  IN      VARCHAR2    DEFAULT NULL,
    DBOP_NAME               IN      VARCHAR2    DEFAULT NULL,
    DBOP_EXEC_ID            IN      INT         DEFAULT NULL,
    SESSION_ID              IN      BIGINT      DEFAULT NULL,
    SESSION_SERIAL          IN      INT         DEFAULT NULL,
```

```
    SQL_EXEC_START          IN      DATETIME DEFAULT  NULL,
    SQL_EXEC_ID             IN      BIGINT           DEFAULT  NULL,
    INST_ID                 IN      INT              DEFAULT  NULL,
    START_TIME_FILTER       IN      DATETIME         DEFAULT  NULL,
    END_TIME_FILTER         IN      DATETIME         DEFAULT  NULL,
    INSTANCE_ID_FILTER      IN      INT              DEFAULT  NULL,
    PARALLEL_FILTER         IN      VARCHAR2         DEFAULT  NULL,
    PLAN_LINE_FILTER        IN      INT              DEFAULT  NULL,
    EVENT_DETAIL            IN      VARCHAR2         DEFAULT  'YES',
    BUCKET_MAX_COUNT        IN      INT              DEFAULT  128,
    BUCKET_INTERVAL         IN      INT              DEFAULT  NULL,
    BASE_PATH               IN      VARCHAR2         DEFAULT  NULL,
    LAST_REFRESH_TIME       IN      DATETIME         DEFAULT  NULL,
    REPORT_LEVEL            IN      VARCHAR2         DEFAULT  'TYPICAL',
    TYPE                    IN      VARCHAR2         DEFAULT  'TEXT',
    SQL_PLAN_HASH_VALUE     IN      INT              DEFAULT  NULL,
    CON_NAME                IN      VARCHAR2         DEFAULT  NULL,
    REPORT_ID               IN      INT              DEFAULT  NULL
)RETURN CLOB;
```

参数详解

- SQL_ID

表示展示 SQL_ID 对应语句的监控报告。SQL_ID 表示 SQL 语句编号，全局递增。相同 SQL 语句对应 SQL_ID 一般相同。

当 SQL_ID 为 NULL 时，表示返回最后一条被监控 SQL 的报告。

- SESSION_ID

展示 session_id 指定会话的监控报告，默认为 NULL。session_id 表示 session 地址信息，session 地址会重用，导致 session_id 相同。

session_id 为-1 表示当前会话。

- SESSION_SERIAL

会话的唯一标识。全局递增的，不会重复。

- SQL_EXEC_ID

表示展示 sql_exec_id 执行号对应语句的监控报告。sql_exec_id 为 SQL 执行编号，全局递增。相同 SQL 语句多次执行，对应 sql_exec_id 也各不相同。

- TYPE

监控报告格式。目前只支持文本"TEXT"格式，其它报错。

- 其它参数

保留参数，暂时忽略，不进行合法性检查。

2. REPORT_SQL_MONITOR_LIST

返回被监控 SQL 的概要信息列表。

语法如下:

```
FUNCTION REPORT_SQL_MONITOR_LIST(
    SQL_ID                  IN      VARCHAR2    DEFAULT NULL,
    SESSION_ID              IN      BIGINT      DEFAULT NULL,
    SESSION_SERIAL          IN      INT         DEFAULT NULL,
    INST_ID                 IN      INT         DEFAULT NULL,
    ACTIVE_SINCE_DATE       IN      DATETIME    DEFAULT NULL,
    ACTIVE_SINCE_SEC        IN      INT         DEFAULT NULL,
```

```
    ACTIVE_BEFORE_DATE      IN      DATETIME DEFAULT    NULL,
    LAST_REFRESH_TIME       IN      DATETIME    DEFAULT    NULL,
    DBOP_NAME               IN      VARCHAR2    DEFAULT    NULL,
    MONITOR_TYPE            IN      INT         DEFAULT    MONITOR_TYPE_ALL,
    MAX_SQLTEXT_LENGTH      IN      INT         DEFAULT    NULL,
    TOP_N_COUNT             IN      INT         DEFAULT    NULL,
    TOP_N_RANKBY            IN      VARCHAR2    DEFAULT    'LAST_ACTIVE_TIME',
    REPORT_LEVEL            IN      VARCHAR2    DEFAULT    'TYPICAL',
    AUTO_REFRESH            IN      INT         DEFAULT    NULL,
    BASE_PATH               IN      VARCHAR2    DEFAULT    NULL,
    TYPE                    IN      VARCHAR2    DEFAULT    'TEXT',
    CON_NAME                IN      VARCHAR2    DEFAULT    NULL,
    TOP_N_DETAIL_COUNT      IN      INT         DEFAULT    NULL
)RETURN CLOB;
```

参数详解

- SQL_ID

表示展示 sql_id 对应语句的监控报告。当 sql_id 为 NULL 时，表示返回所有被监控 SQL 的报告。

- SESSION_ID

展示 session_id 指定会话的监控报告，默认为 NULL。session_id 为-1 表示当前会话。

- SESSION_SERIAL

会话的唯一标识。

- MAX_SQLTEXT_LENGTH

SQL 文本的最大长度，默认为 NULL 表示没限制。SQL 文本长度是字符长度，而不是字

节长度。

- TYPE

  监控报告格式。目前只支持文本“TEXT”格式，其它报错。

- 其它参数

  暂时忽略，不进行合法性检查。

3. CREATE_TUNING_TASK

   创建语句调优任务。

   语法如下:

```
FUNCTION CREATE_TUNING_TASK(
    SQL_TEXT            IN      CLOB,
    BIND_LIST           IN      VARCHAR(128)    DEFAULT NULL,
    USER_NAME           IN      VARCHAR(128)    DEFAULT
SYS_CONTEXT('USERENV', 'CURRENT_USER'),
    SCOPE               IN      VARCHAR     DEFAULT 'COMPREHENSIVE',
    TIME_LIMIT          IN      INT         DEFAULT 1800,
    TASK_NAME           IN      VARCHAR(128)    DEFAULT NULL,
    DESCRIPTION         IN      VARCHAR(256)    DEFAULT NULL,
    CON_NAME            IN      VARCHAR(128)    DEFAULT NULL,
    DATABASE_LINK_TO    IN      VARCHAR(128)    DEFAULT NULL
)RETURN VARCHAR(128);
```

   参数详解

- SQL_TEXT

  调优语句。

- BIND_LIST

  调优语句的绑定参数部分。目前仅为保留参数，无实际作用，只能传入 NULL。

- USER_NAME

  调优任务所属用户的用户名称。

- SCOPE

  调优等级，可取值为 COMPREHENSIVE 或 LIMITED，仅语法支持。

- TIME_LIMIT

  调优任务的最大执行时间，超时则会中断调优任务。默认值为 1800，单位为秒，当设置为 0 时，表示不限制调优任务执行时间。当指定为 NULL 值或空值时，取默认值 1800。

- TASK_NAME

  调优任务名称，值为 NULL 时由系统分配名称。

- DESCRIPTION

  调优任务相关描述。

- CON_NAME

  连接名。目前仅为保留参数，无实际作用，只能传入 NULL。

- DATABASE_LINK_TO

  数据库链接。目前仅为保留参数，无实际作用，只能传入 NULL。

4. EXECUTE_TUNING_TASK

   执行语句调优任务。

   语法如下:

```
FUNCTION EXECUTE_TUNING_TASK(
    TASK_NAME           IN      VARCHAR(128),
    EXECUTION_NAME      IN      VARCHAR(128)    DEFAULT NULL,
```

```
    EXECUTION_PARAMS        IN      VARCHAR(128)    DEFAULT  NULL,
    EXECUTION_DESC          IN      VARCHAR(256)    DEFAULT  NULL,
    DATABASE_LINK_TO        IN      VARCHAR(128)    DEFAULT  NULL
)RETURN VARCHAR;

PROCEDURE EXECUTE_TUNING_TASK(
    TASK_NAME               IN      VARCHAR(128),
    EXECUTION_NAME          IN      VARCHAR(128)    DEFAULT  NULL,
    EXECUTION_PARAMS        IN      VARCHAR(128)    DEFAULT  NULL,
    EXECUTION_DESC          IN      VARCHAR(256)    DEFAULT  NULL,
    DATABASE_LINK_TO        IN      VARCHAR(128)    DEFAULT  NULL
);
```

参数详解

- TASK_NAME

  调优任务的名称。

- EXECUTION_NAME

  调优任务的执行名称，值为 NULL 则由系统分配名称。

- EXECUTION_PARAMS

  调优任务的执行参数。目前仅为保留参数，无实际作用。

- EXECUTION_DESC

  执行描述信息。

- DATABASE_LINK_TO

  数据库链接。目前仅为保留参数，无实际作用，只能传入 NULL。

5. REPORT_TUNING_TASK

输出语句调优报告。

语法如下:

```
FUNCTION REPORT_TUNING_TASK(
    TASK_NAME               IN      VARCHAR(128),
    TYPE                    IN      VARCHAR(128)    DEFAULT  'TEXT',
    LEVEL                   IN      VARCHAR(128)    DEFAULT  'TYPICAL',
    "SECTION"               IN      VARCHAR(128)    DEFAULT  'ALL',
    OBJECT_ID               IN      INT         DEFAULT  NULL,
    RESULT_LIMIT            IN      INT         DEFAULT  NULL,
    OWNER_NAME              IN      VARCHAR(128)    DEFAULT
SYS_CONTEXT('USERENV', 'CURRENT_USER'),
    EXECUTION_NAME          IN      VARCHAR(128)    DEFAULT  NULL,
    DATABASE_LINK_TO        IN      VARCHAR(128)    DEFAULT  NULL
)RETURN CLOB;
```

参数详解

- TASK_NAME

  调优任务的名称。

- TYPE

  报告输出的格式，仅支持“TEXT”格式。

- LEVEL

  报告的等级，可取值为 BASIC、TYPICAL、ALL。BASIC：基本等级；TYPICAL：常规等级；ALL：详细等级。

- "SECTION"

报告的内容，可取值为 SUMMARY、FINDINGS、PLAN、INFORMATION、ERROR、ALL。SUMMARY: 概要; FINDINGS: 优化点; PLAN: 优化计划; INFORMATION: 综合信息; ERROR: 语法错误; ALL: 全部信息。

- OBJECT_ID

  对象 ID，仅兼容支持，无实际意义。仅支持指定为 NULL 或空串。。

- RESULT_LIMIT

  结果集行数限制。目前仅为保留参数，无实际作用。

- OWNER_NAME

  调优任务的所属用户名称。

- EXECUTION_NAME

  调优任务的执行名称。

- DATABASE_LINK_TO

  数据库链接。目前仅为保留参数，无实际作用，只能传入 NULL。

6. DROP_TUNING_TASK

   删除语句调优任务。

   语法如下:

```
PROCEDURE DROP_TUNING_TASK(
    TASK_NAME               IN          VARCHAR      DEFAULT   NULL
);
```

   参数详解

- TASK_NAME

  待删除的调优任务名称。

38.2 举例说明

DBMS_SQLTUNE 包会根据系统参数的不同给出相应的调优建议，因此不同系统中下面例子的运行结果可能会不同。

使用包内的过程和函数之前，如果还未创建过系统包，请先调用系统过程创建系统包。

```
SP_CREATE_SYSTEM_PACKAGES (1,'DBMS_SQLTUNE');
```

例1 监控 SQL 语句。

一 开启 SQL 监控开关。

设置 DM.INI 参数 ENABLE_MONITOR 和 MONITOR_SQL_EXEC 为 1。

```
SP_SET_PARA_VALUE (1,'ENABLE_MONITOR',1);

SP_SET_PARA_VALUE (1,'MONITOR_SQL_EXEC',1);
```

二 执行 SQL 语句。

```
SELECT  * FROM DBA_OBJECTS;

……

已用时间: 146.907(毫秒). 执行号:11

SELECT * FROM DBA_TABLES;

……

已用时间: 544.633(毫秒). 执行号:24.
```

得到两条 SQL 语句的执行号 SQL_EXEC_ID 分别为 11 和 24。

三 查询 SQL 监控报告。

为了能够完整展示 SQL 监控报告，特使用 SET 命令将将 DIsql 环境变量 LONG 设置成一个较大值 999999。

```
SET LONG 999999
```

根据 SQL 执行号来查询监控报告。

```
SELECT DBMS_SQLTUNE.REPORT_SQL_MONITOR(SQL_EXEC_ID=>11) FROM DUAL;

或

SELECT DBMS_SQLTUNE.REPORT_SQL_MONITOR(SQL_EXEC_ID=>24) FROM DUAL;
```

四 查询 SQL 监控报告链表。

```
SELECT DBMS_SQLTUNE.REPORT_SQL_MONITOR_LIST() FROM DUAL;
```

例2 SQL 语句调优。

准备数据表。

```
CREATE TABLE TEST(C1 INT);

INSERT INTO TEST VALUES(1);

INSERT INTO TEST VALUES(2);

INSERT INTO TEST VALUES(3);
```

一 创建语句调优任务。

```
DBMS_SQLTUNE.CREATE_TUNING_TASK('SELECT C1 FROM TEST', TASK_NAME=>'TASK1');
```

二 执行语句调优任务。

```
DBMS_SQLTUNE.EXECUTE_TUNING_TASK('TASK1');
```

三 输出语句调优报告。

```
//首先将环境变量 LONG 设置成一个较大值 999999，以保证完整显示调优报告

SET LONG 999999

SELECT DBMS_SQLTUNE.REPORT_TUNING_TASK('TASK1');
```

查询结果如下:

```
行号        DBMS_SQLTUNE.REPORT_TUNING_TASK('TASK1')

---------- ------------------------------------------------------------

1              GENERAL INFORMATION SECTION

-------------------------------------------------------------------------
```

```
TUNING TASK NAME       : TASK1

TUNING TASK OWNER      : SYSDBA

WORKLOAD TYPE          : SINGLE SQL STATEMENT

EXECUTION COUNT        : 1

CURRENT EXECUTION      : SYSEXECNAME_1

EXECUTION TYPE         : TUNE SQL

SCOPE                  : COMPREHENSIVE

TIME LIMIT(SECONDS) : 1800

COMPLETION STATUS      : COMPLETED

STARTED AT             : 2021-06-30 14:45:38

COMPLETED AT           : 2021-06-30 14:45:38

SQL TEXT               : SELECT C1 FROM TEST



-------------------------------------------------------------------------------



FINDING SECTION (2 FINDINGS)

-------------------------------------------------------------------------------

  1 - STATISTICS FINDING

  ----------------------

     尚未分析表 SYSDBA.TEST。

     RECOMMENDATION

     --------------

     考虑收集此表的优化程序统计信息。

     CALL DBMS_STATS.GATHER_TABLE_STATS('SYSDBA', 'TEST');
```

```
2 - INDEX FINDING

-----------------

  通过创建一个或多个索引可以改进此语句的执行计划。

  RECOMMENDATION

  --------------

  考虑运行创建推荐的索引。

  CREATE INDEX IDX$$_1 ON "SYSDBA"."TEST"("C1");
```

四 删除语句调优任务。

```
DBMS_SQLTUNE.DROP_TUNING_TASK('TASK1');
```

39 DBMS_CRYPTO 包

DBMS_CRYPTO 包提供加密和解密数据的接口，支持几种工业标准的分组加密、流加密算法和散列算法。分组加密算法包括 ADVANCED ENCRYPTION STANDARD(AES)、DATA ENCRYPTION STANDARD(DES)；流加密算法包括 RC4；散列算法包括 MD5、SHA-1、SHA-256、SHA-384、SHA-512。

39.1 相关方法

DBMS_CRYPTO 包中包含的过程和函数如下详细介绍：

1. HASH

对 VARBINARY、BLOB、CLOB 数据进行散列。

语法如下：

```
FUNCTION HASH(
    SRC             IN      VARBINARY/BLOB/CLOB,
    TYPE            IN      INT
) RETURN VARBINARY;
```

参数详解

- SRC 输入参数，需要散列的数据。数据类型可以为 VARBINARY、BLOB 或 CLOB。如果 SRC 为空，则返回值和输出参数均为空。
- TYPE 输入参数，使用的算法，可选值 HASH_MD5、HASH_SH1、HASH_SH256、HASH_SH384、HASH_SH512。HASH_MD5 表示 MD5 信息摘要算法，HASH_SH1 表示安全散列算法 1（SHA-1），HASH_SH256 表示安全散列算法 2（SHA-256），HASH_SH384 表示安全散列算法 2（SHA-384），HASH_SH512 表示安全散列算法 2

（SHA-512）。

返回值

散列后的 VARBINARY 数据。

2. ENCRYPT

对 VARBINARY 数据进行加密。

语法如下：

```
FUNCTION ENCRYPT(
    SRC             IN      VARBINARY,
    TYP             IN      PLS_INTEGER,
    KEY             IN      VARBINARY,
    IV              IN      VARBINARY DEFAULT NULL
) RETURN VARBINARY;
```

参数详解

- SRC 输入参数，需要加密的数据。如果 SRC 为空，则返回值和输出参数均为空。
- TYP 输入参数，使用的算法，不能为空。支持分组加密算法和流加密算法。

  1) 使用分组加密算法时必须同时指定加密算法、工作模式、填充模式。书写规则为：加密算法 + 工作模式 + 填充模式。

  - 分组加密算法：ENCRYPT_DES、ENCRYPT_3DES、ENCRYPT_3DES_2KEY、ENCRYPT_AES128、ENCRYPT_AES192、ENCRYPT_AES256。
  - 分组加密算法的工作模式：CHAIN_ECB、CHAIN_CBC、CHAIN_CFB、CHAIN_OFB。
  - 分组加密算法的填充模式：填充 PAD_PKCS5、不填充 PAD_NONE。是否填充由算法决定。以_NOPAD 结尾的非填充算法名须选 PAD_NONE 模式，其余填充算法均选 PAD_PKCS5 模式。

2）使用流加密算法，仅指定算法名即可。流加密算法：ENCRYPT_RC4。

- KEY 输入参数，加密使用的 KEY，不能为空。KEY 长度必须足够，大于等于算法所需密钥长度，超出长度部分忽略，长度不足将会报错处理。算法的 KEY 值可通过 V$CIPHERS 视图 KH_SIZE 列获取。需要注意的是，对于 ENCRYPT_RC4 算法而言，由于其是流式加密算法，密钥长度可变，不存在长度不足的情况，故其 KH_SIZE 作为默认值，仅供参考。
- IV 输入参数，加密使用的初始向量，缺省为空。IV 值的大小为所使用加密算法的分组大小，可通过 V$CIPHERS 视图 BLOCK_SIZE 列获取。IV 长度不足时，将报错处理；IV 长度大于等于 BLOCK_SIZE 时，均取值 BLOCK_SIZE。工作模式为 CHAIN_ECB 的加密算法不会用到 IV 参数，因此不对 IV 参数值进行校验。

返回值

加密后的 VARBINARY 数据

3. ENCRYPT

对 BLOB、CLOB 数据进行加密。

语法如下：

```
PROCEDURE ENCRYPT(
    DST             IN OUT NOCOPY   BLOB,
    SRC             IN              BLOB/CLOB,
    TYP             IN              PLS_INTEGER,
    KEY             IN              VARBINARY,
    IV              IN              VARBINARY DEFAULT NULL
);
```

参数详解

- DST 输入输出参数，加密后的数据。

- SRC 输入参数，需要加密的数据。如果 SRC 为空，则返回值和输出参数均为空。
- TYP 输入参数，使用的算法，不能为空。用法和 ENCRYPT 方法的 TYP 一样。
- KEY 输入参数，加密使用的 KEY，不能为空。用法和 ENCRYPT 方法的 KEY 一样。
- IV 输入参数，加密使用的 IV，默认为空。用法和 ENCRYPT 方法的 IV 一样

返回值

无

4. DECRYPT

对 VARBINARY 数据进行解密。

语法如下:

```
FUNCTION DECRYPT(
    SRC                 IN      VARBINARY,
    TYP                 IN      PLS_INTEGER,
    KEY                 IN      VARBINARY,
    IV                  IN      VARBINARY DEFAULT NULL
) RETURN VARBINARY;
```

参数详解

- SRC 输入参数，需要解密的数据。如果 SRC 为空，则返回值和输出参数均为空。
- TYP 输入参数，使用算法，不能为空。
- KEY 输入参数，解密使用的 KEY，不能为空。
- IV 输入参数，解密使用的 IV，默认为空。

返回值

解密后的 VARBINARY 数据。

5. DECRYPT

对 BLOB、CLOB 数据进行解密。

语法如下:

```
PROCEDURE DECRYPT(
    DST                 IN OUT NOCOPY   BLOB/CLOB,
    SRC                     IN              BLOB,
    TYP                 IN              PLS_INTEGER,
    KEY                 IN              VARBINARY,
    IV                  IN              VARBINARY DEFAULT NULL
);
```

参数详解

- DST 输入输出参数，解密后的数据。
- SRC 输入参数，需要解密的数据。如果 SRC 为空，则返回值和输出参数均为空。
- TYP 输入参数，使用算法，不能为空。
- KEY 输入参数，解密使用的 KEY，不能为空。
- IV 输入参数，解密使用的 IV，默认为空。

返回值

无

39.2 使用说明

使用 DBMS_CRYPTO 包时，指定的加密算法须存在于数据库中。可以通过查询系统视图 V$CIPHERS 获得。例如：算法名、KEY、工作模式、填充模式、分组长度等。

39.3 举例说明

使用包内的过程和函数之前，如果还未创建过系统包。请先调用系统过程创建系统包。

```
SP_CREATE_SYSTEM_PACKAGES (1,'DBMS_CRYPTO');
```

```
SP_CREATE_SYSTEM_PACKAGES (1,'DBMS_OUTPUT');

SP_CREATE_SYSTEM_PACKAGES(1,'UTL_I18N');

SET SERVEROUTPUT ON;  //DBMS_OUTPUT.PUT_LINE 需要设置这条语句，才能打印出消息
```

例 1 分别使用 MD5 散列算法、SHA-1 算法对 VARBINARY 类型数据进行散列。

```
DECLARE
    RAW_INPUT VARBINARY(128) := '74696765727469676572746967657274';
    MD5_RAW VARBINARY(2048);
    SHA1_RAW VARBINARY(2048);
BEGIN
    DBMS_OUTPUT.PUT_LINE('> ========== BEGIN TEST RAW DATA ==========');
    DBMS_OUTPUT.PUT_LINE('> RAW INPUT          : ' ||                RAW_INPUT);
    MD5_RAW := DBMS_CRYPTO.HASH(RAW_INPUT, DBMS_CRYPTO.HASH_MD5);
    DBMS_OUTPUT.PUT_LINE('> MD5 HASH           : ' ||                  MD5_RAW);
    SHA1_RAW := DBMS_CRYPTO.HASH(RAW_INPUT, DBMS_CRYPTO.HASH_SH1);
    DBMS_OUTPUT.PUT_LINE('> SHA1 HASH          : ' ||                SHA1_RAW);
END;
/
```

结果如下:

```
> ========== BEGIN TEST RAW DATA ==========
> RAW INPUT                    : 74696765727469676572746967657274
> MD5 HASH                     : 831F2AA79221A0F8F330E43A76B53323
> SHA1 HASH                    : 7E48985200C8255A8F7BFF840F9D1F2397A6C65A
```

例 2 使用 DES+ECB+PKCS5 算法对 VARBINARY 类型数据进行加解密。

```
CREATE OR REPLACE FUNCTION ENCRYPT_FUNCTION (V_STR VARCHAR2, V_KEY VARCHAR2)
```

```
    RETURN VARCHAR2
AS
    V_KEY_RAW        RAW (24);
    V_STR_RAW        RAW (2000);
    V_RETURN_STR     VARCHAR2 (2000);
    V_TYPE           PLS_INTEGER;
BEGIN
    V_KEY_RAW := UTL_I18N.STRING_TO_RAW (V_KEY, 'GBK');
    V_STR_RAW := UTL_I18N.STRING_TO_RAW (V_STR, 'GBK');
    V_TYPE :=
          DBMS_CRYPTO.ENCRYPT_DES
        + DBMS_CRYPTO.CHAIN_ECB
        + DBMS_CRYPTO.PAD_PKCS5;
    V_STR_RAW :=
        DBMS_CRYPTO.ENCRYPT (SRC => V_STR_RAW, TYP => V_TYPE, KEY => V_KEY_RAW);
    V_RETURN_STR := RAWTOHEX (V_STR_RAW);
    RETURN V_RETURN_STR;
END;
/


CREATE OR REPLACE FUNCTION DECRYPT_FUNCTION (V_STR VARCHAR2, V_KEY VARCHAR2)
    RETURN VARCHAR2
AS
    V_KEY_RAW        RAW (24);
```

```
    V_STR_RAW         RAW (2000);
    V_RETURN_STR      VARCHAR2 (2000);
    V_TYPE            PLS_INTEGER;
BEGIN
    V_KEY_RAW := UTL_I18N.STRING_TO_RAW (V_KEY, 'GBK');
    V_STR_RAW := HEXTORAW (V_STR);
    V_TYPE :=
          DBMS_CRYPTO.ENCRYPT_DES
        + DBMS_CRYPTO.CHAIN_ECB
        + DBMS_CRYPTO.PAD_PKCS5;
    V_STR_RAW :=
        DBMS_CRYPTO.DECRYPT (SRC => V_STR_RAW, TYP => V_TYPE, KEY => V_KEY_RAW);
    V_RETURN_STR := UTL_I18N.RAW_TO_CHAR (V_STR_RAW, 'GBK');
    RETURN V_RETURN_STR;
END;
/


CREATE TABLE C1
(
    A           VARCHAR2 (2000),
    SNAME       VARCHAR2 (10),
    B           INT,
    C           INT NULL,
    D           INT
```

```
);
INSERT INTO C1 (A,
                    SNAME,
                    B,
                    C,
                    D)
      VALUES (ENCRYPT_FUNCTION ('加密数据列 1', 'ASDASD_123456789'),
                'A',
                102,
                104,
                1);
SELECT DECRYPT_FUNCTION(A, 'ASDASD_123456789') FROM C1;
```

结果如下：

```
SELECT DECRYPT_FUNCTION(A, 'ASDASD_123456789') FROM C1;


行号          DECRYPT_FUNCTION(A,'ASDASD_123456789')
---------- ----------------------------------------
1             加密数据列 1
```

例 3 使用 DES+ECB+PKCS5 算法对 CLOB 类型数据进行加解密。

```
--创建存储过程 COMPUTEXORHASH，用于将 KEY 转换为固定 16 字节长度的 KEY
CREATE OR REPLACE PROCEDURE COMPUTEXORHASH(
    INPUT_STR   IN    VARCHAR2,
    RESULT   IN OUT   BINARY(16)
)
```

```
AS
    I INT DEFAULT 1;
    LEN INT;
    POS INT;
    CHAR_CODE BYTE;
    CURR_BYTE BYTE;
    NEW_BYTE BYTE;
BEGIN
    LEN := LENGTH(INPUT_STR);
    RESULT := 0X00000000000000000000000000000000;
    WHILE I <= LEN LOOP
        CHAR_CODE := ASCII(SUBSTRING(INPUT_STR, I));
        POS := ((I - 1) % 16) * 2 + 1;
        CURR_BYTE := UNHEX(SUBSTRING(RESULT, POS, 2));
        NEW_BYTE := CURR_BYTE ^ CHAR_CODE;
        RESULT := HEXTORAW(CONCAT(SUBSTRING(RESULT, 1, POS - 1),
                                  CHARTOBIN(CHAR(NEW_BYTE)),
                                  SUBSTRING(RESULT, POS + 2)));
        I := I + 1;
    END LOOP;
END;
/


--自定义加密函数，使用 DES+ECB+PKCS5 算法对 CLOB 类型数据进行加密
```

```
CREATE OR REPLACE FUNCTION ENCRYPT_DM (
    V_STR_RAW CLOB,
    V_KEY VARCHAR2
)RETURN VARBINARY
AS
    V_KEY_RAW VARBINARY;
    V_RETURN_STR BLOB;
    V_TYPE PLS_INTEGER;
BEGIN
    COMPUTEXORHASH(V_KEY, V_KEY_RAW);
    V_TYPE :=
          DBMS_CRYPTO.ENCRYPT_DES
        + DBMS_CRYPTO.CHAIN_ECB
        + DBMS_CRYPTO.PAD_PKCS5;
    CALL DBMS_CRYPTO.ENCRYPT (DST => V_RETURN_STR,
                              SRC => V_STR_RAW,
                              TYP => V_TYPE,
                              KEY => V_KEY_RAW);
    RETURN V_RETURN_STR;
END;
/

--自定义解密函数，使用 DES+ECB+PKCS5 算法对 CLOB 类型数据进行解密
CREATE OR REPLACE FUNCTION DECRYPT_DM (
```

```
    V_STR_RAW BLOB,
    V_KEY VARCHAR2
)RETURN VARCHAR2
AS
    V_KEY_RAW VARBINARY;
    V_RETURN_STR CLOB;
    V_TYPE PLS_INTEGER;
BEGIN
    COMPUTEXORHASH(V_KEY, V_KEY_RAW);
    V_TYPE :=
          DBMS_CRYPTO.ENCRYPT_DES
        + DBMS_CRYPTO.CHAIN_ECB
        + DBMS_CRYPTO.PAD_PKCS5;
    CALL DBMS_CRYPTO.DECRYPT (DST => V_RETURN_STR,
                              SRC => V_STR_RAW,
                              TYP => V_TYPE,
                              KEY => V_KEY_RAW);
    RETURN V_RETURN_STR;
END;
/
```

使用自定义函数ENCRYPT_DM加密数据：

```
SELECT ENCRYPT_DM('加密数据','AAAAAAAAAAAAAAAABBBBBBBBBBBBBBBB');


行号     ENCRYPT_DM('加密数据','AAAAAAAAAAAAAAAABBBBBBBBBBBBBBBB')
```

```
---------- ----------------------------------------------------------------

1              0XE6B158239EB3AA89B21C053F63AEF2B8
```

使用自定义函数DECRYPT_DM解密数据:

```
SELECT DECRYPT_DM(ENCRYPT_DM('加密数据','AAAAAAAAAAAAAAAABBBBBBBBBBBBBBBB'),

'AAAAAAAAAAAAAAAABBBBBBBBBBBBBBBB');



行号        DECRYPT_DM(ENCRYPT_DM('加密数据

','AAAAAAAAAAAAAAAABBBBBBBBBBBBBBBB'),'AAAAAAAAAAAAAAAABBBBBBBBBBBBBBBB')

--------------------------------------------------------------------------------

1              加密数据
```

40 DBMS_ERRLOG 包

DBMS_ERRLOG 包提供一种方法给指定表创建对应的错误日志表。

创建错误日志表的方法有两种，一是通过 DBMS_ERRLOG 包中的 CREATE_ERROR_LOG 来创建，二是通过 SQL 语法手动创建。因为使用 DBMS_ERRLOG 包创建错误日志表简单易行，建议用户使用方法一。

40.1 相关方法

给指定表创建对应的错误日志表。不支持指定表中含有列类型为 LONG、CLOB、BLOB、BFILE、自定义类型的列。也不支持指定的表中和要创建的错误日志表含有同名列。

语法如下:

```
PROCEDURE CREATE_ERROR_LOG(
    DML_TABLE_NAME          IN      VARCHAR,
    ERR_LOG_TABLE_NAME      IN      VARCHAR         DEFAULT NULL,
    ERR_LOG_TABLE_OWNER     IN      VARCHAR         DEFAULT NULL,
    ERR_LOG_TABLE_SPACE     IN      VARCHAR         DEFAULT NULL,
    SKIP_UNSUPPORTED        IN      BOOLEAN         DEFAULT FALSE
);
```

参数详解

- DML_TABLE_NAME 输入参数，错误日志记录表所基于的数据库表，支持以前缀形式指定其他用户的表。
- ERR_LOG_TABLE_NAME 输入参数，用户创建的错误日志记录表的名字，默认使用数据库表名加上前缀'ERR$ _'。错误日志记录表前 5 列的固定结构如下:

序号	列	数据类型	说明
1	DM_ERR_NUMBER$	INTEGER	错误码
2	DM_ERR_MESG$	VARCHAR(2000)	错误信息
3	DM_ERR_ROWID$	ROWID	保存出错的行号，INSERT 时为 0（0 将会使用 ROWID 格式显示）
4	DM_ERR_OPTYPE$	VARCHAR(2)	出错的 DML 类型，I 表示插入、 U 表示更新、D 表示删除
5	DM_ERR_TAG$	VARCHAR(2000)	log errors 语法(expression)设定的值，未设置时为空

- ERR_LOG_TABLE_OWNER　输入参数，错误日志记录表的拥有者，默认使用当前连接的用户。
- ERR_LOG_TABLE_SPACE　输入参数，错误日志记录表所在的表空间，需要有权限，默认使用当前用户使用的表空间。
- SKIP_UNSUPPORTED　输入参数，设置为 TRUE 时，列类型不支持的将会被跳过并且不会添加到错误日志记录表，设置为 FALSE 时，一个不支持的列类型将会引发过程中断，默认为 FALSE。若取值为 NULL，实际执行时会调整为 TRUE。

通过 SQL 语法手动创建错误日志表（例如，表名为 LOG_T1）。为了减少出错环节，不推荐用户自己手动创建。

```
CREATE TABLE LOG_T1(
    DM_ERR_NUMBER$ INTEGER,
    DM_ERR_MESG$ VARCHAR(2000),
    DM_ERR_ROWID$ ROWID,
```

```
    DM_ERR_OPTYPE$ VARCHAR(2),
    DM_ERR_TAG$ VARCHAR(2000)
);
```

错误日志表的其他列可以有源表的列，当存在与源表同名的列时，会将源表该列的出错值插入错误日志表的对应列。使用系统包提供的方法创建的错误日志表，除固定的前 5 列外，其余列类型固定为 VARCHAR(4000)，用户自定义列也应尽量使用相同类型，避免出现不必要的错误导致插入数据失败，无法达到预期效果。

40.2 使用说明

1. 使用该包需要有包执行权限，表的操作权限，建表权限。
2. 支持对日志表进行 I/D/U 操作。
3. 暂时不支持对视图建错误日志表。
4. 暂时不支持对系统表建错误日志表。
5. 暂时不支持批量绑定参数执行<log erros>语法。
6. 只有错误发生在 I/D/U 的自身操作符执行期间才会记录日志。
7. 部分错误在进行错误日志记录时无法登记操作相关的数据，如字符串转换出错。更新操作的一些错误发生在获取原始数据前，因此无法登记操作相关数据，比如违反非空约束、数据超过数据类型限制等。
8. 暂时不支持 INSERT ALL 语句中使用<log erros>语法。
9. 虚拟列暂不进行记录数据。

10. 暂时不支持 DBLINK。

11. 错误日志表记录的内容长度须小于 4000 字节，数据超长时将会报错处理。

12. 不支持分区子表作为错误日志表，不支持为分区子表创建错误日志表。

13. 不支持 huge 表，临时表，外部表，对象表，分布表，分区子表。

14. 不支持 MPP 环境。

15. 对于 DPC 环境下的表，所使用的错误日志表必须拥有和源表完全一致的分布情况，因此建表时指定 ERR_LOG_TABLE_SPACE 无效。

16. DPC 环境下的分区表的错误日志表，保留分区列为原始数据类型，以便数据分布查找。

17. DPC 环境下错误日志表中不支持记录分区列更新造成分区更新的错误。

40.3 使用错误日志表

在数据插入语句、数据修改语句或数据删除语句的尾部使用<log errors>，可以将错误操作生成错误记录，登记到错误日志表中。

LOG ERRORS 语法:

```
<数据插入语句><log errors>;

<数据修改语句><log errors>;

<数据删除语句><log errors>;

<log errors>::=LOG ERRORS [INTO <表名>][(<expression>)] REJECT LIMIT <UNLIMITED | <错误行数>>

<数据插入语句>::=请参考《DM8_SQL 语言使用手册》第 5.1 节

<数据修改语句>::=请参考《DM8_SQL 语言使用手册》第 5.2 节

<数据删除语句>::=请参考《DM8_SQL 语言使用手册》第 5.3 节
```

参数详解

- [INTO <表名>] 错误日志表的表名。缺省使用的是默认错误日志表名。
- <expression> 是一个表达式，为错误日志记录的一个标记，即每一行错误日志会有一列存储此列表达式的值，该<expression>需要是字符串或者是可以转换为字符串的内容。
- REJECT LIMIT <UNLIMITED | <错误行数>> 指定允许的最大错误行数。UNLIMITED 表示没有限制；<错误行数> 指定的允许的最大错误行数，取值(0~ 2147483647)，超过此值将回滚 DML 操作，但仍保留已经生成的错误日志。错误日志表生成记录后，不会因为 DML 语句的回滚而回滚，也不会因为基表被删除而同步被删除。其中，空间不足导致的回滚，不会在错误日志表中记录回滚记录。针对不同的 DML 操作，错误日志表中记录的错误类型如下表所示：

DML 操作	字符串转换出错	违反唯一约束条件	不允许 NULL 值	数据超过数据类型限制	分区列更新造成分区更新	插入值不满足分区范围
INSERT	支持	支持	支持	支持	×	支持
UPDATE	×	×	支持	支持	支持	×
DELETE	×	×	×	×	×	×

注：符号“×”表示错误日志表中暂不支持记录该报错信息

注 2：DPC 环境下不支持记录分区列更新造成分区更新的错误

40.4 举例说明

使用包内的过程和函数之前，如果还未创建过系统包，请先调用系统过程 SP_CREATE_SYSTEM_PACKAGES 创建系统包。

```
SP_CREATE_SYSTEM_PACKAGES(1,'DBMS_ERRLOG');
```

首先，创建数据库表 t1，并使用 DBMS_ERRLOG 为 t1 创建一个错误日志表。同时插入 3 条数据。

```
create table t1 (id int primary key,name varchar(20));
begin
  DBMS_ERRLOG.CREATE_ERROR_LOG('T1','LOG_T1');
end;
/
insert into t1 values(1, 'a');
insert into t1 values(2, 'b');
insert into t1 values(10, 'j');
```

其次，在插入数据的时候使用<log errors>。为 t1 插入 20 条数据，其中有三条是违反了唯一性约束的。

```
begin
  for i in 1..20 loop
    insert into t1 values(i,i) LOG ERRORS INTO LOG_T1 ('INSERT ERROR') REJECT LIMIT UNLIMITED;
  end loop;
  commit;
end;
/
```

然后，查看日志信息表。

```
select * from LOG_T1;
```

查询结果如下：

行号	DM_ERR_NUMBER$	DM_ERR_MESG$	DM_ERR_ROWID$	DM_ERR_OPTYPE$	DM_ERR_TAG$	ID	NAME

1	-6602	违反表[T1]唯一性约束条件[CONS134218802]	AAAAAAAAAAAAAAAAAA	1	INSERT ERROR	1	1
2	-6602	违反表[T1]唯一性约束条件[CONS134218802]	AAAAAAAAAAAAAAAAAA	1	INSERT ERROR	2	2
3	-6602	违反表[T1]唯一性约束条件[CONS134218802]	AAAAAAAAAAAAAAAAAA	1	INSERT ERROR	10	10

可见，LOG_T1 中记录了插入失败的数据以及相关信息。

```
select count(*) from t1;
```

查询结果为：20。

可见，t1 中插入了全部未出错的数据。之前的 3 条和后来批量插入的 17 条。

41 UTL_COMPRESS 包

UTL_COMPRESS 包提供对输入的 BLOB、VARBINARY 或 BFILE 类型数据进行压缩或解压的功能。压缩与解压的格式兼容 gzip（-n），压缩时支持指定压缩等级，且支持分段压缩或解压。

UTL_COMPRESS 包解压时支持的最大解压比例和解压后数据的最大长度受 INI 参数 MAX_UNCOMPRESS_RATIO 和 MAX_UNCOMPRESS_LENGTH 的限制，具体可参考《DM8 系统管理员手册》2.1.1.1.8 节。

调用 UTL_COMPRESS 包中的相关函数和过程时需要保证 DM 安装目录的 bin 子目录下存在 libz.dll（windows）或 libz.so（linux）库文件，否则报错。

41.1 相关方法

下面对各个函数和过程进行详细说明。

1. LZ_COMPRESS

对数据进行压缩。

语法如下：

```
FUNCTION LZ_COMPRESS(
    SRC             IN          VARBINARY,
    QUALITY         IN          INTEGER         DEFAULT 6
) RETURN VARBINARY;
FUNCTION LZ_COMPRESS(
    SRC             IN          BLOB,
    QUALITY         IN          INTEGER         DEFAULT 6
```

```
) RETURN BLOB;

FUNCTION LZ_COMPRESS(
    SRC             IN          BFILE,
    QUALITY         IN          INTEGER         DEFAULT 6
) RETURN BLOB;

PROCEDURE LZ_COMPRESS(
    SRC             IN          BLOB,
    DST             IN OUT      BLOB,
    QUALITY         IN          INTEGER         DEFAULT 6
);

PROCEDURE LZ_COMPRESS(
    SRC             IN          BFILE,
    DST             IN OUT      BLOB,
    QUALITY         IN          BINARY_INTEGER  DEFAULT 6
);
```

参数详解

- SRC　输入参数，VARBINARY、BLOB 或 BFILE 类型的待压缩数据。
- QUALITY　输入参数，压缩等级。取值范围 1~9：1 表示 1 级压缩；2 表示 2 级压缩；9 表示 9 级压缩。压缩级别越高，压缩速度越慢，但压缩比越高。默认值为 6。
- DST　输入输出参数，压缩结果。LZ_COMPRESS 过程是将压缩数据返回到现有的 BLOB 类型参数 DST 中，因此当参数 DST 为空时，将报错处理。

返回值

压缩结果。若待压缩数据类型为 VARBINARY，则压缩结果的数据类型也为 VARBINARY；若待压缩数据类型为 BLOB 或 BFILE，则压缩结果的数据类型均为 BLOB。

使用说明:

(1) 待压缩数据不能为空，否则报错;

(2) 若压缩结果的数据类型为 VARBINARY,且压缩结果的数据长度超过 32767 个字节，则返回空;

(3) 若待压缩数据类型为 BFILE，则需要首先调用 DBMS_LOB.FILEOPEN 打开 BFILE，否则报错;

(4) 压缩等级必须在合法范围内，否则报错;

(5) 若返回值类型为 BLOB，或输出参数为临时 BLOB 大对象，则需要调用 DBMS_LOB.FREETEMPORARY 释放临时的 BLOB 大对象;

(6) ORACLE 与 DM 对数据的压缩结果不一定完全相同，但得到压缩数据都可以被 ORACLE 或 DM 解压。

2. LZ_UNCOMPRESS

对数据进行解压。

语法如下:

```
FUNCTION LZ_UNCOMPRESS(
    SRC             IN              VARBINARY
) RETURN VARBINARY;
FUNCTION LZ_UNCOMPRESS(
    SRC             IN              BLOB
) RETURN BLOB;
FUNCTION LZ_UNCOMPRESS(
```

```
    SRC             IN              BFILE
) RETURN BLOB;
PROCEDURE LZ_UNCOMPRESS(
    SRC             IN              BLOB,
    DST             IN OUT          BLOB
);
PROCEDURE LZ_UNCOMPRESS(
    SRC             IN              BFILE,
    DST             IN OUT          BLOB
);
```

参数详解

- SRC　输入参数，VARBINARY、BLOB 或 BFILE 类型的待解压数据。
- DST　输入输出参数，解压结果。

返回值

解压结果。若待解压数据类型为 VARBINARY，则解压结果的数据类型也为 VARBINARY；若待解压数据类型为 BLOB 或 BFILE，则解压结果的数据类型均为 BLOB。

使用说明:

(1) 待解压数据不能为空，否则报错；

(2) 若解压结果的数据类型为 VARBINARY，且解压结果的数据长度超过 32767 个字节，则返回空；

(3) 若待解压数据类型为 BFILE，则需要首先调用 DBMS_LOB.FILEOPEN 打开 BFILE，否则报错；

(4) 若返回值类型为 BLOB，或输出参数为临时 BLOB 大对象，则需要调用

DBMS_LOB.FREETEMPORARY 释放临时的 BLOB 大对象。

3. ISOPEN

判断句柄是否打开。

语法如下:

```
FUNCTION ISOPEN(
    HANDLE          IN      INTEGER
) RETURN BOOLEAN;
```

参数详解

- HANDLE 输入参数，句柄。用 1~5 之间的 5 个整型数字表示句柄，详细介绍请参考 LZ_COMPRESS_OPEN 函数的使用说明。

返回值

TRUE：句柄已打开。

FALSE：句柄未打开。

4. LZ_COMPRESS_OPEN

打开一个压缩句柄，指定一个 BLOB 大对象用于存放压缩结果。

语法如下:

```
FUNCTION LZ_COMPRESS_OPEN (
    DST             IN OUT  BLOB,
    QUALITY         IN      INTEGER     DEFAULT 6
) RETURN INTEGER;
```

参数详解

- DST　输入输出参数，用于存放压缩结果的 BLOB 大对象。
- QUALITY　输入参数，压缩等级。取值范围 1~9：1 表示 1 级压缩；2 表示 2 级压缩；9 表示 9 级压缩。压缩级别越高，压缩速度越慢，但压缩比越高。缺省值为 6。

返回值

压缩句柄的对应值。

使用说明:

(1) BLOB 大对象不能为空，否则报错;

(2) 压缩等级必须在合法范围内，否则报错;

(3) 每个会话最多打开 5 个压缩或解压句柄，分别用 1~5 之间的 5 个整数表示，若打开句柄的个数超过 5 个，则报错。调用 LZ_COMPRESS_OPEN 函数时会优先打开已关闭的句柄中值较小的句柄，例如：新建一个会话，打开三个句柄，则返回三个值为 1、2、3 的句柄，关闭值为 2 的句柄，再打开三个句柄，则返回三个值为 2、4、5 的句柄，由于此时打开的句柄个数为 5 个，若继续打开句柄，则报错。

5. LZ_COMPRESS_ADD

按照压缩句柄指定的压缩等级，压缩数据。

语法如下:

```
PROCEDURE LZ_COMPRESS_ADD (
    HANDLE      IN          INTEGER,
    DST         IN OUT      BLOB,
    SRC             IN          VARBINARY
```

```
);
```

参数详解

- HANDLE　输入参数，压缩句柄。用 1~5 之间的 5 个整型数字表示句柄，详细介绍请参考 LZ_COMPRESS_OPEN 函数的使用说明。
- DST　输入输出参数，压缩句柄对应的 BLOB 大对象，用于存放压缩结果。
- SRC　输入参数，待压缩数据。

使用说明:

(1) 用户需要提前打开压缩句柄，否则报错;

(2) 压缩句柄必须与 BLOB 大对象相对应，否则报错;

(3) 待压缩数据与 BLOB 大对象均不能为空，否则报错;

(4) LZ_COMPRESS_ADD 执行过程中会随时将压缩数据更新到 BLOB 大对象中，更新的压缩数据将被添加到 BLOB 大对象的末尾。

6. LZ_COMPRESS_CLOSE

将缓存中的压缩数据全部写入 BLOB 大对象，关闭压缩句柄。

语法如下:

```
PROCEDURE LZ_COMPRESS_CLOSE (
    HANDLE      IN          INTEGER,
    DST         IN OUT      BLOB
);
```

参数详解

- HANDLE　输入参数，压缩句柄。用 1~5 之间的 5 个整型数字表示句柄，详细介绍请参

考 LZ_COMPRESS_OPEN 函数的使用说明。

- DST　输入输出参数，压缩句柄对应的 BLOB 大对象，用于存放压缩结果。

使用说明:

(1) 若压缩句柄已经处于关闭状态，则报错;

(2) 压缩句柄必须与 BLOB 大对象相对应，否则报错;

(3) 若执行 LZ_COMPRESS_OPEN 后并未执行 LZ_COMPRESS_ADD，则压缩结果为空，即 BLOB 大对象为空，执行 LZ_COMPRESS_OPEN 时 BLOB 大对象的初始值不会影响最后的结果。

7. LZ_UNCOMPRESS_OPEN

打开一个解压句柄，指定一个待解压的 BLOB 大对象。

语法如下:

```
FUNCTION LZ_UNCOMPRESS_OPEN(
    SRC             IN          BLOB
) RETURN BINARY_INTEGER;
```

参数详解

- SRC　输入参数，待解压的 BLOB 大对象。

返回值

解压句柄的对应值。

使用说明:

(1) BLOB 大对象不能为空，否则报错;

(2) 该函数返回句柄值的策略与 LZ_COMPRESS_OPEN 相同，详细介绍请参考

LZ_COMPRESS_OPEN 函数的使用说明。

8. LZ_UNCOMPRESS_EXTRACT

从上次解压结束的位置，继续解压句柄对应的 BLOB 大对象。

语法如下:

```
PROCEDURE LZ_UNCOMPRESS_EXTRACT(
    HANDLE      IN      INTEGER,
    DST         OUT     VARBINARY
);
PROCEDURE LZ_UNCOMPRESS_EXTRACT(
    HANDLE      IN      INTEGER,
    DST         OUT     VARBINARY,
    OUTLEN      IN      INTEGER
);
```

参数详解

- HANDLE　输入参数，解压句柄。用 1~5 之间的 5 个整型数字表示句柄，详细介绍请参考 LZ_COMPRESS_OPEN 函数的使用说明。
- DST　输出参数，解压结果。
- OUTLEN　输入参数，指定解压的数据长度。取值范围为[0,32767]，单位：字节。

使用说明:

(1) 用户需要提前打开解压句柄，否则报错;

(2) 若调用 LZ_UNCOMPRESS_EXTRACT 时解压句柄对应的 BLOB 大对象已经全部解

压完成，则报错；

(3) 若待解压数据过长导致无法被完全解压，则 ORACLE 会根据输出参数 DST 的定义长度，解压相应长度的数据存放在 DST 中，DM 不支持上述策略，当待解压数据无法被完全解压时，调用两个参数的 LZ_UNCOMPRESS_EXTRACT 将始终解压 32767 个字节长度的数据，为此，DM 提供三个参数的 LZ_UNCOMPRESS_EXTRACT，用户可通过 OUTLEN 参数指定解压的数据长度；

(4) 若调用 LZ_UNCOMPRESS_EXTRACT 时，解压句柄对应的 BLOB 大对象还未开始解压，则将检查该 BLOB 大对象是否有 GZIP 格式的压缩头，若没有则报错；

(5) 若解压句柄对应的 BLOB 大对象已经解压完成，则会对该 BLOB 大对象进行 CRC 校验，若校验失败则报错；

(6) ORACLE 对第一次执行 LZ_UNCOMPRESS_EXTRACT 时的待解压 BLOB 大对象中的内容进行解压，而 DM 对执行 LZ_UNCOMPRESS_OPEN 时的待解压 BLOB 大对象中的内容进行解压。即若用户在执行 LZ_UNCOMPRESS_OPEN 之后，且在第一次执行 LZ_UNCOMPRESS_EXTRACT 之前修改了待解压 BLOB 大对象中的内容，则 ORACLE 的解压结果会相应改变，但 DM 的解压结果不会发生变化。但若用户在执行 LZ_UNCOMPRESS_EXTRACT 之后修改了待解压 BLOB 大对象中的内容，则 ORACLE 和 DM 的解压结果均不会发生变化。

9. LZ_UNCOMPRESS_CLOSE

关闭解压句柄。

语法如下：

```
PROCEDURE LZ_UNCOMPRESS_CLOSE(
```

```
    HANDLE          IN          INTEGER
);
```

参数详解

- HANDLE　输入参数，解压句柄。用 1~5 之间的 5 个整型数字表示句柄，详细介绍请参考 LZ_COMPRESS_OPEN 函数的使用说明。

使用说明:

若解压句柄已经处于关闭状态，则报错。

41.2 举例说明

使用包内的过程和函数之前，如果还未创建过系统包，请先调用系统过程 SP_CREATE_SYSTEM_PACKAGES 创建系统包。

```
SP_CREATE_SYSTEM_PACKAGES(1,'UTL_COMPRESS');
```

准备如下数据:

```
CREATE TABLE BFILE_TABLE(ID NUMBER,BFILE_COLUMN BFILE);
CREATE DIRECTORY MYBFILE AS 'D:\testxml\bfile';
INSERT INTO BFILE_TABLE VALUES (1,BFILENAME('MYBFILE','bfiletest.txt'));
CREATE TABLE BLOB_TABLE(ID NUMBER,BLOB_COLUMN BLOB);
COMMIT;
```

其中，D:\testxml\bfile 目录下的 bfiletest.txt 文件中的内容如下:

```
《红楼梦》写的是封建贵族的青年贾宝玉、林黛王、薛宝钗之间的恋爱和婚姻悲剧，而且以此为中心，写出了当时具有代表性的贾、王、史、薛四大家族的兴衰，其中又以贾府为中心，揭露了封建社会后期的种种黑暗和罪恶，及其不可克服的内在矛盾，对腐朽的封建统治阶级和行将崩溃的封建制度作了有力的批判，使读者预感到它必然要走向覆灭的命运。
```

例 1 LZ_COMPRESS 函数的使用。

一 从 BFILE 文件中读取数据并进行压缩。

```
DECLARE
  SRC_BFILE BFILE;
  DST_BLOB BLOB;
BEGIN
  SELECT BFILE_COLUMN INTO SRC_BFILE FROM BFILE_TABLE WHERE ID=1 FOR UPDATE;
  DBMS_LOB.OPEN(SRC_BFILE);  //打开 BFILE 文件
  DST_BLOB := UTL_COMPRESS.LZ_COMPRESS(SRC_BFILE,6);  //进行数据压缩
  INSERT INTO BLOB_TABLE VALUES(1, DST_BLOB);
END;
```

二 查询压缩结果。

```
SELECT * FROM BLOB_TABLE WHERE ID=1;
//查询结果如下:
ID          BLOB_COLUMN
---------- --------------------------------------------------------------------
1
            0x1F8B0800000000000034D90D16E82301486EFF7D4A096A1A8A05B86933AA243D
            1250833C8B4157D187B4ECBD55E61AD2ECB925E347F4FBEEF3FBD5A3DC916B83CE162
            75B53C705EE5B483931C0A1B3853E51E435F27CD6C0CC7529567D8CEE4B07BB56C
            7C0B1B1EC9A1A7EFEA25D279330EC5D16BC2D2106C4FBA9F30EEC3660A29C7D60EB
            AE9F7A9AFACBE383C0BBEC42C1107260E195CDA3AD75E78648239503F615801F942
            DA15FC5DCDD768A51AA8D55A74D781BFFB95461124296CAB7B4920A59A1786462A
```

```
43F65D2D329D19FDEF423F6BE8CEB86E3BCAE42C4E53088648630D91E9409F868F7
01AEAFEB2FE40BB324CBF77C30EC0CF81784807C6E810A06B1947323A9B99FCA8488
0B4364F77388F71C79B492559AA696ADE87C2817283ACFD37036E056C256AAA2BE
9ADA117990FEC1EC14D3453D4179573659166D1C14E0C6E01DB365C88EC546A65AB7
D01C148AD1C69670638AAD525B85AAD871F26B4024BD8010000
```

例 2 LZ_UNCOMPRESS 函数的使用。

一 对给定的压缩数据进行解压。

```
DECLARE
  SRC_BLOB BLOB;
  DST_BLOB BLOB;
BEGIN
  SRC_BLOB:=HEXTORAW('1F8B080000000000000BBB70EEC8B56B17CE1DB940777A204D24CF0E
004F3D66292C010000');    //符合校验的压缩数据
  DST_BLOB := UTL_COMPRESS.LZ_UNCOMPRESS(SRC_BLOB);  //进行数据解压
  INSERT INTO BLOB_TABLE VALUES(2, DST_BLOB);
END;
```

二 查询解压结果。

```
SELECT * FROM BLOB_TABLE WHERE ID=2;
//查询结果如下:
ID          BLOB_COLUMN
---------- --------------------------------------------------------------------
2
```

```
            0xD0CEC4D6D6D0CEC4D0CEC4D6D6D0CEC4D0CEC4D6D6D0CEC4D0CEC4D6D6
            D0CEC4D0CEC4D6D6D0CEC4D0CEC4D6D6D0CEC4D0CEC4D6D6D0CEC4D0CEC4
            D6D6D0CEC4D0CEC4D6D6D0CEC4D0CEC4D6D6D0CEC4D0CEC4D6D6D0CEC4D0
            CEC4D6D6D0CEC4D0CEC4D6D6D0CEC4D0CEC4D6D6D0CEC4D0CEC4D6D6D0CE
            C4D0CED0CEC4D6D6D0CEC4D0CEC4D6D6D0CEC4D0CEC4D6D6D0CEC4D0CEC4
            D6D6D0CEC4D0CEC4D6D6D0CEC4D0CEC4D6D6D0CEC4D0CEC4D6D6D0CEC4D0
            CEC4D6D6D0CEC4D0CEC4D6D6D0CEC4D0CEC4D6D6D0CEC4D0CED0CEC4D6D6
            D0CEC4D0CEC4D6D6D0CEC4D0CEC4D6D6D0CEC4D0CEC4D6D6D0CEC4D0CEC4
            D6D6D0CEC4D0CED0CEC4D6D6D0CEC4D0CEC4D6D6D0CEC4D0CEC4D6D6D0CE
            C4D0CEC4D6D6D0CEC4D0CEC4D6D6D0CEC4D0CEC4D6D6D0CEC4D0CEC4D6D6
            D0
```

例 3 分段压缩。

一 对给定的数据进行分段压缩。

```
DECLARE
  V_HANDLE BINARY_INTEGER;
  DST_BLOB BLOB;
  SRC_VAR VARBINARY;
BEGIN
  SRC_VAR:=HEXTORAW('D0CEC4D6D6D0CEC4D0CEC4D6D6D0CEC4D0CEC4D6D6D0CEC4D0
CEC4D6D6D0CEC4D0CEC4D6D6D0CEC4D0CEC4D6D6D0CEC4D0CEC4D6D6D0CEC4D0CEC4D6
D6D0CEC4D0CEC4D6D6D0CEC4D0CEC4D6D6D0CEC4D0CEC4D6D6D0CEC4D0CEC4D6D6D0CE
C4D0CEC4D6D6D0CEC4D0CEC4D6D6D0CEC4D0CEC4D6D6D0CEC4D0CED0CEC4D6D6D0CEC4
```

```
D0CEC4D6D6D0CEC4D0CEC4D6D6D0CEC4D0CEC4D6D6D0CEC4D0CEC4D6D6D0CEC4D0CEC4
D6D6D0CEC4D0CEC4D6D6D0CEC4D0CEC4D6D6D0CEC4D0CEC4D6D6D0CEC4D0CEC4D6D6D0
CEC4D0CED0CEC4D6D6D0CEC4D0CEC4D6D6D0CEC4D0CEC4D6D6D0CEC4D0CEC4D6D6D0CE
C4D0CEC4D6D6D0CEC4D0CED0CEC4D6D6D0CEC4D0CEC4D6D6D0CEC4D0CEC4D6D6D0CEC4
D0CEC4D6D6D0CEC4D0CEC4D6D6D0CEC4D0CEC4D6D6D0CEC4D0CEC4D6D6D0');
  DBMS_LOB.CREATETEMPORARY(DST_BLOB, TRUE);  //创建临时 BLOB 大对象
  V_HANDLE :=UTL_COMPRESS.LZ_COMPRESS_OPEN(DST_BLOB,5);  //打开压缩句柄
  UTL_COMPRESS.LZ_COMPRESS_ADD(V_HANDLE,DST_BLOB,SRC_VAR);  //进行分段压缩
  UTL_COMPRESS.LZ_COMPRESS_CLOSE(V_HANDLE,DST_BLOB);  //关闭压缩句柄
  INSERT INTO BLOB_TABLE VALUES(3,DST_BLOB);
  DBMS_LOB.FREETEMPORARY(DST_BLOB);  //释放临时 BLOB 大对象
END;
```

二 查询分段压缩结果。

```
SELECT * FROM BLOB_TABLE WHERE ID=3;
//查询结果如下:
ID          BLOB_COLUMN
---------- --------------------------------------------------------------------
3
            0x1F8B0800000000000003BB70EEC8B56B17CE1DB940779AB00A5269C22A485789
            4C03004F3D66292C010000
```

例 4 分段解压。

一 对给定的压缩数据进行分段解压。

```
DECLARE
  SRC_BLOB BLOB;
  DST_TEMP VARBINARY;
  DST_OUT BLOB;
  V_HANDLE BINARY_INTEGER;
BEGIN
  SRC_BLOB:=HEXTORAW('1F8B080000000000000BBB70EEC8B56B17CE1DB940777A204D24CF0E
004F3D66292C010000');  //符合校验的压缩数据
  V_HANDLE :=UTL_COMPRESS.LZ_UNCOMPRESS_OPEN(SRC_BLOB); //打开解压句柄
  DBMS_LOB.CREATETEMPORARY(DST_OUT, TRUE);  //创建临时 BLOB 大对象
  //利用循环结构进行分段解压
  LOOP
    BEGIN
      UTL_COMPRESS.LZ_UNCOMPRESS_EXTRACT(V_HANDLE, DST_TEMP);
      EXCEPTION
        WHEN OTHERS THEN
      EXIT;
    END;
    DBMS_LOB.APPEND(DST_OUT, DST_TEMP);
  END LOOP;
  UTL_COMPRESS.LZ_UNCOMPRESS_CLOSE(V_HANDLE);  //关闭解压句柄
  INSERT INTO BLOB_TABLE VALUES(4,DST_OUT);
  DBMS_LOB.FREETEMPORARY(DST_OUT);  //释放临时 BLOB 大对象
END;
```

二 查询分段解压结果。

```
SELECT * FROM BLOB_TABLE WHERE ID=4;

查询结果为:

ID          BLOB_COLUMN

---------- --------------------------------------------------------------------

4

            0xD0CEC4D6D6D0CEC4D0CEC4D6D6D0CEC4D0CEC4D6D6D0CEC4D0CEC4D6D6
            D0CEC4D0CEC4D6D6D0CEC4D0CEC4D6D6D0CEC4D0CEC4D6D6D0CEC4D0CEC4
            D6D6D0CEC4D0CEC4D6D6D0CEC4D0CEC4D6D6D0CEC4D0CEC4D6D6D0CEC4D0
            CEC4D6D6D0CEC4D0CEC4D6D6D0CEC4D0CEC4D6D6D0CEC4D0CEC4D6D6D0CE
            C4D0CED0CEC4D6D6D0CEC4D0CEC4D6D6D0CEC4D0CEC4D6D6D0CEC4D0CEC4
            D6D6D0CEC4D0CEC4D6D6D0CEC4D0CEC4D6D6D0CEC4D0CEC4D6D6D0CEC4D0
            CEC4D6D6D0CEC4D0CEC4D6D6D0CEC4D0CEC4D6D6D0CEC4D0CED0CEC4D6D6
            D0CEC4D0CEC4D6D6D0CEC4D0CEC4D6D6D0CEC4D0CEC4D6D6D0CEC4D0CEC4
            D6D6D0CEC4D0CED0CEC4D6D6D0CEC4D0CEC4D6D6D0CEC4D0CEC4D6D6D0CE
            C4D0CEC4D6D6D0CEC4D0CEC4D6D6D0CEC4D0CEC4D6D6D0CEC4D0CEC4D6D6
            D0
```

42 DBMS_APPLICATION_INFO 包

DBMS_APPLICATION_INFO 包用于记录数据库正在运行的模块名称以及模块正在执行的事务名称。利用这个包，管理员和开发者可以进行性能追踪以及调试。

42.1 相关方法

DBMS_APPLICATION_INFO 包中包含的过程和函数如下详细介绍：

1. SET_CLIENT_INFO

设置当前会话的客户端信息。

语法如下：

```
PROCEDURE SET_CLIENT_INFO(
    CLIENT_INFO         IN          VARCHAR2
);
```

参数详解

- CLIENT_INFO　输入参数，当前会话的客户端信息，最大长度为 64 字节。

2. SET_MODULE

设置当前运行的模块名称。

语法如下：

```
PROCEDURE SET_MODULE(
    MODULE_NAME         IN          VARCHAR2,
    ACTION_NAME         IN          VARCHAR2
);
```

参数详解

- MODULE_NAME　输入参数，当前运行的模块名称，模块名通常为数据库中的表名或预编译应用的代码段名称，最大长度为 48 字节。当前模块运行结束后需要再次调用 SET_MODULE 过程，若存在下一个待运行模块，则需要将 MODULE_NAME 参数设置为下一个运行模块的名称；若不存在待运行模块，则需要将 MODULE_NAME 参数设置为 NULL。
- ACTION_NAME　输入参数，当前模块正在执行的事务名称，最大长度为 32 字节。用户可将该参数设置为 NULL，之后通过调用 SET_ACTION 过程设置当前模块正在执行的事务名称。

3. SET_ACTION

设置当前模块正在执行的事务名称。

语法如下:

```
PROCEDURE SET_ACTION(
    ACTION_NAME          IN          VARCHAR2
);
```

参数详解

- ACTION_NAME　输入参数，当前模块正在执行的事务名称，最大长度为 32 字节。当前事务运行结束后需要再次调用 SET_ACTION 过程，若存在下一个待执行事务，则需要将 ACTION_NAME 参数设置为下一个执行事务的名称；若不存在待执行事务，则需要将 ACTION_NAME 参数设置为 NULL。

4. READ_CLIENT_INFO

获取当前会话的客户端信息。

语法如下:

```
PROCEDURE READ_CLIENT_INFO(
    CLIENT_INFO         OUT       VARCHAR2
);
```

参数详解

- CLIENT_INFO 输出参数，当前会话的客户端信息。

5. READ_MODULE

获取数据库当前运行的模块名称以及模块正在执行的事务名称。

语法如下:

```
PROCEDURE READ_MODULE(
    MODULE_NAME             OUT       VARCHAR2,
    ACTION_NAME             OUT       VARCHAR2
);
```

参数详解

- MODULE_NAME 输出参数，当前运行的模块名称。
- ACTION_NAME 输出参数，当前模块正在执行的事务名称。

42.2 举例说明

使用包内的过程和函数之前，如果还未创建过系统包，请先调用系统过程 SP_CREATE_SYSTEM_PACKAGES 创建系统包。

```
SP_CREATE_SYSTEM_PACKAGES (1,'DBMS_OUTPUT');
SP_CREATE_SYSTEM_PACKAGES (1,'DBMS_APPLICATION_INFO');
SET SERVEROUTPUT ON;  //DBMS_OUTPUT.PUT_LINE 需要设置这条语句，才能打印出消息
```

记录数据库当前正在运行的模块名称以及模块正在执行的事务名称。

```
DECLARE
    C_N VARCHAR2(100);
    M_N VARCHAR2(100);
    A_N VARCHAR2(100);
BEGIN
    DBMS_APPLICATION_INFO.SET_CLIENT_INFO('MY CLIENT');
    DBMS_APPLICATION_INFO.SET_MODULE('MODULE1', 'ACTION1');
    DBMS_APPLICATION_INFO.SET_ACTION('ACTION2');
    DBMS_APPLICATION_INFO.READ_CLIENT_INFO(C_N);
    DBMS_APPLICATION_INFO.READ_MODULE(M_N, A_N);
    DBMS_OUTPUT.PUT_LINE('CLIENT INFO IS:' || C_N);
    DBMS_OUTPUT.PUT_LINE('MODULE NAME IS:' || M_N);
    DBMS_OUTPUT.PUT_LINE('ACTION NAME IS:' || A_N);
END;
```

执行结果如下：

```
CLIENT INFO IS:MY CLIENT
MODULE NAME IS:MODULE1
ACTION NAME IS:ACTION2
```

43 DBMS_REDEFINITION 包

DBMS_REDEFINITION 包提供了在线重定义数据库表的功能。这种功能尽可能地减少了对表的封锁时间，因此，数据库表在重定义的过程中也能正常提供服务。

使用须知:

1.对于临时表、物化视图日志表、系统表、外部表、记录类型数组或对象类型所用的临时表、动态性能视图、位图连接索引表、位图索引表等类型的表以及含有半透明加密列的表不支持进行重定义。

2.对于 SYSDBA 下的用户表与普通用户表同等对待，允许进行重定义。

3.DM MPP 环境下不支持 DBMS_REDEFINITION 包。

43.1 相关方法

1. CAN_REDEF_TABLE

判断目标表是否可以进行在线重定义。

语法如下:

```
DBMS_REDEFINITION.CAN_REDEF_TABLE (
    UNAME                IN   VARCHAR(128),
    TNAME                IN   VARCHAR(128),
    OPTIONS_FLAG         IN   INT := CONS_USE_PK,
    PART_NAME            IN   VARCHAR(128) := NULL);
```

参数详解

- uname：模式名。
- TNAME：进行重定义的原表。

- options_flag：重定义方式，取值为 dbms_redefinition.cons_use_pk 或 dbms_redefinition.cons_use_rowid。默认为 cons_use_pk。对于不存在主键或是伪主键（某一个 UNIQUE 约束上的所有列都具有 NOT NULL 约束）的表，不允许使用 cons_use_pk 方式进行重定义。
- part_name：暂不支持，仅作参数兼容。分区名，默认为空。可以指定多个分区，用逗号隔开。必须与 START_REDEF_TABLE 指定的分区一致。

2. START_REDEF_TABLE

开始重定义过程。

首先会执行一次 CAN_REDEF_TABLE，如果不能按照指定方式重定义，则会报错。在原表 ORIG_TABLE 上创建一个物化视图日志，创建的过程需要对原表上 X 锁；如果原表上已存在物化视图日志，则判断该物化视图日志的类型（ROWID 还是 PK）与指定的重定义方式是否一致，如果不一致则报错，如果一致，则跳过创建物化视图日志的过程。基于中间表建立物化视图，并进行完全刷新。

语法如下：

```
DBMS_REDEFINITION.START_REDEF_TABLE (
    UNAME                   IN  VARCHAR(128),
    ORIG_TABLE              IN  VARCHAR(128),
    INT_TABLE               IN  VARCHAR(128),
    COL_MAPPING                 IN  VARCHAR := NULL,
    OPTIONS_FLAG            IN  INT := CONS_USE_PK,
    ORDERBY_COLS            IN  VARCHAR(128) := NULL,
    PART_NAME               IN  VARCHAR(128) := NULL,
```

```
CONTINUE_AFTER_ERRORS    IN   BOOLEAN := FALSE,
COPY_VPD_OPT             IN   INT := 1,
REFRESH_DEP_MVIEWS       IN   VARCHAR(128) := 'N');
```

参数详解

- uname：模式名。
- orig_table：进行重定义的原表。
- int_table：按照新定义创建的中间表。中间表的表名、列名、列数都可以和原表不同。但需保证映射的列类型需相同。
- col_mapping: 两个表之间的列映射关系。语法与查询项类似，例"T1.C1 as d1,c2 as d2, c3 as d3"。如果没有指定 COL_MAPPING，则根据原表的列名构造。如果指定列映射，需要包含所有主键列，且不能包含自增列和虚拟列等类型的列。
- options_flag：重定义方式。取值为 dbms_redefinition.cons_use_pk 或 dbms_redefinition.cons_use_rowid。默认为 cons_use_pk。不推荐使用 rowid 的方式进行重定义，可能导致性能较差。
- orderby_cols：暂不支持，仅作参数兼容。转移数据时按照指定列排序，格式与 order by 项类似，例'c2 desc'。
- part_name：暂不支持，仅作参数兼容。分区名，默认为空。可以指定多个分区，用逗号隔开。
- continue_after_errors：暂不支持，仅作参数兼容。批量进行分区重定义时，出错时允许进行下一个分区的重定义。
- copy_vpd_opt：暂不支持，仅作参数兼容。

- refresh_dep_mviews：是否在 START_REDEF_TABLE、SYNC_INTERIM_TABLE 和 FINISH_REDEF_TABLE 执行时对其他依赖该表的物化视图进行快速刷新。Y 是，N 否。

3. COPY_TABLE_DEPENDENTS

将原表上的依赖（索引、约束、授权、统计信息等）复制到新表。DMDPC 环境下暂不支持该方法。

语法如下:

```
DBMS_REDEFINITION.COPY_TABLE_DEPENDENTS(
    UNAME                   IN  VARCHAR(128),
    ORIG_TABLE              IN  VARCHAR(128),
    INT_TABLE               IN  VARCHAR(128),
    COPY_INDEXES            IN  INT := 1,
    COPY_TRIGGERS           IN  BOOLEAN     := TRUE,
    COPY_CONSTRAINTS        IN  BOOLEAN     := TRUE,
    COPY_PRIVILEGES         IN  BOOLEAN     := TRUE,
    IGNORE_ERRORS           IN  BOOLEAN     := FALSE,
    NUM_ERRORS              OUT INT,
    COPY_STATISTICS         IN  BOOLEAN     := FALSE,
    COPY_MVLOG              IN  BOOLEAN     := FALSE);
```

参数详解

- uname：模式名。
- orig_table：进行重定义的原表。

- int_table：按照新定义创建的中间表。
- copy_indexes：是否复制索引
- copy_triggers：暂不支持，仅作参数兼容。是否复制触发器。
- copy_constraints：是否复制约束。
- copy_privileges：是否复制授权。
- ignore_errors：复制过程遇到错误时，是否忽略错误。
- num_errors：出现的错误个数。
- copy_statistics：复制统计信息。
- copy_mvlog：暂不支持，仅作参数兼容。复制物化视图日志。

4. SYNC_INTERIM_TABLE

重定义过程中手动数据同步。为了减轻最后一步 FINISH_REDEF_TABLE 同步的数据量，在重定义的过程中，可使用 SYNC_INTERIM_TABLE 先阶段性地完成部分数据同步。

语法如下：

```
DBMS_REDEFINITION.SYNC_INTERIM_TABLE (
    UNAME                       IN  VARCHAR(128),
    ORIG_TABLE                  IN  VARCHAR(128),
    INT_TABLE                   IN  VARCHAR(128),
    PART_NAME                   IN  VARCHAR(128) := NULL,
    CONTINUE_AFTER_ERRORS       IN  BOOLEAN := FALSE);
```

参数详解

- uname：模式名。
- orig_table：进行重定义的原表。
- int_table：按照新定义创建的中间表。
- part_name：暂不支持，仅作参数兼容。分区名，默认为空。可以指定多个分区，用逗号隔开，数量需要与 int_table 一致。
- continue_after_errors：暂不支持，仅作参数兼容。批量进行分区重定义时，出错时允许进行下一个分区的重定义。

5. SET_PARAM

修改重定义过程中的参数属性。即修改 START_REDEF_TABLE 过程中的各属性值。

语法如下：

```
DBMS_REDEFINITION.SET_PARAM (
    REDEFINITION_ID             IN  VARCHAR(128),
    PARAM_NAME                  IN  VARCHAR(128),
    PARAM_VALUE                     IN  VARCHAR(128));
```

参数详解

- redefinition_id：进行重定义的原表的 ID。
- param_name：需要设置的参数名称。目前仅支持修改 START_REDEF_TABLE 中 refresh_dep_mviews 参数。
- param_value：设置参数的目标值。目标值只能设置为 0 或 1。0 否，1 是。

6. FINISH_REDEF_TABLE

完成重定义过程。完成数据同步并清理重定义环境。根据物化视图日志进行最后的数据同步，并交换原表和中间表的表名。如果原表上的物化视图日志是由 START_REDEF_TABLE 创建的，则删除该物化视图日志，否则保留。本方法全程需要对原表上 X 锁。

语法如下:

```
DBMS_REDEFINITION.FINISH_REDEF_TABLE (
    UNAME                IN   VARCHAR(128),
    ORIG_TABLE           IN   VARCHAR(128),
    INT_TABLE            IN   VARCHAR(128),
    PART_NAME            IN   VARCHAR(128) := NULL,
    DML_LOCK_TIMEOUT     IN   INT := NULL,
    CONTINUE_AFTER_ERRORS IN  BOOLEAN := FALSE);
```

参数详解

- uname：模式名。
- orig_table：进行重定义的原表。
- int_table：按照新定义创建的中间表。
- part_name：暂不支持，仅作参数兼容。分区名，默认为空。可以指定多个分区，用逗号隔开。必须与 START_REDEF_TABLE 指定的分区一致。
- dml_lock_timeout：锁等待的超时时长，取值 0-1000000（秒），NULL 表示不限制等待时长，默认为 NULL。
- continue_after_errors：批量进行分区重定义时，出错时允许进行下一个分区的重定义

7. ABORT_REDEF_TABLE

终止重定义过程。解除原表和中间表之间的依赖关系，清空中间表，并将中间表恢复成普通表状态。如果原表上的物化视图日志是由 START_REDEF_TABLE 创建的，则删除该物化视图日志，否则保留。

语法如下:

```
DBMS_REDEFINITION.ABORT_REDEF_TABLE (
    UNAME                IN  VARCHAR(128),
    ORIG_TABLE           IN  VARCHAR(128),
    INT_TABLE            IN  VARCHAR(128),
    PART_NAME            IN  VARCHAR(128) := NULL);
```

参数详解

- UNAME：模式名。
- ORIG_TABLE：进行重定义的原表。
- INT_TABLE：按照新定义创建的中间表。
- PART_NAME：暂不支持，仅作参数兼容。分区名，默认为空。可以指定多个分区，用逗号隔开。必须与 START_REDEF_TABLE 指定的分区一致。

8. DELETE_REDUNDANT_RECORD

用于删除某些特定情况产生的系统表冗余记录，这些冗余记录可能导致重定义过程执行失败，此时须执行本过程进行删除。例如，如果重定义过程中原表遭到删除，这会导致原表在 SYSOBJECTS 表中的记录不存在，此时，执行本过程会将三张系统表 SYS.SYSREDEFSTATUS、SYS.SYSREDEFOBJECT 和 SYS.SYSREDEFDERROR 中与原表相关的

记录全部删除。

语法如下:

```
DBMS_REDEFINITION.REDUNDANT_RECORD(
    OBJ_ID    IN INT);
```

参数详解

- OBJ_ID 对象 ID。

43.2 系统表

1. SYS.SYSREDEFSTATUS

表重定义过程中，上一步操作的状态。

列名	类型	描述	备注
OBJ_ID	INTEGER	重定义对象的 ID	
REDEF_ID	INTEGER	重定义过程 ID	自增列。该列不作为重定义系统表之间的连接依据
PREV_OPERATION	VARCHAR(128)	操作类型: START_REDEF_TABLE SYNC_INTERIM_TABLE COPY_TABLE_DEPENDENTS FINISH_REDEF_TABLE	
OBJ_OWNER	VARCHAR(128)	对象模式名	
OBJ_NAME	VARCHAR(128)	对象名	
STATUS	VARCHAR(128)	状态：FAILURE 或 SUCCESS	

RESTARTABLE	VARCHAR(1)	是否可以重新开始	暂不支持，一律为'N'
CREATE_MVLOG	INTEGER	START_REDEF_TABLE 是否创建了物化视图日志	用于 ABORT 或 FINISH 时判断是否需要删除物化视图日志
REFRESH_DEP_MV	INTEGER	START_REDEF_TABLE、SYNC_INERTIM_TABLE、FINISH_REDEF_TABLE 时，是否同步刷新表上的物化视图	
COL_MAPPING	VARCHAR	记录用户输入的列映射的原始字符串	
OPTIONS_FLAG	INTEGER	记录是刷新方式 1：PK 2：ROWID	

2. SYS.SYSREDEFOBJECT

表重定义过程中，涉及到的所有对象。仅当进行 COPY_TABLE_DEPENDENTS 时，同一个 OBJ_ID 下可能出现其他对象，否则只会有当前表。

列名	类型	描述	备注
OBJ_ID	INTEGER	重定义对象的 ID	
REDEF_ID	INTEGER	重定义过程 ID	列值直接从 SYSREDEFSTATUS 获取，该列不作为重定义系统表之间的连接依据
OBJ_TYPE	INTEGER	对象类型 1:TABLE 2:INDEX	

		3:CONSTRAINT	
OBJ_OWNER	VARCHAR(128)	对象模式名	
OBJ_NAME	VARCHAR(128)	对象名	
INT_OBJ_ID	INTEGER	中间对象 ID	
INT_OBJ_OWNER	VARCHAR(128)	中间对象模式名	
INT_OBJ_NAME	VARCHAR(128)	中间对象名	
BT_OWNER	VARCHAR(128)	所属基表模式名	
BT_NAME	VARCHAR(128)	所属基表名	
GENFLAG	INTEGER	未知	保留列，无实际意义
TYPFLAG	INTEGER	未知	保留列，无实际意义

3. SYS.SYSREDEFDERROR

记录 COPY_TABLE_DEPENDENTS 过程中出现的错误。

列名	类型	描述	备注
OBJ_ID	INTEGER	重定义对象的 ID	
REDEF_ID	INTEGER	重定义过程 ID	列值直接从 SYSREDEFSTATUS 获取，该列不作为重定义系统表之间的连接依据
OBJ_TYPE	VARCHAR(20)	对象类型	
OBJ_OWNER	VARCHAR(128)	对象模式名	
OBJ_NAME	VARCHAR(128)	对象名	
BT_OWNER	VARCHAR(128)	所属基表模式名	
BT_NAME	VARCHAR(128)	所属基表名	
DDL_TXT	CLOB	定义	
ERR_NO	INTEGER	错误码	

ERR_TXT	VARCHAR(1000)	错误信息	
TIMESTAMP$	BIGINT	错误对应的执行过程开始时的时间戳	用于计算一次执行过程中出现错误的个数

43.3 系统视图

1. DBA_REDEFINITION_STATUS

表重定义过程中，上一步操作的状态。

列名	类型	描述	备注
REDEFINITION_ID	INTEGER	重定义对象的 ID	
BASE_TABLE_OWNER	VARCHAR(128)	基表模式名	
BASE_TABLE_NAME	VARCHAR(128)	基表名	
BASE_PARTITION_NAME	VARCHAR(128)	分区名	暂不支持
INTERIM_OBJECT_OWNER	VARCHAR(128)	中间表模式名	
INTERIM_OBJECT_NAME	VARCHAR(128)	中间表名	
OPERATION	VARCHAR(128)	操作类型: START_REDEF_TABLE SYNC_INTERIM_TABLE COPY_TABLE_DEPENDENTS FINISH_REDEF_TABLE	
STATUS	VARCHAR(128)	状态: FAILURE SUCCESS	

RESTARTABLE	VARCHAR(1)	是否可以重新开始	暂不支持，一律为'N'

2. DBA_REDEFINITION_OBJECTS

表重定义过程中，涉及到的所有对象。仅当进行 COPY_TABLE_DEPENDENTS 时，同一个 OBJ_ID 下可能出现其他对象，否则只会有当前表。

列名	类型	描述	备注
OBJECT_TYPE	VARCHAR(12)	重定义对象类型	
OBJECT_OWNER	VARCHAR(128)	对象模式名	
OBJECT_NAME	VARCHAR(128)	对象名	
BASE_TABLE_OWNER	VARCHAR(128)	基表模式名	
BASE_TABLE_NAME	VARCHAR(128)	基表名	
INTERIM_OBJECT_OWNER	VARCHAR(128)	中间表模式名	
INTERIM_OBJECT_NAME	VARCHAR(128)	中间表名	
EDITION_NAME	VARCHAR(128)	保留字段，将来使用	恒为 NULL

3. DBA_REDEFINITION_ERRORS

记录 COPY_TABLE_DEPENDENTS 过程中出现的错误。

列名	类型	描述	备注
OBJECT_TYPE	VARCHAR(12)	对象类型	
OBJECT_OWNER	VARCHAR(128)	对象模式名	
OBJECT_NAME	VARCHAR(128)	对象名	
BASE_TABLE_OWNER	VARCHAR(128)	所属基表模式名	
BASE_TABLE_NAME	VARCHAR(128)	所属基表名	
DDL_TXT	CLOB	定义	

EDITION_NAME	VARCHAR(128)	保留字段，将来使用	为 NULL
ERR_NO	INTEGER	错误编号	
ERR_TXT	VARCHAR(1000)	错误信息	

43.4 一个完整的重定义过程

一个完整的重定义过程如下：

第1步　可选项。使用 CAN_REDEF_TABLE 判断目标表是否可以进行重定义。是则进入第 2 步，否则结束。提供给用户的预判选项。

第2步　必选项。使用 START_REDEF_TABLE 开始重定义过程。此时，系统会生成一个名为“MVIEW$_+目标表名”的物化视图，用来记录在重定义过程中用户对目标表的数据操作。

第3步　可选项。在重定义过程中，可使用 COPY_TABLE_DEPENDENTS 将原表上的依赖（索引、约束、触发器等）复制到新表。如果用户不再需要这些依赖关系或者想自己手动创建依赖对象，此步骤可省略。

第4步　可选项。在重定义过程中，可使用 SYNC_INTERIM_TABLE 阶段性地手动数据同步。此步骤是为了加速数据同步。如果省略此步骤，数据同步会最终在 FINISH_REDEF_TABLE 步骤中一次性完成。

第5步　可选项。使用 SET_PARAM 设置重定义过程中的参数属性。目前只支持设置同步刷新表上依赖的其他物化视图这一个属性。

第6步　必选项。使用 FINISH_REDEF_TABLE 完成重定义过程，完成数据同步并清理重定义环境。至此，整个重定义过程完成。

注意事项：

1. 在重定义过程中，不允许对原表进行 DDL 操作，否则会产生错误。一旦遇到错误，可使用 ABORT_REDEF_TABLE 终止重定义过程，后续如仍需继续重定义，可在终止后重新

开始一次重定义。

2. 在重定义过程中，如果原表遭到删除，那么 ABORT_REDEF_TABLE 和 FINISH_REDEF_TABLE 无法执行，导致该次重定义过程在系统表中的记录无法删除，产生冗余。对于这些冗余的记录，可以通过 DELETE_REDUNDANT_RECORD 删除。

43.5 举例说明

使用包内的过程和函数之前，如果还未创建过系统包，请先调用系统过程创建系统包。

```
SP_CREATE_SYSTEM_PACKAGES (1,'DBMS_REDEFINITION');
```

1. 建表

```
CREATE SCHEMA "CSCOTT" AUTHORIZATION "SYSDBA";
DROP TABLE "CSCOTT".EMP;
DROP TABLE "CSCOTT".EMP_INT;
//创建原表
CREATE TABLE "CSCOTT"."EMP"(c1 int PRIMARY KEY, c2 int, c3 int) PARTITION BY range(c1)
(
    partition p1 values less than(10),
    partition p2 values less than(20),
    partition p3 values less than(maxvalue)
);
//在原表上创建索引
CREATE INDEX "CSCOTT".emp_i1 on "CSCOTT".EMP(c2);

//创建中间表
```

```
CREATE TABLE "CSCOTT"."EMP_INT"(c1 int, c2 INT, c3 int) ;


INSERT INTO "CSCOTT".EMP(c1,c2,c3) SELECT level,level,level FROM DUAL connect by level <=1000;

INSERT INTO "CSCOTT".EMP_INT(c1) SELECT level FROM DUAL connect by level <=10;
```

2. 使用包中的各方法

```
//可选项。判断原表是否可以重定义。

DBMS_REDEFINITION.CAN_REDEF_TABLE('CSCOTT', 'EMP', DBMS_REDEFINITION.CONS_USE_PK);


//必选项，开始重定义

DBMS_REDEFINITION.START_REDEF_TABLE('CSCOTT', 'EMP', 'EMP_INT',col_mapping =>'C1 as C1, C2
as C2, C3 as C3');


//可选项。在重定义过程中，阶段性地同步数据。

DBMS_REDEFINITION.SYNC_INTERIM_TABLE('CSCOTT', 'EMP', 'EMP_INT');


//可选项。拷贝原表上的依赖关系。

DECLARE

    n_err INT;

BEGIN

     DBMS_REDEFINITION.COPY_TABLE_DEPENDENTS('CSCOTT', 'EMP', 'EMP_INT',
num_errors=>n_err);

     DBMS_OUTPUT.PUT_LINE(n_err);

END;
```

```
//可选项。终止重定义。执行了此步骤，后续不再执行。
DBMS_REDEFINITION.ABORT_REDEF_TABLE('CSCOTT', 'EMP', 'EMP_INT');

//必选项。完成重定义。未进行终止重定义时为必选操作。
DBMS_REDEFINITION FINISH_REDEF_TABLE('CSCOTT', 'EMP', 'EMP_INT');

//可选项。假设重定义过程中原表遭到删除，可手动删除系统表 SYS.SYSREDEFSTATUS、
SYS.SYSREDEFOBJECT 和 SYS.SYSREDEFDERROR 中的数据冗余；下面完整执行该过程，先在重定义过程中
删除原表
DBMS_REDEFINITION.START_REDEF_TABLE('CSCOTT', 'EMP', 'EMP_INT',col_mapping =>'C1 as C1, C2
as C2, C3 as C3');  //开始重定义
DROP TABLE "CSCOTT".EMP CASCADE;  //删除原表
SELECT OBJ_ID FROM SYS.SYSREDEFSTATUS WHERE OBJ_NAME='EMP';  //假定查询到的 ID 为 1128
DBMS_REDEFINITION.DELETE_REDUNDANT_RECORD(1128);  //手动删除系统表 SYS.SYSREDEFSTATUS、
SYS.SYSREDEFOBJECT 和 SYS.SYSREDEFDERROR 中的数据冗余
```

3. 重定义的过程中可查看各系统视图和系统表的变化

```
SELECT * FROM DBA_REDEFINITION_STATUS;
SELECT * FROM DBA_REDEFINITION_OBJECTS;
SELECT * FROM DBA_REDEFINITION_ERRORS;
SELECT * FROM SYS.SYSREDEFSTATUS;
SELECT * FROM SYS.SYSREDEFOBJECT;
SELECT * FROM SYS.SYSREDEFDERROR;
```

44 DBMS_XA 包

DBMS_XA 包为应用提供了和 XA 接口一样功能的方法，以供用户在客户端工具（如 DIsql）或 DMSQL 程序中调用。

仅当 INI 参数 XA_COMPATIBLE_MODE 不为 2 时，支持使用 DBMS_XA 包。

由于当主事务先启动，XA 事务后启动时，XA 事务对主事务不可见，无法查看 XA 事务提交的数据。此时需要开启 INI 参数 TRX_VIEW_POLICY=1，设置事务可见性视图策略为支持脚本中的 SQL 语句更新事务可见性视图，使语句块内的 SQL 及时更新活动事务信息，保证 XA 事务提交后对主事务可见。

44.1 常量

DBMS_XA 包中包含的常量如下：

```
LAST_ERRCODE            INTEGER := 0;



//包中 XA_START 和 XA_END 函数的 FLAG 参数的可取值

TMNOFLAGS               CONSTANT INTEGER :=0X00000000;

TMSUCCESS               CONSTANT INTEGER :=0x04000000;

TMJOIN                  CONSTANT INTEGER :=0x00200000;

TMSUSPEND               CONSTANT INTEGER :=0x02000000;

TMRESUME                CONSTANT INTEGER :=0x08000000;



//包中函数 PLS_INTEGER 类型返回值的可取值

XA_RBBASE               CONSTANT INTEGER := 100;
```

```
XA_RBROLLBACK        CONSTANT INTEGER := XA_RBBASE;

XA_RBCOMMFAIL            CONSTANT INTEGER := XA_RBBASE+1;

XA_RBDEADLOCK            CONSTANT INTEGER := XA_RBBASE+2;

XA_RBINTEGRITY       CONSTANT INTEGER := XA_RBBASE+3;

XA_RBOTHER           CONSTANT INTEGER := XA_RBBASE+4;

XA_RBPROTO           CONSTANT INTEGER := XA_RBBASE+5;

XA_RBTIMEOUT         CONSTANT INTEGER := XA_RBBASE+6;

XA_RBTRANSIENT       CONSTANT INTEGER := XA_RBBASE+7;

XA_RBEND             CONSTANT INTEGER := XA_RBTRANSIENT;


XA_NOMIGRATE         CONSTANT INTEGER := 9;

XA_HEURHAZ           CONSTANT INTEGER := 8;

XA_HEURCOM               CONSTANT INTEGER := 7;

XA_HEURRB            CONSTANT INTEGER := 6;

XA_HEURMIX           CONSTANT INTEGER := 5;

XA_RETRY             CONSTANT INTEGER := 4;

XA_RDONLY            CONSTANT INTEGER := 3;


XA_OK                CONSTANT INTEGER := 0;


XAER_ASYNC           CONSTANT INTEGER := -2;

XAER_RMERR           CONSTANT INTEGER := -3;

XAER_NOTA            CONSTANT INTEGER := -4;

XAER_INVAL           CONSTANT INTEGER := -5;
```

```
XAER_PROTO              CONSTANT INTEGER := -6;

XAER_RMFAIL             CONSTANT INTEGER := -7;

XAER_DUPID              CONSTANT INTEGER := -8;

XAER_OUTSIDE            CONSTANT INTEGER := -9;

XAER_AFFINITY           CONSTANT INTEGER := -10;
```

44.2 数据类型

DBMS_XA 包中包含的数据类型如下:

1. DBMS_XA_XID

DBMS_XA_XID 是 XA 事务标识符。一个 DBMS_XA_XID 代表一个 XA 事务的一个事务分支。

语法如下:

```
CREATE OR REPLACE TYPE SYS.DBMS_XA_XID AS OBJECT(

    FORMATID      INTEGER,

    GTRID         RAW(64),

    BQUAL         RAW(64),


    CONSTRUCTOR FUNCTION DBMS_XA_XID(

        GTRID       IN        INTEGER

    )RETURN SELF AS RESULT,


    CONSTRUCTOR FUNCTION DBMS_XA_XID(

        GTRID       IN        RAW,

        BQUAL       IN        RAW
```

```
    )RETURN SELF AS RESULT,


    CONSTRUCTOR FUNCTION DBMS_XA_XID(
        FORMATID    IN    INTEGER,
        GTRID       IN    RAW,
        BQUAL    IN  RAW  DEFAULT HEXTORAW('00000000000000000000000000000001')
    )RETURN SELF AS RESULT
);
```

类成员说明：

- FORMATID 保留参数，无实际意义。
- GTRID 全局事务 ID。由用户指定，需保证全局唯一。GTRID 相同的事务属于同一个 XA 事务。
- BQUAL 事务分支 ID。由用户指定，需保证全局唯一。缺省为系统内部自动创建。GTRID 相同，BQUAL 不同，表示为同一个 XA 事务的不同事务分支。

44.3 相关方法

DBMS_XA 包中包含的过程和函数如下详细介绍：

1. XA_START

将当前会话与指定 XID 的事务分支关联起来。

语法如下：

```
FUNCTION XA_START(
    XID         IN          DBMS_XA_XID,
    FLAG        IN          PLS_INTEGER
```

```
)RETURN PLS_INTEGER;
```

参数详解

- xid 输入参数，XA 事务标识符。
- flag 输入参数，具体取值可参考 45.1 常量。

2. XA_END

将当前会话与指定 XID 的事务分支取消关联。

语法如下:

```
FUNCTION XA_END (
    XID         IN          DBMS_XA_XID,
    FLAG        IN          PLS_INTEGER
)RETURN PLS_INTEGER;
```

参数详解

- XID 输入参数，XA 事务标识符。
- FLAG 输入参数，具体取值可参考 45.1 常量。

3. XA_SETTIMEOUT

为当前会话设置事务的超时时间。

语法如下:

```
FUNCTION XA_SETTIMEOUT (
    SECONDS     IN          PLS_INTEGER
)RETURN PLS_INTEGER;
```

参数详解

- SECONDS　输入参数，当前会话的事务超时时间，单位：秒。

4. XA_PREPARE

事务分支准备提交。

语法如下:

```
FUNCTION XA_PREPARE (
    XID         IN          DBMS_XA_XID
)RETURN PLS_INTEGER;
```

参数详解

- XID 输入参数，XA 事务标识符。

5. XA_COMMIT

根据指定的 XID 提交全局事务。

语法如下:

```
FUNCTION XA_COMMIT (
    XID             IN      DBMS_XA_XID,
    ONEPHASE        IN      BOLLEAN
)RETURN PLS_INTEGER;
```

参数详解

- XID 输入参数，XA 事务标识符。
- ONEPHASE 输入参数，是否一阶段提交。TRUE：一阶段提交；FALSE：两阶段提交。

6. XA_RECOVER

从 RM 上获取所有准备提交或者启发式完成的事务分支。

语法如下:

```
FUNCTION XA_RECOVER() RETURN DBMS_XA_XID_ARRAY;
```

返回值

RM 上所有准备提交或者启发式完成的事务分支 ID 数组。

7. XA_FORGET

通知 RM 擦除启发式提交或回滚掉的事务分支。目前该方法仅语法兼容。

语法如下:

```
FUNCTION XA_FORGET(
    XID         IN          DBMS_XA_XID
)RETURN PLS_INTEGER;
```

参数详解

- XID 输入参数，XA 事务标识符。

8. XA_ROLLBACK

通知 RM 回滚事务分支所做的修改。

语法如下:

```
FUNCTION XA_ROLLBACK(
    XID         IN          DBMS_XA_XID
)RETURN PLS_INTEGER;
```

参数详解

- XID 输入参数，XA 事务标识符。

9. XA_GETLASTOER

获取最后一个错误码。目前该方法仅语法兼容。

语法如下:

```
FUNCTION XA_GETLASTOER()RETURN PLS_INTEGER;
```

返回值

最后一个错误码。

44.4 举例说明

使用包内的过程和函数之前，如果还未创建过系统包，请先调用系统过程 SP_CREATE_SYSTEM_PACKAGES 创建系统包。

```
SP_CREATE_SYSTEM_PACKAGES(1,'DBMS_XA');

SET SERVEROUTPUT ON;  //DBMS_OUTPUT.PUT_LINE 需要设置这条语句，才能打印出消息
```

准备数据表。

```
CREATE TABLE TESTXA(C1 INT,C2 INT);
```

利用 XA_START 将当前会话与事务分支关联起来。

```
DECLARE
    TESTI INT;
    TESTXID DBMS_XA_XID := DBMS_XA_XID(16);
BEGIN
    TESTI := DBMS_XA.XA_START(TESTXID,DBMS_XA.TMNOFLAGS);
    DBMS_OUTPUT.PUT_LINE(TESTI);
    INSERT INTO TESTXA VALUES(1,1);
END;
```

执行结果如下:

```
0
```

利用 XA_END 将当前会话与事务分支取消关联。

```
DECLARE
    TESTI INT;
    TESTXID DBMS_XA_XID := DBMS_XA_XID(16);
BEGIN
    TESTI := DBMS_XA.XA_END(TESTXID,DBMS_XA.TMSUCCESS);
    DBMS_OUTPUT.PUT_LINE(TESTI);
END;
```

执行结果如下:

```
0
```

利用 XA_PREPARE 和 XA_COMMIT 两阶段提交事务。

```
DECLARE
    TESTI INT;
    TESTXID DBMS_XA_XID := DBMS_XA_XID(16);
BEGIN
    TESTI := DBMS_XA.XA_PREPARE(TESTXID);
    DBMS_OUTPUT.PUT_LINE(TESTI);
    TESTI := DBMS_XA.XA_COMMIT(TESTXID,FALSE);
    DBMS_OUTPUT.PUT_LINE(TESTI);
END;
```

执行结果如下:

```
0
0
```

上述示例返回值均为 0，即 XA_OK，表示语句执行成功，其余返回值含义可参考 44.1 常量。

45 DBMS_XMLDOM 包

DBMS_XMLDOM 包用于将 XML 数据解析成一系列结点，这些结点组成一个 DOM 树状结构。通过对这些结点的增删改操作从而达到对 XML 数据的增删改操作。

一个 XML 数据的 DOM 树形结构由各种不同类型的结点组成。

45.1 数据类型

DBMS_XMLDOM 包定义了若干 XML 数据类型：

1. DOMNODE

DOMNODE 类型结点，是其它类型结点的总称。下文的任何一种结点都属于 DOMNODE。

例 加粗的黑体字为一个 DOMNODE 结点。

```
<A>ABC</A>
```

2. DOMNAMEDNODEMAP

存放<名称-DOM 结点>的映射表，通常用于根据结点名称来检索 DOM 结点。

3. DOMNODELIST

DOM 结点链表。

4. DOMATTR

DOM 属性类型结点。

例 加粗的黑体字为一个 DOMATTR 结点。

```
<A ATTR='VALUE1'></A>
```

5. DOMCDATASECTION

DOM 数据区块类型结点。

例 加粗的黑体字为一个 DOMCDATASECTION 结点。

```
<A><![CDATA[……]]></A>
```

6. DOMCHARACTERDATA

DOM 字符串类型数据结点。

例 加粗的黑体字（省略号代表的部分）为一个 DOMCHARACTERDATA 结点。

```
<A><![CDATA[……]]></A>
```

7. DOMCOMMENT

DOM 注释类型结点。

例 加粗的黑体字为一个 DOM COMMENT 结点。

```
<A> <!-- …… --> </A>
```

8. DOMDOCUMENTFRAGMENT

DOM 分片文档类型结点。

9. DOMELEMENT

DOM 元素类型结点。

例 加粗的黑体字为一个 DOMELEMENT 结点。

```
<A>ELEMENT</A>
```

10. DOMENTITY

DOM 实体类型结点。

例 加粗的黑体字为一个 DOMENTITY 结点。

```
<A><!ENTITY XML "EXTENSIBLE MARKUP LANGUAGE"></A>
```

11. DOMENTITYREFERENCE

DOM 实体引用类型结点。

例 加粗的黑体字为一个 DOMENTITYREFERENCE 结点。

```
<!DOCTYPE DOC [<!ENTITY ENT "HELLO">]>

<DOC>&ENT;</DOC>
```

12. DOMNOTATION

DOM 符号类型结点。

例 加粗的黑体字为一个 DOMNOTATION 结点。

```
<!DOCTYPE EXAMPLE SYSTEM "EXAMPLE.DTD" [<!NOTATION ABC1 SYSTEM "GODI">]>
```

13. DOMPROCESSINGINSTRUCTION

DOM 指令类型结点。

例 加粗的黑体字为一个 DOMPROCESSINGINSTRUCTION 结点。

```
<A><?APP PRINT TEXT?></A>
```

14. DOMTEXT

DOM 文本类型结点。

例 加粗的黑体字为一个 DOM TEXT 结点。

```
<A>THIS IS TEXT NODE</A>
```

15. DOMIMPLEMENTATION

DOM 文档实现体结点。

16. DOMDOCUMENTTYPE

DOM 文档定义类型结点。

例 加粗的黑体字为一个 DOMDOCUMENTTYPE 结点。

```
<!DOCTYPE DOC [<!ENTITY ENT "HELLO">]>
```

17. DOMDOCUMENT

DOM 文档类型结点。

例 加粗的黑体字为一个 DOMDOCUMENT 结点。

```
<?XML VERSION='1.0' ?><A>ELEMENT</A>
```

45.2 相关方法

1. HASFEATURE

DBMS_XMLDOM 包是否支持指定的特性。

语法如下:

```
FUNCTION HASFEATURE(
    DI DOMIMPLEMENTATION,
    FEATURE IN VARCHAR2,
    VERSION IN VARCHAR2
) RETURN BOOLEAN;
```

参数详解

- DI

  DOMIMPLEMENTATION 类型结点。

- FEATURE

  需要检测的特性，字符串类型。如 CORE 或者 XML。

- VERSION

  XML 的版本号，字符串类型。

返回值

TRUE：支持指定特性；FALSE：不支持指定特性。

2. ISNULL

判断 XML 结点是否为空。

语法如下:

```
FUNCTION ISNULL(
    N <DOMTYPE>
) RETURN BOOLEAN;
```

参数详解

- N

  DOM 类型参数。如 DOMNAMEDNODEMAP、DOMNODELIST、DOMATTR 等。

返回值

TRUE：结点为空；FALSE：结点不为空。

3. GETNODENAME

获取结点名。

语法如下:

```
FUNCTION GETNODENAME(
    N DOMNODE
) RETURN VARCHAR2;
```

参数详解

- N

  DOMNODE 类型结点。

返回值

字符串类型的结点名。

4. GETNODEVALUE

获取结点值。

语法如下:

```
FUNCTION GETNODEVALUE (
    N DOMNODE
) RETURN VARCHAR2;
```

参数详解

- N

  DOMNODE 类型结点。

返回值

字符串类型的结点值。

5. SETNODEVALUE

设置结点值。

语法如下:

```
PROCEDURE SETNODEVALUE (
    N DOMNODE,
    NODEVALUE VARCHAR2
);
```

参数详解

- N

DOMNODE 类型结点。

- NODEVALUE

  新的结点值。

6. GETNODETYPE

获取结点类型。

语法如下:

```
FUNCTION GETNODETYPE (
    N DOMNODE
) RETURN INT;
```

参数详解

- N

  DOMNODE 类型结点。

返回值

整数形式的 XML 结点类型。

7. GETPARENTNODE

获取父结点。

语法如下:

```
FUNCTION GETPARENTNODE (
    N DOMNODE
) RETURN DOMNODE;
```

参数详解

- N

DOMNODE 类型结点。

返回值

当前 XML 结点的父结点。

8. GETCHILDNODES

获取孩子结点。

语法如下:

```
FUNCTION GETCHILDNODES (
    N DOMNODE
) RETURN DOMNODELIST;
```

参数详解

- N

  DOMNODE 类型结点。

  返回值

  当前结点的孩子结点列表。

9. GETFIRSTCHILD

获取第一个结点。

语法如下:

```
FUNCTION GETFIRSTCHILD (
    N DOMNODE
) RETURN DOMNODE;
```

参数详解

- N

DOMNODE 类型结点。

返回值

当前结点的第一个孩子结点。

10. GETLASTCHILD

获取最后一个孩子结点。

语法如下:

```
FUNCTION GETLASTCHILD (
    N DOMNODE
) RETURN DOMNODE;
```

参数详解

- N

DOMNODE 类型结点。

返回值

当前结点的最后一个孩子结点

11. GETPREVIOUSSIBLING

获取前一个兄弟结点。

语法如下:

```
FUNCTION GETPREVIOUSSIBLING (
    N DOMNODE
) RETURN DOMNODE;
```

参数详解

- N

DOMNODE 类型结点。

返回值

当前结点的前一个兄弟结点。

12. GETNEXTSIBLING

获取后一个兄弟结点。

语法如下:

```
FUNCTION GETNEXTSIBLING (
    N DOMNODE
) RETURN DOMNODE;
```

参数详解

- N

DOMNODE 类型结点。

返回值

当前结点的后一个兄弟孩子结点。

13. GETATTRIBUTES

获取结点的属性。

语法如下:

```
FUNCTION GETATTRIBUTES (
    N DOMNODE
) RETURN DOMNAMEDNODEMAP;
```

参数详解

- N

DOMNODE 类型结点。

返回值

当前结点的属性列表。

14. GETOWNERDOCUMENT

获取结点所属的文档结点。

语法如下:

```
FUNCTION GETOWNERDOCUMENT (
    N DOMNODE
) RETURN DOMDOCUMENT;
```

参数详解

- N

DOMNODE 类型结点。

返回值

当前结点的文档结点。

15. GETPREFIX

获取结点命名空间标签。

语法如下:

```
FUNCTION GETPREFIX (
    N DOMNODE
) RETURN VARCHAR2;
```

参数详解

- N

DOMNODE 类型结点。

返回值

当前结点的命名空间标签。

16. SETPREFIX

设置结点命名空间标签。

语法如下:

```
FUNCTION SETPREFIX (
    N DOMNODE,
    PREFIX VARCHAR2
);
```

参数详解

- N

  DOMNODE 类型结点。

17. INSERTBEFORE

在指定结点前插入新结点。

语法如下:

```
FUNCTION INSERTBEFORE (
    N DOMNODE,
    NEWCHILD IN DOMNODE,
    REFCHILD IN DOMNODE
) RETURN DOMNODE;
```

参数详解

- N

  DOMNODE 类型结点。

- NEWCHHILD

  新结点。

- REFCHILD

  参照结点。

  返回值

  插入结点。

18. REPLACECHILD

替换指定的结点。

语法如下:

```
FUNCTION REPLACECHILD (
    N DOMNODE,
    NEWCHILD IN DOMNODE,
    OLDCHILD IN DOMNODE
) RETURN DOMNODE;
```

参数详解

- N

  DOMNODE 类型结点。

- NEWCHILD

  新结点。

- OLDCHILD

  被替换结点。

返回值

被替换结点。

19. REMOVECHILD

删除指定的结点。

语法如下:

```
FUNCTION REMOVECHILD (
    N DOMNODE,
    OLDCHILD IN DOMNODE
) RETURN DOMNODE;
```

参数详解

- N

  DOMNODE 类型结点。

- OLDCHILD

  待删除结点。

返回值

待删除结点。

20. APPENDCHILD

追加新结点。

语法如下:

```
FUNCTION APPENDCHILD (
    N DOMNODE,
    NEWCHILD IN DOMNODE
```

```
) RETURN DOMNODE;
```

参数详解

- N

  DOMNODE 类型结点。

- NEWCHILD

  新结点。

返回值

新结点。

21. HASCHILDNODES

判断指定结点是否有孩子结点。

语法如下:

```
FUNCTION HASCHILDNODES (
    N DOMNODE
) RETURN BOOLEAN;
```

参数详解

- N

  DOMNODE 类型结点。

返回值

TRUE：有孩子结点；FALSE：没有孩子结点。

22. HASATTRIBUTES

判断指定结点是否有属性。

语法如下:

```
FUNCTION HASATTRIBUTES (
    N DOMNODE
) RETURN BOOLEAN;
```

参数详解

- N

  DOMNODE 类型结点。

返回值

TRUE：有属性；FALSE：无属性。

23. CLONENODE

拷贝指定结点。

语法如下:

```
FUNCTION CLONENODE (
    N DOMNODE,
    DEEP BOOLEAN
) RETURN DOMNODE;
```

参数详解

- N

  DOMNODE 类型结点。

- DEEP

  是否递归拷贝子孙结点。

返回值

拷贝得到的新结点。

24. WRITETOFILE

将结点格式化后，以纯文本格式写入文件，文件拓展名通常为.xml，但也可以使用其他文本类型（如.txt 等）。

语法如下:

```
PROCEDURE WRITETOFILE (
    N DOMNODE,
    FILENAME VARCHAR2(256)
);
PROCEDURE WRITETOFILE(
    DOC DOMDOCUMENT,
    FILENAME VARCHAR2(256)
);
```

参数详解

- N

  DOMNODE 类型结点。

- DOC

  DOMDOCUMENT 类型结点。

- FILENAME

  文件名，包含文件所在的目录对象名。

25. WRITETOBUFFER

将结点格式化输出到指定缓冲区中。

语法如下:

```
PROCEDURE WRITETOBUFFER (
```

```
    N DOMNODE,
    BUFFER IN OUT VARCHAR2
);
PROCEDURE WRITETOBUFFER(
    DOC DOMDOCUMENT,
    BUFFER IN OUT VARCHAR2
);
```

参数详解

- N

  DOMNODE 类型结点。

- DOC

  DOMDOCUMENT 类型结点。

- BUFFER

  缓冲区。

26. WRITETOCLOB

将结点的内容写入到 CLOB 类型变量里。

语法如下:

```
PROCEDURE WRITETOCLOB (
    N DOMNODE,
    CL IN OUT CLOB
);
PROCEDURE WRITETOCLOB(
    DOC DOMDOCUMENT,
```

```
    CL IN OUT CLOB
);
```

参数详解

- N

  DOMNODE 类型结点。

- DOC

  DOMDOCUMENT 类型结点。

- CL

  CLOB 类型数据。

27. MAKENODE

获取结点名。

语法如下:

```
FUNCTION MAKENODE(
    N <DOMTYPE>
)RETURN DOMNODE;
```

参数详解

- N

  DOM 类型参数。当前支持 DOMATTR、DOMCDATASECTION、DOMCHARACTERDATA、DOMCOMMENT、DOMDOCUMENTFRAGMENT, DOMDOCUMENTTYPE、DOMELEMENT、DOMENTITY、DOMDOCUMENT、DOMNOTATION、DOMPROCESSINGINSTRUCTION、DOMENTITYREFERENCE、DOMTEXT 类型。

返回值

DOMNODE 类型结点。

28. MAKEATTR

将 DOMNODE 类型结点转换为 DOMATTR 类型结点。

语法如下:

```
FUNCTION MAKEATTR (
    N DOMNODE
) RETURN DOMATTR;
```

参数详解

- N

  DOMNODE 类型结点。

返回值

DOMATTR 类型结点。

29. MAKECDATASECTION

将 DOMNODE 类型结点转换为 DOMCDATASECTION 类型结点。

语法如下:

```
FUNCTION MAKECDATASECTION (
    N DOMNODE
) RETURN DOMCDATASECTION;
```

参数详解

- N

  DOMNODE 类型结点。

返回值

DOMCDATASECTION 类型结点。

30. MAKECHARACTERDATA

将 DOMNODE 类型结点转换为 DOMCHARACTERDATA 类型结点。

语法如下:

```
FUNCTION MAKECHARACTERDATA (
    N DOMNODE
) RETURN DOMCHARACTERDATA;
```

参数详解

- N

  DOMNODE 类型结点。

返回值

DOMCHARACTERDATA 类型结点。

31. MAKECOMMENT

将 DOMNODE 类型结点转换为 DOMCOMMENT 类型结点。

语法如下:

```
FUNCTION MAKECOMMENT (
    N DOMNODE
) RETURN DOMCOMMENT;
```

参数详解

- N

  DOMNODE 类型结点。

返回值

DOMCOMMENT 类型结点。

32. MAKEDOCUMENTFRAGMENT

将 DOMNODE 类型结点转换为 DOMDOCUMENTFRAGMENT 类型结点。

语法如下:

```
FUNCTION MAKEDOCUMENTFRAGMENT (
    N DOMNODE
) RETURN DOMDOCUMENTFRAGMENT;
```

参数详解

- N

  DOMNODE 类型结点。

返回值

DOMDOCUMENTFRAGMENT 类型结点。

33. MAKEDOCUMENTTYPE

将 DOMNODE 类型结点转换为 DOMDOCUMENTTYPE 类型结点。

语法如下:

```
FUNCTION MAKEDOCUMENTTYPE (
    N DOMNODE
) RETURN DOMDOCUMENTTYPE;
```

参数详解

- N

  DOMNODE 类型结点

返回值

DOMDOCUMENTTYPE 类型结点。

34. MAKEELEMENT

将 DOMNODE 类型结点转换为 DOMELEMENT 类型结点。

语法如下:

```
FUNCTION MAKEELEMENT (
    N DOMNODE
) RETURN DOMELEMENT;
```

参数详解

- N

  DOMNODE 类型结点。

返回值

DOMELEMENT 类型结点。

35. MAKEENTITY

将 DOMNODE 类型结点转换为 DOMENTITY 类型结点。

语法如下:

```
FUNCTION MAKEENTITY (
    N DOMNODE
) RETURN DOMENTITY;
```

参数详解

- N

  DOMNODE 类型结点。

返回值

DOMENTITY 类型结点。

36. MAKEENTITYREFERENCE

将 DOMNODE 类型结点转换为 DOMENTITYREFERENCE 类型结点。

语法如下:

```
FUNCTION MAKEENTITYREFERENCE (
    N DOMNODE
) RETURN DOMENTITYREFERENCE;
```

参数详解

- N

  DOMNODE 类型结点。

返回值

DOMENTITYREFERENCE 类型结点。

37. MAKENOTATION

将 DOMNODE 类型结点转换为 DOMNOTATION 类型结点。

语法如下:

```
FUNCTION MAKENOTATION (
    N DOMNODE
) RETURN DOMNOTATION;
```

参数详解

- N

  DOMNODE 类型结点。

返回值

DOMNOTATION 类型结点。

38. MAKEPROCESSINGINSTRUCTION

将 DOMNODE 类型结点转换为 DOMPROCESSINGINSTRUCTION 类型结点。

语法如下:

```
FUNCTION MAKEPROCESSINGINSTRUCTION (
    N DOMNODE
) RETURN DOMPROCESSINGINSTRUCTION;
```

参数详解

- N

  DOMNODE 类型结点。

返回值

DOMPROCESSINGINSTRUCTION 类型结点。

39. MAKETEXT

将 DOMNODE 类型结点转换为 DOMTEXT 类型结点。

语法如下:

```
FUNCTION MAKETEXT (
    N DOMNODE
) RETURN DOMTEXT;
```

参数详解

- N

  DOMNODE 类型结点。

返回值

DOMTEXT 类型结点。

40. MAKEDOCUMENT

将 DOMNODE 类型结点转换为 DOMDOCUMENT 类型结点。

语法如下:

```
FUNCTION MAKEDOCUMENT (
    N DOMNODE
) RETURN DOMDOCUMENT;
```

参数详解

- N

  DOMNODE 类型结点。

返回值

DOMDOCUMENT 类型结点。

41. GETNAMESPACE

获取结点的命名空间。

语法如下:

```
PROCEDURE GETNAMESPACE (
    N DOMNODE,
    DATA IN OUT VARCHAR2
);
```

参数详解

- N

  DOMNODE 类型结点。

- DATA

结点命名空间。

42. GETLOCALNAME

获取结点名。

语法如下:

```
PROCEDURE GETLOCALNAME (
    N DOMNODE,
    DATA OUT VARCHAR2
);
```

参数详解

- N

  DOMNODE 类型结点。

- DATA

  结点名。

43. GETEXPANDEDNAME

获取结点扩展名。

语法如下:

```
PROCEDURE GETEXPANDEDNAME (
    N DOMNODE,
    DATA OUT VARCHAR2
);
```

参数详解

- N

DOMNODE 类型结点。

- DATA

  结点扩展名。

44. FREENODE

释放结点。

语法如下:

```
PROCEDURE FREENODE (
    N DOMNODE
);
```

参数详解

- N

  DOMNODE 类型结点。

45. ITEM

获取列表指定下标的结点。

语法如下:

```
FUNCTION ITEM (
    NL DOMNODELIST,
    IDX IN INT
) RETURN DOMNODE;
```

参数详解

- NL

  NODE 列表。

- IDX

  下标号。

  返回值

  指定下标的结点。

46. GETLENGTH

获取结点列表长度。

语法如下:

```
FUNCTION GETLENGTH (
    NL DOMNODELIST
) RETURN INT;
```

参数详解

- NL

  结点列表。

  返回值

  结点列表长度。

47. FREENODELIST

释放结点列表。

语法如下:

```
PROCEDURE FREENODELIST (
    NL DOMNODELIST
);
```

参数详解

- N

  DOMNODE 类型结点。

48. GETNAMEDITEM

获取指定名结点。

语法如下:

```
FUNCTION GETNAMEDITEM(
    NNM DOMNAMEDNODEMAP,
    NAME IN VARCHAR2)
RETURN DOMNODE;
```

参数详解

- NNM

  DOMNAMEDNODEMAP 类型结点。

- NAME

  结点名。

返回值

指定名的结点。

49. SETNAMEDITEM

设置新结点。

语法如下:

```
FUNCTION SETNAMEDITEM(
    NNM DOMNAMEDNODEMAP,
    ARG IN DOMNODE)
```

```
RETURN DOMNODE;
```

参数详解

- NNM

  DOMNAMEDNODEMAP 类型结点。

- ARG

  新结点。

返回值

新结点。

50. REMOVENAMEDITEM

删除指定名结点。

语法如下:

```
FUNCTION REMOVENAMEDITEM(
    NNM DOMNAMEDNODEMAP,
    NAME IN VARCHAR2)
RETURN DOMNODE;
```

参数详解

- NNM

  DOMNAMEDNODEMAP 类型结点。

- NAME

  结点名。

返回值

被删除的结点。

51. ITEM

获取指定下标的结点。

语法如下:

```
FUNCTION ITEM(
    NNM DOMNAMEDNODEMAP,
    IDX IN INT)
RETURN DOMNODE;
```

参数详解

- NNM

  DOMNAMEDNODEMAP 类型结点。

- IDX

  下标。

返回值

指定下标结点。

52. GETLENGTH

获取列表长度。

语法如下:

```
FUNCTION GETLENGTH (
    NNM DOMNAMEDNODEMAP
) RETURN INT;
```

参数详解

- NNM

  DOMNAMEDNODEMAP 类型结点。

返回值

列表长度。

53. GETDATA

获取结点数据。

语法如下:

```
FUNCTION GETDATA (
    CD DOMCHARACTERDATA
) RETURN VARCHAR2;
```

参数详解

- CD

  DOMCHARACTERDATA 类型结点。

返回值

结点数据。

54. SETDATA

为结点设置新数据。

语法如下:

```
PROCEDURE SETDATA(
    CD DOMCHARACTERDATA,
    DATA IN VARCHAR2
);
```

参数详解

- CD

DOMCHARACTERDATA 类型结点。

- DATA

  新数据。

55. GETLENGTH

获取 DOMCHARACTERDATA 类型数据长度。

语法如下:

```
FUNCTION GETLENGTH(
    CD DOMCHARACTERDATA
) RETURN INT;
```

参数详解

- CD

  DOMCHARACTERDATA 类型结点。

返回值

字符类型数据长度。

56. SUBSTRINGDATA

取指定位置下的字符串。

语法如下:

```
FUNCTION SUBSTRINGDATA (
    CD DOMCHARACTERDATA,
    OFFSET IN INT,
    CNT IN INT
) RETURN VARCHAR2;
```

参数详解

- CD

  DOMCHARACTERDATA 类型结点。

- OFFSET

  偏移。

- CNT

  字符数量。

返回值

字符串子串。

57. APPENDDATA

追加字符串数据。

语法如下:

```
PROCEDURE APPENDDATA(
    CD DOMCHARACTERDATA,
    ARG IN VARCHAR2
);
```

参数详解

- CD

  DOMCHARACTERDATA 类型结点。

- ARG

  追加的字符串。

58. INSERTDATA

插入新字符串。

语法如下:

```
PROCEDURE INSERTDATA(
    CD DOMCHARACTERDATA,
    OFFSET IN INT,
    ARG IN VARCHAR2
);
```

参数详解

- CD

  DOMCHARACTERDATA 类型结点。

- OFFSET

  偏移位置。

- ARG

  插入的字符串。

59. DELETEDATA

删除指定位置下的字符串。

语法如下:

```
PROCEDURE DELETEDATA(
    CD DOMCHARACTERDATA,
    OFFSET IN INT,
    CNT IN INT
);
```

参数详解

- CD

  DOMCHARACTERDATA 类型结点。

- OFFSET

  偏移位置。

- CNT

  删除字符数量。

60. REPLACEDATA

替换字符数据。

语法如下:

```
PROCEDURE REPLACEDATA(
    CD DOMCHARACTERDATA,
    OFFSET IN INT,
    CNT IN INT,
    ARG IN VARCHAR2
);
```

参数详解

- CD

  DOMCHARACTERDATA 类型结点。

- OFFSET

  偏移位置。

- CNT

  替换字符数量。

- ARG

新字符串。

61. GETNAME

获取属性名。

语法如下:

```
FUNCTION GETNAME (
    A DOMATTR
) RETURN VARCHAR2;
```

参数详解

- A

  DOMATTR 类型结构。

返回值

属性名。

62. GETOWNERELEMENT

获取属性所属元素类型结点。

语法如下:

```
FUNCTION GETOWNERELEMENT (
    A DOMATTR
) RETURN DOMELEMENT;
```

参数详解

- A

  DOMATTR 类型结构。

返回值

元素类型结点。

63. GETVALUE

获取属性值。

语法如下:

```
FUNCTION GETVALUE (
    A DOMATTR
) RETURN VARCHAR2;
```

参数详解

- A

  DOMATTR 类型结构。

返回值

属性值。

64. SETVALUE

设置属性值。

语法如下:

```
PROCEDURE SETVALUE (
    A DOMATTR,
    NEWVALUE IN VARCHAR2
);
```

参数详解

- A

  DOMATTR 类型结构。

- NEWVALUE

  新的属性值。

65. GETQUALIFIEDNAME

获取属性全名。

语法如下:

```
FUNCTION GETQUALIFIEDNAME (
    A DOMATTR
) RETURN VARCHAR2;
```

参数详解

- A

  DOMATTR 类型结构。

返回值

字符串类型属性全名。

66. GETNAMESPACE

获取属性的命名空间。

语法如下:

```
FUNCTION GETNAMESPACE (
    A DOMATTR
) RETURN VARCHAR2;
```

参数详解

- A

  DOMATTR 类型结构

返回值

属性的命名空间。

67. GETLOCALNAME

获取属性名。

语法如下:

```
FUNCTION GETLOCALNAME (
    A DOMATTR
) RETURN VARCHAR2;
```

参数详解

- A

  DOMATTR 类型结构。

返回值

属性名。

68. GETEXPANDEDNAME

获取属性的扩展名。

语法如下:

```
FUNCTION GETEXPANDEDNAME (
    A DOMATTR
) RETURN VARCHAR2;
```

参数详解

- A

  DOMATTR 类型结构。

返回值

属性扩展名。

69. GETTAGNAME

获取元素结点名。

语法如下:

```
FUNCTION GETTAGNAME(
    ELEM DOMELEMENT
)RETURN VARCHAR2;
```

参数详解

- ELEM

  DOMELEMENT 类型结点。

返回值

元素结点名。

70. GETATTRIBUTE

获取元素的属性值。

语法如下:

```
FUNCTION GETATTRIBUTE(
    ELEM DOMELEMENT,
    NAME IN VARCHAR2
)RETURN VARCHAR2;
FUNCTION GETATTRIBUTE(
    ELEM DOMELEMENT,
```

```
    NAME IN VARCHAR2,
    NS IN VARCHAR2
)RETURN VARCHAR2;
```

参数详解

- N

  DOMNODE 类型结点。

- NAME

  属性名。

- NS

  命名空间。

返回值

元素属性值。

71. HASATTRIBUTE

元素结点是否有指定属性。

语法如下:

```
FUNCTION HASATTRIBUTE(
    ELEM DOMELEMENT,
    NAME IN VARCHAR2
)RETURN BOOLEAN;
FUNCTION HASATTRIBUTE(
    ELEM DOMELEMENT,
    NAME IN VARCHAR2,
    NS IN VARCHAR2
```

```
)RETURN BOOLEAN;
```

参数详解

- ELEM

  DOMELEMENT 类型结点。

- NAME

  属性名。

- NS

  命名空间。

返回值

TRUE：有指定属性；FALSE：指定属性不存在。

72. SETATTRIBUTE

设置属性。

语法如下:

```
PROCEDURE SETATTRIBUTE(
    ELEM DOMELEMENT,
    NAME IN VARCHAR2,
    NEWVALUE IN VARCHAR2
);
PROCEDURE SETATTRIBUTE(
    ELEM DOMELEMENT,
    NAME IN VARCHAR2,
    NEWVALUE IN VARCHAR2,
    NS IN VARCHAR2
```

```
);
```

参数详解

- ELEM

  DOMELEMENT 类型结点。

- NAME

  属性名。

- NEWVALUE

  新的属性值。

- NS

  命名空间。

73. REMOVEATTRIBUTE

移除指定属性。

语法如下:

```
PROCEDURE REMOVEATTRIBUTE(
    ELEM DOMELEMENT,
    NAME IN VARCHAR2
);
PROCEDURE REMOVEATTRIBUTE(
    ELEM DOMELEMENT,
    NAME IN VARCHAR2,
    NS IN VARCHAR2
);
```

参数详解

- ELEM

  DOMELEMENT 类型结点。

- NAME

  属性名。

- NS

  命名空间。

74. GETATTRIBUTENODE

获取指定的属性结点。

语法如下:

```
FUNCTION GETATTRIBUTENODE(
    ELEM DOMELEMENT,
    NAME IN VARCHAR2
)RETURN DOMATTR;
FUNCTION GETATTRIBUTENODE(
    ELEM DOMELEMENT,
    NAME IN VARCHAR2,
    NS IN VARCHAR2
)RETURN DOMATTR;
```

参数详解

- ELEM

  DOMELEMENT 类型结点。

- NAME

  属性名。

- NS

  命名空间。

  返回值

  属性结点。

75. SETATTRIBUTENODE

将属性结点设置到元素结点上。

语法如下:

```
FUNCTION SETATTRIBUTENODE(
    ELEM DOMELEMENT,
    NEWATTR IN DOMATTR
)RETURN DOMATTR;
FUNCTION SETATTRIBUTENODE(
    ELEM DOMELEMENT,
    NEWATTR IN DOMATTR,
    NS IN VARCHAR2
)RETURN DOMATTR;
```

参数详解

- ELEM

  DOMELEMENT 类型结点。

- NEWATTR

  新的属性结点。

- NS

  命名空间。

返回值

新设置的属性结点。

76. REMOVEATTRIBUTENODE

移除元素结点上的属性结点。

语法如下:

```
FUNCTION REMOVEATTRIBUTENODE(
    ELEM DOMELEMENT,
    OLDATTR IN DOMATTR
)RETURN DOMATTR;
```

参数详解

- ELEM

  DOMELEMENT 类型结点。

- OLDATTR

  待移除的属性结点。

返回值

移除的属性结点。

77. GETELEMENTSBYTAGNAME

根据元素结点名查找元素结点。

语法如下:

```
FUNCTION GETELEMENTSBYTAGNAME(
    ELEM DOMELEMENT,
    NAME IN VARCHAR2
```

```
)RETURN DOMNODELIST;

FUNCTION GETELEMENTSBYTAGNAME(

    DOC DOMDOCUMENT,

    NAME IN VARCHAR2

)RETURN DOMNODELIST;
```

参数详解

- ELEM

  DOMELEMENT 类型结点。

- DOC

  DOMDOCUMENT 类型结点。

- NAME

  元素结点名。

返回值

元素结点列表。

78. GETCHILDRENBYTAGNAME

获取元素结点下的孩子结点。

语法如下:

```
FUNCTION GETCHILDRENBYTAGNAME(

    ELEM DOMELEMENT,

    NAME VARCHAR2

)RETURN DOMNODELIST;

FUNCTION GETCHILDRENBYTAGNAME(

    ELEM DOMELEMENT,
```

```
    NAME VARCHAR2,
    NS VARCHAR2
)RETURN DOMNODELIST;
```

参数详解

- ELEM

  DOMELEMENT 类型结点。

- NAME

  结点名。

- NS

  命名空间。

返回值

孩子结点列表。

79. FREEELEMENT

释放元素结点。

语法如下:

```
PROCEDURE FREEELEMENT(
    ELEM DOMELEMENT
);
```

参数详解

- ELEM

  DOMELEMENT 类型结点。

80. SPLITTEXT

将指定文本结点分隔成两个文本结点。

语法如下:

```
FUNCTION SPLITTEXT(
    T DOMTEXT,
    OFFSET IN INT
)RETURN DOMTEXT;
```

参数详解

- T

  DOMTEXT 类型结点。

- OFFSET

  分割位置。

返回值

分割后的文本结点。

81. GETDOCTYPE

获取文档的定义描述结构。

语法如下:

```
FUNCTION GETDOCTYPE(
    DOC DOMDOCUMENT
)RETURN DOMDOCUMENTTYPE;
```

参数详解

- DOC

  DOMDOCUMENT 类型结点。

返回值

文档定义描述结构。

82. NEWDOMDOCUMENT

构建 XML 文档结点。

语法如下:

```
FUNCTION NEWDOMDOCUMENT RETURN DOMDOCUMENT;或

FUNCTION NEWDOMDOCUMENT(

    XMLDOC IN VARCHAR2

)RETURN DOMDOCUMENT;

FUNCTION NEWDOMDOCUMENT

    CL IN CLOB)

)RETURN DOMDOCUMENT;
```

参数详解

- XMLDOC

  VARCHAR 类型的 XML 数据。

- CL

  CLOB 类型的 XML 数据。

返回值

DOMDOCUMENT 类型结点。

83. GETDOCUMENTELEMENT

获取文档的根元素。

语法如下:

```
FUNCTION GETDOCUMENTELEMENT(
```

```
    DOC DOMDOCUMENT

)RETURN DOMELEMENT;
```

参数详解

- DOC

  DOMDOCUMENT 类型结点。

返回值

文档根元素结点。

84. CREATEELEMENT

创建元素结点。

语法如下:

```
FUNCTION CREATEELEMENT(

    DOC DOMDOCUMENT,

    TAGNAME IN VARCHAR2

)RETURN DOMELEMENT;

FUNCTION CREATEELEMENT(

    DOC DOMDOCUMENT,

    TAGNAME IN VARCHAR2,

    NS IN VARCHAR2

)RETURN DOMELEMENT;
```

参数详解

- DOC

  DOMDOCUMENT 类型结点。

- TAGNAME

元素结点名。

- NS

  命名空间。

  返回值

  元素类型结点。

85. CREATETEXTNODE

创建文本类型结点。

语法如下:

```
FUNCTION CREATETEXTNODE(
    DOC DOMDOCUMENT,
    DATA IN VARCHAR2
)RETURN DOMTEXT;
```

参数详解

- DOC

  DOMDOCUMENT 类型结点。

- DATA

  文本内容。

  返回值

  文本类型结点。

86. CREATECOMMENT

创建注释结点。

语法如下:

```
FUNCTION CREATECOMMENT(
    DOC DOMDOCUMENT,
    DATA IN VARCHAR2
)RETURN DOMCOMMENT;
```

参数详解

- DOC

  DOMDOCUMENT 类型结点。

- DATA

  注释内容。

返回值

注释类型结点。

87. CREATECDATASECTION

创建字符区类型结点。

语法如下:

```
FUNCTION CREATECDATASECTION(
    DOC DOMDOCUMENT,
    DATA IN VARCHAR2
)RETURN DOMCDATASECTION;
```

参数详解

- DOC

  DOMDOCUMENT 类型结点。

- DATA

  字符区内容。

返回值

字符区类型结点。

88. CREATEPROCESSINGINSTRUCTION

创建指令类型结点。

语法如下:

```
FUNCTION CREATEPROCESSINGINSTRUCTION(
    DOC DOMDOCUMENT,
    TARGET IN VARCHAR2,
    DATA IN VARCHAR2
)RETURN DOMPROCESSINGINSTRUCTION;
```

参数详解

- DOC

  DOMDOCUMENT 类型结点。

- TARGET

  指令结点名。

- DATA

  指令结点内容。

返回值

指令类型结点。

89. CREATEATTRIBUTE

创建属性类型结点。

语法如下:

```
FUNCTION CREATEATTRIBUTE(
    DOC DOMDOCUMENT,
    NAME IN VARCHAR2
)RETURN DOMATTR;
```

参数详解

- DOC

  DOMDOCUMENT 类型结点。

- NAME

  属性名。

返回值

属性类型结点。

90. FREEDOCUMENT

释放文档类型结点。

语法如下:

```
PROCEDURE FREEDOCUMENT(
    DOC DOMDOCUMENT
);
```

参数详解

- DOC

  DOMDOCUMENT 类型结点。

91. FREEDOCTYPE

释放文档定义描述类型结点。

语法如下:

```
PROCEDURE FREEDOCTYPE(
    DT DOMDOCUMENTTYPE
);
```

参数详解

- DT

  DOMDOCUMENTTYPE 类型结点。

92. IMPORTNODE

向指定文档中导入新结点，与 ADOPTNODE 接口相比，IMPORTNODE 是拷贝新结点到指定 DOC 上。

语法如下:

```
FUNCTION IMPORTNODE(
    DOC                 DOMDOCUMENT,
    IMPORTEDNODE        DOMNODE,
    DEEP                BOOLEAN
) RETURN DOMNODE;
```

参数详解

- DOC

  DOCUMENT 文档结点。

- IMPORTEDNODE

  待导入的结点。

- DEEP

  是否级联导入孩子结点。

93. ADOPTNODE

用于收养新结点，与 IMPORTNODE 的区别在于，ADOPTNODE 会将结点从源 DOC 中摘除。

语法如下:

```
FUNCTION ADOPTNODE(
    DOC                 DOMDOCUMENT,
    ADOPTEDNODE         DOMNODE
) RETURN DOMNODE;
```

参数详解

- DOC

  DOCUMENT 文档结点。

- ADOPTNODE

  待收养的结点。

94. CREATEDOCUMENTFRAGMENT

将 DOMDOCUMENT 类型结点转换为 DOMDOCUMENTFRAGMENT 类型结点。

语法如下:

```
FUNCTION CREATEDOCUMENTFRAGMENT(
    DOC                 DOMDOCUMENT
) RETURN DOMDOCUMENTFRAGMENT;
```

参数详解

- DOC

  DOCUMENT 文档结点。

返回值

DOMDOCUMENTFRAGMENT 类型结点。

45.3 举例说明

使用包内的过程和函数之前，如果还未创建过系统包，请先调用系统过程创建系统包。

```
SP_CREATE_SYSTEM_PACKAGES (1,'DBMS_XMLDOM');
```

以下是对 XML 数据进行解析，生成 DOM 结构，并对 DOM 类型数据进行操作。

```
DECLARE
SRC     VARCHAR(1000);
BUF     VARCHAR(1000);
DOC  DBMS_XMLDOM.DOMDOCUMENT;
ELE     DBMS_XMLDOM.DOMELEMENT;
NODE       DBMS_XMLDOM.DOMNODE;
SUBNODE         DBMS_XMLDOM.DOMNODE;
LST     DBMS_XMLDOM.DOMNODELIST;
ATTR    DBMS_XMLDOM.DOMATTR;
BEGIN
SRC := '<A ATTR1="VAL1">TXT<B>SUB TXT</B></A>';
//解析并生成生成文档结构
DOC := DBMS_XMLDOM.NEWDOMDOCUMENT(SRC);
//获取根元素结点
ELE := DBMS_XMLDOM.GETDOCUMENTELEMENT(DOC);
//获取元素的 ATTR1 属性
ATTR := DBMS_XMLDOM.GETATTRIBUTENODE(ELE, 'ATTR1');
```

```
DBMS_OUTPUT.PUT_LINE('ATTR VALUE IS ' || DBMS_XMLDOM.GETVALUE(ATTR));
//获取 ELE 的孩子结点列表
NODE := DBMS_XMLDOM.MAKENODE(ELE);
LST := DBMS_XMLDOM.GETCHILDNODES(NODE);
DBMS_OUTPUT.PUT_LINE('LIST LEN IS ' || DBMS_XMLDOM.GETLENGTH(LST));
//第一个孩子结点为文本结点
SUBNODE := DBMS_XMLDOM.ITEM(LST, 0);
DBMS_OUTPUT.PUT_LINE('NODE VALUE IS ' || DBMS_XMLDOM.GETNODEVALUE(SUBNODE));
//第二个孩子结点为元素结点
SUBNODE := DBMS_XMLDOM.ITEM(LST, 1);
DBMS_XMLDOM.WRITETOBUFFER(SUBNODE, BUF);
DBMS_OUTPUT.PUT_LINE(BUF);
//获取第二个孩子结点的信息
IF DBMS_XMLDOM.HASCHILDNODES(SUBNODE) = TRUE THEN
  DBMS_OUTPUT.PUT_LINE('NODE HAS CHILD');
  SUBNODE := DBMS_XMLDOM.GETFIRSTCHILD(SUBNODE);
  DBMS_OUTPUT.PUT_LINE('SUB NODE VALUE IS ' || DBMS_XMLDOM.GETNODEVALUE(SUBNODE));
END IF;
//释放文档
DBMS_XMLDOM.FREEDOCUMENT(DOC);
END;
/
```

46 DBMS_DEBUG 包

DBMS_DEBUG 包用于调试 PL/SQL 语句块。利用 DBMS_DEBUG 系统包调试 PL/SQL 时，至少需要两个会话，一个会话用于执行目标程序，另一个会话用于调试目标程序。

DBMS_DEBUG 系统包调试 PL/SQL 的主要步骤如下:

1. 初始化会话 1;
2. 会话 1 打开调试模式;
3. 会话 1 执行目标程序，目标程序阻塞;
4. 会话 2 启动调试;
5. 会话 2 设置断点;
6. 会话 2 执行单步调试，直至目标程序执行结束;
7. 会话 1 和会话 2 结束调试。

46.1 相关方法

1. PROBE_VERSION

DBMS_DEBUG 系统包的版本号。

语法如下:

```
PROCEDURE PROBE_VERSION(
    MAJOR OUT INTEGER,
    MINOR OUT INTEGER
);
```

参数详解

- MAJOR

主要版本号。

- MINOR

  次要版本号。随着功能的增加而迭代。

  返回值

  无

2. INITIALIZE

初始化当前会话以进行调试。

语法如下:

```
FUNCTION INITIALIZE(
    DEBUG_SESSION_ID    IN VARCHAR2     := NULL,
    DIAGNOSTICS         IN INTEGER      := 0,
    DEBUG_ROLE          IN VARCHAR2     := NULL,
    DEBUG_ROLE_PWD      IN VARCHAR2     := NULL
) RETURN VARCHAR2;
```

参数详解

- DEBUG_SESSION_ID

  指定调试会话 ID。

- DIAGNOSTICS

  暂不支持。

- DEBUG_ROLE

  暂不支持。

- DEBUG_ROLE_PWD

  暂不支持。

返回值

新注册的调试会话 ID，返回值与 DEBUG_SESSION_ID 相同。若 DEBUG_SESSION_ID 为 NULL，则自动生成唯一的调试会话 ID，不同会话返回值不同，相同会话返回值相同。

3. DEBUG_ON

打开调试模式。

语法如下:

```
PROCEDURE DEBUG_ON(
    NO_CLIENT_SIDE_PLSQL_ENGINE  BOOLEAN          := TRUE,
    IMMEDIATE                    BOOLEAN          := FALSE
);
```

参数详解

- NO_CLIENT_SIDE_PLSQL_ENGINE

  暂不支持。

- IMMEDIATE

  暂不支持。

返回值

无

4. DEBUG_OFF

关闭调试模式。

语法如下:

```
PROCEDURE DEBUG_OFF;
```

参数详解

无

返回值

无

5. ATTACH_SESSION

将待调试目标程序与当前会话相关联，启动调试。

语法如下:

```
PROCEDURE ATTACH_SESSION(
    DEBUG_SESSION_ID      IN    VARCHAR2,
    DIAGNOSTICS           IN    INTEGER  := 0
);
```

参数详解

- DEBUG_SESSION_ID

待调试目标程序所在的调试会话 ID。即会话使用 DBMS_DEBUG.INITIALIZE 进行初始化时的返回值。

- DIAGNOSTICS

暂不支持。

返回值

无

6. DETACH_SESSION

结束调试。

语法如下:

```
PROCEDURE DETACH_SESSION;
```

参数详解

无

返回值

无

7. SET_BREAKPOINT

设置断点。若指定行无法设置断点，则在距离指定行最近的行设置断点。若同一行重复设置断点，则只会添加一个断点。

语法如下:

```
FUNCTION SET_BREAKPOINT(
    PROGRAM          IN    PROGRAM_INFO,
    LINE#            IN    INTEGER,
    BREAKPOINT#  OUT       INTEGER,
    FUZZY            IN    INTEGER := 0,
    ITERATIONS       IN    INTEGER := 0
) RETURN INTEGER;
```

参数详解

- PROGRAM

  仅支持输入 NULL。

- LINE#

  指定需要设置断点的行号。

- BREAKPOINT#

  输出断点编号，断点从 1 开始依次编号。

- FUZZY

暂不支持。

- ITERATIONS

暂不支持。

返回值

返回值为 0 表示成功执行；为 12 表示不能在找到指定行设置断点，如：行号过大、加密文本解密失败；为 14 表示无法设置断点，如：对象不存在；为 16 表示其它错误导致无法设置断点。

8. "CONTINUE"

单步运行目标程序执行调试。需要注意的是，该函数名必须为大写。

语法如下：

```
FUNCTION "CONTINUE"(
    RUN_INFO            IN OUT      RUNTIME_INFO,
    BREAKFLAGS          IN          INTEGER,
    INFO_REQUESTED      IN          INTEGER := NULL
) RETURN INTEGER;
```

参数详解

- RUN_INFO

程序运行相关信息。数据类型为 DBMS_DEBUG.RUNTIME_INFO。目前 DBMS_DEBUG.RUNTIME_INFO 中包含四个属性，分别为：Breakpoint：断点编号；Line#：语句行号；Program.Owner：调用语句块所属模式名；Program.Name：调用语句块名称。

- BREAKFLAGS

指定目标程序单步执行方式。可取值：0：表示依次执行，若遇到断点则会停靠，否则依次执行至完成；DBMS_DEBUG.break_next_line：等同于 32，表示单步执行，不管是否

遇到断点，会停靠在下一行可以停靠的位置，单步调用；DBMS_DEBUG.break_any_call：等同于 12，表示在目标程序语句的下一行中断，分步调用。

- INFO_REQUESTED

指定目标程序停止时需要返回的信息。可取值 DBMS_DEBUG.info_getBreakpoint：获取断点编号；DBMS_DEBUG.info_getLineinfo：获取行号；DBMS_DEBUG.info_getStackDepth：获取堆栈的当前深度；DBMS_DEBUG.info_getOerInfo：获取当前语句块相关信息。支持同时指定多个值，不同值之间使用“+”相连。

返回值

单步执行完成，返回 DBMS_DEBUG.SUCCESS。

46.2 举例说明

使用包内的过程和函数之前，如果还未创建过系统包，请先调用系统过程创建系统包。

```
SP_CREATE_SYSTEM_PACKAGES (1,'DBMS_DEBUG');
```

用户可调用 PROBE_VERSION 过程查看 DBMS_DEBUG 系统包的当前版本。

```
//DBMS_OUTPUT.PUT_LINE 需要设置这条语句，才能打印出消息
SET SERVEROUTPUT ON



//查看 DBMS_DEBUG 系统包的当前版本
DECLARE
    MAJOR INTEGER;
    MINOR INTEGER;
BEGIN
    DBMS_DEBUG.PROBE_VERSION(MAJOR,MINOR);
```

```
    DBMS_OUTPUT.PUT_LINE('PROBE VERSION IS: ' || MAJOR || '.' || MINOR);
END;
/


//打印信息如下，表示 DBMS_DEBUG 系统包的当前版本为 2.0
PROBE VERSION IS: 2.0
```

例 1 调试目标程序，调试过程中 BREAKFLAGS=0。

步骤一 创建函数 SYSDBA.F1，以便在目标程序中进行调用。

```
CREATE OR REPLACE FUNCTION SYSDBA.F1(C1 INT) RETURN INT
AS
    A INT;
    B INT;
BEGIN
    A = C1 * C1;
    B = C1 + C1;
    RETURN A+B;
END;
/
```

步骤二 创建会话 1 并初始化，然后打开调试模式。

```
//DBMS_OUTPUT.PUT_LINE 需要设置这条语句，才能打印出消息
SET SERVEROUTPUT ON


//初始化会话
```

```
SELECT DBMS_DEBUG.INITIALIZE FROM DUAL;

//返回值

行号        INITIALIZE

---------- --------------

1               0x7f18fc011278



EXEC DBMS_DEBUG.DEBUG_ON;
```

步骤三 会话 1 执行目标程序，此时目标程序阻塞。

```
DECLARE

V1 INT;

V2 INT;

BEGIN

      V1 := 10;

      V2 := SYSDBA.F1(V1);

      DBMS_OUTPUT.PUT_LINE(V2);

END;

/
```

步骤四 创建会话 2，执行 DBMS_DEBUG.ATTACH_SESSION 启动调试。

```
//DBMS_OUTPUT.PUT_LINE 需要设置这条语句，才能打印出消息

SET SERVEROUTPUT ON



//需要输入会话 1 执行 DBMS_DEBUG.INITIALIZE 时的返回值

CALL DBMS_DEBUG.ATTACH_SESSION(?);

请输入参数 1 的值:0x7f18fc011278
```

步骤五 会话 2 设置断点。

```
//在目标程序的第 5 行设置断点
DECLARE
    BP INTEGER;
    RET INTEGER;
BEGIN
    RET := DBMS_DEBUG.SET_BREAKPOINT(NULL, ?,  BP);
    DBMS_OUTPUT.PUT_LINE('Ret := '||RET);
    IF RET = DBMS_DEBUG.SUCCESS THEN
        DBMS_OUTPUT.PUT_LINE('Breakpoint := '||BP);
    END IF;
END;
/
请输入参数 1 的值:5

//打印信息如下，表示第一个断点设置成功
Ret := 0
Breakpoint := 1

//在目标程序的第 6 行设置断点
DECLARE
    BP INTEGER;
    RET INTEGER;
BEGIN
```

```
    RET := DBMS_DEBUG.SET_BREAKPOINT(NULL, ?,  BP);
    DBMS_OUTPUT.PUT_LINE('Ret := '||RET);
    IF RET = DBMS_DEBUG.SUCCESS THEN
        DBMS_OUTPUT.PUT_LINE('Breakpoint := '||BP);
    END IF;
END;
/
请输入参数 1 的值:6


//打印信息如下，表示第二个断点设置成功
Ret := 0
Breakpoint := 2
```

步骤六 会话 2 运行目标程序执行调试，调试过程中依次执行语句，若遇到断点则会停靠。

```
//第 1 次执行
DECLARE
    RUN_INFO DBMS_DEBUG.RUNTIME_INFO;
    RET INTEGER;
BEGIN
    RET := DBMS_DEBUG."CONTINUE"(RUN_INFO, 0,
    0
    + DBMS_DEBUG.info_getBreakpoint
    + DBMS_DEBUG.info_getLineinfo);
    DBMS_OUTPUT.PUT_LINE('Ret := '||RET);
```

```
    IF RET = DBMS_DEBUG.SUCCESS THEN

        DBMS_OUTPUT.PUT_LINE('Breakpoint := '||RUN_INFO.Breakpoint);

        DBMS_OUTPUT.PUT_LINE('Line# := '||RUN_INFO.Line#);

    END IF;

END;

/

//打印信息如下，表示命中第 1 个断点，行号为 5

Ret := 0

Breakpoint := 1

Line# := 5



//第 2 次执行，语句同上，打印信息如下，表示命中第 2 个断点，行号为 6

Ret := 0

Breakpoint := 2

Line# := 6



//第 3 次执行，语句同上，打印信息如下，表示没有命中断点或某一行

Ret := 16

//此时切换到会话 1 会发现目标程序已经执行完成
```

步骤七 会话 1 和会话 2 结束调试。

```
// 会话 1 关闭调试模式，此时语句会阻塞

EXEC DBMS_DEBUG.DEBUG_OFF;



//会话 2 执行"CONTINUE"，唤醒会话 1 继续执行
```

```
DECLARE
        RUN_INFO DBMS_DEBUG.RUNTIME_INFO;
        RET INTEGER;
BEGIN
        RET := DBMS_DEBUG."CONTINUE"(RUN_INFO, 0,   0);
        DBMS_OUTPUT.PUT_LINE('Ret := '||RET);
END;
/
//打印信息如下，表示没有命中断点或某一行，此时切换到会话 1 会发现目标程序已经执行完成
Ret := 28


//会话 2 结束调试
EXEC DBMS_DEBUG.DETACH_SESSION;
```

例 2 调试目标程序，调试过程中进行分步调用。

除步骤六外，其余步骤与例 1 相同，用户可参考例 1 中的步骤介绍。

若在调试过程中进行分步调用，则会话 2 单步运行目标程序执行调试的过程如下：

```
//第 1 次单步执行，碰到调用时执行跳入，并输出当前调用语句块名称以及所属模式名
DECLARE
      RUN_INFO DBMS_DEBUG.RUNTIME_INFO;
      RET INTEGER;
BEGIN
      RET := DBMS_DEBUG."CONTINUE"(RUN_INFO,
      DBMS_DEBUG.break_any_call,
```

```
    0
    + DBMS_DEBUG.info_getBreakpoint
    + DBMS_DEBUG.info_getLineinfo
    + DBMS_DEBUG.info_getStackDepth
    + DBMS_DEBUG.info_getOerInfo);
    DBMS_OUTPUT.PUT_LINE('Ret := '||RET);
    IF RET = DBMS_DEBUG.SUCCESS THEN
        DBMS_OUTPUT.PUT_LINE('Breakpoint := '||RUN_INFO.Breakpoint);
        DBMS_OUTPUT.PUT_LINE('Line# := '||RUN_INFO.Line#);
        IF RUN_INFO.Program.Owner IS NOT NULL THEN
            DBMS_OUTPUT.PUT_LINE('Owner := '||RUN_INFO.Program.Owner);
        END IF;
        IF  RUN_INFO.Program.Name IS NOT NULL THEN
            DBMS_OUTPUT.PUT_LINE('Name := '||RUN_INFO.Program.Name);
        END IF;
    END IF;
END;
/
//打印信息如下，表示命中第 1 个断点，行号为 5
Ret := 0
Breakpoint := 1
Line# := 5

//第 2 次单步执行，语句同上，打印信息如下，表示命中第 2 个断点，行号为 6
```

```
Ret := 0

Breakpoint := 2

Line# := 6



//第 3 次单步执行，语句同上，此时执行跳入，打印信息如下，表示执行到 SYSDBA.F1 的第 6 行

Ret := 0

Breakpoint :=

Line# := 6

Owner := SYSDBA

Name := F1



//第 4 次单步执行，语句同上，打印信息如下，表示执行到 SYSDBA.F1 的第 7 行

Ret := 0

Breakpoint :=

Line# := 7

Owner := SYSDBA

Name := F1



//第 5 次单步执行，语句同上，打印信息如下，表示执行到 SYSDBA.F1 的第 8 行

Ret := 0

Breakpoint :=

Line# := 8

Owner := SYSDBA

Name := F1
```

```
//第 6 次单步执行，语句同上，此时跳出调用，执行到目标程序的第 7 行
Ret := 0
Breakpoint :=
Line# := 7

//第 7 次单步执行，语句同上，打印信息如下，表示没有命中断点或某一行
Ret := 16
//此时切换到会话 1 会发现目标程序已经执行完成
```

47 ODCICONST 包

ODCICONST 包中定义了一系列适用于 DMSQL 程序的常量。

47.1 相关常量

下表中列出了 ODCICONST 系统包中的常量。

表 49.1 ODCICONST 系统包中的常量

常量	值	常量	值	常量	值
Success	0	RowOption	2	FirstCall	1
Error	1	EstimateStats	1	IntermediateCall	2
Warning	2	ComputeStats	2	FinalCall	3
ErrContinue	3	Validate	4	RebuildIndex	4
Fatal	4	AlterIndexNone	0	RebuildPMO	5
PredExactMatch	1	AlterIndexRename	1	StatsGlobal	6
PredPrefixMatch	2	AlterIndexRebuild	2	StatsGlobalAndPartition	7
PredIncludeStart	4	AlterIndexRebuildOnline	3	StatsPartition	8
PredIncludeStop	8	AlterIndexModifyCol	4	FetchOp	1
PredObjectFunc	16	AlterIndexUpdBlockRefs	5	PopulateOp	2
PredObjectPkg	32	AlterIndexRenameCol	6	Sample	1
PredObjectType	64	AlterIndexRenameTab	7	SampleBlock	2
PredMultiTable	128	AlterIndexMigrate	8	TRUE	1
PredNotEqual	256	Local	1	FALSE	0

QueryFirstRows	1	RangePartn	2	QueryCoordinator	1
QueryAllRows	2	HashPartn	4	Shadow	2
QuerySortAsc	4	Online	8	Slave	4
QuerySortDesc	8	Parallel	16	FetchEOS	1
QueryBlocking	16	Unusable	32	CompFilterByCol	1
CleanupCall	1	IndexOnIOT	64	CompOrderByCol	2
RegularCall	2	TransTblspc	128	CompOrderDscCol	4
ObjectFunc	1	FunctionIdx	256	CompUpdatedCol	8
ObjectPkg	2	ListPartn	512	CompRenamedCol	16
ObjectType	4	UpdateGlobalIndexes	1024	CompRenamedTopADT	32
ArgOther	1	DefaultDegree	32767	ColumnExpr	1
ArgCol	2	DebuggingOn	1	AncOpExpr	2
ArgLit	3	NoData	2	SortAsc	1
ArgAttr	4	UserParamString	4	SortDesc	2
ArgNull	5	RowMigration	8	NullsFirst	4
ArgCursor	6	IndexKeyChanged	16	AddPartition	1
PercentOption	1	None	0	DropPartition	2

47.2 举例说明

使用包内的过程和函数之前，如果还未创建过系统包，请先调用系统过程创建系统包。

```
SP_CREATE_SYSTEM_PACKAGES(1,'ODCICONST');
```

举一个使用 ODCICONST 包中常量 SUCCESS 的例子。

```
CREATE OR REPLACE FUNCTION TEST_F(A IN INT) RETURN INT AS

BEGIN
```

```
    IF A > 60 THEN
        RETURN ODCICONST.SUCCESS;
    ELSE
        RETURN ODCICONST.ERROR;
    END IF;
END;
/
SELECT TEST_F(100) FROM DUAL;
```

结果:

```
行号        TEST_F(100)
---------- -----------
1                0
```

48 DBMS_JSON 包

DBMS_JSON 包中提供了五个 JSON 相关类, 具体包括: JDOM_T、JSON_ELEMENT_T、JSON_SCALAR_T、JSON_ARRAY_T 和 JSON_OBJECT_T。

类说明:

JDOM_T: JSON 基础方法类。

JSON_ELEMENT_T: JSON 元素类，包含一个 JDOM_T 类的属性 DOM，此类不能实例化。JSON 元素实体可以为 JSON_SCALAR_T、JSON_ARRAY_T 或 JSON_OBJECT_T。

JSON_SCALAR_T: JSON 标量类, 继承于 JSON_ELEMENT_T。JSON 标量包括: JSON STRING、NUMBER、true、false 和 null。

JSON_ARRAY_T: JSON ARRAY 类，继承于 JSON_ELEMENT_T。

JSON_OBJECT_T: JSON OBJECT 类，继承于 JSON_ELEMENT_T。

48.1 数据类型

1. JSON_KEY_LIST

用于存放 JSON OBJECT 键值对名称。

语法如下:

```
CREATE OR REPLACE TYPE SYS.JSON_KEY_LIST IS TABLE OF VARCHAR(4000);
```

48.2 JDOM_T 类方法

1. PARSE

分析文本，构造 JSON 元素，包括 JSON_SCALAR_T、JSON_ARRAY_T 和 JSON_OBJECT_T。

语法如下:

```
STATIC FUNCTION PARSE(JSN IN VARCHAR2) RETURN JDOM_T

或

STATIC FUNCTION PARSE(JSN IN CLOB) RETURN JDOM_T

或

STATIC FUNCTION PARSE(JSN IN BLOB) RETURN JDOM_T

或

STATIC FUNCTION PARSE(JSN IN BLOB, FORMAT IN VARCHAR2) RETURN JDOM_T
```

参数详解

- JSN：表示 JSON 文本。
- FORMAT：暂不支持。

返回值

JDOM_T 类

使用说明:

1） 若 JSN 为 NULL，则返回 NULL;

2） 若 JSN 为非法的 JSON 文本，则报错。

2. CREATE_OBJ

构造一个空的 JSON OBJECT，不包含任何键值对，等价于“{}”。

语法如下:

```
STATIC FUNCTION CREATE_OBJ() RETURN JDOM_T
```

返回值

JDOM_T 类

3. CREATE_ARR

构造一个空的 JSON ARRAY，不包含任何值，等价于“[]”。

语法如下:

```
STATIC FUNCTION CREATE_ARR() RETURN JDOM_T
```

返回值

JDOM_T 类

4. STRINGIFY

将 JSON 元素转换为 VARCHAR2 文本。

语法如下:

```
MEMBER FUNCTION STRINGIFY() RETURN VARCHAR2
```

返回值

表示 JSON 元素的 VARCHAR2 文本。

5. TO_STRING

将 JSON 元素以 VARCHAR2 文本返回。

语法如下:

```
MEMBER FUNCTION TO_STRING() RETURN VARCHAR2
```

返回值

表示 JSON 元素的 VARCHAR2 文本。

6. TO_CLOB

将 JSON 元素以 CLOB 文本返回。

语法如下:

```
MEMBER FUNCTION TO_CLOB(C  IN OUT  CLOB) RETURN CLOB
```

参数详解

- C: 输入输出参数。

返回值

表示 JSON 元素的 CLOB 文本。

使用说明:

若 C 非 NULL，C 输出值与返回值相同。

7. TO_BLOB

将 JSON 元素以 BLOB 文本返回。

语法如下:

```
MEMBER FUNCTION TO_BLOB(B  IN OUT  BLOB) RETURN BLOB
```

参数详解

- B: 输入输出参数。

返回值

表示 JSON 元素的 BLOB 文本。

使用说明:

若 B 非 NULL，B 输出值与返回值相同。

8. TO_BOOLEAN

将 JSON 元素以 BOOLEAN 数据类型返回。

语法如下:

```
MEMBER FUNCTION TO_BOOLEAN() RETURN BOOLEAN
```

返回值

表示 JSON 元素的 BOOLEAN 值。

9. TO_NUMBER

将 JSON 元素以 NUMBER 数据类型返回。

语法如下:

```
MEMBER FUNCTION TO_NUMBER() RETURN NUMBER
```

返回值

表示 JSON 元素的 NUMBER 值。

10. TO_DATE

将 JSON 元素以 DATE 数据类型返回。

语法如下:

```
MEMBER FUNCTION TO_DATE() RETURN DATE
```

返回值

表示 JSON 元素的 DATE 值。

11. TO_TIMESTAMP

将 JSON 元素以 TIMESTAMP 数据类型返回。

语法如下:

```
MEMBER FUNCTION TO_TIMESTAMP() RETURN TIMESTAMP
```

返回值

表示 JSON 元素的 TIMESTAMP 值。

12. IS_OBJECT

检查 JSON 元素是否为 JSON OBJECT 类型。

语法如下:

```
MEMBER FUNCTION IS_OBJECT() RETURN BOOLEAN
```

返回值

TRUE：表示 JSON 元素是 JSON OBJECT 类型;

FALSE：表示 JSON 元素不是 JSON OBJECT 类型。

13. IS_ARRAY

检查 JSON 元素是否为 JSON ARRAY 类型。

语法如下:

```
MEMBER FUNCTION IS_ARRAY() RETURN BOOLEAN
```

返回值

TRUE：表示 JSON 元素是 JSON ARRAY 类型;

FALSE：表示 JSON 元素不是 JSON ARRAY 类型。

14. IS_SCALAR

检查 JSON 元素是否为 JSON 标量类型，包括：JSON STRING、NUMBER、true、false 和 null。

语法如下:

```
MEMBER FUNCTION IS_SCALAR() RETURN BOOLEAN
```

返回值

TRUE：表示 JSON 元素是 JSON 标量类型；

FALSE：表示 JSON 元素不是 JSON 标量类型。

15. IS_STRING

检查 JSON 元素是否为 JSON STRING 类型。

语法如下：

```
MEMBER FUNCTION IS_STRING() RETURN BOOLEAN
```

返回值

TRUE：表示 JSON 元素是 JSON STRING 类型；

FALSE：表示 JSON 元素不是 JSON STRING 类型。

16. IS_NUMBER

检查 JSON 元素是否为 JSON NUMBER 类型。

语法如下：

```
MEMBER FUNCTION IS_NUMBER() RETURN BOOLEAN
```

返回值

TRUE：表示 JSON 元素是 JSON NUMBER 类型；

FALSE：表示 JSON 元素不是 JSON NUMBER 类型。

17. IS_BOOLEAN

检查 JSON 元素是否为 JSON 布尔类型，包括：true、false。

语法如下：

```
MEMBER FUNCTION IS_BOOLEAN() RETURN BOOLEAN
```

返回值

TRUE：表示 JSON 元素是 JSON 布尔类型；

FALSE：表示 JSON 元素不是 JSON 布尔类型。

18. IS_TRUE

检查 JSON 元素是否为 JSON 布尔类型 true。

语法如下：

```
MEMBER FUNCTION IS_TRUE() RETURN BOOLEAN
```

返回值

TRUE：表示 JSON 元素是 JSON true；

FALSE：表示 JSON 元素不是 JSON true。

19. IS_FALSE

检查 JSON 元素是否为 JSON 布尔类型 false。

语法如下：

```
MEMBER FUNCTION IS_FALSE() RETURN BOOLEAN
```

返回值

TRUE：表示 JSON 元素是 JSON false；

FALSE：表示 JSON 元素不是 JSON false。

20. IS_NULL

检查 JSON 元素是否为 JSON null。

语法如下：

```
MEMBER FUNCTION IS_NULL() RETURN BOOLEAN
```

返回值

TRUE：表示 JSON 元素是 JSON null;

FALSE：表示 JSON 元素不是 JSON null。

21. IS_DATE

检查 JSON 元素是否可以转换为 DATE 数据类型。

语法如下:

```
MEMBER FUNCTION IS_DATE() RETURN BOOLEAN
```

返回值

TRUE：表示 JSON 元素可以转换为 DATE;

FALSE：表示 JSON 元素不可以转换为 DATE。

22. IS_TIMESTAMP

检查 JSON 元素是否可以转换为 TIMESTAMP 数据类型。

语法如下:

```
MEMBER FUNCTION IS_TIMESTAMP() RETURN BOOLEAN
```

返回值

TRUE：表示 JSON 元素可以转换为 TIMESTAMP;

FALSE：表示 JSON 元素不可以转换为 TIMESTAMP。

23. GET_SIZE

获取 JSON 元素的长度。

语法如下:

```
MEMBER FUNCTION GET_SIZE() RETURN NUMBER
```

返回值

针对 JSON OBJECT 类型，返回键值对个数；

针对 JSON ARRAY 类型，返回数组元素个数；

针对 JSON 标量，返回 1。

24. CLONE

克隆 JSON 元素。

语法如下：

```
MEMBER FUNCTION CLONE() RETURN JDOM_T
```

返回值

JDOM_T 类。

25. PUT

针对 JSON OBJECT 或者 JSON ARRAY，PUT 函数的用法不同。

- 针对 JSON OBJECT

为 JSON OBJECT 添加一个键值对。若指定的名称已存在，则覆盖指定名称对应的旧值。

语法如下：

```
MEMBER FUNCTION PUT(KEY VARCHAR2, VAL VARCHAR2) RETURN BOOLEAN

或

MEMBER FUNCTION PUT(KEY VARCHAR2, VAL NUMBER) RETURN BOOLEAN

或

MEMBER FUNCTION PUT(KEY VARCHAR2, VAL BOOLEAN) RETURN BOOLEAN

或
```

```
MEMBER FUNCTION PUT(KEY VARCHAR2, VAL DATE) RETURN BOOLEAN
```

或

```
MEMBER FUNCTION PUT(KEY VARCHAR2, VAL TIMESTAMP) RETURN BOOLEAN
```

或

```
MEMBER FUNCTION PUT(KEY VARCHAR2, VAL CLOB) RETURN BOOLEAN
```

或

```
MEMBER FUNCTION PUT(KEY VARCHAR2, VAL BLOB) RETURN BOOLEAN
```

或

```
MEMBER FUNCTION PUT(KEY VARCHAR2, VAL JDOM_T) RETURN BOOLEAN
```

参数详解

- KEY：表示键值对的名称。
- VAL：表示键值对的值。

返回值

TRUE：表示添加成功；

FALSE：表示添加失败。

- 针对 JSON ARRAY

在 JSON ARRAY 指定位置添加一个数组元素。

语法如下：

```
MEMBER FUNCTION PUT(POS NUMBER, VAL VARCHAR2, OVERWRITE BOOLEAN
) RETURN BOOLEAN
```

或

```
MEMBER FUNCTION PUT(POS NUMBER, VAL NUMBER, OVERWRITE BOOLEAN
) RETURN BOOLEAN
```

或

MEMBER FUNCTION PUT(POS NUMBER, VAL BOOLEAN, OVERWRITE BOOLEAN

) RETURN BOOLEAN

或

MEMBER FUNCTION PUT(POS NUMBER, VAL DATE, OVERWRITE BOOLEAN

) RETURN BOOLEAN

或

MEMBER FUNCTION PUT(POS NUMBER, VAL TIMESTAMP, OVERWRITE BOOLEAN

) RETURN BOOLEAN

或

MEMBER FUNCTION PUT(POS NUMBER, VAL CLOB, OVERWRITE BOOLEAN

) RETURN BOOLEAN

或

MEMBER FUNCTION PUT(POS NUMBER, VAL BLOB, OVERWRITE BOOLEAN

) RETURN BOOLEAN

或

MEMBER FUNCTION PUT(POS NUMBER, VAL JDOM_T, OVERWRITE BOOLEAN

) RETURN BOOLEAN

参数详解

- POS：表示添加元素的数组下标。
- VAL：表示添加元素的值。
- OVERWRITE：表示是否覆盖指定下标的旧值。

返回值

TRUE：表示添加成功；

FALSE：表示添加失败。

使用说明：

1） 若 POS 位置不存在则报错；

2） 若 OVERWRITE 为 TRUE，则 VAL 直接覆盖 POS 下标的元素；

3） 若 OVERWRITE 为 FALSE，则 VAL 作为 POS 下标的元素，原始 POS 下标的元素以及 POS 之后的所有元素向后移动（即下标加 1）。

26. PUT_NULL

针对 JSON OBJECT 或者 JSON ARRAY，PUT_NULL 函数的用法不同。

- 针对 JSON OBJECT

为 JSON OBJECT 添加一个键值对，键值对的值为 null。

语法如下：

```
MEMBER FUNCTION PUT_NULL(KEY VARCHAR2) RETURN BOOLEAN
```

参数详解

- KEY：表示键值对的名称。

返回值

TRUE：表示添加成功；

FALSE：表示添加失败。

- 针对 JSON ARRAY

在 JSON ARRAY 指定位置添加一个数组元素，数组元素的值为 null。

语法如下：

```
MEMBER FUNCTION PUT_NULL(POS NUMBER, OVERWRITE BOOLEAN) RETURN BOOLEAN
```

参数详解

- POS：表示添加元素的数组下标。
- OVERWRITE：表示是否覆盖指定下标的旧值。

返回值

TRUE：表示添加成功；

FALSE：表示添加失败。

使用说明：

1） 若 POS 位置不存在则报错；

2） 若 OVERWRITE 为 TRUE，则 null 直接覆盖 POS 下标的元素；

3） 若 OVERWRITE 为 FALSE，则 null 作为 POS 下标的元素，原始 POS 下标的元素以及 POS 之后的所有元素向后移动（即下标加 1）。

27. "GET"

将 JSON OBJECT 指定名称的 JSON 元素，或者 JSON ARRAY 指定下标的 JSON 元素，以 JDOM_T 方式返回。

语法如下：

```
MEMBER FUNCTION "GET"(KEY VARCHAR2) RETURN JDOM_T

或

MEMBER FUNCTION "GET"(POS NUMBER) RETURN JDOM_T
```

参数详解

- KEY：表示键值对的名称。
- POS：表示数组下标。

返回值

表示 JSON 元素的 JDOM_T 类。

使用说明：

DM 的 GET 为关键字，使用 GET 函数请加上双引号。

28. GET_STRING

将 JSON OBJECT 指定名称的 JSON 元素, 或者 JSON ARRAY 指定下标的 JSON 元素, 以 VARCHAR2 文本返回。

语法如下:

```
MEMBER FUNCTION GET_STRING(KEY VARCHAR2) RETURN VARCHAR2

或

MEMBER FUNCTION GET_STRING(POS NUMBER) RETURN VARCHAR2
```

参数详解

- KEY：表示键值对的名称。
- POS：表示数组下标。

返回值

表示 JSON 元素的 VARCHAR2 文本。

29. GET_BOOLEAN

将 JSON OBJECT 指定名称的 JSON 元素, 或者 JSON ARRAY 指定下标的 JSON 元素, 以 BOOLEAN 数据类型返回。

语法如下:

```
MEMBER FUNCTION GET_BOOLEAN(KEY VARCHAR2) RETURN BOOLEAN

或

MEMBER FUNCTION GET_BOOLEAN(POS NUMBER) RETURN BOOLEAN
```

参数详解

- KEY：表示键值对的名称。

- POS：表示数组下标。

返回值

表示 JSON 元素的 BOOLEAN 值。

30. GET_NUMBER

将 JSON OBJECT 指定名称的 JSON 元素，或者 JSON ARRAY 指定下标的 JSON 元素，以 NUMBER 数据类型返回。

语法如下：

```
MEMBER FUNCTION GET_NUMBER(KEY VARCHAR2) RETURN NUMBER

或

MEMBER FUNCTION GET_NUMBER(POS NUMBER) RETURN NUMBER
```

参数详解

- KEY：表示键值对的名称。
- POS：表示数组下标。

返回值

表示 JSON 元素的 NUMBER 值。

31. GET_DATE

将 JSON OBJECT 指定名称的 JSON 元素，或者 JSON ARRAY 指定下标的 JSON 元素，以 DATE 数据类型返回。

语法如下：

```
MEMBER FUNCTION GET_DATE(KEY VARCHAR2) RETURN DATE

或

MEMBER FUNCTION GET_DATE(POS NUMBER) RETURN DATE
```

参数详解

- KEY：表示键值对的名称。
- POS：表示数组下标。

返回值

表示 JSON 元素的 DATE 值。

32. GET_TIMESTAMP

将 JSON OBJECT 指定名称的 JSON 元素，或者 JSON ARRAY 指定下标的 JSON 元素，以 TIMESTAMP 数据类型返回。

语法如下：

```
MEMBER FUNCTION GET_TIMESTAMP(KEY VARCHAR2) RETURN TIMESTAMP

或

MEMBER FUNCTION GET_TIMESTAMP(POS NUMBER) RETURN TIMESTAMP
```

参数详解

- KEY：表示键值对的名称。
- POS：表示数组下标。

返回值

表示 JSON 元素的 TIMESTAMP 值。

33. GET_CLOB

将 JSON OBJECT 指定名称的 JSON 元素，或者 JSON ARRAY 指定下标的 JSON 元素，以 CLOB 文本返回。

语法如下：

```
MEMBER FUNCTION GET_CLOB(KEY VARCHAR2, C IN OUT CLOB) RETURN CLOB
```

或

```
MEMBER FUNCTION GET_CLOB(POS NUMBER, C IN OUT CLOB) RETURN CLOB
```

参数详解

- KEY：表示键值对的名称。
- C： 输入输出参数。
- POS：表示数组下标。

返回值

表示 JSON 元素的 CLOB 文本。

使用说明:

若 C 非 NULL，则输出值 C 与返回值相同。

34. GET_BLOB

将 JSON OBJECT 指定名称的 JSON 元素, 或者 JSON ARRAY 指定下标的 JSON 元素, 以 BLOB 文本返回。

语法如下:

```
MEMBER FUNCTION GET_BLOB(KEY VARCHAR2, B IN OUT BLOB) RETURN BLOB
```

或

```
MEMBER FUNCTION GET_BLOB(POS NUMBER, B IN OUT BLOB) RETURN BLOB
```

参数详解

- KEY：表示键值对的名称。
- B：输入输出参数。
- POS：表示数组下标。

返回值

表示 JSON 元素的 BLOB 文本。

使用说明:

若 B 非 NULL，则输出值 B 与返回值相同。

35. HAS_KEY

JSON OBJECT 是否存在指定名称的键值对。

语法如下:

```
MEMBER FUNCTION HAS_KEY(KEY VARCHAR2) RETURN BOOLEAN
```

参数详解

- KEY：表示键值对的名称。

返回值

TRUE：表示 JSON OBJECT 存在名称为 KEY 的键值对;

FALSE：表示 JSON OBJECT 不存在名称为 KEY 的键值对。

36. GET_KEYS

获取 JSON OBJECT 的所有键值对名称。

语法如下:

```
MEMBER FUNCTION GET_KEYS() RETURN JSON_KEY_LIST
```

返回值

表示 JSON OBJECT 的所有键值对名称的链表。

37. GET_TYPE

获取 JSON OBJECT 指定名称对应 JSON 元素的 JSON 类型，或者 JSON ARRAY 指定下标对应 JSON 元素的 JSON 类型。

语法如下:

```
MEMBER FUNCTION GET_TYPE(KEY VARCHAR2) RETURN VARCHAR2

或

MEMBER FUNCTION GET_TYPE(POS NUMBER) RETURN VARCHAR2
```

参数详解

- KEY：表示键值对的名称。
- POS：表示数组下标。

返回值

表示 JSON 元素类型的字符串，包括：SCALAR、OBJECT 和 ARRAY，分别表示 JSON 标量、JSON OBJECT 和 JSON ARRAY。

38. REMOVE

删除 JSON OBJECT 指定名称的键值对，或者 JSON ARRAY 指定下标的数组元素。

语法如下:

```
MEMBER FUNCTION REMOVE(KEY VARCHAR2) RETURN BOOLEAN

或

MEMBER FUNCTION REMOVE(POS NUMBER) RETURN BOOLEAN
```

参数详解

- KEY：表示键值对的名称。
- POS：表示数组下标。

返回值

TRUE：表示删除成功;

FALSE：表示删除失败。

39. RENAME_KEY

重命名 JSON OBJECT 中的键值对名称。

语法如下:

```
MEMBER FUNCTION RENAME_KEY(KEYOLD VARCHAR2, KEYNEW VARCHAR2) RETURN BOOLEAN
```

参数详解

- KEYOLD：表示键值对的旧名称。
- KEYNEW：表示键值对的新名称。

返回值

TRUE：表示重命名成功;

FALSE：表示重命名失败。

40. APPEND

在 JSON ARRAY 末尾添加一个数组元素。

语法如下:

```
MEMBER FUNCTION APPEND(VAL VARCHAR2) RETURN BOOLEAN

或

MEMBER FUNCTION APPEND(VAL NUMBER) RETURN BOOLEAN

或

MEMBER FUNCTION APPEND(VAL BOOLEAN) RETURN BOOLEAN

或

MEMBER FUNCTION APPEND(VAL DATE) RETURN BOOLEAN

或

MEMBER FUNCTION APPEND(VAL TIMESTAMP) RETURN BOOLEAN

或

MEMBER FUNCTION APPEND(VAL CLOB) RETURN BOOLEAN
```

或

```
MEMBER FUNCTION APPEND(VAL BLOB) RETURN BOOLEAN
```

或

```
MEMBER FUNCTION APPEND(VAL JDOM_T) RETURN BOOLEAN
```

参数详解

- VAL：表示添加数组元素的值。

返回值

TRUE：表示添加成功；

FALSE：表示添加失败。

41. APPEND_NULL

在 JSON ARRAY 末尾添加一个数组元素，数组元素的值为 null。

语法如下:

```
MEMBER FUNCTION APPEND_NULL() RETURN BOOLEAN
```

返回值

TRUE：表示添加成功；

FALSE：表示添加失败。

48.3 JSON_ELEMENT_T 类方法

1. PARSE

分析文本，构造 JSON 元素，包括 JSON_SCALAR_T、JSON_OBJECT_T 和 JSON_ARRAY_T。

语法如下:

```
STATIC FUNCTION PARSE(JSN IN VARCHAR2) RETURN JSON_ELEMENT_T
```

或

```
STATIC FUNCTION PARSE(JSN IN CLOB) RETURN JSON_ELEMENT_T
```

或

```
STATIC FUNCTION PARSE(JSN IN BLOB) RETURN JSON_ELEMENT_T
```

或

```
STATIC FUNCTION PARSE(JSN IN BLOB, FORMAT IN VARCHAR2) RETURN JSON_ELEMENT_T
```

参数详解

- JSN：表示 JSON 文本。
- FORMAT：暂不支持。

返回值

JSON_ELEMENT_T 类。

使用说明:

1） 若 JSN 为 NULL，则返回 NULL;

2） 若 JSN 为非法的 JSON 文本，则报错;

3） 若 JSN 为 JSON 标量，则报错，只支持 JSON ARRAY 和 JSON OBJECT。

2. STRINGIFY

将 JSON 元素转换为 VARCHAR2 文本。

语法如下:

```
MEMBER FUNCTION STRINGIFY() RETURN VARCHAR2
```

返回值

表示 JSON 元素转换的 VARCHAR2 文本。

3. TO_STRING

将 JSON 元素以 VARCHAR2 文本返回。

语法如下:

```
MEMBER FUNCTION TO_STRING() RETURN VARCHAR2
```

返回值

表示 JSON 元素的 VARCHAR2 文本。

4. TO_BOOLEAN

将 JSON 元素以 BOOLEAN 数据类型返回。

语法如下:

```
MEMBER FUNCTION TO_BOOLEAN() RETURN BOOLEAN
```

返回值

表示 JSON 元素的 BOOLEAN 值。

5. TO_CLOB

将 JSON 元素以 CLOB 文本返回。

语法如下:

```
MEMBER FUNCTION TO_CLOB() RETURN CLOB

或

MEMBER PROCEDURE TO_CLOB(C IN OUT CLOB)
```

参数详解

- C: 输入输出参数。

返回值

表示 JSON 元素的 CLOB 文本。

6. TO_BLOB

将 JSON 元素以 BLOB 文本返回。

语法如下:

```
MEMBER FUNCTION TO_BLOB() RETURN BLOB

或

MEMBER PROCEDURE TO_BLOB(B IN OUT BLOB)
```

参数详解

- b: 输入输出参数。

返回值

表示 JSON 元素的 BLOB 文本。

7. TO_NUMBER

将 JSON 元素以 NUMBER 数据类型返回。

语法如下:

```
MEMBER FUNCTION TO_NUMBER() RETURN NUMBER
```

返回值

表示 JSON 元素的 NUMBER 值。

8. TO_DATE

将 JSON 元素以 DATE 数据类型返回。

语法如下:

```
MEMBER FUNCTION TO_DATE() RETURN DATE
```

返回值

表示 JSON 元素的 DATE 值。

9. TO_TIMESTAMP

将 JSON 元素以 TIMESTAMP 数据类型返回。

语法如下:

```
MEMBER FUNCTION TO_TIMESTAMP() RETURN TIMESTAMP
```

返回值

表示 JSON 元素的 TIMESTAMP 值。

10. IS_OBJECT

检查 JSON 元素是否为 JSON OBJECT 类型。

语法如下:

```
MEMBER FUNCTION IS_OBJECT() RETURN BOOLEAN
```

返回值

TRUE：表示 JSON 元素是 JSON OBJECT 类型;

FALSE：表示 JSON 元素不是 JSON OBJECT 类型。

11. IS_ARRAY

检查 JSON 元素是否为 JSON ARRAY 类型。

语法如下:

```
MEMBER FUNCTION IS_ARRAY() RETURN BOOLEAN
```

返回值

TRUE：表示 JSON 元素是 JSON ARRAY 类型；

FALSE：表示 JSON 元素不是 JSON ARRAY 类型。

12. IS_SCALAR

检查 JSON 元素是否为 JSON 标量类型，包括：JSON STRING、NUMBER、true、false 和 null。

语法如下：

```
MEMBER FUNCTION IS_SCALAR() RETURN BOOLEAN
```

返回值

TRUE：表示 JSON 元素是 JSON 标量类型；

FALSE：表示 JSON 元素不是 JSON 标量类型。

13. IS_STRING

检查 JSON 元素是否为 JSON STRING 类型。

语法如下：

```
MEMBER FUNCTION IS_STRING() RETURN BOOLEAN
```

返回值

TRUE：表示 JSON 元素是 JSON STRING 类型；

FALSE：表示 JSON 元素不是 JSON STRING 类型。

14. IS_NUMBER

检查 JSON 元素是否为 JSON NUMBER 类型。

语法如下：

```
MEMBER FUNCTION IS_NUMBER() RETURN BOOLEAN
```

返回值

TRUE：表示 JSON 元素是 JSON NUMBER 类型；

FALSE：表示 JSON 元素不是 JSON NUMBER 类型。

15. IS_BOOLEAN

检查 JSON 元素是否为 JSON 布尔类型，包括：true、false。

语法如下:

```
MEMBER FUNCTION IS_BOOLEAN() RETURN BOOLEAN
```

返回值

TRUE：表示 JSON 元素是 JSON 布尔类型；

FALSE：表示 JSON 元素不是 JSON 布尔类型。

16. IS_TRUE

检查 JSON 元素是否为 JSON 布尔类型 true。

语法如下:

```
MEMBER FUNCTION IS_TRUE() RETURN BOOLEAN
```

返回值

TRUE：表示 JSON 元素是 JSON true；

FALSE：表示 JSON 元素不是 JSON true。

17. IS_FALSE

检查 JSON 元素是否为 JSON 布尔类型 false。

语法如下:

```
MEMBER FUNCTION IS_FALSE() RETURN BOOLEAN
```

返回值

TRUE：表示 JSON 元素是 JSON false;

FALSE：表示 JSON 元素不是 JSON false。

18. IS_NULL

检查 JSON 元素是否为 JSON null。

语法如下:

```
MEMBER FUNCTION IS_NULL() RETURN BOOLEAN
```

返回值

TRUE：表示 JSON 元素是 JSON null;

FALSE：表示 JSON 元素不是 JSON null。

19. IS_DATE

检查 JSON 元素是否可以转换为 DATE 数据类型。

语法如下:

```
MEMBER FUNCTION IS_DATE() RETURN BOOLEAN
```

返回值

TRUE：表示 JSON 元素可以转换为 DATE;

FALSE：表示 JSON 元素不可以转换为 DATE。

20. IS_TIMESTAMP

检查 JSON 元素是否可以转换为 TIMESTAMP 数据类型。

语法如下:

```
MEMBER FUNCTION IS_TIMESTAMP() RETURN BOOLEAN
```

返回值

TRUE：表示 JSON 元素可以转换为 TIMESTAMP；

FALSE：表示 JSON 元素不可以转换为 TIMESTAMP。

21. GET_SIZE

获取 JSON 元素的长度。

语法如下：

```
MEMBER FUNCTION GET_SIZE() RETURN NUMBER
```

返回值

针对 JSON OBJECT 类型，返回键值对个数；

针对 JSON ARRAY 类型，返回数组元素个数；

针对 JSON 标量，返回 1。

48.4 JSON_SCALAR_T 类方法

1. 构造函数

构造 JSON_SCALAR_T 类。

语法如下：

```
CONSTRUCTOR FUNCTION JSON_SCALAR_T(JSN JDOM_T) RETURN SELF AS RESULT

或

CONSTRUCTOR FUNCTION JSON_SCALAR_T(JSN JSON_ELEMENT_T) RETURN SELF AS RESULT
```

参数详解

- JSN：表示 JDOM_T 类或 JSON_ELEMENT_T 类。

返回值

JSON_SCALAR_T 类。

使用说明:

若 JSN 对应 JSON 元素不是 JSON 标量，则无法构造 JSON_SCALAR_T，函数报错。

2. CLONE

克隆 JSON_SCALAR_T 类。

语法如下:

```
MEMBER FUNCTION CLONE() RETURN JSON_SCALAR_T
```

返回值

JSON_SCALAR_T 类。

48.5 JSON_ARRAY_T 类方法

1. 构造函数

构造 JSON_ARRAY_T 类。

语法如下:

```
CONSTRUCTOR FUNCTION JSON_ARRAY_T() RETURN SELF AS RESULT

或

CONSTRUCTOR FUNCTION JSON_ARRAY_T(JSN JDOM_T) RETURN SELF AS RESULT

或

CONSTRUCTOR FUNCTION JSON_ARRAY_T(JSN JSON_ELEMENT_T) RETURN SELF AS RESULT

或

CONSTRUCTOR FUNCTION JSON_ARRAY_T(JSN VARCHAR2) RETURN SELF AS RESULT

或

CONSTRUCTOR FUNCTION JSON_ARRAY_T(JSN CLOB) RETURN SELF AS RESULT

或
```

```
CONSTRUCTOR FUNCTION JSON_ARRAY_T(JSN BLOB) RETURN SELF AS RESULT
```

或

```
CONSTRUCTOR FUNCTION JSON_ARRAY_T(JSN BLOB, FORMAT IN VARCHAR2) RETURN SELF AS
RESULT
```

参数详解

- JSN：表示 JSON 文本。
- FORMAT：暂不支持。

返回值

JSON_ARRAY_T 类。

使用说明:

1） 若没有 JSN 参数，则构造一个空的 JSON ARRAY，不包含任何值，等价于“[]”;

2） 若 JSN 为非法的 JSON 文本，则报错;

3） 若 JSN 对应 JSON 类型不是 JSON ARRAY，则报错。

2. PARSE

分析文本，构造 JSON_ARRAY_T。

语法如下:

```
STATIC FUNCTION PARSE(JSN IN VARCHAR2, NO_USE_1 INT DEFAULT NULL
) RETURN JSON_ARRAY_T
```

或

```
STATIC FUNCTION PARSE(JSN IN CLOB, NO_USE_1 INT DEFAULT NULL) RETURN JSON_ARRAY_T
```

或

```
STATIC FUNCTION PARSE(JSN IN BLOB, NO_USE_1 INT DEFAULT NULL) RETURN JSON_ARRAY_T
```

或

```
STATIC FUNCTION PARSE(JSN IN BLOB, FORMAT IN VARCHAR2, NO_USE_1 INT DEFAULT NULL
) RETURN JSON_ARRAY_T
```

参数详解

- JSN：表示 JSON 文本。
- NO_USE_1：暂不支持。
- FORMAT：暂不支持。

返回值

JSON_ARRAY_T 类。

使用说明:

1） 若 JSN 为 NULL，则返回 NULL;

2） 若 JSN 为非法的 JSON 文本，则报错;

3） 若 JSN 对应 JSON 类型不是 JSON ARRAY，则报错。

3. CLONE

克隆 JSON_ARRAY_T 类。

语法如下:

```
MEMBER FUNCTION CLONE() RETURN JSON_ARRAY_T
```

返回值

JSON_ARRAY_T 类。

4. "GET"

将 JSON ARRAY 指定下标的 JSON 元素以 JSON_ELEMENT_T 方式返回。

语法如下:

```
MEMBER FUNCTION "GET"(POS NUMBER) RETURN JSON_ELEMENT_T
```

参数详解

- POS：表示数组下标。

返回值

表示 JSON 元素的 JSON_ELEMENT_T 类。

使用说明:

DM 的 GET 为关键字，使用 GET 函数请加上双引号。

5. GET_STRING

将 JSON ARRAY 指定下标的 JSON 元素以 VARCHAR2 文本返回。

语法如下:

```
MEMBER FUNCTION GET_STRING(POS NUMBER) RETURN VARCHAR2
```

参数详解

- POS：表示数组下标。

返回值

表示 JSON 元素的 VARCHAR2 文本。

6. GET_BOOLEAN

将 JSON ARRAY 指定下标的 JSON 元素以 BOOLEAN 数据类型返回。

语法如下:

```
MEMBER FUNCTION GET_BOOLEAN(POS NUMBER) RETURN BOOLEAN
```

参数详解

- POS：表示数组下标。

返回值

表示 JSON 元素的 BOOLEAN 值。

7. GET_NUMBER

将 JSON ARRAY 指定下标的 JSON 元素以 NUMBER 数据类型返回。

语法如下:

```
MEMBER FUNCTION GET_NUMBER(POS NUMBER) RETURN NUMBER
```

参数详解

- POS：表示数组下标。

返回值

表示 JSON 元素的 NUMBER 值。

8. GET_DATE

将 JSON ARRAY 指定下标的 JSON 元素以 DATE 数据类型返回。

语法如下:

```
MEMBER FUNCTION GET_DATE(POS NUMBER) RETURN DATE
```

参数详解

- POS：表示数组下标。

返回值

表示 JSON 元素的 DATE 值。

9. GET_TIMESTAMP

将 JSON ARRAY 指定下标的 JSON 元素以 TIMESTAMP 数据类型返回。

语法如下:

```
MEMBER FUNCTION GET_TIMESTAMP(POS NUMBER) RETURN TIMESTAMP
```

参数详解

- POS：表示数组下标。

返回值

表示 JSON 元素的 TIMESTAMP 值。

10. GET_CLOB

将 JSON ARRAY 指定下标的 JSON 元素以 CLOB 文本返回。

语法如下:

```
MEMBER FUNCTION GET_CLOB(POS NUMBER) RETURN CLOB

或

MEMBER PROCEDURE GET_CLOB(POS NUMBER, C IN OUT CLOB)
```

参数详解

- POS：表示数组下标。
- C： 输入输出参数。

返回值

表示 JSON 元素的 CLOB 文本。

11. GET_BLOB

将 JSON ARRAY 指定下标的 JSON 元素以 BLOB 文本返回。

语法如下:

```
MEMBER FUNCTION GET_BLOB(POS NUMBER) RETURN BLOB

或

MEMBER PROCEDURE GET_BLOB(POS NUMBER, B IN OUT BLOB)
```

参数详解

- POS：表示数组下标。
- B： 输入输出参数。

返回值

表示 JSON 元素的 BLOB 文本。

12. APPEND

在 JSON ARRAY 末尾添加一个数组元素。

语法如下:

```
MEMBER PROCEDURE APPEND(VAL VARCHAR2)

或

MEMBER PROCEDURE APPEND(VAL NUMBER)

或

MEMBER PROCEDURE APPEND(VAL BOOLEAN)

或

MEMBER PROCEDURE APPEND(VAL DATE)

或

MEMBER PROCEDURE APPEND(VAL TIMESTAMP)

或

MEMBER PROCEDURE APPEND(VAL CLOB)

或

MEMBER PROCEDURE APPEND(VAL BLOB)

或

MEMBER PROCEDURE APPEND(VAL JSON_ELEMENT_T)
```

参数详解

- VAL：表示添加数组元素的值。

13. APPEND_NULL

在 JSON ARRAY 末尾添加一个数组元素，数组元素的值为 null。

语法如下:

```
MEMBER PROCEDURE APPEND_NULL()
```

14. PUT

在 JSON ARRAY 指定位置添加一个数组元素。

语法如下:

```
MEMBER PROCEDURE PUT(POS NUMBER, VAL VARCHAR2, OVERWRITE BOOLEAN DEFAULT FALSE)

或

MEMBER PROCEDURE PUT(POS NUMBER, VAL NUMBER, OVERWRITE BOOLEAN DEFAULT FALSE)

或

MEMBER PROCEDURE PUT(POS NUMBER, VAL BOOLEAN, OVERWRITE BOOLEAN DEFAULT FALSE)

或

MEMBER PROCEDURE PUT(POS NUMBER, VAL DATE, OVERWRITE BOOLEAN DEFAULT FALSE)

或

MEMBER PROCEDURE PUT(POS NUMBER, VAL TIMESTAMP, OVERWRITE BOOLEAN DEFAULT FALSE)

或

MEMBER PROCEDURE PUT(POS NUMBER, VAL CLOB, OVERWRITE BOOLEAN DEFAULT FALSE)

或

MEMBER PROCEDURE PUT(POS NUMBER, VAL BLOB, OVERWRITE BOOLEAN DEFAULT FALSE)

或
```

```
MEMBER PROCEDURE PUT(POS NUMBER, VAL JSON_ELEMENT_T, OVERWRITE BOOLEAN DEFAULT FALSE)
```

参数详解

- POS：表示添加元素的数组下标。
- VAL：表示添加元素的值。
- OVERWRITE：表示是否覆盖指定下标的旧值。

使用说明:

1） 若 POS 位置不存在则报错;

2） 若 OVERWRITE 为 TRUE，则 VAL 直接覆盖 POS 下标的元素;

3） 若 OVERWRITE 为 FALSE，则 VAL 作为 POS 下标的元素，原始 POS 下标的元素以及 POS 之后的所有元素向后移动（即下标加 1）。

15. PUT_NULL

为 JSON ARRAY 添加一个数组元素，数组元素的值为 null。

语法如下:

```
MEMBER PROCEDURE PUT_NULL(POS NUMBER, OVERWRITE BOOLEAN DEFAULT FALSE)
```

参数详解

- POS：表示添加元素的数组下标。
- OVERWRITE：表示是否覆盖指定下标的旧值。

使用说明:

1） 若 POS 位置不存在则报错;

2） 若 OVERWRITE 为 TRUE，则 VAL 直接覆盖 POS 下标的元素;

3） 若 OVERWRITE 为 FALSE，则 VAL 作为 POS 下标的元素，原始 POS 下标的元素以及 POS 之后的所有元素向后移动（即下标加 1）。

16. REMOVE

删除 JSON ARRAY 指定下标的 JSON 元素。

语法如下:

```
MEMBER PROCEDURE REMOVE(POS NUMBER)
```

参数详解

- POS：表示数组下标。

17. GET_TYPE

获取 JSON ARRAY 指定下标对应 JSON 元素的 JSON 类型。

语法如下:

```
MEMBER FUNCTION GET_TYPE(POS NUMBER) RETURN VARCHAR2
```

参数详解

- POS：表示数组下标。

返回值

表示 JSON 元素类型的字符串，包括：SCALAR、OBJECT 和 ARRAY，分别表示 JSON 标量、JSON OBJECT 和 JSON ARRAY。

48.6 JSON_OBJECT_T 类方法

1. 构造函数

构造 JSON_OBJECT_T 类。

语法如下:

```
CONSTRUCTOR FUNCTION JSON_OBJECT_T() RETURN SELF AS RESULT
```

或

CONSTRUCTOR FUNCTION JSON_OBJECT_T(JSN JDOM_T) RETURN SELF AS RESULT

或

CONSTRUCTOR FUNCTION JSON_OBJECT_T(JSN JSON_ELEMENT_T) RETURN SELF AS RESULT

或

CONSTRUCTOR FUNCTION JSON_OBJECT_T(JSN VARCHAR2) RETURN SELF AS RESULT

或

CONSTRUCTOR FUNCTION JSON_OBJECT_T(JSN CLOB) RETURN SELF AS RESULT

或

CONSTRUCTOR FUNCTION JSON_OBJECT_T(JSN BLOB) RETURN SELF AS RESULT

或

CONSTRUCTOR FUNCTION JSON_OBJECT_T(JSN BLOB, FORMAT IN VARCHAR2) RETURN SELF AS RESULT

参数详解

- JSN：表示 JSON 文本。
- FORMAT：暂不支持。

返回值

JSON_OBJECT_T 类。

使用说明:

1） 若没有 JSN 参数，则构造一个空的 JSON OBJECT，不包含任何值，等价于“{}”;

2） 若 JSN 为非法的 JSON 文本，则报错;

3） 若 JSN 对应 JSON 类型不是 JSON OBJECT，则报错。

2. PARSE

分析文本，构造 JSON_OBJECT_T。

语法如下:

```
STATIC FUNCTION  PARSE(JSN IN VARCHAR2, NO_USE_1 INT DEFAULT NULL
) RETURN JSON_OBJECT_T
或
STATIC FUNCTION  PARSE(JSN IN CLOB, NO_USE_1 INT DEFAULT NULL) RETURN JSON_OBJECT_T
或
STATIC FUNCTION  PARSE(JSN IN BLOB, NO_USE_1 INT DEFAULT NULL) RETURN JSON_OBJECT_T
或
STATIC FUNCTION  PARSE(JSN IN BLOB, FORMAT IN VARCHAR2, NO_USE_1 INT DEFAULT NULL
) RETURN JSON_OBJECT_T
```

参数详解

- JSN：表示 JSON 文本。
- NO_USE_1：暂不支持。
- FORMAT：暂不支持。

返回值

JSON_OBJECT_T 类。

使用说明:

1） 若 JSN 为 NULL，则返回 NULL;

2） 若 JSN 为非法的 JSON 文本，则报错;

3） 若 JSN 对应 JSON 类型不是 JSON OBJECT，则报错。

3. CLONE

克隆 JSON_OBJECT_T 类。

语法如下:

```
MEMBER FUNCTION CLONE() RETURN JSON_OBJECT_T
```

返回值

JSON_OBJECT_T 类。

4. "GET"

将 JSON OBJECT 指定名称的 JSON 元素以 JSON_ELEMENT_T 方式返回。

语法如下:

```
MEMBER FUNCTION "GET"(KEY VARCHAR2) RETURN JSON_ELEMENT_T
```

参数详解

- KEY：表示键值对的名称。

返回值

表示 JSON 元素的 JSON_ELEMENT_T 类。

使用说明:

DM 的 GET 为关键字，使用 GET 函数请加上双引号。

5. GET_OBJECT

将 JSON OBJECT 指定名称的 JSON 元素以 JSON_OBJECT_T 类方式返回。

语法如下:

```
MEMBER FUNCTION GET_OBJECT(KEY VARCHAR2) RETURN JSON_OBJECT_T
```

参数详解

- KEY：表示键值对的名称。

返回值

表示 JSON 元素的 JSON_OBJECT_T 类。

6. GET_ARRAY

将 JSON OBJECT 指定名称的 JSON 元素以 JSON_ARRAY_T 类方式返回。

语法如下:

```
MEMBER FUNCTION GET_ARRAY(KEY VARCHAR2) RETURN JSON_ARRAY_T
```

参数详解

- KEY: 表示键值对的名称。

返回值

表示 JSON 元素的 JSON_ARRAY_T 类。

7. GET_STRING

将 JSON OBJECT 指定名称的 JSON 元素以 VARCHAR2 文本返回。

语法如下:

```
MEMBER FUNCTION GET_STRING(KEY VARCHAR2) RETURN VARCHAR2
```

参数详解

- KEY: 表示键值对的名称。

返回值

表示 JSON 元素的 VARCHAR2 文本。

8. GET_BOOLEAN

将 JSON OBJECT 指定名称的 JSON 元素以 BOOLEAN 数据类型返回。

语法如下:

```
MEMBER FUNCTION GET_BOOLEAN(KEY VARCHAR2) RETURN BOOLEAN
```

参数详解

- KEY：表示键值对的名称。

返回值

表示 JSON 元素的 BOOLEAN 值。

9. GET_NUMBER

将 JSON OBJECT 指定名称的 JSON 元素以 NUMBER 数据类型返回。

语法如下:

```
MEMBER FUNCTION GET_NUMBER(KEY VARCHAR2) RETURN NUMBER
```

参数详解

- KEY：表示键值对的名称。

返回值

表示 JSON 元素的 NUMBER 值。

10. GET_DATE

将 JSON OBJECT 指定名称的 JSON 元素以 DATE 数据类型返回。

语法如下:

```
MEMBER FUNCTION GET_DATE(KEY VARCHAR2) RETURN DATE
```

参数详解

- KEY：表示键值对的名称。

返回值

表示 JSON 元素的 DATE 值。

11. GET_TIMESTAMP

将 JSON OBJECT 指定名称的 JSON 元素以 TIMESTAMP 数据类型返回。

语法如下:

```
MEMBER FUNCTION GET_TIMESTAMP(KEY VARCHAR2) RETURN TIMESTAMP
```

参数详解

- KEY：表示键值对的名称。

返回值

表示 JSON 元素的 TIMESTAMP 值。

12. GET_CLOB

将 JSON OBJECT 指定名称的 JSON 元素以 CLOB 文本返回。

语法如下:

```
MEMBER FUNCTION GET_CLOB(KEY VARCHAR2) RETURN CLOB

或

MEMBER PROCEDURE GET_CLOB(KEY VARCHAR2, C IN OUT CLOB)
```

参数详解

- KEY：表示键值对的名称。
- C： 输入输出参数。

返回值

表示 JSON 元素的 CLOB 文本。

13. GET_BLOB

将 JSON OBJECT 指定名称的 JSON 元素以 BLOB 文本返回。

语法如下:

```
MEMBER FUNCTION GET_BLOB(KEY VARCHAR2) RETURN BLOB
```

或

```
MEMBER PROCEDURE GET_BLOB(KEY VARCHAR2, B IN OUT BLOB)
```

参数详解

- KEY：表示键值对的名称。
- B： 输入输出参数。

返回值

表示 JSON 元素的 BLOB 文本。

14. PUT

为 JSON OBJECT 添加一个键值对。若指定的名称已存在，则覆盖指定名称对应的旧值。

语法如下：

```
MEMBER PROCEDURE PUT(KEY VARCHAR2, VAL VARCHAR2)

或

MEMBER PROCEDURE PUT(KEY VARCHAR2, VAL NUMBER)

或

MEMBER PROCEDURE PUT(KEY VARCHAR2, VAL BOOLEAN)

或

MEMBER PROCEDURE PUT(KEY VARCHAR2, VAL DATE)

或

MEMBER PROCEDURE PUT(KEY VARCHAR2, VAL TIMESTAMP)

或

MEMBER PROCEDURE PUT(KEY VARCHAR2, VAL CLOB)

或

MEMBER PROCEDURE PUT(KEY VARCHAR2, VAL BLOB)
```

或

```
MEMBER PROCEDURE PUT(KEY VARCHAR2, VAL JSON_ELEMENT_T)
```

参数详解

- KEY：表示键值对的名称。
- VAL：表示键值对的值。

15. PUT_NULL

为 JSON OBJECT 添加一个键值对，键值对的值为 NULL。

语法如下:

```
MEMBER PROCEDURE PUT_NULL(KEY VARCHAR2)
```

参数详解

- KEY：表示键值对的名称。

16. HAS

JSON OBJECT 是否存在指定名称的键值对。

语法如下:

```
MEMBER FUNCTION HAS(KEY VARCHAR2) RETURN BOOLEAN
```

参数详解

- KEY：表示键值对的名称。

返回值

TRUE：表示 JSON OBJECT 存在名称为 KEY 键值对;

FALSE：表示 JSON OBJECT 不存在名称为 KEY 键值对。

17. GET_KEYS

获取 JSON OBJECT 的所有键值对名称。

语法如下:

```
MEMBER FUNCTION GET_KEYS() RETURN JSON_KEY_LIST
```

返回值

表示 JSON OBJECT 的所有键值对名称的链表。

18. GET_TYPE

获取 JSON OBJECT 指定名称对应 JSON 元素的 JSON 类型。

语法如下:

```
MEMBER FUNCTION GET_TYPE(KEY VARCHAR2) RETURN VARCHAR2
```

参数详解

- KEY：表示键值对的名称。

返回值

表示 JSON 元素类型的字符串，包括：SCALAR、OBJECT 和 ARRAY，分别表示 JSON 标量、JSON OBJECT 和 JSON ARRAY。

19. REMOVE

删除 JSON OBJECT 指定名称的键值对。

语法如下:

```
MEMBER PROCEDURE REMOVE(KEY VARCHAR2)
```

参数详解

- KEY：表示键值对的名称。

20. RENAME_KEY

重命名 JSON OBJECT 中的键值对名称。

语法如下:

```
MEMBER PROCEDURE RENAME_KEY(KEYOLD VARCHAR2, KEYNEW VARCHAR2)
```

参数详解

- KEYOLD：表示键值对的旧名称。
- KEYNEW：表示键值对的新名称。

48.7 举例说明

使用包内的过程和函数之前，如果还未创建过系统包，请先调用系统过程 SP_CREATE_SYSTEM_PACKAGES 创建系统包。

如果 INI 参数 COMPATIBLE_MODE 为 2, 则须同时设置 INI 参数 ORA_DATE_FMT 为 1, 否则下面创建语句无法执行通过。

```
SP_CREATE_SYSTEM_PACKAGES(1,'DBMS_JSON');

SET SERVEROUTPUT ON;  //DBMS_OUTPUT.PUT_LINE 需要设置这条语句，才能打印出消息
```

可以查询 SYSOBJECTS 系统表判断当前 JSON 类是否处于有效状态，如下所示表示 JSON 类均处于有效状态:

```
SELECT NAME,TYPE$,SUBTYPE$,VALID FROM SYSOBJECTS WHERE NAME LIKE 'J%';

行号     NAME              TYPE$   SUBTYPE$ VALID

---------- -------------- ------ -------- -----

1          JDOM_T            SCHOBJ CLASS     Y

2          JDOM_T            DSYNOM NULL      Y

3          JSON_ARRAY_T      SCHOBJ CLASS     Y

4          JSON_ARRAY_T      DSYNOM NULL      Y
```

```
5               JSON_ELEMENT_T SCHOBJ CLASS       Y
6               JSON_ELEMENT_T DSYNOM NULL        Y
7               JSON_KEY_LIST   SCHOBJ TYPE       Y
8               JSON_KEY_LIST   DSYNOM NULL       Y
9               JSON_OBJECT_T   SCHOBJ CLASS      Y
10              JSON_OBJECT_T   DSYNOM NULL       Y
11              JSON_SCALAR_T   SCHOBJ CLASS      Y
12              JSON_SCALAR_T DSYNOM NULL         Y
```

下面将以 JDOM_T 为例进行举例说明，若用户需要使用系统包中的其余类，只需将 JDOM_T 替换为其他类型，并根据对应类的方法说明进行调用即可。

例 1 举例说明 PARSE 和 GET_NUMBER 方法。

语句如下:

```
DECLARE
    P_CLOB CLOB := '{"AGE":123}';
    //构造一个 JDOM_T 类型元素 DOM
    DOM JDOM_T := JDOM_T.PARSE(P_CLOB);
    //将 DOM 中指定名称对应的值以 NUMBER 数据类型返回
    V_AGE_ID NUMBER := DOM.GET_NUMBER('AGE');
BEGIN
    DBMS_OUTPUT.PUT_LINE(V_AGE_ID);
END;
/
```

执行结果如下:

```
123
```

例 2 举例说明 PUT(KEY,VAL)和 TO_STRING 方法。

语句如下:

```
DECLARE
    P_CLOB CLOB := '{"AGE":123}';
    //构造一个 JDOM_T 类型元素 DOM
    DOM JDOM_T := JDOM_T.PARSE(P_CLOB);
    RET BOOLEAN;
BEGIN
    //在 DOM 中添加一个键值对
    RET := DOM.PUT('NAME', '张三');
    //将 DOM 以 VARCHAR2 文本返回并进行打印
    DBMS_OUTPUT.PUT_LINE(DOM.TO_STRING);
END;
/
```

执行结果如下:

```
{"AGE":123,"NAME":"张三"}
```

例 3 举例说明 PUT(POS,VAL,OVERWRITE)、IS_OBJECT 和 IS_ARRAY 方法。

语句如下:

```
DECLARE
    P_CLOB CLOB := '[1,2]';
    //构造一个 JDOM_T 类型元素 DOM
```

```
    DOM JDOM_T := JDOM_T.PARSE(P_CLOB);
    RET BOOLEAN;
BEGIN
    //依次判断 DOM 的 JSON 元素类型
    IF (DOM.IS_OBJECT) THEN
        DBMS_OUTPUT.PUT_LINE('OBJECT');
    ELSIF (DOM.IS_ARRAY) THEN
        DBMS_OUTPUT.PUT_LINE('ARRAY');
    ELSE
        DBMS_OUTPUT.PUT_LINE('SCALAR');
    END IF;
    //在 DOM 中添加一个数组元素
    RET := DOM.PUT(1, '张三', FALSE);
    DBMS_OUTPUT.PUT_LINE(DOM.TO_STRING);
END;
/
```

执行结果如下：

```
ARRAY
[1,"张三",2]
```

例 4 举例说明 CLONE、REMOVE 和 GET_KEYS 方法。

语句如下：

```
DECLARE
    P_CLOB CLOB := '{"A":1,"B":2,"C":3}';
```

```
    //构造一个 JDOM_T 类型元素 DOM
    DOM JDOM_T := JDOM_T.PARSE(P_CLOB);
    DOM2 JDOM_T;
    RET BOOLEAN;
    V JSON_KEY_LIST;
    N NUMBER;
BEGIN
    //将 DOM 克隆至 DOM2，DOM2 与 DOM 完全相同
    DOM2 := DOM.CLONE();
    //删除 DOM2 中名称为“B”的键值对
    RET := DOM2.REMOVE('B');
    DBMS_OUTPUT.PUT_LINE(DOM.STRINGIFY);
    DBMS_OUTPUT.PUT_LINE(DOM2.STRINGIFY);
    //获取 DOM 中所有键值对名称，并进行打印
    V := DOM.GET_KEYS();
    N := V.COUNT();
    DBMS_OUTPUT.PUT_LINE('KEYS_NUM:'||N);
    FOR I IN 1..N LOOP
        DBMS_OUTPUT.PUT_LINE(V(I));
    END LOOP;
END;
/
```

执行结果如下：

```
{"A":1,"B":2,"C":3}
```

```
{"A":1,"C":3}

KEYS_NUM:3

A

B

C
```

例 5 举例说明 CREATE_ARR、APPEND、APPEND_NULL 和"GET"方法。

```
DECLARE
    //构造一个空的 JSON ARRAY，等价于“[]”
    DOM JDOM_T := JDOM_T.CREATE_ARR();
    DOM2 JDOM_T;
    RET BOOLEAN;
BEGIN
    //在 DOM 的末尾添加一个数组元素 1
    RET := DOM.APPEND(1);
    //在 DOM 的末尾添加一个数组元素 null
    RET := DOM.APPEND_NULL();
    DBMS_OUTPUT.PUT_LINE(DOM.STRINGIFY);
    //获取 DOM 中下标为 0 的数组元素，结果以 JDOM_T 方式返回
    DOM2 := DOM."GET"(0);
    DBMS_OUTPUT.PUT_LINE(DOM2.TO_STRING());
    //在 DOM 中下标为 0 的位置添加一个数组元素 null，指定覆盖
    RET := DOM.PUT_NULL(0, TRUE);
    //在 DOM 中下标为 1 的位置添加一个数组元素 20，指定覆盖
```

```
    RET := DOM.PUT(1, 20, TRUE);
    DBMS_OUTPUT.PUT_LINE(DOM.STRINGIFY);
END;
/
```

执行结果如下:

```
[1,null]
1
[null,20]
```

例 6 举例说明 GET_SIZE 和 GET_TYPE 方法。

```
DECLARE
    P_CLOB CLOB := '[1,"ABC",[2],{"A":1}]';
    //构造一个 JDOM_T 类型元素 DOM
    DOM JDOM_T := JDOM_T.PARSE(P_CLOB);
    N NUMBER;
BEGIN
    //获取 DOM 中数组元素的个数
    N := DOM.GET_SIZE;
    DBMS_OUTPUT.PUT_LINE('GET_SIZE:'||N);
    //依次打印 DOM 中每个数组元素的 JSON 类型
    FOR I IN 1..N LOOP
        DBMS_OUTPUT.PUT_LINE(DOM.GET_TYPE(I-1));
    END LOOP;
END;
```

```
/
```

执行结果如下:

```
GET_SIZE:4

SCALAR

SCALAR

ARRAY

OBJECT
```

例 7 举例说明 HAS_KEY 和 RENAME_KEY 方法。

```
DECLARE
    P_CLOB CLOB := '{"张三":13, "李四":14}';
    //构造一个 JDOM_T 类型元素 DOM
    DOM JDOM_T := JDOM_T.PARSE(P_CLOB);
    N NUMBER;
    RET BOOLEAN;
BEGIN
    //判断 DOM 中是否含有名称为“张三”的键值对
    IF (DOM.HAS_KEY('张三')) THEN
        //将 DOM 中的键值对名称“张三”改为“张三峰”
        RET := DOM.RENAME_KEY('张三', '张三峰');
    END IF;
    DBMS_OUTPUT.PUT_LINE(DOM.STRINGIFY);
END;
/
```

执行结果如下:

```
{"李四":14,"张三峰":13}
```

49 DBMS_FLASHBACK 包

使用 DBMS_FLASHBACK 包，可以查询在指定时间或 LSN 时刻的数据库版本。DM DPC 环境下 DBMS_FLASHBACK 包不支持查询指定 LSN 时刻的数据库版本，但是可以查询指定时间的数据库版本。

49.1 相关方法

1.DISABLE

关闭会话闪回模式。

语法如下:

```
PROCEDURE DISABLE;
```

2.ENABLE_AT_SYSTEM_CHANGE_NUMBER

以指定 LSN 打开会话闪回模式。

语法如下:

```
PROCEDURE ENABLE_AT_SYSTEM_CHANGE_NUMBER(QUERY_SCN IN BIGINT);
```

参数详解

- QUERY_SCN 输入参数，指定闪回时刻 LSN。

3. ENABLE_AT_TIME

以指定时间打开会话闪回模式。

语法如下:

```
PROCEDURE ENABLE_AT_TIME(QUERY_TIME IN TIMESTAMP);
```

参数详解

- QUERY_TIME 输入参数，指定闪回时间。

4. GET_SYSTEM_CHANGE_NUMBER

获取系统当前 LSN 值。

语法如下:

```
FUNCTION GET_SYSTEM_CHANGE_NUMBER RETURN BIGINT;
```

返回值.

返回系统当前 LSN

49.2 使用说明

1. 使用 DBMS_FLASHBACK 包，需要打开闪回 INI 参数 ENABLE_FLASHBACK。
2. 打开会话闪回模式下不能执行 DML 和 DDL 操作，需要先调用包中方法 DISABLE 关闭闪回模式才能正常执行。
3. 会话闪回模式下可以执行带闪回查询子句的语句，也可以执行不带闪回查询子句的语句;如果指定闪回查询子句中的 LSN/TIME 和会话闪回模式中的 LSN/TIME 相同，则查询结果一致，若不同则以指定的闪回查询子句为主。
4. 使用 DBMS_FLASHBACK 包进行闪回查询的对象不支持临时表、列存储表、外部表与视图。
5. 在数据守护环境下，备库不支持使用 DBMS_FLASHBACK 包。

6. DM MPP 和 DM DPC 环境下都不支持 DBMS_FLASHBACK 包。

49.3 举例说明

使用包内的过程和函数之前，如果还未创建过系统包。请先调用系统过程创建系统包，默认已经创建。

```
SP_CREATE_SYSTEM_PACKAGES (1,'DBMS_FLASHBACK');
```

以指定 LSN 为例使用闪回包进行查询。

```
DROP TABLE IF EXISTS T1;

CREATE TABLE T1(C1 INT, C2 INT);

INSERT INTO T1 VALUES(1,1);

COMMIT;



SELECT DBMS_FLASHBACK.GET_SYSTEM_CHANGE_NUMBER;   //假设查询结果为 117846



INSERT INTO T1 VALUES(2,2);

COMMIT;



DBMS_FLASHBACK.ENABLE_AT_SYSTEM_CHANGE_NUMBER(117846);

SELECT * FROM T1;



DBMS_FLASHBACK.DISABLE;
```

50 DBMS_SYSTEM 包

使用 DBMS_SYSTEM 包，可以在 INI 参数 COMPATIBLE_MODE=2 的前提下，通过执行 DBMS_SYSTEM.KSDWRT 包过程实现分别往数据库告警日志 ELOG 和 TRACE 文件写入信息。需要注意的是，只有在 INI 参数 TRC_LOG 为开启状态，且 INI 参数 TRC_LOG_MODULE 取值包含 12 时，向 TRACE 文件写入信息才能写入成功。

50.1 相关方法

1.KSDWRT

向数据库告警日志 ELOG 和 TRACE 文件写入用户指定信息。

语法如下:

```
PROCEDURE KSDWRT(DEST_ID INT, WRT_STR VARCHAR2);
```

参数详解

- DEST_ID 输入参数，指定信息写入目的地。

  目的地参数有三个选项:

  1） DEST_ID=1，向 TRACE 文件写入 WRT_STR;

  2） DEST_ID=2，向 ELOG 文件写入 WRT_STR;

  3） DEST_ID=3，同时向 TRACE 文件与 ELOG 文件写入 WRT_STR。

  若输入的目的地参数不合法或为空，则报错。

- WRT_STR 输入参数，向日志写入的具体内容。

  若日志写入内容为空串，则向 LOG 文件中写入空白；若写入内容过长，则进行截断处理，每 1000 个字节进行一次截断，并在被截断串末尾添加省略号"..."。

50.2 举例说明

使用包内的过程和函数之前，如果还未创建过系统包。请先调用系统过程创建系统包。

```
SP_CREATE_SYSTEM_PACKAGES (1,'DBMS_SYSTEM');
```

以指定内容为例使用 DBMS_SYSTEM 包。

例 1 INI 参数 TRC_LOG 开启，且 INI 参数 TRC_LOG_MODULE 取值包含 12 时，设置目的参数为 1，向 trace 文件写入信息。

```
CALL DBMS_SYSTEM.KSDWRT(1,'TEST_TRACE_FILE');
```

进入对应路径查看 trace 文件信息 dmtrc_DMSERVER.log，已记录“TEST_TRACE_FILE”和执行时间相关信息，结果如下：

```
2023-05-10 15:11:01.333 [INF] P2911112:T2908348 TEST_TRACE_FILE [IFUN_SYS2.C:28804]
```

例 2 设置目的参数为 2，向告警 elog 文件写入信息。

```
CALL DBMS_SYSTEM.KSDWRT(2,'TEST_ELOG_FILE');
```

进入对应路径查看 elog 文件信息 dm_DMSERVER_202305.log，已记录“TEST_ELOG_FILE”和执行时间相关信息，结果如下：

```
2023-05-10 15:11:12.358 [INFO] database P0002911112 T0000000000002908348  TEST_ELOG_FILE
```

例 3 INI 参数 TRC_LOG 开启，且 INI 参数 TRC_LOG_MODULE 取值包含 12 时，设置目的参数为 3，向告警 elog 文件与 trace 文件同时写入信息。

```
CALL DBMS_SYSTEM.KSDWRT(3,'TEST_TRACE_AND_ELOG_FILE');
```

进入对应路径查看 elog 文件与 trace 文件，已记录“TEST_TRACE_AND_ELOG_FILE”和执行时间相关信息，结果如下：

```
TRACE 文件:

2023-05-10 15:11:27.891 [INF] P2911112:T2908348 TEST_TRACE_AND_ELOG_FILE

[IFUN_SYS2.C:28804]
```

ELOG 文件:

2023-05-10 15:11:27.890 [INFO] database P0002911112 T0000000000002908348

TEST_TRACE_AND_ELOG_FILE

51 DBMS_AUDIT 包

用户可以使用 DBMS_AUDIT 包获取指定审计日志下的审计记录，并通过获取的信息进行更深入的分析。DPC 和 MPP 环境下暂不支持使用 DBMS_AUDIT 系统包。

51.1 相关方法

1. ADD_AUDFILE

在审计日志列表中增加日志文件，该函数只影响当前会话的 AUD。

语法如下:

```
PROCEDURE ADD_AUDFILE(
    LogFileName     IN VARCHAR,
);
```

参数详解

- LogFileName

增加的文件的全路径。

2. REMOVE_AUDFILE

从日志列表中移除某个日志文件,此处的日志列表为 AUD 预加载的审计日志文件列表,可通过 V$DBMSAUDIT_FILE 查看，该函数只影响当前会话的 AUD。

语法如下:

```
PROCEDURE REMOVE_AUDFILE(
    LogFileName     IN VARCHAR
```

```
);
```

参数详解

- LogFileName

待移除的日志的文件全路径。

3. START_AUD

用于打开某个会话上的 AUD，且开启 AUD 后会加载审计日志列表 V$DBMSAUDIT_FILE 中的文件，若审计列表为空，会加载所有审计文件。使用该函数后，不能再对审计日志列表中的文件进行进行增加或删除。

语法 1:

根据指定的模式和条件来开始某个会话上的 AUD，一个会话上仅能开始一个 AUD。

具体语法如下:

```
PROCEDURE    START_AUD(
    startTime          IN   DATETIME DEFAULT TO_DATE('1988/1/1', 'YYYY-MM-DD'),
    endTime            IN   DATETIME DEFAULT TO_DATE('2110/12/31', 'YYYY-MM-DD'),
    userName           IN   VARCHAR DEFAULT '',
    ip                 IN   VARCHAR DEFAULT '',
    succFlag           IN   INT DEFAULT 0
);
```

参数详解

- startTime

分析时或者加载时的过滤条件，审计记录起始时间，默认为 1988/1/1。

- endTime

分析时或者加载时的过滤条件，审计记录结束时间，默认为 2110/12/31。

- userName

分析时或者加载时的过滤条件，当前审计记录的用户名，默认为空。

- ip

分析时或者加载时的过滤条件，当前审计记录的 IP 地址，默认为空。

- succFlag

分析时或者加载时的过滤条件，当前审计记录执行后是否成功，默认为 0。0 表示无限制，1 表示只加载成功审计记录，2 表示只加载失败审计记录。只能为这三个值。

语法 2:

开始某个会话上的 AUD，一个会话上仅能 START 一个 AUD。使用该方法将不对审计记录进行筛选。

具体语法如下:

```
PROCEDURE    START_AUD( );
```

4. END_AUD

结束 AUD。使用该函数后，会清空审计列表中的信息。

语法如下:

```
PROCEDURE    END_AUD();
```

51.2 举例说明

使用包内的过程和函数之前，如果还未创建过系统包。请先调用系统过程创建系统包。

```
SP_CREATE_SYSTEM_PACKAGES (1,'DBMS_AUDIT');
```

以下示例展示日志分析工具使用流程。由于只有审计用户才有审计视图的查询权限，所以只有审计用户执行以下操作才能查询到正确视图。

第1步 打开审计开关

(1)通过 disql 设置打开审计开关，审计开关打开设置可查阅 DM 安全管理，如下:

```
SP_SET_ENABLE_AUDIT (1);
```

第2步 添加审计日志文件(最多添加 64 个文件)

(1) 查询有哪些审计日志（可查看《DM8 系统管理员手册》的附录二第 19 小节中对于 V$AUDIT_FILE 的说明)。

```
SELECT FILEPATH,BEGIN_TIME,DBNAME FROM V$AUDIT_FILE;
```

查询结果如下:

```
D:\DMDATA\ARCH\AUDIT_TEST_01AC9E1D46814CD482E58154AF1D663F_2023-4-14-9-28-38.L
OG 2023-04-30 15:51:06.000000    DAMENG
```

(2) 添加一个或多个需要分析的审计日志文件

```
DBMS_AUDIT.ADD_AUDFILE('D:\DMDATA\ARCH\AUDIT_TEST_01AC9E1D46814CD482E58154AF1D
663F_2023-4-14-9-28-38.LOG')
```

如要查看通过 ADD_AUDFILE 添加的审计日志文件，可以通过动态视图 V$DBMSAUDIT_FILE 进行查询。

```
SELECT FILEPATH,BEGIN_TIME,DBNAME FROM V$DBMSAUDIT_FILE;
```

查询结果如下:

```
D:\DMDATA\ARCH\AUDIT_TEST_01AC9E1D46814CD482E58154AF1D663F_2023-4-14-9-28-38.L
OG 2023-04-30 15:51:06.000000    DAMENG
```

第3步 启动审计日志文件分析

```
DBMS_AUDIT.START_AUD();
```

在启动审计日志文件的同时，也可以添加相应的过滤条件提前过滤审计日志的内容。

```
DBMS_AUDIT.START_AUD(SUCCFLAG=>1 , STARTTIME=>TO_DATE('2023-03-04
17:36:00','YYYY-MM-DD HH24:MI:SS') , ENDTIME=>TO_DATE('2023-06-04 17:36:02','YYYY-MM-DD
HH24:MI:SS'));
```

如要查看审计日志文件的分析结果，可以通过动态视图 V$AUDITRECORDS 进行查询。

```
SELECT *  FROM V$AUDITRECORDS;
```

也可以在查询的同时增加起始、结束时间、用户、IP、对象、成功、失败等条件过滤。

```
SELECT * FROM V$AUDITRECORDS WHERE SUCC_FLAG = 'Y';
```

第4步 终止审计日志文件分析

```
DBMS_AUDIT.END_AUD();
```

52 DBMS_XPLAN 包

为了更好展示历史执行计划，达梦提供了 DBMS_XPLAN 系统包。DBMS_XPLAN 包用于展示历史执行计划，以及对持久化的历史执行计划进行清除。

仅当历史执行计划持久化功能打开（USE_PLN_POOL!=0，ENABLE_MONITOR_PLNHIST=1）时，使用 DBMS_XPLAN 包才有意义。不支持 DMDPC 环境下记录历史执行计划。

52.1 相关方法

DBMS_XPLAN 包目前共提供三种方法。

1. DISPLAY_PLANHIST

展示指定原始 SQL 语句的历史执行计划信息。历史执行计划信息来源于系统表 SYSPLANHIST。

语法如下:

```
DBMS_XPLAN.DISPLAY_PLANHIST (
    SQL_TEXT                IN  VARCHAR(8818));
```

参数详解

- SQL_TEXT：具体的 SQL 语句。

2. DISPLAY_PLANHIST

展示指定哈希值的 SQL 语句的历史执行计划信息。历史执行计划信息来源于系统表 SYSPLANHIST。

语法如下:

```
DBMS_XPLAN.DISPLAY_PLANHIST (
    HASH_VALUE              IN  INTEGER);
```

参数详解

- HASH_VALUE：SQL 语句的哈希值。

3. CLEAR_PLANHIST

清理系统表 SYSPLANHIST 中指定时间范围内的历史执行计划记录。

语法如下：

```
DBMS_XPLAN.CLEAR_PLANHIST (
    START_TIME          IN DATETIME,
    END_TIME            IN DATETIME);
```

参数详解

- START_TIME：指定时间范围起始值。指定为 NULL 时，表示不限制起始时间。
- END_TIME：指定时间范围终止值。指定为 NULL 时，表示不限制终止时间。终止时间不应该早于起始时间。

52.2 举例说明

使用包内的过程和函数之前，如果还未创建过系统包。请先调用系统过程创建系统包。

```
SP_CREATE_SYSTEM_PACKAGES (1,'DBMS_XPLAN');
```

执行前确保计划复用和历史计划持久化功能都开启，即 USE_PLN_POOL!=0，ENABLE_MONITOR_PLNHIST=1。

1. 建表

```
CREATE TABLE T1(C1 INT,C2 INT);
INSERT INTO T1 SELECT LEVEL,LEVEL FROM DUAL CONNECT BY LEVEL <= 100;
SELECT C1 FROM T1;
CREATE INDEX IDX1 ON T1(C1);
```

```
SELECT C1 FROM T1;

SP_FLUSH_HIST_PLAN; //将历史执行计划信息刷盘到系统表 SYSPLANHIST 中
```

2. 展示 SQL 语句的历史执行计划信息

```
DBMS_XPLAN.DISPLAY_PLANHIST('SELECT C1 FROM T1;');
```

53 DBMS_DDL 包

DBMS_DDL 包用于加密 CREATE OR REPLACE 语句。

DBMS_DDL 包的使用依赖 DBMS_SQL 包，在创建 DBMS_DDL 包时，系统会自动创建 DBMS_SQL 包。

53.1 相关方法

1. CREATE_WRAPPED

该过程用于加密 CREATE OR REPLACE 语句，同时执行该语句创建相应的对象。

语法如下:

```
PROCEDURE CREATE_WRAPPED (
    DDL         VARCHAR2
);
或
PROCEDURE CREATE_WRAPPED(
    DDL         DBMS_SQL.VARCHAR2A,
    LB          PLS_INTEGER,
    UB          PLS_INTEGER
);
或
PROCEDURE CREATE_WRAPPED(
    DDL         DBMS_SQL.VARCHAR2S,
    LB          PLS_INTEGER,
```

```
    UB          PLS_INTEGER
);
```

参数详解

- DDL：CREATE OR REPLACE 语句，可以创建包规范、包主体、函数、过程、类型、类型体。
- LB：指定 CREATE OR REPLACE 语句的字符串表中索引的下限。
- UB：指定 CREATE OR REPLACE 语句的字符串表中索引的上限。

返回值

无

2. WRAP

该函数用于加密 CREATE OR REPLACE 语句。

语法如下：

```
FUNCTION WRAP(
    DDL         VARCHAR2
) RETURN CLOB;
或
FUNCTION WRAP(
    DDL         DBMS_SQL.VARCHAR2S,
    LB          PLS_INTEGER,
    UB          PLS_INTEGER
) RETURN DBMS_SQL.VARCHAR2S;
或
FUNCTION WRAP(
    DDL         DBMS_SQL.VARCHAR2A,
```

```
    LB          PLS_INTEGER,
    UB          PLS_INTEGER
) RETURN DBMS_SQL.VARCHAR2A;
```

参数详解

- DDL：CREATE OR REPLACE 语句，可以创建包规范、包主体、函数、过程、类型、类型体。
- LB：指定 CREATE OR REPLACE 语句的字符串表中索引的下限。
- UB：指定 CREATE OR REPLACE 语句的字符串表中索引的上限。

返回值

加密后的 CREATE OR REPLACE 语句。

53.2 举例说明

使用包内的过程和函数之前，如果还未创建过系统包，请先调用系统过程创建系统包。

```
SP_CREATE_SYSTEM_PACKAGES (1,'DBMS_DDL');
SET SERVEROUTPUT ON;  //PRINT 需要设置这条语句，才能打印出消息
```

使用 CREATE_WRAPPED 过程加密 CREATE OR REPLACE 语句。

```
DECLARE
    l_source DBMS_SQL.VARCHAR2A;
    l_wrap DBMS_SQL.VARCHAR2A;
BEGIN
    l_source(1) := 'CREATE OR REPLACE FUNCTION get_date_string RETURN VARCHAR2 AS ';
    l_source(2) := 'BEGIN ';
    l_source(3) := 'RETURN TO_CHAR(SYSDATE, "DD-MON-YYYY"); ';
    l_source(4) := 'END get_date_string;';
```

```
    SYS.DBMS_DDL.CREATE_WRAPPED(ddl => l_source, lb => 1, ub => l_source.count);

END;

/
```

CREATE_WRAPPED 过程加密语句的同时会执行该语句创建相应的对象，上例中的 get_date_string 函数已被创建完成，如下所示:

```
SELECT get_date_string();

行号        GET_DATE_STRING()

---------- ------------------

1             20-5 月-2025
```

查看加密后的 CREATE OR REPLACE 语句。

```
SELECT TEXT FROM USER_SOURCE WHERE NAME = 'GET_DATE_STRING' AND TYPE = 'FUNCTION';

行号        TEXT

---------- ----------------------------------------------------------------

1             CREATE   OR REPLACE   FUNCTION GET_DATE_STRING WRAPPED

'D5611521085 1B1C1856505156 1F1B1080885511555 85B10808954185C12141858C0841C508A0088151D1

61B108A0088551155585B10815F1F5C121418528C565C5114115513808D811117871F1B17865656565D868

ECA00851B11108D15115F51141155 1F5C5158561B1D1EC08A0';
```

54 DBMS_LDAP 包

LDAP 是一种轻量级的目录访问协议，DBMS_LDAP 是 DM 数据库提供的一个 PL/SQL 系统包，用于在数据库内部与 LDAP 服务器之间交互，实现目录服务的查询、认证、修改和管理。

54.1 数据类型

DBMS_LDAP 包中支持的数据类型语法如下：

```
SUBTYPE SESSION IS BINARY(32);

SUBTYPE MESSAGE IS BINARY(32);

SUBTYPE MOD_ARRAY IS BINARY(32);

SUBTYPE BERVAL IS BINARY(32000);

TYPE STRING_COLLECTION IS TABLE OF VARCHAR2(32767) INDEX BY BINARY_INTEGER;

TYPE BERVAL_COLLECTION IS TABLE OF VARBINARY(32767) INDEX BY BINARY_INTEGER;

TYPE BLOB_COLLECTION IS TABLE OF BLOB INDEX BY BINARY_INTEGER;
```

54.2 相关方法

1. INIT

建立与 LDAP 服务器的连接，并返回通信会话句柄。

语法如下：

```
FUNCTION INIT(
    HOSTNAME        IN  VARCHAR2,
    PORTNUM         IN  PLS_INTEGER
```

```
)RETURN SESSION;
```

参数详解

- HOSTNAME

  LDAP 服务器主机名。

- PORTNUM

  LDAP 服务器端口。

返回值

通信会话句柄。

2. CREATE_MOD_ARRAY

创建用于存放条目信息的数组。

语法如下:

```
FUNCTION CREATE_MOD_ARRAY(
    NUM                 IN  PLS_INTEGER
) RETURN MOD_ARRAY;
```

参数详解

- NUM

  数组大小，不超过 8192。

返回值

存放条目信息的数组。

3. POPULATE_MOD_ARRAY

填充条目信息。

语法如下:

```
PROCEDURE POPULATE_MOD_ARRAY(
    MODPTR              IN  MOD_ARRAY,
    MOD_OP              IN  PLS_INTEGER,
    MOD_TYPE            IN  VARCHAR2,
    MODVAL              IN  STRING_COLLECTION
);

PROCEDURE POPULATE_MOD_ARRAY(
    MODPTR              IN  MOD_ARRAY,
    MOD_OP              IN  PLS_INTEGER,
    MOD_TYPE            IN  VARCHAR2,
    MODBVAL             IN  BERVAL_COLLECTION
);

PROCEDURE POPULATE_MOD_ARRAY(
    MODPTR              IN  MOD_ARRAY,
    MOD_OP              IN  PLS_INTEGER,
    MOD_TYPE            IN  VARCHAR2,
    MODBVAL             IN  BLOB_COLLECTION
);
```

参数详解

- MODPTR

  条目数组。

- MOD_OP

条目操作，可指定为 MOD_ADD、MOD_DELETE、MOD_REPLACE。

- MOD_TYPE

  条目类型

- MODBVAL

  条目数据。

4. ADD_S

向服务器添加条目。

语法如下:

```
FUNCTION ADD_S(
    LD                  IN  SESSION,
    ENTRYDN             IN  VARCHAR2,
    MODPTR              IN  MOD_ARRAY
) RETURN PLS_INTEGER;
```

参数详解

- LD

  SESSION 类型句柄。

- ENTRYDN

  条目路径

- MODPTR

  条目数组。

5. FREE_MOD_ARRAY

释放条目数组。

语法如下:

```
PROCEDURE FREE_MOD_ARRAY(
    MODPTR              IN   MOD_ARRAY
);
```

参数详解

- MODPTR

  条目数组。

6. MODIFY_S

修改条目信息。

语法如下:

```
FUNCTION MODIFY_S(
    LD                  IN   SESSION,
    ENTRYDN             IN   VARCHAR2,
    MODPTR              IN   MOD_ARRAY
) RETURN PLS_INTEGER;
```

参数详解

- LD

  SESSION 类型句柄。

- ENTRYDN

  条目路径。

- MODPTR

  条目数组

7. SIMPLE_BIND_S

登录 LDAP 服务器。

语法如下:

```
FUNCTION SIMPLE_BIND_S(
    LD                  IN  SESSION,
    DN                  IN  VARCHAR2,
    PASSWD              IN  VARCHAR2
) RETURN PLS_INTEGER;
```

参数详解

- LD

  SESSION 类型句柄。

- DN

  条目路径。

- PASSWD

  登录密码。

8. UNBIND_S

断开与 LDAP 服务器的连接。

语法如下:

```
FUNCTION UNBIND_S(
    LD                  IN OUT  SESSION
) RETURN PLS_INTEGER;
```

参数详解

- LD

SESSION 类型句柄。

9. SEARCH_S

查找条目。

语法如下:

```
FUNCTION SEARCH_S(
    LD              IN  SESSION,
    BASE            IN  VARCHAR2,
    SCOPE           IN  PLS_INTEGER,
    FILTER          IN  VARCHAR2,
    ATTRS           IN  STRING_COLLECTION,
    ATTRONLY        IN  PLS_INTEGER,
    RES             OUT MESSAGE
) RETURN PLS_INTEGER;
```

参数详解

- LD

  SESSION 类型句柄。

- BASE

  基准路径。

- SCOPE

  搜索范围。

- FILTER

  过滤条件。

- ATTRS

需要返回的属性数组。

- ATTRSONLY

  属性返回模式。

- RES

  输出的结果集。

10. COUNT_ENTRIES

计算条目数量。

语法如下:

```
FUNCTION COUNT_ENTRIES (
    LD                  IN  SESSION,
    MSG                 IN  MESSAGE
) RETURN PLS_INTEGER;
```

参数详解

- LD

  SESSION 类型句柄。

- MSG

  结果集句柄。

11. FIRST_ENTRY

获取第一个条目。

语法如下:

```
FUNCTION FIRST_ENTRY(
    LD                  IN  SESSION,
```

```
    MSG                 IN    MESSAGE

) RETURN MESSAGE;
```

参数详解

- LD

  SESSION 类型句柄。

- MSG

  结果集句柄。

返回值

返回第一个条目。

12. NEXT_ENTRY

获取下一条条目。

语法如下:

```
FUNCTION NEXT_ENTRY (

    LD                  IN    SESSION,

    MSG                 IN    MESSAGE

) RETURN MESSAGE;
```

参数详解

- LD

  SESSION 类型句柄。

- MSG

  结果集句柄。

返回值

返回下一个条目。

13. GET_DN

获取条目路径。

语法如下:

```
FUNCTION GET_DN(
    LD                  IN  SESSION,
    LDAPENTRY           IN  MESSAGE
) RETURN VARCHAR2;
```

参数详解

- LD

  SESSION 类型句柄。

- LDAPENTRY

  条目句柄。

返回值

返回条目对应的路径。

14. GET_VALUES

从 LDAP 条目中获取指定属性的字符串类型值。

语法如下:

```
FUNCTION GET_VALUES(
    LD                  IN  SESSION,
    LDAPENTRY           IN  MESSAGE,
    ATTR                IN  VARCHAR2
) RETURN STRING_COLLECTION;
```

参数详解

- LD

  SESSION 类型句柄。

- LDAPENTRY

  条目句柄。

- ATTR

  属性名。

返回值

获取的指定属性的字符串类型值。

15. RENAME_S

重命名条目。

语法如下:

```
FUNCTION RENAME_S(
    LD                  IN  SESSION,
    DN                  IN  VARCHAR2,
    NEWRDN              IN  VARCHAR2,
    NEWPARENT           IN  VARCHAR2,
    DELETEOLDRDN        IN  PLS_INTEGER,
    SERVERCTRLS         IN  LDAPCONTROL DEFAULT NULL,
    CLIENTCTRLS         IN  LDAPCONTROL DEFAULT NULL
) RETURN PLS_INTEGER;
```

参数详解

- LD

SESSION 类型句柄。

- DN

  条目路径。

- NEWRDN

  新的条目相对路径。

- NEWPARENT

  父路径。

- DELETEOLDRDN

  是否删除原先的条目相对路径。

- SERVERCTRLS

  仅兼容支持，无实际意义。

- CLIENTCTRLS

  仅兼容支持，无实际意义。

16. DELETE_S

同步方式删除条目。

语法如下:

```
FUNCTION DELETE_S(
    LD                  IN  SESSION,
    ENTRYDN             IN  VARCHAR2
) RETURN PLS_INTEGER;
```

参数详解

- LD

  SESSION 类型句柄。

- ENTRYDN

  条目路径。

17. MSGFREE

释放结果集。

语法如下:

```
FUNCTION MSGFREE(
    LM                  IN  MESSAGE
) RETURN PLS_INTEGER;
```

参数详解

- LM

  结果集句柄。

18. ERR2STRING

根据返回的 LDAP 错误码获取错误描述。

语法如下:

```
FUNCTION ERR2STRING(
    LDAP_ERR            IN  PLS_INTEGER
) RETURN VARCHAR2;
```

参数详解

- LDAP_ERR

  错误码。

返回值

错误码对应的错误描述。

54.3 举例说明

使用包内的过程和函数之前，如果还未创建过系统包。请先调用系统过程创建系统包。

```
SP_CREATE_SYSTEM_PACKAGES (1,'DBMS_LDAP');
```

向 LDAP 添加条目：

```
declare
  l_retval     PLS_INTEGER;
  l_add_vals     DBMS_LDAP.STRING_COLLECTION;
  l_array       DBMS_LDAP.MOD_ARRAY;
  l_session         DBMS_LDAP.SESSION;
begin
     l_session := dbms_ldap.init('192.168.0.1', 389);
     l_retval := dbms_ldap.simple_bind_s(l_session,
'cn=admin,dc=dm_ldap,dc=com','mypwd');
     --增加数据
     l_array := dbms_ldap.create_mod_array(10);
     l_add_vals(1) := 'test_user1';
     dbms_ldap.populate_mod_array(l_array,
                                  dbms_ldap.MOD_ADD,
                                  'cn',
                                  l_add_vals);
    l_add_vals(1) := 'Doe1';
  dbms_ldap.populate_mod_array(l_array,
                                  dbms_ldap.MOD_ADD,
```

```
                                        'sn',
                                        l_add_vals);
  l_add_vals(1) := 'top';
  l_add_vals(2) := 'person';
  dbms_ldap.populate_mod_array(l_array,
                                        dbms_ldap.MOD_ADD,
                                        'objectClass',
                                        l_add_vals);
l_retval := dbms_ldap.add_s(l_session, 'cn=test_user1, dc=dm_ldap, dc=com', l_array);
    dbms_ldap.free_mod_array(l_array);
    l_retval := dbms_ldap.unbind_s(l_session);
end;
/
```

55 DBMS_XMLPARSER 包

DBMS_XMLPARSER 包用于新建 PARSER，并通过 PARSER 可以访问 XML 文档的内容和结构。

55.1 数据类型

DBMS_XMLPARSER 包定义了若干 XML 数据类型：

1. PARSER

XML 解析器，用于构建一个可通过 DOM API 访问的解析树。

55.2 相关方法

1. NEWPARSER

创建新的 PARSER。

语法如下：

```
FUNCTION NEWPARSER()RETURN DBMS_XMLPARSER.PARSER;
```

返回值

创建的 PARSER 结点。

2. FREEPARSER

释放 PARSER。

语法如下：

```
PROCEDURE FREEPARSER(
    P IN DBMS_XMLPARSER.PARSER
);
```

参数详解

P：PARSER 类型结点。

3. PARSEBUFFER

解析存储在给定字符串缓冲区中的 XML。

语法如下:

```
PROCEDURE PARSEBUFFER(
    P IN DBMS_XMLPARSER.PARSER,
    DOC IN VARCHAR2
);
```

参数详解

P：PARSER 类型结点。

DOC：字符串类型 XML 数据。

4. PARSECLOB

解析存储在给定字符串缓冲区中的 XML。

语法如下:

```
PROCEDURE PARSECLOB(
    P IN DBMS_XMLPARSER.PARSER,
    DOC IN CLOB
);
```

参数详解

P：PARSER 类型结点。

DOC：CLOB 类型 XML 数据。

5. GETDOCUMENT

获取 PARSER 中的 DOCUMENT 结点。

语法如下:

```
FUNCTION GETDOCUMENTR(
    P IN DBMS_XMLPARSER.PARSER
)RETURN DBMS_XMLDOM.DOMDOCUMENT;
```

参数详解

P: PARSER 类型结点。

返回值

PARSER 中存储的 DOMDOCUMENT 类型结点。

6. GETDOCTYPE

获取 PARSER 中的 DOMDOCUMENTTYPE 结点。

语法如下:

```
FUNCTION GETDOCTYPE(
    P IN DBMS_XMLPARSER.PARSER
)RETURN DBMS_XMLDOM.DOMDOCUMENTTYPE;
```

参数详解

P: PARSER 类型结点。

返回值

PARSER 中存储的 DOMDOCUMENTTYPE 类型结点。

7. SETDOCTYPE

设置 PARSER 中的 DOMDOCUMENTTYPE 结点。

语法如下:

```
PROCEDURE SETDOCTYPE(
    P IN DBMS_XMLPARSER.PARSER,
    DTD IN DBMS_XMLDOM.DOMDOCUMENTTYPE
);
```

参数详解

P：PARSER 类型结点。

DTD：DOMDOCUMENTTYPE 结点。

8. GETRELEASEVERSION

返回用于 PL/SQL 的 XML 分析器发布版本。

语法如下:

```
FUNCTION RESETPARAMS()RETURN VARCHAR2;
```

返回值

版本信息字符串。

9. SETVALIDATIONMODE

设置验证模式。

语法如下:

```
PROCEDURE SETVALIDATIONMNODE(
    P IN DBMS_XMLPARSER.PARSER,
    YES IN BOOLEAN
);
```

参数详解

P：PARSER 类型结点。

YES：设置模式：TRUE 为验证，FALSE 为不验证。

10. GETVALIDATIONMODE

获取验证模式。

语法如下：

```
FUNCTION SETVALIDATIONMNODE(
    P IN DBMS_XMLPARSER.PARSER
)RETURN BOOLEAN;
```

参数详解

P：PARSER 类型结点。

返回值

是否验证模式，TRUE 为验证，FALSE 为不验证。

11. PARSE

解析存储在 URL 或文件中的 XML。

语法如下：

```
FUNCTION PARSE(
    URL IN VARCHAR2
)RETURN DBMS_XMLDOM.DOMDOCUMENT;

PROCEDURE PARSE(
    P IN DBMS_XMLPARSER.PARSER,
    URL IN VARCHAR2
);
```

参数详解

P：PARSER 类型结点。

URL：待解析的 URL/文件的完整路径。

返回值

解析生成的 DOMDOCUMENT 类型节点。

12. SETBASEDIR

设置用于解析相对 URL 的基础目录。

语法如下:

```
PROCEDURE SETBASEDIR (
    P IN DBMS_XMLPARSER.PARSER,
    DIR IN VARCHAR2
);
```

参数详解

P：PARSER 类型结点。

DIR：字符串类型基础路径。

13. GETBASEDIR

获取 PARSER 中解析相对 URL 的基础目录。

语法如下:

```
FUNCTION GETBASEDIR (
    P IN DBMS_XMLPARSER.PARSER
)RETURN VARCHAR2;
```

参数详解

P：PARSER 类型结点。

返回值

字符串类型基础路径。

14. PARSEDTD

解析存储在给定 URL 或文件中的 DTD。

语法如下:

```
PROCEDURE PARSEDTD(
    P IN DBMS_XMLPARSER.PARSER,
    URL IN VARCHAR2,
    ROOT IN VARCHAR2
);
```

参数详解

P: PARSER 类型结点。

URL: 待解析的 URL/文件的完整路径。

ROOT: 根元素的名称。

15. PARSEDTDBUFFER

解析存储在给定缓冲区中的 DTD。

语法如下:

```
PROCEDURE PARSEDTDBUFFER(
    P IN DBMS_XMLPARSER.PARSER,
    DTD IN VARCHAR2,
    ROOT IN VARCHAR2
);
```

参数详解

P: PARSER 类型结点。

DTD：字符串类型 DTD。

ROOT：根元素的名称。

16. PARSEDTDCLOB

解析存储在给定 CLOB 数据中的 DTD。

语法如下：

```
PROCEDURE PARSEDTDCLOB(
    P IN DBMS_XMLPARSER.PARSER,
    DTD IN CLOB,
    ROOT IN VARCHAR2
);
```

参数详解

P：PARSER 类型结点。

DTD：CLOB 类型 DTD。

ROOT：根元素的名称。

55.3 举例说明

使用包内的过程和函数之前，如果还未创建过系统包，请先调用系统过程创建系统包。

```
SP_CREATE_SYSTEM_PACKAGES (1,'DBMS_XMLPARSER');
```

以下是新建一个 PARSER，通过 PARSERBUFFER 解析 XML 字符串，之后获取解析生成的 DTD 与 DOCUMENT，最后释放 PARSER。

```
DECLARE
    myparser DBMS_XMLPARSER.PARSER;
    doctype DBMS_XMLDOM.DOMDOCUMENTTYPE;
```

```
    doc DBMS_XMLDOM.DOMDOCUMENT;
BEGIN
    myparser := DBMS_XMLPARSER.newParser;
    DBMS_XMLPARSER.PARSEBUFFER(myparser, '<?xml version="1.0"?><!DOCTYPE test SYSTEM
"test.dtd"><test/>');
    doctype :=DBMS_XMLPARSER.GETDOCTYPE(myparser);
    doc := DBMS_XMLPARSER.GETDOCUMENT(myparser);
    DBMS_XMLPARSER.FREEPARSER(myparser);
END;
/
```

DM 技术支持

如果您在安装或使用 DM 及其相应产品时出现了问题，请首先访问我们的 Web 站点 http://www.dameng.com/。在此站点我们收集整理了安装使用过程中一些常见问题的解决办法，相信会对您有所帮助。

您也可以通过以下途径与我们联系，我们的技术支持工程师会为您提供服务。

武汉达梦数据库股份有限公司

地址：武汉市东湖新技术开发区甲铺岭街 39 号达梦中国数据库产业基地

电话：(+86)027-87588000

传真：(+86)027-87588810

武汉达梦数据技术有限公司

地址：武汉市东湖新技术开发区甲铺岭街 39 号达梦中国数据库产业基地 13-14 层

电话：(+86)027-87588000

传真：(+86)027-87588810

北京达梦数据库技术有限公司

地址：北京市海淀区中关村南大街 2 号数码大厦 B 座 1003

电话：(+86)010-51727900

传真：(+86)010-51727983

上海达梦数据库有限公司

地址：上海市张江高科技园区亮秀路 112 号 A 座 401

电话：(+86)021-50111338

四川蜀天梦图数据科技有限公司

地址：成都市天府新区湖畔西路 99 号 B7 栋（天府英才中心）6 层

电话：(+86)028-61458838

达梦数据技术（江苏）有限公司

地址：苏州吴中经济开发区越溪街道吴中大道 1431 号苏州（太湖）软件产业园智慧谷园区 1601 室 03-04 号

电话：(+86)0512-65285955

传真：(+86)0512-65286955

重庆达梦大数据有限公司

地址：重庆市沙坪坝区重庆师范大学（沙坪坝校区）第二教学楼 1002

电话：(+86)023-68165682

达梦数据技术（深圳）有限公司

地址：深圳市龙岗区园山街道荷坳社区龙岗大道 8288 号大运软件小镇 17 栋 101

技术服务：

咨询热线：400-991-6599

技术支持：dmtech@dameng.com

商务咨询：400@dameng.com

手册版本

版本号：V8

发版日期：2026 二季度版-26876



武汉达梦数据库股份有限公司
官方网站：www.dameng.com

达梦数据版权所有，侵权必究©